

رُست‌خانه دانه

فروشگاه قهوه تخصصی با پنل مدیریت

تحلیل کامل معماری، مدل داده، الگوریتم‌ها و تصمیم‌های مهندسی یک فروشگاه اینترنتی روی MongoDB و Express و Next.js – نوشته شده بر پایه خود کد، نه بر پایه حدس.

۸	۱. معرفی و نمای کلی پروژه
۸	۱.۱ این پروژه چیست
۸	۱.۲ یک پوشه در ریشه که جزء برنامه نیست
۹	۱.۳ سه نوع کالا، یک مدل
۹	۱.۴ داده اولیه
۱۰	۱.۵ نقشه صفحه‌ها
۱۲	۱.۶ اعداد پروژه
۱۲	۱.۷ اجرا در دو دستور
۱۴	۲. پشته فناوری و دلیل انتخاب هر ابزار
۱۴	۲.۱ فهرست کامل وابستگی‌ها
۱۵	۲.۲ چرا MERN؟
۱۶	۲.۳ چرا Mongoose و نه درایور خام؟
۱۷	۲.۴ چرا Express و نه Fastify یا NestJS؟
۱۸	۲.۵ چرا JWT و نه session؟
۱۹	۲.۶ چرا bcryptjs و نه bcrypt؟
۱۹	۲.۷ چرا React با Context و نه Redux؟
۲۰	۲.۸ چرا Next.js – و چرا اولش نه
۲۱	۲.۹ مسیریابی: از React Router به پوشه‌ها
۲۲	۲.۱۰ چرا multer با تنظیمات سفارشی؟
۲۳	۲.۱۱ ابزارهایی که عمداً استفاده نشده‌اند
۲۴	۲.۱۲ متغیرهای محیطی
۲۷	۳. ساختار پوشه‌ها و معماری کلی
۲۷	۳.۱ درخت کامل پروژه
۳۰	۳.۲ معماری لایه‌به‌لایه
۳۱	۳.۳ اصل‌های سازمان‌دهی کد
۳۳	۳.۴ چرخه عمر یک درخواست
۳۴	۳.۵ لایه ارتباط با سرور
۳۵	۳.۶ نقشه کامل API
۳۷	۳.۷ مدیریت خطا در سرور
۳۷	۳.۸ استراتژی راه‌اندازی و تجربه توسعه‌دهنده
۳۸	۳.۹ لایه‌های امنیتی سرور
۴۰	۳.۱۰ تست، لینت و یکپارچه‌سازی

۴. مدل داده و ساختار بانک اطلاعاتی ۴۳

- ۴.۱ نمای کلی ۴۳
- ۴.۲ نمودار ER ۴۳
- ۴.۳ مدل Item – قلب پروژه ۴۴
- ۴.۴ قلاب pre('validate') مدل Item ۴۷
- ۴.۵ مدل Order – سفارش به عنوان عکس لحظه‌ای ۴۸
- ۴.۶ مدل Admin ۵۱
- ۴.۷ مدل ClubMember ۵۲
- ۴.۸ مدل Content – الگوی کلید/مقدار ۵۳
- ۴.۹ چرا slug و نه ObjectId ؟ ۵۴
- ۴.۱۰ داده اولیه ۵۵
- ۴.۱۱ داده‌های سمت مرورگر ۵۷
- ۴.۱۲ جمع‌بندی الگوهای داده‌ای ۵۸

۵. تحلیل فایل به فایل ۵۹

بخش الف – سرور ۶۰

- server/src/index.js – نقطه ورود سرور ۶۰
- server/src/db.js – اتصال پایگاه داده ۶۱
- server/src/middleware/auth.js – نگهبان مسیرها ۶۲
- server/src/middleware/rateLimit.js – محدودیت نرخ ۶۳
- server/src/lib/cors.js – تصمim CORS ۶۴
- server/src/lib/stock.js – رزرو اتمی موجودی ۶۴
- server/src/lib/track.js – پیگیری سفارش ۶۵
- server/src/lib/upload.js – آپلود تصویر ۶۶
- server/src/models/*.js ۶۷
- server/src/routes/auth.js – احراز هویت ۶۸
- server/src/routes/items.js – مدیریت کالا ۶۹
- server/src/routes/orders.js – ثبت و مدیریت سفارش ۷۰
- server/src/routes/content.js – متن‌های سایت ۷۲
- server/src/routes/club.js – باشگاه مشتریان ۷۲
- server/src/routes/stats.js – وبترین ۷۳
- server/src/routes/reports.js – گزارش و پشتیبان ۷۳
- */server/src/data ۷۵

بخش الف-۲ – بسته مشترک ۷۷

- shared/package.json ۷۷
- shared/pricing.js – تنها نسخه محاسبه قیمت ۷۷

۷۷	shared/taxonomy.js — تنها نسخهٔ کلیدهای مجاز
۷۸	shared/seo.js — آدرس، متن صفحه و داده‌های ساختاریافته
۸۰	بخش ب — وب: زیرساخت
۸۰	web/src/app/layout.jsx — پوستهٔ همهٔ صفحه‌ها
۸۱	web/src/app/App.jsx — جدول مسیرها به‌جای App.jsx
۸۱	web/next.config.mjs — پراکسی، بستهٔ مشترک، و بهینه‌ساز تصویر
۸۳	web/src/proxy.js — سیاست امنیتی صفحه‌ها
۸۳	web/src/app/fonts.js — فونت روی همین دامنه
۸۴	web/src/lib/data.js — خواندن داده روی سرور
۸۴	web/src/lib/metadata.js — از توصیفِ <head> به شیء metadata
۸۵	web/src/lib/site.js — آدرس پایه و تصویر پیش‌نمایش
۸۵	web/src/lib/cartStorage.js — سبد، بیرون از کامپوننت
۸۵	web/src/lib/seo.js — پلِ JSON-LD
۸۶	web/src/lib/share.js — نشانی، کپی، و اینکه کدام راه
۸۷	web/src/lib/toastStore.js — پیام کوتاه، بیرون از هر کانتکست
۸۸	web/src/lib/img.js — کدام تصویر بهینه می‌شود
۸۹	web/src/lib/useStorefrontRefresh.js — جانشین reload() در پل
۸۹	web/src/app/_item — یک پیاده‌سازی، سه پوشه
۹۰	web/src/app/@modal — مودال پشت یک آدرس
۹۱	web/src/app/track/page.jsx — و چرا Suspense
۹۱	web/src/components/HomeShell.jsx و SiteHeader.jsx
۹۱	web/src/components/ItemBody.jsx
۹۱	web/src/context/ShopContext.jsx — قلب فروشگاه
۹۵	web/src/context/AuthContext.jsx
۹۵	web/src/lib/format.js
۹۶	web/src/lib/cartLine.js — شکل یک ردیف سبد
۹۶	web/src/lib/blendVisual.js — ریاضی و زمان‌بندی حالت تصویری
۹۸	web/src/lib/useReveal.js
۹۹	بخش ج — وب: کامپوننت‌های فروشگاه
۹۹	Header.jsx
۹۹	Hero.jsx — «ترازوی قیمت»
۱۰۰	CatalogSection.jsx — یک کامپوننت برای سه فهرست
۱۰۱	ItemCard.jsx
۱۰۱	ItemDetail.jsx
۱۰۲	CartDrawer.jsx

۱۰۳	Receipt.jsx — یک شکل، دو صداکننده
۱۰۴	PrintSheet.jsx — میزبان چاپ
۱۰۴	ShareButton.jsx — «برای دوستت بفرست»
۱۰۶	Toast.jsx — بیست و هفت خط، و یک هوک
۱۰۶	BlendsSection.jsx — میزبان هر دو حالت
۱۰۷	MixVisual.jsx — حالت تصویری ساز میکس
۱۰۹	BeanPicker.jsx — فهرست افزودن دانه
۱۰۹	CardArt.jsx
۱۱۱	ContentSections.jsx
۱۱۲	StaticSections.jsx
۱۱۲	web/src/components/Track.jsx — پیگیری سفارش
۱۱۵	بخش د — وب: پنل مدیریت
۱۱۵	AdminShell.jsx
۱۱۶	AdminLogin.jsx
۱۱۶	AdminItems.jsx
۱۱۶	AdminItemForm.jsx — ۹۶۹ خط
۱۱۷	OrderDetail.jsx و AdminOrders.jsx
۱۱۸	AdminReports.jsx — ۵۹۹ خط
۱۱۸	AdminContent.jsx + contentSchema.jsx
۱۱۹	AdminSettings.jsx و AdminClub.jsx
۱۲۰	بخش ه — تست‌ها و پیکربندی
۱۲۰	vitest.config.mjs
۱۲۰	tests/*.test.{js,jsx} — بیست و دو فایل، ۳۸۰ سنج
۱۲۲	prettierrc و eslint.config.mjs
۱۲۳	۶. جریان‌های اصلی کاربر
۱۲۳	۶.۱ جریان اول — بارگذاری اولیه صفحه
۱۲۵	۶.۲ جریان دوم — مرور، فیلتر و افزودن یک قهوه ساده
۱۲۷	۶.۳ جریان سوم — صفحه اختصاصی کالا، و همان کالا به شکل مودال
۱۳۰	۶.۴ جریان چهارم — ساختن میکس دلخواه
۱۳۱	۶.۵ جریان پنجم — تسویه و ثبت سفارش
۱۳۷	۶.۶ جریان ششم — پیگیری سفارش بدون حساب کاربری
۱۳۹	۶.۷ جریان هفتم — ورود مدیر
۱۴۱	۶.۸ جریان هشتم — ویرایش کالا با پیش‌نمایش زنده
۱۴۲	۶.۹ جریان نهم — گزارش فروش و پشتیبان
۱۴۳	۶.۱۰ جریان جانبی — عضویت در باشگاه

۱۴۳	۶.۱۱ جریان جانبی – هم‌رسانی یک کالا
۱۴۶	۶.۱۲ نمودار وضعیت سفارش
۱۴۷	۷. الگوریتم‌ها و منطق تجاری
۱۴۷	۷.۰ کدام الگوریتم کجا اجرا می‌شود
۱۴۸	۷.۱ محاسبه قیمت – shared/pricing.js
۱۵۴	۷.۲ ریاضی میکس – web/src/lib/blend.js
۱۵۷	۷.۳ حالت تصویری میکس – web/src/lib/blendVisual.js
۱۵۹	۷.۴ قلاب اعتبارسنجی مدل Item – خط‌به‌خط
۱۶۲	۷.۵ رزرو اتمی موجودی – server/src/lib/stock.js
۱۶۳	۷.۶ پیگیری سفارش بدون حساب – server/src/lib/track.js
۱۶۵	۷.۷ تقویم شمسی – server/src/lib/jalali.js
۱۷۰	۷.۸ الگوریتم پرفروش‌ها – server/src/routes/stats.js
۱۷۱	۷.۹ تولید تصویر – web/src/lib/art.js
۱۷۵	۷.۱۰ قالب‌بندی اعداد فارسی
۱۷۶	۷.۱۱ الگوریتم‌های کوچک‌تر
۱۷۸	۷.۱۲ نگه‌بان‌های ShopContext – الگوریتمی که هیچ عددی حساب نمی‌کند
۱۸۱	۸. سیستم طراحی و استایل
۱۸۱	۸.۱ فلسفه بصری
۱۸۱	۸.۲ توکن‌های طراحی
۱۸۳	۸.۳ تایپوگرافی
۱۸۶	۸.۴ فاصله‌گذاری و چیدمان
۱۸۷	۸.۵ راست‌به‌چپ – مهم‌ترین جنبه فنی استایل
۱۸۸	۸.۶ قرارداد نام‌گذاری کلاس‌ها
۱۸۹	۸.۷ اجزای رابط کاربری
۱۹۲	۸.۸ جزئیات بصری خاص
۱۹۳	۸.۹ واکنش‌گرایی
۱۹۵	۸.۱۰ استایل چاپ رسید
۱۹۹	۸.۱۱ دسترس‌پذیری در سطح CSS
۱۹۹	۸.۱۲ استایل پنل مدیریت
۲۰۱	۹. تصمیمات معماری: «چرا این روش؟»
۲۰۱	تصمیم ۱ – یک مدل Item برای هر سه نوع کالا
۲۰۲	تصمیم ۲ – slug به‌عنوان کلید پیوند، نه ObjectId
۲۰۳	تصمیم ۳ – قیمت‌ها در سرور دوباره حساب می‌شوند
۲۰۳	تصمیم ۴ – یک بسته مشترک، نه دو کپی آینه‌ای
۲۰۵	تصمیم ۵ – تقویم شمسی دست‌نویس

۲۰۶	تصمیم ۶ – تصویر تولیدی به جای عکس محصول
۲۰۷	تصمیم ۷ – محتوای سایت در پایگاه داده با schema بیرونی
۲۰۸	تصمیم ۸ – Context API به جای کتابخانه مدیریت حالت
۲۰۸	تصمیم ۹ – کلید مرکب برای ردیف سب
۲۰۹	تصمیم ۱۰ – تخفیف بر پایه وزن، نه مبلغ
۲۱۰	تصمیم ۱۱ – پرفروش‌ها از سفارش‌های واقعی + دو سوپاپ دستی
۲۱۱	تصمیم ۱۲ – پیام‌های کنسول انگلیسی، پیام‌های مرورگر فارسی
۲۱۱	تصمیم ۱۳ – tokenVersion به جای فهرست سیاه توکن
۲۱۲	تصمیم ۱۴ – مسیر نسبی و پراکسی، به جای آدرس مطلق API
۲۱۳	تصمیم ۱۵ – seed غیرمخرب به صورت پیش‌فرض
۲۱۴	تصمیم ۱۶ – پیش‌نمایش زنده با همان کامپوننت واقعی
۲۱۵	تصمیم ۱۷ – سب در localStorage با پاک‌سازی هوشمند
۲۱۶	تصمیم ۱۸ – دو فرآیند و یک مبدأ
۲۱۷	تصمیم ۱۹ – بدون درگاه پرداخت
۲۱۷	تصمیم ۲۰ – بدون حساب کاربری برای مشتری
۲۱۸	تصمیم ۲۱ – رفتن به Next.js، بعد از اینکه یک بار «نه» گفته بودیم
۲۱۹	تصمیم ۲۲ – سیاست امنیتی با nonce، و بهایش: force-dynamic
۲۱۹	تصمیم ۲۳ – مودال کالا پشت یک آدرس واقعی
۲۲۰	تصمیم ۲۴ – og:type: product را خود صفحه رندر می‌کند
۲۲۱	تصمیم ۲۵ – صفحه کالا فهرست کامل را فقط وقتی می‌خواند که سب لازمش دارد
۲۲۲	تصمیم ۲۶ – توست از کانتکتست بیرون آمد، به یک انبار مازولی
۲۲۳	تصمیم ۲۷ – چاپ با display: none، به بهای یک رندر دوم
۲۲۴	تصمیم ۲۸ – هم‌رسانی را نوع نشانگر تعیین می‌کند، نه بودن navigator.share
۲۲۵	تصمیم ۲۹ – تصویر در زمان سرو بهینه می‌شود، نه در زمان آپلود
۲۲۷	جدول جمع‌بندی
۲۲۸	۱۰. پرسش و پاسخ آماده
۲۲۸	بخش الف – معماری کلی
۲۳۰	بخش ب – قیمت‌گذاری و منطق تجاری
۲۳۲	بخش ج – امنیت
۲۳۶	بخش د – پایگاه داده و مدل‌سازی
۲۳۸	بخش ه – تقویم شمسی و گزارش‌ها
۲۴۰	بخش و – رابط کاربری و RTL
۲۴۳	بخش ز – تغییرهای تازه
۲۴۸	بخش ح – مهاجرت به Next.js
۲۵۱	بخش ط – هم‌رسانی، چاپ و تصویر

۱۱. نقاط ضعف فعلی و مسیر بهبود ۲۵۶

وضعیت فعلی ۲۵۶

۱۱.۱ امنیت ۲۵۸

۱۱.۲ تست و کیفیت کد ۲۶۵

۱۱.۳ کارایی ۲۷۳

۱۱.۴ SEO ۲۷۸

۱۱.۵ دسترس‌پذیری ۲۸۵

۱۱.۶ قابلیت‌های تجاری غایب ۲۸۶

۱۱.۷ زیرساخت ۲۸۹

۱۱.۸ نقشه راه پیشنهادی ۲۹۰

۱۱.۹ آنچه نباید عوض شود ۲۹۲

۱۲. واژه‌نامه فنی ۲۹۴

۱۲.۱ اصطلاحات عمومی توسعه وب ۲۹۴

۱۲.۲ پایگاه داده و مدل‌سازی ۲۹۵

۱۲.۳ امنیت ۲۹۶

۱۲.۴ اصطلاحات دامنه قهوه ۲۹۷

۱۲.۵ اصطلاحات کسب‌وکار پروژه ۲۹۹

۱۲.۶ اصطلاحات رابط کاربری و طراحی ۳۰۰

۱۲.۷ اصطلاحات تقویم و بومی‌سازی ۳۰۲

۱۲.۸ الگوها و اصول ۳۰۲

۱۲.۹ تست و ابزار توسعه ۳۰۴

۱۲.۱۰ فرمان‌ها و فایل‌های کلیدی ۳۰۴

۱۲.۱۱ اصطلاحات معماری (App Router) Next ۳۰۶

هر ارجاع در این سند به فایل و تابع واقعی مخزن اشاره می‌کند. تکه‌کدها عیناً از منبع برداشته شده‌اند و نمودارها همگی به‌صورت SVG درون‌خطی رسم شده‌اند.

۱. معرفی و نمای کلی پروژه

۱.۱ این پروژه چیست

رُست‌خانه دانه یک فروشگاه اینترنتی کامل برای یک رُستری قهوه تخصصی است: از ویتترین محصولات و سبد خرید گرفته تا پنل مدیریت کالاها، سفارش‌ها، محتوای سایت، باشگاه مشتریان و گزارش‌های فروش با تقویم شمسی.

کل پروژه با پشته MERN نوشته شده — Node.js و MongoDB، Express، React — و تمام رابط کاربری‌اش فارسی و راست‌به‌چپ است.

اما آنچه این پروژه را از یک فروشگاه معمولی جدا می‌کند، سه تصمیم است که در تک‌تک لایه‌ها ردّ خودشان را گذاشته‌اند:

یکم — قهوه به گرم فروخته می‌شود، نه به بسته. هیچ محصولی «بسته ۲۵۰ گرمی» نیست. قیمت ذخیره‌شده هر قهوه، قیمت هر کیلوگرم است و مشتری وزن دلخواهش را انتخاب می‌کند. این یک تصمیم تجاری است که مستقیماً به یک تصمیم فنی تبدیل شده: در `shared/pricing.js` تابع `priceFor(pricePerKg, grams)` هسته همه محاسبات است — یک فایل در بسته مشترک `@ghahve/shared` که مرورگر و سرور هر دو همان را import می‌کنند — و در `server/src/models/Item.js` فیلد `price` توضیح صریح دارد: «قهوه و پودر: قیمت هر کیلوگرم — ابزار: قیمت هر عدد».

دوم — مشتری می‌تواند میکس خودش را بسازد. پنج میکس ویژه خانه وجود دارد (`espresso-100`، `espresso-70-30`، `cold-brew`، `shabneshin`، `espresso-50-50`) که هرکدام ترکیب پیشنهادی ما را دارند — ولی مشتری می‌تواند با اهرم‌ها نسبت هر دانه را عوض کند، دانه‌ای را بردارد، یا هر قهوه دیگری از فهرست فروشگاه را جایش بگذارد. قیمت هر کیلوی میکس همان لحظه از میانگین وزنی قیمت دانه‌ها به‌علاوه دستمزد میکس حساب می‌شود. ریاضی جابه‌جایی اهرم‌ها در `web/src/lib/blend.js` است و خود قیمت در `shared/pricing.js` (تابع `blendPricePerKg`) — همان تابعی که سرور هم موقع ثبت سفارش صدا می‌زند.

سوم — هیچ عکس محصولی وجود ندارد. تصویر هر یک از ۱۰۶ کالای اولیه، یک SVG است که در همان لحظه از روی `slug` کالا تولید می‌شود. فایل `web/src/lib/art.js` با ۱,۰۲۵ خط کد، ۳۶ طرح ابزار و ۱۴ طرح پودر و یک مولد دانه قهوه دارد. مولد عدد شبه‌تصادفی `seeded(slug)` باعث می‌شود طرح هر کالا همیشه یکسان بماند. مدیر البته می‌تواند عکس واقعی آپلود کند؛ آن‌وقت عکس جای طرح تولیدی می‌نشیند (`web/src/components/CardArt.jsx`).

این SVG روی سرور ساخته می‌شود، نه در مرورگر: `art.js` تابع خالص است و `CardArt` هیچ هوکی ندارد، پس کارتها داخل همان HTML اولی می‌آیند که سرور می‌فرستد. بهایش حجم HTML است و سودش این است که صفحه بدون اجرای هیچ جاوااسکریپتی تصویر دارد — بحث کاملش در فصل ۱۰، پرسش ۳۸.

۱.۲ یک پوشه در ریشه که جزء برنامه نیست

وقتی مخزن را باز می‌کنید، در ریشه یک پوشه می‌بینید که ممکن است گمراه‌کننده باشد:

img/ ۱۱ فایل SVG

این بازمانده نسخه اولیه ایستای پروژه است: یک فروشگاه تک‌فایلی بدون سرور که داده محصولات داخل خود جاوااسکریپتش به‌صورت آرایه نوشته شده بود. سه فایل اصلی آن نسخه — `index.html`، `script.js` و `style.css` — دیگر در مخزن نیستند؛ فقط همین پوشه تصویر مانده.

و همان تصویرها یک نسخه دوم هم دارند، در `web/public/img/`. آن یکی است که سایت واقعاً سرو می‌کند. پس پوشه ریشه امروز نه اجرا می‌شود و نه خوانده می‌شود — فقط مرجع تاریخی است (مورد ۱۸ فصل ۱۱).

web/ Next.js (App Router) → (فروشگاه و پنل) پورت ۳۰۰۰
 server/ Express + Mongoose → (فقط API) پورت ۴۰۰۰
 shared/ منطق مشترک هر دو → @ghahve/shared

پوشه `client/` – که تا پیش از مورد ۲۶ فروشگاه را روی ویت اجرا می‌کرد – دیگر وجود ندارد. هر جای این سند که نامش می‌آید، اشاره به گذشته است، نه به فایلی که امروز در مخزن باشد.

اهمیت این نکته در کامنت‌های کد پیداست. مثلاً بالای `web/src/lib/format.js` نوشته شده «قالب‌بندی اعداد و وزن – عیناً مثل نسخه اولیه سایت» و بالای `web/src/lib/art.js` نوشته «تولید تصویر محصولات – عیناً از نسخه اولیه سایت». یعنی هنگام بازنویسی به MERN، تصمیم گرفته شده که **ظاهر و رفتار دقیقاً حفظ شود** و فقط لایه داده و مدیریت عوض شود. این را در فصل ۹ به‌عنوان یک تصمیم معماری بررسی می‌کنیم.

۱.۳ سه نوع کالا، یک مدل

فروشگاه سه دسته کالا دارد، اما در پایگاه داده فقط یک مجموعه به‌نام `items` وجود دارد. فیلد `kind` نوع را مشخص می‌کند:

kind	فارسی	واحد فروش	ویژگی خاص
coffee	قهوه	گرم (قیمت هر کیلو)	آسیاب‌شدنی، می‌تواند میکس باشد
gear	ابزار	عدد (قیمت هر عدد)	فهرست «سازگار با» دارد
powder	پودر	گرم (قیمت هر کیلو)	فهرست «پیشنهاد سرو» دارد

جدول `IS_WEIGHED` در `shared/taxonomy.js` این تفاوت را تعریف می‌کند:

```
export const IS_WEIGHED = { coffee: true, powder: true, gear: false };
```

و در سمت کلاینت، جدول `KINDS` در `web/src/lib/groups.js` همان تفاوت را با جزئیات بیشتر – برچسب، وزن‌های پیش‌فرض، نام سنج، متن مرتب‌سازی – نگهداری می‌کند. همین یک جدول باعث می‌شود یک کامپوننت `ItemCard` بتواند هر سه نوع کالا را رندر کند.

هر نوع کالا دسته‌های خودش را دارد (`GROUPS_BY_KIND` در `shared/taxonomy.js`):

- قهوه: `light`، `medium`، `dark`، `espresso`، `decaf`
- ابزار: `pourover`، `stovetop`، `grinder`، `kettle`، `cups`، `barista`
- پودر: `chocolate`، `matcha`، `masala`، `spice`، `milk`، `sweet`

۱.۴ داده اولیه

اسکرپت `server/src/seed.js` پایگاه داده را با ۱۰۶ کالا پر می‌کند: ۳۲ قهوه (۸ روشن، ۱۱ متوسط، ۶ تیره، ۵ اسپرسو، ۲ بدون کافئین)، ۴۰ ابزار و ۳۴ پودر. این داده در `server/src/data/seed-items.js` است. سپس `server/src/data/seed-enrich.js` سه چیز اضافه می‌کند:

1. `TASTE_TAGS` – برچسب طعمی هر یک از ۳۲ قهوه (مثلاً `yirgacheffe` → `['floral', 'fruity']`). این برچسب‌ها پایه بخش «چه طعمی دوست دارید؟» هستند.
2. `STORIES` – متن معرفی کامل هر قهوه در سه بخش: `story` (از کجا می‌آید)، `taste` (در فنان چه می‌چشید)، `recommend` (چطور دم‌ش کنید).
3. `BLENDS` – ترکیب پنج میکس ویژه خانه با درصدها و دستمزد میکس.

نکته مهم در طراحی seed این است که غیرمخرب است. تابع `enrichExisting` فقط فیلدهایی را پر می‌کند که خالی باشند، و کامنت خط ۱۰۳ فایل `seed.js` صریحاً توضیح می‌دهد چرا `featured` را تعیین نمی‌کند:

«پیشنهاد ما» و «پرفروش‌ها» را seed تعیین نمی‌کند. این‌ها انتخاب مدیرند و فقط در فرم خود کالا (بخش «وبترین») روشن می‌شوند – وگرنه هر بار seed، سلیقه مدیر را بازنویسی می‌کرد.

اگر بخواهید همه‌چیز از نو ساخته شود، `npm run seed:reset` سوییچ `--reset` را می‌فرستد و همه‌چیز پاک و بازسازی می‌شود.

۱.۵ نقشه صفحه‌ها

فروشگاه یک صفحه بلند است (`app/page.jsx`) که بدنه‌اش در `web/src/components/HomeShell.jsx` است و با لنگر پیمایش می‌شود. ترتیب بخش‌ها دقیقاً همان ترتیبی است که در `HomeShell` رندر می‌شوند:

ترتیب	بخش	کامپوننت	id
۱	هدر چسبان	Header	top
۲	تصویر اصلی + ترازوی قیمت	Hero	—
۳	نوار اطمینان	Strip	—
۴	چرا از ما بخرید	WhyUsSection	why
۵	پیشنهاد روز	PicksSections	daily
۶	پرفروش‌ها	PicksSections	bestsellers
۷	میز قهوه‌ها	CatalogSection	products
۸	چه طعمی دوست دارید؟	SuggestSection	suggest
۹	میکس‌های ویژه	BlendsSection	blends
۱۰	روش‌های دم‌آوری	BrewSection	brew
۱۱	قفسه ابزار	CatalogSection	gear
۱۲	میز پودرها	CatalogSection	powders
۱۳	داستان کارگاه	StorySection	story
۱۴	خرید کیلویی	BulkSection	bulk
۱۵	راهنمای رست	GuideSection	guide
۱۶	باشگاه مشتریان	ClubSection	club

ترتیب	بخش	کامپوننت	id
۱۷	درباره ما	AboutSection	about
۱۸	گالری کافه	CafeSection	cafe
۱۹	فوتر	Footer	contact

روی همه این‌ها، `CartDrawer` به صورت کشوی کناری و یک `toast` شناور نشسته‌اند.

نکته ظریفی در ترتیب هست که ارزش گفتن دارد: `WhyUsSection` بیرون از شرط `loading` رندر می‌شود. کامنتش در `HomeShell.jsx` توضیح می‌دهد:

چرا از ما بخريد — منتش فقط به محتوا وابسته است، پس منتظر آمدن فهرست کالاها نمی‌ماند.

با رندر سمت سرور این شرط عملاً هیچ‌وقت روشن نمی‌شود — فهرست کالاها پیش از ساخته شدن HTML خوانده شده است — ولی `reload()` هنوز وجود دارد و بعد از آن ممکن است لازم شود. پس شاخه‌ها نگه داشته شده‌اند.

سه صفحه دیگر هم هست که با مورد ۲۶ و ۲۷ اضافه شدند:

آدرس	فایل	چیست
<code>/coffee/:slug</code> <code>/gear/:slug</code> <code>/powder/:slug</code>	<code>app/coffee/[slug]/</code> <code>app/_item/itemRoute.jsx</code>	صفحه اختصاصی هر کالا، روی سرور رندر شده
همان آدرس‌ها، وقتی از داخل سایت کلیک شوند	<code>app/@modal/(.)coffee/[slug]/</code> <code>app/_item/itemModalRoute.jsx</code>	همان کالا به شکل مودال، روی صفحه فعلی
<code>/track</code>	<code>app/track/page.jsx</code> <code>components/Track.jsx</code>	پیگیری سفارش با «کد + موبایل»

پنل مدیریت هشت صفحه زیر دروازه نشست دارد (`web/src/app/admin/(panel)/`)، به علاوه صفحه ورود و یک تغییر مسیر که بیرون از دروازه‌اند:

مسیر	صفحه	کار
<code>/admin/login</code>	<code>AdminLogin</code>	ورود — بیرون از دروازه
<code>/admin</code>	<code>/admin/items</code> →	تغییر مسیر (<code>app/admin/page.jsx</code>)
<code>/admin/items</code>	<code>AdminItems</code>	فهرست، روشن/خاموش، حذف
<code>/admin/items/new</code>	<code>AdminItemForm</code>	افزودن
<code>/admin/items/:id</code>	<code>AdminItemForm</code>	ویرایش با پیش‌نمایش زنده
<code>/admin/orders</code>	<code>AdminOrders</code>	سفارش‌ها و تغییر وضعیت
<code>/admin/reports</code>	<code>AdminReports</code>	گزارش شمسی + خروجی
<code>/admin/content</code>	<code>AdminContent</code>	ویرایش متن‌های سایت
<code>/admin/club</code>	<code>AdminClub</code>	اعضای باشگاه + CSV
<code>/admin/settings</code>	<code>AdminSettings</code>	تغییر نام کاربری و رمز

۱.۶ اعداد پروژه

مقدار	سنجه
۸۶ فایل (web/src)	فایل‌های منبع وب
۲۷ فایل (server/src)	فایل‌های منبع سرور
۳ (seo.js , taxonomy.js , pricing.js)	فایل‌های بسته مشترک
web/src/style.css (۳,۲۹۹ خط)	بزرگ‌ترین فایل
web/src/lib/art.js (۱,۰۲۵ خط)	بزرگ‌ترین فایل JS
۳۲ endpoint (شامل /api/health) در ۷ روتر	مسیرهای API
۲ - /robots.txt و /sitemap.xml	مسیرهای غیر-API سرور
صفحه اصلی، سه صفحه کالا، سه مودال رهگیری‌شده، /track، ده مسیر پنل	مسیرهای Next
۵ (Content , ClubMember , Admin , Order , Item)	مدل‌های Mongoose
۱۱	وابستگی‌های تولیدی سرور
۴ (@ghahve/shared , react-dom , react , next)	وابستگی‌های تولیدی وب
۳۸۰ سنجه در ۲۲ فایل	تست‌ها

توجه کنید که هیچ کتابخانه مدیریت حالت، هیچ UI kit، هیچ CSS framework، هیچ کتابخانه تقویم شمسی، هیچ کتابخانه نمودار و هیچ کتابخانه سئویی در کار نیست. نمودار میله گزارش‌ها یک `<i style={{inlineSize: '...%'}}>` ساده است، تقویم شمسی در ۲۸۳ خط دست‌نویس شده، و مسیریابی خود ساختار پوشه‌های `web/src/app` است – نه یک کتابخانه.

۱.۷ اجرا در دو دستور

```
npm run setup # پر کردن پایگاه داده + workspace یک نصب برای هر سه
npm run dev   # روی Next ۳۰۰۰ و API اجرای همزمان
```

یک `npm install` در ریشه کافی است: `shared`، `server` و `web` هر سه workspace همان یک نصباند و فقط یک `package-lock.json` وجود دارد.

پیش‌نیاز فقط این است که سرویس MongoDB روی همین ماشین روشن باشد. اگر نبود، `server/src/db.js` پیام روشنی می‌دهد و دستور `net start MongoDB` را نشان می‌دهد.

بعد از اجرا:

- فروشگاه: `http://localhost:3000`
- پنل: `http://localhost:3000/admin`
- API: `http://localhost:4000/api`

حساب اولیه مدیر از `server/.env` خوانده می‌شود و پیش‌فرضش `admin` / `change-me` است.

در حالت تولید، `npm run build` خروجی Next را در `web/.next` می‌سازد و `npm start` دو فرآیند را بالا می‌آورد: Next روی ۳۰۰۰ و Express روی ۴۰۰۰. تا پیش از مورد ۲۶ یک فرآیند کافی بود، چون همان Express فایل‌های ساخته‌شده فرانت را هم سرو می‌کرد؛ حالا صفحه‌ها روی سرور رندر می‌شوند و Express فقط API است.

از دید بازدیدکننده همه‌چیز یک مبدأ است: مرورگر فقط با ۳۰۰۰ حرف می‌زند و Next درخواست‌های `/api`، `/uploads`، `/sitemap.xml` و `/robots.txt` را به Express می‌فرستد. متن قبلی – که سرو شدن فرانت از Express را توضیح می‌داد – دیگر معتبر نیست:

```
const CLIENT_DIST = path.join(__dirname, '..', '..', 'client', 'dist');
app.use(express.static(CLIENT_DIST, { fallthrough: true }));
app.get(/^(!\s*\api\s*\uploads).*/ (req, res, next) => {
  res.sendFile(path.join(CLIENT_DIST, 'index.html'), (err) => {
    if (err) next();
  });
});
```

آن regex منفی‌نگر (`negative lookahead`) کاری می‌کرد که هر آدرسی که با `/api` یا `/uploads` شروع نشود، به `index.html` برسد – الگوی استاندارد **SPA fallback** تا مسیرهای React Router مثل `/admin/orders` بعد از refresh هم کار کنند. امروز هیچ‌کدام از این چند خط در `server/src/index.js` نیست: آدرس ناشناخته در Express یک JSON چهار کلمه‌ای می‌گیرد (`{ 'این آدرس وجود ندارد': error }`) و صفحه ۴۰۴ واقعی را Next می‌سازد.

۲. پشتۀ فناوری و دلیل انتخاب هر ابزار

۲.۱ فهرست کامل وابستگی‌ها

پروژه سه `package.json` دارد. ریشه فقط هماهنگ‌کننده است و هیچ وابستگی تولیدی ندارد.

ریشه — `package.json`

```
{
  "name": "ghahve",
  "private": true,
  "workspaces": ["shared", "server", "web"],
  "scripts": {
    "setup": "npm install && npm run seed",
    "dev": "concurrently -n \"api,web\" -c \"yellow,magenta\" \"npm:dev:server\" \"npm:dev:web\"",
    "dev:server": "npm run dev -w server",
    "dev:web": "npm run dev -w ghahve-web",
    "build": "npm run build -w ghahve-web",
    "start": "concurrently -n \"api,web\" -c \"yellow,magenta\" \"npm:start:server\" \"npm:start:web\"",
    "test": "vitest run",
    "docs:pdf": "npm run docs:build && npm run docs:check && npm run docs:render"
  }
}
```

سه چیز در همین چند خط تغییر کرده و هر سه معنا دارند: `workspaces` جای `npm --prefix` را گرفت (یک نصب، یک `package-lock.json`)، `dev:client` شد `dev:web`، و `start` دیگر یک فرآیند نیست — دو تاست، چون Express و Next جدا اجرا می‌شوند.

وابستگی‌های توسعه ریشه هم فهرست کوتاهی نیست دیگر: `concurrently` برای اجرای همزمان، `jsdom` و `vitest` برای تست، `eslint` و `prettier` برای کیفیت کد، و `marked` و `playwright` و `pdfjs-dist` که فقط برای ساختن همین سند (`npm run docs:pdf`) لازم‌اند.

نقش	نسخه	بسته
چارچوب HTTP	^4.19.2	express
ODM برای MongoDB	^8.6.0	mongoose
امضا و بررسی توکن ورود مدیر	^9.0.2	jsonwebtoken
هش رمز عبور	^2.4.3	bcryptjs
آپلود تصویر (multipart/form-data)	^2.2.0	multer
فشرده‌سازی gzip پاسخ‌ها (مورد ۲۴)	^1.8.1	compression
هدرهای امنیتی پاسخ‌های API (مورد ۷)	^8.3.0	helmet
سه محدودکننده نرخ (مورد ۱ و ۲)	8.6.2	express-rate-limit
اجازه درخواست از مبدأ فرانت	^2.8.5	cors
خواندن server/.env	^16.4.5	dotenv
منطق مشترک با مرورگر (workspace محلی)	*	@ghahve/shared

نقش	نسخه	بسته
فریم‌ورک: مسیریابی، رندر سمت سرور، build	^16.3.1	next
کتابخانه رابط کاربری	^19.2.0	react-dom + react
منطق مشترک با سرور	*	@ghahve/shared

جمعاً ۱۱ وابستگی تولیدی در سرور و ۴ در وب — و در هر دو فهرست، `@ghahve/shared` یک بسته محلی است نه چیزی که از رجیستری بیاید. این عدد کم، خودش یک تصمیم است و در ادامه دلیلش را می‌بینیم.

مسیریابی در این فهرست نیست، و این عمده است: جدول مسیرها ساختار پوشه‌های `web/src/app` است. `react-router-dom` با مورد ۲۶ حذف شد.

۲.۲ چرا MERN؟

پروژه یک فروشگاه با ساختار داده ناهمگن است. سه نوع کالا داریم که بعضی فیلدهایشان مشترک است (نام، قیمت، دسته، تصویر) و بعضی فقط برای یک نوع معنی دارد:

- `mat` (جنس) فقط برای ابزار
- `tone` (رنگ پودر) فقط برای پودر
- `components` و `surcharge` فقط برای میکس‌های قهوه

در یک پایگاه داده رابطه‌ای، این یعنی یا جدول‌های جداگانه با join، یا یک جدول پهن با ستون‌های NULL فراوان، یا الگوی EAV. در MongoDB، سند فقط فیلدهایی را دارد که لازم است. کامنت بالای itemSchema در server/src/models/Item.js دقیقاً همین را می‌گوید:

```
/*
یک مدل واحد برای هر سه نوع کالا: قهوه، ابزار، پودر.
تفاوت‌ها با فیلد kind مشخص می‌شود. یکی بودن مدل باعث
می‌شود پتل مدیریت و مسیرهای API هم یکی باشند و جای
کمتری برای اشتباه بماند.
*/
```

دلیل دوم، مدل Content است. متن‌های سایت در سندهایی با شکل {key: String, data: Mixed} ذخیره می‌شوند. Mixed یعنی «هر شکلی که خواستی». به همین دلیل، اضافه کردن یک بخش تازه به سایت هیچ مهاجرت پایگاه داده‌ای لازم ندارد. کامنت server/src/models/Content.js این را صریح می‌گوید:

هر بخش یک کلید دارد و شکل داده خودش را – تا وقتی بخواهیم بخش تازه‌ای اضافه کنیم مجبور به مهاجرت پایگاه داده نشویم.

دلیل سوم، یک‌زبانه بودن است. همان pricing.js که در مرورگر اجرا می‌شود، عیناً در سرور هم اجرا می‌شود. اگر بک‌اند PHP یا Python بود، باید فرمول قیمت‌گذاری دو بار به دو زبان نوشته می‌شد و همگام نگه‌داشتنش سخت‌تر بود.

اما هزینه‌اش را هم بشناسید: MongoDB یکپارچگی ارجاعی ندارد. در این پروژه، Item.components[].slug به یک قهوه دیگر اشاره می‌کند اما هیچ FOREIGN KEY ای وجود ندارد. برای همین در server/src/routes/items.js تابعی به نام checkBlend نوشته شده که این را دستی بررسی می‌کند:

```
const found = await Item.find({ slug: { $in: slugs }, kind: 'coffee' })
    .select('slug isBlend').lean();
const bySlug = new Map(found.map((f) => [f.slug, f]));

for (const s of slugs) {
  const bean = bySlug.get(s);
  if (!bean) return `دانه «${s}» میکس گذاشت`;
  if (bean.isBlend) return `خودش یک میکس است و نمی‌تواند جزء میکس دیگری باشد «${s}»`;
}
```

این کاری است که در SQL با یک FOREIGN KEY و یک CHECK رایگان به دست می‌آید.

۲.۳ چرا Mongoose و نه درایور خام؟

Mongoose سه چیز به پروژه اضافه می‌کند که هر سه در کد استفاده جدی دارند:

(۱) اعتبارسنجی با پیام فارسی. هر فیلد پیام خطای خودش را دارد:

```
name: { type: String, required: [true, 'نام کالا الزامی است'], trim: true },
meter: { type: Number, default: 3,
  min: [1, 'کمترین مقدار ۱ است'],
  max: [5, 'بیشترین مقدار ۵ است'],
}
```

و در server/src/index.js، مبدل خطای سراسری همین پیام‌ها را مستقیم به کاربر می‌رساند:

```
if (err.name === 'ValidationError') {
  const first = Object.values(err.errors)[0];
  return res.status(400).json({ error: first?.message || 'اطلاعات وارد شده کامل نیست' });
}
```

یعنی مدیر به جای `E11000 duplicate key error collection...` پیام «این شناسه قبلاً استفاده شده است» می‌بیند.

۲) **قلاب** `pre('validate')`. پیچیده‌ترین منطق مدل `Item` – که در فصل ۷ خطبه‌خط بررسی می‌شود – در یک قلاب اجرا می‌شود که هم موقع `create` و هم موقع `save` کار می‌کند. این باعث می‌شود قواعدی مثل «فقط قهوه آسیاب می‌شود» و «مجموع درصدهای میکس باید ۱۰۰ باشد» از هر مسیری که کالا ذخیره شود اعمال شوند.

توجه کنید که به همین دلیل، مسیر ویرایش عمداً از `findByIdAndUpdate` استفاده نمی‌کند:

```
// server/src/routes/items.js
const item = await Item.findById(req.params.id);
Object.assign(item, data);
await item.save(); // تا pre('validate') اجرا شود
```

`findByIdAndUpdate` قلاب‌های سند را دور می‌زند و این قواعد اجرا نمی‌شدند.

۳) `virtual` و `toJSON` سفارشی. فیلد `weighed` در پایگاه داده ذخیره نمی‌شود، بلکه محاسبه می‌شود:

```
itemSchema.virtual('weighed').get(function () {
  return IS_WEIGHED[this.kind] === true;
});
```

و `toJSON` در مدل `Admin` باعث می‌شود `passwordHash` هیچ‌وقت – حتی به اشتباه – در پاسخ API نیاید:

```
toJSON: {
  transform(_d, ret) { delete ret.passwordHash; delete ret.__v; return ret; }
}
```

این «امنیت در عمق» است: حتی اگر برنامه‌نویس در یک مسیر اشتباهاً کل سند مدیر را برگرداند، هش رمز بیرون نمی‌رود.

۲.۴ چرا Express و نه Fastify یا NestJS؟

Express حداقلی است و همین‌جا برازنده است. کل `server/src/index.js` فقط ۹۳ خط است و شامل: تنظیم CORS، محدودیت بدنه، سرو استاتیک، ثبت هفت روتر، `SPA fallback`، `۴۰۴` و مبدل خطا.

الگوی معماری، روتر به ازای هر منبع است:

```
/api/auth    → routes/auth.js
/api/items   → routes/items.js
/api/orders  → routes/orders.js
/api/content → routes/content.js
/api/club    → routes/club.js
/api/stats   → routes/stats.js
/api/reports → routes/reports.js
```

NestJS با DI container، decorator، و ماژول‌بندی، برای تیم‌های بزرگ و پروژه‌های پیچیده ارزش دارد؛ اینجا فقط سربار مفهومی اضافه می‌کرد. Fastify سریع‌تر است اما این پروژه هیچ‌جا محدود به توان سرور نیست.

نکته ظریفی که Express اجازه داده، **استریم کردن خروجی** است. در `server/src/routes/reports.js` فایل CSV به‌جای ساخته شدن در حافظه، تکه‌تکه نوشته می‌شود:

```
const cursor = Order.find(filter).sort({ createdAt: -1 }).lean().cursor();
for await (const o of cursor) {
  if (mode === 'lines') { for (const row of lineRows(o)) res.write(csvRow(row)); }
  else { res.write(csvRow(orderRow(o))); }
}
res.end();
```

کامنت بالای این بخش دلایلش را می‌گوید: «فایل‌ها را تکه‌تکه می‌فرستیم تا حافظه سرور با پشتیبان بزرگ پر نشود.»

۲.۵ چرا JWT و نه session؟

انتخاب JWT در این پروژه سه دلیل عملی دارد:

- یک – سرور بدون حالت (stateless). هیچ فروشگاه session ای لازم نیست؛ نه Redis، نه جدول `sessions` در MongoDB.
- دو – فقط یک کاربر واقعی دارد. کل سیستم احراز هویت برای مدیر است. مشتری‌ها اصلاً حساب کاربری ندارند؛ سفارش با نام و شماره ثبت می‌شود و عضویت باشگاه هم بدون رمز است (کامنت `ClubMember.js`: «رمز عبوری در کار نیست»).
- سه – مشکل ابطال با یک ترفند ساده حل شده. ایراد کلاسیک JWT این است که نمی‌شود توکن صادرشده را باطل کرد. راه‌حل این پروژه یک عدد در سند مدیر است:

```
// server/src/models/Admin.js
tokenVersion: { type: Number, default: 0 }

// server/src/middleware/auth.js
export function signToken(admin) {
  return jwt.sign({ sub: String(admin._id), v: admin.tokenVersion },
    process.env.JWT_SECRET, { expiresIn: `${hours}h` });
}

export async function requireAdmin(req, res, next) {
  const payload = jwt.verify(token, process.env.JWT_SECRET);
  const admin = await Admin.findById(payload.sub);
  if (admin.tokenVersion !== payload.v) {
    return res.status(401).json({ error: 'رمز عوض شده است، دوباره وارد شوید' });
  }
  ...
}
```

و در `routes/auth.js` هنگام تغییر رمز:

```
await admin.setPassword(next_);
admin.tokenVersion += 1; // توکن‌های قدیمی باطل می‌شوند
await admin.save();
```

با یک عدد، همه نشست‌های قدیمی از کار می‌افتند – بدون جدول blacklist. و بلافاصله توکن تازه برگردانده می‌شود تا خود مدیر از پنل بیرون نیفتد.

نکته

requireAdmin در هر درخواست یک findById به پایگاه داده می‌زند. یعنی این JWT دیگر کاملاً stateless نیست. این معامله آگاهانه‌ای است: یک کوئری ارزان در ازای امکان ابطال فوری.

۲.۶ چرا bcryptjs و نه bcrypt؟

bcrypt (بدون js) یک ماژول native است که هنگام نصب باید کامپایل شود و روی ویندوز به Visual Studio Build Tools نیاز دارد. bcryptjs پیاده‌سازی خالص جاوااسکریپت است: کندتر، اما بدون هیچ دردسر نصبی.

با توجه به اینکه در کل عمر برنامه شاید چند ده بار عملیات هش انجام شود (ورود مدیر و تغییر رمز)، این کندی کاملاً بی‌اهمیت است. ضریب کار روی ۱۲ تنظیم شده که استاندارد امروزی است:

```
adminSchema.methods.setPassword = async function (plain) {  
  this.passwordHash = await bcrypt.hash(plain, 12);  
};
```

۲.۷ چرا React با Context و نه Redux؟

سه دلیل که هر سه از خود کد پیداست:

یک – **حجم حالت سراسری کم است.** کل چیزی که در ShopContext نگه داشته می‌شود: فهرست کالاها، متن‌های سایت، سبد خرید، آسیاب پیش‌فرض و یک toast. همین. بقیه حالت‌ها محلی‌اند: فیلترها در HomeShell.jsx، ترتیب و جست‌وجو در CatalogSection، ترکیب میکس در BlendCard، مرحله تسویه در CartDrawer.

با آمدن رندر سمت سرور (مورد ۲۶) نقش این Provider هم کمی عوض شد: دیگر خودش داده نمی‌گیرد. کالاها و متن‌ها روی سرور خوانده و به شکل پارامتر تحویل داده می‌شوند، و Provider فقط نگه‌دارنده حالت است. تنها استثنا پنل مدیریت است که صفحه‌هایش سروری نیستند و با پرچم LoadOnMount خودش می‌خواند.

دو – الگوی «Context + hook سفارشی» ارزان و خواناست:

```
const ShopContext = createContext(null);  
export const useShop = () => useContext(ShopContext);
```

هر کامپوننتی که به داده نیاز دارد، const { cart, addWeighed } = useShop() می‌نویسد. Redux یعنی action و reducer و selector و middleware – برای این اندازه، سربار خالص.

سه – مشکل rerender با useMemo مدیریت شده. ایراد اصلی Context این است که هر تغییر مقدار، همه مصرف‌کنندگان را rerender می‌کند. در این پروژه ساختارهای گران با useMemo و توابع با useCallback تثبیت شده‌اند:

```
const bySlug = useMemo(() => new Map(items.map((i) => [i.slug, i])), [items]);  
const lookup = useCallback((slug) => bySlug.get(slug), [bySlug]);  
const totals = useMemo(() => computeTotals(resolved, lookup), [resolved, lookup]);
```

شیء `value` که به `Provider` داده می‌شود، خودش با `useMemo` پوشیده نشده است. یعنی در هر رندر `ShopProvider` یک شیء تازه ساخته می‌شود و همهٔ مصرف‌کنندگان `rerender` می‌شوند. با اندازهٔ فعلی صفحه مشکلی ایجاد نمی‌کند، ولی نقطه‌ای است که در فصل ۱۱ به‌عنوان مسیر بهبود ذکر شده است.

۲.۸ چرا Next.js – و چرا اولش نه

این بخش دو نسخه دارد، چون تصمیمش یک بار عوض شد. متن اول عمداً می‌ماند: استدلال رد شدن، بخشی از تاریخ همین انتخاب است.

آنچه اول انتخاب شد: Vite

در برابر **CRA: CRA** عملاً بایگانی شده است. `Vite` در توسعه از ماژول‌های `ES` بومی مرورگر استفاده می‌کند، پس سرور در چند صد میلی‌ثانیه بالا می‌آید و `HMR` تقریباً آنی است.

در برابر **Next.js – استدلال آن روز**: `Next` رندر سمت سرور می‌دهد که برای `SEO` یک فروشگاه واقعی حیاتی است. اما `Next` با خودش یک مدل مسیریابی مبتنی بر فایل، مرز `server/client component` و زنجیرهٔ `build` سنگین‌تر می‌آورد. برای پروژه‌ای که هدفش نمایش معماری یک فروشگاه با پنل مدیریت است، `Vite` ساده‌تر و شفاف‌تر به‌نظر می‌رسید.

چه چیزی آن استدلال را برگرداند

سه چیز، و هیچ‌کدام «سلیقه» نبود (شرح کامل پای مورد ۲۶ در فصل ۱۱):

- ۱) راه میانی – تزریق `<head>` داخل `index.html` – کارت تلگرام را درست کرد ولی متن **صفحه** را نه، که همان مسئلهٔ اصلی بود.
- ۲) نگه‌داشتن آن راه میانی خودش هزینه داشت: `<head>` هر صفحهٔ کالا دو بار ساخته می‌شد (یک بار رشته در سرور، یک بار عنصر `DOM` در مرورگر) و یک پشتهٔ سه‌حالتی و یک تست همگامی لازم داشت تا واگرا نشوند.
- ۳) «`prerender` جواب نمی‌دهد» درست بود، ولی راه سومی هم بود که آن روز دیده نشد: رندر سمت سرور در هر درخواست. نه `build` تازه برای هر کالا، نه فهرستِ موقع ساخت.

آنچه Next در این پروژه واقعاً می‌کند

- ۱) مسیریابی بدون کتابخانه. پوشه = مسیر. سه پوشهٔ `coffee/[slug]`، `gear/[slug]` و `powder/[slug]` یک پیاده‌سازی مشترک را صدا می‌زنند، و `tests/next-routes.test.js` نگه می‌دارد که هیچ نوعی بی‌مسیر نماند.
- ۲) `generateMetadata` به‌جای مدیر `head`. همان توصیفی که `shared/seo.js` می‌سازد به `Next` داده می‌شود و `Next` رندرش می‌کند – یک رندرکننده، نه دو تا.
- ۳) مسیرهای رهگیری‌شده. مودال کالا با `app/@modal` بازسازی شد: کلیک از داخل سایت مودال را روی همان صفحه باز می‌کند، بازدید سرد صفحهٔ کامل را می‌دهد. جانشین `state.backgroundLocation` قبلی.
- ۴) پراکسی، همان‌طور که ویت می‌کرد. جای `vite.config.js` را `next.config.mjs` گرفت:

```
// web/next.config.mjs
async rewrites() {
  return [
    { source: '/api/:path*', destination: `${API_URL}/api/:path*` },
    { source: '/uploads/:path*', destination: `${API_URL}/uploads/:path*` },
    { source: '/sitemap.xml', destination: `${API_URL}/sitemap.xml` },
    { source: '/robots.txt', destination: `${API_URL}/robots.txt` }
  ];
}
```

نتیجه عملی همان است که با ویت بود: در `web/src/lib/api.js` هیچ جا `http://localhost:4000` نوشته نشده و همه جا مسیر نسبی است.

```
items: () => request('/api/items'),
```

تفاوت این است که حالا یک مصرف‌کننده دوم هم هست – خود سرور. برای همین `web/src/lib/data.js` جدا وجود دارد: روی سرور `fetch` مبدأ ندارد، پس آدرس Express از محیط (`API_URL`) می‌آید.

بهای که پرداخت شد

- کش ایستا نداریم. سیاست امنیتی صفحه‌ها با nonce کار می‌کند و nonce برای هر درخواست تازه است، پس همه صفحه‌ها پویا رندر می‌شوند.
- جاوااسکریپت بیشتر: ۲۲۵ کیلوبایت gzip در برابر ۱۲۹ کیلوبایت باندل ویت – ولی دیگر مانع اولین رنگ نیست.
- دو فرایند به جای یک.

۲.۹ مسیریابی: از React Router به پوشه‌ها

فروشگاه تقریباً یک صفحه است، اما پنل مدیریت نه. پنل هشت مسیر دارد و سه قابلیت لازم داشت. تا پیش از مورد ۲۶ این سه را `react-router-dom` می‌داد؛ حالا هر سه از خود App Router می‌آیند:

نیاز	پیش‌تر (react-router)	حالا (App Router)
layout مشترک	+ <code><Route element={<AdminLayout/>}></code> <code><Outlet/></code>	+ <code>app/admin/(panel)/layout.jsx</code> <code>children</code>
پارامتر مسیر	<code>useParams()</code> از <code>react-router</code>	<code>useParams()</code> از <code>next/navigation</code>
محافظت از مسیر	<code><Navigate to="/admin/login" replace /></code>	هدایت در افکت، با <code>router.replace</code>

دو نکته ریز که در این جابه‌جایی معنا داشتند:

۱) صفحه ورود بیرون از دروازه است. در نسخه قبل `/admin/login` یک مسیر جدا بیرون از `AdminLayout` بود. همان تقسیم با یک گروه مسیر حفظ شد: `app/admin/layout.jsx` نشست را برای همه فراهم می‌کند (صفحه ورود هم لازمش دارد تا بفهمد از قبل وارد شده‌اید یا نه)، و دروازه یک لایه پایین‌تر در `(panel)/layout.jsx` است.

۲) هدایت دیگر در رندر انجام نمی‌شود. App Router کامپوننتی مثل `<Navigate>` ندارد و هدایت باید بیرون از رندر باشد:

```
useEffect(() => {
  if (!checking && !admin) router.replace('/admin/login');
}, [checking, admin, router]);

if (checking || !admin) return <div className="admin-boot">در حال بررسی نشست</div>;
```

تا رفتنِ کاربر، همان پیام «بررسی نشست» دیده می‌شود — نه پنبلی که یک لحظه برق بزند و بعد برود.

۲.۱۰ چرا multer با تنظیمات سفارشی؟

فایل `server/src/lib/upload.js` سه محافظ دارد:

```
/* SVG عمداً در این فهرست نیست (مورد ۵ سند ضعیفها) */
export const ALLOWED = {
  'image/jpeg': '.jpg', 'image/png': '.png', 'image/webp': '.webp',
  'image/gif': '.gif', 'image/avif': '.avif'
};

const storage = multer.diskStorage({
  destination: (_req, _file, cb) => cb(null, UPLOAD_DIR),
  filename: (_req, file, cb) => {
    /* نام تصادفی - نام اصلی فایل ممکن است فارسی یا تکراری باشد */
    const ext = ALLOWED[file.mimetype] || path.extname(file.originalname).toLowerCase() || '.bin';
    cb(null, crypto.randomBytes(12).toString('hex') + ext);
  }
});

export const upload = multer({
  storage,
  limits: { fileSize: 4 * 1024 * 1024 }, // ۴ مگابایت
  fileFilter: (_req, file, cb) => {
    if (!ALLOWED[file.mimetype]) {
      return cb(new Error('فقط تصاویر JPG، PNG، WebP، AVIF یا GIF قابل آپلود است'));
    }
    cb(null, true);
  }
});
```

- نام تصادفی ۲۴ نویسه‌ای هم مشکل نام فارسی را حل می‌کند، هم برخورد نام‌ها را، و هم مهم‌تر: حملهٔ **path traversal** را می‌بندد. اگر نام اصلی فایل حفظ می‌شد، مهاجم می‌توانست `../../server/src/index.js` بفرستد.
- **whitelist** بر پایهٔ **MIME** نه پسوند.
- سقف ۴ مگابایت روی حجم.

روزی در این فهرست بود و برداشته شد (مورد ۵). SVG یک سند XML است، نه تصویر خام، و می‌تواند `<script>` داشته باشد؛ چون فایل‌ها از `/uploads` روی همان دامنه سرو می‌شوند، آن اسکریپت در مبدأ خود فروشگاه اجرا می‌شد و به توکن مدیر در `localStorage` هم دسترسی داشت.

بستن فهرست به‌تنهایی کافی نبود، چون هرچه پیش‌تر آپلود شده بود هنوز روی دیسک است. پس `setUploadHeaders` در همان فایل یک لایه دوم گذاشت: `Cross-Content-Security-Policy: default-src 'none'; sandbox`، `nosniff`، `Origin-Resource-Policy: same-origin`، و برای پسوندهای سندی (`.svg`، `.xml`، `.html`، ...) `Content-Disposition: attachment`. `tests/upload.test.js` هر دو لایه را می‌سنجد — از جمله اینکه تصویر عادی دانلود اجباری نشود، وگرنه کارت‌ها می‌شکستند.

۲.۱۱ ابزارهایی که عمداً استفاده نشده‌اند

این فهرست به‌اندازه فهرست وابستگی‌ها گویاست:

ابزار رایج	جایگزین در این پروژه
Redux / Zustand / Jotai	<code>useCallback</code> / <code>useMemo</code> + Context API
Tailwind / Bootstrap / MUI	CSS دست‌نویس با متغیرهای CSS
<code>date-fns</code> / <code>moment-jalaali</code> <code>jalaali</code>	<code>server/src/lib/jalali.js</code> (۲۸۳ خط دست‌نویس)
Chart.js / Recharts	یک <code></code> با <code>inlineSize</code> درصدی
<code>axios</code>	<code>fetch</code> بومی با یک wrapper به‌نام <code>request()</code>
Formik / <code>react-hook-form</code>	<code>useState</code> ساده در هر فرم
<code>lodash</code>	چند تابع کوچک در <code>format.js</code> و <code>blend.js</code>
TypeScript	JavaScript ساده با کامنت‌های توضیحی
Jest	تست‌ها با <code>Vitest</code> نوشته شده‌اند؛ دو تست کامپوننتی محیط خود را به <code>jsdom</code> عوض می‌کنند
Playwright برای تست	هست، ولی فقط برای رندر PDF همین مستندات (<code>npm run docs:pdf</code>) — نه برای تست end-to-end
کتابخانه سئو (<code>next-seo</code>) و مانندش)	<code>shared/seo.js</code> دست‌نویس + <code>Metadata API</code> خود Next
ESLint / Prettier (config)	فقط چند <code>eslint-disable-next-line</code> پراکنده
Docker	اجرای مستقیم روی ماشین

درباره `axios`: کل لایه شبکه یک تابع ۳۵ خطی است که همان کاری را می‌کند که پروژه لازم دارد — تنظیم هدرها، مدیریت توکن، تجزیه JSON و تبدیل خطای شبکه به پیام فارسی:


```

} catch {
  /* سرور خاموش است یا شبکه قطع است */
  throw new Error('ارتباط با سرور برقرار نشد. مطمئن شوید سرور روشن است');
}

```

این پیام برای کاربر ایرانی روی ویندوز، بسیار مفیدتر از `Network Error` است.

دربارهٔ تست: مورد ۱۱ فصل ۱۱ رفع شد. منطق قیمت‌گذاری در `pricing.js` و ریاضی میکس در `blend.js` توابع خالص (pure function) اند و همین باعث شد تست‌نویسی ارزان تمام شود – بیست‌ودو فایل و ۳۸۰ سنج، بدون مونگو.

۲.۱۲ متغیرهای محیطی

دو فایل تنظیمات جدا داریم، چون دو فرآیند جدا داریم: `server/.env` → `server/.env.example` و `web/.env.example` → `web/.env.local`. هیچ‌کدام از آن دو فایل واقعی در مخزن نیستند.

سرور – `server/.env.example`

```

PORT=4000
MONGODB_URI=mongodb://127.0.0.1:27017/ghahve
JWT_SECRET=یک-رشته-تصادفی-بلند-اینجا-بگذارید
TOKEN_HOURS=12
ADMIN_USERNAME=admin
ADMIN_PASSWORD=change-me
SITE_URL=http://localhost:3000
CLIENT_ORIGIN=http://localhost:3000
RATE_LIMIT_LOGIN_MAX=5
RATE_LIMIT_LOGIN_WINDOW_MIN=15
RATE_LIMIT_ORDER_MAX=5
RATE_LIMIT_ORDER_WINDOW_MIN=60
RATE_LIMIT_TRACK_MAX=20
RATE_LIMIT_TRACK_WINDOW_MIN=15
# RATE_LIMIT_DISABLED=1
# TRUST_PROXY=1

```

متغیر	استفاده	فایل	اگر نباشد
PORT	پورت شنود API	<code>index.js</code>	۴۰۰۰
MONGODB_URI	رشتهٔ اتصال؛ برای Atlas فقط همین عوض می‌شود	<code>db.js</code>	سرور بالا نمی‌آید
JWT_SECRET	کلید امضای توکن	<code>middleware/auth.js</code>	ورود کار نمی‌کند
TOKEN_HOURS	عمر توکن	<code>middleware/auth.js</code>	۱۲ ساعت
ADMIN_USERNAME ADMIN_PASSWORD	حساب اولیه، فقط در seed	<code>seed.js</code>	<code>/admin</code> <code>change-me</code>
SITE_URL	آدرس عمومی سایت – پایهٔ لینک‌های <code>sitemap.xml</code> و <code>robots.txt</code>	<code>lib/siteUrl.js</code>	از هدر <code>Host</code> ساخته می‌شود

متغیر	استفاده	فایل	اگر نباشد
CLIENT_ORIGIN	مبدأهای مجاز CORS، جدا با کاما	lib/cors.js	در توسعه باز، در تولید خطای مرگبار
RATE_LIMIT_LOGIN_*	سقف ورود مدیر	middleware/rateLimit.js	۵ در ۱۵ دقیقه
RATE_LIMIT_ORDER_*	سقف ثبت سفارش	middleware/rateLimit.js	۵ در ۶۰ دقیقه
RATE_LIMIT_TRACK_*	سقف پیگیری سفارش	middleware/rateLimit.js	۲۰ در ۱۵ دقیقه
RATE_LIMIT_DISABLED	خاموش کردن دستی محدودیت	middleware/rateLimit.js	روشن (در تست همیشه خاموش)
TRUST_PROXY	خواندن IP واقعی از X-Forwarded-For	index.js	خاموش، مگر در تولید

در این فایل نوشته نمی‌شود؛ آن را دستور اجرا تعیین می‌کند. با `production` سه چیز عوض می‌شود: `CLIENT_ORIGIN` اجباری می‌شود، HSTS روشن می‌شود، و اعتماد به پراکسی خودبه‌خود فعال می‌شود.

وب — `web/.env.example`

```
API_URL=http://localhost:4000
NEXT_PUBLIC_SITE_URL=http://localhost:3000
NEXT_PUBLIC_OG_IMAGE=/img/og-default.png
```

متغیر	استفاده	فایل	اگر نباشد
API_URL	آدرس Express — فقط روی سرور: پراکسی و خواندن داده	next.config.mjs lib/data.js	http://localhost:4000
NEXT_PUBLIC_SITE_URL	پایه canonical و og:url و آدرس‌های Product schema	lib/site.js	آدرس‌ها نسبی می‌شوند
NEXT_PUBLIC_OG_IMAGE	تصویر پیش‌نمایش لینک وقتی کالا عکس ندارد	lib/site.js	/img/og-default.png

هرچه با `NEXT_PUBLIC_` شروع شود به مرورگر هم می‌رسد؛ `API_URL` عمداً این پیشوند را ندارد، چون آدرس داخلی است و مرورگر همیشه مسیر نسبی می‌زند.

دو جفتی که باید با هم بخوانند

۱) `SITE_URL` و `NEXT_PUBLIC_SITE_URL` باید دقیقاً یکی باشند. این دو یک چیز را می‌سازند از دو طرف: Next با دومی `canonical` و `og:url` هر صفحه را می‌نویسد، و Express با اولی لینک‌های داخل `sitemap.xml` و خط `Sitemap:` در `robots.txt` را. اگر فرق کنند، نقشه سایت به آدرسی اشاره می‌کند که `canonical` همان صفحه تأییدش نمی‌کند — و گوگل دو نسخه از یک صفحه می‌بیند.

۲) هر دو آدرسِ سایت‌اند، نه آدرس API. این نکته‌ای است که به‌سادگی اشتباه می‌شود: `SITE_URL` در فایل سرور نوشته می‌شود ولی مقدارش پورت ۳۰۰۰ است، نه ۴۰۰۰. بازدیدکننده صفحه‌ها را از Next می‌گیرد، پس `<loc>` های نقشه سایت هم باید به همان‌جا اشاره کنند؛ با ۴۰۰۰، هر لینکِ نقشه سایت به سروری می‌رسید که فقط JSON می‌دهد. همین برای `CLIENT_ORIGIN` : مبدایی که مرورگر از آن درخواست می‌زند ۳۰۰۰ است.

هشدار امنیتی

این هشدار رفع شده (مورد ۶): تصمیم به `server/src/lib/cors.js` منتقل شد و در تولید، نبودِ `CLIENT_ORIGIN` خطای مرگبار است، نه پیش‌فرضِ نرم. متن زیر تاریخِ همان تصمیم است.

پیش‌تر نوشته بود:

```
app.use(cors({ origin: process.env.CLIENT_ORIGIN?.split(',') || true }));
```

اگر `CLIENT_ORIGIN` تنظیم نشده باشد، مقدار `true` یعنی هر مبدایی مجاز است. در توسعه راحت است، در تولید خطرناک. باید در محیط تولید حتماً مقداردهی شود.

فایل `.env` در `.gitignore` است و فقط `.env.example` در مخزن می‌ماند — با هشدار صریح در خودش: «این فایل را در جایی منتشر نکنید.»

۳. ساختار پوشه‌ها و معماری کلی

۳.۱ درخت کامل پروژه

```
Ghahve/
├── README.md           معرفی انگلیسی پروژه
├── .gitignore           .gitattributes .git-blame-ignore-revs
├── .prettierrc          .prettierignore eslint.config.mjs vitest.config.mjs
├── package.json         ها + اسکریپت‌ها workspace ریشه
├── img/                تنها بازمانده نسخه ایستای اولیه - SVG فایل ۱۱
                        (نسخه‌شان در web/public/img است و همان سرو می‌شود)
├── .github/workflows/ci.yml   build • تست • Prettier • ESLint • نصب
├── tests/              Vitest - ریشه، روی هر سه
├──   ├── pricing.test.js   پله‌های تخفیف، ارسال، قیمت میکس
├──   ├── blend.test.js     جابه‌جایی درصدها
├──   ├── blend-visual.test.js زمان‌بندی و هندسه حالت تصویری
├──   ├── card-art.test.js   SVG شدن نام کالا داخل escape
├──   ├── cart-storage.test.js بازاعتبارسنجی سبد ذخیره‌شده
├──   ├── cart-hydration.test.jsx (jsdom) «hydrate» سبد در چرخه «رندر سرور
├──   ├── item-card.test.jsx   (jsdom) لینک واقعی کارت به صفحه کالا
├──   ├── item-detail.test.jsx (jsdom) ShopProvider مودال کالا بدون
├──   ├── share.test.js       نشانی هم‌رسانی و انتخاب راه بر پایه نشانیگر
├──   ├── toast-store.test.js سرور snapshot عمر پیام، مشترک‌ها، و
├──   ├── item-routes.test.js شکل آدرس کالا و نگاشت نوع ⇔ بخش
├──   ├── next-routes.test.js هر نوع کالا پوشه مسیر خودش را دارد
├──   ├── next-metadata.test.js نکست metadata شیء ⇔ headTags
├──   ├── json-ld.test.js     و بی‌خطر بودن متن Product و LocalBusiness
├──   ├── sitemap.test.js    نقشه سایت و بیرون ماندن کالای خاموش XML
├──   ├── stock.test.js      رزرو و پس‌دادن موجودی
├──   ├── track.test.js      پیگیری سفارش و نمای عمومی
├──   ├── upload.test.js     /uploads فهرست مجاز و هدرهای
├──   ├── cors.test.js       CORS تصمیم
├──   ├── rate-limit.test.js خاموش بودن محدودیت در محیط تست
├──   ├── jalali.test.js     تقویم شمسی و بازه‌بندی
├──   └── taxonomy.test.js   و برچسب‌ها shared هم‌پوشانی کلیدهای
├── docs/               README-fa.md + همین مستندات
├── scripts/
├──   ├── og-image.mjs      (og-default.png) ساخت تصویر پیش‌نمایش لینک
├──   ├── docs/             فارسی PDF و documentation.html ساخت
├──   │   ├── build.mjs      diagrams.mjs paginator.js
├──   │   ├── render.mjs     check-svg.mjs zoom.mjs
├──   │   └── fonts/         HTML درون‌خط در JetBrains Mono، وزیرمتن و
├── shared/              ★ workspace - مشترک @ghahve/shared
├──   ├── package.json      بدون هیچ وابستگی، export سه
├──   ├── pricing.js         تنها نسخه محاسبه قیمت
├──   ├── taxonomy.js        تنها نسخه کلیدهای مجاز
├──   └── seo.js             <head>، JSON-LD، sitemap و robots متن
├── web/                 Next.js (App Router) - فروشگاه و پنل
├──   ├── package.json
├──   └── next.config.mjs    پراکسی، بسته مشترک، و بهینه‌ساز تصویر
```

.env.example	API_URL و NEXT_PUBLIC_*
public/img/	۱۱ همان SVG + og-default.png
.next/	build خروجی (در .gitignore)
src/	
proxy.js	بقیه هدرهای امنیتی صفحه + nonce با CSP
style.css	استایل فروشگاه
admin.css	استایل پنل + چاپ رسید
app/	جدول مسیرها = ساختار پوشه‌ها ★
layout.jsx	force-dynamic، فونت‌ها، شکاف مودال، توست، rtl، پوسته
page.jsx	صفحه اصلی - کالاها و متن‌ها را روی سرور می‌خواند
fonts.js	Vazirmatn + Lalezar با next/font
error.jsx	خطای ساخت صفحه (۵۰۰)
not-found.jsx	آدرس ناشناخته (۴۰۴)
_item/	پیاده‌سازی مشترک صفحه کالا، مودال و ۴۰۴ کالا
itemRoute.jsx	generateMetadata + صفحه کامل (سروری)
itemModalRoute.jsx	نیمه سروری مودال
ItemModal.jsx	(router.back) نیمه مشتری مودال
ItemBuyCard.jsx	ستون خرید - تنها تکه مشتری صفحه
ItemNotFound.jsx	خودش metadata کالا، با ۴۰۴
coffee/[slug]/	page.jsx + not-found.jsx سه پوشه، یک پیاده‌سازی
gear/[slug]/	
powder/[slug]/	
@modal/	مسیرهای رهگیری‌شده: مودال روی صفحه فعلی
track/	پیگیری سفارش
admin/	
layout.jsx	نشست (بیرون دروازه - ورود هم لازم دارد)
page.jsx	/admin/items هدایت به
login/	
(panel)/	دروازه + هشت صفحه
context/	
ShopContext.jsx	کالاها، سبد، محتوا، آسیاب
AuthContext.jsx	نشست مدیر
lib/	
api.js	تنها لایه ارتباط مرورگر با سرور
data.js	(ری‌اکت cache) خواندن داده روی سرور
metadata.js	نکست metadata به شيء <head> از توصیف
site.js	آدرس پایه و تصویر پیش‌نمایش، از محیط
cartStorage.js	خواندن و نوشتن سبد، با حافظه تزریق‌شده
seo.js	و ساعت کار ماشین‌خوان JSON-LD
share.js	نشانی هم‌رسانی، کپی، و قاعده نوع نشانگر
toastStore.js	انباره پیام کوتاه، بیرون از هر کانتکست
img.js	کدام تصویر بهینه می‌شود و کدام خام می‌ماند
blend.js	ریاضی جابه‌جایی درصدها
blendVisual.js	زمان‌بندی و هندسه حالت تصویری
cartLine.js	شکل ردیف سبد - مشترک بین دو حالت
groups.js	shared برچسب فارسی کلیدهای
format.js	عدد، پول، وزن، تاریخ
art.js	SVG + esc تولید
useReveal.js	انیمیشن ورود کارتها
useStorefrontRefresh.js	باطل‌کردن کش مسیرپای بعد از تغییر
components/	
Header.jsx	هدر چسبان + منو + دکمه سبد
SiteHeader.jsx	هدر + کشوی سبد (صاحب حالت باز/بسته)
HomeShell.jsx	بدنه صفحه اصلی - نگاه‌دارنده فیلترها
Hero.jsx	«تصویر اصلی + «ترازوی قیمت
CatalogSection.jsx	یک کامپوننت برای هر سه فهرست
ItemCard.jsx	کارت واحد سه‌نوعه

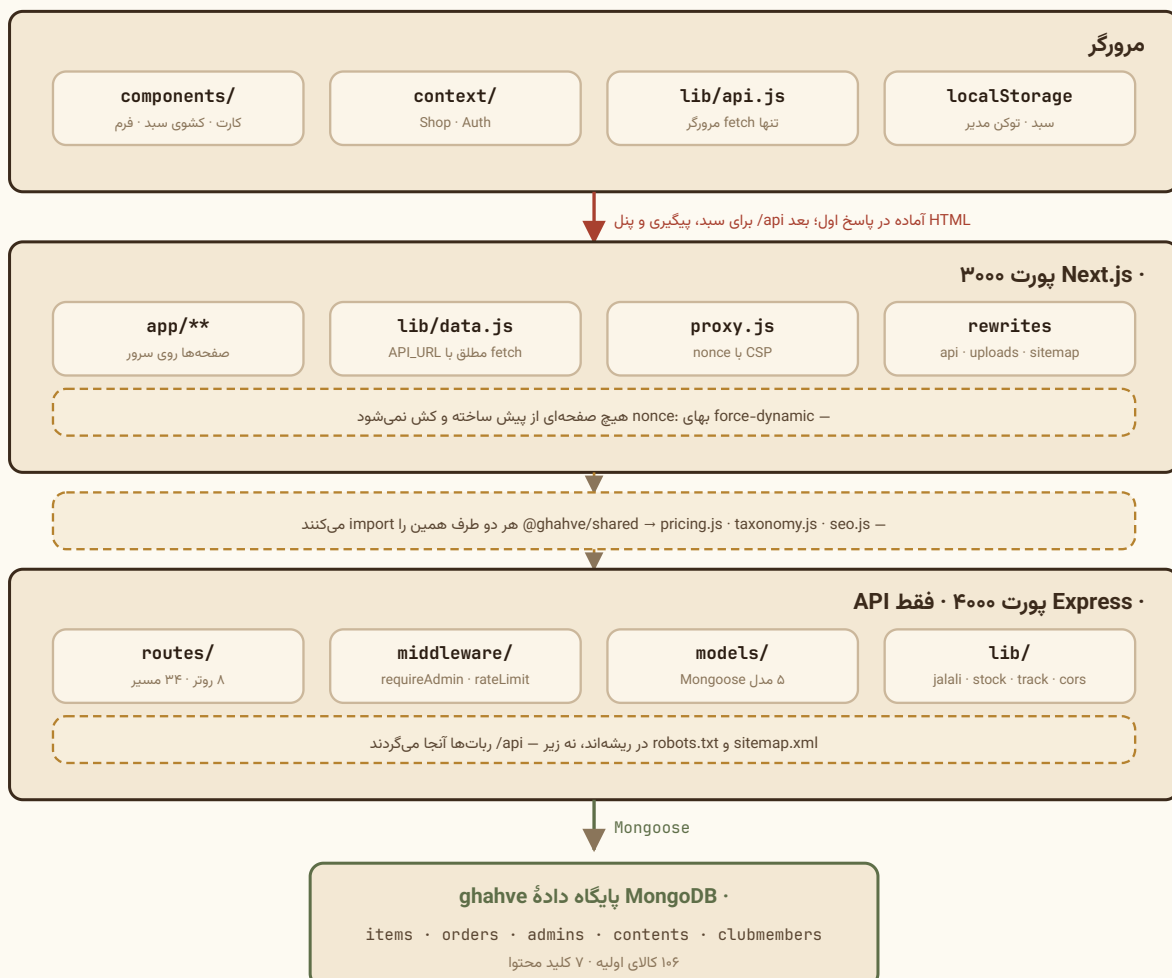
ItemDetail.jsx	«مودال» دربارهٔ این قهوه
ItemBody.jsx	متن کالا - سروری، مشترک صفحه و مودال
CardArt.jsx	تصویر کارت - طرح تولیدی یا عکس بهینه‌شده
ShareButton.jsx	دکمه هم‌رسانی - روی کارت و مودال
Toast.jsx	پیام کوتاه - یک بار، در پوستهٔ ریشه
CartDrawer.jsx	سبد + فرم + رسید
Receipt.jsx	شکل رسید - مشترک کشو و صفحهٔ پیگیری
PrintSheet.jsx	میزبان چاپ - رسید، فرزند مستقیم بدنه
BlendsSection.jsx	ساز میکس - میزبان هر دو حالت
MixVisual.jsx	حالت تصویری: کیسه، قیف، دستگاه
BeanPicker.jsx	فهرست افزودن دانه (مشترک)
ContentSections.jsx	بخش‌های قابل‌ویرایش از پنل
StaticSections.jsx	بخش‌های ثابت
Track.jsx	فرم و نتیجهٔ پیگیری سفارش
admin/	
AdminShell.jsx	نوار بالا + دروازهٔ نشست
AdminLogin.jsx	
AdminItems.jsx	فهرست کالاها
AdminItemForm.jsx	فرم با پیش‌نمایش زنده
AdminOrders.jsx	
OrderDetail.jsx	قطعهٔ مشترک سفارش‌ها و گزارش‌ها
AdminReports.jsx	گزارش شمسی
AdminContent.jsx	schema فرم ساخته‌شده از
contentSchema.js	توصیف فیلدهای محتوا
AdminClub.jsx	
AdminSettings.jsx	
server/	API - Express + Mongoose
package.json	
.env	محرمانه (در .gitignore)
.env.example	نمونهٔ قابل انتشار
uploads/	(git در .gitkeep فقط) تصویرهای آپلودی
src/	
index.js	روتورها، خطاها، CORS، helmet، compression
db.js	اتصال Mongoose
seed.js	پر کردن اولیهٔ پایگاه داده
middleware/	
auth.js	signToken + requireAdmin
ratelimit.js	سه محدودکننده: ورود، سفارش، پیگیری
lib/	
jalali.js	تقویم شمسی + بازبندی گزارش
stock.js	رزرو و پس‌دادن اتمی موجودی
track.js	پیگیری سفارش + نمای عمومی
cors.js	جدا و تست‌پذیر CORS، تصمیم
siteUrl.js	یا از خود درخواست SITE_URL آدرس پایهٔ عمومی - از
upload.js	multer /uploads هدرهای سخت‌گیر +
models/	
Item.js	stock + مدل واحد سه‌نوعه
Order.js	قیمت snapshot سفارش با
Admin.js	حساب مدیر
ClubMember.js	عضو باشگاه
Content.js	متن‌های سایت
routes/	
auth.js	ورود، بررسی نشست، تغییر رمز
items.js	کالا + آپلود + یک کالای عمومی CRUD
orders.js	ثبت سفارش + پیگیری + مدیریت
content.js	خواندن/نوشتن/بازنشانی محتوا

club.js	CSV + عضویت
stats.js	پرفروش‌ها و پیشنهاد ما
reports.js	گزارش شمسی + خروجی استریمی
seo.js	sitemap.xml و robots.txt در ریشه، نه زیر - /api
data/	
default-content.js	متن اولیه هفت بخش
seed-items.js	کالا ۱۰۶
seed-enrich.js	برچسب طعمی، معرفی، ترکیب میکس

نکته

این پروژه یک **npm workspace** است. یک **npm install** در ریشه هر سه بسته **server**، **shared** و **web** را نصب می‌کند و فقط یک **package-lock.json** در ریشه وجود دارد. در **server** یا **web** جداگانه **npm install** نکنید.

۳.۲ معماری لایه‌به‌لایه



چهار لایه و یک بسته مشترک – مرورگر، Express، Next و MongoDB

چهار لایه، سه بسته، و یکی از آن‌ها زیر بقیه. تا پیش از مورد ۲۶ سه لایه بود؛ حالا Next بین مرورگر و Express نشسته و صفحه‌ها را همان‌جا می‌سازد. `shared/` در هیچ‌کدام از این لایه‌ها نیست – زیر هر سه است.

و یک نکته تازه از مورد ۲۶: «مرورگر» دیگر تنها مصرف‌کننده API نیست. Next هم روی سرور از همان API می‌خواند – با آدرس مطلق، چون `fetch` روی سرور مبدأ ندارد. برای همین دو فایل جدا هست: `lib/api.js` برای مرورگر (مسیر نسبی) و `lib/data.js` برای سرور (`API_URL`). هر چیزی که دو طرف باید یکسان ببینند آنجاست و هر دو طرف با همان نام import می‌کنند:

```
// هر دو طرف
import { computeTotals } from '@ghahve/shared/pricing.js';
// هر دو طرف
import { KINDS } from '@ghahve/shared/taxonomy.js';
```

۳.۳ اصل‌های سازمان‌دهی کد

الف) پوشه بر پایه نقش، نه بر پایه ویژگی

در سمت وب، فایل‌ها بر اساس نقش فنی گروه‌بندی شده‌اند (`context/`، `lib/`، `components/`) نه بر اساس دامنه (`pages/`، `features/cart/`، `features/blend/`). پوشه `app/` گرفته و آنجا ساختار پوشه‌ها جدول مسیره‌است، نه یک گروه‌بندی سلیقه‌ای.

این انتخاب برای پروژه‌ای در این اندازه مناسب است، چون منطق دامنه‌ها درهم‌تنیده است: قیمت‌گذاری هم در کارت کالا، هم در سبد، هم در ساز میکس، هم در تصویر اصلی استفاده می‌شود. تقسیم بر پایه ویژگی، این ماژول‌های مشترک را به یک پوشه مشترک می‌راند و عملاً به همین‌جا برمی‌گشتیم.

در سمت سرور، تقسیم‌بندی کلاسیک MVC است: `models/` (داده و قواعد) · `routes/` (کنترلر) · `lib/` (منطق دامنه) · `middleware/` (میان‌افزار) · `data/` (متن پیش‌فرض و seed).

ب) `lib/` = منطق خالص، بدون React و بدون Express

هیچ‌کدام از فایل‌های `web/src/lib/*` (به‌جز `useReveal.js` و `useStorefrontRefresh.js` که هوک‌اند) به React وابسته نیستند. `blend.js`، `blendVisual.js`، `cartLine.js`، `format.js`، `art.js`، `groups.js` و `img.js` توابع و ثابت‌های خالص‌اند.

دو تای تازه‌تر همین قاعده را یک پله جلوتر می‌برند: `share.js` به `navigator` و `document` نیاز دارد ولی هر دو را پارامتر می‌گیرد، و `toastStore.js` حالت نگه می‌دارد ولی با `subscribe/getSnapshot` – یعنی یک انبار ساده که ری‌اکت از راه `useSyncExternalStore` به آن وصل می‌شود، نه چیزی که به ری‌اکت وابسته باشد. نتیجه‌اش این است که `tests/share.test.js` و `tests/toast-store.test.js` هر دو در محیط `node` اجرا می‌شوند، بدون `jsdom`.

در سمت سرور هم `lib/jalali.js`، `lib/stock.js`، `lib/track.js` و `lib/cors.js` هیچ ارجاعی به `req` و `res` ندارند و مستقل از Express اند. دو تای آخر عمداً از دل `index.js` و روتر بیرون کشیده شده‌اند تا بشود تستشان کرد: `resolveCorsOrigin` تصمیمی می‌گیرد که قبلاً کنارش `process.exit` بود، و `findOrderForTracking` مدل سفارش را به‌عنوان پارامتر می‌گیرد تا بدون مونگو اجرا شود. همین قاعده در `stock.js` هم هست: `reserveStock(lines, ItemModel)`.

این خلط‌کش ساده قاعده کار پروژه است: اگر منطقی نوشتید که تست کردنش به مونگو نیاز دارد، احتمالاً باید به یک تابع خالص جدا شود.

ج) shared/ = منبع یگانه حقیقت برای هر دو طرف

هر چیزی که مرورگر و سرور باید دقیقاً یکسان ببینند در بسته `@ghahve/shared` است، نه در دو کپی آینه‌ای. سه فایل آنجاست:

فایل	چه چیزی	چه کسی می‌خواند
<code>shared/pricing.js</code>	پله‌های تخفیف، ارسال، گرد کردن، قیمت میکس	وب برای نمایش، سرور برای ثبت
<code>shared/taxonomy.js</code>	کلیدهای مجاز: نوع، دسته، طرح، جنس، رنگ، طعم، آسیاب	سرور برای اعتبارسنجی، وب برای برچسب
<code>shared/seo.js</code>	<code>itemPath</code> ، عنوان و توضیح صفحه، JSON-LD، <code>buildRobots</code> ، <code>buildSitemap</code>	وب برای <code>generateMetadata</code> ، سرور برای <code>sitemap.xml</code>

`seo.js` تنها جای این بسته است که متن فارسی دارد، و این یک استثنای مستند است نه بی‌دقتی: عنوان و توضیح صفحه کالا هم در متادیتای Next لازم است و هم – تا وقتی نقشه سایت را Express می‌سازد – در سرور. دو نسخه شدنش یعنی چیزی که در تلگرام دیده می‌شود با تب مرورگر فرق کند. مرزش دقیق است: محتوای صفحه در `shared`، برچسب رابط کاربری در `web/src/lib/groups.js`.

مدل `Item` مستقیماً از همان بسته اعتبارسنجی می‌کند:

```
import { KINDS, GROUPS_BY_KIND, GEAR_SHAPES, GEAR_MATERIALS,
  POWDER_SHAPES, POWDER_TONES, IS_WEIGHED, TASTE_KEYS }
  from '@ghahve/shared/taxonomy.js';
```

این یک زنجیره ظریف است: `taxonomy.js` (shared) ← `art.js` (کلاینت). اگر شکلی در فهرست طرح‌ها نباشد، مدل اجازه نمی‌دهد ذخیره شود و کارت خالی رندر نمی‌شود.

د) تقسیم کلید و برچسب – عمدی، نه تکراری

`shared/taxonomy.js` فقط کلید دارد؛ برچسب فارسی هر کلید در `web/src/lib/groups.js` است. این دوتایی دوگانگی نیست، تقسیم کار است: سرور با متن فارسی کاری ندارد و بردن آن به `shared` فقط بار اضافه بود.

قاعده کار: کلید تازه اول در `shared/taxonomy.js` اضافه می‌شود، بعد برچسبش در `groups.js`. و برای اینکه کسی نیمه دوم را فراموش نکند، یک تست هر دو سمت را می‌سند:

```
// tests/taxonomy.test.js
it('دسته‌های قهوه', () => sameKeys(GROUPS, GROUPS_BY_KIND.coffee));
it('طرح‌های ابزار', () => sameKeys(GEAR_SHAPES, GEAR_SHAPE_KEYS));
```

`sameKeys` نه یکی کم را می‌بخشد و نه یکی زیاد را – یعنی کلید بی‌برچسب و برچسب بی‌کلید هر دو CI را قرمز می‌کنند.

نکته

پیش از این، `pricing.js` واقعاً دو نسخه داشت (یکی در `client`، یکی در `server`) که باید دستی همگام می‌ماندند. خطرش این بود که کسی پله‌های تخفیف را در یکی عوض کند و عددی که مشتری می‌بیند با عددی که ثبت می‌شود فرق کند – بی هیچ خطایی. حالا یک فایل بیشتر نیست، پس واگرایی از اساس ممکن نیست. این جای بازمحاسبه سمت سرور را نمی‌گیرد؛ فقط تضمین می‌کند نتیجه با آنچه کاربر دیده یکی دربیاید.

۳.۴ چرخه عمر یک درخواست

بیا یک درخواست کامل را از ابتدا تا انتها دنبال کنیم – «ثبت سفارش».

(۱) مرورگر. `CartDrawer.submit()` فراخوانی می‌شود.

```
const res = await api.placeOrder({
  lines: resolved.map(l => l.kind === 'gear'
    ? { slug: l.slug, qty: l.qty }
    : { slug: l.slug, grams: l.grams,
      ...(l.item.grindable ? { grind: l.grind || 'whole' } : {}),
      ...(l.item.isBlend && l.mix?.length ? { mix: l.mix } : {})) },
  customer: { name, phone, address, note }
});
```

توجه کنید که هیچ قیمتی فرستاده نمی‌شود.

(۲) لایه `api`. تابع `request()` در `web/src/lib/api.js`:

```
res = await fetch(url, { method, headers, body: JSON.stringify(body) });
```

آدرس `'/api/orders'` نسبی است.

(۳) `Next proxy`. این درخواست را به `localhost:4000` می‌فرستد (بند `rewrites` در `next.config.mjs`). در تولید، همان سروری که HTML را داد، درخواست را می‌گیرد.

(۴) میان‌افزارهای `Express`. به ترتیب `helmet` (هدرهای امنیتی و CSP)، `cors` (با میداهای مجاز از `CLIENT_ORIGIN`)، سپس `express.json({limit: '1mb'})` که بدنه را تجزیه می‌کند.

(۵) مسیریابی. `app.use('/api/orders', orderRoutes)` → روتر `POST /`. این مسیر `requireAdmin` ندارد چون مشتری باید بتواند سفارش بدهد؛ به جای آن `orderLimiter` جلوی نشسته: پیش‌فرض ۵ سفارش در ساعت از هر IP.

(۶) منطق تجاری. بازخوانی کالاها از پایگاه داده، اعتبارسنجی هر ردیف، و بازمحاسبه کامل قیمت با `computeTotals` (جزئیات در فصل ۶ و ۷).

(۷) رزرو موجودی. آخرین کار پیش از ثبت: `reserveStock(lines, item)` هر ردیف شمرده‌شده را با یک `findOneAndUpdate` اتمی کم می‌کند. اگر ردیفی نرسد، هرچه تا آنجا برداشته شده پس داده می‌شود و پاسخ ۴۰۹ برمی‌گردد – نه ۴۰۰، چون ورودی مشتری غلط نبود؛ انبار عوض شده بود.

(۸) لایه داده. `Order.create(...)` → قلاب `pre('validate')` شماره سفارش را می‌سازد → Mongoose اعتبارسنجی می‌کند → درج در MongoDB. اگر همین درج شکست بخورد، `releaseStock(taken, item)` رزروها را برمی‌گرداند و بعد خطا را بالا می‌فرستد.

(۹) پاسخ. از روی سند ذخیره‌شده ساخته می‌شود:

```
res.status(201).json({
  ok: true, code: order.code, createdAt: order.createdAt,
  lines: order.lines, customer: order.customer, totals: order.totals
});
```

۱۰ مسیر خطا. اگر هر جای این زنجیره خطایی بیفتد، `next(err)` آن را به مبدل خطای سراسری در `index.js` می‌فرستد که به پیام فارسی و کد وضعیت مناسب تبدیلش می‌کند.

۱۱ بازگشت به مرورگر. `setDone(res)` → `clearCart()` → کامپوننت `Receipt` رسید را از روی پاسخ سرور می‌سازد. شماره سفارش روی رسید همان چیزی است که بعداً در `/track` به‌کار می‌آید.

۳.۵ لایه ارتباط با سرور

فایل `web/src/lib/api.js` تنها جایی است که `fetch` صدا زده می‌شود. ساختارش سه بخش دارد:

بخش یک – مدیریت توکن:

```
const TOKEN_KEY = 'ghahve.admin.token';
export const getToken = () => localStorage.getItem(TOKEN_KEY) || '';
export const setToken = (t) => localStorage.setItem(TOKEN_KEY, t);
export const clearToken = () => localStorage.removeItem(TOKEN_KEY);

let onUnauthorized = () => {};
export const setUnauthorizedHandler = (fn) => { onUnauthorized = fn; };
```

آن `onUnauthorized` یک الگوی ظریف است: `api.js` هیچ وابستگی‌ای به React ندارد، اما باید بتواند به React خبر بدهد که «نشست منقضی شد». به‌جای import کردن context (که وابستگی حلقوی می‌ساخت)، یک فلاب می‌گذارد و `AuthContext` آن را پر می‌کند:

```
// web/src/context/AuthContext.jsx
useEffect(() => {
  setUnauthorizedHandler(() => setAdmin(null));
}, []);
```

بخش دو – تابع `request()` یکتا:

```
async function request(url, { method = 'GET', body, auth = false, raw = false } = {}) {
  const headers = {};
  if (!raw && body !== undefined) headers['Content-Type'] = 'application/json';
  if (auth) headers.Authorization = `Bearer ${getToken()}`;
  ...
  if (!res.ok) {
    if (res.status === 401 && auth) { clearToken(); onUnauthorized(); }
    throw new Error(data?.error || `خطای سرور (${res.status})`);
  }
  return data;
}
```

پرچم `raw` برای آپلود فایل است: وقتی بدنه `FormData` باشد، نباید `Content-Type` دستی تنظیم شود، چون مرورگر باید خودش `boundary` را اضافه کند.

بخش سه – شیء `api` با ۳۰ متد که به گروه‌های منطقی تقسیم شده‌اند: فروشگاه، باشگاه، ورود، مدیریت کالا، مدیریت سفارش، محتوا، گزارش.

علاوه بر این، یک تابع `download()` جداگانه وجود دارد برای فایل‌های محافظت‌شده. مشکل این است که لینک ساده (`<a>` `href>`) نمی‌تواند هدر `Authorization` بفرستد. راه‌حل: فایل را با `fetch` بگیر، به `Blob` تبدیل کن، یک `<a download>` موقت بساز، کلیک کن و پاکش کن:

```
const blob = await res.blob();
const href = URL.createObjectURL(blob);
const a = document.createElement('a');
a.href = href; a.download = name;
document.body.appendChild(a); a.click(); a.remove();
URL.revokeObjectURL(href);
```

و نام فایل از هدر `Content-Disposition` سرور خوانده می‌شود — با پشتیبانی از شکل `filename*=UTF-8'...'` که برای نام‌های فارسی لازم است.

۳.۶ نقشهٔ کامل API

متد	مسیر	دسترسی	خروجی
GET	/api/health	عمومی	{ok:true}
POST	/api/auth/login	عمومی (با محدودیت نرخ)	{token, admin}
GET	/api/auth/me	مدیر	{admin}
POST	/api/auth/change-password	مدیر	{ok, token, admin}
GET	/api/items?kind=	عمومی	آرایهٔ کالاهای active
GET	/api/items/:kind/:slug	عمومی	یک کالای روشن؛ ۴۰۴ اگر نباشد یا خاموش باشد
GET	/api/items/admin/all?kind=&q=	مدیر	همه، با جست‌وجو
GET	/api/items/admin/:id	مدیر	یک کالا
POST	/api/items	مدیر	کالای ساخته‌شده (۲۰۱)
PUT	/api/items/:id	مدیر	کالای به‌روزشده
PATCH	/api/items/:id/active	مدیر	کالا با active معکوس
DELETE	/api/items/:id	مدیر	{ok, id}
POST	/api/items/upload	مدیر	{url}
POST	/api/orders	عمومی (با محدودیت نرخ)	رسید کامل (۲۰۱)
GET	/api/orders/track?code=&phone=	عمومی (با محدودیت نرخ)	نمای عمومی سفارش
GET	/api/orders?status=	مدیر	{orders, counts}
GET	/api/orders/:id	مدیر	یک سفارش
PATCH	/api/orders/:id/status	مدیر	سفارش به‌روزشده
DELETE	/api/orders/:id	مدیر	{ok, id}

متد	مسیر	دسترسی	خروجی
GET	/api/content	عمومی	شیء هفت‌کلیده
PUT	/api/content/:key	مدیر	{ok, key, data}
POST	/api/content/:key/reset	مدیر	{ok, key, data}
POST	/api/club	عمومی	{ok, already, name}
GET	/api/club?q=	مدیر	{members, total}
DELETE	/api/club/:id	مدیر	{ok, id}
GET	/api/club/export.csv	مدیر	فایل CSV
GET	/api/stats/top-sellers?limit=	عمومی	آرایهٔ کالا با sales
GET	/api/stats/featured?limit=	عمومی	آرایهٔ کالا
GET	/api/reports/summary?period=&count=	مدیر	{buckets, windowTotals, allTime}
GET	/api/reports/orders?...	مدیر	{orders, total, page, pages, topItems}
GET	/api/reports/export.csv?mode=	مدیر	فایل CSV استریمی
GET	/api/reports/export.json	مدیر	فایل JSON استریمی
GET	/sitemap.xml	عمومی	XML از فهرست زندهٔ کالاهای روشن
GET	/robots.txt	عمومی	متن، با اشاره به sitemap

دو سطر آخر عمداً زیر **/api** نیستند: ربات‌ها فقط در ریشه دنبال‌شان می‌گردند. تنها آدرس‌های غیر-API این سرورند، و در تولید پراکسی باید همین دو را هم – کنار **/api** و **/uploads** – به Express بفرستد، نه به Next.

با `GET /api/items/:kind/:slug` ۲۶ مورد اضافه شد: صفحهٔ کالا روی سرور رندر می‌شود و بی‌معنی بود که برای نشان دادن یک کالا صد و شش کالا خوانده شود. الگوی **:kind** از خود **taxonomy** ساخته می‌شود، پس هیچ‌وقت با `/api/items/admin/...` اشتباه گرفته نمی‌شود.

قرارداد پاسخ

پاسخ موفق همیشه JSON است. پاسخ خطا همیشه شکل `{ "error": "پیام فارسی" }` دارد و همین باعث می‌شود کلاینت با یک خط بتواند همهٔ خطاها را نشان دهد:

```
throw new Error(data?.error || `خطای سرور`);
```

کدهای وضعیت به‌کاررفته: ۲۰۰ موفق · ۲۰۱ ساخته شد · ۴۰۰ ورودی نامعتبر · ۴۰۱ نیاز به ورود یا نشست منقضی · ۴۰۴ پیدا نشد · ۴۰۹ تعارض با وضعیت فعلی انبار – موجودی کافی نیست · ۴۲۹ عبور از محدودیت نرخ · ۵۰۰ خطای غیرمنتظره.

فرق ۴۰۰ و ۴۰۹ عمدی است: ۴۰۰ یعنی «آنچه فرستادی غلط بود»، ۴۰۹ یعنی «درست بود، ولی دنیا عوض شده». سبب مشتری در حالت دوم دست‌نخورده می‌ماند و فقط پیام موجودی نشان داده می‌شود.

پاسخ ۴۲۹ هم همان شکل `{ error }` را دارد، به‌علاوهٔ هدرهای استاندارد **RateLimit-*** (نسخهٔ ۷-draft) تا کلاینت بداند چقدر اعتبار مانده است.

۳.۷ مدیریت خطا در سرور

مبدل خطای سراسری در انتهای `server/src/index.js` سه دسته خطای Mongoose را به پیام فارسی تبدیل می‌کند:

```
app.use((err, _req, res, _next) => {
  if (err.name === 'ValidationError') {
    const first = Object.values(err.errors)[0];
    return res.status(400).json({ error: first?.message || 'اطلاعات وارد شده کامل نیست' });
  }

  if (err.code === 11000) {
    const field = Object.keys(err.keyPattern || {})[0];
    const label = { slug: 'شناسه', username: 'نام کاربری', code: 'شماره سفارش' }[field] || 'مقدار';
    return res.status(400).json({ error: `قبلاً استفاده شده است این ${label}` });
  }

  if (err.name === 'CastError') {
    return res.status(400).json({ error: 'شناسه درخواستی معتبر نیست' });
  }

  console.error(err);
  res.status(500).json({ error: 'خطای غیرمنتظره در سرور' });
});
```

سه نکته:

(۱) `ValidationError` پیام خود مدل را برمی‌گرداند – همان پیام فارسی که در schema نوشته شده. (۲) خطای `11000` (کلید تکراری) نام فیلد را به فارسی ترجمه می‌کند. (۳) `CastError` وقتی رخ می‌دهد که `ObjectId` نامعتبر در آدرس باشد؛ بدون این، هر آدرس اشتباهی یک ۵۰۰ می‌داد.

خطای پیش‌بینی‌نشده `console.error` می‌شود (برای لاگ سرور) اما به کاربر فقط پیام کلی می‌رسد – تا جزئیات داخلی نشت نکند.

۳.۸ استراتژی راه‌اندازی و تجربه توسعه‌دهنده

نکته‌ای که در بیشتر پروژه‌ها نادیده گرفته می‌شود اما اینجا به آن توجه شده، پیام‌های خطای راه‌اندازی است.

اگر MongoDB روشن نباشد، `server/src/db.js` این را چاپ می‌کند:

```
X Could not connect to MongoDB.
URI:   mongodb://127.0.0.1:27017/ghahve
Error: connect ECONNREFUSED 127.0.0.1:27017

Make sure the MongoDB service is running.
In PowerShell as administrator: net start MongoDB
```

و اگر پورت اشغال باشد، `index.js` دقیقاً دستور رفعش را می‌دهد:

```
X Port 4000 is already in use.
Another copy of the server is still running.
Close it, or change PORT in server/.env

To find and stop it on Windows (PowerShell):
Get-NetTCPConnection -LocalPort 4000 -State Listen | Select-Object OwningProcess
Stop-Process -Id <number-from-above> -Force
```

نکته

این پیام‌ها عمداً انگلیسی هستند. کامنت `db.js` دلیلش را می‌گوید: «پیام‌های این فایل عمداً انگلیسی‌اند: اینجا خروجی خط فرمان است، و پنجرهٔ cmd ویندوز حروف فارسی را «?» چاپ می‌کند. متن‌هایی که مشتری در سایت می‌بیند همچنان فارسی‌اند.» این یک تصمیم آگاهانه بر پایهٔ محیط اجرا است، نه بی‌دقتی.

اتصال قطع‌شده هم مدیریت می‌شود – بدون خروج از برنامه:

```
mongoose.connection.on('disconnected', () => console.warn('.. Database disconnected, retrying'));
mongoose.connection.on('reconnected', () => console.log('OK Database reconnected'));
```

با کامنت: «اگر وسط کار اتصال قطع شد، فقط لاگ می‌کنیم؛ mongoose خودش دوباره وصل می‌شود.»

۳.۹ لایه‌های امنیتی سرور

امنیت اینجا یک میان‌افزار نیست، چند لایهٔ مستقل است که هر کدام یک در را می‌بندد. ترتیبشان در `index.js` همان ترتیبی است که درخواست از آن‌ها رد می‌شود.

الف) `trust proxy` – پیش‌شرط درست کار کردن بقیه

محدودیت نرخ روی `req.ip` کلید می‌خورد. پشت nginx یا کلادفلر، `req.ip` آدرس خودِ پراکسی است – یعنی همهٔ بازدیدکنندگان یک کلید مشترک می‌گیرند و بعد از ۵ ورود ناموفق کل سایت برای همه قفل می‌شود.

```
if (process.env.NODE_ENV === 'production' || process.env.TRUST_PROXY === '1') {
  app.set('trust proxy', 1);
}
```

اما همیشه روشن هم نمی‌شود: جایی که پراکسی نیست، هر کلاینتی می‌تواند `X-Forwarded-For` جعلی بفرستد و با هر درخواست یک آدرس تازه بسازد – و محدودیت نرخ عملاً بی‌اثر شود. پس فقط در تولید، یا با پرچم صریح.

ب) `helmet` و `CSP`

این سیاست دیگر مال صفحه‌ها نیست. تا پیش از مورد ۲۶ همین سرور HTML فروشگاه را هم می‌داد، پس باید هر چیزی را که یک صفحهٔ واقعی لازم دارد پوشش می‌داد: فونت از گوگل، استایل درون‌خطی برای نوار نمودارها، و مانند این‌ها. حالا این سرور فقط JSON و XML و تصویر آپلودی می‌دهد، و آنچه مانده دو بند است:

```
helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ['self'],
      imgSrc: ['self', 'data:']
    }
  },
  strictTransportSecurity: process.env.NODE_ENV === 'production'
})
```

بند	چرا
defaultSrc: 'self'	پایهٔ محافظه‌کارانه؛ بقیهٔ بندها از پیش‌فرض helmet می‌آیند و برای یک API درست‌اند
imgSrc: 'data:'	تصویرهای data URI که ممکن است در پاسخ‌های همین سرور دیده شوند
scriptSrc دست‌نخورده ('self')	پاسخ‌های این سرور اسکریپتی ندارند
بندهای گوگل‌فونتس	برداشته شدند – فونت‌ها با next/font روی همان دامنه‌اند (مورد ۲۵)

سه سیاست جدا، و این عمدی است: صفحه‌ها سیاست خودشان را از `web/src/proxy.js` می‌گیرند (CSP با `nonce` بدون `'unsafe-inline'` برای اسکریپت)، پاسخ‌های API از همین helmet، و فایل‌های `/uploads` از `lib/upload.js`. اگر چیزی در مرورگر بسته شد، اول ببینید کدامیک از این سه گفته است.

`strictTransportSecurity` فقط در تولید روشن است: روی `http://localhost` دامنهٔ localhost را برای پروژه‌های دیگر هم به https قفل می‌کرد – دردسری که پاک کردنش از مرورگر سخت است.

ج CORS که در تولید سکوت نمی‌کند

قاعده در `lib/cors.js` است، جدا از `index.js`، چون تصمیمی که کنارش `process.exit` باشد تست‌پذیر نیست:

```
export function resolveCorsOrigin(env = process.env) {
  const origins = parseOrigins(env.CLIENT_ORIGIN);
  if (!origins.length && env.NODE_ENV === 'production') {
    throw new Error('CLIENT_ORIGIN must be set in production');
  }
  return origins.length ? origins : true;
}
```

پیش از این، نبود `CLIENT_ORIGIN` بی‌صدا به `origin: true` تبدیل می‌شد – یعنی «هر مبدایی مجاز است»، یک تنظیم جاافتاده و بی‌هیچ هشدار. حالا در تولید خطای مرگبار است و سرور با پیام روشن بالا نمی‌آید. در توسعه هنوز باز است تا کسی که تازه مخزن را کلون کرده بدون `.env` هم بتواند اجرا کند.

د) محدودیت نرخ روی سه مسیر بی‌محافظ

محدودکننده	مسیر	پیش‌فرض	خطر
loginLimiter	POST /api/auth/login	۵ در ۱۵ دقیقه	امتحان کردن هزاران رمز
orderLimiter	POST /api/orders	۵ در ۶۰ دقیقه	سیل سفارش جعلی
trackLimiter	GET /api/orders/track	۲۰ در ۱۵ دقیقه	حدس زدن شمارهٔ سفارش

هر سه عدد از `.env` خوانده می‌شوند تا بدون دست زدن به کد سفت یا شل شوند. سقف پیگیری سخاوتمندتر است چون مشتری واقعی ممکن است چند بار غلط تایپ کند، ولی آن‌قدر کم که جست‌وجوی فراگیر معنا نداشته باشد.

در محیط تست هر سه خاموش‌اند (`skip: isOff`)، وگرنه مجموعه تست بعد از پنج فراخوانی به ۴۲۹ می‌خورد. `RATE_LIMIT_DISABLED=1` همین کار را برای آزمایش دستی می‌کند.

هشدار امنیتی

شمارنده `express-rate-limit` درون‌حافظه‌ای است. با چند پروسه (یا چند نمونه سرور) هر کدام شمارنده خودش را دارد و سقف واقعی چند برابر می‌شود. برای استقرار چندپروسه‌ای باید انبار مشترک (مثلاً Redis) گذاشت.

ه) پوشه `/uploads` – دو لایه، نه یکی

لایه اول، فهرست سفید نوع فایل در `lib/upload.js`: فقط JPEG، PNG، WebP، GIF و SVG. AVIF عمداً نیست – یک سند XML است، نه تصویر خام، و می‌تواند `<script>` داشته باشد؛ و چون از همان دامنه سایت سرو می‌شود آن اسکریپت به توکن مدیر در `localStorage` هم دسترسی داشت.

لایه دوم، هدرهای خود پوشه – چون بستن فهرست فقط جلوی آپلود تازه را می‌گیرد و هرچه قبلاً آپلود شده هنوز روی دیسک است:

```
res.setHeader('X-Content-Type-Options', 'nosniff');
res.setHeader('Content-Security-Policy', "default-src 'none'; sandbox");
res.setHeader('Cross-Origin-Resource-Policy', 'same-origin');
res.setHeader('X-Frame-Options', 'DENY');
// و برای پسوندهای سندی (.html .xml .svg ...)
res.setHeader('Content-Disposition', 'attachment');
```

سند را در مبدأ یکتا و بدون اسکریپت می‌گذارد، پس حتی SVG قدیمی آلوده هم دیگر به مبدأ فروشگاه دسترسی ندارد. `Content-Disposition` روی `<img src>` اثری ندارد، پس تصویرهای عادی سالم می‌مانند. سقف حجم ۴ مگابایت است و نام فایل تصادفی (۱۲ بایت hex) تا نام اصلی فارسی یا تکراری در دسر نسازد.

۳.۱۰ تست، لینت و یکپارچه‌سازی

تست‌ها در ریشه می‌نشینند

`tests/` نه در web است و نه در server، چون از هر سه workspace می‌خواند: منطق مشترک از `shared`، رزرو موجودی و پیگیری از `server`، و جدول برچسب‌ها از `web`.

```
// vitest.config.mjs
export default defineConfig({
  test: {
    environment: 'node',
    include: ['tests/**/*.test.{js,jsx}'],
    env: { TZ: 'Asia/Tehran' }
  }
});
```

`TZ` ثابت شده چون گزارش‌ها روی وقت تهران بسته می‌شوند و تست هم باید مستقل از ساعت ماشینی باشد که اجرا می‌کند.

محیط پیش‌فرض `node` است و سه استثنا دارد: `item-card.test.jsx`، `cart-hydration.test.jsx` و `item-` `detail.test.jsx` با کامنت `// @vitest-environment jsdom` بالای خودشان محیط را عوض می‌کنند – همان‌جا، نه در این پیکربندی، تا بقیه فایل‌ها بی‌جهت داخل `jsdom` اجرا نشوند.

نکته

در دوران مهاجرت، این پیکربندی دو «پروژه» داشت: `client` روی ری‌اکت ۱۸ و `web` روی ۱۹، و تستی که کامپوننت رندر می‌کند باید رندرکننده و کامپوننت را از یک نسخه بگیرد. با برداشته شدن `client` فقط یک نسخه ری‌اکت ماند و آن تقسیم هم برداشته شد.

هیچ تستی به پایگاه داده دست نمی‌زند

قاعده ثابت پروژه: هر چیزی که تست می‌شود یا تابع خالص است یا وابستگی‌اش تزریق می‌شود. مدل `reserveStock` را پارامتر می‌گیرد، `findOrderForTracking` هم. تست‌ها یک مدل قلابی چند خطی می‌سازند که همان `findOneAndUpdate` را با یک شیء در حافظه شبیه‌سازی می‌کند – و شرط `$gte` را واقعاً بررسی می‌کند، وگرنه تست «یک گرم بیشتر از موجودی» بی‌معنی بود. نتیجه‌اش این است که CI به سرویس MongoDB نیاز ندارد.

بیست‌ودو فایل تست

فایل	چه چیزی را نگه می‌دارد
<code>pricing.test.js</code>	مرزهای دقیق پله‌ها (۹۹۹ در برابر ۱۰۰۰ گرم)، ارسال، قیمت میکس
<code>blend.test.js</code>	مجموع درصدها بعد از هزار حرکت تصادفی هم ۱۰۰ می‌ماند
<code>blend-visual.test.js</code>	بودجه زمانی، سطح کیفی، هندسه کیسه‌ها، رنگ میکس
<code>card-art.test.js</code>	نام خصمانه کالا نمی‌تواند صفتی به تگ <code><svg></code> اضافه کند
<code>stock.test.js</code>	مرز دقیق موجودی، پس دادن رزروها در شکست
<code>track.test.js</code>	یکسان بودن پاسخ «کد نیست» و «شماره غلط»، فیلدهای نمای عمومی
<code>upload.test.js</code>	نمود SVG در فهرست مجاز، هدرهای <code>/uploads</code>
<code>cors.test.js</code>	خطا در تولید بدون <code>CLIENT_ORIGIN</code>
<code>rate-limit.test.js</code>	خاموش بودن محدودیت در محیط تست
<code>jalali.test.js</code>	نوروزهای شناخته‌شده، شنبه بودن شروع هفته
<code>taxonomy.test.js</code>	هر کلید <code>shared</code> یک برچسب فارسی دارد – نه کم، نه زیاد
<code>item-routes.test.js</code>	شکل آدرس کالا و نگاشت دوطرفه نوع → بخش آدرس
<code>json-ld.test.js</code>	شکل <code>Product</code> و <code>LocalBusiness</code> ، و بی‌خطر بودن متن مدیر داخل <code><script></code>
<code>sitemap.test.js</code>	XML نقشه سایت، <code>escape</code> ، و بیرون ماندن کالای خاموش
<code>next-metadata.test.js</code>	هر تگی که <code>headTags</code> تعریف می‌کند به شیء <code>metadata</code> نکست می‌رسد
<code>next-routes.test.js</code>	هر نوع کالا پوشه مسیر خودش را دارد – در هر دو جهت
<code>cart-storage.test.js</code>	سبد ذخیره‌شده بازاعتبارسنجی می‌شود، و حافظه خطادار فروشگاه را زمین نمی‌زند
<code>share.test.js</code>	نشانی فرستاده‌شده همان <code>canonical</code> صفحه است، و دسکتاپ به کپی می‌رود نه به برگه سیستم

فایل	چه چیزی را نگه می‌دارد
toast-store.test.js	پیام سر وقت پاک می‌شود، پیام دوم شمارش تازه می‌گیرد، و snapshot سرور همیشه خالی است
cart-hydration.test.jsx	سبد از چرخه «رندر سرور ← hydrate» دست‌نخورده بیرون می‌آید
item-card.test.jsx	کارت هر نوع کالا <a href> واقعی به صفحه خودش دارد
item-detail.test.jsx	مودال کالا بدون ShopProvider رندر می‌شود و دکمه هم‌رسانی‌اش کار می‌کند

CI

روی هر push و هر pull request چهار گام را همان‌طور اجرا می‌کند که یک نفر روی ماشین خودش می‌زند:

```
npm ci → npm run lint → npm run format:check → npm test → npm run build
```

یک `npm ci` برای هر سه workspace کافی است و کش npm روی همان تک `package-lock.json` ریشه بسته می‌شود. اینجا استقراری در کار نیست — CI فقط می‌گوید کد سالم است.

پی‌کربندی ESLint (`eslint.config.mjs`) از نوع flat config است و برای هر سه بخش قاعده خودش را دارد؛ `eslint-config-` هم آخر صف می‌نشیند تا قاعده‌های سلیقه‌ای قالب‌بندی با Prettier دعوا نکنند. متن‌های فارسی `*.md` در `.prettierrignore` کنار گذاشته شده‌اند، چون تنها کار Prettier با آن‌ها هم‌تراز کردن ستون‌های جدول است و با پهنای حروف فارسی نتیجه به‌هم‌ریخته‌تر می‌شود.

۴. مدل داده و ساختار بانک اطلاعاتی

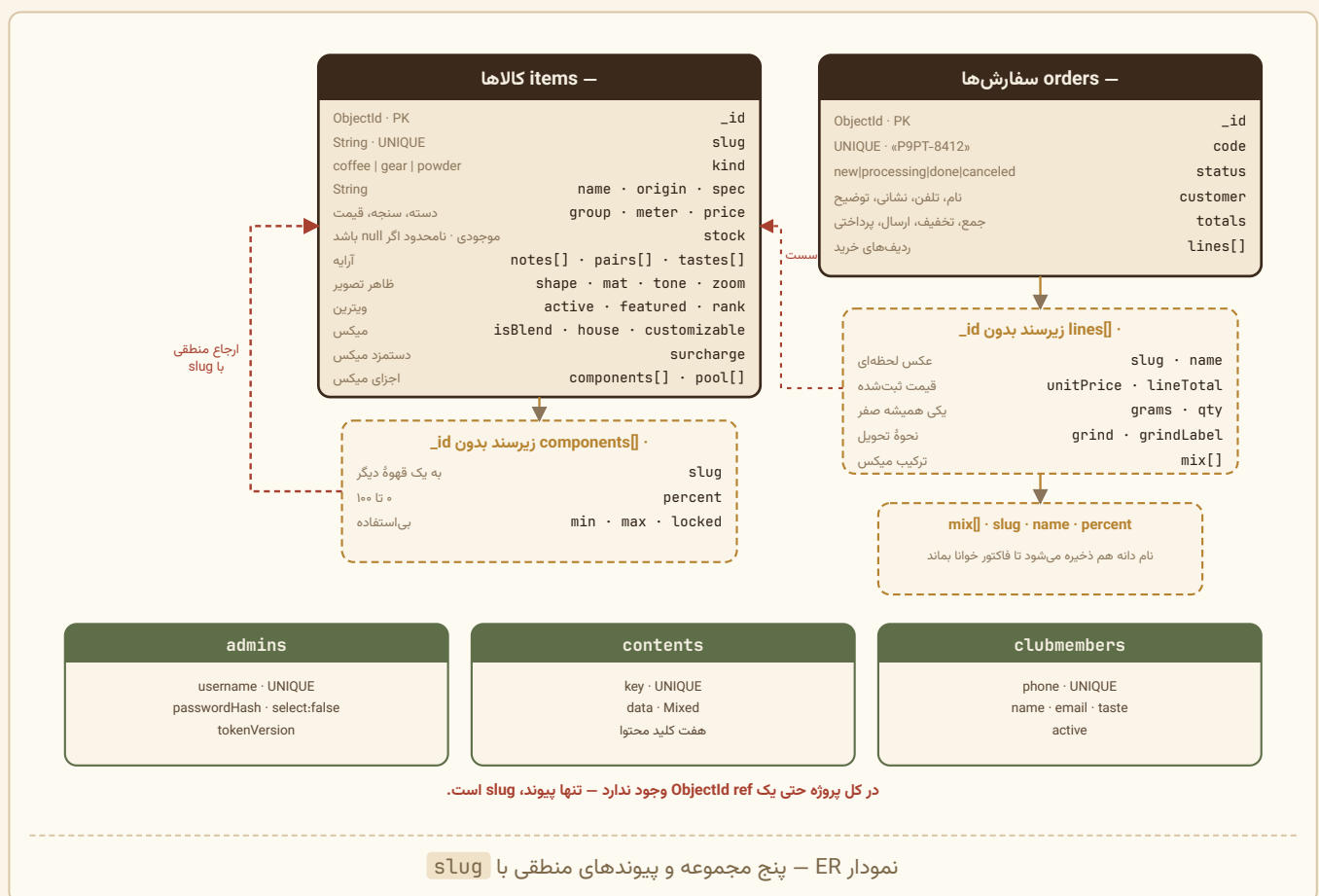
۴.۱ نمای کلی

پایگاه داده ghahve پنج مجموعه (collection) دارد:

مجموعه	مدل	تعداد اولیه	نقش
items	Item	۱۰۶	همه کالاهای — قهوه، ابزار، پودر
orders	Order	۰	سفارش‌های ثبت‌شده
admins	Admin	۱	حساب مدیر
contents	Content	۷	متن‌های قابل ویرایش سایت
clubmembers	ClubMember	۰	اعضای باشگاه مشتریان

نکته ساختاری مهم: در کل پروژه حتی یک **ObjectId ref** وجود ندارد. هیچ **populate** ای صدا زده نمی‌شود. تنها پیوند بین مجموعه‌ها، فیلد **slug** است که به عنوان کلید منطقی به کار می‌رود.

۴.۲ نمودار ER



۴.۳ مدل Item – قلب پروژه

فایل: `server/src/models/Item.js` (۲۳۶ خط)

الف) زیرسند `componentSchema`

```
const componentSchema = new mongoose.Schema(
  {
    slug: { type: String, required: true },
    percent: { type: Number, required: true, min: 0, max: 100 },
    min: { type: Number, default: 0, min: 0, max: 100 },
    max: { type: Number, default: 100, min: 0, max: 100 },
    locked: { type: Boolean, default: false }
  },
  { _id: false }
);
```

`{ _id: false }` یعنی Mongoose برای هر جزء یک `ObjectId` جداگانه نسازد. این زیرسندها هویت مستقل ندارند و همیشه با سند والد خوانده و نوشته می‌شوند، پس `_id` فقط حجم اضافه بود.

نکته

`min`، `max` و `locked` در مدل تعریف شده‌اند و در داده seed هم مقدار واقعی دارند (`{slug:'cerrado', percent:70,` `min: 0, max: 100, locked: false` همیشه `AdminItemForm` اما `{min:40, max:90}`)، می‌فرستد و `BlendsSection` هم اصلاً به آن‌ها نگاه نمی‌کند – اهرم‌ها همیشه ۰ تا ۱۰۰ حرکت می‌کنند. کامنت خط ۲۶۹ `AdminItemForm.jsx` این را عمدی نشان می‌دهد: «ترکیب ما فقط پیشنهاد است – بازه مجاز و قفل نداریم، چون مشتری در انتخابش کاملاً آزاد است.» یعنی سیاست تجاری عوض شده اما فیلدها در مدل باقی مانده‌اند.

ب) جدول کامل فیلدها

فیلد	نوع	پیش‌فرض	قیدها	برای کدام kind
kind	String	–	required, enum, index	همه
slug	String	–	required, unique, lowercase, trim, <code>/^[a-z0-9][a-z0-9-]*\$/</code>	همه
name	String	–	required, trim	همه
origin	String	''	trim	همه
spec	String	''	trim	همه
group	String	–	required, index, عضو <code>GROUPS_BY_KIND[kind]</code>	همه
meter	Number	3	۵..۱	همه
price	Number	–	required, <code>min: 0</code>	همه
stock	Number null	null	<code>min: 0</code> , گرد به عدد صحیح	همه
notes	[String]	[]	–	همه

فیلد	نوع	پیش فرض	قیدها	برای کدام kind
pairs	[String]	[]	برای قهوه پاک می شود	ابزار، پودر
tag	String	''	trim	همه
shape	String	''	باید در whitelist باشد	ابزار، پودر
mat	String	''	GEAR_MATERIALS	ابزار
tone	String	''	POWDER_TONES	پودر
zoom	Number	1	۲۰۰/۴	ابزار، پودر
image	String	''	مسیر /uploads/...	همه
rank	Number	999	—	همه
active	Boolean	true	—	همه
story	String	''	trim	همه (عملاً قهوه)
taste	String	''	trim	همه
recommend	String	''	trim	همه
tastes	[String]	[]	زیرمجموعه TASTE_KEYS	قهوه
isBlend	Boolean	false	—	قهوه
house	Boolean	false	—	قهوه
customizable	Boolean	false	—	قهوه
components	[componentSchema]	[]	۲ ≤ و ۱۰۰ = Σ اگر isBlend	قهوه
pool	[String]	[]	—	قهوه
surcharge	Number	0	min: 0	قهوه
featured	Boolean	false	—	همه
pinnedTop	Boolean	false	—	همه
excludeTop	Boolean	false	—	همه
grindable	Boolean	false	خودکار تنظیم می شود	قهوه = true

به علاوه createdAt و updatedAt که با { timestamps: true } خودکارند.

ب-۲) فیلد stock - و چرا null با صفر یکی نیست

این فیلد با مورد ۳۴ اضافه شد و تنها فیلدی است که سه حالت معنادار دارد، نه دو تا:

/* — موجودی انبار —
 واحدش همان واحد فروش است: گرم برای قهوه و پودر،
 عدد برای ابزار.

null یعنی «نامحدود»، و پیش‌فرض هم همین است — چون
 تا پیش از این هیچ موجودی‌ای در کار نبود و همه
 کالاهای موجود باید بدون مهاجرت مثل قبل کار کنند.
 صفر یعنی «تمام شد»، که با null یکی نیست. */
 stock: { type: Number, default: null, min: [0, 'می‌تواند منفی باشد'] },

مقدار	معنا	در ثبت سفارش
null	نامحدود — شمرده نمی‌شود	اصلاً لمس نمی‌شود
عددی > 0	همان‌قدر مانده	با \$inc اتمی کم می‌شود
0	تمام شد	هر سفارشی ۴۰۹ می‌گیرد

چرا null پیش‌فرض است و نه صفر؟ چون این فیلد به یک پایگاه دادهٔ پُر اضافه شد. اگر پیش‌فرضش صفر بود، لحظه‌ای که کد بالا می‌آمد هر ۱۰۶ کالا «ناموجود» می‌شدند. null یعنی «این کالا هنوز شمرده نشده» و رفتار پیش از مورد ۳۴ را دقیقاً حفظ می‌کند — بدون هیچ مهاجرتی.

سه جای دیگر هم به همین سه‌حالتی بودن حساس‌اند:

- `server/src/lib/stock.js` → `isTracked(item)` که فقط عدد متناهی را «شمرده‌شده» می‌داند؛
- قلاب `pre('validate')` که موجودی را مثل قیمت گرد و منفی‌نشده می‌کند (`Math.max(0, ...)`)
- `Math.round(this.stock)`، چون مقایسهٔ اتمی روی عدد صحیح انجام می‌شود و اعشار فقط در دسر است؛
- فرم کالا در پنل، که خالی گذاشتن کادر را به null ترجمه می‌کند نه به صفر.

هشدار

موجودی یک میکس از خود سند میکس کم می‌شود، نه از دانه‌های اجزایش. پس اگر برای میکسی عدد بگذارید، فروشش موجودی دانه‌ها را کم نمی‌کند. میکس‌ها را معمولاً null بگذارید. لغو سفارش هم موجودی را برنمی‌گرداند — هر دو در مورد ۳۴ فصل ۱۱ به‌عنوان باقی‌ماندهٔ باز ثبت شده‌اند.

ج) فیلد مجازی weighed

```
itemSchema.virtual('weighed').get(function () {
  return IS_WEIGHED[this.kind] === true;
});
```

این فیلد در پایگاه داده ذخیره نمی‌شود اما در پاسخ API می‌آید، چون:

```
toJSON: { virtuals: true, transform(_doc, ret) { delete ret.__v; return ret; } }
```

و در کوئری‌ها هم صریحاً درخواست می‌شود:

```
const items = await Item.find(filter).sort({ rank: 1, name: 1 }).lean({ virtuals: true });
```

نکته ظریف: `.lean()` معمولاً سند ساده برمی‌گرداند و `virtual` ها را ندارد؛ `.lean({ virtuals: true })` در Mongoose 8 آن‌ها را هم می‌آورد. بدون این، کلاینت نمی‌فهمید کالا وزنی است یا عددی.

د) ایندکس‌ها

```
itemSchema.index({ kind: 1, rank: 1 });
itemSchema.index({ name: 'text', origin: 'text', spec: 'text' });
```

به‌علاوه ایندکس‌های تک‌فیلدی که در تعریف فیلدها آمده‌اند (`kind`، `group`) و ایندکس یکتای `slug`.

نکته صادقانه

ایندکس متنی (`text`) ساخته شده اما هیچ‌جا استفاده نمی‌شود. جست‌وجوی پنل مدیریت به‌جای `$text` از `RegExp` استفاده می‌کند:

```
const rx = new RegExp(q.replace(/[*+?^$(){[\]\\\]/g, '\\$&'), 'i');
filter.$or = [{ name: rx }, { slug: rx }, { origin: rx }, { spec: rx }, { tag: rx }];
```

دلیل احتمالی: `$text` روی فارسی stemming درستی ندارد و جست‌وجوی جزئی (مثل «کنی» برای «کنیا») را پشتیبانی نمی‌کند، در حالی که `regex` می‌کند. اما نتیجه این است که یک ایندکس بدون استفاده روی دیسک نگه داشته می‌شود.

۴.۴ قلاب `pre('validate')` مدل `Item`

این قلاب مهم‌ترین بخش منطق مدل است و هفت گروه قاعده را اعمال می‌کند. در فصل ۷ خطبه‌خط بررسی می‌شود؛ اینجا خلاصه رفتاری:

شماره	قاعده	نتیجه
۱	<code>group</code> باید عضو <code>GROUPS_BY_KIND[kind]</code> باشد	خطای «دسته X» برای این نوع کالا تعریف نشده است
۲	<code>gear</code> : پیش‌فرض <code>shape='mug'</code> ، <code>mat='steel'</code> ؛ <code>tone</code> پاک	جلوگیری از کارت خالی
۳	<code>powder</code> : پیش‌فرض <code>shape='scoop'</code> ، <code>tone='cocoa'</code> ؛ <code>mat</code> پاک	همان
۴	<code>coffee</code> : <code>shape/mat/tone/pairs</code> پاک، <code>grindable = true</code>	«هر قهوه‌ای آسیاب‌شدنی است»
۵	<code>grindable=false</code> و همه فیلدهای میکس پاک	«فقط قهوه میکس می‌شود»
۶	اگر <code>isBlend</code> : $2 \leq$ جزء، $1 \leq \Sigma - 100 $ ، بدون تکرار، <code>min ≤ max</code>	خطاهای فارسی جداگانه
۷	اگر <code>!isBlend</code> : <code>components/pool</code> پاک، <code>customizable/house=false</code>	تمیز ماندن سند
۸	گرد کردن <code>price</code> ، <code>meter</code> و <code>stock</code>	بدون اعشار سرگردان؛ <code>null</code> دست‌نخورده

و در انتها:


```

if (typeof this.price === 'number') this.price = Math.round(this.price);
if (typeof this.meter === 'number') this.meter = Math.round(this.meter);

/* موجودی هم همین‌طور - نیم گرم و نیم عدد معنا ندارد.
مقایسه اتمی هنگام ثبت سفارش روی همین عدد صحیح
انجام می‌شود، پس اعشار فقط در دسر است. */
if (typeof this.stock === 'number') this.stock = Math.max(0, Math.round(this.stock));

```

با کامنت: «قیمت را گرد می‌کنیم تا اعشار سرگردان در پایگاه داده نماند.» شرط `typeof === 'number'` در سطر آخر همان چیزی است که `null` را دست‌نخورده می‌گذارد: «نامحدود» گرد نمی‌شود.

اهمیت این قلاب: این قواعد در لایه مدل هستند، نه در روت. یعنی چه از `POST /api/items` بیاید، چه از `PUT`، چه از `seed.js`، همه اعمال می‌شوند. به همین دلیل است که مسیر ویرایش عمداً `findByIdAndUpdate` را استفاده نمی‌کند (که قلاب‌های سند را دور می‌زند) بلکه `findById` + `Object.assign` + `save()` می‌نویسد.

۴.۵ مدل Order – سفارش به‌عنوان عکس لحظه‌ای

فایل: `server/src/models/Order.js` (۱۰۹ خط)

کامنت بالای فایل، فلسفه طراحی را در چهار خط می‌گوید:

```

/* سفارش‌ها. قیمت‌ها هنگام ثبت از روی پایگاه داده دوباره
حساب می‌شوند (به عدد ارسالی از مرورگر اعتماد نمی‌کنیم)
ولی همان لحظه در سفارش ذخیره می‌شوند تا تغییر قیمت
بعدی، فاکتور قدیمی را عوض نکند. */

```

این دو جمله به‌ظاهر متناقض، در واقع دو نگرانی جدا را حل می‌کنند:

- «اعتماد نمی‌کنیم» = مسئله امنیت. مشتری نمی‌تواند قیمت را دستکاری کند.
- «ذخیره می‌شوند» = مسئله درستی حسابداری. اگر فردا قیمت یرگچف بالا برود، فاکتور دیروز نباید عوض شود.

الف) ساختار سه‌لایه تودرتو

```

Order
├── lines[]      (lineSchema، بدون _id)
│   └── mix[]    (mixSchema، بدون _id)

```

```

const mixSchema = new mongoose.Schema(
  { slug: String, name: String, percent: Number },
  { _id: false }
);

```

توجه کنید که `name` هم ذخیره می‌شود. کامنت می‌گوید چرا:

ترکیب میکس، همان‌طور که مشتری سفارش داده – با نام دانه‌ها، تا فاکتور بعداً هم خوانا بماند.

اگر فقط `slug` ذخیره می‌شد و مدیر بعداً نام «یرگچف» را به «یرگچف اتیوپی» عوض می‌کرد یا کالا را حذف می‌کرد، فاکتور قدیمی `yirgacheffe` خام نشان می‌داد.

```
const lineSchema = new mongoose.Schema(
  {
    kind:      { type: String, required: true },
    slug:      { type: String, required: true },
    name:      { type: String, required: true },
    unitPrice: { type: Number, required: true }, // هر کیلو یا هر عدد
    grams:     { type: Number, default: 0 },    // قهوه و پودر
    qty:       { type: Number, default: 0 },    // ابزار
    lineTotal: { type: Number, required: true },

    grind:     { type: String, default: '' },
    grindLabel: { type: String, default: '' },

    mix: { type: [mixSchema], default: [] }
  },
  { _id: false }
);
```

نکته طراحی: `grams` و `qty` هر دو روی هر ردیف هستند، اما همیشه یکی صفر است. برای کالای وزنی `qty = 0` و برای ابزار `grams = 0`. این باعث می‌شود جمع‌زدن در `aggregate` ساده شود:

```
// server/src/routes/stats.js
grams: { $sum: '$lines.grams' },
qty:   { $sum: '$lines.qty' },
```

بدون نیاز به `$cond`.

مشابهاً `grind` و `grindLabel` هر دو ذخیره می‌شوند: `grind` کلید ماشینی ('espresso') و `grindLabel` متن فارسی همان لحظه ('آسیاب اسپرسو'). اگر مدیر بعداً برچسب را عوض کند، سفارش قدیمی همان چیزی را نشان می‌دهد که مشتری دیده بود.

ج) تولید شماره سفارش

```
orderSchema.pre('validate', function (next) {
  if (!this.code) {
    const stamp = Date.now().toString(36).slice(-4).toUpperCase();
    const rand = Math.floor(Math.random() * 9000 + 1000);
    this.code = `${stamp}-${rand}`;
  }
  next();
});
```

خطبه خط:

- `Date.now()` → عدد میلی‌ثانیه، مثلاً 1754906421337
- `.toString(36)` → مبنای ۳۶ (رقم + حرف)، مثلاً '1jkm3p9pt'
- `.slice(-4)` → چهار نویسه آخر، '9pt' ... در واقع 'p9pt'
- `.toUpperCase()` → 'P9PT'
- `rand` عددی بین ۱۰۰۰ و ۹۹۹۹

نتیجه: P9PT-8412. کوتاه، خواناتر از ObjectId بیست و چهار نویسه‌ای، و قابل خواندن روی تلفن.

هشدار

این تولیدکننده قطعاً یکتا نیست. فیلد code قید unique دارد، پس برخورد باعث خطای 11000 می‌شود که به پیام «این شماره سفارش قبلاً استفاده شده است» تبدیل می‌شود – و هیچ منطق تلاش مجددی وجود ندارد. احتمالش کم است (نیاز به دو سفارش در یک بازهٔ 4^{36} میلی‌ثانیه‌ای با همان عدد تصادفی) اما صفر نیست. در فصل ۱۱ راه‌حلش را می‌آوریم.

د totals – چرا ذخیره می‌شود؟

```
totals: {
  grams:      { type: Number, default: 0 },
  pieces:     { type: Number, default: 0 },
  base:       { type: Number, default: 0 },
  discount:   { type: Number, default: 0 },
  discountLabel: { type: String, default: '' },
  shipping:   { type: Number, default: 0 },
  total:      { type: Number, default: 0 }
}
```

این‌ها همه از lines قابل محاسبه‌اند، پس داده‌های اضافه (denormalized) هستند. دلیلش کارایی گزارش‌ها است. در `server/src/routes/reports.js`:

```
const rows = await Order.find(
  { createdAt: { $gte: windowStart } },
  { createdAt: 1, status: 1, totals: 1 } // projection سبک
).lean();
```

اگر totals ذخیره نمی‌شد، برای هر گزارش باید همهٔ lines خوانده و دوباره جمع زده می‌شدند. با این طراحی، گزارش ماهانه فقط سه فیلد کوچک از هر سفارش می‌خواند.

همچنین discountLabel (رشتهٔ '۱۵٪') ذخیره می‌شود تا اگر فردا پله‌های تخفیف عوض شوند، فاکتور قدیمی همان درصدی را نشان دهد که واقعاً اعمال شده بود.

```
const adminSchema = new mongoose.Schema(
  {
    username: {
      type: String, required: [true, 'نام کاربری الزامی است'],
      unique: true, trim: true, lowercase: true,
      minlength: [3, 'نام کاربری دستکم ۳ نویسه باشد']
    },
    passwordHash: { type: String, required: true, select: false },
    tokenVersion: { type: Number, default: 0 }
  },
  {
    timestamps: true,
    toJSON: { transform(_d, ret) { delete ret.passwordHash; delete ret.__v; return ret; } }
  }
);
```

سه لایه محافظت از رمز:

(۱) `select: false` – در کوئری معمولی اصلاً خوانده نمی‌شود. برای خواندنش باید صریحاً بخواهید:

```
const admin = await Admin.findOne({ username }).select('+passwordHash');
```

(۲) `toJSON.transform` – حتی اگر خوانده شد، در سریال‌سازی حذف می‌شود.

(۳) هیچ‌وقت رمز خام ذخیره نمی‌شود:

```
adminSchema.methods.setPassword = async function (plain) {
  this.passwordHash = await bcrypt.hash(plain, 12);
};
adminSchema.methods.checkPassword = function (plain) {
  return bcrypt.compare(plain, this.passwordHash);
};
```

روی `username` یعنی `Admin` و `admin` یکی‌اند – و در مسیر ورود هم `lowercase: true` اعمال می‌شود، پس نداشت یک‌به‌یک است. `String(req.body.username).trim().toLowerCase()`

```
phone: {
  type: String,
  required: [true, 'شمارهٔ موبایل را وارد کنید'],
  unique: true,
  trim: true,
  match: [/^0\d{10}$/, 'شمارهٔ موبایل باید ۱۱ رقم و با ۰ شروع شود'],
},

email: {
  type: String, default: '', trim: true, lowercase: true,
  validate: {
    validator: (v) => v === '' || /^[^@\\s]+@[^@\\s]+\.[^@\\s]+$/.test(v),
    message: 'ایمیل معتبر نیست'
  }
}
```

دو نکته:

(۱) **phone** کلید یکتای طبیعی است. چون رمز عبوری در کار نیست، شماره همان هویت است. مسیر عضویت به جای خطا دادن، به روزرسانی می‌کند:

```
const existing = await ClubMember.findOne({ phone });
if (existing) {
  existing.name = name;
  if (email) existing.email = email;
  if (taste) existing.taste = taste;
  existing.active = true;
  await existing.save();
  return res.json({ ok: true, already: true, name: existing.name });
}
```

کامنت: «اگر قبلاً عضو شده، اطلاعاتش را تازه می‌کنیم و همان پیام خوشامد را می‌دهیم — نه پیام خطا.» و کلاینت از **already** استفاده می‌کند تا پیام مناسب نشان دهد.

(۲) **اعتبارسنجی ایمیل با `v === ''` شروع می‌شود** — چون ایمیل اختیاری است و رشتهٔ خالی باید بگذرد، اما اگر چیزی نوشته شد باید معتبر باشد.

ایندکس `clubSchema.index({ createdAt: -1 })` برای مرتب‌سازی «تازه‌ترین اول» در پنل است.

```
const contentSchema = new mongoose.Schema(
  {
    key: { type: String, required: true, unique: true, trim: true },
    data: { type: mongoose.Schema.Types.Mixed, default: {} }
  },
  { timestamps: true, minimize: false,
    toJSON: { transform(_d, ret) { delete ret.__v; return ret; } } }
);
```

Mixed یعنی **Mongoose** هیچ اعتبارسنجی روی **data** انجام نمی‌دهد. کامنت این را عمدی نشان می‌دهد:

ساختارش به کلید بستگی دارد؛ اعتبارسنجی شکل داده در لایهٔ مسیرها انجام می‌شود، نه اینجا.

minimize: false نکتهٔ ظریفی است. به‌طور پیش‌فرض **Mongoose** شیء‌های خالی (**{}**) را از سند حذف می‌کند. اگر مدیر همهٔ فیلدهای یک بخش را خالی کند، بدون این تنظیم آن کلید از سند ناپدید می‌شد و بخش به پیش‌فرض برمی‌گشت – که رفتار گیج‌کننده‌ای بود.

هفت کلید و شکل داده‌شان

مصرف‌کننده	کلید	شکل data
WhyUsSection n	whyUs	{eyebrow, title, lead, reasons:[{icon,title,text}], ctaText, ctaHref}
SuggestSection on	suggest	{eyebrow, title, lead, profiles:[{key,label,hint,advice,picks:[slug]}]}
PicksSection s	picks	{eyebrow, title, lead, featuredTitle, featuredNote, topTitle, topNote, emptyTop}
BlendsSection n	blends	{eyebrow, title, lead, customNote}
ClubSection	club	{eyebrow, title, lead, benefits:[String], successTitle, successText, askTaste, askEmail}
AboutSection n + رسید	about	{eyebrow, title, lead, image, imageCaption, facts:[{value,label}], sections:[{title,text}], addressTitle, address, hours, phone, email}
کارت قهوه، سبد، سرور	grinds	{options:[{value,label,desc}]}

تابع کمکی loadContent

```
export async function loadContent() {
  const rows = await Content.find().lean();
  return Object.fromEntries(rows.map((r) => [r.key, r.data]));
}
```

آرایهٔ اسناد را به یک شیء تخت تبدیل می‌کند: **{whyUs: {...}, about: {...}}**.

در `server/src/routes/content.js`:

```
const KEYS = Object.keys(DEFAULT_CONTENT);

router.get('/', async (_req, res, next) => {
  const stored = await loadContent();
  const out = {};
  for (const key of KEYS) {
    out[key] = stored[key] !== undefined ? stored[key] : DEFAULT_CONTENT[key];
  }
  res.json(out);
});
```

سه فایده:

- سایت هیچ‌وقت با بخش خالی بالا نمی‌آید حتی اگر seed اجرا نشده باشد.
- `KEYS` از `DEFAULT_CONTENT` مشتق می‌شود، پس **whitelist خودکار** است: مسیر `PUT /:key` هر کلید ناشناسی را رد می‌کند.
- اضافه کردن بخش تازه فقط یعنی یک کلید به `DEFAULT_CONTENT` و یک ورودی به `contentSchema.js` – بدون مهاجرت پایگاه داده.

۴.۹ چرا slug و نه ObjectId؟

فیلد `slug` در این پروژه چهار نقش همزمان دارد:۱) شناسهٔ خوانا – `yirgacheffe` به جای `66b2f1e8a4c9d20012ab34ef`.۲) بذر تولید تصویر – در `web/src/lib/art.js`:

```
const rnd = seeded(p.slug);
const uid = 'k' + p.slug.replace(/^[a-z0-9]/gi, '');
```

کامنت مدل هم این را می‌گوید: «شناسهٔ انگلیسی و یکتا – هم در آدرس‌ها استفاده می‌شود و هم بذر تولید تصویر کارت است، پس نباید تکراری باشد.»

۳) کلید میکس – `components[].slug` و `pool[]`.۴) کلید سفارش – `lines[].slug` و `mix[].slug`.

قید فرمت آن سخت‌گیرانه است:

```
match: [/^[a-z0-9][a-z0-9-]*$/i, 'شناسه فقط می‌تواند حروف کوچک انگلیسی، عدد و خط تیره باشد']
```

باید با حرف یا رقم شروع شود (نه خط تیره)، و فقط حروف کوچک لاتین، رقم و خط تیره. این تضمین می‌کند که `uid` تولیدشده در SVG یک شناسهٔ معتبر باشد.

برای راحتی مدیر، فرم کالا یک مبدل حرف‌نویسی فارسی به لاتین دارد (`suggestSlug` در `AdminItemForm.jsx`):

```
const map = { 'ا': 'a', 'آ': 'a', 'ب': 'b', 'پ': 'p', 'ت': 't', ... 'ی': 'y', ' ': '-', '': '-' };
return [...String(name).toLowerCase()]
  .map((ch) => (/[a-z0-9]/.test(ch) ? ch : map[ch] ?? ''))
  .join('').replace(/-+/g, '-').replace(/^-|-$/g, '').slice(0, 40);
```

توجه کنید که «`''`» (نیم‌فاصله، U+200C) هم در نگاشت هست و به خط تیره تبدیل می‌شود – جزئیاتی که فقط کسی که با متن فارسی کار کرده به آن فکر می‌کند.

هزینهٔ این انتخاب

بدون یکپارچگی ارجاعی، سه محافظ دستی لازم شده:

(۱) هنگام ذخیرهٔ میکس (`checkBlend` → `routes/items.js`)

```
if (slugs.includes(selfSlug)) return 'یک میکس نمی‌تواند خودش را به‌عنوان جزء داشته باشد';
if (!bean) return `پیدا نشد - فقط قهوه‌ها را می‌شود در میکس گذاشت «${s}» دانهٔ`;
if (bean.isBlend) return `خودش یک میکس است و نمی‌تواند جزء میکس دیگری باشد «${s}»`;
```

(۲) هنگام ثبت سفارش (`routes/orders.js`) – همان بررسی‌ها دوباره روی ترکیب ارسالی مشتری.

(۳) هنگام بارگذاری سبد (`ShopContext.jsx`)

```
const kept = prev.filter(
  (l) => bySlug.has(l.slug) && (l.mix || []).every((m) => bySlug.has(m.slug))
);
```

کامنت: «اگر مدیر کالایی را پاک یا خاموش کند، ردیفش از سبد کاربر هم برداشته می‌شود تا موقع ثبت سفارش خطا نگیرد.»

۴.۱۰ دادهٔ اولیه

توزیع ۱۰۶ کالا

۳۲ قهوه بر اساس `group`:

دسته	تعداد	نمونه
light	۸	یرگاف، گوجی، کنیا AA، گیشا پاناما
medium	۱۱	هویدا، آنتیگو، سرادو، بلو مانتین
dark	۶	سوماترا مندلینگ، مونسون مالابار، رست فرانسوی
espresso	۵	میکس ۷۰/۳۰، ۱۰۰٪ عربیکا، شب‌نشین، کلد برو
decaf	۲	کلمبیا بدون کافئین، اسپرسو بدون کافئین

۴۰ ابزار در شش دسته و ۳۴ پودر در شش دسته.

بازۀ قیمت جالب است: از ۲۹۰،۰۰۰ تومان (لیوان کورتادو) تا ۹۶،۰۰۰،۰۰۰ تومان (ماشین اسپرسو تک‌گروپ E61) و در قهوه‌ها از ۸۲۰،۰۰۰ (میکس ۵۰/۵۰) تا ۸،۵۰۰،۰۰۰ (بلو مانتین) تومان هر کیلو.

پنج میکس اولیه

از `server/src/data/seed-enrich.js`:

slug	ترکیب پیشنهادی	دستمزد میکس
espresso-70-30	سرادو ۷۰٪ + مونسون ۳۰٪	۴۰,۰۰۰
espresso-100	سرادو ۴۰٪ + هویلا ۳۵٪ + یرگاجف ۲۵٪	۵۰,۰۰۰
espresso-50-50	سرادو ۵۰٪ + مونسون ۵۰٪	۳۵,۰۰۰
shabneshin	هویلا ۴۵٪ + سرادو ۳۵٪ + بوگیسو ۲۰٪	۴۵,۰۰۰
cold-brew	سرادو ۶۰٪ + سیدامو ۴۰٪	۴۰,۰۰۰

هر پنج تا `house: true` و `customizable: true` هستند.

الگوریتم seed

فایل `server/src/seed.js` پنج مرحله دارد و ترتیبشان مهم است:

```
await seedItems();      // ۱ کالاهای + متن + برچسب طعمی
await enrichExisting(); // ۲ پر کردن فیلدهای خالی کالاهای قدیمی
await seedBlends();     // ۳ پیکربندی میکسها ← بعد از ساخته شدن دانهها
await seedContent();    // ۴ متنهای سایت
await seedAdmin();      // ۵ حساب مدیر
```

کامنت `seedBlends` دلیل ترتیب را می‌گوید: «میکسها بعد از ساخته شدن همه قهوهها تنظیم می‌شوند، چون اجزایشان باید از قبل وجود داشته باشند.»

و `seedBlends` هوشمندانه فقط دانه‌های موجود را نگه می‌دارد:

```
const beans = await Item.find({
  slug: { $in: [...blend.components.map((c) => c.slug), ...(blend.pool || [])] }
}).select('slug').lean();
const have = new Set(beans.map((b) => b.slug));

const components = blend.components.filter((c) => have.has(c.slug));
if (components.length < 2) continue; // ← میکس ناقص را رها می‌کند
```

ویژگی کلیدی: غیرمخرب بودن. بدون `--reset`:

```
const exists = await Item.exists({ slug: data.slug });
if (exists) { skipped++; continue; }
```

و در `enrichExisting`:

```
for (const field of ['story', 'taste', 'recommend']) {
  if (!item[field] && story[field]) { item[field] = story[field]; changed = true; }
}
```

فقط اگر فیلد خالی باشد پر می‌شود. یعنی می‌توانید بارها `npm run seed` بزنید بدون اینکه ویرایش‌هایتان از بین برود.

۴.۱۱ داده‌های سمت مرورگر

دو چیز در `localStorage` نگه داشته می‌شوند:

کلید	محتوا	خوانده و نوشته می‌شود در
<code>ghahve.cart</code>	آرایه ردیف‌های سبد	<code>lib/cartStorage.js</code> ، از راه <code>ShopContext</code>
<code>ghahve.admin.token</code>	مدیر JWT	<code>lib/api.js</code>

شکل هر ردیف سبد:

```
{
  key: 'espresso-70-30|espresso|cerrado:60,monsooned:40',
  slug: 'espresso-70-30',
  kind: 'coffee',
  grams: 500,
  qty: 0,
  grind: 'espresso',
  mix: [{ slug: 'cerrado', percent: 60 }, { slug: 'monsooned', percent: 40 }]
}
```

توجه کنید که **قیمت ذخیره نمی‌شود**. قیمت هر بار از روی کالاهای تازه از سرور محاسبه می‌شود، پس اگر مدیر قیمت را عوض کند، سبد کاربر خودکار به‌روز می‌شود.

خواندن سبد با بی‌اعتمادی کامل انجام می‌شود – و از **مورد ۲۶** به بعد، بیرون از کامپوننت. منطقش به `web/src/lib/cartStorage.js` منتقل شد و حافظه به‌شکل پارامتر تزریق می‌شود:

```
export function parseCart(raw) {
  let data;
  try { data = JSON.parse(raw || '[]'); } catch { return []; }
  if (!Array.isArray(data)) return [];

  return data
    .filter((l) => l && typeof l.slug === 'string')
    .map((l) => ({
      /* شناسه از نو ساخته می‌شود، نه از فایل خوانده –
      وگرنه ردیفی با شناسه جعلی می‌توانست با ردیف
      دیگری قاطی شود. */
      key: lineKey(l.slug, grind, mix),
      slug: l.slug, kind: l.kind,
      grams: Number(l.grams) || 0, qty: Number(l.qty) || 0,
      grind, mix
    }));
}

export function loadCart(storage) {
  if (!storage) return [];
  try { return parseCart(storage.getItem(CART_KEY)); } catch { return []; }
}

export const browserStorage = () =>
  typeof window === 'undefined' ? null : window.localStorage;
```

دو چیز در این جابه‌جایی عوض شد و هر دو به رندر سمت سرور برمی‌گردند:

(یک) روی سرور `localStorage` وجود ندارد. `browserStorage()` آنجا `null` می‌دهد و `loadCart` سبد خالی برمی‌گرداند – بی هیچ خطایی.

(دو) ترتیب حالا حساس است. اگر پیش از خوانده شدن سبد ذخیره‌شده چیزی نوشته شود، سبد مشتری با یک آرایهٔ خالی پاک می‌شود. برای همین `ShopContext` یک پرچم `hydrated` دارد و نوشتن پشت آن است. شرحش در بخش ۷.۱۲ آمده و `tests/cart-hydration.test.jsx` نگرانش است.

کامنت بالای فایل: «سبد ذخیره‌شده را می‌خوانیم ولی به آن اعتماد نمی‌کنیم؛ هرچه در `localStorage` است دست‌کاربر بوده. پس هر ردیف از نو ساخته می‌شود.» ردیف‌هایی که کالایشان دیگر وجود ندارد بعد از رسیدن فهرست کالاها پاک می‌شوند – با دو نگرهان، که در ۷.۱۲ توضیح داده شده‌اند.

قیمت اینجا هم ذخیره نمی‌شود: تنها چیزی که در حافظه می‌ماند شناسه و مقدار و ترکیب است.

۴.۱۲ جمع‌بندی الگوهای داده‌ای

الگو	جا	چرا
Single-table inheritance	<code>Item</code> یا <code>kind</code>	یک UI و یک API برای سه نوع
Embedded subdocument	<code>components</code> ، <code>lines</code> ، <code>mix</code>	همیشه با والد خوانده می‌شوند
Snapshot / point-in-time	<code>Order.lines[].name/unitPrice/grindLabel</code>	فاکتور نباید با تغییر کالا عوض شود
Denormalization	<code>Order.totals</code>	گزارش‌ها بدون خواندن ردیف‌ها
Key-value یا schema بیرونی	<code>Content</code>	افزودن بخش بدون مهاجرت
کلید طبیعی به‌جای کلید مصنوعی	<code>ClubMember.phone</code> ، <code>Item.slug</code>	خوانایی و نقش چندگانه
Soft delete نمایشی	<code>Item.active</code>	پنهان کردن بدون از دست دادن تاریخچه
Optimistic token invalidation	<code>Admin.tokenVersion</code>	ابطال بدون جدول blacklist
سه‌حالتی با <code>null</code>	<code>Item.stock</code>	«نشمرده»، «مانده» و «تمام شد» سه چیز جدا هستند
رزرو اتمی	<code>findOneAndUpdate</code> با شرط <code>\$gte</code>	فروش بیش از موجودی از اساس ممکن نیست

۵. تحلیل فایل به فایل

این فصل هر فایل مهم پروژه را جداگانه بررسی می‌کند: مسئولیتش چیست، چه توابعی دارد، به چه چیزی وابسته است و چه کسی از آن استفاده می‌کند.

server/src/index.js – نقطه ورود سرور

مسئولیت: ساخت اپلیکیشن Express، ثبت میانافزارها و روترها، سرو کردن فایل‌های ساخته‌شده فرانت، و تبدیل خطاها به پیام فارسی.

وابستگی‌ها: `dotenv/config`، `express`، `cors`، `helmet`، `node:path`، `node:url`، هفت روتر، `./db.js`، `./lib/cors.js`، `./lib/upload.js`

ترتیب میانافزارها (که در Express حیاتی است):

```
/* فقط در تولید یا با پرچم صریح - وگرنه X-Forwarded-For جعلی
محدودیت نرخ را بی‌اثر می‌کند
*/
if (process.env.NODE_ENV === 'production' || process.env.TRUST_PROXY === '1') {
  app.set('trust proxy', 1);
}

app.use(helmet({ contentSecurityPolicy: { directives: { ... } },
  strictTransportSecurity: process.env.NODE_ENV === 'production' }));
app.use(cors({ origin: corsOrigin })); // lib/cors.js از corsOrigin //
app.use(express.json({ limit: '1mb' }));

app.use('/uploads', express.static(UPLOAD_DIR, {
  maxAge: '7d', index: false, dotfiles: 'ignore', setHeaders: setUploadHeaders
}));

app.get('/api/health', (_req, res) => res.json({ ok: true }));

app.use('/api/auth', authRoutes);
app.use('/api/items', itemRoutes);
app.use('/api/orders', orderRoutes);
app.use('/api/content', contentRoutes);
app.use('/api/club', clubRoutes);
app.use('/api/stats', statsRoutes);
app.use('/api/reports', reportRoutes);
```

`helmet` اول می‌آید تا هدرهای امنیتی روی هر پاسخی بنشیند، حتی پاسخ‌های خطا. جزئیات هر بند CSP در فصل ۳ (بخش ۳.۹) آمده است.

روی `/uploads` سه گزینه بیش از پیش‌فرض تنظیم شده: `index: false` تا فهرست پوشه لو نرود، `dotfiles: 'ignore'` و `setHeaders` که هدرهای سخت‌گیر `lib/upload.js` را می‌گذارد.

خروج مرکبار وقتی CORS تنظیم نشده:

```
let corsOrigin;
try {
  corsOrigin = resolveCorsOrigin();
} catch {
  console.error('\nX CLIENT_ORIGIN must be set in production.');
```

خودِ تصمیم در `lib/cors.js` است و `throw` می‌کند؛ `index.js` فقط آن را به پیام خط فرمان و `exit` تبدیل می‌کند. این تقسیم عمده است – تابعی که `process.exit` صدا بزند تست‌پذیر نیست.

آنچه اینجا بود و رفت – بازگشت SPA.

تا پیش از مهاجرت به Next (مورد ۲۶)، همین فایل خروجی build فرانت را هم سرو می‌کرد: یک `express.static` روی `client/dist` و یک الگوی negative-lookahead که هر آدرسی جز `/api` و `/uploads` را به `index.html` می‌فرستاد تا `react-router` مسیریابی کند.

هر سه تکه حذف شدند – `express.static(CLIENT_DIST)`، بازگشت SPA، و `routes/itemPages.js` که برای `<head>` تگ‌های `/coffee/:slug` را داخل همان HTML تزریق می‌کرد. حالا صفحه‌ها را Next می‌سازد و این سرور هیچ HTML ای نمی‌دهد. آخرین میان‌افزارش این است:

```
app.use((_req, res) => res.status(404).json({ error: 'این آدرس وجود ندارد' }));
```

یعنی پاسخ آدرس ناشناخته JSON است، نه صفحه – که برای یک API درست است. تنها آدرس‌های غیر-API که مانده‌اند `sitemap.xml` و `robots.txt` اند، چون ربات‌ها فقط در ریشه دنبالشان می‌گردند و ساختنشان به مونگو نیاز دارد.

فشرده‌سازی (مورد ۲۴). پیش از helmet و مسیرها می‌نشیند تا خروجی‌های استریمی هم از آن رد شوند:

```
app.use(compression());
```

فهرست کالاها با همین یک خط از ۵۳ کیلوبایت به ۱۱.۷ کیلوبایت می‌رسد؛ متن فارسی در UTF-8 سه بایت برای هر حرف می‌گیرد و خوب فشرده می‌شود.

راهنمای EADDRINUSE: بخشی که کمتر در پروژه‌ها دیده می‌شود:

```
server.on('error', (err) => {
  if (err.code === 'EADDRINUSE') {
    console.error(`\nX Port ${PORT} is already in use.`);
    console.error('  Another copy of the server is still running.');
```

```
    console.error('  Close it, or change PORT in server/.env');
```

```
    console.error('\n  To find and stop it on Windows (PowerShell):');
```

```
    console.error('    Get-NetTCPConnection -LocalPort ${PORT} -State Listen | Select-Object
```

```
OwningProcess`);
```

```
    console.error('    Stop-Process -Id <number-from-above> -Force\n');
```

```
  }
```

```
  process.exit(1);
});
```

به جای stack trace بیست‌خطی Node، دقیقاً دستور رفع مشکل داده می‌شود.

server/src/db.js – اتصال پایگاه داده

توابع: فقط `connectDB()`

```
mongoose.set('strictQuery', true);
await mongoose.connect(uri, { serverSelectionTimeoutMS: 8000 });
```

یعنی فیلدهایی که در schema نیستند در فیلتر کوئری نادیده گرفته می‌شوند (نه اینکه خطا بدهند) – رفتار امن‌تر در برابر پارامترهای ناخواسته کاربر.

به‌جای ۳۰ ثانیه پیش‌فرض، خطا را سریع‌تر نشان می‌دهد. `serverSelectionTimeoutMS: 8000`

اگر `MONGODB_URI` نباشد یا اتصال شکست بخورد، `process.exit(1)` می‌کند – یعنی سرور بدون پایگاه داده بالا نمی‌آید.

نگهبان مسیرها – `server/src/middleware/auth.js`

توابع:

تابع	امضا	کار
<code>signToken</code>	<code>(admin) → String</code>	ساخت JWT با <code>{sub, v}</code>
<code>requireAdmin</code>	<code>(req, res, next)</code>	بررسی توکن و پر کردن <code>req.admin</code>

```
export async function requireAdmin(req, res, next) {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;

  if (!token) return res.status(401).json({ error: 'برای این کار باید وارد شوید' });

  try {
    const payload = jwt.verify(token, process.env.JWT_SECRET);
    const admin = await Admin.findById(payload.sub);

    if (!admin) return res.status(401).json({ error: 'حساب مدیر پیدا نشد' });
    if (admin.tokenVersion !== payload.v) {
      return res.status(401).json({ error: 'رمز عوض شده است، دوباره وارد شوید' });
    }

    req.admin = admin;
    next();
  } catch {
    return res.status(401).json({ error: 'نشست شما منقضی شده است، دوباره وارد شوید' });
  }
}
```

سه لایه بررسی: وجود توکن → معتبر بودن امضا و انقضا → همخوانی `tokenVersion`. سه پیام خطای متفاوت هم به کاربر کمک می‌کند بفهمد چه شده.

`catch` خالی هر خطای `jwt.verify` (امضای غلط، انقضا، فرمت خراب) را به یک پیام واحد تبدیل می‌کند – تا جزئیات فنی به مهاجم اطلاعات ندهد.

server/src/middleware/rateLimit.js — محدودیت نرخ

صادرات: `loginLimiter`، `orderLimiter`، `trackLimiter`.

هر سه از یک سازندهٔ مشترک می‌آیند تا رفتارشان یکی باشد:

```
function make({ windowMs, limit, error }) {
  return rateLimit({
    windowMs,
    limit,
    standardHeaders: 'draft-7', // هدرهای *RateLimit //
    legacyHeaders: false,       // قدیمی لازم نیست *X-RateLimit //
    skip: isOff,
    message: { error }          // همان شکل { error } بقیهٔ مسیرها //
  });
}
```

عمدی است: رابط کاربری برای ۴۲۹ هیچ کد خاصی ندارد، چون شکل پاسخ با بقیهٔ خطاها یکی است و همان یک خط `data?.error` کافی است.

اعداد از محیط، با پیش‌فرض امن:

```
const num = (raw, fallback) => {
  const n = Number(raw);
  return Number.isFinite(n) && n > 0 ? n : fallback;
};
```

مقدار خالی، منفی یا بی‌معنی در `.env` به پیش‌فرض برمی‌گردد — نه به `NaN` که عملاً محدودیت را باز می‌کرد.

پیش‌فرض	متغیر محیطی
۱۵ دقیقه / ۵	<code>RATE_LIMIT_LOGIN_MAX</code> / <code>RATE_LIMIT_LOGIN_WINDOW_MIN</code>
۶۰ دقیقه / ۵	<code>RATE_LIMIT_ORDER_MAX</code> / <code>RATE_LIMIT_ORDER_WINDOW_MIN</code>
۱۵ دقیقه / ۲۰	<code>RATE_LIMIT_TRACK_MAX</code> / <code>RATE_LIMIT_TRACK_WINDOW_MIN</code>

خاموشی در تست:

```
const isOff = () =>
  process.env.NODE_ENV === 'test' || process.env.RATE_LIMIT_DISABLED === '1';
```

مجموعهٔ تست باید بتواند پشت‌سرهم ده‌ها بار همان مسیر را صدا بزند؛ `RATE_LIMIT_DISABLED` هم برای آزمایش دستی روی ماشین خودی است. `tests/rate-limit.test.js` دقیقاً همین را می‌سنجد — که در محیط تست خیلی بیشتر از سقف هم عبور می‌کند.

CORS – تصمیم server/src/lib/cors.js

صادرات: `parseOrigins(raw)` و `resolveCorsOrigin(env)`.

سه خط منطق که یک فایل جدا گرفته‌اند، چون تنها جایی است که تصمیم می‌گیرد «چه کسی حق دارد به API ما درخواست بزند»:

```
export function resolveCorsOrigin(env = process.env) {
  const origins = parseOrigins(env.CLIENT_ORIGIN);
  if (!origins.length && env.NODE_ENV === 'production') {
    throw new Error('CLIENT_ORIGIN must be set in production');
  }
  return origins.length ? origins : true;
}
```

`env` پارامتر است، نه `process.env` مستقیم – همین یک تغییر کوچک اجازه داده `tests/cors.test.js` هر چهار ترکیب (تولید/توسعه × تنظیم‌شده/نشده) را بدون دست‌کاری محیط واقعی بسنجد.

`parseOrigins` با کاما می‌شکند، فاصله‌ها را می‌گیرد و خالی‌ها را حذف می‌کند، پس `"https://a.com, , https://b.com"` دو مبدأ می‌دهد.

رزرو اتمی موجودی server/src/lib/stock.js

صادرات: `unitsFor`، `isTracked`، `reserveStock`، `releaseStock`.

مسئله: دو مشتری هم‌زمان آخرین ۵۰۰ گرم یرگاف را سفارش می‌دهند. با الگوی «اول بخوان، بعد بنویس» هر دو «۵۰۰ گرم موجود است» را می‌بینند و هر دو سفارش ثبت می‌شود – یک کیلو فروخته‌ایم که نداریم.

راه‌حل: شرط و کاهش در یک عملیات، زیر یک قفل سند:

```
const updated = await ItemModel.findOneAndUpdate(
  { slug: item.slug, stock: { $gte: need } },
  { $inc: { stock: -need } },
  { new: true, projection: { stock: 1 } }
);
```

دقیقاً یکی از آن دو مشتری برنده می‌شود و دیگری `null` می‌گیرد. هیچ transaction ای هم لازم نیست – که روی مونگوی تک‌گرهی اصلاً در دسترس نیست.

سه تصمیم ظریف:

(۱) کالای نامحدود اصلاً لمس نمی‌شود. `isTracked` اول بررسی می‌کند:

```
export function isTracked(item) {
  return typeof item?.stock === 'number' && Number.isFinite(item.stock);
}
```

`$inc` روی `null` خطا می‌دهد و شرط `{ $gte: need }` هم با `null` جور نمی‌شود؛ یعنی حتی اگر اشتباهاً صدا زده شود، نتیجه به‌جای خراب شدن «ناموجود» می‌شود.

۲) واحد از نوع کالا می‌آید. `unitsFor` برای ابزار `qty` و برای وزنی `grams` را برمی‌دارد و گرد می‌کند؛ ورودی نامعتبر صفر می‌شود، نه `NaN`.

۳) همه یا هیچ. هر ردیف جدا رزرو می‌شود، پس ممکن است ردیف سوم شکست بخورد بعد از آنکه دو ردیف اول کم شده‌اند:

```
if (!updated) {  
  // هرچه برداشته‌ایم برمی‌گردد  
  await releaseStock(taken, ItemModel);  
  const fresh = await ItemModel.findOne({ slug: item.slug }, { stock: 1 }).lean();  
  const left = typeof fresh?.stock === 'number' ? fresh.stock : 0;  
  return { error: shortMessage(item, left), taken: [] };  
}
```

موجودی برای پیام خطا دوباره از پایگاه داده خوانده می‌شود، چون عددی که در حافظه داشتیم ممکن است مال چند لحظه پیش باشد.

پیام هم واحدش را از نوع کالا می‌گیرد: «فقط ۳۰۰ گرم مانده است» در برابر «فقط ۲ عدد مانده است»، و اگر صفر باشد «فعلاً موجود نیست».

server/src/lib/track.js – پیگیری سفارش

صادرات: `normalizePhone`، `normalizeCode`، `safeEqual`، `publicOrderView`، `NOT_FOUND_MESSAGE`، `findOrderForTracking`

مشتری حساب کاربری ندارد، پس تنها چیزی که مالکیتش را ثابت می‌کند جفت «شماره سفارش + شماره موبایل» است: کد را فقط کسی دارد که رسید را دیده و شماره را فقط کسی که خودش سفارش داده.

سه قاعده‌ای که این فایل نگه می‌دارد:

۱) هیچ‌وقت لو نده که کدی وجود دارد یا نه. کد درست با شماره غلط باید دقیقاً همان پاسخ «کد اصلاً وجود ندارد» را بگیرد، وگرنه صفحه پیگیری به ابزار شمارش سفارش‌ها تبدیل می‌شود.

۲) مقایسه در زمان ثابت – وگرنه خود زمان پاسخ همان چیزی را لو می‌دهد که در بند ۱ پنهانش کردیم:

```
export function safeEqual(a, b) {  
  const ha = crypto.createHash('sha256').update(String(a ?? ''), 'utf8').digest();  
  const hb = crypto.createHash('sha256').update(String(b ?? ''), 'utf8').digest();  
  return crypto.timingSafeEqual(ha, hb);  
}
```

اول هش، بعد مقایسه – چون `timingSafeEqual` طول‌های نابرابر را قبول نمی‌کند و خود پرتاب خطا یک نشانه زمانی است. خروجی sha256 همیشه ۳۲ بایت است، پس مقایسه همیشه یک شکل دارد.

و در `findOrderForTracking`، حتی وقتی سفارشی پیدا نشده هم یک مقایسه انجام می‌شود:

```
const stored = normalizePhone(doc?.customer?.phone);  
const matches = safeEqual(stored || 'no-order', wantedPhone);  
if (!doc || !matches) return { status: 404, error: NOT_FOUND_MESSAGE };
```

۳) فقط فیلدهایی که مشتری لازم دارد. `publicOrderView` یک whitelist صریح می‌سازد، نه یک `delete` روی سند:

```
return {
  code, status, createdAt,
  customer: { name: order.customer?.name || '' },
  lines: (order.lines || []).map((l) => ({ kind, name, grams, qty,
    unitPrice, lineTotal, grindLabel, mix })),
  totals: { base, discount, discountLabel, shipping, total }
};
```

عمداً بیرون مانده‌اند: `_id` و `__v` و `updatedAt` (داخلی)، و نشانی و شماره تماس و یادداشت مشتری. دیدن وضعیت و فهرست کالاها به نشانی نیازی ندارد، و اگر روزی کسی جفت کد و شماره را حدس زد نباید نشانی خانه کسی را هم برداشته باشد. مزیت whitelist این است که هر فیلد داخلی‌ای که فردا به مدل اضافه شود، به‌طور پیش‌فرض بیرون می‌ماند – و `track.test.js` دقیقاً همین را می‌سجد.

نرمال‌سازی ورودی. مشتری با کیبورد فارسی تایپ می‌کند و شکل‌های مختلف باید یکی شمرده شوند:

```
normalizePhone('۰۹۱۲ ۱۲۳-۴۵۶۷') // '09121234567'
normalizePhone('+989121234567') // '09121234567'
normalizeCode('a1b23456') // 'A1B2-3456'
```

`normalizePhone` قرینه `toLatinDigits` در کلاینت است و `0098...` / `98...` / `9...` را همه به `09...` می‌رساند. `normalizeCode` خط تیره را خودش می‌گذارد، چون شکل تولیدی همیشه چهار نویسه + چهار رقم است. ورودی ناقص (کد یا شماره خالی) `400` می‌گیرد با پیام راهنما – نه `404`، چون اینجا چیزی برای پنهان کردن نیست.

server/src/lib/upload.js – آپلود تصویر

صادرات: `UPLOAD_DIR`، `ALLOWED`، `upload` (نمونه `multer`)، و `setUpUploadHeaders`.

```
export const UPLOAD_DIR = path.join(__dirname, '..', '..', 'uploads');
fs.mkdirSync(UPLOAD_DIR, { recursive: true });
```

پوشه در زمان `import` ساخته می‌شود (`recursive: true`) یعنی اگر بود خطا نده).

فهرست سفید – و غیبتِ عمده SVG:

```
export const ALLOWED = {
  'image/jpeg': '.jpg',
  'image/png': '.png',
  'image/webp': '.webp',
  'image/gif': '.gif',
  'image/avif': '.avif'
};
```

SVG یک سند XML است، نه تصویر خام: می‌تواند `<script>` داشته باشد. و چون از `/uploads` روی همان دامنه سایت سرو می‌شود، آن اسکریپت در مبدأ خودِ فروشگاه اجرا می‌شد – یعنی به توکن مدیر در `localStorage` هم دسترسی داشت. تصویرهای برداری خودِ سایت (`public/img`) از این راه نمی‌آیند و دست‌نخورده‌اند؛ این فهرست فقط درباره چیزی است که مدیر از پنل آپلود می‌کند.

پیام رد کردن هم عمداً اسم SVG را نمی‌آورد، فقط می‌گوید چه چیزی مجاز است: «فقط تصویر JPG، PNG، WebP، AVIF یا GIF قابل آپلود است».

نام تصادفی و سقف حجم:

```
const ext = ALLOWED[file.mimetype] || path.extname(file.originalname).toLowerCase() || '.bin';
cb(null, crypto.randomBytes(12).toString('hex') + ext);
// limits: { fileSize: 4 * 1024 * 1024 }
```

پسوند از **mimetype** ساخته می‌شود نه از نام فرستاده‌شده، و نام اصلی (که ممکن است فارسی یا تکراری باشد) اصلاً استفاده نمی‌شود.

لایه دوم – هدرهای پوشه. بستن فهرست مجاز فقط جلوی آپلود تازه را می‌گیرد؛ هرچه پیش از این آپلود شده هنوز روی دیسک است و سرو می‌شود:

```
export function setUpUploadHeaders(res, filePath) {
  res.setHeader('X-Content-Type-Options', 'nosniff');
  res.setHeader('Content-Security-Policy', "default-src 'none'; sandbox");
  res.setHeader('Cross-Origin-Resource-Policy', 'same-origin');
  res.setHeader('X-Frame-Options', 'DENY');

  if (DOCUMENT_LIKE.has(path.extname(String(filePath || '')).toLowerCase())) {
    res.setHeader('Content-Disposition', 'attachment');
  }
}
```

چهری را می‌بندد	هدر
مرورگر حق ندارد نوع فایل را از محتوایش حدس بزند؛ PNG با محتوای HTML همان PNG می‌ماند	<code>nosniff</code>
سند حق هیچ درخواستی ندارد و در مبدأ یکتا و بدون اسکریپت می‌نشیند – SVG قدیمی آلوده هم دیگر به مبدأ فروشگاه دسترسی ندارد	<code>default-src 'none';</code> <code>sandbox</code>
سایت دیگری نمی‌تواند این فایل‌ها را به منابع خودش وصل کند	<code>Cross-Origin-Resource-Policy</code>
باز کردن مستقیم <code>/uploads/x.svg</code> به جای رندر، دانلود می‌شود	<code>Content-Disposition:</code> <code>attachment</code>

روی `DOCUMENT_LIKE` شامل `.svg` `.svgz` `.xml` `.html` `.htm` `.xhtml` است. مهم است که `Content-Disposition` اثری ندارد، پس تصویرهای عادی کارتها سالم می‌مانند – و `upload.test.js` همین را جداگانه می‌سنجد.

`server/src/models/*.js`

جزئیات کامل در فصل ۴. خلاصه نقش هر کدام:

فایل	صادرات	نکته کلیدی
Item.js	Item	قلاب <code>pre('validate')</code> با هفت گروه قاعده
Order.js	Order	سه سطح زیرسند، تولید <code>code</code>
Admin.js	Admin	<code>select:false</code> ، <code>checkPassword</code> / <code>setPassword</code>
ClubMember.js	ClubMember	کلید یکتا <code>phone</code>
Content.js	<code>loadContent</code> ، <code>Content</code>	<code>minimize:false</code> + <code>Mixed</code>

server/src/routes/auth.js – احراز هویت

سه مسیر:

POST /login – نکته امنیتی اصلی اش پیام یکسان است:

```
const admin = await Admin.findOne({ username }).select('+passwordHash');

/* پیام یکسان برای نام کاربری اشتباه و رمز اشتباه،
تا نشود فهمید کدام نام کاربری وجود دارد. */
const ok = admin && (await admin.checkPassword(password));
if (!ok) return res.status(401).json({ error: 'نام کاربری یا رمز عبور درست نیست' });
```

اگر پیام‌ها فرق می‌کردند («کاربر یافت نشد» در برابر «رمز غلط»)، مهاجم می‌توانست فهرست نام‌های کاربری معتبر را کشف کند (user enumeration).

GET /me – فقط `requireAdmin` و برگرداندن نام کاربری. کلاینت با این مسیر در بارگذاری اول اعتبار توکن ذخیره‌شده را می‌سنجد.

POST /change-password – پنج بررسی به ترتیب:

```
if (!(await admin.checkPassword(current))) → 'رمز فعلی درست نیست'
if (next_.length < 6) → 'رمز جدید دست‌کم ۶ نویسه باشد'
if (next_ !== confirm) → 'رمز جدید و تکرارش یکی نیستند'
if (next_ === current) → 'رمز جدید با رمز فعلی فرقی ندارد'
if (newUsername && taken) → 'این نام کاربری قبلاً گرفته شده است'
```

سپس:

```
await admin.setPassword(next_);
admin.tokenVersion += 1; // توکن‌های قدیمی باطل می‌شوند
await admin.save();

res.json({ ok: true, token: signToken(admin), admin: {...},
  message: 'رمز عبور با موفقیت عوض شد' });
```

بلافاصله توکن تازه برگردانده می‌شود تا خودِ مدیر از پنل بیرون نیفتد. کامنت: «توکن تازه برمی‌گردانیم تا کاربر از پنل بیرون نیفتد.»

توابع کمکی:

whitelist و نرمال‌سازی. مهم‌ترین محافظ در برابر **mass assignment**: `pickBody(body)`

```
const WRITABLE = ['kind', 'slug', 'name', 'origin', 'spec', 'group', 'meter', 'price', 'stock',
  'notes', 'pairs', 'tag', 'shape', 'mat', 'tone', 'zoom', 'image', 'rank', 'active',
  'story', 'taste', 'recommend', 'tastes',
  'isBlend', 'house', 'customizable', 'components', 'pool', 'surcharge',
  'featured', 'pinnedTop', 'excludeTop'];

function pickBody(body) {
  const out = {};
  for (const key of WRITABLE) {
    if (body[key] === undefined) continue;
    out[key] = body[key];
  }
  ...
}
```

هر فیلدی که در `WRITABLE` نباشد — از جمله `_id`، `createdAt`، `grindable`، `weighed` — بی‌صدا حذف می‌شود.

`stock` نرمال‌سازی خودش را دارد، چون سه حالت متفاوت دارد و «خالی» با «صفر» یکی نیست:

```
/* موجودی جدا از بقیهٔ عددهاست، چون «خالی» معنای خودش
را دارد: نامحدود (null)، نه صفر. اگر با NUMBERS
حساب می‌شد، رشتهٔ خالی دست‌نخورده می‌ماند و مونگو
موقع cast خطا می‌داد. */
if (out.stock !== undefined) {
  const raw = out.stock === null ? '' : String(out.stock).trim();
  const n = raw === '' ? NaN : Number(raw);
  out.stock = Number.isFinite(n) ? Math.max(0, Math.round(n)) : null;
}
```

فیلد خالی در فرم یعنی `null` یعنی **نامحدود**؛ صفر یعنی **تمام شد**. اگر این دو یکی می‌شدند، پاک کردن عدد از فرم به‌جای «نامحدود» می‌شد «ناموجود».

سپس سه نرمال‌سازی نوع:

```
/* فهرست‌ها ممکن است به‌صورت رشتهٔ خط‌به‌خط بیایند */
for (const key of LISTS) {
  if (typeof out[key] === 'string') {
    out[key] = out[key].split('\n').map((s) => s.trim()).filter(Boolean);
  }
  if (Array.isArray(out[key])) {
    out[key] = out[key].map((s) => String(s).trim()).filter(Boolean);
  }
}

for (const key of NUMBERS) if (out[key] !== undefined && out[key] !== '') out[key] = Number(out[key]);
for (const key of BOOLEANS) if (out[key] !== undefined) out[key] = out[key] === true || out[key] === 'true';
```

پذیرش `'true'` رشته‌ای در کنار `true` بولی، برای سازگاری با فرم‌های HTML است.

– `checkBlend(data, selfSlug)` سه بررسی یکپارچگی که در فصل ۴ آمد.

– `removeImageFile(imagePath)` پاکسازی فایل یتیم:

```
function removeImageFile(imagePath) {
  if (!imagePath || !imagePath.startsWith('/uploads/')) return;
  const file = path.join(UPLOAD_DIR, path.basename(imagePath));
  fs.promises.unlink(file).catch(() => {}); // نبودن فایل مشکلی نیست
}
```

دو محافظ امنیتی: شرط `startsWith('/uploads/')` و `path.basename()` که هر `../` را حذف می‌کند. `.catch(() => {})` هم یعنی اگر فایل نبود، برنامه نمی‌شکند.

هفت مسیر:

مسیر	نکته
GET /	<code>lean({virtuals:true}) + sort({rank:1, name:1}) + {active: true}</code>
GET /admin/all	بدون فیلتر <code>active</code> ، با جست‌وجوی <code>regex</code> روی پنج فیلد
GET /admin/:id	یک کالا برای فرم
POST /	<code>pickBody</code> → <code>checkBlend</code> → <code>Item.create</code> → ۲۰۱
PUT /:id	<code>findById</code> → <code>checkBlend</code> → پاک کردن عکس قدیمی → <code>Object.assign</code> → <code>save()</code>
PATCH /:id/active	<code>item.active = !item.active</code>
DELETE /:id	<code>removeImageFile</code> + <code>findByIdAndDelete</code>
POST /upload	multer با مدیریت خطای دستی

مسیر آپلود شکل خاصی دارد چون خطاهای `multer` باید به فارسی تبدیل شوند:

```
router.post('/upload', requireAdmin, (req, res) => {
  upload.single('image')(req, res, (err) => {
    if (err) return res.status(400).json({ error: err.message });
    if (!req.file) return res.status(400).json({ error: 'فایلی انتخاب نشده است' });
    res.json({ url: `/uploads/${req.file.filename}` });
  });
});
```

به‌جای اینکه `upload.single('image')` به‌عنوان میان‌افزار زنجیره شود، دستی صدا زده می‌شود تا `callback` خطا در اختیار باشد.

– ثبت و مدیریت سفارش `server/src/routes/orders.js`

مهم‌ترین فایل امنیتی پروژه. تحلیل کامل در فصل ۶ و ۷.

تابع کمکی `: grindMap()`

```

async function grindMap() {
  const doc = await Content.findOne({ key: 'grinds' }).lean();
  const options = Array.isArray(doc?.data?.options) && doc.data.options.length
    ? doc.data.options
    : DEFAULT_GRINDS;
  return new Map(options.map((o) => [o.value, o.label]));
}

```

گزینه‌های آسیاب از پایگاه داده خوانده می‌شوند (چون مدیر می‌تواند عوضشان کند) با fallback به `DEFAULT_GRINDS`.

ثابت: `const MIN_GRAMS = { coffee: 100, powder: 50 };`

شش مسیر: `POST /` (عمومی)، `GET /track` (عمومی)، `GET /` (فهرست + شمارش)، `GET /:id`، `PATCH /:id/status`، `DELETE /:id`.

ترتیب `/track` قبل از `/:id` اجباری است. Express اولین مسیرِ چوردآمده را برمی‌دارد و `/:id` همه‌چیز را می‌گیرد؛ اگر جابه‌جا شوند، `/track` پشت `requireAdmin` گیر می‌کند و مشتری هرگز به آن نمی‌رسد. کامنت بالای مسیر همین را می‌گوید.

خود مسیر پیگیری تقریباً خالی است، چون منطقش در `lib/track.js` نشسته تا بدون مونگو تست شود:

```

router.get('/track', trackLimiter, async (req, res, next) => {
  try {
    const result = await findOrderForTracking(
      { code: req.query.code, phone: req.query.phone }, Order
    );
    if (result.status !== 200) {
      return res.status(result.status).json({ error: result.error });
    }
    res.json(result.order);
  } catch (err) { next(err); }
});

```

رزرو موجودی، درست پیش از ثبت:

```

/* آخرین کاری است که پیش از ثبت انجام می‌شود: تا اینجا
  هر خطای اعتبارسنجی بدون لمس انبار برگشته است، پس
  چیزی برای پس دادن نمی‌ماند. */
const { error: stockError, taken } = await reserveStock(lines, Item);
if (stockError) return res.status(409).json({ error: stockError });

let order;
try {
  order = await Order.create({ ... });
} catch (err) {
  await releaseStock(taken, Item); // انبار نباید بی‌دلیل کم بماند
  throw err;
}

```

جای این چند خط در تابع مهم است: بعد از همهٔ اعتبارسنجی‌ها و قبل از `Order.create`. هر خطای زودتر (وزن نامعتبر، دانهٔ ناشناخته، مجموع درصدهای غلط) پیش از لمس انبار برمی‌گردد، پس نیازی به پس دادن ندارد.

مسیر فهرست، شمارش هر وضعیت را هم می‌دهد تا پتل بتواند عدد کنار هر دکمهٔ فیلتر را نشان دهد:

```

const counts = await Order.aggregate([
  { $group: { _id: '$status', n: { $sum: 1 } } }
]);
res.json({ orders, counts: Object.fromEntries(counts.map((c) => [c._id, c.n])) });

```


متن‌های سایت — server/src/routes/content.js

سه مسیر ساده. نکته اصلی، whitelist خودکار کلیدها است:

```
const KEYS = Object.keys(DEFAULT_CONTENT);

if (!KEYS.includes(key)) return res.status(400).json({ error: 'این بخش محتوا وجود ندارد' });
```

و اعتبارسنجی حداقلی نوع:

```
const data = req.body?.data;
if (data === undefined || data === null || typeof data !== 'object') {
  return res.status(400).json({ error: 'محتوای ارسالی درست نیست' });
}
```

ذخیره با `upsert` انجام می‌شود تا اگر بخش هنوز در پایگاه داده نبود، ساخته شود:

```
const doc = await Content.findOneAndUpdate(
  { key }, { key, data },
  { new: true, upsert: true, setDefaultsOnInsert: true }
);
```

هشدار

`typeof data !== 'object'` تنها بررسی است. یعنی اگر مدیر `{reasons: "not an array"}` بفرستد، ذخیره می‌شود و بعداً `WhyUsSection` با `Array.isArray(c.reasons)` آن را رد می‌کند و بخش اصلاً رندر نمی‌شود. رفتار امن است اما بازخوردی به مدیر نمی‌دهد.

باشگاه مشتریان — server/src/routes/club.js

تابع `toLatin`:

```
const toLatin = (s) =>
  String(s ?? '')
    .replace(/[\u06F0-\u06F9]/g, (d) => String('۰۱۲۳۴۵۶۷۸۹'.indexOf(d)))
    .replace(/[\u0660-\u0669]/g, (d) => String('۰۱۲۳۴۵۶۷۸۹'.indexOf(d)))
    .replace(/[\s-]/g, '');
```

سه تبدیل: رقم فارسی (U+06F0–U+06F9) → لاتین، رقم عربی (U+0660–U+0669) → لاتین، و حذف فاصله و خط تیره. پس ۰۹۱۲ به 09121234567 تبدیل می‌شود.

توجه کنید که رقم فارسی و عربی دو بلوک یونیکد جدا هستند. کاربر ایرانی معمولاً رقم فارسی می‌زند اما بعضی کیبوردها رقم عربی می‌دهند؛ هر دو پوشش داده شده‌اند.

: GET /export.csv

```
const esc = (v) => `"${String(v ?? '').replace(/"/g, '"')}"`;
const rows = [
  ,('')map(esc).join(['تاریخ عضویت', 'سلیقه', 'ایمیل', 'موبایل', 'نام'])
  ...members.map(m) => [m.name, m.phone, m.email, m.taste,
    new Date(m.createdAt).toISOString()].map(esc).join(',')
].join('\r\n');

res.setHeader('Content-Type', 'text/csv; charset=utf-8');
res.setHeader('Content-Disposition', 'attachment; filename="club-members.csv"');
/* BOM تا اکسل فارسی را درست باز کند */
res.send('' + rows);
```

سه جزئیات مهم:

- برای escape کردن نقل قول داخل مقدار (استاندارد RFC 4180). `"`
- به عنوان جداکننده خط (اکسل ویندوز). `\r\n`
- **BOM** () در ابتدا — بدون آن، اکسل فایل UTF-8 را با کدپیچ محلی می‌خواند و فارسی به هم می‌ریزد.

ویترین — server/src/routes/stats.js

GET /top-sellers — تحلیل کامل الگوریتم در فصل ۷.

GET /featured — ساده:

```
const items = await Item.find({ active: true, featured: true })
  .sort({ rank: 1, name: 1 }).limit(limit).lean({ virtuals: true });
```

هر دو مسیر `Math.min(Number(req.query.limit) || 8, 24)` دارند تا کاربر نتواند با `?limit=100000` سرور را خسته کند.

گزارش و پشتیبان — server/src/routes/reports.js

بزرگ‌ترین روتر پروژه (۳۸۵ خط).

ثابت‌ها:

```
const DEFAULT_COUNT = { day: 30, week: 12, month: 12, year: 6 };
const MAX_COUNT      = { day: 366, week: 260, month: 120, year: 40 };
```

buildFilter(query) — فیلتر مشترک بین چهار مسیر:

```
function buildFilter(query) {
  const filter = {};

  const from = parseDate(query.from);
  const to = parseDate(query.to);
  if (from || to) {
    filter.createdAt = {};
    if (from) filter.createdAt.$gte = from;
    if (to) filter.createdAt.$lt = to;
  }

  if (query.status && query.status !== 'all') {
    if (!STATUSES.includes(query.status)) {
      const err = new Error('وضعیت نامعتبر است'); err.status = 400; throw err;
    }
    filter.status = query.status;
  }

  const q = String(query.q || '').trim();
  if (q) {
    const rx = new RegExp(q.replace(/[\.*+?^${}()|[\]\|\|]/g, '\\\\&'), 'i');
    filter.$or = [{ code: rx }, { 'customer.name': rx },
      { 'customer.phone': rx }, { 'lines.name': rx }];
  }

  return filter;
}
```

توجه کنید که بازه `$gte` تا `$lt` است (نه `$lte`) - یعنی بسته چپ، باز راست. این با `bucket.end` که دقیقاً `start` بازه بعدی است جور درمی‌آید و از شمارش دوباره جلوگیری می‌کند.

جست‌وجو حتی داخل `lines.name` هم می‌گردد، پس می‌شود پرسید «کدام سفارش‌ها برگارچ داشتند؟»

csvCell(v) – محافظ CSV injection:

```
/* اکسل هر سلولی را که با + = - @ شروع شود «فرمول» حساب
می‌کند. نشانی و توضیح مشتری را خود مشتری نوشته، پس
ابتدایش یک آپاستروف می‌گذاریم تا متن بماند. */
function csvCell(v) {
  let s = String(v ?? '');
  if (/^[+=\@-@t\r]/.test(s)) s = "'" + s;
  return `"${s.replace(/"/g, '""')}"`;
}
```

این یکی از معدود جاهایی است که پروژه به حمله **Formula Injection** فکر کرده. اگر مشتری در فیلد نشانی بنویسد `=cmd|' /c` و مدیر فایل را در اکسل باز کند، بدون این محافظ دستور اجرا می‌شد.

دو حالت خروجی CSV:

mode	یک ردیف به ازای	ستون‌ها
orders	هر سفارش	۱۸ ستون شامل خلاصه کالاها
lines	هر ردیف کالا	۱۷ ستون شامل ترکیب میکس

توابع کمکی `mixText(l)`، `linesSummary(o)`، `orderRow(o)`، `lineRows(o)`.

استریم کردن: هر دو خروجی با `cursor()` نوشته می‌شوند. برای JSON، حتی ساختار خارجی هم دستی نوشته می‌شود:

```
res.write('{\n');
res.write(`  "exportedAt": ${JSON.stringify(new Date().toISOString())},\n`);
res.write('  "orders": [\n');

const cursor = Order.find(filter).sort({ createdAt: 1 }).lean().cursor();
let n = 0;
for await (const o of cursor) {
  res.write((n ? ',\n' : '') + '    ' + JSON.stringify(o));
  n += 1;
}
res.write(`\n  ],\n  "count": ${n}\n`);
res.end();
```

کاماهای بین عناصر آرایه را می‌گذارد بدون اینکه کامای اضافه در انتها بیفتد. `(n ? ',\n' : '')`

مدیریت خطا در حین استریم:

```
} catch (err) {
  if (res.headersSent) return res.destroy(err);
  ...
}
```

اگر هدرها رفته باشند دیگر نمی‌شود کد وضعیت عوض کرد؛ تنها کار درست، قطع کردن اتصال است تا کلاینت بفهمد فایل ناقص است.

نام فایل با پشتیبانی یونیکد:

```
res.setHeader('Content-Disposition',
  `attachment; filename="${name}"; filename*=UTF-8'${encodeURIComponent(name)}`);
```

شکل `filename*=UTF-8'...` طبق RFC 5987 برای نام‌های غیر-ASCII است و مرورگرهای مدرن آن را ترجیح می‌دهند.

`server/src/data/*`

فایل	صادرات	نکته
<code>default-content.js</code>	<code>DEFAULT_CONTENT</code>	هفت کلید؛ <code>KEYS</code> روتر از اینجا مشتق می‌شود
<code>seed-items.js</code>	<code>SEED_ITEMS</code>	۱۰۶ شیء JSON یک‌خطی
<code>seed-enrich.js</code>	<code>BLENDS</code> , <code>STORIES</code> , <code>TASTE_TAGS</code>	متن معرفی + ترکیب میکس

`seed-items.js` عمداً یک‌خطی نوشته شده؛ کامنت بالایش می‌گوید:

این فایل به‌صورت خودکار از نسخهٔ اولیهٔ سایت ساخته شده و فقط برای پر کردن اولیهٔ پایگاه داده است. بعد از `seed`، منبع حقیقت پایگاه داده و پزل مدیریت است، نه این فایل.

هر دو فایل `seed` در `.prettierrignore` هستند: هر ردیف عمداً یک خط بلند است تا کاتالوگ در یک نگاه خوانده شود، و شکستنشان خوانایی را کم می‌کند.

قبلاً اینجا بود. حالا در `shared/taxonomy.js` است تا کلاینت هم بتواند همان کلیدها را import کند – بخش بعدی.

بخش الف-۲ – بسته مشترک

shared/package.json

بسته خصوصی @ghahve/shared با دقیقاً سه export و بدون هیچ وابستگی:

```
{
  "name": "@ghahve/shared",
  "private": true,
  "type": "module",
  "exports": {
    "./pricing.js": "./pricing.js",
    "./taxonomy.js": "./taxonomy.js",
    "./seo.js": "./seo.js"
  }
}
```

exports صریح یعنی هیچ‌کس نمی‌تواند به فایل سومی از این بسته دست ببرد؛ هر چیزی که اضافه شود باید عمداً اینجا اعلام شود.

چون در workspaces ریشه ثبت شده، npm خودش در node_modules پیوندش می‌کند و هر دو طرف با همان نام import می‌کنند – بدون مسیر نسبی، بدون symlink دستی، بدون مرحله build.

shared/pricing.js – تنها نسخه محاسبه قیمت

صادرات: unitPriceFor, blendPricePerKg, priceFor, FREE_SHIPPING_FROM, SHIPPING, TIERS, lineTotal, computeTotals

تحلیل کامل فرمول‌ها در فصل ۷. آنچه اینجا اهمیت دارد جای فایل است. کامنت بالایش داستان را می‌گوید:

تا پیش از این دو نسخه یکسان از این فایل وجود داشت، یکی در client و یکی در server، که باید دستی همگام می‌ماندند. خطرش این بود که کسی پله‌های تخفیف را در یکی عوض کند و عددی که مشتری می‌بیند با عددی که ثبت می‌شود فرق کند – بی هیچ خطایی.

و بلافاصله یادآوری می‌کند که این جایگزین بازمحاسبه سمت سرور نیست:

سرور همچنان قیمت را از نو حساب می‌کند و به عدد ارسالی از مرورگر اعتماد نمی‌کند. یکی بودن فرمول جای آن بازمحاسبه را نمی‌گیرد؛ فقط تضمین می‌کند نتیجه با چیزی که کاربر دیده یکی دربیاید.

جدول TIERS با // prettier-ignore علامت خورده و اعشارها کامل نوشته شده‌اند (0.10 نه 0.1) تا ستون‌به‌ستون خوانده شود – این یک جدول است، نه چند سطر کد.

shared/taxonomy.js – تنها نسخه کلیدهای مجاز

صادرات: POWDER_SHAPES (۱۴)، GEAR_MATERIALS (۱۱)، GEAR_SHAPES (۳۶)، GROUPS_BY_KIND, KINDS, POWDER_TONES (۱۶)، IS_WEIGHED, DEFAULT_GRINDS (۷)، TASTE_KEYS (۸).

مدل Item با همین‌ها اعتبارسنجی می‌شود تا هیچ‌وقت کالایی ذخیره نشود که طرح تصویرش وجود ندارد، و برچسب فارسی هر کلید را کنارش می‌گذارد. web/src/lib/groups.js

تقسیم کار عمدی است: **کلید** اینجاست چون هر دو طرف باید یکی ببینندش؛ **برچسب فارسی** آنجاست چون فقط رابط کاربری به آن نیاز دارد و سرور کاری با متن ندارد. `tests/taxonomy.test.js` می‌سنجد که جدول برچسب‌ها دقیقاً همین کلیدها را پوشش بدهد – نه یکی کم، نه یکی زیاد.

shared/seo.js – آدرس، متن صفحه و داده‌های ساختاریافته

سومین فایل این بسته و تازه‌ترین‌شان (موردهای ۲۷ تا ۲۹). چهار دسته چیز دارد و همه‌شان دو مصرف‌کننده:

دسته	صادرات	وب	سرور
آدرس کالا	<code>itemPath</code> , <code>KIND_SEGMENTS</code> , <code>SEGMENT_KIND</code> , <code>KIND_SEGMENT</code> , <code>normalizeBaseUrl</code> , <code>absoluteUrl</code> , <code>itemUrl</code>	مسیر صفحه‌ها و لینک کارتها	لینک‌های <code>sitemap.xml</code>
متن صفحه	<code>SITE_LOCALE</code> , <code>SITE_DIR</code> , <code>SITE_LANG</code> , <code>SITE_NAME</code> , <code>itemTitle</code> , <code>HOME_DESCRIPTION</code> , <code>HOME_TITLE</code> , <code>clamp</code> , <code>itemImage</code> , <code>itemDescription</code>	<code>generateMeta</code> هر صفحه <code>data</code>	–
توصیف <code><head></code>	<code>headTags</code> , <code>notFoundHeadState</code> , <code>itemHeadState</code> , <code>identityOf</code>	ورودی <code>lib/metadata.js</code>	–
ماشین‌خوان	<code>jsonLdText</code> , <code>organizationJsonLd</code> , <code>productJsonLd</code> , <code>itemSitemapEntries</code> , <code>buildRobots</code> , <code>buildSitemap</code> , <code>lastmodOf</code> , <code>escapeXml</code>	JSON-LD صفحه‌ها	<code>routes/seo.js</code>

چرا اینجا و نه در web؟ چون دو مصرف‌کننده دارد و باید دقیقاً یک آدرس بسازند. کامنت بالای فایل می‌گوید اگر دو نسخه می‌شد، «روزی که بخش قهوه از `/coffee` به `/beans` می‌رفت، نقشه سایت بی‌سروصدا به صفحه‌های ۴۰۴ اشاره می‌کرد».

و چرا متن فارسی دارد، برخلاف قاعده ۶ پروژه؟ این تنها استثناست و خودش مستند شده: عنوان و توضیح صفحه کالا هم در متادیتای Next لازم است و هم – تا وقتی نقشه سایت را Express می‌سازد – در سرور. دو نسخه شدنشان یعنی چیزی که در تلگرام دیده می‌شود با تب مرورگر فرق کند. مرز دقیق است: **محتوای صفحه اینجا، برچسب رابط کاربری در `groups.js`**.

دو نکته ریز که ارزش دیدن دارند:

واحد پول در JSON-LD. قیمت‌های پایگاه داده به تومان‌اند، ولی `schema.org` کد ISO 4217 می‌خواهد و کد رسمی ایران ریال است. پس:

```
export const CURRENCY = 'IRR';
export const TOMAN_TO_RIAL = 10;
export const toRial = (toman) => Math.round(Number(toman) || 0) * TOMAN_TO_RIAL;
```

«IRT» کد استاندارد نیست و گوگل ردش می‌کند. و برای کالای وزنی یک `UnitPriceSpecification` با `unitCode: 'KGM'` اضافه می‌شود، وگرنه گوگل قیمت هر کیلو را قیمت یک بسته می‌فهمید.

بی‌خطر کردن JSON پیش از رفتن داخل `<script>`:

```
export function jsonLdText(value) {
  return JSON.stringify(value)
    .replace(/</g, '\\u003c').replace(/>/g, '\\u003e').replace(/&/g, '\\u0026')
    .replace(/\\u2028/g, '\\u2028').replace(/\\u2029/g, '\\u2029');
}
```

نام کالا را مدیر می‌نویسد. با این تبدیل، نامی مثل `</script><script>` هیچ پارسر HTML ای را گول نمی‌زند، ولی `JSON.parse` همان مقدار را می‌خواند. `U+2028` و `U+2029` هم بسته می‌شوند: در JSON مجازند ولی در جاوااسکریپت پایان خط حساب می‌شوند.

نگهبان‌هایش: `tests/item-routes.test.js`، `tests/json-ld.test.js`، `tests/sitemap.test.js` و `tests/next-metadata.test.js`.

بخش ب – وب: زیرساخت

– پوسته همه صفحه‌ها web/src/app/layout.jsx

جانشین client/index.html و client/src/main.jsx. چهار کار می‌کند:

```
export default function RootLayout({ children, modal }) {
  return (
    <html lang={SITE_LANG} dir={SITE_DIR} className={` ${vazirmatn.variable} ${lalezar.variable}`} >
      <body>
        {children}
        {modal}
        <Toast />
      </body>
    </html>
  );
}
```

یک (زبان، جهت و فونت). `lang` و `dir` از `shared/seo.js` می‌آیند، نه ثابت نوشته شده. کلاس‌های فونت متغیرهای `--font-vazirmatn` و `--font-lalezar` را روی `<html>` می‌گذارند و استایل‌نامه از همان‌ها می‌خواند – چون `next/font` نام خانواده را خودش می‌سازد.

(دو) شکاف دوم صفحه. `modal` یک مسیر موازی است (`app/@modal`). در حال عادی خالی است؛ با کلیک روی کارتی از داخل سایت، مسیر رهگیری شده آن را پر می‌کند و `children` – یعنی صفحه زیر – دست‌نخورده می‌ماند. این جانشین `state.backgroundLocation` در نسخه react-router است.

(سه) یک توست، برای هر دو شکاف. `<Toast />` یک بار و بالای هر دو شکاف رندر می‌شود. جایش عمداً همین‌جاست: پیام‌ها از یک انبار مازولی می‌آیند نه از کانتکست، پس هم صفحه و هم مودال رهگیری شده می‌توانند پیام بدهند و هر دو همان یک نوار پایین صفحه را روشن می‌کنند. تا پیش از این چند خط داخل `HomeShell` بود، یعنی فقط صفحه اصلی توست داشت و صفحه کالا و مودالش هر دو بی‌صدا بودند.

چهار) پیش‌فرض `<head>`. یک `metadata` صادر می‌کند که عنوان و توضیح و Open Graph پیش‌فرض را می‌دهد. برخلاف `index.html` قدیم، این پیش‌فرض واقعاً پیش‌فرض است: هر مسیری که `generateMetadata` خودش را داشته باشد جایش را می‌گیرد و Next تضمین می‌کند تگ دوباره ساخته نشود. به همین دلیل `client/src/lib/head.js` – با آن پشته سه‌حالت و صفت‌های `data-head-*` – دیگر وجود ندارد.

و یک تصمیم که جایش همین‌جاست:

```
export const dynamic = 'force-dynamic';
```

سیاست امنیتی صفحه‌ها با `nonce` کار می‌کند و `nonce` برای هر درخواست تازه ساخته می‌شود؛ صفحه‌ای که موقع `build` ساخته شده باشد آن را ندارد و اسکریپت‌هایش بسته می‌شوند. پس بین «کش ایستا» و «سیاست سخت‌گیرانه»، دومی انتخاب شد. دلیل کامل داخل خود فایل نوشته شده.

هر دو فایل CSS همین‌جا `import` می‌شوند. بازدیدکننده فروشگاه هنوز CSS پل را هم می‌گیرد – همان ضعف کوچک قبلی، که با کد اسپلیت (مورد ۲۰) قابل رفع است.

web/src/app/ — جدول مسیرها به جای App.jsx

در نسخهٔ react-router یک فایل (App.jsx) همهٔ مسیرها را در یک <Routes> فهرست می‌کرد. حالا ساختار پوشه‌ها همان جدول است:

آدرس	پوشه
/	app/page.jsx
/coffee/:slug و همتاها	app/coffee/[slug]/ (سه پوشه، یک پیاده‌سازی در app/_item/)
همان آدرس‌ها، به شکل مودال	app/@modal/(.)coffee/[slug]/
/track	app/track/
/admin/login	app/admin/login/
هشت صفحهٔ پنل	app/admin/(panel)/*
آدرس ناشناخته	app/not-found.jsx ۴۰۴ →

سه تفاوت که ارزش گفتن دارند:

۱) **Provider ها جای ثابتی ندارند.** پیش‌تر AuthProvider و ShopProvider دور کل جدول مسیرها می‌پیچیدند. حالا هرکس آن‌جا که لازم است گذاشته می‌شود: ShopProvider در app/page.jsx (با کالاهای خوانندهٔ سرور)، در صفحهٔ کالا (با فهرست کوچک‌تر)، و در پنل (با loadOnMount)؛ AuthProvider فقط در app/admin/layout.jsx. نتیجه‌اش این است که صفحهٔ پیگیری سفارش هیچ‌کدام را حمل نمی‌کند.

۲) آدرس ناشناخته دیگر بی‌صدا به خانه نمی‌رود. سطر آخر جدول قبلی <Route path="*" element={<Navigate to="/" replace /> /> بود که پاسخ ۲۰۰ می‌داد — از دید موتور جست‌وجو یعنی «این آدرس وجود دارد». حالا ۴۰۴ واقعی است.

۳) /track هنوز تنها صفحهٔ عمومی دوم است و همچنان بیرون از دروازهٔ پنل می‌نشیند — همان تقسیم قبلی، این بار با پوشه.

web/next.config.mjs — پراکسی، بستهٔ مشترک، و بهینه‌ساز تصویر

پنج بند، و هر پنج لازم:

```
outputFileTracingRoot: path.join(__dirname, '..'), // پروژه workspace است
transpilePackages: ['@ghahve/shared'], // بستهٔ محلی ترنسپایل‌نشده
images: { /* AVIF/WebP · localPatterns · imageSizes · minimumCacheTTL */ }
async headers() { /* next/image/ روی دو هدر */ }
async rewrites() { /* /api · /uploads · /sitemap.xml · /robots.txt → Express */ }
```

rewrites جانشین بند server.proxy در vite.config.js است و همان نتیجه را می‌دهد: مرورگر همچنان آدرس نسبی می‌زند، پس lib/api.js دست‌نخورده منتقل شد و مسئلهٔ CORS اصلاً پیش نمی‌آید.

هم عمداً در همین فهرست‌اند — همان‌جا از فهرست زندهٔ کالاها ساخته می‌شوند و دلیلی برای دو نسخه شدنشان نیست. robots.txt و sitemap.xml

```
images: {
  formats: ['image/avif', 'image/webp'],
  localPatterns: [{ pathname: '/uploads/**' }, { pathname: '/img/**' }],
  imageSizes: [32, 48, 64, 96, 128, 256, 384, 448],
  minimumCacheTTL: 604800
}
```

بند	چرا این مقدار
formats	به ترتیب اولویت: مرورگری که AVIF بفهمد AVIF می‌گیرد، وگرنه WebP، وگرنه همان قالب اصلی
localPatterns	قفلِ عمدی. بدون آن هر مسیر محلی‌ای بهینه‌شدنی است و <code>/_next/image</code> عملاً یک واکنش عمومی می‌شود. این دو تنها جاهایی‌اند که تصویر واقعی از آن‌ها می‌آید
imageSizes	448 به فهرست پیش‌فرض اضافه شده: کارت فهرست روی دسکتاپ ۳۸۵ و روی تبلت تا ۴۳۱ پیکسل عرض می‌گیرد، و با فهرست پیش‌فرض نزدیک‌ترین نسخه بزرگ‌تر برای هر دو ۶۴۰ بود
minimumCacheTTL	هفت روز — هم‌تراز با <code>maxAge</code> پوشه <code>uploads</code> در سرور. نام فایل آپلودی تصادفی است و هیچ‌وقت بازنویسی نمی‌شود، پس کش طولانی بی‌خطر است

مسیر داخلی، نه بیرونی. بهینه‌ساز برای آدرس‌های محلی درخواست را از همین مسیریاب Next رد می‌کند، یعنی از همان `rewrites` پایین‌تر. پس `/uploads/x.jpg` به Express می‌رسد بدون اینکه آدرس داخلی Express به مرورگر درز کند و بدون اینکه `remotePatterns` لازم شود.

و آنچه عمداً دست نخورد: `dangerouslyAllowSVG` خاموش ماند (مورد ۵: SVG یک سند اجرashدنی است)، و `contentTypeDisposition` و `contentSecurityPolicy` پیش‌فرض Next اند که همان سیاست `sandbox` پوشه `uploads` را می‌دهند. کد `lib/img.js` مطمئن می‌شود هیچ SVG ای اصلاً به اینجا نرسد.

بند headers() — دو محافظ که Next نمی‌گذارد

```
async headers() {
  return [{
    source: '/_next/image',
    headers: [
      { key: 'x-content-type-options', value: 'nosniff' },
      { key: 'cross-origin-resource-policy', value: 'same-origin' },
      { key: 'x-frame-options', value: 'DENY' }
    ]
  }];
}
```

تصویرهای `/uploads` دو لایه محافظ داشتند (پای `setUploadHeaders` در `server/src/lib/upload.js`). حالا مرورگر دیگر مستقیم آن آدرس را نمی‌خواند و پاسخ را از `/_next/image` می‌گیرد — پس هدرهای Express روی آن پاسخ نیستند.

خود Next دوتای مهم را می‌گذارد: همان `script-src 'none'; frame-src 'none'; sandbox` و `Content-Disposition: attachment`. سه‌تای بالا را نمی‌گذارد و اینجا اضافه شده‌اند تا سیاست هر دو راه یکی بماند.

این کار از `proxy.js` برنمی‌آمد: آن میان‌افزار عمداً `_next/image` را رد می‌کند (مثل بقیه دارایی‌های `_next`)، چون CSP با `nonce` مال صفحه‌هاست نه فایل‌های ایستا.

سیاست امنیتی صفحه‌ها — `web/src/proxy.js`

نامش در ۱۶ Next `proxy` است؛ همان چیزی که پیش‌تر `middleware` بود. برای هر درخواست یک `nonce` تازه می‌سازد و آن را هم روی درخواست می‌گذارد (تا Next روی اسکریپت‌های خودش بنشاندش) و هم روی پاسخ:

```
const nonce = Buffer.from(crypto.randomUUID()).toString('base64');
requestHeaders.set('x-nonce', nonce);
response.headers.set('content-security-policy', policyFor(nonce));
```

چرا اصلاً لازم شد؟ سیاست پیشین می‌گفت `script-src 'self'` و برای یک باندل ویت کافی بود، چون هیچ اسکریپت درون خطی نداشت. Next اما داده رندر سمت سرور را به شکل چند `<script>` درون خطی داخل صفحه می‌گذارد و همان سیاست دقیقاً آن‌ها را می‌بست — یعنی صفحه رندر می‌شد ولی هیچ‌وقت زنده نمی‌شد.

راه ساده `'unsafe-inline'` بود که عملاً کل ارزش سیاست را دور می‌ریخت. به‌جایش `nonce`، به‌علاوه `'strict-dynamic'` تا باندل‌هایی که خودشان اسکریپت می‌سازند اعتبار را به ارث ببرند.

سه چیز عمداً باز مانده و هر سه کامنت دارند:

بند	چرا باز است
<code>style-src 'unsafe-inline'</code>	صفت <code>style</code> با <code>nonce</code> کار نمی‌کند؛ نوار نمودار گزارش و متغیرهای CSS حالت تصویری میکس هر دو درون خطی‌اند
<code>img-src data:</code>	تصویر تولیدی SVG کارت‌ها و بافت نویز <code>style.css</code>
<code>'unsafe-eval'</code> و <code>ws:</code>	فقط در توسعه — بازآوری داغ Next بی این‌ها کار نمی‌کند

و دو چیز نسبت به سیاست قدیم Express بسته‌تر شد: `font-src` دیگر `fonts.gstatic.com` را ندارد و `style-src` دیگر `fonts.googleapis.com` را — چون فونت‌ها با `next/font` روی همین دامنه‌اند (مورد ۲۵).

`matcher` مسیرهایی را که Express جواب می‌دهد یا فایل ایستا هستند کنار می‌گذارد: نه `nonce` لازم دارند نه سیاست.

فونت روی همین دامنه — `web/src/app/fonts.js`

فایل فونت را موقع `build` می‌گیرد و کنار بقیه دارایی‌ها می‌گذارد؛ در زمان اجرا هیچ درخواستی به بیرون نمی‌رود. برای مخاطب ایرانی این فقط یک بهینه‌سازی نیست: `fonts.googleapis.com` برای بسیاری کند یا بسته است و صفحه تا تایم‌اوت شدن درخواست با فونت جایگزین دیده می‌شد.

```
export const vazirmatn = Vazirmatn({ subsets: ['arabic', 'latin'], variable: '--font-vazirmatn', display: 'swap' });
export const laleazar = Lalezar({ subsets: ['arabic', 'latin'], weight: '400', variable: '--font-laleazar', display: 'swap' });
```

وزیرمتن روی گوگل‌فونتس فونت متغیر است، پس وزن جداگانه نمی‌گیرد — همان یک فایل همه وزن‌های ۳۰۰ تا ۹۰۰ را پوشش می‌دهد. لاله‌زار متغیر نیست و وزنش اجباری است.

نام خانواده‌ای که Next می‌سازد هش‌دار است، پس هیچ‌جای استایل‌نامه 'Vazirmatn' نوشته نمی‌شود؛ همه‌چیز از راه متغیر می‌آید. جزئیاتش در فصل ۸.

هشدار

`npm run build` به شبکه نیاز دارد، چون همان موقع فایل فونت‌ها گرفته می‌شود. در محیط بی‌اینترنت، build شکست می‌خورد – نه اجرا.

خواندن داده روی سرور — `web/src/lib/data.js`

دو فایل شبکه داریم و این عمدی است: `api.js` برای مرورگر، `data.js` برای سرور. دو فایل، چون دو محیط – نه چون دو نسخه‌اند.

```
const API_URL = process.env.API_URL || 'http://localhost:4000';
const FETCH_OPTS = { cache: 'no-store' };

export const getItem = cache(async (kind, slug) => { /* null → ۴۰۴ */ });
export const getItems = cache(async (kind = '') => { /* فهرست روشن‌ها */ });
export const getContent = cache(async () => { /* خطا */ });
```

سه تصمیم:

۱) **آدرس مطلق.** `fetch` روی سرور «مبدأ صفحه» ندارد؛ `/api/items` برایش معنایی ندارد. این فایل هیچ‌وقت در مرورگر اجرا نمی‌شود، پس نشأت آدرس داخلی هم ممکن نیست.

۲) `cache` از ری‌اکت. `generateMetadata` و خود صفحه هر دو همان کالا را می‌خواهند؛ بی‌این، هر بازدید از صفحه کالا دو درخواست به Express می‌زد.

۳) `no-store`. داده‌ها زنده‌اند: مدیر هر لحظه ممکن است کالایی را خاموش یا موجودی را کم کند. نسخه کش‌شده یعنی فروختن چیزی که نداریم.

و یک تفاوت رفتاری که ارزش گفتن دارد: نبود کالاهای خطا می‌دهد، نبود متن‌ها نه. اگر Express خاموش باشد باید خطا بدهیم نه ۴۰۴ – «کالا پیدا نشد» درباره کالایی که شاید هست، هم به کاربر دروغ می‌گوید و هم به موتور جست‌وجو. ولی `getContent` در خطا `{}` می‌دهد: بخش‌های متنی رندر نمی‌شوند و فروشگاه کار می‌کند – همان قاعده `api.content().catch(() => ({}))` که در نسخه ویت بود.

`web/src/lib/metadata.js` — از توصیف `<head>` به شیء `metadata`

`toNextMetadata(state)` خروجی `itemHeadState` یا `notFoundHeadState` را به شکلی که Next می‌فهمد ترجمه می‌کند. قراردادش صریح است: خروجی باید دقیقاً همان مقادیری را داشته باشد که `headTags(state)` می‌ساخت – و `tests/next-metadata.test.js` تگ‌به‌تگ همین را می‌سنجد.

پیش از مورد ۲۶، همان توصیف دو مصرف‌کننده داشت که باید مو به مو یکی می‌مانند: یکی در مرورگر که به عنصر DOM تبدیلش می‌کرد و یکی در سرور که رشته HTML می‌ساخت. حالا یک رندرکننده بیشتر نیست: خود Next.

و یک استثنا: `og:type`.

```
export const NEXT_OG_TYPES = new Set(['website', 'article', 'book', 'profile', *.video و *.music /*
*/]);

export const ogTypeFallback = (state = {}) =>
  state.ogType && !NEXT_OG_TYPES.has(state.ogType) ? state.ogType : '';
```

صفحه کالا `og:type: product` می‌خواهد — همان چیزی که تلگرام و فیس‌بوک برای کارت کالا می‌خوانند. Next فهرست بسته خودش را دارد و برای هر چیز دیگری موقع رندر خطا می‌دهد؛ یعنی کل `<head>` خالی می‌ماند، نه اینکه فقط یک تگ بیفتد. اولین بار همین شد: صفحه کالا بدنه درست داشت و هیچ عنوانی نداشت.

دو راه دیگر بررسی و رد شدند:

- `metadata.other` → تگ را با `name` می‌نویسد نه `property`، و خزنده Open Graph سراغ `property` می‌رود. تگی که هست ولی خوانده نمی‌شود.
- عوض کردنش به `'website'` → کالا دیگر کالا نیست؛ عقب‌گرد واقعی.

پس همان یک تگ را خود صفحه رندر می‌کند و ری‌اکت ۱۹ به `<head>` می‌بردش. اگر روزی Next فهرستش را باز کند، فقط همین مجموعه بزرگ می‌شود و آن یک خط از صفحه برداشته می‌شود.

web/src/lib/site.js — آدرس پایه و تصویر پیش‌نمایش

سه تابع کوچک روی متغیرهای محیطی: `siteBaseUrl`، `siteOgImage` و `metadataBase`. نکته مهمشان این است که روی سرور هم صدا زده می‌شوند، جایی که `window` وجود ندارد:

```
export function siteBaseUrl(env = process.env) {
  const fromEnv = normalizeBaseUrl(env.NEXT_PUBLIC_SITE_URL);
  if (fromEnv) return fromEnv;
  return typeof window === 'undefined' ? '' : window.location.origin;
}
```

در نبود متغیر محیطی، به جای حدس زدن رشته خالی برمی‌گردد و Next مسیر نسبی می‌سازد — «آدرس نسبی بهتر از آدرس غلط است». فقط `metadataBase` استثناست: Next آنجا یک URL کامل می‌خواهد، وگرنه هشدار می‌دهد؛ پس در توسعه `http://localhost:3000` گذاشته می‌شود.

web/src/lib/cartStorage.js — سبد، بیرون از کامپوننت

`browserStorage()` و `saveCart(storage, cart)`، `loadCart(storage)`، `parseCart` است؛ آنچه اینجا اهمیت دارد این است که حافظه پارامتر است، نه `localStorage` سراسری — همان کاری که `reserveStock` در سرور با مدل مونگوس می‌کند. نتیجه‌اش این است که `tests/cart-storage.test.js` بدون مرورگر و بدون ری‌اکت اجرا می‌شود.

web/src/lib/seo.js — پل JSON-LD

آنچه در `shared/seo.js` تابع خالص است، اینجا با برچسب‌های فارسی `groups.js` و آدرس پایه `site.js` پر می‌شود: `productSchema(item)`، `organizationSchema(content)`، `canonicalUrl(path)` و `asJsonLd(obj)`.

تقسیمش همان مرز همیشگی است: `shared` نمی‌داند برچسب فارسی دسته کالا چیست و نباید بداند؛ پس `category` و `brandName` و `soldOut` را از اینجا پارامتر می‌گیرد.

web/src/lib/share.js — نشانی، کپی، و اینکه کدام راه

۱۴۸ خط، چهار تابع صادرشده و دو پیام. همهٔ منطقِ دکمهٔ «برای دوستت بفرست» اینجاست و هیچ‌کدامش در کامپوننت نیست — چون تابع خالص تست می‌شود و کامپوننت نه.

یک نشانی از `canonical` می‌آید، نه از رشته‌بافی دستی:

```
export function itemShareUrl(item, env) {
  return itemHeadState(item, { baseUrl: siteBaseUrl(env) }).canonical;
}
```

این یک خط، همان قاعدهٔ ۴ پروژه است. لینکی که مشتری برای دوستش می‌فرستد باید دقیقاً همان آدرسی باشد که در `canonical` و `og:url` صفحه نوشته شده؛ وگرنه پیش‌نمایش تلگرام و واتساپ به یک آدرس نگاه می‌کند و لینک به آدرسی دیگر می‌رود. اگر کسی روزی `/coffee/' + item.slug` بنویسد، روز عوض‌شدن مسیرها یا آدرس پایه لینک‌های فرستاده‌شده به ۴۰۴ می‌رسند و هیچ چیزی خبردار نمی‌شود. `tests/share.test.js` صریحاً خروجی این تابع را با `itemHeadState(...).canonical` و با `itemUrl(...)` — همان که `sitemap.xml` می‌سازد — می‌سنجد.

دو ملاکِ دوراهی: نوعِ نشانگر، نه بودنِ `navigator.share`:

```
export function isTouchDevice(opts = {}) {
  const { nav = globalThis.navigator, win = globalThis } = opts;

  const mq = win?.matchMedia?.('(pointer: coarse)');
  if (typeof mq?.matches === 'boolean') return mq.matches;

  return Number(nav?.maxTouchPoints) > 0;
}
```

`(pointer: coarse)` یعنی نشانگر اصلیِ دستگاه انگشت است، نه ماوس. لپ‌تاپ لمسی که ماوس هم دارد `fine` گزارش می‌دهد و درست هم همین است. `maxTouchPoints` فقط تکیه‌گاه آخر است، برای مرورگر بی `matchMedia`؛ به‌تنهایی ملاک بدی است چون هر نمایشگر لمسی وصل به دسکتاپ آن را بالای صفر می‌کند. چرایی کامل این انتخاب در تصمیم ۲۸ فصل ۹ است.

سه (کپی، با یک راه دوم:

```
export async function copyText(text, opts = {}) {
  const { nav = globalThis.navigator, doc = globalThis.document } = opts;

  if (nav?.clipboard?.writeText) {
    try {
      await nav.clipboard.writeText(text);
      return true;
    } catch {
      /* رد شدن اجازه یا زمینه ناامن — راه دوم را امتحان کن */
    }
  }
  ...
}
```

`navigator.clipboard` فقط در «زمینه امن» (https یا localhost) هست. فروشگاهی که روی http ساده بالا آمده باشد اصلاً چنین چیزی ندارد، و آنجا همان ترفند قدیمی — `textarea` موقت و `execCommand('copy')` — تنها راهی است که کار می‌کند. منسوخ است، ولی جایگزین کارکننده‌ای ندارد و بی‌خطر است: اگر هم نبود، فقط `false` برمی‌گرداند. `textarea` عمداً با `opacity: 0` و بیرون قاب پنهان می‌شود و نه با `display: none` — عنصری که رندر نشود انتخاب هم نمی‌شود.

چهار) `shareItem` که چهار پایان دارد، نه دو تا:

mode	یعنی	چه چیزی به مشتری گفته می‌شود
'shared'	برگه سیستم باز شد و کاربر فرستاد	هیچ — خود برگه بازخورد داده
'canceled'	کاربر برگه را بست	هیچ — نظرش عوض شده
'copied'	لینک در کلیپ‌بورد است	توست تأیید
'failed'	هیچ‌کدام نشد	توست راهنمایی

جدا کردن `canceled` از `failed` ریز به‌نظر می‌رسد ولی نیست: بستن برگه یک `AbortError` می‌دهد و اگر آن را شکست حساب می‌آوریم، هر بار که کسی نظرش عوض می‌شد یک پیام خطا می‌گرفت.

```
if (typeof nav?.share === 'function' && isTouchDevice({ nav, win })) {
  try {
    await nav.share({ title, url });
    return { mode: 'shared', url };
  } catch (err) {
    if (err?.name === 'AbortError') return { mode: 'canceled', url };
  }
}

return { mode: (await copyText(url, { nav, doc })) ? 'copied' : 'failed', url };
```

خطای غیر از `AbortError` عمداً از `try` بیرون می‌افتد و به همان مسیر کپی می‌رسد — یعنی اگر `share` از کار افتاده باشد، دست مشتری خالی نمی‌ماند.

و پنج) همه‌چیز تزریق‌پذیر است. `nav`، `doc`، `win` و `env` هر چهار پارامترند و در عمل هیچ‌کس آن‌ها را نمی‌دهد. همان قاعده `reserveStock` در سرور، این بار برای مرورگر: `tests/share.test.js` بدون jsdom اجرا می‌شود.

پیام کوتاه، بیرون از هر کانتکست — `web/src/lib/toastStore.js`

۸۰ خط. یک انبار مازولی ساده با سه تکه حالت و چهار تابع:


```
export const TOAST_LIFETIME = 4500;

let message = '';
let timer = null;
const listeners = new Set();

export function toast(msg) {
  message = String(msg ?? '');
  clearTimeout(timer);
  timer = setTimeout(() => { message = ''; timer = null; emit(); }, TOAST_LIFETIME);
  emit();
}

export function subscribeToast(listener) {
  listeners.add(listener);
  return () => listeners.delete(listener);
}

export const getToast = () => message;
export const getServerToast = () => '';
```

چرا از **ShopContext** بیرون آمد. توست هیچ وقت واقعاً حالت فروشگاه نبود؛ فقط آنجا زندگی می کرد چون اولین صداکننده اش سبد بود. مرز غلط، و مودال کالا رویش گیر کرد: مودال در شکاف **@modal** رندر می شود، یعنی بیرون از **ShopProvider** صفحه. هر چیزی داخل مودال که بخواهد «شد» بگوید، به چیزی نیاز دارد که به Provider وابسته نباشد. شرح کاملش در تصمیم ۲۶ فصل ۹.

سه ریزه کاری که هر کدام یک اشکال نادیدنی را می بندند:

۱) **clearTimeout** پیش از **setTimeout**. بدون آن، پیام دومی که پشت پیام اول بیاید شمارش اولی را به ارث می برد و زودتر از موعد می رود. یک تست دقیقاً همین را می سنجد.

۲) **getServerToast** که همیشه خالی است. متغیرهای این مازول روی سرور بین درخواست ها مشترک اند. **toast()** فقط از دل رویدادهای مرورگر صدا زده می شود پس در عمل پر نمی شوند، ولی snapshot جدا تضمین می کند که حتی در بدترین حالت هم پیام یک بازدیدکننده داخل HTML بازدیدکننده بعدی ننشیند – و **hydrate** هم شکایتی نداشته باشد.

۳) عمر پیام ۴۵۰۰ میلی ثانیه است، و این عدد عوض شد. ۲,۶۰۰ بود، از روزی که تنها پیام ها کوتاه بودند («به سبد اضافه شد»). بلندترین پیام حالا پیام هم رسانی است – «لینک این کالا کپی شد – حالا می توانید بفرستیدش»، نزدیک به پنجاه نویسه. و آن ۲.۶ ثانیه همه اش خواندنی نبود: ۰.۳ ثانیه اولش محو ورود است (**transition: opacity 0.3s**) روی **.toast** در **style.css**، پس عملاً ۲.۳ ثانیه می ماند برای جمله ای که چشم فارسی خوان دست کم سه ثانیه لازمش دارد.

نکته

عدد در همین مازول است، نه در کامپوننت – چون هم **Toast.jsx** و هم **tests/toast-store.test.js** باید از یک جا بخوانندش. تست با **vi.useFakeTimers()** روی همین ثابت جلو می رود، پس عوض کردن عدد تست را نمی شکند.

— کدام تصویر بهینه می شود **web/src/lib/img.js**

۳۹ خط، دو تابع. کوچک ترین فایل **lib/** و در عین حال نگهبان یک اشکال پرهزینه: SVG ای که به بهینه ساز Next برسد قاب خالی می دهد، نه عکس بهینه نشده.

```
const VECTOR = /\.svgz?(?:[?#]|$)/i;

export function isVector(src) {
  return VECTOR.test(String(src || ''));
}

export function isOptimizable(src) {
  const s = String(src || '');
  return s.startsWith('/') && !s.startsWith('//') && !isVector(s);
}
```

سه شرط `isOptimizable` هر کدام یک چیز را بیرون می‌گذارند:

شرط	چه چیزی را رد می‌کند	چرا
<code>startsWith('/')</code>	آدرس مطلق بیرونی	بهینه‌ساز برای میزبان ثبت‌نشده در <code>remotePatterns</code> پاسخ ۴۰۰ می‌دهد
<code>!startsWith('//')</code>	آدرس بدون پروتکل (<code>//cdn...</code>)	همان مورد بالا، با ظاهر مسیر محلی
<code>!isVector(s)</code>	SVG و SVGZ	<code>dangerouslyAllowSVG</code> خاموش است (مورد ۵)

الگوی `VECTOR` عمداً `[?#]` را هم می‌پذیرد: آدرس تصویر بخش «درباره ما» را مدیر دستی می‌نویسد و ممکن است `?v=2` هم داشته باشد.

دو صداکننده دارد — `CardArt.jsx` و بخش «درباره ما» — و همین دلیل وجودش است: قاعده یک‌جا باشد، نه دو تا.

— جانشین `reload()` در پنل `web/src/lib/useStorefrontRefresh.js`

سه صفحه پنل (کالاها، فرم کالا، محتوای سایت) بعد از هر تغییر باید فروشگاه را تازه کنند. در نسخه ویت این `useShop().reload()` بود، چون `ShopContext` فهرست کالاها را یک بار می‌گرفت و در حافظه نگه می‌داشت.

در Next آن حافظه وجود ندارد — هر صفحه فروشگاه در هر درخواست از نو ساخته می‌شود — ولی یک کش دیگر هست: کش مسیریاب سمت مشتری. اگر مدیر پیش از رفتن به پنل صفحه اصلی را دیده باشد، بازگشتش می‌تواند از همان نسخه کش‌شده بیاید.

```
export function useStorefrontRefresh() {
  const router = useRouter();
  return useCallback(() => router.refresh(), [router]);
}
```

شکل صدا زدن در آن سه فایل دست‌نخورده ماند و فقط موتورش عوض شد — و پنل به `ShopProvider` نیازی پیدا نکرد که فقط برای این کار باشد.

— یک پیاده‌سازی، سه پوشه `web/src/app/_item/`

پوشه با `_` شروع می‌شود، پس Next آن را مسیر حساب نمی‌کند. پنج فایل:

فایل	چیست
itemRoute.jsx	صفحه کامل کالا + itemMetadata(segment, props) – سروری
itemModalRoute.jsx	نیمه سروری مودال: فقط داده جمع می‌کند
ItemModal.jsx	نیمه مشتری: ItemDetail + router.back() برای بستن
ItemBuyCard.jsx	ستون خرید – تنها تکه مشتری صفحه کالا
ItemNotFound.jsx	بدنه و metadata صفحه ۴۰۴ کالا

و هر پوشه مسیر فقط چند خط است:

```
// app/coffee/[slug]/page.jsx
const SEGMENT = 'coffee';
export const generateMetadata = (props) => itemMetadata(SEGMENT, props);
export default function Page(props) { return <ItemRoute segment={SEGMENT} {...props} />; }
```

در نسخه ویت این سه مسیر با یک حلقه روی KIND_SEGMENTS ساخته می‌شدند، پس نوع تازه در taxonomy خودبه‌خود مسیر هم می‌گرفت. App Router برای هر مسیر یک پوشه واقعی می‌خواهد و چنین حلقه‌ای ممکن نیست – آنچه از دست رفت با tests/next-routes.test.js جبران شده: اگر segment پوشه نداشته باشد، تست می‌شکند.

سه نکته دیگر در itemRoute.jsx:

ItemNotFound metadata جدا دارد، با اینکه صفحه خودش generateMetadata دارد. دلیلش این است که وقتی notFound() صدا زده می‌شود، Next نتیجه generateMetadata همان مسیر را دور می‌ریزد و به metadata لایه بالاتر برمی‌گردد؛ بدون آن بلوک، صفحه ۴۰۴ عنوان و og:image صفحه اصلی را می‌گرفت. description و alternates هم صریحاً null اند، چون در Next کلیدی که نویسی از والد به ارث می‌رسد و «نوشتن» با «نداشتن» یکی نیست.

دانه‌های میکس فقط برای میکس‌ها خوانده می‌شوند. کالای معمولی یک درخواست می‌دهد و تمام؛ میکس یکی بیشتر، چون هم نام دانه‌ها در متن لازم است و هم قیمتشان برای میانگین وزنی.

فهرست عمداً ناقص است. صفحه فقط خود کالا (و دانه‌هایش) را به ShopProvider می‌دهد و fullCatalogue={false} می‌فرستد. دلیلش و خطرش در ۷.۱۲ و ۹ (تصمیم ۲۵) آمده.

web/src/app/@modal/ – مودال پشت یک آدرس

سه پوشه رهگیری‌شده (coffee/[slug]) و هم‌تاها) و یک default.jsx. مسیر موازی @modal در app/layout.jsx رندر می‌شود، کنار children.

کلیک از داخل سایت به این شکاف می‌رسد و صفحه زیر سر جایش می‌ماند؛ بازدید سرد یا رفرش به صفحه کامل می‌رود. همان دو رفتاری که در نسخه ویت با state.backgroundLocation دستی ساخته می‌شد.

default.jsx که فقط null برمی‌گرداند، لازم است: بدون آن Next برای بازدید سرد نمی‌داند شکاف را با چه چیزی پر کند و صفحه ۴۰۴ می‌شود.

و یک تصمیم که به سبد گره خورده: داده به مودال به شکل نگاشت ساده می‌رسد (beanNames, grindLabels)، نه از کانتکست. چون این مودال در شکاف layout ریشه رندر می‌شود، یعنی بیرون از ShopProvider ی که app/page.jsx می‌سازد. می‌شد یک Provider دوم اینجا هم گذاشت، ولی آن‌وقت دو سبد جدا می‌داشتیم و افزودن از یکی روی دیگری اثر نمی‌کرد – دامی که بعداً پیدا کردنش سخت است.

خود پیگیری نمی‌تواند سروری باشد: مشتری باید کد و موبایلش را بنویسد و آن جفت هیچ‌وقت در آدرس نمی‌نشیند (و نباید بنشیند). پس فرم و نتیجه سمت مشتری ماندند و آنچه سروری شد، تگ‌های `<head>` است — که برای `noindex` مهم است: رباتی که جاوااسکریپت اجرا نمی‌کند هم آن را می‌بیند.

`Suspense` لازم است چون `Track` از `useSearchParams` استفاده می‌کند (برای پیش‌پر کردن کد از لینک رسید) و `Next` می‌خواهد چنین تکه‌ای مرز `Suspense` خودش را داشته باشد.

دو کامپوننتی که از تقسیم `pages/Home.jsx` قدیمی به‌وجود آمدند.

`HomeShell` بدنه صفحه اصلی است و `'use client'` دارد — نه از سلیقه، بلکه چون فیلترهای دسته (`coffeeFilter`) و هم‌تاهایش) بالای چند بخش زندگی می‌کنند: کارت‌های «روش دم‌آوری» و «راهنمای رست» هم عوضشان می‌کنند.

این به معنای «سمت مشتری رندر شدن» نیست. کامپوننت مشتری هم روی سرور رندر می‌شود؛ متن همه این بخش‌ها داخل خود HTML می‌آید. تفاوت فقط این است که جاوااسکریپتشان هم برای مرورگر فرستاده می‌شود — همان چیزی که در نسخه ویت هم بود.

`SiteHeader` یک لایه نازک است: `Header` یک تابع می‌خواهد (`onOpenCart`) و تابع از مرز سروری به مشتری رد نمی‌شود — فقط داده قابل‌سرپال‌سازی رد می‌شود. پس صفحه سروری نمی‌تواند خودش `Header` را با کنترلر سبد رندر کند. همین یک جا صاحب حالت سبد است، پس هر صفحه‌ای که هدر دارد سبد هم دارد — صفحه اصلی و صفحه کالا رفتارشان یکی است.

متن معرفی کالا: «این قهوه از کجا می‌آید»، «در فنجان چه می‌چشید»، «پیشنهاد ما»، و برای میکس‌ها ترکیبش. یک کامپوننت سروری بدون هیچ هوکی، که هم مودال از آن استفاده می‌کند و هم صفحه کامل — پس آنچه گوگل می‌خواند و آنچه بازدیدکننده می‌بیند یکی است. `headingLevel` پارامتر دارد چون در صفحه کامل زیر `<h1>` می‌نشیند و در مودال نه.

بزرگ‌ترین مخزن منطق سمت کلاینت، و تنها جایی که سبد عوض می‌شود.

شکل ردیف از `lib/cartLine.js` می‌آید

`mixSignature` و `lineKey` قبلاً همین‌جا تعریف می‌شدند؛ حالا از `cartLine.js` می‌آیند و فقط دوباره صادر می‌شوند تا مصرف‌کننده‌های قدیمی نشکنند:

```
import { mixSignature, lineKey, buildLine } from '../lib/cartLine.js';
export { mixSignature, lineKey };
```

دلیل جابه‌جایی: میکس را حالا می‌شود از دو راه ساخت — اهرم‌های ساده و حالت تصویری — و شکل ردیف باید جایی می‌بود که هر دو راه از آن رد شوند و بشود بی‌مرورگر تستش کرد. `ShopContext` همچنان تنها جایی است که سبد را عوض می‌کند؛ فقط شکل ردیف را می‌سازد، بدون هیچ حالتی.

```
const [items, setItems] = useState(initialItems); // ← از سرور
const [content, setContent] = useState(initialContent); // ← از سرور
const [loading, setLoading] = useState(false);
const [loadError, setLoadError] = useState('');
const [catalogueComplete, setCatalogueComplete] = useState(fullCatalogue);
const [cart, setCart] = useState([]); // ← همیشه خالی
const [hydrated, setHydrated] = useState(false);
const [grind, setGrind] = useState('whole');
```

دو تای اول با مورد ۲۶ عوض شدند: پیش‌تر خالی شروع می‌شدند و یک افکت موقع mount از سرور می‌گرفتشان (loading) هم برای همین (true شروع می‌شد). حالا داده روی سرور خوانده و به شکل پارامتر تحویل داده می‌شود.

سه تای بعدی تازه‌اند و هر سه به یک چیز مربوط‌اند: سبد نباید بی‌صدا پاک شود.

و یکی از این فهرست بیرون رفت. تا پیش از این یک `const [toastMsg, setToastMsg] = useState('')` هم اینجا بود. توست حالا در `lib/toastStore.js` زندگی می‌کند و از این کانتکست فقط یک پوششِ دوخطی مانده، تا ده صداکننده فعلی دست نخورند:

```
/* خودِ توست دیگر حالتِ این کانتکست نیست و در
lib/toastStore.js زندگی می‌کند - تا مودال کالا هم،
که بیرون از این Provider رندر می‌شود، بتواند پیام
بدهد. */
const toast = useCallback((msg) => showToast(msg), []);
```

دلیل کاملش پای `lib/toastStore.js` در همین فصل، و در تصمیم ۲۶ فصل ۹.

سبد و hydration – سه نگرهبان

سبد پیش‌تر در همان اولین رندر خوانده می‌شد: `useState(loadCart)`. زیر رندر سمت سرور این یک خط سه مسئله می‌سازد و هر سه به یک نتیجه می‌رسند:

۱) روی سرور `localStorage` وجود ندارد. ۲) اگر سرور سبد خالی بفرستد و مرورگر در همان رندر اول سبد پُر بسازد، ریاکت ناسازگاری می‌بیند. ۳) افکت ذخیره اگر پیش از افکت خواندن اجرا شود، آرایه خالی اولیه را روی سبد ذخیره‌شده می‌نویسد.

جواب سه نگرهبان است:

```
/* یک بار، فقط بخوان */
useEffect(() => {
  const saved = loadCart(browserStorage());
  if (saved.length) setCart(saved);
  setHydrated(true);
}, []);

/* پیش از خواندن، هیچ‌وقت ننویس */
useEffect(() => {
  if (!hydrated) return;
  saveCart(browserStorage(), cart);
}, [cart, hydrated]);
```

و نگرهبان سوم `catalogueComplete` است، که پای افکت پاک‌سازی توضیح داده می‌شود.

بهایش یک فریم است: تا اجرا نشدن افکت، شمارندهٔ سبدها در هدر «گرم» است. این تفاوت واقعی با نسخهٔ قبل است و عمداً با `suppressHydrationWarning` پنهان نشده — آن پرچم هشدار را خاموش می‌کند، نه مسئله را. `tests/cart-hydration.test.jsx` یک چرخهٔ کامل «رندر سرور ← hydrate» را اجرا می‌کند و هر سه شکست را جدا می‌سجد.

`reload()` — دیگر موقع mount صدا زده نمی‌شود

```
const [data, texts] = await Promise.all([
  api.items(),
  api.content().catch(() => ({}))
]);
```

روی محتوا اما نه روی کالاها. کامنت دلیل را می‌گوید: «متن‌ها نباید فهرست کالاها را زمین بزنند؛ اگر نیامدند بخش‌های متنی رندر نمی‌شوند ولی فروشگاه کار می‌کند.» این یک تصمیم **تنزل مطبوع** (graceful degradation) است.

ساختارهای مشتق

```
const bySlug = useMemo(() => new Map(items.map((i) => [i.slug, i])), [items]);
const lookup = useCallback((slug) => bySlug.get(slug), [bySlug]);

const byKind = useMemo(() => {
  const out = { coffee: [], gear: [], powder: [] };
  for (const it of items) if (out[it.kind]) out[it.kind].push(it);
  return out;
}, [items]);

const houseBlends = useMemo(
  () => (byKind.coffee || []).filter((c) => c.isBlend && c.house).sort((a, b) => a.rank - b.rank),
  [byKind]
);
```

به جای `find()` تکراری: در سبدها با ۱۰ ردیف و ۱۰۶ کالا، `find()` یعنی حداکثر ۱۰۶۰ مقایسه؛ `Map.get()` یعنی ۱۰ عملیات `O(1)`.

افکت پاک‌سازی سبدها

```
useEffect(() => {
  if (!hydrated || !catalogueComplete || loading || items.length === 0) return;
  setCart((prev) => {
    const kept = prev.filter(
      (l) => bySlug.has(l.slug) && (l.mix || []).every((m) => bySlug.has(m.slug))
    );
    return kept.length === prev.length ? prev : kept;
  });
}, [hydrated, catalogueComplete, loading, items.length, bySlug]);
```

خط آخر درون `setCart` مهم است: اگر چیزی حذف نشد، همان آرایهٔ قبلی برگردانده می‌شود نه یک آرایهٔ تازه. این از `rerender` بی‌مورد و حلقهٔ بی‌نهایت افکت جلوگیری می‌کند.

ولی دو شرط اول افکت مهم‌ترند، و هر دو با مورد ۲۶ اضافه شدند.

این افکت هر ردیفی را که کالایش در `bySlug` نیست حذف می‌کند — منطقی، چون مدیر ممکن است کالایی را خاموش کرده باشد. مشکل آنجاست که صفحهٔ یک کالا عمداً **فهرست ناقص** می‌گیرد: فقط خود کالا و دانه‌های میکسش، نه صد و شش کالا. بدون شرط `catalogueComplete`، همین یک بازدید کل سبد مشتری را پاک می‌کرد.

و یک لایهٔ دیگر هم لازم بود. نگه داشتن ردیف در سبد کافی نیست: `resolved` ردیف‌ها را با سند کالا جفت می‌کند و ردیف بی‌سند را کنار می‌گذارد – و `CartDrawer` بدنهٔ سفارش را از همان `resolved` می‌سازد. یعنی سبد در حافظه سالم می‌ماند ولی **سفارش با ردیف‌های کمتر ثبت می‌شود**، بی‌صدا. برای همین افکتی هست که اگر سبد به کالایی بی‌رد اشاره کند، فهرست را کامل می‌کند:

```
useEffect(() => {
  if (!hydrated || catalogueComplete || fillTried.current || cart.length === 0) return;
  const missing = cart.some(
    (l) => !bySlug.has(l.slug) || (l.mix || []).some((m) => !bySlug.has(m.slug))
  );
  if (!missing) return;
  fillTried.current = true;
  reload();
}, [hydrated, catalogueComplete, cart, bySlug, reload]);
```

برای بازدیدکنندهٔ بی‌سبد – یعنی بیشترشان – این افکت هیچ کاری نمی‌کند و صفحهٔ کالا سبک می‌ماند.

عملیات سبد

تابع	امضا	کار
<code>addWeighed</code>	<code>(slug, grams, opts)</code>	افزودن/جمع‌زدن ردیف وزنی
<code>addPiece</code>	<code>(slug, qty=1)</code>	افزودن/جمع‌زدن ابزار
<code>changeWeight</code>	<code>(key, delta)</code>	تغییر وزن با کف <code>minWeight</code>
<code>changeQty</code>	<code>(key, delta)</code>	تغییر تعداد با کف ۱
<code>setLineGrind</code>	<code>(key, nextGrind)</code>	تغییر آسیاب + ادغام ردیف‌های همسان
<code>removeLine</code>	<code>(key)</code>	حذف با <code>toast</code> مناسب نوع کالا
<code>clearCart</code>	<code>()</code>	خالی کردن

`setLineGrind` پیچیده‌ترین است چون تغییر آسیاب، **کلید ردیف را عوض می‌کند** و ممکن است با ردیف دیگری یکی شود:

```
const twin = prev.findIndex((l, idx) => idx !== i && l.key === newKey);

if (twin === -1) { /* کلید تازه یکتاست */ }

/* ادغام: وزن‌ها جمع می‌شوند و ردیف تکراری حذف */
return prev
  .map((l, idx) => idx === twin ? { ...l, grams: l.grams + line.grams, qty: l.qty + line.qty } : l)
  .filter((_, idx) => idx !== i);
```

مقادیر مشتق برای نمایش

```
const resolved = useMemo(
  () => cart.map((l) => ({ ...l, item: bySlug.get(l.slug) })).filter((l) => l.item),
  [cart, bySlug]
);

const totals = useMemo(() => computeTotals(resolved, lookup), [resolved, lookup]);
```

`resolved` ردیف سبد را با سند کالا جفت می‌کند — همان شکلی که `computeTotals` انتظار دارد. `filter((l) => l.item)` لایهٔ دفاعی دومی است در برابر کالاهای حذف‌شده.

```
const nextTier = useMemo(() => {
  if (totals.grams <= 0) return null;
  return [...TIERS].reverse().find((t) => t.min > totals.grams) || null;
}, [totals.grams]);
```

`TIERS` نزولی است (۵۰۰۰، ۳۰۰۰، ۱۰۰۰، ۰)؛ `reverse()` آن را صعودی می‌کند و `find(t => t.min > grams)` نزدیک‌ترین پلهٔ بعدی را می‌دهد. با ۶۰۰ گرم، جواب پلهٔ ۱۰۰۰ است و سبد پیام می‌دهد: «۴۰۰ گرم دیگر اضافه کنید تا تخفیف ۵٪ فعال شود.»

`web/src/context/AuthContext.jsx`

سه حالت: `admin` (شیء یا `null`)، `checking` (بولی)، و توابع.

الگوی `checking` مهم است: بدون آن، در لحظهٔ اول بارگذاری `admin === null` است و `AdminLayout` کاربر را به صفحهٔ ورود می‌فرستاد — حتی اگر توکن معتبر داشت. با `checking`، صفحهٔ «در حال بررسی نشست...» نشان داده می‌شود تا `api.me()` جواب بدهد.

`web/src/lib/format.js`

```
export const toFa = (n) => Number(n || 0).toLocaleString('fa-IR');
export const money = (n) => toFa(n) + ' تومان';

export function formatWeight(g) {
  if (g < 1000) return toFa(g) + ' گرم';
  const kg = g / 1000;
  return toFa(Number.isInteger(kg) ? kg : kg.toFixed(2)) + ' کیلوگرم';
}
```

دو کار همزمان می‌کند: رقم‌ها را فارسی می‌کند و جداکنندهٔ هزارگان می‌گذارد. `toLocaleString('fa-IR')` می‌شود `1850000` . `1,۸۵۰,۰۰۰`

`formatWeight` نکتهٔ ظریفی دارد: `Number.isInteger(kg)` تعیین می‌کند کیلوگرم ۲ نوشته شود یا کیلوگرم ۱/۵۰.

`faDate` از `Intl.DateTimeFormat('fa-IR')` استفاده می‌کند که تقویم شمسی بومی مرورگر است — پس در کلاینت نیازی به `jalali.js` نیست. سرور آن را لازم دارد چون باید بازه‌ها را روی مرز روزهای شمسی ببندد.

`toLatinDigits` قرینهٔ `toLatin` در `server/src/lib/track.js` است: هر جایی که کاربر عدد تایپ می‌کند (وزن، قیمت، شمارهٔ موبایل، شمارهٔ سفارش) باید رقم فارسی هم پذیرفته شود.

شکل یک ردیف سبد — web/src/lib/cartLine.js

صادرات: `buildLine`، `normalizeMix`، `lineKey`، `mixSignature`

بدون هیچ حالتی و بدون React — یعنی هم `BlendsSection` (اهرم‌ها) و هم `MixVisual` (حالت تصویری) از همین‌جا رد می‌شوند و تضمین «یک میکس، از هر راهی که ساخته شود، دقیقاً همان ردیف سبد» تست‌پذیر می‌ماند.

```
export function mixSignature(mix) {
  if (!Array.isArray(mix) || mix.length === 0) return '';
  return [...mix]
    .filter((m) => Number(m.percent) > 0)
    .map((m) => `${m.slug}:${Math.round(Number(m.percent) || 0)}%`)
    .sort()
    .join(',');
}

export const lineKey = (slug, grind, mix) => `${slug}|${grind || ''}|${mixSignature(mix)}%`;
```

`.sort()` حیاتی است: ترکیب `[cerrado 60, monsooned 40]` و `[monsooned 40, cerrado 60]` باید یک امضا بدهند، وگرنه دو ردیف تکراری در سبد می‌ساختند. توجه کنید که مرتب‌سازی فقط داخل امضا اتفاق می‌افتد، نه روی خود آرایه.

`normalizeMix` دانه‌های صفرشده را کنار می‌گذارد و درصدها را گرد می‌کند:

```
export function normalizeMix(item, mix) {
  if (!item?.isBlend) return [];
  if (Array.isArray(mix) && mix.length) {
    return mix
      .filter((m) => Number(m.percent) > 0)
      .map((m) => ({ slug: m.slug, percent: Math.round(Number(m.percent)) }));
  }
  return (item.components || []).map((c) => ({ slug: c.slug, percent: c.percent }));
}
```

ترتیب دانه‌ها همان ترتیب ورودی می‌ماند و عمداً مرتب نمی‌شود — حالت تصویری با همین ترتیب کیسه‌ها را می‌ریزد.

`buildLine` هم آسیاب را فقط برای قهوه می‌گذارد (پودر آسیاب نمی‌خورد) و ردیف نهایی را با کلیدش برمی‌گرداند:

```
export function buildLine(item, grams, opts = {}, fallbackGrind = '') {
  const grind = item.grindable ? (opts.grind ?? fallbackGrind) : '';
  const mix = normalizeMix(item, opts.mix);
  return { key: lineKey(item.slug, grind, mix), slug: item.slug,
    kind: item.kind, grams, qty: 0, grind, mix };
}
```

ریاضی و زمان‌بندی حالت تصویری — web/src/lib/blendVisual.js

صادرات: `pourPlan`، `SETTLE_MS`، `MIX_MS`، `POUR_MIN`، `POUR_BUDGET`، `stageMessage`، `STAGE_TITLE`، `STAGES`، `prefersReducedMotion`، `hopperLevel`، `sackLayout`، `SCENE`، `pourTotal`

هیچ درصدی اینجا حساب نمی‌شود. هرچه به درصدها مربوط است از `applyPercent` در `lib/blend.js` می‌آید و همان‌جا می‌ماند. آنچه اینجا است تصمیم‌های نمایشی است – و همه تابع خالص‌اند تا بشود بدون مرورگر تستشان کرد و تا هیچ‌کدام لازم نباشد فریم‌به‌فریم در جاوااسکریپت حساب شوند. عددها یک بار درمی‌آیند و بقیه‌اش کار CSS است.

بودجه زمانی به‌جای زمان هر کیسه. کل مرحله ریختن باید چند ثانیه باشد، نه چند ثانیه به‌ازای هر کیسه:

```
export const POUR_BUDGET = 2600; // میلی‌ثانیه، کل مرحله ۲
export const POUR_MIN = 320; // کوتاه‌ترین ریختن قابل‌دیدن
```

`pourPlan` این بودجه را بین کیسه‌ها به نسبت سهمشان پخش می‌کند، با یک کف کوتاه تا کیسه ۵٪ هم دیده شود:

```
const share = live.length * min >= budget; // بودجه به کف همه نمی‌رسد
const extra = budget - live.length * min;
const ms = share
  ? Math.round(budget / live.length) // مساوی پخش کن
  : Math.round(min + (percent / sum) * extra);
```

و باقیمانده گردکردن روی آخرین کیسه می‌نشیند تا جمع زمان دقیقاً همان بودجه باشد و مرحله ۲ کش نیاید – چیزی که `blend-visual.test.js` مستقیماً می‌سنجد.

هندسه صحنه. `SCENE` یک SVG با دستگاه سمت چپ و کیسه‌های سمت راست توصیف می‌کند: جهت خواندن فارسی از راست به چپ است، پس قهوه هم از راست به سمت دستگاه می‌رود. `first: 296` مرکز راست‌ترین کیسه (اولین انتخاب) است.

ظریف‌ترین محاسبه فایل در `sackLayout` است. کیسه حول پایه‌اش می‌چرخد، پس با چرخش `tilt` دهانه‌اش از پایه فاصله می‌گیرد – و مقدار این فاصله به بزرگی کیسه بستگی دارد:

```
const rad = (scene.tilt * Math.PI) / 180;
const up = [Math.sin(rad), -Math.cos(rad)];
const reach = scene.mouth * scale;

hx: scene.hopper.x - reach * up[0],
hy: scene.hopper.y - reach * up[1]
```

برای اینکه دهانه هر کیسه – کوچک و بزرگ – دقیقاً سر قیف بایستد، پایه‌اش به‌اندازه همین بردار عقب برده می‌شود. بدون این، کیسه کوچک بالای هوا خالی می‌کرد.

کم‌حرکتی. `prefersReducedMotion()` تنها جایی است که به `window` دست می‌زند (با نگرهبان `typeof window !== 'undefined'` تا در محیط node تست بشکند نه). اگر روشن باشد، هیچ مرحله‌ای تایمر ندارد: با یک کلیک مستقیم به نتیجه می‌رسد – همان بسته، همان قیمت، همان ردیف سبد، فقط بدون نمایش.

```
export function useReveal(deps = []) {
  useEffect(() => {
    const cards = [...document.querySelectorAll('.card:not(.is-in)')];
    if (cards.length === 0) return;

    if (window.matchMedia('(prefers-reduced-motion: reduce)').matches) {
      cards.forEach((c) => c.classList.add('is-in'));
      return;
    }

    const observer = new IntersectionObserver((entries) => {
      entries.forEach((e, i) => {
        if (!e.isIntersecting) return;
        setTimeout(() => e.target.classList.add('is-in'), i * 55);
        observer.unobserve(e.target);
      });
    }, { threshold: 0.12 });

    cards.forEach((c) => observer.observe(c));
    return () => observer.disconnect();
  }, deps);
}
```

سه نکته:

- `:not(.is-in)` یعنی کارتهایی که قبلاً وارد شده‌اند دوباره رصد نمی‌شوند.
- `prefers-reduced-motion` قبل از ساختن observer بررسی می‌شود و کارتها فوراً نمایان می‌شوند.
- `i * 55` تأخیر پلکانی می‌سازد؛ `unobserve` بعد از ورود، کار observer را کم می‌کند.

این یکی از محدود جاهایی است که پروژه مستقیماً با DOM کار می‌کند (به جای state) – انتخاب آگاهانه‌ای برای اینکه ۱۰۶ کارت با هر اسکرول rerender نشوند.

بخش ج – وب: کامپوننت‌های فروشگاه

Header.jsx

هدر چسبان با لوگوی SVG دست‌نویس، ۱۰ لینک لنگری در آرایه `LINKS`، دکمهٔ سبد که `cartSummary(true)` را نشان می‌دهد، و دکمهٔ برگ با `aria-controls` / `aria-expanded`.

لنگرها بیرون از صفحهٔ اصلی. همهٔ آن ۱۰ لینک به بخش‌های صفحهٔ اصلی اشاره می‌کنند. روی خود صفحهٔ اصلی، لنگر ساده درست‌ترین کار است؛ ولی روی صفحهٔ اختصاصی کالا لنگر تنها هیچ کاری نمی‌کند. پس شکل لینک از مسیر فعلی تصمیم گرفته می‌شود:

```
const onHome = usePathname() === '/';
```

روی صفحهٔ اصلی `<a href="#products">` و جای دیگر `<Link href="#products">`.

و یک لینک سوم در `header-actions`: پیگیری سفارش.

```
<Link className="btn-track" href="/track" aria-label="پیگیری سفارش">
  <svg viewBox="0 0 24 24" aria-hidden="true" className="icon">...</svg>
  <span>پیگیری سفارش</span>
</Link>
```

تا پیش از این فقط فوتر و خود رسید به `/track` راه داشتند؛ مشتری برگشته باید تا ته صفحه اسکرول می‌کرد تا سفارشش را پیدا کند. حالا همان‌جایی است که چشم دنبال کار حساب می‌گردد – کنار سبد. لینک فوتر سر جایش ماند: کسی که به ته صفحه رسیده هم نباید برگردد بالا.

روی خود لینک لازم است و نه فقط روی آیکن، چون زیر ۷۶۰ پیکسل برچسب متنی `display: none` می‌شود و از درخت دسترس‌پذیری هم بیرون می‌رود – آن وقت لینک بی‌نام می‌ماند:

```
.btn-cart span,
.btn-track span { display: none; }
```

«ترازوی قیمت» Hero.jsx

امضای بصری صفحه. یک `<select>` با `<optgroup>` بر پایهٔ دستهٔ رست، یک اسلایدر ۱۰۰ تا ۲۰۰۰ گرم با گام ۵۰، و نمایش زندهٔ قیمت.

```
useEffect(() => {
  if (coffees.length === 0) return;
  if (!coffees.some((c) => c.slug === slug)) {
    const first = [...coffees].sort((a, b) => a.rank - b.rank)[0];
    setSlug(first.slug);
  }
}, [coffees, slug]);
```

این افکت دو کار می‌کند: انتخاب خودکار اولین قهوه در بارگذاری، و بازبایی اگر قهوهٔ انتخاب‌شده حذف شود.

عدد قهوه `{toFa(coffees.length)}` در «hero-facts» زنده است و با تعداد واقعی کالاهای عوض می‌شود.

– یک کامپوننت برای سه فهرست CatalogSection.jsx

```
<CatalogSection kind="coffee" id="products" eyebrow="فهرست این هفته" ... />
<CatalogSection kind="gear" id="gear" eyebrow="قفسه ابزار" ... />
<CatalogSection kind="powder" id="powders" eyebrow="غیر از قهوه" ... />
```

همه تفاوت‌ها از `KINDS[kind]` می‌آید و بقیه `prop` است.

جست‌وجوی `debounce` شده:

```
const timer = useRef(null);
useEffect(() => {
  clearTimeout(timer.current);
  timer.current = setTimeout(() => setQuery(queryInput), 160);
  return () => clearTimeout(timer.current);
}, [queryInput]);
```

دو `state` جدا (`queryInput` برای `input` و `query` برای فیلتر) الگوی استاندارد `debounce` کنترل‌شده است.

فیلتر و مرتب‌سازی:

```
const hay = (p) => [p.name, p.origin, p.spec, ...p.notes, ...p.pairs,
  meta.groups[p.group]?.label || ''].join(' ');
list = list.filter((p) => hay(p).includes(q));
```

جست‌وجو در شش منبع، از جمله برچسب فارسی دسته – پس «رست تیره» هم نتیجه می‌دهد.

```
const by = {
  rank: (a, b) => a.rank - b.rank,
  'price-asc': (a, b) => a.price - b.price,
  'price-desc': (a, b) => b.price - a.price,
  'meter-asc': (a, b) => a.meter - b.meter || a.rank - b.rank,
  'meter-desc': (a, b) => b.meter - a.meter || a.rank - b.rank,
  name: (a, b) => a.name.localeCompare(b.name, 'fa')
};
```

`a.rank - b.rank` مرتب‌سازی پایدار می‌سازد؛ `localeCompare(_, 'fa')` الفبای فارسی را درست مرتب می‌کند (نه ترتیب کد یونیکد).

حالت گروه‌بندی‌شده:

```
const grouped = filter === 'all' && sort === 'rank' && !query.trim();
```

فقط وقتی هیچ فیلتری فعال نیست، کالاها زیر عنوان دسته‌شان دسته‌بندی می‌شوند. کامنت: «وقتی همه‌چیز نمایش داده می‌شود و ترتیب پیشنهادی است، کالاها دسته‌بندی‌شده نشان داده می‌شوند.»

یک کارت برای هر سه نوع. تفاوت‌ها از `KINDS[item.kind]` می‌آید:

```
const CARD_CLASS = { coffee: 'card', gear: 'card card-gear', powder: 'card card-powder' };
```

نمایش موجودی. کارت سه حالت را از هم جدا می‌کند: نامحدود (هیچ نشانه‌ای نشان نمی‌دهد)، موجود شمرده‌شده (عدد باقی‌مانده)، و ناموجود:

```
const soldOut = isSoldOut(item);
...
{soldOut ? <span className="badge badge-out">ناموجود</span> : null}
...
<button disabled={soldOut}>{soldOut ? 'افزودن' : 'ناموجود'}</button>
```

همین منطق در `Hero.jsx` («ترازوی قیمت») هم هست. توجه کنید که این فقط پیشگیری در رابط کاربری است؛ حرف آخر را رزرو اتمی سمت سرور می‌زند و اگر کالا بین بارگذاری صفحه و ثبت سفارش تمام شود، پاسخ ۴۰۹ می‌آید.

همگام‌سازی آسیاب:

```
const [pick, setPick] = useState(grind);
const [touched, setTouched] = useState(false);

useEffect(() => { if (!touched) setPick(grind); }, [grind, touched]);
```

کامنت توضیح می‌دهد: «آسیاب این کارت با پیش‌فرض سایت شروع می‌شود و اگر مشتری از بخش «روش دم‌آوری» پیش‌فرض را عوض کند، کارت‌هایی که دست نخورده‌اند هم همراهش می‌آیند.» یعنی وقتی کاربر روی کارت «اسپرسو» در بخش دم‌آوری می‌زند، همه کارت‌های دست‌نخورده به آسیاب اسپرسو می‌روند اما کارتی که خودش انتخاب کرده دست‌نخورده می‌ماند.

قیمت میکس:

```
const perKg = item.isBlend ? blendPrice(item, item.components) : item.price;
```

روی کارت، قیمت میکس با ترکیب رسمی حساب می‌شود؛ در بخش میکس‌ها با ترکیب دلخواه کاربر.

مودال معرفی. تابع صادرشده `hasStory` توسط `ItemCard` استفاده می‌شود تا تصمیم بگیرد دکمه «درباره این قهوه» را نشان دهد یا نه:

```
export const hasStory = (item) => Boolean(item?.story || item?.taste || item?.recommend);
```

سه بلوک با `.filter((b) => b.text)` ساخته می‌شوند، پس بخش خالی اصلاً رندر نمی‌شود.

مدیریت فوکوس و اسکرول در `useEffect`:

```
lastFocus.current = document.activeElement;
document.body.style.overflow = 'hidden';
panel.current?.focus();
// ...
return () => {
  document.body.style.overflow = '';
  document.removeEventListener('keydown', onKey);
  lastFocus.current?.focus?().();
};
```

نوار کنش‌های بالای پنجره. کنار دکمه بستن، دکمه هم‌رسانی هم اینجا است:

```
<div className="sheet-actions">
  <ShareButton item={item} className="sheet-share" />
  <button className="btn-close sheet-close" onClick={onClose} aria-label="بستن"></button>
</div>
```

جایش تصادفی نیست. بیشتر بازدیدکننده‌ها کالا را از همین پنجره می‌بینند، نه از صفحه کامل؛ نبودن دکمه اینجا یعنی نبودن عملی قابلیت. و از مورد ۲۷ به بعد این پنجره آدرس واقعی خودش را دارد، پس چیزی که فرستاده می‌شود همان آدرسی است که در نوار مرورگر است.

نکته

این پنجره در شکاف `@modal` رندر می‌شود، یعنی بیرون از `ShopProvider`. برای همین `ShareButton` تأییدش را از انباره ماژولی `lib/toastStore.js` می‌گیرد و نه از `useShop()`. نگهداریش `tests/item-detail.test.jsx` است: مودال را بدون هیچ `Provider`ی رندر می‌کند. اگر روزی چیزی داخل این پنجره دوباره به کانتکست وابسته شود، بقیه تست‌ها سبز می‌مانند و فقط پرکاربردترین راه دیدن کالا می‌ترکد.

CartDrawer.jsx

۳۸۴ خط، سه حالت: `cart` → `form` → رسید. حالت با ترکیب `step` و `done` تعیین می‌شود:

```
{</فرم> : <فهرست سبد> ? step === 'form' : <Receipt .../> : done ?
```

بازنشانی با تأخیر:

```
setTimeout(() => { setStep('cart'); setError(''); setDone(null); }, 350);
```

۳۵۰ میلی‌ثانیه صبر می‌کند تا انیمیشن بسته شدن کشو تمام شود، وگرنه کاربر می‌دید که محتوا وسط انیمیشن عوض می‌شود.

کامپوننت `Receipt` از پاسخ سرور ساخته می‌شود نه از سبد. حتی شماره تماس فروشگاه از `content.about` خوانده می‌شود:

```
<Receipt order={done} shop={content?.about} />
```

با کامنت: «راه تماس — از «درباره ما» خوانده می‌شود تا اگر مدیر شماره را عوض کرد، رسید هم به‌روز باشد.»

و همان رسید یک بار دیگر رندر می‌شود — این بار داخل `PrintSheet`، که آن را به شکل فرزند مستقیم `body` می‌گذارد:

```

<div className="drawer-body">
  <Receipt order={done} shop={content?.about} />
</div>

/* همان رسید، این بار فرزند مستقیم body - تنها چیزی که چاپگر می‌بیند */
<PrintSheet>
  <Receipt order={done} shop={content?.about} />
</PrintSheet>

<div className="drawer-foot">
  <button className="btn btn-primary btn-block" onClick={() => window.print()}>
    چاپ یا ذخیرهٔ رسید
  </button>

  ""
</div>

```

دو بار رندر شدن اسراف به‌نظر می‌رسد و نیست: یکی از آن دو روی صحنه `display: none` است و هیچ‌وقت هم‌زمان با دیگری دیده نمی‌شود. چرایی کاملش پای `PrintSheet.jsx` در همین فصل، و در تصمیم ۲۷ فصل ۹.

Receipt.jsx - یک شکل، دو صداکننده

۲۰۵ خط. از `CartDrawer` بیرون کشیده شد، چون دو جا همین برگه را نشان می‌دهند: کشوی سبد بعد از ثبت سفارش، و صفحهٔ پیگیری برای کسی که رسیدش را بسته است. اگر دو نسخه می‌شد، روزی یکی «تخفیف وزنی» را نشان می‌داد و دیگری نه. چهار پارامتر تفاوت‌های آن دو جا را می‌پوشانند:

پارامتر	پیش‌فرض	چرا هست
<code>heading</code>		در کشو خبر ثبت است، در پیگیری فقط عنوانِ سند: «رسید سفارش»
<code>showMark</code>	<code>true</code>	تیک سبز فقط وقتی معنی دارد که همین الان ثبت شده باشد - سفارش لغوشده تیک نمی‌خواهد
<code>statusLabel</code>	<code>''</code>	وضعیت سفارش؛ فقط صفحهٔ پیگیری آن را می‌داند
<code>showTracking</code>	<code>true</code>	لینک «وضعیت سفارش را ببینید» روی خود صفحهٔ پیگیری بی‌معنی است

دو شکل داده که هر دو باید کار کنند. پاسخ `POST /api/orders` کامل است - با نشانی و تلفن؛ پاسخ `GET /api/orders/track` نمای عمومی است و عمداً نشانی و تلفن و توضیح را ندارد (`server/src/lib/track.js`). برای همین هر تکه‌ای که ممکن است نباشد شرط دارد، و یک شرط ترکیبی هم هست تا بلوک «تحويل به» برای یک نام تنها یک قاب خالی نسازد:

```
const hasDelivery = Boolean(customer?.phone || customer?.address);
```

```
<h4 className="receipt-h">{hasDelivery ? 'به نام' : 'تحويل به'}</h4>
```

و در رسید، وزن هر دانه میکس محاسبه می‌شود:

```
<small>{formatWeight(Math.round((l.grams * m.percent) / 100))}</small>
```

یعنی مشتری می‌بیند «سرادو ۶۰٪ - ۳۰۰ گرم». کلید حلقه میکس هم عمداً شماره است نه `slug`: نمای عمومی پیگیری فقط نام و درصد می‌دهد.

۶۷ خط، یک `useState` و یک `portal`. کوچک‌ترین کامپوننت پروژه، و جانشین ترفندی که ۳۳ صفحه کاغذ سفید می‌داد.

```
const FLAG = 'has-print-sheet';

export default function PrintSheet({ children }) {
  const [ready, setReady] = useState(false);

  useEffect(() => {
    setReady(true);
    document.body.classList.add(FLAG);
    return () => document.body.classList.remove(FLAG);
  }, []);

  if (!ready) return null;

  return createPortal(
    <div className="print-sheet" aria-hidden="true">{children}</div>,
    document.body
  );
}
```

مسئله‌ای که حل می‌کند. نسخهٔ پیشین رسید را با `visibility: hidden` روی `body * body` چاپ می‌کرد و فقط کشوی سید را برمی‌گرداند. `visibility` عنصر را از چیدمان حذف نمی‌کند: کل فروشگاه – هدر، سه فهرست کالا، کارت‌ها و SVG‌هایشان، متن‌ها و فوتر – همچنان ارتفاع واقعی خودشان را داشتند و مرورگر همان ارتفاع را صفحه‌بندی می‌کرد. نتیجه اندازه‌گیری شد: ۳۳ صفحه، که ۳۱ تای آخرش خالی بود.

`display: none` این را حل می‌کند، ولی نمی‌شود مستقیم روی نیاکان رسید زدش – رسید چند لایه داخل درخت است و پنهان کردن هر نیایی خودش را هم می‌برد. پس رسید یک بار دیگر و به شکل فرزند مستقیم `body` رندر می‌شود، و آن وقت قاعدهٔ چاپ یک خط است.

سه جزئیات که هر کدام یک اشکال را می‌بندند:

(۱) `ready`. `createPortal` به `document` نیاز دارد و روی سرور چنین چیزی نیست. تا وقتی `mount` نشده هیچ چیزی بر نمی‌گرداند، پس HTML سرور و اولین رندر مرورگر یکی‌اند و `hydrate` شکایتی ندارد.

(۲) `aria-hidden`. روی صحنه این میزبان `display: none` است، ولی صفحه‌خوان نباید همان رسید را دو بار بخواند.

(۳) `FLAG` روی `body`. بدون این نشانه، قاعدهٔ «هر فرزند `body` به جز میزبان را بردار» روی هر صفحه‌ای اجرا می‌شد و `Ctrl+P` روی خود فروشگاه یک برگ سفید می‌داد. با آن، قاعده فقط وقتی زنده است که واقعاً رسیدی برای چاپ وجود داشته باشد – و پاک‌سازی `useEffect` برش می‌دارد.

سبک چاپ در ۸.۱۰ خط‌به‌خط آمده.

ShareButton.jsx – «برای دوستت بفرست»

۷۹ خط. روی سه جا می‌نشیند و در هر سه همان یک کار را می‌کند:

جا	کلاس	از کجا رندر می‌شود
کارت فهرست	card-share	ItemCard.jsx، در ردیف عنوان
صفحه اختصاصی کالا	card-share	همان کارت، از راه ItemBuyCard
مودال رهگیری شده	sheet-share	ItemDetail.jsx، در sheet-actions

منطق آدرس و کپی اینجا نیست، در lib/share.js است. آنچه اینجا می‌ماند چهار چیز است و هر چهار عمدی:

```
const onShare = async (e) => {
  e.preventDefault();
  e.stopPropagation();

  if (busy) return;
  setBusy(true);
  try {
    const { mode } = await shareItem(item);
    if (mode === 'copied') toast(SHARE_COPIED);
    else if (mode === 'failed') toast(SHARE_FAILED);
  } finally {
    setBusy(false);
  }
};
```

(۱) `<button type="button">` واقعی است، نه `<a>` و نه `div` کلیک‌پذیر: نه ناوبری می‌کند و نه فرمی را می‌فرستد.

(۲) `stopPropagation` دارد، چون داخل کارتی می‌نشیند که پر از کنترل است؛ کلیکش نباید به هیچ چیز دیگری برسد. همین را می‌سنجد. tests/item-card.test.jsx

(۳) دو حالت ساکت. `shared` و `canceled` هیچ توستی نمی‌دهند — اولی چون برگه سیستم خودش بازخورد داده، دومی چون کاربر نخواست.

(۴) `aria-label` نام کالا را دارد، وگرنه در فهرستی از صد کارت، صد دکمه «هم‌رسانی» بی‌تفاوت می‌شدند:

```
aria-label={`این کالا | ${item?.name || 'هم‌رسانی'}`}
```

و `busy` تا وقتی برگه سیستم باز است دکمه را قفل می‌کند تا دو بار پشت‌سرهم زده نشود.

نکته

توست را از انباره مازولی می‌گیرد، نه از `ShopContext`. همین یک خط است که می‌گذارد این دکمه داخل مودال هم کار کند، جایی که هیچ `Provider` ای بالای سرش نیست.

```
export default function Toast() {
  const message = useSyncExternalStore(subscribeToast, getToast, getServerToast);

  return (
    <div className={`toast ${message ? 'is-on' : ''}`.trim()} role="status" aria-live="polite">
      {message}
    </div>
  );
}
```

`useSyncExternalStore` سه آرگومان می‌گیرد و هر سه از `toastStore.js` می‌آیند: مشترک‌شدن، snapshot مرورگر، و snapshot سرور. آرگومان سوم اختیاری است ولی اینجا لازم است – بدون آن Next موقع رندر سروری خطا می‌دهد، و با آن تضمین می‌شود که HTML سرور همیشه یک توست خالی دارد.

`aria-live="polite"` و `role="status"` یعنی صفحه‌خوان پیام را می‌خواند بدون اینکه کاری را که کاربر در دست دارد قطع کند. عنصر همیشه در DOM هست و فقط کلاس `is-on` می‌گیرد، چون `opacity` باید بتواند گذار کند.

– BlendsSection.jsx میزبان هر دو حالت

`BlendCard` پنج حالت محلی دارد: `mix`، `grams`، `pick` (آسیاب)، `detail`، و `visual`.

`visual` تنها چیزی است که بین دو حالت فرق می‌کند. `mix`، `grams` و `pick` بالای هر دو می‌مانند، پس رفت و برگشت بین حالت ساده و حالت تصویری هیچ انتخابی را از دست نمی‌دهد. حالت ساده پیش‌فرض است:

```
const [visual, setVisual] = useState(false);
...
{visual ? <MixVisual item={item} mix={mix} available={available}
  canEdit={Boolean(item.customizable)}
  onPercent={change} onDrop={drop} onJoin={join}
  onAdd={add} error={error} /> : null}

{visual ? null : (<{*/ اهرم‌های نسبت */><BeanPicker ... /></>)}}
```

`MixVisual` هیچ حالتی از خودش درباره ترکیب ندارد؛ همان `mix` را می‌خواند و همان `change` (یعنی `applyPercent`) را صدا می‌زند. یعنی منطق میکس فورک نشده، فقط دو نما گرفته است.

دکمه تعویض حالت برچسبش کاری را می‌گوید که انجام می‌دهد («خودم ترکیب کنم» ↔ «حالت ساده»). کامنت توضیح می‌دهد چرا `aria-pressed` اینجا نیست: با آن، صفحه‌خوان «حالت ساده، فشرده» می‌خواند که معنایش برعکس است.

تنها یک راه به سبب:

```
const add = useCallback(
  () => addWeighed(item.slug, grams, { grind: pick, mix }),
  [addWeighed, item.slug, grams, pick, mix]
);
```

دکمه پاورقی در هر دو حالت سر جایش است و دکمه پایان مرحله ۴ در `MixVisual` دقیقاً همین `add` را صدا می‌زند. راه انداختن دستگاه یک خوشی اختیاری است، نه دروازه خرید.

```
const available = useMemo(
  () => (byKind.coffee || [])
    .filter((b) => !b.isBlend && b.slug !== item.slug && !mix.some((p) => p.slug === b.slug))
    .sort((a, b) => a.name.localeCompare(b.name, 'fa')),
  [byKind, item.slug, mix]
);
```

توجه: از همه قهوه‌ها استفاده می‌کند، نه از `item.pool`. کامنت می‌گوید: «هر قهوه‌ای که خودش میکس نباشد می‌تواند وارد ترکیب شود – مشتری آزاد است، ترکیب ما فقط پیشنهاد است.» به همین دلیل فیلد `pool` عملاً بی‌استفاده مانده.

تشخیص دست‌کاری:

```
const changed = useMemo(() => {
  const base = startingMix(item);
  if (base.length !== mix.length) return true;
  return base.some((b) => mix.find((m) => m.slug === b.slug)?.percent !== b.percent);
}, [item, mix]);
```

فقط اگر `changed` باشد دکمه «بازگشت به پیشنهاد ما» نشان داده می‌شود.

MixVisual.jsx – حالت تصویری ساز میکس

۴۸۵ خط. یک صفحه SVG با دستگاه ترکیب سمت چپ و کیسه‌های قهوه سمت راست، در چهار مرحله:

مرحله	کلید	چه اتفاقی می‌افتد
۱	<code>pick</code>	چیدن کیسه‌ها – درصد‌ها همین‌جا تنظیم می‌شوند
۲	<code>pour</code>	کیسه‌ها یکی‌یکی در قیف خالی می‌شوند
۳	<code>mix</code>	کار دستگاه – کوتاه، با امکان رد کردن
۴	<code>done</code>	بسته آماده، و دکمه افزودن به سبد

هیچ مرحله‌ای خودبه‌خود جلو نمی‌رود مگر مشتری بخواهد؛ مرحله ۱ تا وقتی «شروع ترکیب» زده نشود همان‌جا می‌ماند.

تنها یک تایمر در هر لحظه:

```
useEffect(() => {
  if (reduced) return undefined;
  if (stage === 'pour') {
    const cur = plan[poured];
    if (!cur) return undefined;
    const t = setTimeout(() => {
      const done = poured + 1;
      setPoured(done);
      if (done >= plan.length) setStage('mix');
    }, cur.ms);
    return () => clearTimeout(t);
  }
  if (stage === 'mix') {
    const t = setTimeout(() => setStage('done'), MIX_MS);
    return () => clearTimeout(t);
  }
  return undefined;
}, [stage, poured, plan, reduced]);
```

نه حلقه `requestAnimationFrame` و نه محاسبه فریم به فریم: جاوااسکریپت فقط **مرز** مرحله‌ها را می‌زند و حرکت بین دو مرز کار CSS است. مدت هر ریختن با یک متغیر CSS به صحنه داده می‌شود:

```
<div className="mixv" data-stage={stage}
  style={{ '--pour': `${span}ms`, '--settle': `${SETTLE_MS}ms` }}>
```

پس هم انیمیشن کیسه و هم بالا آمدن قیف با یک عدد کوچک می‌شوند و مو به مو هم‌زمان تمام می‌شوند.

بازنشانی هنگام تغییر ترکیب – **حین رندر**، نه در افکت:

```
const [ranWith, setRanWith] = useState(signature);
if (ranWith !== signature) {
  setRanWith(signature);
  setStage('pick');
  setPoured(0);
}
```

اگر ترکیب عوض شود (چه با اهرم‌های همین‌جا، چه در حالت ساده و بازگشت به اینجا) آنچه دستگاه ساخته دیگر معتبر نیست. با `useEffect` کاربر یک فریم بسته قبلی را می‌دید و بعد پرش می‌خورد؛ این الگو – که خود React اسمش را «تنظیم state در حین رندر» می‌گذارد – آن فریم را حذف می‌کند.

قیف یک کیسه عقب نمی‌ماند:

```
const pouring = stage === 'pour' ? plan[poured] : null;
const filled = poured + (pouring ? 1 : 0);
const hopperTone = useMemo(() => mixTone(beans.slice(0, filled)), [beans, filled]);
const level = hopperLevel(plan, filled);
```

در مرحله ۲ کیسه شماره `poured` وسط خالی شدن است، پس هم سطح قیف و هم رنگش باید شامل همان کیسه باشد و در طول همان ریختن به مقصد برسد.

رنگ از همان جدول کارت‌ها می‌آید. `roastTone(meter)` و `mixTone(parts)` از `art.js` صادر شده‌اند تا یک دانه در کارت و در ساز میکس دقیقاً یک رنگ داشته باشد – نه یک کپی دوم از جدول رنگ رست.

شناسه یکتا برای `clipPath`:

```
const uid = useMemo(() => 'm' + String(item.slug).replace(/^[a-z0-9]/gi, ''), [item.slug]);
```

چند میکس می‌توانند هم‌زمان صحنه خودشان را باز کرده باشند و شناسه `clipPath` در کل صفحه یکتا است، نه در هر SVG. **دسترس‌پذیری.** خود صحنه برای صفحه‌خوان یک `role="img"` با شرح کوتاه است؛ هر کاری که می‌شود کرد، دکمه و اهرم واقعی پایین است. یک ناحیه `aria-live="polite"` هم پیام هر مرحله را با لحن آرام می‌خواند:

```
// «مرحله ۲ از ۴ - کیسه‌ها یکی‌یکی در دستگاه خالی می‌شوند» stageMessage('pour')
```

و وقتی بسته می‌رسد، تمرکز صفحه‌کلید روی دکمه «افزودن به سبد» می‌نشیند — پایان مسیر همان‌جایی است که مشتری برایش آمده:

```
useEffect(() => {  
  if (stage === 'done') addRef.current?.focus({ preventScroll: true });  
}, [stage]);
```

چهار برچسب مرحله کنار هم در کارت باریک بریده می‌شد، پس نوار مرحله فقط شماره دارد به‌علاوه عنوان مرحله جاری، و کلاً `aria-hidden` است — خواندنش با صفحه‌خوان از راه همان پیام زنده انجام می‌شود. **کم‌حرکتی:** `reduced` یک بار خوانده می‌شود و بعد به تغییرش گوش داده می‌شود (کاربر می‌تواند وسط کار در سیستم‌عامل عوضش کند). با آن روشن، `start()` یک‌راست به مرحله ۴ می‌پرد.

فهرست افزودن دانه — BeanPicker.jsx

یک `<select>` کوچک که هر دو حالت از آن استفاده می‌کنند، تا قاعده «چه دانه‌ای می‌شود اضافه کرد» یک جا بماند.

```
<option key={b.slug} value={b.slug} disabled={out}>  
  {b.name} — {b.origin}{out ? ' (ناموجود)' : ''}  
</option>
```

دانه ناموجود از فهرست حذف نمی‌شود، خاموش می‌شود: نبودنش این حس را می‌داد که دیگر این قهوه را نداریم، در حالی‌که فقط موقتاً تمام شده. `option` با صفت `disabled` هم با صفحه‌کلید رد می‌شود و هم صفحه‌خوان «در دسترس نیست» را می‌خواند. تشخیص ناموجودی از `isSoldOut` در `groups.js` می‌آید:

```
export const isSoldOut = (item) => isTracked(item) && item.stock < minBuyable(item);
```

یعنی «موجودی شمرده می‌شود و از کمینه قابل‌خرید کمتر است» — ۵۰ گرم باقی‌مانده از قهوه‌ای که کمینه‌اش ۱۰۰ گرم است، عملاً ناموجود است.

CardArt.jsx

پل بین `art.js` و DOM:

```
export function artSVG(item) {
  if (!item) return '';
  try {
    if (item.kind === 'gear') return gearArt(item);
    if (item.kind === 'powder') return powderArt(item);
    return productArt(item);
  } catch {
    return ''; // طرح ناقص نباید کل صفحه را از کار بیندازد
  }
}
```

و انتخاب بین سه راه — که با نیمهٔ دوم مورد ۲۴ از دو راه به سه رسید:

```
if (item?.image) {
  const box = SLOTS[slot] || SLOTS.card;

  /* عکس برداری یا آدرس بیرونی: خام، همان‌طور که بود. */
  if (!isOptimizable(item.image)) {
    return <img className={className} src={item.image} alt={item.name} loading="lazy" />;
  }

  return (
    <Image className={className} src={item.image} alt={item.name}
      width={box.w} height={box.h} sizes={box.sizes} priority={priority} />
  );
}

return <span className="art-holder" dangerouslySetInnerHTML={{ __html: svg }} />;
```

چرا `next/image` . عکس آپلودی تا چهار مگابایت خام سرو می‌شد، در قابی که بزرگ‌ترینش ۳۸۵ پیکسل عرض دارد و کوچک‌ترینش ۷۰. یعنی مشتری موبایل عکسی را می‌گرفت که هیچ‌وقت بیش از یک بیستمش را نمی‌دید. حالا همان فایل از `/_next/image` رد می‌شود: به اندازهٔ همان قاب کوچک می‌شود، به AVIF یا WebP تبدیل می‌شود اگر مرورگر بگوید می‌فهمد، و روی دیسک کش می‌شود. عده‌های اندازه‌گیری‌شده در مورد ۲۴ فصل ۱۱ آمده.

چرا شرط `isOptimizable` . طرح تولیدی SVG و آدرس بیرونی هر دو از بهینه‌ساز پاسخ ۴۰۰ می‌گیرند — یعنی قاب خالی، نه عکس بهینه‌نشده. قاعده‌اش در `lib/img.js` است تا با بخش «دربارهٔ ما» یکی بماند.

جدول SLOTS — چرا `sizes` را جایگاه تعیین می‌کند نه کلاس

یک کلاس (`.card-art`) در پنج قاب با پنج اندازهٔ خیلی متفاوت استفاده می‌شود، پس خود کلاس نمی‌تواند به مرورگر بگوید کدام نسخه را بردارد. فراخوان جایگاهش را نام می‌برد و اندازه‌ها از اینجا می‌آیند:

```
const SLOTS = {
  card: { w: 385, h: 152,
    sizes: '(min-width: 1305px) 385px, (min-width: 975px) 30vw, '
      + '(min-width: 645px) 45vw, 92vw' },
  sheet: { w: 120, h: 96, sizes: '120px' }, // سرصفحهٔ صفحهٔ کالا و مودالش
  blend: { w: 78, h: 64, sizes: '78px' }, // کارت میکس
  thumb: { w: 70, h: 52, sizes: '70px' } // بندانگشتی سبد و فهرست پِنل
};
```

آن چهار پلهٔ `card` از خود چیدمان `style.css` درآمده‌اند، نه از حدس. شبکه‌اش `auto-fill` با ستون کمینهٔ ۲۸۵px و شکاف ۲۲px است، داخل `.wrap` که `min(1200px, 92%)` عرض دارد — یعنی تعداد ستون‌ها پله‌ای عوض می‌شود و عرض کارت با عرض پنجره خطی بالا نمی‌رود:

عرض پنجره	چیدمان	عرض کارت
$1305px \leq$	سه ستون	۳۸۵px ثابت
۱۳۰۵–۹۷۵	سه ستون	حدود ۳۰۷۷
۹۷۵–۶۴۵	دو ستون	حدود ۴۵۷۷
$645 >$	یک ستون	۹۲۷۷

عدهای درصدی مهم‌اند: Next از **کوچک‌ترین** آن‌ها تصمیم می‌گیرد کوچک‌ترین نسخه `srcset` چقدر باشد. با ۳۰۷۷، نسخه‌های ۲۵۶ و ۳۸۴ و ۴۴۸ پیکسلی هم ساخته می‌شوند – همان‌هایی که یک کارت واقعاً لازم دارد.

نکته

`w` و `h` فقط نسبت ابعاد را می‌دهند تا مرورگر پیش از رسیدن عکس جای درست را نگه دارد؛ اندازه واقعی را همچنان CSS تعیین می‌کند. هر دو بُعد داده می‌شوند، وگرنه Next هشدار «یک بُعد عوض شده» می‌دهد. قاب تازه‌ای که اضافه می‌کنید یا باید یکی از این چهار جایگاه را بگیرد یا ردیف خودش را در `SLOTS` – وگرنه بی‌صدا `sizes` کارت را می‌گیرد.

فقط جایی داده می‌شود که تصویر بالای صفحه است (سرفصله کالا و مودالش)؛ پیش‌فرض `next/image` خودش `priority` است. `lazy` است.

در `admin.css` کلاس `art-holder` با `display: contents` تعریف شده تا این `<span>` اضافی چیدمان را به هم نزند:

```
/* پوسته نامرئی دور تصویر تولیدی، تا چیدمان کارت عوض نشود */
.art-holder{ display:contents; }
```

ContentSections.jsx

پنج بخش که همه یک الگوی مشترک دارند:

```
const c = content?.whyUs;
if (!c || !Array.isArray(c.reasons) || c.reasons.length === 0) return null;
```

اگر محتوا نبود، بخش اصلاً رندر نمی‌شود – نه یک قاب خالی.

`SuggestSection` منطق جالبی دارد:

```
const chosen = (current.picks || []).map((s) => bySlug.get(s)).filter(Boolean);
if (chosen.length >= 3) return chosen.slice(0, 6);

const seen = new Set(chosen.map((i) => i.slug));
const extra = (byKind.coffee || []).
  .filter((i) => !seen.has(i.slug) && (i.tastes || []).includes(current.key))
  .sort((a, b) => a.rank - b.rank);

return [...chosen, ...extra].slice(0, 6);
```

اول انتخاب دستی مدیر، و اگر کمتر از سه تا بود، از روی برچسب `tastes` خود قهوه‌ها پر می‌شود. کامنت: «این‌طوری با اضافه شدن قهوه تازه، پیشنهادها هم زنده می‌مانند.»

PicksSections مستقیماً API را صدا می‌زند و به items وابسته است:

```
useEffect(() => { ... }, [items]);
```

با کامنت: «وگرنه کالایی که مدیر همین حالا پنهانش کرده تا بازگذاری بعدی صفحه اینجا می‌ماند.»

StaticSections.jsx

هفت کامپوننت ثابت. دو تای‌شان با صفحه تعامل می‌کنند:

```
// BrewSection
const pick = (card) => {
  setGrind(card.grind);
  toast(`تنظیم شد «${grindLabel(card.grind)}» آسیاب پیش‌فرض روی`);
  setCoffeeFilter(card.filter);
  setGearFilter(card.gear);
  document.getElementById('products')?.scrollIntoView({ behavior: 'smooth' });
};
```

یک کلیک، چهار کار: آسیاب پیش‌فرض، پیام، دو فیلتر و پیمایش.

Footer یک فرم خبرنامه صرفاً نمایشی دارد که فقط پیام محلی نشان می‌دهد و هیچ درخواستی نمی‌فرستد – برخلاف ClubSection که واقعاً ثبت می‌کند.

پیگیری سفارش – web/src/components/Track.jsx

۳۴۳ خط. تنها صفحه عمومی دوم سایت. مشتری حساب کاربری ندارد و بعد از بستن رسید هیچ راهی نداشت بفهمد سفارشش کجاست؛ اینجا با همان دو چیزی که دستش هست – شماره سفارش و شماره موبایلی که داده – وضعیت و محتوای سفارش را می‌بیند.

کد از آدرس پیش‌پر می‌شود. رسید در Receipt.jsx لینک می‌دهد:

```
<Link href={` /track?code=${encodeURIComponent(order.code)}`}>وضعیت سفارش را ببینید</Link>
```

و صفحه آن را برمی‌دارد، پس مشتری فقط شماره‌اش را می‌نویسد:

```
const [code, setCode] = useState(params.get('code') || '');
```

اعتبارسنجی روی همان چیزی که فرستاده می‌شود:

```
const c = toLatinDigits(code).trim();
const p = toLatinDigits(phone).trim();

if (!c) return setError('شماره سفارش را وارد کنید - روی رسید نوشته شده است');
if (!/^0\d{10}$/.test(p.replace(/[\s-]/g, ''))) {
  return setError('شماره موبایل را کامل و با ۰ اول وارد کنید، مثل ۰۹۱۲۱۲۳۴۵۶۷');
}
```

سرور هم همین نرمال‌سازی را دارد (normalizePhone)، ولی بررسی این طرف باید روی رشته نهایی باشد نه روی چیزی که کاربر دیده – وگرنه پیام خطا درباره متن می‌بود که اصلاً فرستاده نشده.

صفحه بین دو نوع شکست فرق نمی‌گذارد. سرور برای «کد وجود ندارد» و «شماره جور نیست» یک پاسخ می‌دهد، و این صفحه هم همان پیام را همان‌طور نشان می‌دهد – هیچ منطق اضافه‌ای که تفاوت را بازسازی کند اینجا نیست:

```
catch (err) { setError(err.message); }
```

نوار وضعیت. سه گام معمول (new → processing → done) به صورت یک با aria-current="step" روی گام جاری. «لغو شده» گام نیست، پس جداگانه و با role="status" نشان داده می‌شود:

```
if (status === 'canceled') {
  return <p className="track-canceled" role="status">
    این سفارش لغو شده است. اگر فکر می‌کنید اشتباهی رخ داده، با ما تماس بگیرید.
  </p>;
}
```

کلیدهای STEPS همان enum مدل Order اند و برچسب‌ها از ORDER_STATUS در groups.js می‌آیند – همان جدولی که پنل مدیریت هم از آن می‌خواند.

آنچه این صفحه نشان نمی‌دهد: نشانی، شماره تماس و یادداشت مشتری. این تصمیم سمت سرور گرفته شده (publicOrderView)، نه اینجا – یعنی حتی اگر کسی مستقیماً API را صدا بزند هم چیزی بیشتر نمی‌بیند.

چاپ، بدون اینکه صفحه دو رسید متفاوت پیدا کند

مشتری‌ای که رسیدش را بسته، هیچ راه دیگری برای گرفتن یک نسخه کاغذی نداشت. دکمه چاپ اینجا همان برگه‌ای را می‌دهد که کشوی سبد می‌داد – همان کامپوننت Receipt و همان سبک چاپ:

```
<div className="track-actions">
  <button className="btn btn-ghost" type="button" onClick={() => window.print()}>
    <svg viewBox="0 0 24 24" aria-hidden="true">...</svg>
  </button>
</div>

<PrintSheet>
  <Receipt
    order={order}
    shop={shop}
    heading="رسید سفارش"
    showMark={false}
    statusLabel={ORDER_STATUS[order.status] || order.status}
    showTracking={false}
  />
</PrintSheet>
```

چهار پارامتر همان چهار تفاوت جدول Receipt.jsx اند: عنوان به «رسید سفارش» عوض می‌شود، تیک سبز «ثبت شد» برداشته می‌شود (این سفارش ممکن است لغو شده باشد)، وضعیت اضافه می‌شود، و لینک «وضعیت سفارش را ببینید» می‌رود – مشتری همین حالا رویش ایستاده.

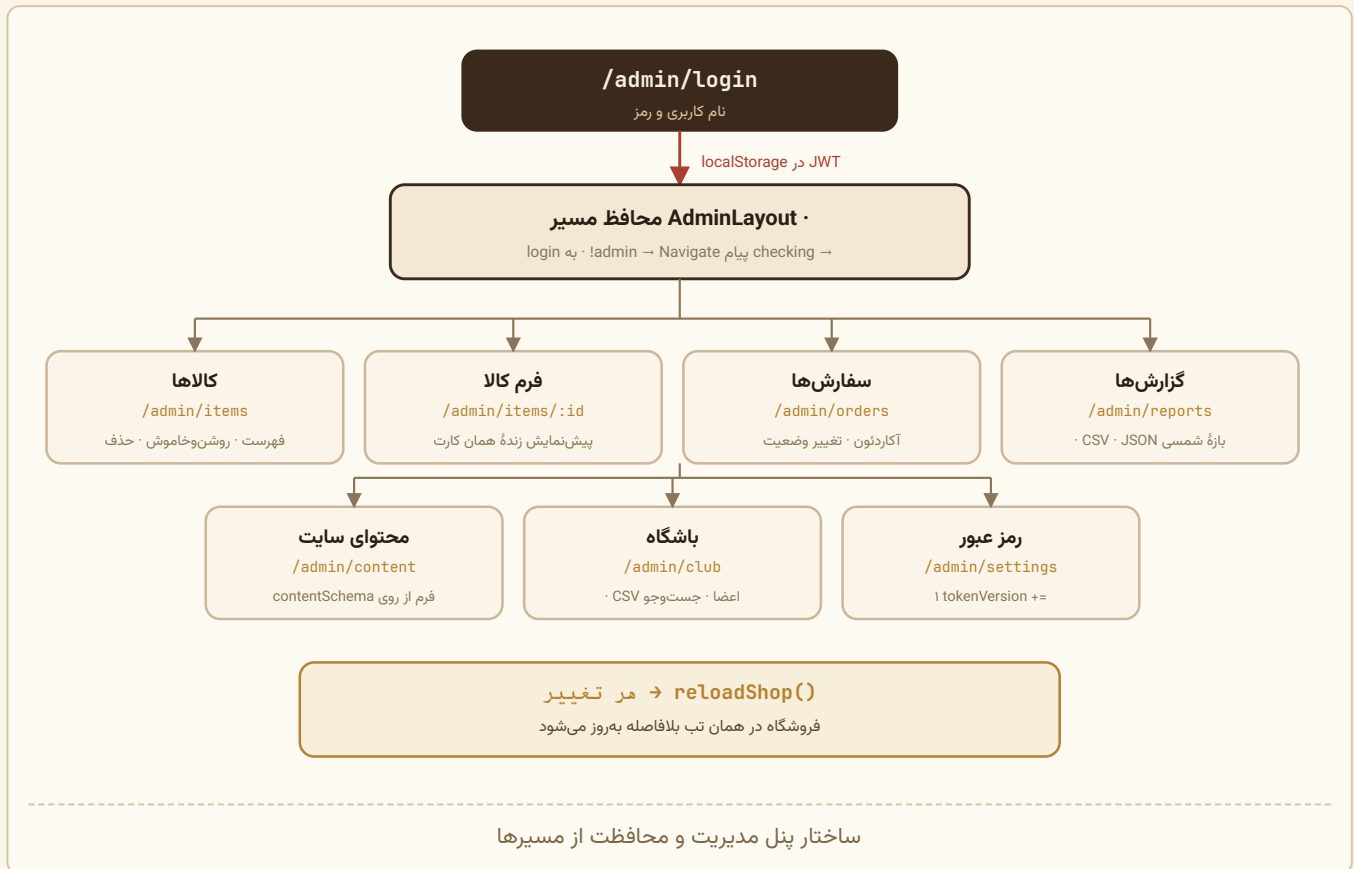
آنچه روی صفحه دیده می‌شود دست نخورده است. نوار مرحله و بلوک وضعیت مال صفحه‌اند و در PrintSheet نیستند؛ برگه چاپی جای خودش را دارد.

و یک درخواست دوم که شکست خوردنش مهم نیست:

```
const [found, texts] = await Promise.all([
  api.trackOrder(c, p),
  api.content().catch(() => null)
]);
setOrder(found);
setShop(texts?.about || null);
```

`content()` فقط برای بلوک تماس برگه چاپی لازم است – تلفن و نشانی و ساعت کار فروشگاه. `catch(() => null)` عمدی است: نیامدنش نباید پیگیری را زمین بزند، همان قاعده‌ای که `ShopContext` هم دربارهٔ متن‌ها دارد.

بخش د – وب: پنل مدیریت



AdminShell.jsx

نوار بالا با شش لینک، دروازه نشست، و `children`. در نسخه ویت نامش `AdminLayout` بود و `<Outlet />` رندر می‌کرد؛ حالا فقط یک خط است که همین را دوباره صادر می‌کند.

سه چیز در جابه‌جایی عوض شد:

۱) دروازه در افکت است، نه در رندر. App Router کامپوننتی مثل `<Navigate>` ندارد و هدایت نباید حین رندر انجام شود:

```
useEffect(() => {
  if (!checking && !admin) router.replace('/admin/login');
}, [checking, admin, router]);

if (checking || !admin) return <div className="admin-boot">بررسی نشست</div>;
```

تا رفتن کاربر همان پیام «بررسی نشست» دیده می‌شود – نه پنلی که یک لحظه برق بزند و بعد برود.

۲) لینک فعال با `usePathname` سنجیده می‌شود. `NavLink` در `react-router` خودش می‌دانست کدام مسیر فعال است. اینجا `startsWith` لازم است، چون فرم کالا (`/admin/items/new`) و (`/admin/items/:id`) زیرمسیر «کالاها» است و باید همان دکمه را فعال نشان بدهد.

۳) `ShopProvider` فقط دور محتوا می‌پیچد، نه دور نوار بالا – و با پرچم `loadOnMount`، چون صفحه‌های پنل سروری نیستند که کسی فهرست کالاها را برایشان آماده کند. نوار بالا به فهرست کالاها کاری ندارد.

فرم ساده. نکته خوب: `dir="ltr"` روی هر دو input و `autoComplete="username"` / `"current-password"` برای password manager.

فهرست با فیلتر نوع (سرور) + فیلتر دسته (کلاینت) + جست‌وجو (سرور، ۲۵۰ms)

- هفت گزینه مرتب‌سازی (کلاینت).

کامنت توضیح می‌دهد چرا تقسیم شده: «فیلتر دسته و ترتیب نمایش – هر دو سمت مرورگر انجام می‌شوند؛ فهرست کالاها کوچک است و رفت‌وبرگشت لازم ندارد.»

هر عملیات مخرب تأیید دومرحله‌ای دارد:

```
{confirmId === item._id ? (
  <span className="confirm">
    <button className="admin-btn danger" onClick={() => remove(item)}>حذف قطعی</button>
    <button className="admin-btn" onClick={() => setConfirmId(null)}>انصراف</button>
  </span>
) : (
  <button className="admin-btn danger" onClick={() => setConfirmId(item._id)}>حذف</button>
)}
```

بدون `window.confirm` – تأیید درون‌خطی و قابل استایل‌دهی.

ستون «موجودی» هم در همین جدول است و سه حالت را به زبان مدیر می‌گوید – `stockText(item)` یا «نامحدود» می‌دهد یا «۷۵۰ گرم» یا «۳ عدد»، و ردیف ناموجود کلاس هشدار می‌گیرد.

شش `fieldset`: نوع و دسته · معرفی کالا · معرفی کامل · میکس (فقط قهوه) · ویتترین · قیمت و ترتیب (شامل موجودی) · تصویر کارت.

کادر موجودی، سه‌حالته. برچسبش با نوع کالا عوض می‌شود («موجودی انبار (گرم)» یا «(عدد)») و خالی بودن معنا دارد:

```
/* موجودی: کادر خالی یعنی نامحدود (null)، نه صفر */
const stockRaw = toLatinDigits(form.stock).trim();
const stock = stockRaw === '' ? null : Math.round(Number(stockRaw));
if (stock !== null && (!Number.isFinite(stock) || stock < 0)) {
  return setError('موجودی باید عددی مثبت باشد، یا خالی بماند برای نامحدود');
}
```

لازم است چون مدیر ممکن است با کیبورد فارسی «۷۵۰» بنویسد. `toLatinDigits`

راهنمای زیر کادر برای میکس‌ها متن دیگری می‌گوید: «برای میکس‌ها معمولاً خالی درست‌تر است: میکس از همان دانه‌ها ساخته می‌شود و موجودی جدایی ندارد» – همان دام شناخته‌شده‌ای که در فصل ۱۱ هم آمده است.

ویژگی برجسته: پیش‌نمایش زنده. یک شیء موقتی ساخته می‌شود و به همان `ItemCard` واقعی سایت داده می‌شود:

```
const preview = useMemo(() => ({
  _id: 'preview',
  kind: form.kind,
  slug: form.slug || 'preview',
  name: form.name || 'نام کافه',
  price: Number(toLatinDigits(form.price)) || 0,
  notes: String(form.notes).split('\n').map((s) => s.trim()).filter(Boolean),
  ...
  grindable: form.kind === 'coffee',
  isBlend: form.kind === 'coffee' && form.isBlend,
}), [form, meta]);
```

با کامنت: «تا پیش‌نمایش دقیقاً مثل سایت رفتار کند.» چون کامپوننت یکی است، هیچ‌وقت پیش‌نمایش با واقعیت فرق نمی‌کند.

فهرست دانه‌ها از مسیر مدیریتی خوانده می‌شود:

```
api.adminItems({ kind: 'coffee' })
  .then((list) => setBeans(Array.isArray(list) ? list : []).filter((b) => !b.isBlend)))
```

کامنت: «از مسیر مدیریتی خوانده می‌شود تا قهوه‌های خاموش هم دیده شوند — مدیر ممکن است دانه‌ای را موقتاً از سایت برداشته باشد ولی هنوز در میکس داشته باشد.» و در `<option>` هم علامت می‌خورد: `{b.name}{b.active ? '' : '(خاموش)'}`.

دسته پیش‌فرض هنگام تغییر نوع:

```
useEffect(() => {
  const meta = KINDS[form.kind];
  if (!meta.groups[form.group]) {
    setForm((f) => ({ ...f, group: meta.order[0],
      shape: f.kind === 'gear' ? 'dripper' : f.kind === 'powder' ? 'scoop' : '' }));
  }
}, [form.kind]);
```

جلوگیری از حالت نامعتبر: اگر کاربر از «قهوه/light» به «ابزار» برود، `light` دسته معتبری برای ابزار نیست.

`OrderDetail.jsx` و `AdminOrders.jsx`

`AdminOrders` فهرست آکاردئونی با چیپ‌های وضعیت. `OrderDetail` قطعه مشترکی است که هم اینجا و هم در مودال گزارش‌ها استفاده می‌شود — کامنت بالایش می‌گوید چرا:

هم صفحه «سفارش‌ها» و هم پنجره «گزارش‌ها» از همین یک قطعه استفاده می‌کنند تا هیچ‌وقت یکی‌شان اطلاعاتی را نشان ندهد که آن یکی نمی‌دهد.

`MixBreakdown` وزن هر دانه را برای کارگاه محاسبه می‌کند:

```
{grams ? <small className="mix-grams">{formatWeight(Math.round((grams * m.percent) / 100))}</small> :
null}
```

کامنت: «کارگاه باید بداند دقیقاً از هر دانه چند گرم بریزد.»

چهار بخش: کارت‌های KPI · جدول بازه‌ها با میله سهم · فهرست سفارش‌ها · جعبه خروجی.

نمودار میله‌ای بدون کتابخانه:

```
const maxRevenue = Math.max(1, ...buckets.map((b) => b.revenue));
...
<span className="bar" aria-hidden="true">
  <i style={{ inlineSize: `${Math.round((b.revenue / maxRevenue) * 100)}%`} />
</span>
```

`Math.max(1, ...)` تقسیم بر صفر را می‌بندد. `inlineSize` به جای `width` یعنی در RTL هم از راست پر می‌شود.

دسترس‌پذیری ردیف‌های کلیک‌پذیر:

```
<tr tabIndex={0} role="button" aria-pressed={active}
  onClick={...}
  onKeyDown={(e) => { if (e.key === 'Enter' || e.key === ' ') { e.preventDefault(); ... } } }>
```

بازخوانی نسخه تازه هنگام باز کردن سفارش:

```
const openOrder = async (order) => {
  setOpen(order); // فوری، از فهرست
  try {
    const fresh = await api.order(order._id);
    setOpen((cur) => (cur && cur._id === fresh._id ? fresh : cur));
  } catch { /* همان نسخه فهرست را نشان می‌دهیم */ }
};
```

الگوی stale-while-revalidate: فوراً چیزی نشان بده، بعد به‌روزش کن. شرط `cur._id === fresh._id` از race condition جلوگیری می‌کند (اگر کاربر سریع سفارش دیگری باز کند).

contentSchema.js + AdminContent.jsx

فرم از روی توصیف داده ساخته می‌شود. `contentSchema.js` شش نوع فیلد تعریف می‌کند: `check`، `textarea`، `text`، `list`، `lines`، `select`.

کامپوننت `Field` هر نوع را رندر می‌کند و `ListField` فهرست موردهای چندفیلدی را با دکمه‌های افزودن/حذف/بالا/پایین:

```
const move = (i, dir) => {
  const j = i + dir;
  if (j < 0 || j >= rows.length) return;
  const next = [...rows];
  [next[i], next[j]] = [next[j], next[i]];
  onChange(next);
};
```

کامنت: «ترتیب همان‌طور که اینجاست در سایت دیده می‌شود.»

هر بخش دکمه ذخیره مستقل دارد: «هر بخش جدا ذخیره می‌شود تا اگر وسط کار پشیمان شدید، بقیه دست‌نخورده بمانند.»

فایده معماری این طراحی: اضافه کردن یک فیلد تازه به سایت فقط دو فایل عوض می‌کند – `default-content.js` (سرور) و `contentSchema.js` (کلاینت) – و فرم پنل خودکار به‌روز می‌شود.

`AdminSettings.jsx` و `AdminClub.jsx`

`AdminClub`: جدول اعضا با جست‌وجوی ۲۰۰ms، حذف با تأیید، دکمه CSV.

`AdminSettings`: تغییر نام کاربری و رمز با اعتبارسنجی سمت کلاینت، چک‌باکس «نمایش رمزها»، و پیام روشن درباره اثر تغییر:

بعد از تغییر رمز، دستگاه‌های دیگری که با رمز قبلی وارد شده‌اند از پنل بیرون می‌آیند. همین مرورگر باز می‌ماند.

که دقیقاً همان رفتار `tokenVersion` را به زبان کاربر توضیح می‌دهد.

vitest.config.mjs

```
export default defineConfig({
  test: {
    environment: 'node',
    include: ['tests/**/*.test.{js,jsx}'],
    env: { TZ: 'Asia/Tehran' }
  }
});
```

پیش‌فرض است چون تقریباً هیچ تستی به DOM نیاز ندارد – حتی `blend-visual.test.js` که `environment: 'node'` را می‌سنجد. ظاهراً دربارهٔ انیمیشن است، فقط عددهای خالص `blendVisual.js` را می‌سنجد.

سه استثنا هست و هر سه عمدی‌اند: `item-card.test.jsx`، `cart-hydration.test.jsx` و `item-detail.test.jsx` با کامنت `// @vitest-environment jsdom` بالای خودشان محیط را عوض می‌کنند – همان‌جا، نه اینجا، تا بقیهٔ فایل‌ها بی‌جهت داخل `jsdom` اجرا نشوند. JSX هم پیکربندی نمی‌خواهد: ترنسفورمر ویت‌تست ۴ خودش ران‌تایم خودکار را می‌گیرد.

`share.test.js` و `toast-store.test.js` عمداً در این فهرست نیستند، با اینکه هر دو دربارهٔ رفتار مرورگرند. `share.js` وابستگی‌هایش را پارامتر می‌گیرد و `toastStore.js` یک انبارهٔ ساده است، پس هر دو در محیط `node` اجرا می‌شوند – همان قاعدهٔ ۹.

`TZ` ثابت شده چون گزارش‌ها روی وقت تهران بسته می‌شوند؛ بدون آن، `jalali.test.js` روی ماشین CI (که UTC است) نتیجهٔ دیگری می‌داد.

نکته

در دوران مهاجرت این پیکربندی دو «پروژه» داشت: `client` روی ری‌اکت ۱۸ و `web` روی ۱۹. تستی که کامپوننت رندر می‌کند باید رندرکننده و کامپوننت را از یک نسخه بگیرد، پس یک پروژهٔ جدا با `alias` لازم بود. با برداشته شدن `client` آن تقسیم هم برداشته شد.

– بیست‌ودو فایل، ۳۸۰ سنجه `tests/**/*.test.{js,jsx}`

هیچ‌کدام به پایگاه داده دست نمی‌زنند. جایی که کد واقعاً به مونگو نیاز دارد، مدل تزریق می‌شود و تست یک بدل چند خطی می‌سازد:

```
Item مدل - tests/stock.test.js //
async findOneAndUpdate(filter, update) {
  const row = rows.find((r) => r.slug === filter.slug);
  if (!row) return null;
  /* شرط gte$ عمداً با null جور نمی‌شود - همان رفتار مونگو */
  const need = filter.stock.$gte;
  if (typeof row.stock !== 'number' || row.stock < need) return null;
  row.stock += update.$inc.stock;
  return { slug: row.slug, stock: row.stock };
}
```

بدل عمداً همان ضمانت مهم مونگو را تقلید می‌کند و نه بیشتر: شرط و کاهش یکجا. کامنت خودِ فایل می‌گوید چرا این کافی است – جاوااسکریپت تکنخی است، پس هر عملیات هم‌زمان‌ناپذیر است، دقیقاً مثل تکی سند مونگو که در لحظهٔ به‌روزرسانی قفل است.

حتی جزئیاتی مثل شکل بازگشتی هم رعایت شده: `findOne` عمداً `async` نیست، چون کد واقعی `findOne(...).lean()` می‌نویسد و باید شیئی با متد `lean` بگیرد، نه یک `Promise`.

چند نمونه از چیزهایی که این تست‌ها واقعاً نگه می‌دارند:

فایل	یک ادعای نمونه
<code>pricing.test.js</code>	«مرز ۳۰۰۰ گرم: ۲۹۹۹ هنوز ۵٪ است، ۳۰۰۰ می‌شود ۱۰٪»
<code>blend.test.js</code>	«هزار حرکت تصادفی هم مجموع را خراب نمی‌کند»
<code>blend-visual.test.js</code>	«دهانه هر کیسه – کوچک یا بزرگ – دقیقاً سر قیف می‌ایستد»
<code>card-art.test.js</code>	«نام نمی‌تواند تگ را ببندد و <code><script></code> باز کند»
<code>stock.test.js</code>	«شکست ردیف سوم، دو ردیف اول را پس می‌دهد»
<code>track.test.js</code>	«کد درست با شماره غلط، دقیقاً همان پاسخ کد ناموجود را می‌دهد»
<code>upload.test.js</code>	«تصویر عادی داندلود اجباری نمی‌شود – وگرنه کارت‌ها می‌شکستند»
<code>taxonomy.test.js</code>	«همان آرایه است، نه آرایه‌ای شبیه آن»
<code>item-routes.test.js</code>	« <code>itemPath</code> هیچ‌وقت با / تمام نمی‌شود»
<code>next-routes.test.js</code>	«هر <code>segment</code> در <code>KIND_SEGMENTS</code> یک پوشه مسیر دارد، و هر پوشه یک <code>segment</code> »
<code>next-metadata.test.js</code>	«هر تگی که <code>headTags</code> می‌سازد در شیء <code>metadata</code> پیدا می‌شود»
<code>json-ld.test.js</code>	«نام کالایی مثل <code></script></code> از <code><script></code> بیرون نمی‌زند»
<code>sitemap.test.js</code>	«کالای خاموش در نقشه سایت نمی‌آید»
<code>cart-storage.test.js</code>	«شناسه ردیف از نو ساخته می‌شود، نه از حافظه خوانده»
<code>share.test.js</code>	«کروم ویندوز <code>navigator.share</code> دارد، ولی آنجا هم باید کپی شود»
<code>toast-store.test.js</code>	« <code>getServerToast</code> حتی وقتی پیامی روی میز باشد خالی است»
<code>cart-hydration.test.jsx</code>	«سبد ذخیره‌شده بعد از <code>hydrate</code> هنوز سر جایش است»
<code>item-card.test.jsx</code>	«کارت هر نوع کالا <code><a href></code> واقعی دارد»
<code>item-detail.test.jsx</code>	«مودال بدون <code>ShopProvider</code> رندر می‌شود»

سه فایل از این‌ها با مورد ۲۶ اضافه شدند و هدفشان یک چیز است: نگه داشتن چیزی که دیده نمی‌شود. یک لینک شکسته را آدم می‌بیند؛ سبدي که بی‌صدا پاک شود را نه.

سه فایل تازه‌تر هم دقیقاً همان منطق را دنبال می‌کنند:

- `share.test.js` یک دام مشخص را می‌بندد. یک روز شرطِ دوراهی هم‌رسانی فقط `typeof nav.share === 'function'` بود، با این فرض که «تقریباً هیچ مرورگر دسکتاپی share ندارد». کروم ویندوز آن را دارد، ولی برگه‌ای که باز می‌کند بی‌کار بسته می‌شود و در آن مسیر هیچ‌وقت کاری به کلیک‌بورد نمی‌رسید – یعنی کاربر دسکتاپ دکمه را می‌زد، پنبلی می‌آمد و می‌رفت، و لینک هیچ‌جا کپی نشده بود. چهار تستِ بخشی «دسکتاپی که `navigator.share` دارد» تنها چیزی‌اند که جلوی برگشتنِ آن رفتار را می‌گیرند.
- `toast-store.test.js` دو خرابیِ نادیدنی را می‌گیرد: پیامی که پاک نشود و تا ابد بماند، و پیام دومی که شمارشِ اولی را به ارث ببرد.
- `item-detail.test.jsx` سومین استثنای قاعدهٔ ۹ است و دامنه‌اش یک چیز بیشتر نیست: مودال باید بدون هیچ Provider ای رندر شود. اگر روزی `ShareButton` دوباره سراغ `useShop` برود، همه‌چیز در صفحهٔ اصلی و صفحهٔ کالا سبز می‌ماند و فقط مودال می‌ترکد – یعنی راهی که بیشترِ بازدیدکننده‌ها کالا را از آن می‌بینند.
- `card-art.test.js` یک ویژگی کمیاب دارد: خودِ **سنجه را هم می‌سنجد**. یک تست عمداً رشتهٔ `escape` می‌سازد و انتظار دارد سنجه شکست را ببیند – تا اگر روزی `regex` بررسی خراب شد، بقیهٔ تست‌ها بی‌صدا سبز نمانند.
- `taxonomy.test.js` هم به `toBe` تکیه می‌کند نه `toEqual`: ترتیب‌های `UI` باید همان آرایهٔ `shared` باشند، نه کپی برابرِ آن. یک کپی امروز برابر است و فردا واگرا می‌شود.

9 `eslint.config.mjs` و `.prettierrc`

- `flat config` با سه ناحیه: `Node` در `server/` و `shared/`، `کد مرورگر` در `web/` (با `eslint-plugin-react` و `react-hooks`)، و تست‌ها. `eslint-config-prettier` آخر صف می‌نشیند تا قاعده‌های سلیقه‌ای قالب‌بندی با Prettier دعوا نکنند.
- در `.prettierrc` پنج دستهٔ کنارگذاشته وجود دارد و هر پنج کامنت دارند: نسخهٔ ایستای ریشه (که امروز فقط `img/` از آن مانده – مورد ۱۸)، دو فایل `seed` (هر ردیف عمداً یک خط بلند است)، خروجی ساخته‌شدهٔ `docs/documentation.html`، چهار فایل ستون‌تراز `scripts/docs/`، و همهٔ `*.md` – چون تنها کار Prettier با متن فارسی هم‌تراز کردن ستون‌های جدول است و با پهنای حروف فارسی نتیجه به‌هم‌ریخته‌تر می‌شود.
- جاهایی که ستون‌بندی دستی معنا دارد ولی فایل در `ignore` نیست، با `// prettier-ignore` علامت خورده‌اند: جدول `TIERS`، جدول‌های `shared/taxonomy.js`، `WRITABLE` در روتر کالا، و `schema` های ستون‌تراز در مدل‌ها.

۶. جریان‌های اصلی کاربر

این فصل ده جریان کامل را از اولین کلیک تا آخرین درج در پایگاه داده دنبال می‌کند، و با نمودار وضعیت سفارش تمام می‌شود. برای هر مرحله، فایل و تابع دقیق ذکر شده است.

سه جریان با مورد ۲۶ و ۲۷ و ۳۵ عوض شدند یا تازه‌اند و علامت خورده‌اند: بارگذاری اولیه (۶.۱)، صفحه کالا و مودالش (۶.۳)، و پیگیری سفارش (۶.۶). جریان هم‌رسانی کالا (۶.۱۱) از همه تازه‌تر است، و گام چاپ رسید در ۶.۵ از پایه بازنویسی شد.

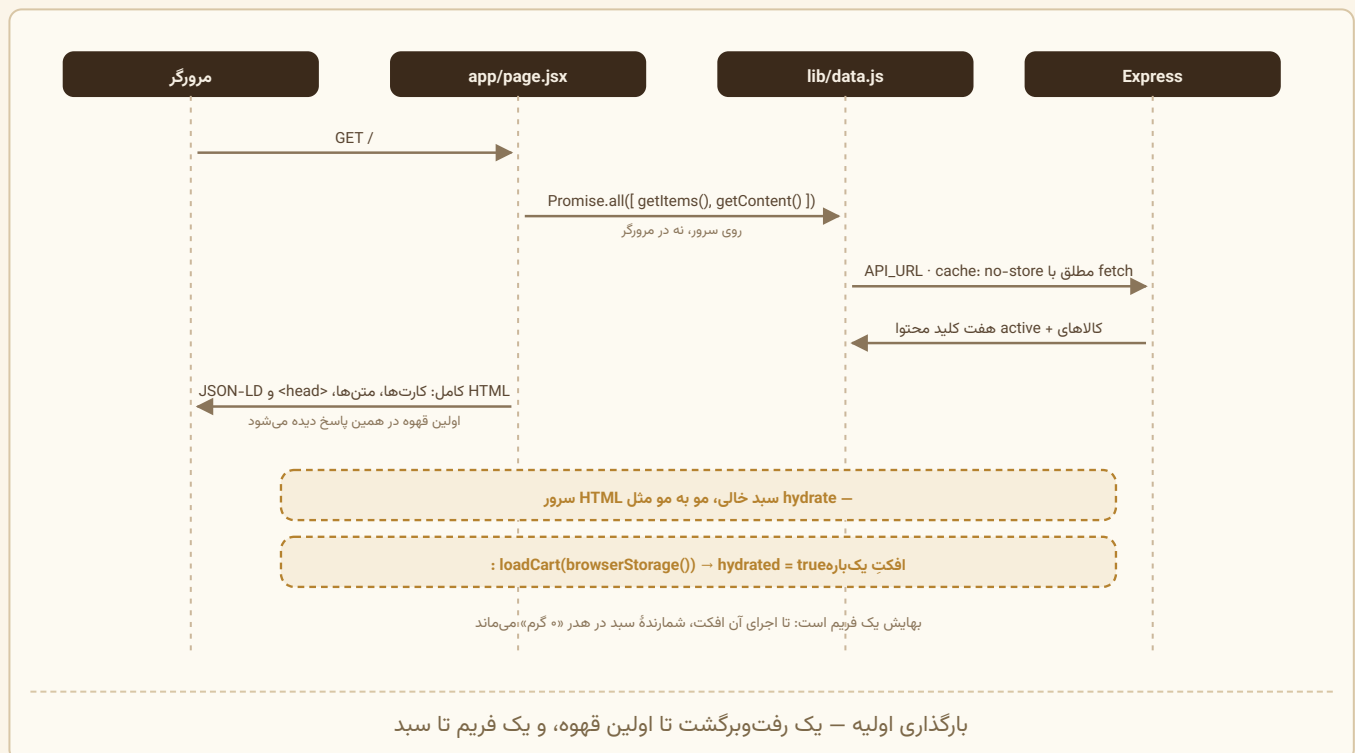
۶.۱ جریان اول – بارگذاری اولیه صفحه

این جریان بیش از هر جریان دیگری با مورد ۲۶ عوض شد، پس اول تفاوت را صریح بگوییم.

پیش‌تر (ویت): مرورگر یک `index.html` تقریباً خالی می‌گرفت، بعد باندل جاوااسکریپت را دانلود و اجرا می‌کرد، بعد `ShopContext` موقع mount دو درخواست به `/api/items` و `/api/content` می‌فرستاد، و بعد اولین قهوه دیده می‌شد. سه رفت‌وبرگشت.

حالا: همان دو درخواست روی سرور انجام می‌شوند و نتیجه‌شان داخل همان HTML اول است. یک رفت‌وبرگشت.

نمودار توالی



مرحله به مرحله

(۱) `web/src/proxy.js` پیش از هر چیز اجرا می‌شود: یک `nonce` تازه می‌سازد، روی هدر درخواست می‌گذارد (تا Next روی اسکریپت‌های خودش بنشاندش) و سیاست امنیتی را روی پاسخ.

(۲) `web/src/app/layout.jsx` پوسته صفحه را می‌سازد: `lang="fa"`، `dir="rtl"` (هر دو از `shared/seo.js`)، کلاس‌های فونت `next/font` روی `<html>`، و دو شکاف `children` و `modal`.

```
export default async function HomePage() {
  /* هر دو با هم، نه یکی پس از دیگری */
  const [items, content] = await Promise.all([getItems(), getContent()]);
  const nonce = (await headers()).get('x-nonce') || undefined;

  const orgJsonLd = content?.about ? asJsonLd(organizationSchema(content)) : '';

  return (
    <ShopProvider initialItems={items} initialContent={content}>
      <HomeShell />
      {orgJsonLd ? (
        <script type="application/ld+json" nonce={nonce}
          dangerouslySetInnerHTML={{ __html: orgJsonLd }} />
      ) : null}
    </ShopProvider>
  );
}
```

سه چیز در همین چند خط:

- داده به شکل `prop` از مرز سروری به مشتری رد می‌شود – همان چیزی که تا دیروز با `fetch` در مرورگر گرفته می‌شد.
- `LocalBusiness` اگر مدیر بخش «درباره ما» را پر نکرده باشد اصلاً منتشر نمی‌شود: نشانی خالی در داده ساختاریافته از نبودش بدتر است.
- `nonce` از همان چیزی می‌آید که `proxy.js` روی درخواست گذاشته. بدون آن، سیاست امنیتی این `<script>` را هم می‌بست – هرچند داده است و اجرا نمی‌شود.

(۴) `web/src/lib/data.js` روی سرور می‌خواند، با آدرس مطلق و `cache: 'no-store'`:

```
export const getItems = cache(async (kind = '') => {
  const res = await fetch(`${API_URL}/api/items${kind ? `?kind=${kind}` : ''}`, FETCH_OPTS);
  if (!res.ok) throw new Error(`خطای سرور: ${res.status}`);
  return res.json();
});
```

`cache` از ری‌اکت است، نه کش HTTP: در طول یک درخواست هر بار که همین تابع صدا زده شود فقط یک بار از سرور پرسیده می‌شود. برای صفحه اصلی مهم نیست؛ برای صفحه کالا هست، چون `generateMetadata` و خود صفحه هر دو همان کالا را می‌خواهند.

(۵) سرور – `GET /api/items`:

```
const filter = { active: true };
if (req.query.kind) filter.kind = req.query.kind;
const items = await Item.find(filter).sort({ rank: 1, name: 1 }).lean({ virtuals: true });
```

و `GET /api/content` که هفت کلید را با fallback به `DEFAULT_CONTENT` برمی‌گرداند. نبود متن‌ها صفحه را زمین نمی‌زند: `getContent` در خطا `{}` می‌دهد.

(۶) `ShopProvider` دیگر خودش نمی‌خواند. `initialItems` و `initialContent` را می‌گیرد، `loading` از همان اول `false` است، و ساختارهای مشتق ساخته می‌شوند: `bySlug` (Map)، `byKind` (سه آرایه)، `houseBlends` (میکس‌های `house`، مرتب با `rank`).

شاخه‌های «در حال خواندن...» حذف نشده‌اند، چون `reload()` هنوز وجود دارد و پس از آن ممکن است لازم شوند. ولی در بارگذاری اولیه هیچ‌وقت دیده نمی‌شوند.

۷) `HomeShell.jsx` – `'use client'`، چون فیلترهای دسته بالای چند بخش زندگی می‌کنند. این به معنای «سمت مشتری رندر شدن» نیست: متن همه بخش‌ها داخل همان HTML اول است.

۸) **hydration** – حساس‌ترین نقطه این جریان. سبد همیشه با آرایه خالی شروع می‌شود، پس HTML سرور و اولین رندر مرورگر مو به مو یکی‌اند. خواندن واقعی در یک افکت یک‌باره است:

```
useEffect(() => {
  const saved = loadCart(browserStorage());
  if (saved.length) setCart(saved);
  setHydrated(true);
}, []);
```

بهایش یک فریم است: تا اجرای این افکت، شمارنده سبد در هدر «گرم» نشان می‌دهد. عمداً با `suppressHydrationWarning` پنهان نشده – آن پرچم هشدار را خاموش می‌کند، نه مسئله را. سه نگره‌بانی که این ترتیب لازم دارد در ۷.۱۲ خطبه‌خط آمده‌اند.

۹) **نشست مدیر اینجا نیست**. در نسخه ویت `AuthProvider` دور کل برنامه می‌پیچید و همان اول `api.me()` را صدا می‌زد. حالا فقط در `app/admin/layout.jsx` است، پس بازدیدکننده فروشگاه هیچ درخواستی برای بررسی نشست نمی‌فرستد و کد پنل هم برایش دانلود نمی‌شود.

۱۰) **رفع باگ لنگر** – جزئیاتی که ماند، ولی دلیلش عوض شد:

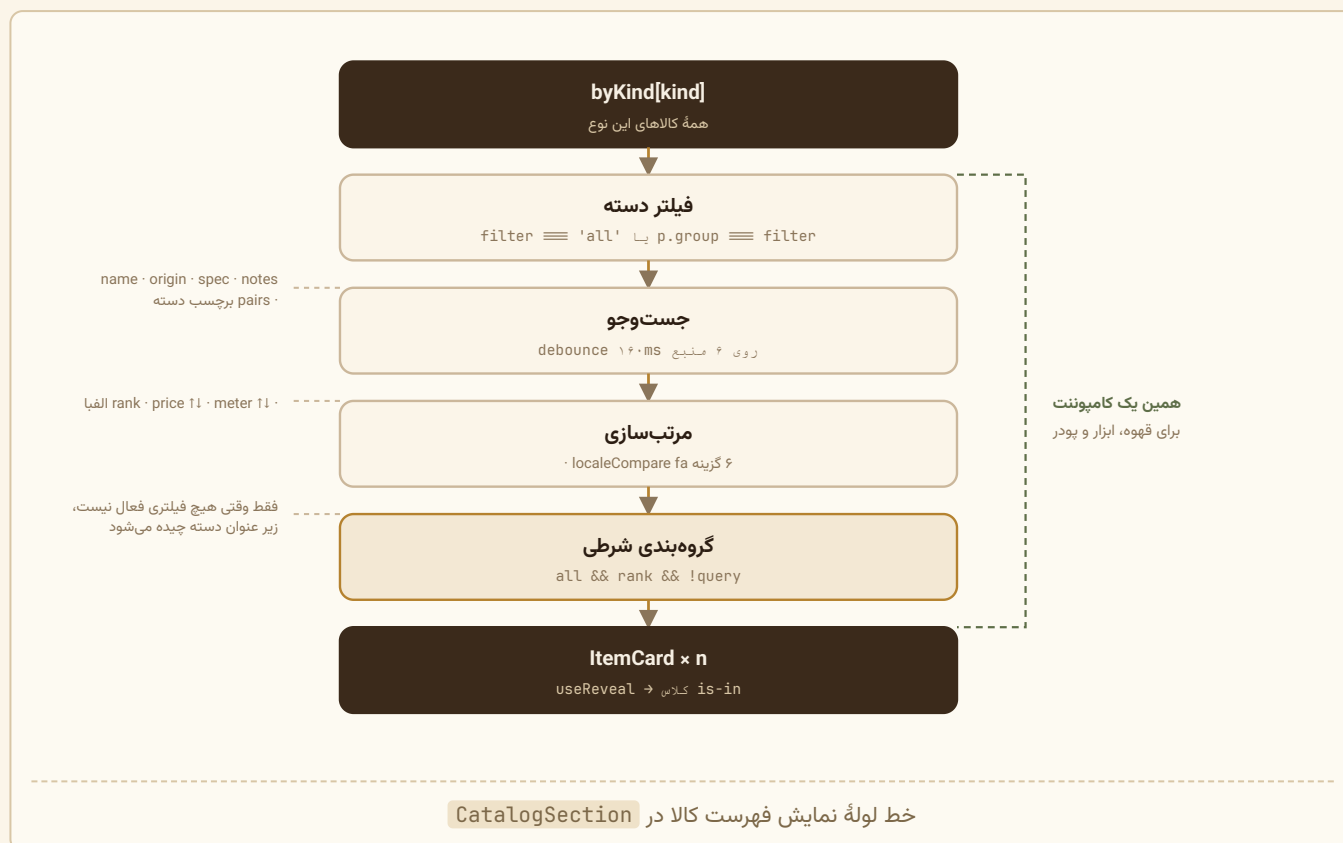
```
useEffect(() => {
  if (loading || loadError) return;
  const id = window.location.hash.slice(1);
  if (!id) return;
  const t = setTimeout(() => {
    document.getElementById(id)?.scrollIntoView({ behavior: 'auto' });
  }, 60);
  return () => clearTimeout(t);
}, [loading, loadError]);
```

کامنت `HomeShell.jsx` تفاوت را می‌گوید:

اگر آدرس با لنگر باز شده باشد (مثل `/#cafe`)، مرورگر همان اول صفحه را جابه‌جا می‌کند. در نسخه ویت آن لحظه هنوز فهرست کالاها نیامده بود و بخش‌ها کوتاه‌تر بودند، پس جای اشتباهی می‌ایستاد. حالا داده از همان اول در HTML است و بخش‌ها ارتفاع نهایی خودشان را دارند – ولی تصویرها ممکن است هنوز نرسیده باشند، پس همان یک فریم صبر سر جایش می‌ماند.

یعنی همان شصت میلی‌ثانیه ماند، ولی از «منتظر داده» به «منتظر چیدمان تصویرها» تبدیل شد.

۶.۲ جریان دوم – مرور، فیلتر و افزودن یک قهوه ساده



جزئیات کلیدی

فیلتر و جست و جو در `web/src/components/CatalogSection.jsx` (تابع `visible` در `useMemo`) – تحلیل خطبه خط در فصل ۵.

افزودن به سبد در `web/src/context/ShopContext.jsx`. خودِ شکلی ردیف دیگر اینجا ساخته نمی شود – از `lib/cartLine.js` می آید:

```
const addWeighed = useCallback((slug, grams, opts = {}) => {
  const item = bySlug.get(slug);
  if (!item) return;

  const line = buildLine(item, grams, opts, grind);

  setCart((prev) => {
    const i = prev.findIndex((l) => l.key === line.key);
    if (i === -1) return [...prev, line];
    const next = [...prev];
    next[i] = { ...next[i], grams: next[i].grams + grams };
    return next;
  });

  toast(`به سبد اضافه شد ${formatWeight(grams)} ${item.name}`);
}, [bySlug, grind, toast]);
```

و `buildLine` همان سه تصمیم قدیمی را دارد، این بار به شکل تابع خالص:

```
export function buildLine(item, grams, opts = {}, fallbackGrind = '') {
  /* آسیاب فقط برای قهوه معنی دارد؛ پودر آسیاب نمی‌خورد */
  const grind = item.grindable ? (opts.grind ?? fallbackGrind) : '';
  const mix = normalizeMix(item, opts.mix);

  return { key: lineKey(item.slug, grind, mix), slug: item.slug, kind: item.kind,
    grams, qty: 0, grind, mix };
}
```

سه تصمیم:

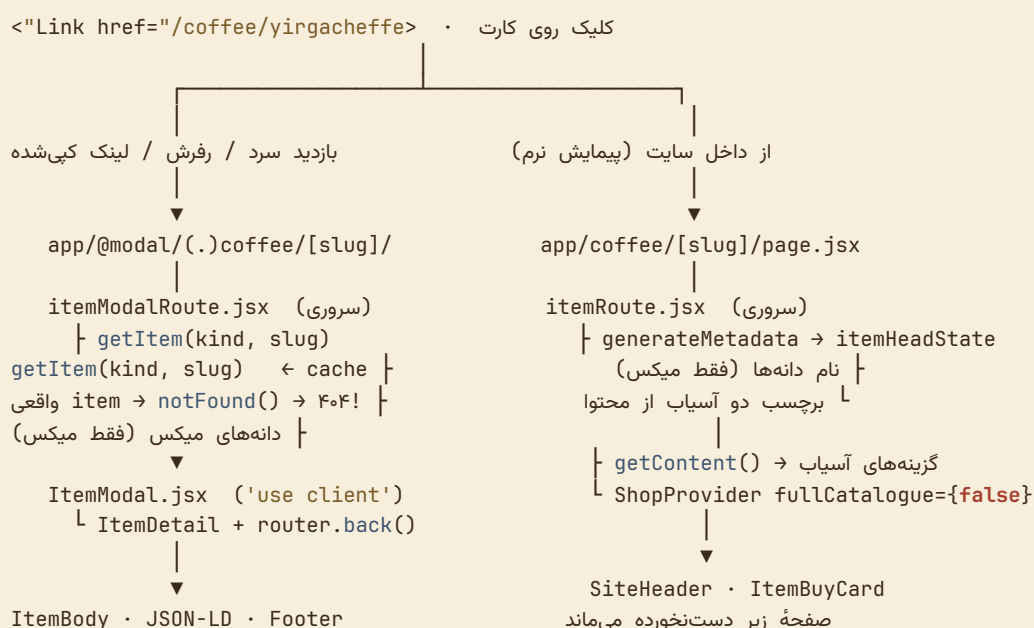
- `opts.grind ?? fallbackGrind` – اگر کارت آسیاب خودش را داد استفاده کن، وگرنه پیش‌فرض سایت. (`??` نه `||` ، چون رشته خالی باید معتبر باشد).
- `normalizeMix` : اگر میکس بود ولی ترکیب دلخواه نداد، ترکیب رسمی گذاشته می‌شود؛ دانه‌های صفرشده کنار می‌روند.
- کلید تکراری یعنی «همان چیز دقیقاً» → وزن‌ها جمع می‌شوند.

چرا بیرون کشیده شد؟ چون ساز میکس دو حالت پیدا کرد (اهرم‌های ساده و حالت تصویری) و هر دو باید دقیقاً همان ردیف را بسازند. حالا «یک میکس، از هر راهی که ساخته شود، دقیقاً همان ردیف سبد» یک ادعای تست‌پذیر است نه یک امید. `ShopContext` همچنان تنها جایی است که سبد را عوض می‌کند؛ `cartLine.js` فقط شکل می‌سازد، بدون هیچ حالتی.

نکته رندر: خودِ کارت‌ها روی سرور رندر می‌شوند و متنشان در HTML اول هست؛ ولی `ItemCard` یک کامپوننت مشتری است، چون وزن انتخابی و دکمه افزودن حالت و رویداد می‌خواهند. کلیک روی خودِ کارت هم دیگر فقط یک مودال باز نمی‌کند: یک `<a href>` واقعی به صفحه کالاست (مورد ۲۷) – جریان بعدی.

۶.۳ جریان سوم – صفحه اختصاصی کالا، و همان کالا به شکل مودال

تا پیش از مورد ۲۷، کلیک روی «درباره این قهوه» فقط یک مودال باز می‌کرد و آدرس صفحه عوض نمی‌شد. یعنی هیچ کالایی آدرس قابل‌اشتراک نداشت و گوگل هیچ صفحه‌ای برای ایندکس کردن نمی‌دید. حالا **همان محتوا** دو راه دارد، و انتخاب بین آن دو کار `Next` است، نه کار ما.



چرا مسیر رهگیری‌شده، و چه چیزی را جایگزین کرد

در نسخه ویت، لینک کارتها موقعیت فعلی را با `state={{ backgroundLocation }}` همراه خودشان می‌فرستادند و `App.jsx` با دیدن آن، جدول مسیرها را دو بار رندر می‌کرد: یک بار برای صفحه زیر و یک بار برای مودال.

App Router خودش این را دارد. `app/@modal` یک مسیر موازی است و `(.)coffee/[slug]` یک مسیر رهگیری‌شده: پیمایش نرم به آن شکاف می‌رسد، بازدید سرد به مسیر کامل. همان دو رفتار، بی هیچ حالت دستی.

سه فایل کوچک این را کامل می‌کنند:

فایل	کارش
<code>app/layout.jsx</code>	شکاف <code>modal</code> را کنار <code>children</code> رندر می‌کند
<code>app/@modal/default.jsx</code>	برای آدرسی که هیچ مودالی ندارد <code>null</code> می‌دهد — بدون آن، بازدید سرد ۴۰۴ می‌شود
<code>_item/ItemModal.jsx</code>	بستن مودال = <code>router.back()</code> ، پس دکمه بستن و دکمه <code>back</code> مرورگر یک کار می‌کنند

<head> مودال — و چرا اصلاً ساخته نمی‌شود

`itemModalRoute.jsx` هیچ `generateMetadata` ندارد، و این عمدی است.

در نسخه ویت مجبور بودیم عنوان صفحه را موقع باز شدن مودال عوض کنیم و با بستنش برگردانیم — که همان چیزی بود که آن مدیر `head` با پشتۀ سه‌حالتۀ را لازم می‌کرد. حالا مودال روی صفحه‌ای باز می‌شود که خودش عنوان و `canonical` دارد، و آدرس واقعاً عوض شده: اگر کاربر لینک را کپی کند یا ربانی سراغ همان آدرس برود، مسیر کامل همان تگ‌ها را می‌دهد. پس چیزی برای عوض کردن نمانده.

اگر کالا پیدا نشد، مودال چیزی رندر نمی‌کند: صفحه زیر دست‌نخورده می‌ماند و آدرسی کامل خودش ۴۰۴ می‌دهد.

دادهٔ مودال به شکل نگاشت می‌آید، نه از کانتکست

```
<ItemDetail
  item={item}
  onClose={close}
  beanName={slug} => beanNames[slug] || slug}
  grindLabel={value} => grindLabels[value] || ''}
/>
```

مودال در شکافِ layout ریشه رندر می‌شود، یعنی **بیرون** از **ShopProvider** ی که **app/page.jsx** می‌سازد؛ پس نمی‌تواند **bySlug** و **grindLabel** را از کانتکست بگیرد. می‌شد یک **Provider** دوم اینجا هم گذاشت، ولی آن وقت **دو سبب جدا** می‌داشتیم و افزودن از یکی روی دیگری اثر نمی‌کرد — دامی که بعداً پیدا کردنش سخت است. پس فقط همان دو چیزی که لازم است، به شکل دادهٔ ساده از سرور می‌آید.

به همین دلیل **itemModalRoute** فقط برچسب **دو آسیاب** را می‌خواند (**espresso** و **french**)، نه هر هفت تا؛ متن **ItemBody** فقط به همان دو اشاره می‌کند («از اسپرسو تا فرنچ پرس»).

صفحهٔ کامل: چه چیزی سروری است و چه چیزی نه

itemRoute.jsx تقریباً تماماً سروری است. تنها تکهٔ مشتری، **ستون خرید** است (**ItemBuyCard.jsx**) — چون وزن انتخابی حالت است. متن کالا (**ItemBody**) سروری است و از داخل همان درخت پاس داده می‌شود، پس هیچ جاوااسکریپتی برایش فرستاده نمی‌شود.

و یک تگ که از راه **Metadata API** نمی‌رود:

```
const ogType = ogTypeFallback(itemHeadState(item, { baseUrl: siteBaseUrl() }));
...
{ogType ? <meta property="og:type" content={ogType} /> : null}
```

og:type: product در فهرست بستهٔ **Next** نیست و اگر به **generateMetadata** داده شود، رندر متادیتا **خطا می‌دهد** — یعنی کل **<head>** از دست می‌رود، نه فقط یک تگ. پس همان یک تگ را خود صفحه رندر می‌کند و ری‌اکت ۱۹ به **<head>** می‌بردش. شرح کاملش در ۵ (**lib/metadata.js**) و ۹ (تصمیم ۲۴).

دو تلهٔ سبب در همین جریان

تلهٔ اول: فهرست ناقص. صفحهٔ کالا فقط خود کالا (و دانه‌های میکس) را به **ShopProvider** می‌دهد، نه صد و شش کالا — خواندن کل فهرست برای نشان دادن یکی بی‌معنی است. ولی سبب مالِ کل سایت است: مشتری می‌تواند دو قهوه در سبد داشته باشد و بعد صفحهٔ یک ابزار را باز کند. بدون نگهبان، افکتِ پاک‌سازی همهٔ آن ردیف‌ها را «کالایش در فهرست نیست» می‌دید و پاک می‌کرد.

تلهٔ دوم: بدنهٔ سفارش. **CartDrawer** سفارش را از **resolved** می‌سازد — ردیف‌هایی که با سند کالا جفت شده‌اند. روی فهرست ناقص، ثبت سفارش از صفحهٔ یک کالا **بی‌صدا** بقیهٔ ردیف‌ها را می‌انداخت.

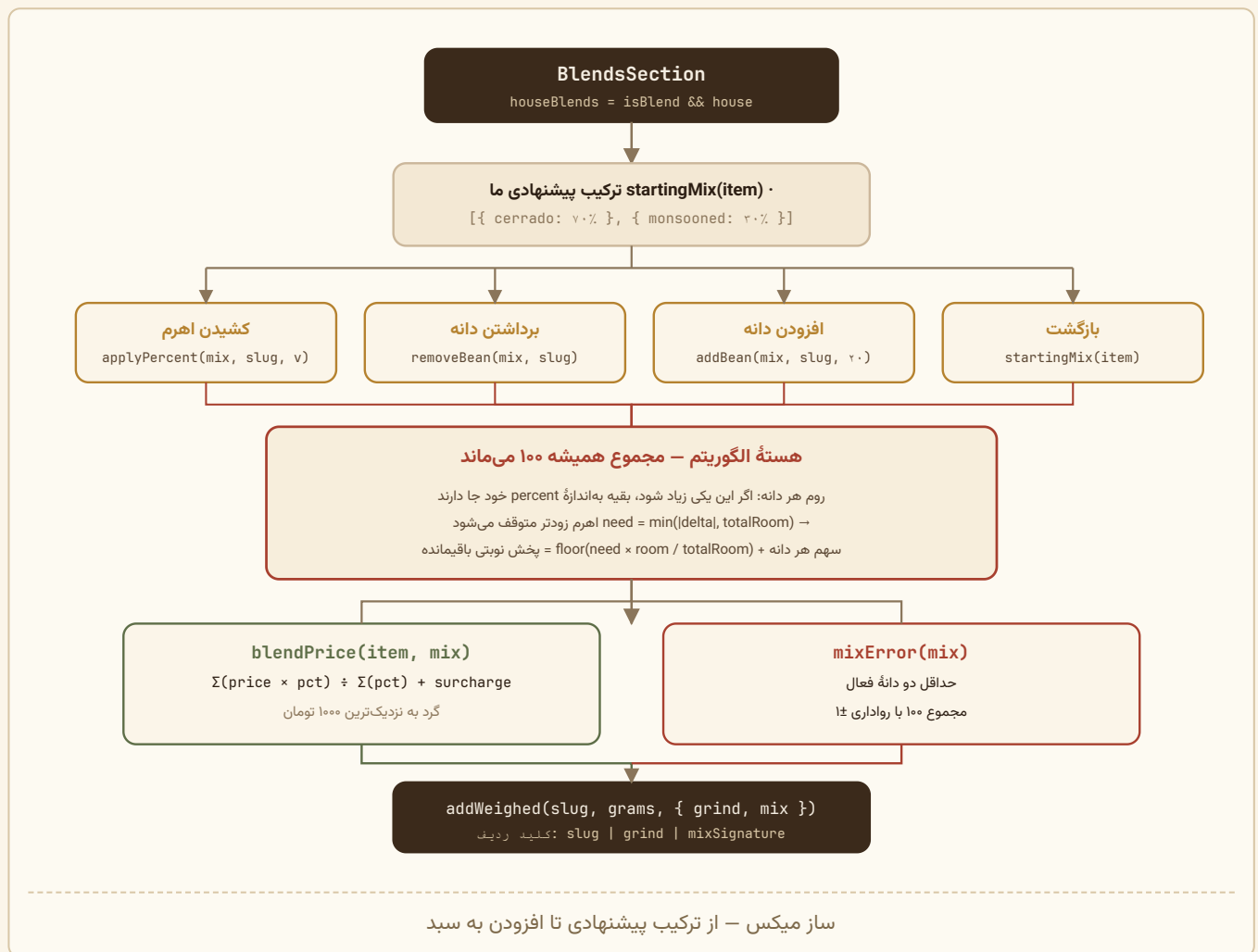
جوابش پرچم **fullCatalogue={false}** و افکت «کامل کردن فهرست» است: به محض اینکه سبب به کالایی بی‌رد اشاره کند، فهرست کامل می‌شود. برای بازدیدکنندهٔ بی‌سبب — یعنی بیشترشان — این افکت هیچ کاری نمی‌کند و صفحهٔ کالا سبک می‌ماند. کد و نگهبان‌هایش در ۷.۱۲.

فقط کالای `active` می‌دهد. پس شناسه ناموجود و کالایی که مدیر خاموشش کرده از بیرون یکی دیده می‌شوند: هر دو `notFound()` می‌گیرند، هر دو ۴۰۴ واقعی با `noindex`، و هیچ‌کدام در `sitemap.xml` نمی‌آیند. عمداً ۴۱۰ نیست: خاموش کردن یعنی «فعلاً فروخته نمی‌شود»، نه «برای همیشه رفت». برای دومی یک فیلد صریح لازم است.

۶.۴ جریان چهارم – ساختن میکس دلخواه

این متمایزترین جریان پروژه است.

نمودار



نکته کلیدی: هر نسبت، یک ردیف جدا

چون `mixSignature` در کلید ردیف است، مشتری می‌تواند یک میکس را دو بار با دو نسبت متفاوت سفارش دهد و هر کدام ردیف مستقل خودش را دارد. مثلاً:

espresso-70-30 | espresso | cerrado:70,monsooned:30 ← گرم ۵۰۰
espresso-70-30 | french | cerrado:50,monsooned:50 ← گرم ۲۵۰

همگام‌سازی با سرور

مهم است بدانید که همان دو قاعده‌ای که `mixError` سمت کلاینت بررسی می‌کند، سمت سرور هم دوباره بررسی می‌شوند:

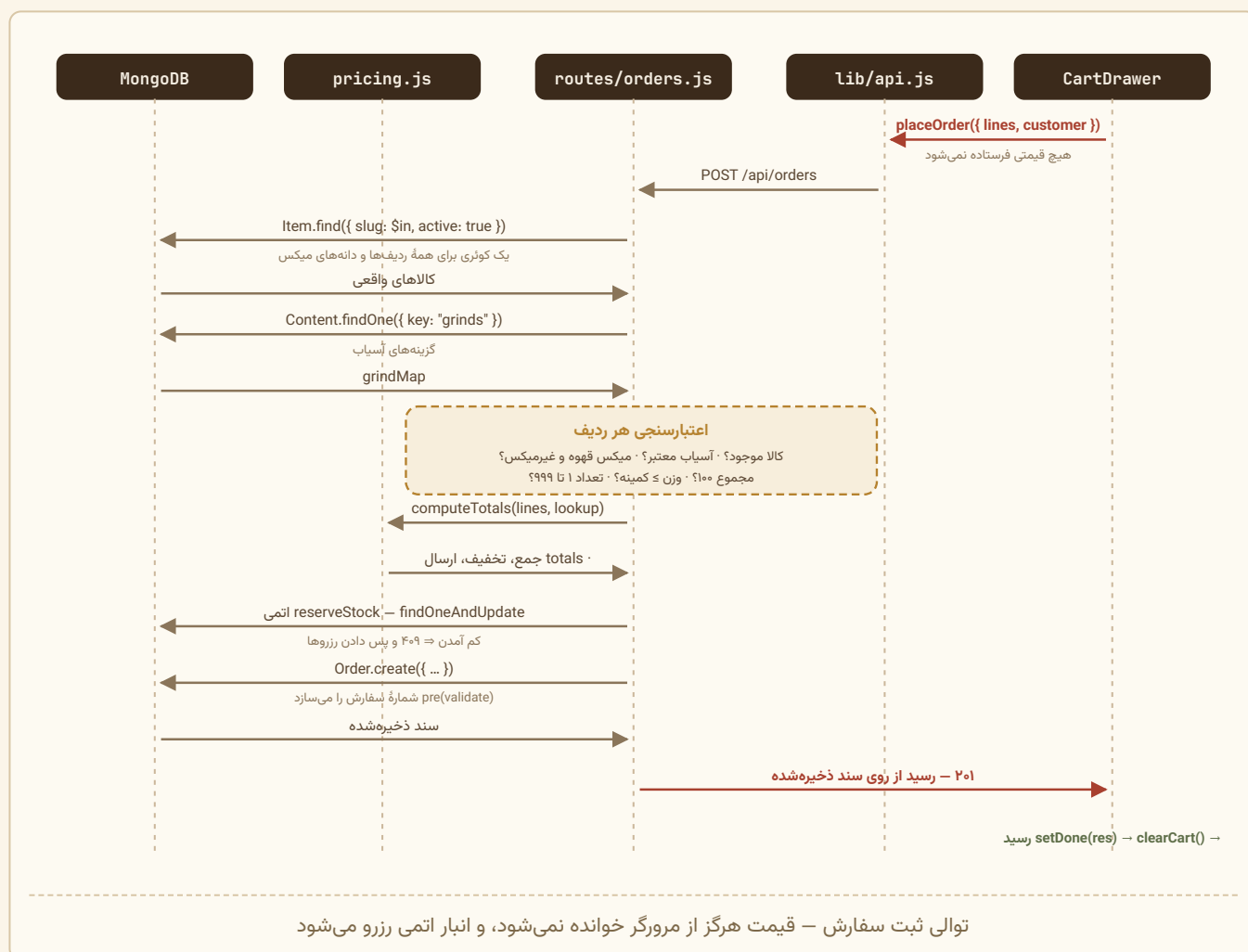
قاعده	کلاینت (<code>blend.js</code>)	سرور (<code>orders.js</code>)
حداقل دو دانه	<code>if (live.length < 2)</code>	<code>if (mix.length < 2)</code>
مجموع ۱۰۰ (±)	<code>Math.abs(sum - 100) > 1</code>	<code>Math.abs(sum - 100) > 1</code>
فقط قهوه	(فهرست <code>available</code> فیلتر شده)	<code>if (bean.kind !== 'coffee')</code>
میکس در میکس نه	(فیلتر <code>!b.isBlend</code>)	<code>if (bean.isBlend)</code>

کلاینت برای تجربه کاربری بررسی می‌کند، سرور برای درستی.

۶.۵ جریان پنجم – تسویه و ثبت سفارش

این مهم‌ترین جریان از نظر امنیت است.

نمودار توالی کامل



web/src/components/CartDrawer.jsx

```
const phone = toLatinDigits(form.phone).trim();

if (!form.name.trim()) return setError('نام گیرنده را بنویسید');
if (!/^0\d{10}$/.test(phone)) {
  return setError('شمارهٔ موبایل را کامل و با ۰ اول وارد کنید، مثل ۰۹۱۲۱۲۳۴۵۶۷');
}
if (form.address.trim().length < 10) return setError('نشانی را کامل‌تر بنویسید');
```

toLatinDigits قبل از regex اجرا می‌شود، چون کاربر با کیبورد فارسی ۰۹۱۲۱۲۳۴۵۶۷ می‌نویسد.

گام ۲ – ساخت بار درخواست

```
lines: resolved.map((l) =>
  l.kind === 'gear'
    ? { slug: l.slug, qty: l.qty }
    : {
      slug: l.slug,
      grams: l.grams,
      ...(l.item.grindable ? { grind: l.grind || 'whole' } : {}),
      ...(l.item.isBlend && l.mix?.length ? { mix: l.mix } : {}),
    },
),
```

فقط چهار چیز: شناسه، مقدار، آسیاب، ترکیب. نه قیمت، نه جمع، نه تخفیف. عملگر spread شرطی (cond ? {k:v} : ...) ({}) فیلدهای بی‌ربط را حذف می‌کند.

گام ۳ – بازسازی داده در سرور

server/src/routes/orders.js

```
const wanted = new Set(raw.map((l) => String(l.slug || '')));
/* اجزای میکس هم لازم‌اند تا قیمت را دوباره حساب کنیم */
for (const l of raw) {
  if (Array.isArray(l.mix)) for (const m of l.mix) wanted.add(String(m.slug || ''));
}

const items = await Item.find({ slug: { $in: [...wanted] }, active: true });
const bySlug = new Map(items.map((i) => [i.slug, i]));
const lookup = (slug) => bySlug.get(slug);
```

یک کوئری برای همه‌چیز. اگر سبد ۱۰ ردیف با ۳ میکس داشته باشد، همچنان یک find است، نه ۱۰ تا. و شرط active: true یعنی کالای پنهان‌شده قابل سفارش نیست.

```
const item = bySlug.get(String(l.slug || ''));
if (!item) {
  return res.status(400).json({ error: `دیگر موجود نیست. سید را به روز کنید «${l.slug}» کالای ` });
}

let grind = '';
if (item.grindable) {
  grind = String(l.grind || 'whole');
  if (!grinds.has(grind)) {
    return res.status(400).json({ error: `نامعتبر است «${item.name}» نحوه تحویل ` });
  }
}
```

توجه: `item.grindable` از پایگاه داده خوانده می‌شود، نه از ورودی. اگر مشتری برای یک ماگ `grind: 'espresso'` بفرستد، بی‌سروصدا نادیده گرفته می‌شود چون `grindable` آن `false` است.

بررسی میکس دو شاخه دارد:

```
if (!item.customizable) {
  /* مشتری اجازه تغییر ندارد - ترکیب رسمی خودمان را می‌گذاریم */
  mix = item.components.map((c) => ({ slug: c.slug, percent: c.percent }));
} else {
  /* بررسی کامل ترکیب ارسالی */
}
```

اگر مدیر `customizable` را خاموش کرده باشد، هر چه مشتری بفرستد دور ریخته می‌شود و ترکیب رسمی جایگزین می‌شود. این یک الگوی امنیتی خوب است: به جای خطا دادن، حالت امن را اعمال کن.

در شاخه `customizable`:

```
if (!bean) → 'در میکس موجود نیست «X» دانه'
if (seen.has(slug)) → 'یک دانه دو بار در میکس آمده است'
if (bean.kind !== 'coffee') → 'قهوه نیست و در میکس نمی‌آید «X»'
if (bean.isBlend) → 'خودش یک میکس است و جزء میکس دیگری نمی‌شود «X»'
if (!Number.isFinite(percent) || percent < 0 || percent > 100) → 'درصدهای میکس نامعتبرند'
if (percent === 0) continue; // دانه برداشته شده
if (mix.length < 2) → 'میکس باید دست کم دو دانه داشته باشد'
if (Math.abs(sum - 100) > 1) → `است ${sum} مجموع درصدهای میکس باید ۱۰۰ باشد، الان`
```

بررسی مقدار:

```

if (item.kind === 'gear') {
  const qty = Math.floor(Number(l.qty));
  if (!Number.isFinite(qty) || qty < 1 || qty > 999) {
    return res.status(400).json({ error: `نامعتبر است «${item.name}» تعداد` });
  }
  lines.push({ kind: 'gear', item, qty, grams: 0, grind: '', mix: [] });
} else {
  const grams = Math.round(Number(l.grams));
  const min = MIN_GRAMS[item.kind] || 50;
  if (!Number.isFinite(grams) || grams < min || grams > 100000) {
    return res.status(400).json({ error: `نامعتبر است «${item.name}» وزن` });
  }
  lines.push({ kind: item.kind, item, grams, qty: 0, grind, mix });
}
}

```

Number.isFinite هم NaN را می‌گیرد و هم Infinity را — بررسی کامل‌تری از !isNaN.

گام ۵ — بازمحاسبهٔ قیمت

```
const totals = computeTotals(lines, lookup);
```

همان تابعی که مرورگر استفاده کرده، اما با `item` هایی که مستقیم از پایگاه داده آمده‌اند. اگر مشتری قیمت را دستکاری کرده بود، اثری ندارد چون قیمت اصلاً از ورودی خوانده نمی‌شود.

گام ۶ — رزرو موجودی (مورد ۳۴)

آخرین کاری است که پیش از ثبت انجام می‌شود، و ترتیبش عمدی است: تا اینجا هر خطای اعتبارسنجی بدون لمس انبار برگشته است، پس چیزی برای پس دادن نمی‌ماند.

```

const { error: stockError, taken } = await reserveStock(lines, item);
if (stockError) return res.status(409).json({ error: stockError });

```

درون `server/src/lib/stock.js`، هر ردیف با یک عملیات اتمی رزرو می‌شود:

```

const updated = await ItemModel.findOneAndUpdate(
  { slug: item.slug, stock: { $gte: need } }, // شرط
  { $inc: { stock: -need } }, // کاهش
  { new: true, projection: { stock: 1 } }
);

```

شرط و کاهش زیر یک قفل سند اجرا می‌شوند، پس دو مشتری که هم‌زمان آخرین ۵۰۰ گرم را می‌خواهند دقیقاً یکی‌شان برنده می‌شود و دیگری `null` می‌گیرد. هیچ transaction ای لازم نیست — که روی مونگوی تک‌گرهی اصلاً در دسترس نیست.

سه جزئیات که این گام را درست نگه می‌دارند:

(۱) کالای نامحدود اصلاً لمس نمی‌شود. `isTracked(item)` فقط عدد متناهی را «شمرده‌شده» می‌داند؛ `$inc` روی `null` خطا می‌دهد و شرط `$gte` هم با `null` جور نمی‌شود.

(۲) همه یا هیچ. ردیف سوم می‌تواند بعد از موفقیت دو ردیف اول شکست بخورد. `taken` هر رزرو موفق را نگه می‌دارد و در شکست همه با هم برمی‌گردند. و اگر خود `Order.create` هم خطا بدهد، روتر همان `releaseStock(taken, item)` را صدا می‌زند:

```

} catch (err) {
  /* ثبت سفارش شکست خورد - انبار نباید بی‌دلیل کم بماند */
  await releaseStock(taken, Item);
  throw err;
}

```

۳) پیام از موجودی همان لحظه ساخته می‌شود، نه از عددی که در حافظه داشتیم – که ممکن است مال چند لحظه پیش باشد: «موجودی «یرگاچف» کافی نیست – فقط ۳۰۰ گرم مانده است».

چرا ۴۰۹ و نه ۴۰۰؟ چون ورودی مشتری غلط نبود؛ دنیا عوض شده بود. با ۴۰۹، رابط کاربری سبد را دست‌نخورده نگه می‌دارد و فقط پیام موجودی را نشان می‌دهد.

گام ۷ – درج

```

const order = await Order.create({
  lines: lines.map((l) => ({
    kind: l.kind,
    slug: l.item.slug,
    name: l.item.name,
    unitPrice: unitPriceFor(l, lookup),
    grams: l.grams,
    qty: l.qty,
    lineTotal: lineTotal(l, lookup),
    grind: l.grind,
    grindLabel: l.grind ? grinds.get(l.grind) || '' : '',
    mix: l.mix.map((m) => ({
      slug: m.slug,
      name: bySlug.get(m.slug)?.name || m.slug,
      percent: m.percent
    })))
  })),
  customer: { name: ..., phone: ..., address: ..., note: ... },
  totals
});

```

هر ردیف snapshot کامل می‌شود: نام، قیمت واحد، جمع ردیف، برچسب فارسی آسیاب، و نام هر دانه میکس.

گام ۸ – پاسخ

```

/* رسید را از روی سفارش ذخیره‌شده برمی‌گردانیم، نه از
روی چیزی که مرورگر فرستاده – تا مشتری دقیقاً همان
چیزی را ببیند که ثبت شده و بعداً آماده می‌شود. */
res.status(201).json({
  ok: true, code: order.code, createdAt: order.createdAt,
  lines: order.lines, customer: order.customer, totals: order.totals
});

```

گام ۹ – رسید

کامپوننت `Receipt` در `CartDrawer.jsx` رسید را از `done` (پاسخ سرور) می‌سازد، نه از سبد – پس قیمت‌هایش همان چیزی است که واقعاً ثبت شده. حتی شماره تماس فروشگاه از `content.about` خوانده می‌شود، تا اگر مدیر آن را عوض کرد رسید هم به‌روز باشد.

یک کامپوننت مشترک است و همین صفحه تنها صداکننده‌اش نیست؛ صفحه پیگیری هم همان را رندر می‌کند. تفاوت‌های آن دو با چهار پارامتر (`heading`، `showMark`، `statusLabel`، `showTracking`) پوشانده می‌شود – جدولش در فصل ۵.

گام ۱۰ – چاپ

این گام هم از کنشوی سبد شروع می‌شود و هم از صفحه پیگیری، و در هر دو مسیر یک چیز است:

```
onClick -> window.print()
|
+-- <body class="has-print-sheet">      PrintSheet added it on mount
|
v
@media print                          web/src/admin.css
|
+-- body.has-print-sheet > *:not(.print-sheet) { display: none }
|      header, catalogues, cards, footer, drawer, toast
|
+-- .print-sheet { display: block }    direct child of body, via portal
    |
    v
    <Receipt />    10pt, black on white, break-inside: avoid
```

چرا رسید دو بار رندر می‌شود. یکی داخل درخت صفحه (آنچه کاربر روی نمایشگر می‌بیند) و یکی به شکل فرزند مستقیم `body`:

```
<div className="drawer-body">
  <Receipt order={done} shop={content?.about} />
</div>

<PrintSheet>
  <Receipt order={done} shop={content?.about} />
</PrintSheet>
```

بدون نسخه دوم، قاعده چاپ باید نیاکان رسید را پنهان می‌کرد – و هر نیایی که پنهان شود خود رسید را هم می‌برد. با نسخه دوم، قاعده یک خط می‌شود: «هر فرزند `body` به جز این یکی».

نسخه پیشین و اینکه چرا عوض شد. تا پیش از این، سبک چاپ همه چیز را با `visibility: hidden` نامرئی می‌کرد و فقط کنشو را برمی‌گرداند. `visibility` عنصر را از چیدمان حذف نمی‌کند: کل فروشگاه همچنان ارتفاع واقعی خودش را داشت و مرورگر همان ارتفاع را صفحه‌بندی می‌کرد. نتیجه اندازه‌گیری شد – ۳۳ صفحه، که ۳۱ تای آخرش کاغذ سفید خالی بود. شرح کامل سبک در ۸.۱۰ و چرایی تصمیم در تصمیم ۲۷ فصل ۹.

نکته

کلاس `has-print-sheet` را `PrintSheet` موقع mount روی `body` می‌گذارد و موقع unmount برمی‌دارد. بدون آن، قاعده «هر فرزند `body` را بردار» روی هر صفحه‌ای اجرا می‌شد و `Ctrl+P` روی خود فروشگاه یک برگ سفید می‌داد.

۶.۶ جریان ششم – پیگیری سفارش بدون حساب کاربری

مشتری حساب کاربری ندارد؛ بعد از بستن رسید، تا پیش از مورد ۳۵ هیچ راهی نداشت بفهمد سفارشش کجاست. این جریان با همان دو چیزی کار می‌کند که دستش هست: شماره سفارش و شماره موبایلی که داده.

نمودار جریان

```

/track یا /track?code=K7B2-4193 (از لینک رسید)
↓
app/track/page.jsx ← کامپوننت سروری
  | metadata = toNextMetadata({ ..., robots: 'noindex, follow' })
  | <Suspense> ... </Suspense>
  ↓
components/Track.jsx ('use client')
  | useSearchParams() → code پیش‌پر کردن
  | toLatinDigits(phone) و toLatinDigits(code)
  | هر دو خالی؟ → پیام محلی، بی درخواست
  ↓
api.trackOrder({ code, phone })
  | GET /api/orders/track?code=&phone= (عمومی)
  ↓
trackLimiter تلاش در ۱۵ دقیقه ۲۰
  ↓
lib/track.js → findOrderForTracking({ code, phone }, Order)
  | ورودی ناقص → ۴۰۰ با پیام راهنما
  | Order.findOne({ code })
  | safeEqual(normalizePhone(...), wantedPhone) ← زمان ثابت
  | نبود **یا** جور نبود → ۴۰۴ با پیام **یکسان**
  | جور بود → publicOrderView(order)
  ↓
Track.jsx: StatusBar + جمع‌ها + ردیف‌ها
```

چه چیزی با مورد ۲۶ عوض شد

خود پیگیری نمی‌تواند سروری باشد، و این یک محدودیت واقعی است نه سلیقه: مشتری باید کد و موبایلش را بنویسد، و آن جفت هیچ‌وقت در آدرس نمی‌نشیند – و نباید بنشیند، وگرنه در تاریخچه مرورگر و لاگ پراکسی می‌ماند. پس فرم و نتیجه سمت مشتری ماندند.

آنچه سروری شد، `<head>` است:

```

export const metadata = toNextMetadata({
  title: `${SITE_NAME} - پیگیری سفارش`,
  description: 'وضعیت سفارشات را با شماره سفارش و شماره موبایل ببینید',
  /* همان چیزی که robots.txt هم می‌گوید - ولی این یکی
  ایندکس شدن را هم می‌بندد، نه فقط خزیدن را. */
  robots: 'noindex, follow'
});
```

در نسخه ویت این تگ‌ها را یک هوک در زمان اجرا می‌نوشت. برای `noindex` این فرق می‌کند: رباتی که جاوااسکریپت اجرا نمی‌کند هم حالا آن را می‌بیند.

`Suspense` هم از همین جا آمد: `Track` از `useSearchParams` استفاده می‌کند و Next می‌خواهد چنین تکه‌ای مرز `Suspense` خودش را داشته باشد، تا اگر روزی این مسیر پیش‌رندر شد، بقیه صفحه گروگان پارامترهای آدرس نماند. `robots.txt` هم `/track` را می‌بندد – آدرس‌های `?code=...` نباید در نتایج جست‌وجو بنشینند.

دو محافظ که این جریان را از یک ابزار شمارش جدا می‌کنند

(یک پیام یکسان. «کد وجود ندارد» و «شماره جور نیست» دقیقاً یک پاسخ می‌گیرند، از یک ثابت صادرشده:

```
export const NOT_FOUND_MESSAGE = 'سفارشی با این شماره و شماره موبایل پیدا نشد';
```

اگر فرق می‌کرد، یک اسکریپت می‌توانست کدها را یکی‌یکی امتحان کند و بدون دانستن هیچ شماره‌ای بفهمد کدام کدها وجود دارند – و از آنجا تعداد سفارش‌های فروشگاه و نرخ رشدشان.

(دو زمان یکسان. پیام یکسان کافی نیست: اگر برای کد ناموجود سریع‌تر پاسخ بدهیم، خود زمان همان چیزی را لو می‌دهد که پنهانش کردیم.

```
const stored = normalizePhone(doc?.customer?.phone);
const matches = safeEqual(stored || 'no-order', wantedPhone);
if (!doc || !matches) return { status: 404, error: NOT_FOUND_MESSAGE };
```

مقایسه هم زمان‌ثابت است و اول هش می‌شود، چون `timingSafeEqual` طول‌های نابرابر را قبول نمی‌کند و پرتاب خطا می‌کند – که خودش یک نشئت زمانی است. تحلیل کاملش در ۷.۶ و در پرسش ۳۲ فصل ۱۰.

آنچه برگردانده می‌شود، و آنچه نمی‌شود

`publicOrderView` سند را بازسازی می‌کند، نه اینکه فیلدهای ناخواسته را حذف کند: `code`، `status`، `createdAt`، فقط نام مشتری، ردیف‌ها و جمع‌ها. نشانی، شماره تماس و یادداشت مشتری بیرون می‌مانند. اثبات مالکیت اینجا یک جفت کوتاه است و از یک رمز عبور خیلی ضعیف‌تر. پس فرض را بر این می‌گذاریم که روزی یک حدس موفق می‌شود و می‌پرسیم: آن وقت چه چیزی لو رفته؟ با طراحی فعلی، نه نشانی خانه کسی.

وضعیت، به زبان مشتری

```
const STEPS = ['new', 'processing', 'done'];
```

سه گام معمول، با کلیدهایی که همان `enum` مدل `Order` اند. «لغو شده» گام نیست، پس جدا و با یک پیام نشان داده می‌شود – نه به عنوان مرحله چهارم.

و یک نسخه کاغذی، از همان جا

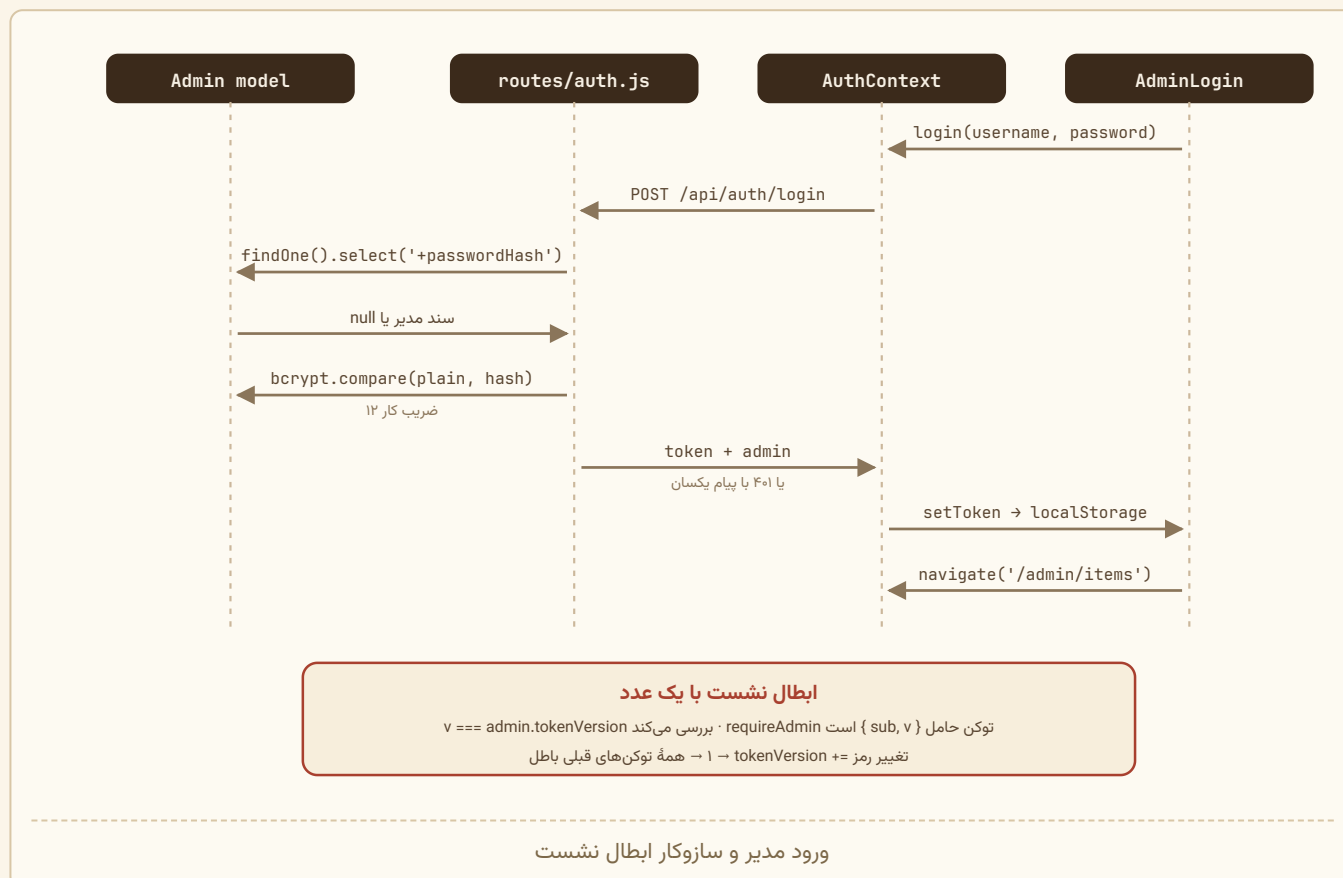
این جریان یک پایان دوم هم دارد. مشتری‌ای که رسیدش را بسته، تا پیش از این هیچ راهی برای گرفتن یک نسخه چاپی نداشت؛ حالا دکمه «چاپ یا ذخیره رسید» همان برگه‌ای را می‌دهد که کشوی سبد می‌داد – همان کامپوننت `Receipt` و همان سبک چاپ، پس دو رسید متفاوت به وجود نمی‌آید.

چهار پارامتر تفاوت‌ها را می‌پوشانند: عنوان «رسید سفارش» می‌شود، تیک سبز «ثبت شد» برداشته می‌شود (این سفارش ممکن است لغو شده باشد)، وضعیت اضافه می‌شود، و لینک «وضعیت سفارش را ببینید» می‌رود – مشتری همین حالا رویش ایستاده.

آنچه روی نمایشگر است دست نمی‌خورد: نوار مرحله و بلوک وضعیت مال صفحه‌اند و در `PrintSheet` نیستند. گردش کامل چاپ در گام ۱۰ همین فصل و سبکش در ۸.۱۰.

بلوک تماسِ برگهٔ چاپی به متن‌های سایت نیاز دارد، پس این صفحه کنار `trackOrder` یک `api.content()` هم می‌زند - با `.catch(() => null)`. نیامدنش نباید پیگیری را زمین بزند؛ فقط تلفن و نشانی از پای رسید می‌افتند.

۶.۷ جریان هفتم - ورود مدیر



کجای درخت چه چیزی است. این جریان با مورد ۲۶ سه‌تکه شد و تقسیمش عیناً همان چیزی است که در نسخهٔ ویت بود، فقط با پوشه به‌جای `<Route>`:

فایل	نقش
app/admin/layout.jsx	AuthProvider + metadata با robots: noindex - بیرون از دروازه، چون صفحهٔ ورود هم لازمش دارد تا بفهمد از قبل وارد شده‌اید یا نه
app/admin/login/page.jsx	یک خط: AdminLogin را دوباره صادر می‌کند
app/admin/(panel)/layout.jsx	یک خط: AdminShell - همین دروازه است

گروه مسیر (panel) در آدرس دیده نمی‌شود؛ فقط برای همین تقسیم است.

سپس AdminShell محافظت می‌کند - و اینجا یک تفاوت واقعی با react-router هست: App Router کامپوننتی مثل `<Navigate>` ندارد و هدایت نباید در رندر انجام شود.

```
useEffect(() => {
  if (!checking && !admin) router.replace('/admin/login');
}, [checking, admin, router]);

if (checking || !admin) return <div className="admin-boot">در حال بررسی نشست</div>;
```

شرط `if` دوم فقط برای زیبایی نیست: بین اجرای افکت و رسیدن به صفحه ورود یک لحظه فاصله هست، و بدون آن پنل یک لحظه برق می‌زد و بعد می‌رفت. همین الگو در `AdminLogin` هم هست، این بار برای حالت معکوس: اگر از قبل وارد شده‌اید، به جای فرم بی‌فایده همان پیام «بررسی نشست» دیده می‌شود تا هدایت انجام شود.

`app/admin/page.jsx` هم همان کاری را می‌کند که `<Route index element={<Navigate to="/admin/items" />} />` می‌کرد، ولی روی سرور:

```
export default function AdminIndex() {
  redirect('/admin/items');
}
```

و از این به بعد، هر درخواست مدیریتی هدر `Authorization` می‌گیرد:

```
if (auth) headers.Authorization = `Bearer ${getToken()}`;
```

خروج خودکار در صورت انقضا:

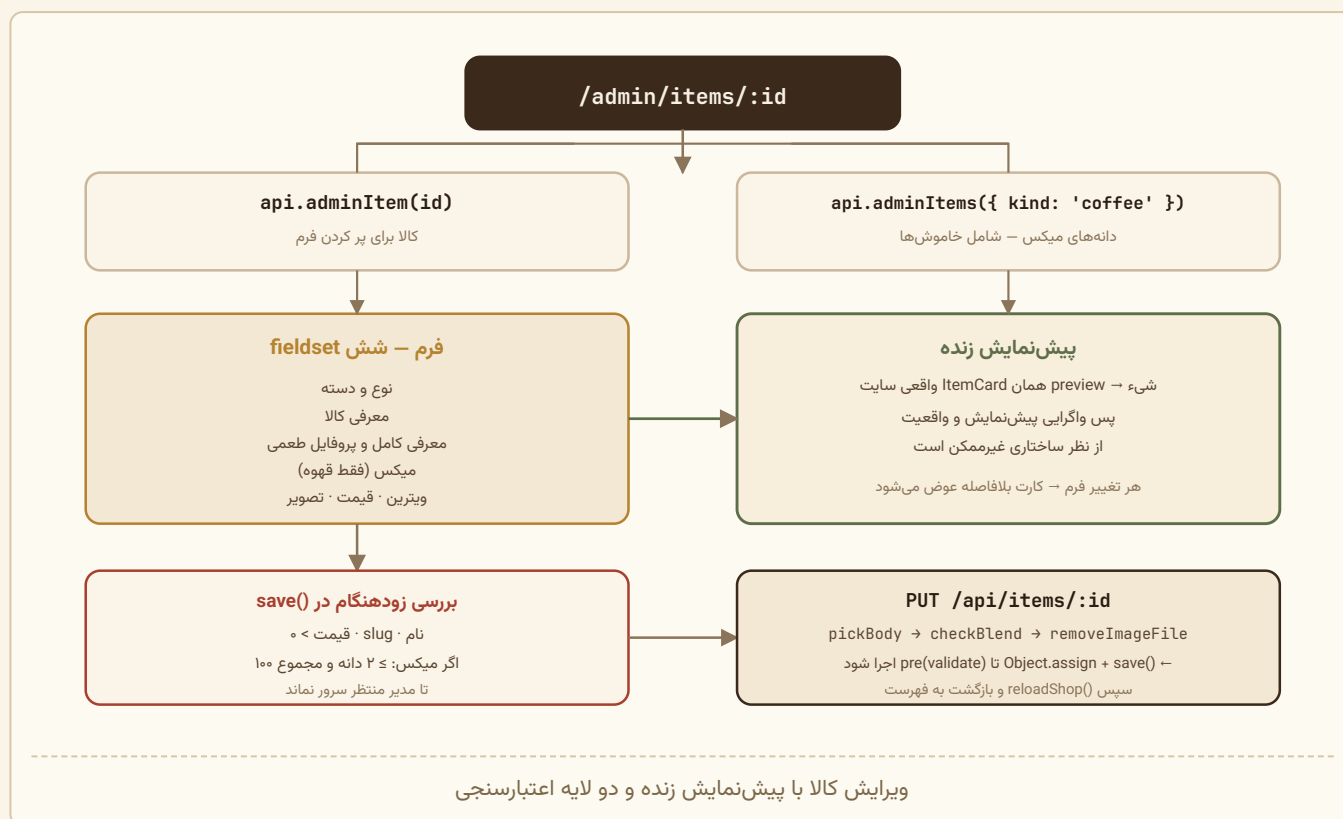
```
if (!res.ok) {
  if (res.status === 401 && auth) {
    clearToken();
    onUnauthorized(); // ← AuthContext.setAdmin(null)
  }
  throw new Error(data?.error || `خطای سرور (${res.status})`);
}
```

باعث `rerender` `AdminShell` می‌شود، همان افکت دروازه دوباره اجرا می‌شود و `router.replace` کاربر را به صفحه ورود می‌برد – بدون اینکه هیچ کامپوننتی صریحاً چنین دستوری داده باشد.

نکته

این کل زنجیره فقط زیر `/admin` زندگی می‌کند. `AuthProvider` در `app/admin/layout.jsx` است، نه در ریشه؛ پس بازدیدکننده فروشگاه نه `api.me()` می‌فرستد و نه کد پنل را دانلود می‌کند – تقسیم باندل بر اساس مسیر را Next خودش انجام می‌دهد (نیمه حل‌شده مورد ۲۰).

۶.۸ جریان هشتم – ویرایش کالا با پیش‌نمایش زنده



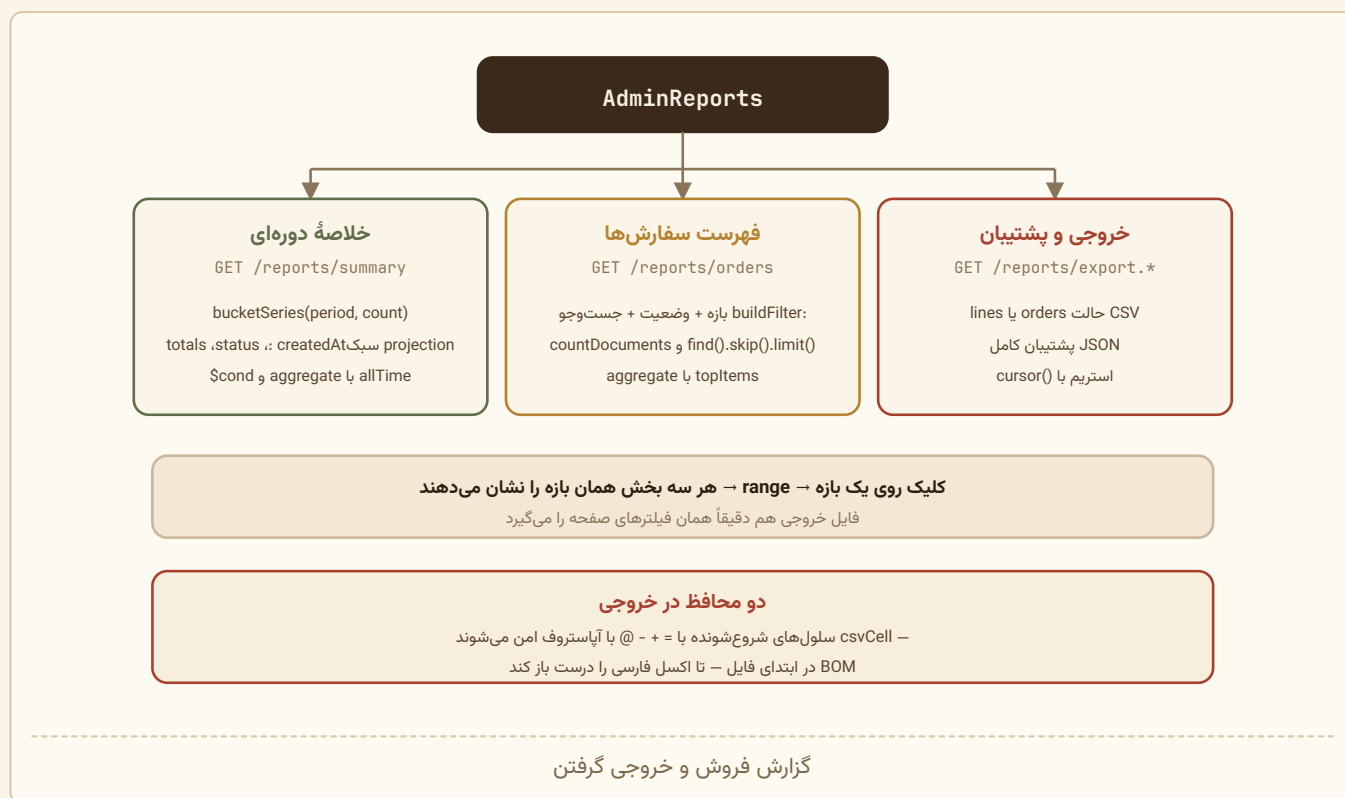
نکته معماری: پیش‌نمایش از همان کامپوننت `ItemCard` استفاده می‌کند که در سایت واقعی رندر می‌شود. یعنی هیچ‌وقت پیش‌نمایش و واقعیت واگرا نمی‌شوند. کامنت `preview` این را می‌گوید: «تا پیش‌نمایش دقیقاً مثل سایت رفتار کند.»

دو لایه اعتبارسنجی: بررسی زودهنگام در `save()` با کامنت «بررسی زودهنگام تا مدیر برای خطای ساده منتظر سرور نماند»، و بررسی قطعی در `pre('validate')` مدل.

«فروشگاه را تازه کن» چه چیزی را تازه می‌کند. در نسخه ویت این `useShop().reload()` بود، چون `ShopContext` فهرست کالاها را یک بار موقع بالا آمدن سایت می‌گرفت و در حافظه نگه می‌داشت. در Next آن حافظه وجود ندارد – هر صفحه فروشگاه در هر درخواست از نو ساخته می‌شود – ولی کش مسیریاب سمت مشتری هست: اگر مدیر پیش از رفتن به پنل صفحه اصلی را دیده باشد، بازگشتش می‌تواند از همان نسخه کش‌شده بیاید.

`useStorefrontRefresh()` دقیقاً همین را باطل می‌کند (`router.refresh()`). شکل صدا زدن در سه فایل پنل دست‌نخورده ماند؛ فقط موتورش عوض شد.

و چرا پنل هنوز `ShopProvider` دارد. پیش‌نمایش زنده به `ItemCard` واقعی نیاز دارد و آن کارت برای میکس‌ها قیمت را از میانگین وزنی دانه‌ها حساب می‌کند – یعنی به فهرست کالاها. صفحه‌های پنل سروری نیستند که کسی فهرست را برایشان آماده کند، پس `AdminShell` با پرچم `loadOnMount` آن را در مرورگر می‌گیرد. این تنها جای پروژه است که `ShopProvider` هنوز خودش داده می‌خواند.



نکته مهم UX: خروجی‌ها همان فیلترهای صفحه را می‌گیرند. متن جعبه پشتیبان این را صریح می‌گوید:

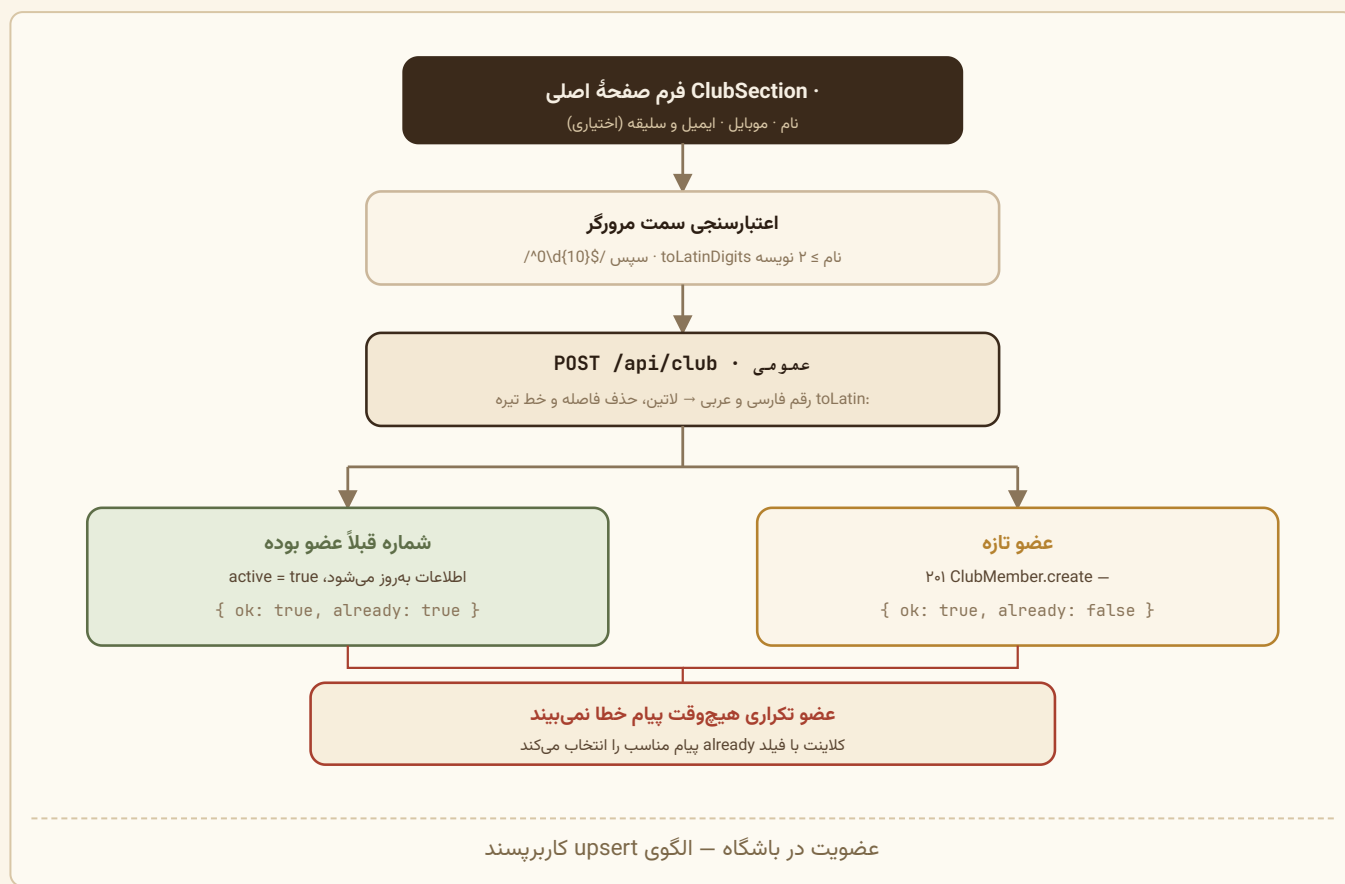
فایل‌ها دقیقاً همان سفارش‌هایی را می‌گیرند که الان انتخاب کرده‌اید (مرداد ۱۴۰۵). برای پشتیبان واقعی، فایل JSON را نگه دارید: همه چیز بی‌کم‌وکاست در آن هست.

و این با کد جور است – همان params هم به reportOrders و هم به reportCsv / reportJson داده می‌شود.

تفاوت CSV و JSON:

JSON	CSV	
پشتیبان کامل	تحلیل در اکسل	هدف
عیناً سند MongoDB	ستون‌های منتخب و قالب‌بندی‌شده	محتوا
فقط ISO	شمسی + ISO	تاریخ
ندارد (لازم نیست)	دارد (csvCell)	محافظ فرمول
ندارد	دارد	BOM

۶.۱۰ جریان جانبی – عضویت در باشگاه



الگوی **upsert کاربرپسند**: عضو تکراری خطا نمی‌گیرد، بلکه پیام مناسب می‌بیند. این تصمیم در کامنت روتر آمده: «اگر قبلاً عضو شده، اطلاعاتش را تازه می‌کنیم و همان پیام خوشامد را می‌دهیم – نه پیام خطا.»

۶.۱۱ جریان جانبی – هم‌رسانی یک کالا

کوتاه‌ترین جریان سایت – یک کلیک و یک پاسخ – ولی سه جای متفاوت شروعش می‌کنند و چهار پایان متفاوت دارد.


```

ShareButton.onShare(e)                                ItemCard | ItemBuyCard | ItemDetail
|
+-- e.preventDefault() + e.stopPropagation()
+-- setBusy(true)
v
shareItem(item)                                       web/src/lib/share.js
|
+-- url = itemHeadState(item, {baseUrl}).canonical
|
+-- navigator.share && isTouchDevice()
|   |
|   +-- nav.share({title, url}) ----> mode = 'shared'
|   +-- err.name === 'AbortError' ----> mode = 'canceled'
|   +-- any other error -----+
|   |
+-----+
v
copyText(url)
+-- navigator.clipboard.writeText(url) ----> mode = 'copied'
+-- textarea + document.execCommand() ----> mode = 'copied'
+-- neither ----> mode = 'failed'
|
v
ShareButton
  'shared' | 'canceled' -> ---
  'copied' -> toast(SHARE_COPIED)
  'failed' -> toast(SHARE_FAILED)
|
v
toastStore.toast(msg) -> <Toast /> in app/layout.jsx

```

سه چیز در همین نمودار عمده‌اند:

- `preventDefault` و `stopPropagation` چون دکمه داخل کارتی می‌نشیند که پر از کنترل است؛ این کلیک نباید به هیچ چیز دیگری برسد.
- `setBusy(true)` دکمه را تا پایان کار قفل می‌کند، تا دو بار پشت‌سرهم زده نشود.
- خطای غیر از `AbortError` به مسیر کپی می‌ریزد، نه به شکست: اگر `share` از کار افتاده باشد، دست مشتری خالی نمی‌ماند.

چرا نشانی از `canonical` می‌آید

لینکی که مشتری برای دوستش می‌فرستد باید دقیقاً همان آدرسی باشد که در `canonical` و `og:url` صفحه نوشته شده. اگر نبود، پیش‌نمایشی که تلگرام و واتساپ می‌سازند به یک آدرس نگاه می‌کند و لینک به آدرسی دیگر می‌رود.

پس هیچ رشته‌ای دستی ساخته نمی‌شود: همان توصیف صفحه ساخته می‌شود و `canonical` اش برداشته می‌شود — همان چیزی که `lib/metadata.js` به Next می‌دهد و همان چیزی که `buildSitemap` در `shared/seo.js` می‌نویسد. سه مصرف‌کننده، یک منبع.

این را صریح می‌سنجد، برای هر سه نوع کالا: `tests/share.test.js`

```

const canonical = itemHeadState(item, { baseUrl: 'https://daneh.example' }).canonical;

expect(itemShareUrl(item, ENV)).toBe(canonical);
expect(itemShareUrl(item, ENV)).toBe(itemUrl('https://daneh.example', item));

```

دوراهیِ وسط نمودار، و دامی که در آن بود

ملاکِ «برگهٔ سیستم یا کپی؟» نوعِ نشانگر است، نه بودنِ `navigator.share`. تحلیلِ کاملش در تصمیم ۲۸ فصل ۹ است؛ خلاصه‌اش این: کرومِ ویندوز `navigator.share` دارد، ولی برگه‌ای که باز می‌کند پنجرهٔ خالیِ خودِ ویندوز است که بی‌کار بسته می‌شود – و در آن مسیر هیچ‌وقت کاری به کلیک‌بورد نمی‌رسید. یعنی کاربر دسکتاپ دکمه را می‌زد، پنبلی می‌آمد و می‌رفت، و لینک هیچ‌جا کپی نشده بود.

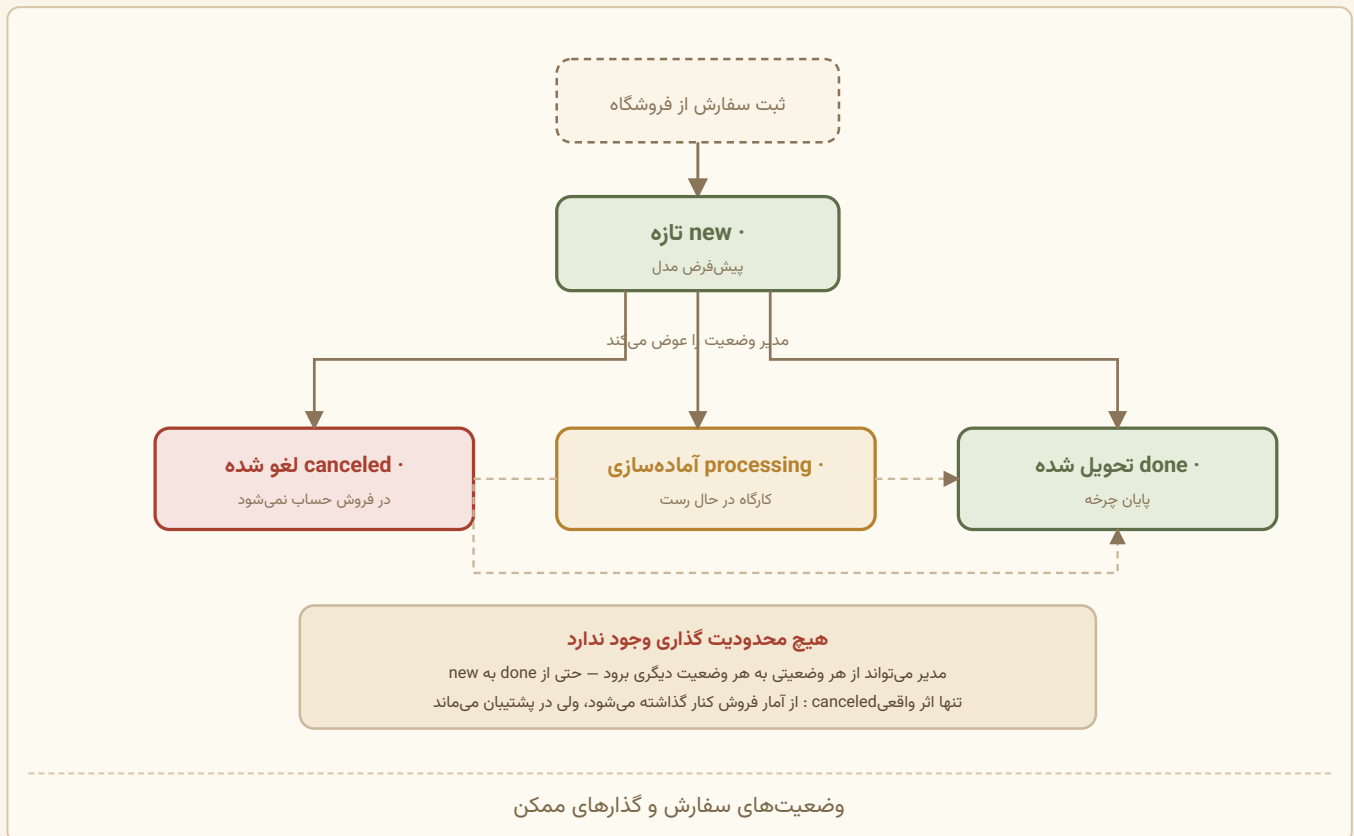
چهار پایان، و اینکه دو تایشان ساکت‌اند

mode	چه اتفاقی افتاده	توست
shared	برگهٔ سیستم باز شد و کاربر فرستاد	ندارد – خودِ برگه بازخورد داده
canceled	کاربر برگه را بست	ندارد – نظرش عوض شده، خطایی نبوده
copied	لینک در کلیک‌بورد است	«لینک این کالا کپی شد – حالا می‌توانید بفرستیدش»
failed	نه <code>share</code> و نه کپی	«کپی نشد – نشانی این صفحه را از نوار مرورگر بردارید»

پیام `failed` عمداً راهنمایی است نه اعلام خطا: کاری که کاربر می‌خواست هنوز شدنی است، فقط دستی.

و اینکه چرا توست از کانتکست نمی‌آید

سومین نقطهٔ شروع این جریان، مودالِ رهگیری‌شده است – و آن در شکافِ `@modal` رندر می‌شود، بیرون از `ShopProvider` صفحه. پس پیام از انبارهٔ ماژولی `lib/toastStore.js` می‌آید و `<Toast />` یک بار در `app/layout.jsx` می‌نشیند، بالای هر دو شکاف. تصمیم ۲۶ فصل ۹.



نکته مهم: canceled تنها وضعیتی است که در محاسبات آماری اثر دارد:

• در stats.js: `{ $match: { status: { $ne: 'canceled' } } }`

• در reports.js تابع addOrder:

```
if (o.status === 'canceled') { stats.canceled += 1; return; }
```

• اما در خروجی پشتیبان می‌ماند — کامنت بالای reports.js: «سفارش لغوشده در «فروش» حساب نمی‌شود ولی در فهرست و در فایل پشتیبان می‌ماند — پشتیبان باید کامل باشد.»

انتقال وضعیت هیچ محدودیتی ندارد؛ مدیر می‌تواند از done به new برگردد. برای این اندازه از کسب‌وکار (یک رستری کوچک) منطقی است، اما در سیستم بزرگ‌تر یک ماشین حالت با گذارهای مجاز لازم بود.

۷. الگوریتم‌ها و منطق تجاری

این فصل هر الگوریتم غیرپیش‌پاافتاده پروژه را با تکه‌کد واقعی و توضیح خط‌به‌خط بررسی می‌کند.

۷.۰ کدام الگوریتم کجا اجرا می‌شود

پیش از هر چیز، یک نقشه. با مورد ۲۶ پرسش «این کد کجا اجرا می‌شود؟» سه جواب پیدا کرد، نه دو تا – و برای خواندن بقیه فصل لازم است بدانید هر تابع در کدام ستون می‌نشیند.

اجرا می‌شود در	یعنی چه	چه چیزی آنجاست
فرآیند Express	نود، با دسترسی مستقیم به مونگو	، روترها، <code>siteUrl.js</code> ، <code>cors.js</code> ، <code>track.js</code> ، <code>stock.js</code> ، <code>jalali.js</code> ، قلاب‌های مدل
فرآیند Next، روی سرور	نود، بدون مونگو – از راه API می‌خواند	کامپوننت‌های سروری <code>lib/metadata.js</code> ، <code>lib/data.js</code> ، <code>app/**</code> ، <code>lib/seo.js</code> ، و <code>art.js</code> هنگام رندر اولیه
مرورگر	بعد از hydrate	، <code>ShopContext</code> ، <code>blend.js</code> ، <code>blendVisual.js</code> ، <code>cartStorage.js</code> ، و همه رویدادها <code>useReveal.js</code>

و بسته مشترک در هر سه اجرا می‌شود. این تازه است: پیش از مهاجرت، `pricing.js` دو مصرف‌کننده داشت (مرورگر و Express)؛ حالا سه تا.

فایل shared	در Express	در Next روی سرور	در مرورگر
<code>pricing.js</code>	بازمحاسبه قطعی سفارش	قیمت کارتها و جمع سبد در اولین رندر	همان جمع، زنده با هر کلیک
<code>taxonomy.js</code>	اعتبارسنجی مدل <code>Item</code>	برچسب‌ها هنگام رندر کارت	همان، در تعامل
<code>seo.js</code>	<code>sitemap.xml</code> و <code>robots.txt</code>	<code>generateMetadata</code> ، JSON-LD، <code>itemPath</code>	فقط <code>itemPath</code> برای لینک کارتها

سه نتیجه عملی از این جدول:

(۱) هیچ‌کدام از این فایل‌ها اجازه ندارند به `window` یا به مونگوس دست بزنند. توابع خالص‌اند و همین است که اجازه می‌دهد در هر سه محیط اجرا شوند. `siteBaseUrl` در `web/src/lib/site.js` تنها جایی است که `window` را نگاه می‌کند، و صریحاً `typeof window === 'undefined'` را می‌سنجد.

(۲) قیمتی که مشتری می‌بیند حالا دو بار پیش از ثبت حساب می‌شود – یک بار روی سرور موقع ساختن HTML، یک بار در مرورگر بعد از hydrate – و هر دو باید یک عدد بدهند، وگرنه ری‌اکت ناسازگاری می‌بیند. چون `computeTotals` تابع خالص است و ورودی‌اش در هر دو یکی است، می‌دهند.

نکته

یک استثنا هست: «پیشنهادهای روز» با `Date.now()` انتخاب می‌شود و تنها محاسبه پروژه است که به لحظه اجرا وابسته است. جزئیاتش در ۷.۱۱.

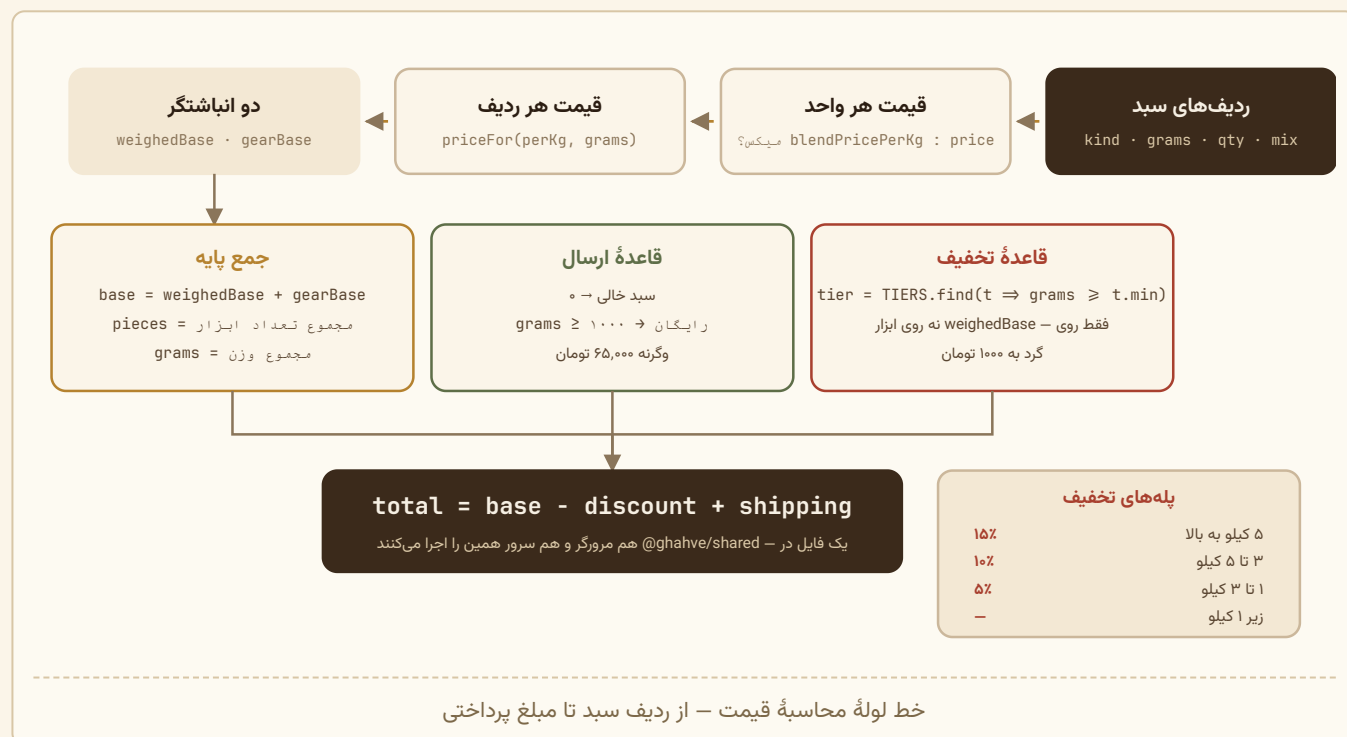
۳) بازمحاسبه سمت سرور سر جایش هست. هیچ‌کدام از این دو محاسبه جای `computeTotals` در `routes/orders.js` را نمی‌گیرد. سرور همچنان کالاها را از پایگاه داده می‌خواند و قیمت را از نو حساب می‌کند – یکی بودن فرمول فقط تضمین می‌کند نتیجه با آنچه کاربر دیده یکی دربیاید.

۷.۱ محاسبه قیمت – `shared/pricing.js`

یک فایل، در بسته مشترک `@ghahve/shared`. هم مرورگر و هم سرور همین را `import` می‌کنند:

```
import { computeTotals, lineTotal, unitPriceFor } from '@ghahve/shared/pricing.js';
```

پیش از این دو نسخه آینه‌ای وجود داشت (یکی در `client`، یکی در `server`) که باید دستی همگام می‌ماندند؛ حالا واگرایی از اساس ممکن نیست. این جای بازمحاسبه سمت سرور را نمی‌گیرد – سرور همچنان به عدد ارسالی از مرورگر اعتماد نمی‌کند – فقط تضمین می‌کند نتیجه با آنچه کاربر دیده یکی دربیاید.



ثابت‌ها

```
export const TIERS = [
  { min: 5000, rate: 0.15, label: '۱۵%' },
  { min: 3000, rate: 0.10, label: '۱۰%' },
  { min: 1000, rate: 0.05, label: '۵%' },
  { min: 0, rate: 0, label: '' }
];

export const SHIPPING = 65000; // تومان
export const FREE_SHIPPING_FROM = 1000; // گرم
```

چرا ترتیب نزولی؟ چون بعداً با `.find()` جست‌وجو می‌شود:

```
const tier = TIERS.find((t) => grams >= t.min);
```

`Array.prototype.find` اولین تطبیق را برمی‌گرداند. با ترتیب نزولی، اولین تطبیق همیشه بالاترین پله واجد شرایط است. با ۳۵۰۰ گرم: `3500 >= 5000` ؟ نه. `3500 >= 3000` ؟ بله $\rightarrow 10\%$.

و ردیف `{min: 0, rate: 0}` نقش `else` را بازی می‌کند: `.find()` هیچ‌وقت `undefined` بر نمی‌گرداند، پس نیازی به بررسی `null` نیست.

تابع `priceFor`

```
/**
 * قیمت را به نزدیک‌ترین ۱۰۰۰ تومان گرد می‌کند */
export const priceFor = (pricePerKg, grams) =>
  Math.round((pricePerKg * grams) / 1000 / 1000) * 1000;
```

سه عملیات در یک خط:

مرحله	عملیات	مثال (۱۸۵۰۰۰۰ تومان/کیلو، ۲۵۰ گرم)
۱	<code>pricePerKg * grams</code>	<code>۱۸۵۰۰۰۰ * ۲۵۰ = ۴۶۲,۵۰۰,۰۰۰</code>
۲	<code>/ 1000</code> (گرم $\rightarrow$ کیلو)	<code>۴۶۲,۵۰۰</code>
۳	<code>/ 1000 ... * 1000</code>	<code>۴۶۲,۵۰۰ \rightarrow \text{round}(۴۶۲,۵) = ۴۶۳</code>

نتیجه: ۴۶۳,۰۰۰ تومان.

آن `*1000 ... /1000` تکنیک استاندارد گرد کردن به مضرب است. عبارت `Math.round(x / n) * n` عدد را به نزدیک‌ترین مضرب `n` می‌برد.

چرا گرد کردن؟ قهوه به گرم فروخته می‌شود و ضرب‌های اعشاری اجتناب‌ناپذیر است. عدد `۴۶۲,۵۰۰` برای فاکتور دستی و پرداخت نقدی ناخوشایند است؛ `۴۶۳,۰۰۰` نیست.

نکته

این گرد کردن رو به بالا و پایین است (`Math.round`)، نه همیشه بالا. پس گاهی به نفع مشتری و گاهی به نفع فروشنده است. فروشگاه‌هایی که همیشه رو به بالا گرد می‌کند از `Math.ceil` استفاده می‌کند.

```
export function blendPricePerKg(item, mix, lookup) {
  const parts = Array.isArray(mix) && mix.length ? mix : item.components || [];
  if (!parts.length) return item.price;

  let total = 0;
  let percentSum = 0;

  for (const p of parts) {
    const bean = lookup(p.slug);
    if (!bean) return item.price; // ترکیب ناقص - به قیمت پایه برگرد
    const percent = Number(p.percent) || 0;
    total += bean.price * percent;
    percentSum += percent;
  }

  if (percentSum <= 0) return item.price;

  /* percentSum معمولاً ۱۰۰ است؛ تقسیم بر خودش یعنی اگر
  کمی کمتر یا بیشتر بود هم نتیجه معنی‌دار بماند. */
  const weighted = total / percentSum;
  return Math.round((weighted + (item.surcharge || 0)) / 1000) * 1000;
}
```

خط ۲: اگر ترکیب دلخواه داده شده، همان؛ وگرنه ترکیب رسمی کالا. این همان چیزی است که به کارت کالا اجازه می‌دهد قیمت رسمی را نشان دهد و به ساز میکس اجازه می‌دهد قیمت زنده ترکیب کاربر را.

خط ۳ و ۱۱ و ۱۷ – سه fallback امن. هر سه به `item.price` برمی‌گردند:

- ترکیب خالی است
 - یکی از دانه‌ها پیدا نشد (مثلاً مدیر حذفش کرده)
 - مجموع درصدها صفر یا منفی است
- نتیجه: هیچ‌وقت `NaN` یا `Infinity` روی صفحه نمی‌آید.

خط ۱۹ – چرا تقسیم بر `percentSum` و نه بر ۱۰۰؟

فرض کنید به‌خاطر گرد شدن، مجموع ۹۹ شده باشد:

```
۷۹,۲۰۰,۰۰۰ = ۱,۲۰۰,۰۰۰ × %۶۶    سرادو
۴۸,۸۴۰,۰۰۰ = ۱,۴۸۰,۰۰۰ × %۳۳    مونسون
total = ۱۲۸,۰۴۰,۰۰۰
```

- تقسیم بر ۱۰۰: `۱,۲۸۰,۴۰۰` ← ۱٪ قیمت گم شده
- تقسیم بر ۹۹: `۱,۲۹۳,۳۳۳` ← میانگین وزنی درست

با تقسیم بر مجموع واقعی، نسبت‌ها حفظ می‌شوند حتی اگر جمع دقیقاً ۱۰۰ نباشد.

مثال کامل

میکس `espresso-70-30` با ترکیب پیش‌فرض:

```

۸۴,۰۰۰,۰۰۰ = ۱,۲۰۰,۰۰۰ × %۷۰    سرادو
۴۴,۴۰۰,۰۰۰ = ۱,۴۸۰,۰۰۰ × %۳۰    مونسون
total = ۱۲۸,۴۰۰,۰۰۰
percentSum = ۱۰۰

weighted = ۱۲۸,۴۰۰,۰۰۰ / ۱۰۰ = ۱,۲۸۴,۰۰۰
surcharge = ۴۰,۰۰۰
۱,۳۲۴,۰۰۰ = جمع
گرد round(۱۳۲۴) × ۱۰۰۰ = ۱,۳۲۴,۰۰۰ = تومان هر كيلو

```

حالا اگر مشتری رويستا را به ۶۰٪ ببرد:

```

۴۸,۰۰۰,۰۰۰ = ۱,۲۰۰,۰۰۰ × %۴۰    سرادو
۸۸,۸۰۰,۰۰۰ = ۱,۴۸۰,۰۰۰ × %۶۰    مونسون
total = ۱۳۶,۸۰۰,۰۰۰
weighted = ۱,۳۶۸,۰۰۰ + ۴۰,۰۰۰ = ۱,۴۰۸,۰۰۰    تومان هر كيلو

```

قيمت همان لحظه روي کارت عوض مي‌شود.

تابع `unitPriceFor`

```

export function unitPriceFor(line, lookup) {
  if (line.item.isBlend && lookup) {
    return blendPricePerKg(line.item, line.mix, lookup);
  }
  return line.item.price;
}

```

يك لايه نازك تصميم‌گيري: ميكس → محاسبه، غيرميكس → قيمت ثابت. شرط `&& lookup` محافظ است براي وقتي كه تابع بدون lookup صدا زده شود.


```
export function computeTotals(lines, lookup) {
  let grams = 0, pieces = 0;
  let weighedBase = 0, gearBase = 0;

  for (const l of lines) {
    if (l.kind === 'gear') {
      pieces += l.qty;
      gearBase += l.item.price * l.qty;
    } else {
      grams += l.grams;
      weighedBase += priceFor(unitPriceFor(l, lookup), l.grams);
    }
  }

  const base = weighedBase + gearBase;
  const tier = TIERS.find((t) => grams >= t.min);
  const discount = Math.round((weighedBase * tier.rate) / 1000) * 1000;
  const shipping = lines.length === 0 ? 0 : grams >= FREE_SHIPPING_FROM ? 0 : SHIPPING;

  return {
    grams, pieces, base, discount,
    discountLabel: tier.label,
    shipping,
    total: base - discount + shipping
  };
}
```

چهار انباشتگر جدا نگه داشته می‌شوند: `grams`، `pieces`، `weighedBase`، `gearBase`. دلیلش سه قاعده تجاری متفاوت است:

قاعده ۱ – تخفیف فقط روی کالای وزنی.

```
const discount = Math.round((weighedBase * tier.rate) / 1000) * 1000;
```

توجه: `weighedBase` نه `base`. اگر مشتری ۵ کیلو قهوه (۶ میلیون تومان) و یک آسیاب ۷,۸۰۰,۰۰۰ تومانی بخرد، ۱۵٪ فقط روی ۶ میلیون اعمال می‌شود، نه روی ۱۳/۸ میلیون.

متن سایت هم این را صریح می‌گوید (در `HomeShell.jsx`، توضیح بخش ابزار):

قیمت این‌ها به ازای هر عدد است و تخفیف پلکانی فقط روی کالاهای وزنی (قهوه و پودر) اعمال می‌شود.

قاعده ۲ – آستانه تخفیف بر پایه وزن است، نه پول.

```
const tier = TIERS.find((t) => grams >= t.min);
```

خرید ۸,۵۰۰,۰۰۰ تومان بلو مانیتین در ۱۰۰ گرم، هیچ تخفیفی نمی‌گیرد. خرید ۱ کیلو میکس ۵۰/۵۰ به قیمت ۸۲۰,۰۰۰ تومان، ۵٪ می‌گیرد.

این تصمیم با مدل کسب‌وکار جور است: تخفیف حجمی برای صرفه‌جویی در رست و بسته‌بندی است، نه پاداش خرید گران.

قاعده ۳ – ارسال هم بر پایه وزن.

```
const shipping = lines.length === 0 ? 0 : grams >= FREE_SHIPPING_FROM ? 0 : SHIPPING;
```

سه حالت:

- سبد خالی → صفر (تا در سبد خالی «۶۵,۰۰۰ تومان ارسال» نشان داده نشود)
- وزن $1000 \leq$ گرم → رایگان
- وگرنه → ۶۵,۰۰۰

هشدار

سبدي که فقط ابزار دارد (`grams === 0`) همیشه ۶۵,۰۰۰ تومان ارسال می‌دهد – حتی اگر یک ماشین اسپرسو ۹۶ میلیون تومانی باشد. طبق کد این رفتار عمدی به نظر می‌رسد (چون شرط بر پایه `grams` است) اما احتمالاً در عمل باید بازبینی شود.

قاعدهٔ ۴ – تخفیف هم گرد می‌شود.

```
Math.round((weighedBase * tier.rate) / 1000) * 1000
```

پس مبلغ تخفیف هم عددی گرد است و فاکتور نهایی هیچ اعشاری ندارد.

تابع `lineTotal`

```
export function lineTotal(l, lookup) {  
  return l.kind === 'gear'  
    ? l.item.price * l.qty  
    : priceFor(unitPriceFor(l, lookup), l.grams);  
}
```

برای نمایش قیمت هر ردیف و ذخیره در `Order.lines[].lineTotal`

ردیف ۱: یرگاجف، ۵۰۰ گرم، هر کیلو ۱,۸۵۰,۰۰۰

`priceFor(1850000, 500) = round(925) × 1000 = ۹۲۵,۰۰۰`

ردیف ۲: میکس ۷۰/۳۰ با نسبت ۵۰/۵۰، ۷۰۰ گرم

`blendPricePerKg = (1200000×50 + 1480000×50)/100 + 40000`

`= ۱,۳۴۰,۰۰۰ + ۴۰,۰۰۰ = ۱,۳۸۰,۰۰۰`

`priceFor(1380000, 700) = round(966) × 1000 = ۹۶۶,۰۰۰`

ردیف ۳: کیف وی ۲،۶۰۰ عدد × ۱,۲۸۰,۰۰۰ = ۲,۵۶۰,۰۰۰

`grams = ۵۰۰ + ۷۰۰ = ۱۲۰۰`

`pieces = ۲`

`weighedBase = ۹۲۵,۰۰۰ + ۹۶۶,۰۰۰ = ۱,۸۹۱,۰۰۰`

`gearBase = ۲,۵۶۰,۰۰۰`

`base = ۴,۴۵۱,۰۰۰`

`tier: ۱۲۰۰ ≥ ۱۰۰۰ → ۵%`

`discount = round(1891000 × 0.05 / 1000) × 1000`

`= round(94.55) × 1000 = ۹۵,۰۰۰`

`shipping: ۱۲۰۰ ≥ ۱۰۰۰ → (۰) رایگان`

`total = ۴,۴۵۱,۰۰۰ - ۹۵,۰۰۰ + ۰ = ۴,۳۵۶,۰۰۰ تومان`

۷.۲ ریاضی میکس — `web/src/lib/blend.js`

مسئله: مشتری یک اهرم را می‌کشد، بقیه باید طوری جابه‌جا شوند که مجموع دقیقاً ۱۰۰ بماند.

تابع `applyPercent` — خطبه‌خط

```
export function applyPercent(parts, slug, raw) {
  const self = parts.find((p) => p.slug === slug);
  if (!self) return parts;

  const others = parts.filter((p) => p.slug !== slug);
  if (others.length === 0) return parts;

  const want = clamp(Math.round(Number(raw) || 0), 0, 100);
  let delta = want - self.percent;
  if (delta === 0) return parts;
```

خط ۲-۶: محافظ‌های ورودی. اگر دانه پیدا نشد یا تنها دانه بود، هیچ کاری نکن.

خط ۸: `clamp` مقدار را در بازه ۰..۱۰۰ نگه می‌دارد.

خط ۱۰: اگر تغییری نبود، همان آرایه قبلی برگردانده می‌شود. این برای React مهم است: مرجع یکسان یعنی `rerender` اضافه نمی‌شود.

```

/* جای خالی بقیه در جهتی که باید حرکت کنند:
اگر این دانه زیاد می‌شود، بقیه باید کم شوند. */
const roomOf = (p) => (delta > 0 ? p.percent : 100 - p.percent);

const totalRoom = others.reduce((s, p) => s + Math.max(0, roomOf(p)), 0);
if (totalRoom <= 0) return parts;

```

مفهوم «جای خالی» (room) هسته الگوریتم است:

- اگر این دانه زیاد می‌شود ($\text{delta} > 0$)، بقیه باید کم شوند. حداکثر کاهش هر دانه = مقدار فعلی‌اش (p.percent).
- اگر این دانه کم می‌شود، بقیه باید زیاد شوند. حداکثر افزایش هر دانه = $100 - \text{p.percent}$.

یعنی هیچ‌جایی برای حرکت نیست (مثلاً همه صفرند و می‌خواهیم این یکی را کم کنیم). $\text{totalRoom} \leq 0$

```

const need = Math.min(Math.abs(delta), totalRoom);
const dir = delta > 0 ? 1 : -1;
const target = self.percent + dir * need;

```

need مقدار واقعی حرکت است. اگر کاربر بخواهد از ۳۰ به ۹۰ برود ($\text{delta} = ۶۰$) اما بقیه فقط ۴۰ جا داشته باشند، حرکت به ۷۰ محدود می‌شود. یعنی اهرم زودتر از انتها متوقف می‌شود — رفتار درست، چون مجموع نمی‌تواند از ۱۰۰ بیشتر شود.

```

/* سهم هر دانه به نسبت جای خالی‌اش */
const shares = others.map((p) => {
  const room = Math.max(0, roomOf(p));
  return { slug: p.slug, room, take: Math.floor((need * room) / totalRoom) };
});

```

تخصیص نسبی: دانه‌ای که جای خالی بیشتری دارد، سهم بیشتری از حرکت می‌گیرد. Math.floor تضمین می‌کند هیچ‌کس بیش از سهمش نگیرد، اما باقیمانده می‌ماند.

```

/* باقیمانده گردکردن را یکی‌یکی پخش می‌کنیم تا
مجموع مو به مو درست دربیاید. */
let left = need - shares.reduce((s, x) => s + x.take, 0);
for (let i = 0; left > 0 && i < shares.length * 200; i++) {
  const s = shares[i % shares.length];
  if (s.take < s.room) { s.take++; left--; }
}

```

پخش باقیمانده به شیوه round-robin. مثلاً اگر $\text{need} = 10$ و سه دانه هرکدام $\text{floor}(3.33) = 3$ بگیرند، جمع ۹ می‌شود و ۱ واحد می‌ماند. این حلقه آن ۱ واحد را به اولین دانه‌ای که جا دارد می‌دهد.

شرط $\text{s.take} < \text{s.room}$ جلوگیری می‌کند از اینکه دانه‌ای بیش از ظرفیتش بگیرد.

یک **محافظ حلقه بی‌نهایت** است. از نظر ریاضی نباید لازم باشد (چون $\text{need} \leq \text{totalRoom}$)، اما اگر ورودی خراب باشد، برنامه قفل نمی‌کند.

```
const takeBy = new Map(shares.map((s) => [s.slug, s.take]));

return parts.map((p) => {
  if (p.slug === slug) return { ...p, percent: target };
  const take = takeBy.get(p.slug);
  if (!take) return p;
  return { ...p, percent: p.percent - dir * take };
});
}
```

اعمال نهایی به صورت تغییرناپذیر (immutable). توجه کنید که `if (!take) return p;` همان شیء قبلی را برمی‌گرداند – بهینه‌سازی برای React.

`p.percent - dir * take`: اگر `dir = 1` (این دانه زیاد شد)، بقیه کم می‌شوند؛ اگر `dir = -1`، بقیه زیاد می‌شوند.

مثال عددی

ترکیب اولیه: `A=50, B=30, C=20`. کاربر A را به ۸۰ می‌برد.

```
delta = 80 - 50 = +30 → dir = +1
roomOf(B) = 30 (می‌تواند تا ۳۰ کم شود B، چون delta > 0)
roomOf(C) = 20
totalRoom = 50
need = min(30, 50) = 30
target = 50 + 30 = 80

shares:
  B: take = floor(30 * 30 / 50) = floor(18) = 18
  C: take = floor(30 * 20 / 50) = floor(12) = 12
left = 0 → 30 = جمع

نتیجه: A=80, B=30-18=12, C=20-12=8
✓ 100 = 8+12+80 = مجموع
```

حالت کرانی: کاربر A را به ۱۰۰ می‌برد.

```
delta = +50, totalRoom = 50, need = 50
B: floor(50*30/50) = 30 → B=0
C: floor(50*20/50) = 20 → C=0
نتیجه: mixError → A=100, B=0, C=0: «میکس باید دست‌کم دو دانه داشته باشد»
```

دکمه «افزودن» غیرفعال می‌شود چون `mixError` فقط دانه‌های `percent > 0` را می‌شمارد.

توابع کمکی

```
export function addBean(parts, slug, share = 20) {
  if (parts.some((p) => p.slug === slug)) return parts;
  const withNew = [...parts, { slug, percent: 0 }];
  return applyPercent(withNew, slug, share);
}
```

هوشمندانه: دانه با صفر درصد اضافه می‌شود (پس مجموع همچنان ۱۰۰ است) و بعد `applyPercent` آن را به ۲۰ می‌برد و از بقیه کم می‌کند. یعنی هیچ منطق جداگانه‌ای برای «افزودن» لازم نیست.

```
export function removeBean(parts, slug) {
  if (parts.length <= 2) return parts;
  const zeroed = applyPercent(parts, slug, 0);
  return zeroed.filter((p) => p.slug !== slug);
}
```

همان الگو برعکس: اول صفر کن (تا سهمش بین بقیه پخش شود)، بعد حذف کن.

قاعدهٔ «میکس حداقل دو دانه» را در سطح اا اعمال می‌کند. و در `BlendsSection` این بی‌اثری تشخیص داده می‌شود:

```
const drop = (slug) =>
  setMix((prev) => {
    const next = removeBean(prev, slug);
    if (next === prev) toast('میکس دست‌کم به دو دانه نیاز دارد');
    return next;
  });
```

مقایسهٔ مرجع (`next === prev`) کافی است چون تابع در حالت بی‌اثر همان آرایه را برمی‌گرداند.

```
export function mixError(parts) {
  const live = parts.filter((p) => p.percent > 0);
  if (live.length < 2) return 'میکس باید دست‌کم دو دانه داشته باشد';
  const sum = sumOf(live);
  if (Math.abs(sum - 100) > 1) return `است ${sum} مجموع درصدها باید ۱۰۰ باشد، الان`;
  return '';
}
```

کامنت بالایش: «همان قواعدی که سرور هم بررسی می‌کند.» رواداری `> 1` برای خطای گرد کردن است.

۷.۳ حالت تصویری میکس — `web/src/lib/blendVisual.js`

حالت تصویری هیچ درصدی حساب نمی‌کند. هرچه به نسبت‌ها مربوط است از `applyPercent` بالا می‌آید و همان‌جا می‌ماند؛ آنچه اینجاست سه محاسبهٔ نمایشی است — و هر سه تابع خالص‌اند تا بشود بدون مرورگر تستشان کرد.

(۱) `pourPlan` — بودجهٔ زمانی، نه زمان هر کیسه

مسئله: اگر هر کیسه زمان ثابتی بگیرد، میکس دو دانه‌ای یک ثانیه طول می‌کشد و میکس هفت دانه‌ای سه و نیم ثانیه. تجربهٔ یکسان یعنی کل مرحله ثابت باشد و سهم هر کیسه از آن درآید.

```
export const POUR_BUDGET = 2600; // کل مرحله ۲
export const POUR_MIN = 320; // کوتاه‌ترین ریختن قابل‌دیدن
```

```
const sum = live.reduce((s, p) => s + Number(p.percent), 0);
const share = live.length * min >= budget;
const extra = budget - live.length * min;

const ms = share
  ? Math.round(budget / live.length)
  : Math.round(min + (percent / sum) * extra);
```

دو حالت:

- **حالت عادی** – بودجه به کف همه کیسه‌ها می‌رسد. هر کیسه `min` می‌گیرد به‌علاوه سهمی از `extra` به نسبت درصدش. پس کیسه ۶۰٪ محسوس‌تر می‌ریزد ولی کیسه ۵٪ هم دیده می‌شود.
 - **حالت شلوغ** (`share`) – آن‌قدر دانه هست که `live.length * min` از بودجه بیشتر شده. اینجا نسبت رها می‌شود و بودجه مساوی پخش می‌شود، وگرنه مرحله ۲ کش می‌آمد.
- و باقیمانده گردکردن روی آخرین کیسه می‌نشیند:

```
const drift = budget - plan.reduce((s, x) => s + x.ms, 0);
last.ms = Math.max(120, last.ms + drift);
```

بدون این خط، هفت بار `Math.round` می‌توانست چند ده میلی‌ثانیه اختلاف بسازد. `Math.max(120, ...)` هم جلوی منفی شدن را می‌گیرد.

هر ردیف برنامه، سطح قیف را هم پیش و پس از خودش حمل می‌کند:

```
const from = (done / sum) * 100;
done += percent;
return { slug, percent, index: i, ms, from, to: (done / sum) * 100 };
```

تقسیم بر `sum` (نه بر ۱۰۰) یعنی حتی اگر مجموع دقیقاً ۱۰۰ نبود، قیف در پایان کاملاً پر می‌شود.

۲) `sackLayout` – نگه داشتن دهانه کیسه سر قیف

بزرگی هر کیسه با سهمش عوض می‌شود:

```
const scale = Number((0.72 + Math.min(1, Number(p.percent) / 100) * 0.46).toFixed(3));
```

بازه ۰٫۷۲ تا ۱٫۱۸ – ملایم، تا تفاوت دیده شود ولی کیسه کوچک هم هنوز کیسه باشد.

اما همین مقیاس یک مسئله هندسی می‌سازد. کیسه حول پایه‌اش می‌چرخد، پس با چرخش `tilt` دهانه‌اش به‌اندازه `mouth * scale` از پایه فاصله می‌گیرد – و این فاصله برای کیسه کوچک و بزرگ فرق دارد. بدون تصحیح، کیسه کوچک بالای هوا خالی می‌کرد.

```
const rad = (scene.tilt * Math.PI) / 180;
const up = [Math.sin(rad), -Math.cos(rad)]; // بردار یکه محور کیسه
const reach = scene.mouth * scale;

hx: scene.hopper.x - reach * up[0],
hy: scene.hopper.y - reach * up[1]
```

یعنی: «پایه را از سر قیف، به‌اندازه `reach` در خلاف جهت محور عقب ببر.» نتیجه این است که دهانه هر کیسه – با هر بزرگی – دقیقاً روی `SCENE.hopper` می‌ایستد. `blend-visual.test.js` همین را با محاسبه معکوس می‌سنجد.

فاصله کیسه‌ها هم با تعدادشان فشرده می‌شود تا از قاب بیرون نزنند:

```
const room = scene.first - scene.last;
const slot = live.length > 1 ? Math.min(scene.slot, room / (live.length - 1)) : 0;
```

و `x: scene.first - i * slot` یعنی اولین انتخاب مشتری راست‌ترین کیسه است – جهت خواندن فارسی.

```
export function hopperLevel(plan, k) {
  if (!plan.length || k <= 0) return 0;
  const i = Math.min(k, plan.length) - 1;
  return Number((plan[i].to / 100).toFixed(4));
}
```

عدد بازگشتی بین ۰ و ۱ است و مستقیماً به یک متغیر CSS می‌رود. نکته اصلی‌اش این است که سطح از درصد تجمعی می‌آید نه از «چند کیسه از چند کیسه»: با ترکیب ۹۰٪ و ۱۰٪، بعد از کیسه اول قیف باید ۹۰٪ پر باشد، نه ۵۰٪.

رنگ میکس — mixTone در art.js

رنگ قیف و بسته پایانی، میانگین وزنی رنگ رست دانه‌هاست:

```
export const roastTone = (meter) => ROAST_TONE[meter] || ROAST_TONE[3];

export function mixTone(parts) { /* میانگین وزنی در فضای sRGB */ }
```

میانگین در همان فضای sRGB گرفته می‌شود؛ چون هر پنج رنگ رست روی یک طیف قهوه‌ای نزدیک به هم‌اند، میانگین ساده هم رنگ باورپذیری می‌دهد و نتیجه‌اش قابل پیش‌بینی و تست‌پذیر است.

مهم‌تر اینکه **ROAST_TONE** همان جدولی است که کارتها از آن رنگ می‌گیرند — صادر شده، نه کپی‌شده. وگرنه یک دانه در کارت یک رنگ می‌شد و در ساز میکس رنگی دیگر.

کم حرکتی

```
export const prefersReducedMotion = () =>
  typeof window !== 'undefined' &&
  typeof window.matchMedia === 'function' &&
  window.matchMedia('(prefers-reduced-motion: reduce)').matches;
```

دو نگرهبان **typeof** لازم‌اند تا همین فایل در محیط node (تست) هم import شود. با روشن بودن این تنظیم، **MixVisual** هیچ تایمری نمی‌سازد و با یک کلیک مستقیم به مرحله ۴ می‌رود: همان بسته، همان قیمت، همان ردیف سبد — فقط بدون نمایش.

۷.۴ قلاب اعتبارسنجی مدل Item — خط به خط

```
.itemSchema.pre('validate', ...), server/src/models/Item.js
```

قاعده ۱ — سازگاری دسته و نوع

```
const allowedGroups = GROUPS_BY_KIND[this.kind] || [];
if (this.kind && !allowedGroups.includes(this.group)) {
  this.invalidate('group', `دسته «${this.group}»`);
}
```

جلوگیری از حالت‌های نامعتبر مثل «ابزار با دسته light».

`this.invalidate(field, message)` روش Mongoose برای افزودن خطای اعتبارسنجی است. برخلاف `throw`، اجازه می‌دهد بقیه بررسی‌ها هم انجام شوند و همه خطاها یک‌جا جمع شوند.

قاعده ۲ و ۳ – پیش‌فرض‌های ظاهری

```
if (this.kind === 'gear') {
  if (!this.shape) this.shape = 'mug';
  if (!GEAR_SHAPES.includes(this.shape)) this.invalidate('shape', 'طرح ابزار نامعتبر است');
  if (!this.mat) this.mat = 'steel';
  if (!GEAR_MATERIALS.includes(this.mat)) this.invalidate('mat', 'جنس ابزار نامعتبر است');
  this.tone = '';
}

if (this.kind === 'powder') {
  if (!this.shape) this.shape = 'scoop';
  if (!POWDER_SHAPES.includes(this.shape)) this.invalidate('shape', 'طرح پودر نامعتبر است');
  if (!this.tone) this.tone = 'cocoa';
  if (!POWDER_TONES.includes(this.tone)) this.invalidate('tone', 'رنگ پودر نامعتبر است');
  this.mat = '';
}
```

الگوی «پیش‌فرض بگذار، بعد اعتبارسنجی کن»: خالی بودن خطا نیست (مقدار معقول می‌گیرد) اما مقدار نامعتبر خطاست. کامنت بالایش دلیل را می‌گوید: «شکل/جنس/رنگ باید جزو طرح‌های موجود باشند وگرنه کارت خالی رندر می‌شود». یعنی این اعتبارسنجی مستقیماً از یک نگرانی سمت کلاینت (`art.js`) می‌آید.

`this.tone = ''` و `this.mat = ''` پاک‌سازی متقاطع است: ابزار رنگ پودر ندارد و پودر جنس ندارد.

قاعده ۴ و ۵ – قهوه در برابر غیرقهوه

```
if (this.kind === 'coffee') {
  /* قهوه طرح اختصاصی ندارد؛ تصویرش از درجه رست و دسته ساخته می‌شود */
  this.shape = '';
  this.mat = '';
  this.tone = '';
  this.pairs = [];
  this.grindable = true; // هر قهوه‌ای آسیاب‌شدنی است
} else {
  /* فقط قهوه میکس می‌شود و آسیاب می‌خورد */
  this.grindable = false;
  this.isBlend = false;
  this.house = false;
  this.customizable = false;
  this.components = [];
  this.pool = [];
  this.surcharge = 0;
}
```

`grindable` یک فیلد مشتق است. در `WRITABLE` روتر نیست، پس مدیر نمی‌تواند مستقیماً تنظیمش کند. همیشه از `kind` محاسبه می‌شود.

این باعث می‌شود قاعده تجاری «فقط قهوه آسیاب می‌شود» غیرقابل نقض باشد – نه با فرم پنل، نه با درخواست دستی API.

```
if (this.isBlend) {
  if (this.components.length < 2) {
    this.invalidate('components', 'یک میکس دستکم به دو دانه نیاز دارد');
  } else {
    const sum = this.components.reduce((s, c) => s + c.percent, 0);
    /* یک واحد رواداری، چون درصدها گرد می‌شوند */
    if (Math.abs(sum - 100) > 1) {
      this.invalidate('components', `مجموع درصدها باید ۱۰۰ باشد، الان ` + `${Math.round(sum)}` + ` است`);
    }
    const seen = new Set();
    for (const c of this.components) {
      if (seen.has(c.slug)) {
        this.invalidate('components', `دو بار در میکس آمده است «${c.slug}» دانه`);
      }
      seen.add(c.slug);
      if (c.min > c.max) {
        this.invalidate('components', 'کمینه درصد نمی‌تواند از بیشینه بزرگ‌تر باشد');
      }
    }
  }
}
```

چهار بررسی: تعداد، مجموع، تکرار، و بازهٔ معتبر. `Set` برای تشخیص تکرار در $O(n)$ به جای $O(n^2)$.

پیام خطا ``${Math.round(sum)}`` را نشان می‌دهد تا مدیر بدانند چقدر فاصله دارد.

قاعدهٔ ۷ – پاک‌سازی غیرمیکس

```
} else {
  /* غیرمیکس نه جزء دارد، نه ویژهٔ خانه است */
  this.components = [];
  this.pool = [];
  this.customizable = false;
  this.house = false;
  this.surcharge = 0;
}
```

اگر مدیر تیک «میکس» را بردارد، بقایای ترکیب قبلی در سند نمی‌ماند.

گرد کردن نهایی

```
if (typeof this.price === 'number') this.price = Math.round(this.price);
if (typeof this.meter === 'number') this.meter = Math.round(this.meter);

/* موجودی هم همین‌طور - نیم گرم و نیم عدد معنا ندارد.
مقایسهٔ اتمی هنگام ثبت سفارش روی همین عدد صحیح
انجام می‌شود، پس اعشار فقط در دسر است. */
if (typeof this.stock === 'number') this.stock = Math.max(0, Math.round(this.stock));
```

کامنت: «قیمت را گرد می‌کنیم تا اعشار سرگردان در پایگاه داده نماند.»

شرط `typeof ... === 'number'` روی `stock` عمدی است و با `!= null` فرق دارد: `null` یعنی نامحدود و باید دست‌نخورده رد شود. `Math.max(0, ...)` هم لایهٔ دوم است کنار `min: [0]` خود `schema`.

مسئله: رقابت بین دو مشتری

دو مشتری هم‌زمان آخرین ۵۰۰ گرم یرگاجف را سفارش می‌دهند. الگوی ساده «اول بخوان، بعد بنویس» این‌طور شکست می‌خورد:

زمان	مشتری الف	مشتری ب
t1	را می‌خواند → stock ۵۰۰	
t2		را می‌خواند → stock ۵۰۰
t3	۵۰۰ ≥ ۵۰۰ ✓	
t4		۵۰۰ ≥ ۵۰۰ ✓
t5	می‌نویسد stock = 0	
t6		می‌نویسد stock = 0

نتیجه: یک کیلو فروخته شد، ۵۰۰ گرم داشتیم

فاصله بین خواندن و نوشتن همان چیزی است که باید حذف شود.

راه‌حل: شرط و کاهش در یک عملیات

```
const updated = await ItemModel.findOneAndUpdate(
  { slug: item.slug, stock: { $gte: need } }, // شرط
  { $inc: { stock: -need } }, // کاهش
  { new: true, projection: { stock: 1 } }
);
```

مونگو این دو را زیر یک قفل سند انجام می‌دهد. همان سناریو با این کد:

```
t1    الف: findOneAndUpdate → ۵۰۰ ≤ ۵۰۰، سند قفل، stock = 0
t2    ب:   findOneAndUpdate → ۵۰۰ ≤ ۰، سند قفل، null برمی‌گردد
```

دقیقاً یکی برنده می‌شود. هیچ transaction ای لازم نیست — که روی مونگوی تک‌گرهی اصلاً در دسترس نیست.

۰ یعنی نامحدود، و دو بار محافظت می‌شود

```
export function isTracked(item) {
  return typeof item?.stock === 'number' && Number.isFinite(item.stock);
}
...
if (!isTracked(item)) continue; // نامحدود — اصلاً لمس نمی‌شود
```

اگر این نگهبان نبود چه می‌شد؟ `$inc` روی `null` خطا می‌دهد و شرط `{ $gte: need }` هم با `null` جور نمی‌شود. یعنی حتی در صورت فراموشی، نتیجه به‌جای خراب شدن «ناموجود» می‌شد — یک شکست امن، ولی اشتباه. پس نگهبان صریح لازم است.

همه یا هیچ

هر ردیف جدا رزرو می‌شود، پس ردیف سوم می‌تواند بعد از موفقیت دو ردیف اول شکست بخورد. `taken` هر رزرو موفق را نگه می‌دارد:

```
const taken = [];
for (const line of lines) {
  ...
  if (!updated) {
    await releaseStock(taken, ItemModel); // همه را با هم برگردان
    const fresh = await ItemModel.findOne({ slug: item.slug }, { stock: 1 }).lean();
    const left = typeof fresh?.stock === 'number' ? fresh.stock : 0;
    return { error: shortMessage(item, left), taken: [] };
  }
  taken.push({ slug: item.slug, amount: need });
}
```

و `releaseStock` که با `Promise.all` همه را موازی برمی‌گرداند:

```
export async function releaseStock(taken, ItemModel) {
  if (!taken?.length) return;
  await Promise.all(
    taken.map((t) => ItemModel.updateOne({ slug: t.slug }, { $inc: { stock: t.amount } }))
  );
}
```

سه جا صدا زده می‌شود: وسط شکست حلقه، و در روتر اگر `Order.create` خطا بدهد. نتیجه یک ضمانت ساده است: سفارش یا کامل ثبت می‌شود یا اصلاً.

چرا موجودی برای پیام خطا دوباره خوانده می‌شود؟

عددی که در `item.stock` داریم از لحظه `Item.find` در ابتدای مسیر آمده و ممکن است چند صد میلی‌ثانیه قدیمی باشد. اگر همان را در پیام بگذاریم، مشتری می‌خواند «فقط ۳۰۰ گرم مانده» و بعد سفارش ۳۰۰ گرمی هم رد می‌شود. یک `findOne` اضافه فقط در مسیر شکست اجرا می‌شود، پس هزینه‌اش روی مسیر عادی صفر است.

دو دام باقی‌مانده

هشدار

۱) موجودی میکس‌ها. `stock` از خود کالای میکس کم می‌شود، نه از دانه‌های اجزایش. پس فروش یک میکس، موجودی یرگافِ داخلش را کم نمی‌کند. توصیه: میکس‌ها را نامحدود (`null`) بگذارید. ۲) رزرو بی‌بازگشت. اگر سفارشی بعداً «لغو» شود، موجودی خودکار برنمی‌گردد – تغییر وضعیت هیچ کاری با انبار ندارد. مدیر باید دستی عدد را اصلاح کند.

۷.۶ پیگیری سفارش بدون حساب – `server/src/lib/track.js`

مشتری حساب کاربری ندارد، پس اثبات مالکیت باید از چیزی بیاید که فقط خودش دارد: جفتِ «شماره سفارش + شماره موبایل». کد را فقط کسی دارد که رسید را دیده، و شماره را فقط کسی که خودش سفارش داده. سه الگوریتم کوچک این را نگه می‌دارند.

۱) نرمال‌سازی – یک شکل واحد برای هر شکل تایپ

```
export function normalizePhone(raw) {
  const digits = toLatin(raw).replace(/\\D/g, '');
  if (!digits) return '';

  let d = digits;
  if (d.startsWith('0098')) d = d.slice(4);
  else if (d.length === 12 && d.startsWith('98')) d = d.slice(2);

  if (d.length === 10 && d.startsWith('9')) d = '0' + d;
  return d;
}
```

ورودی	خروجی
۰۹۱۲۱۲۳۴۵۶۷	09121234567
0912 123-4567	09121234567
+989121234567	09121234567
00989121234567	09121234567
9121234567	09121234567

ترتیب شرط‌ها مهم است: اول پیش‌شماره کشور برداشته می‌شود، بعد صفر ابتدایی اضافه. شرط `d.length === 12` جلوی برداشتن اشتباهی «۹۸» از شماره‌ای که واقعاً با ۹۸ شروع می‌شود را می‌گیرد.

اگر شکل ورودی اصلاً موبایل ایرانی نبود، همان رقم‌ها برمی‌گردند – تصمیم درباره معتبر بودن با صداکننده است، نه با این تابع.

هم همان کار را برای کد می‌کند: رقم فارسی، حروف کوچک و نبود خط تیره را می‌پذیرد و همیشه شکل ذخیره‌شده `normalizeCode` (A1B2-3456) را برمی‌گرداند.

۲) مقایسه در زمان ثابت

```
export function safeEqual(a, b) {
  const ha = crypto.createHash('sha256').update(String(a ?? ''), 'utf8').digest();
  const hb = crypto.createHash('sha256').update(String(b ?? ''), 'utf8').digest();
  return crypto.timingSafeEqual(ha, hb);
}
```

چرا اول هش؟ `crypto.timingSafeEqual` طول‌های نابرابر را قبول نمی‌کند و پرتاب خطا می‌کند – که خودش یک نشت زمانی است: شماره ۱۱ رقمی در برابر شماره ۵ رقمی فوراً خطا می‌داد، شماره ۱۱ رقمی غلط کندتر.

خروجی sha256 همیشه ۳۲ بایت است، پس مقایسه همیشه یک شکل و یک هزینه دارد.

۳) پاسخ یکسان برای دو شکست متفاوت

```
const doc = await OrderModel.findOne({ code: wantedCode }).lean();

/* حتی وقتی سفارشی پیدا نشده، یک مقایسه انجام می‌دهیم -
تا زمان پاسخ در هر دو حالت یکی باشد. */
const stored = normalizePhone(doc?.customer?.phone);
const matches = safeEqual(stored || 'no-order', wantedPhone);

if (!doc || !matches) {
  return { status: 404, error: NOT_FOUND_MESSAGE };
}
```

دو نکته در این پنج خط:

الف) !doc جداگانه بررسی نمی‌شود تا زودتر برگردد. اگر برای کد ناموجود سریع‌تر پاسخ می‌دادیم، خود زمان پاسخ می‌گفت «این کد وجود دارد» – و صفحه پیگیری به ابزار شمارش سفارش‌ها تبدیل می‌شد.

ب) رشته نگهبان no-order عمداً با فاصله شروع می‌شود، پس هیچ‌وقت با یک شماره نرمال‌شده واقعی برابر نمی‌شود.

NOT_FOUND_MESSAGE یک ثابت صادرشده است، نه رشته درجا – تا اگر روزی کسی خواست پیام را عوض کند، مجبور شود در یک جا عوضش کند و دو پیام متفاوت درنیاید.

whitelist خروجی

سند سفارش را بازسازی می‌کند، نه اینکه فیلدهای ناخواسته را از آن حذف کند: `publicOrderView`

```
return {
  code: order.code,
  status: order.status,
  createdAt: order.createdAt,
  customer: { name: order.customer?.name || '' },
  lines: (order.lines || []).map((l) => ({ kind: l.kind, name: l.name, ... })),
  totals: { base: ..., discount: ..., shipping: ..., total: ... }
};
```

فرق این دو رویکرد در آینده است: با `delete`، هر فیلدی که فردا به مدل اضافه شود به‌طور پیش‌فرض بیرون می‌رود؛ با `whitelist`، به‌طور پیش‌فرض داخل نمی‌آید. `track.test.js` دقیقاً همین را می‌سنجد – یک سفارش با فیلدهای ساختگی `adminNote` و `profitMargin` می‌سازد و انتظار دارد هیچ اثری از آن‌ها در خروجی نباشد.

نشانی، شماره تماس و یادداشت مشتری عمداً بیرون‌اند: دیدن وضعیت و فهرست کالاها به نشانی نیازی ندارد، و اگر روزی کسی جفت کد و شماره را حدس زد نباید نشانی خانه کسی را هم برداشته باشد. نام می‌ماند، چون به مشتری اطمینان می‌دهد سفارش خودش را می‌بیند و چیزی بیشتر از آنچه خودش نوشته نشان نمی‌دهد.

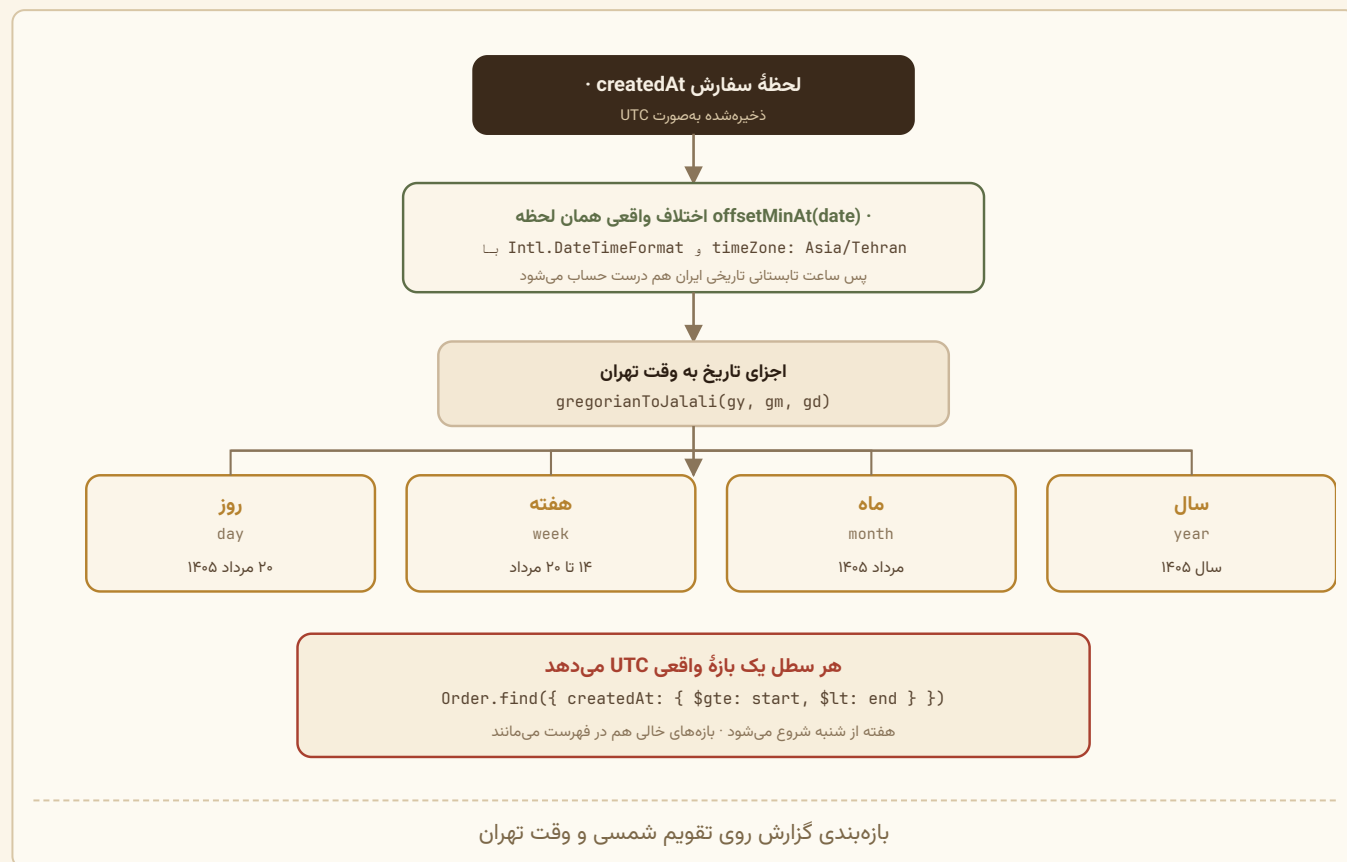
۷.۷ تقویم شمسی – `server/src/lib/jalali.js`

چرا دست‌نویس؟

کامنت بالای فایل:

گزارش‌ها باید با تقویمی دسته‌بندی شوند که مدیر می‌بیند: «مرداد» یعنی مرداد، نه August. پس بازه‌ها روی تقویم جلالی بسته می‌شوند، نه میلادی.

یک ماه شمسی هیچ تطابقی با ماه میلادی ندارد. مرداد ۱۴۰۵ از ۲۳ ژوئیه تا ۲۲ اوت ۲۰۲۶ است. اگر گزارش ماهانه روی تقویم میلادی بسته می‌شد، هر «ماه» بین دو ماه شمسی تقسیم می‌شد.



هستهٔ تبدیل – الگوریتم jalali

```
function jalCal(jy) {  
  const breaks = [  
    -61, 9, 38, 199, 426, 686, 756, 818, 1111, 1181, 1210,  
    1635, 1701, 1866, 2020, 2620, 3220, 3628  
  ];  
  ...  
}
```

این آرایه نقاط شکست چرخهٔ کبیسه در تقویم جلالی است. برخلاف تقویم میلادی که قاعدهٔ ساده‌ای دارد، تقویم جلالی بر پایهٔ اعتدال بهاری واقعی است و الگوی کبیسه‌اش در بازه‌های ۳۳ ساله (گاهی ۲۹ یا ۳۷) تکرار می‌شود.

توابع `g2d`، `d2g`، `d2jInternal` از شمارهٔ روز ژولین (Julian Day Number) به‌عنوان پل بین دو تقویم استفاده می‌کنند – تکنیک استاندارد تبدیل تقویم.

```

/* ایران از سال ۱۴۰۱ ساعت تابستانی ندارد و اختلافش با
UTC همیشه ۳:۳۰ است - ولی سفارش‌های قدیمی‌تر ممکن است
از دورهٔ ساعت تابستانی (۴:۳۰) باشند. پس به‌جای عدد
ثابت، اختلاف واقعی همان لحظه را از خود سیستم
می‌پرسیم تا هیچ سفارشی یک روز جابه‌جا نیفتد. */
const TEHRAN = 'Asia/Tehran';
const FALLBACK_OFFSET_MIN = 210;

const tzFormat = new Intl.DateTimeFormat('en-US', {
  timeZone: TEHRAN, hour12: false,
  year: 'numeric', month: '2-digit', day: '2-digit',
  hour: '2-digit', minute: '2-digit', second: '2-digit'
});

function offsetMinAt(date) {
  try {
    const p = zoned(date);
    const asUTC = Date.UTC(p.year, p.month - 1, p.day, p.hour, p.minute, p.second);
    return Math.round((asUTC - date.getTime()) / 60000);
  } catch {
    return FALLBACK_OFFSET_MIN;
  }
}

```

ترفند: `Intl.DateTimeFormat` با `timeZone: 'Asia/Tehran'` اجزای تاریخ محلی تهران را می‌دهد. اگر آن اجزا را دوباره به‌عنوان UTC تفسیر کنیم و از زمان واقعی کم کنیم، اختلاف منطقهٔ زمانی به دست می‌آید.

این کار پایگاه دادهٔ IANA داخل موتور جاوااسکریپت را به کار می‌گیرد، که تاریخچهٔ کامل ساعت تابستانی ایران را دارد.

```

function zoned(date) {
  const p = {};
  for (const part of tzFormat.formatToParts(date)) {
    if (part.type !== 'literal') p[part.type] = Number(part.value);
  }
  /* ساعت ۲۴ در برخی نسخه‌ها یعنی نیمه‌شب */
  if (p.hour === 24) p.hour = 0;
  return p;
}

```

آن `if (p.hour === 24)` یک باگ شناخته‌شدهٔ برخی نسخه‌های V8 را دور می‌زند.

تصحیح دومرحله‌ای نیمه‌شب

```

/* نیمه‌شب تهران یک روز میلادی، به‌صورت لحظهٔ واقعی (UTC).
دو بار تصحیح می‌کنیم تا اگر آن روز مرز ساعت تابستانی
بوده هم به نیمه‌شب درست برسیم. */
function tehranMidnight(gy, gm, gd) {
  const naive = Date.UTC(gy, gm - 1, gd);
  let t = naive - FALLBACK_OFFSET_MIN * 60000;
  for (let i = 0; i < 2; i += 1) t = naive - offsetMinAt(new Date(t)) * 60000;
  return new Date(t);
}

```


مسئله: برای دانستن اختلاف زمانی در یک لحظه، باید آن لحظه را بدانیم؛ اما برای دانستن آن لحظه، باید اختلاف را بدانیم. راه حل: تکرار.

با حدس اولیه ۳:۳۰، اختلاف واقعی را بپرس، تصحیح کن، دوباره بپرس. دو تکرار برای همگرایی کافی است.

بازه بندی

```
export function bucketOf(date, period) {
  const p = tehranParts(date);

  if (period === 'day') {
    const j = gregorianToJalali(p.gy, p.gm, p.gd);
    return makeBucket('day', j.jy, j.jm, j.jd);
  }

  if (period === 'week') {
    /* getUTCDay: ۰ یکشنبه ... ۶ شنبه. فاصله تا شنبه قبل: */
    const back = (p.weekday + 1) % 7;
    const sat = new Date(Date.UTC(p.gy, p.gm - 1, p.gd) - back * DAY_MS);
    const j = gregorianToJalali(sat.getUTCFullYear(), sat.getUTCMonth() + 1, sat.getUTCDate());
    return makeBucket('week', j.jy, j.jm, j.jd);
  }
  ...
}
```

فرمول $(\text{weekday} + 1) \% 7$ تعداد روزهای عقب‌گرد تا شنبه قبل را می‌دهد:

یعنی	$(w+1)\%7$	روز	weekday
خودش	۰	شنبه	۶
یک روز عقب	۱	یکشنبه	۰
دو روز عقب	۲	دوشنبه	۱
شش روز عقب	۶	جمعه	۵

هفته ایرانی از شنبه شروع می‌شود – نه یکشنبه (آمریکا) و نه دوشنبه (ISO). این جزئیاتی است که کتابخانه‌های عمومی معمولاً اشتباه می‌گیرند.

برچسب‌های هوشمند هفته

```
} else if (period === 'week') {
  end = new Date(start.getTime() + 7 * DAY_MS);
  const last = toJalali(new Date(end.getTime() - DAY_MS));
  key = `w:${jy}-${pad(jm)}-${pad(jd)}`;
  label = jm === last.jm
    ? `${faDigits(jd)} تا ${faDigits(last.jd)} ${JMONTHS[jm - 1]} ${faDigits(jy)}`
    : `${faDigits(jd)} ${JMONTHS[jm - 1]} تا ${faDigits(last.jd)} ${JMONTHS[last.jm - 1]}
      ${faDigits(last.jy)}`;
}
```

اگر هفته داخل یک ماه باشد: «۱۴ تا ۲۰ مرداد ۱۴۰۵». اگر بین دو ماه باشد: «۲۹ مرداد تا ۴ شهریور ۱۴۰۵».

```
export function bucketSeries(period, count, now = new Date()) {
  const list = [];
  let b = bucketOf(now, period);
  for (let i = 0; i < count; i += 1) {
    list.unshift(b);
    b = previousBucket(b, period);
  }
  return list;
}

export function previousBucket(bucket, period) {
  return bucketOf(new Date(bucket.start.getTime() - DAY_MS), period);
}
```

ترفند `previousBucket`: به جای محاسبهٔ حسابی «ماه قبل» (که با طول‌های متغیر ماه شمسی پیچیده است)، یک روز از شروع بازه کم می‌کند و می‌پرسد «این لحظه در کدام بازه است؟» — چون یک روز قبل از اول مرداد، حتماً در تیر است.

`unshift` باعث می‌شود آرایه از قدیم به جدید مرتب باشد. و بازه‌های خالی هم در فهرست می‌مانند — پس نمودار گزارش حفره ندارد.

دو قالب عددی

```
const faDigits = (v) => String(v).replace(/\d/g, (d) => '۰۱۲۳۴۵۶۷۸۹'[d]);

export function jalaliDateString(date) {
  const j = toJalali(date);
  return `${j.jy}/${pad(j.jm)}/${pad(j.jd)}`; // ← رقم لاتین
}
```

کامنت دلیل تفاوت را می‌گوید:

برچسب‌ها با رقم فارسی نوشته می‌شوند تا کنار بقیهٔ عددهای پنل یکدست باشند. رقم لاتین فقط در فایل خروجی می‌ماند، چون اکسل باید عدد را بشناسد.

```
const rows = await Order.aggregate([
  { $match: { status: { $ne: 'canceled' } } },
  { $unwind: '$lines' },
  {
    $group: {
      _id: '$lines.slug',
      /* واحدها متفاوت‌اند، پس «تعداد دفعات سفارش» را
      ملاک می‌گیریم که بین وزنی و عددی قابل مقایسه است. */
      orders: { $sum: 1 },
      grams: { $sum: '$lines.grams' },
      qty: { $sum: '$lines.qty' },
      revenue: { $sum: '$lines.LineTotal' }
    }
  },
  { $sort: { orders: -1, revenue: -1 } },
  { $limit: 60 }
]);
```

چرا `$sum: 1` و نه وزن یا مبلغ؟ کامنت جواب می‌دهد: نمی‌شود ۵۰۰ گرم قهوه را با ۲ عدد ماگ مقایسه کرد. اما «چند بار سفارش داده شده» برای هر دو معنی یکسانی دارد.

به‌عنوان معیار دوم مرتب‌سازی، تساوی‌ها را می‌شکنند. `revenue`

سپس مرحلهٔ دوم در جاوااسکریپت:

```
const salesBySlug = new Map(rows.map((r) => [r._id, r]));

const items = await Item.find({ active: true, excludeTop: { $ne: true } })
  .lean({ virtuals: true });

const scored = items
  .map((it) => {
    const s = salesBySlug.get(it.slug);
    return { ...it, sales: s ? {...} : null };
  })
  .filter((it) => it.pinnedTop || it.sales)
  .sort((a, b) => {
    /* سنجاق‌شده‌ها همیشه بالا */
    if (a.pinnedTop !== b.pinnedTop) return a.pinnedTop ? -1 : 1;
    const ao = a.sales?.orders || 0;
    const bo = b.sales?.orders || 0;
    if (bo !== ao) return bo - ao;
    return (a.rank || 999) - (b.rank || 999);
  })
  .slice(0, limit);
```

سه‌سطحی بودن مرتب‌سازی: ۱. سنجاق‌شده‌ها اول ۲. تعداد سفارش (نزولی) ۳. `rank` مدیر (صعودی) — برای شکستن تساوی

چرا `join` در جاوااسکریپت و نه `$lookup`؟ چون فروش از `orders` و فیلترها (`active`, `excludeTop`, `pinnedTop`) از `items` می‌آیند، و تعداد کالاها کوچک است (۱۰۶). دو کوئری ساده خواناتر از یک `aggregate` پیچیده با `$lookup` است.

به‌جای `excludeTop: false` — چون اسناد قدیمی ممکن است این فیلد را اصلاً نداشته باشند و `$ne: true` هم `false` و هم `undefined` را می‌گیرد.

در `slice(0, limit)` و `aggregate` در `{ $limit: 60 }` در انتها: مرحله اول سقف معقولی می‌گذارد، مرحله دوم برش نهایی را می‌زند.

۷.۹ تولید تصویر – `web/src/lib/art.js`

نکته

این فایل تابع خالص است و `CardArt` هیچ هوکی ندارد، پس روی سرور اجرا می‌شود: SVG کارتها داخل همان HTML اولی می‌آید که Next می‌فرستد. بهایش حجم HTML است و سودش این است که صفحه بدون اجرای هیچ جاوااسکریپتی تصویر دارد. بحث کاملش در فصل ۱۰، پرسش ۳۸.

بی‌خطر سازی متن پیش از رفتن داخل SVG

این باید اول بیاید، چون بر همه تولیدکننده‌های این فایل حاکم است.

خروجی `art.js` یک رشته است، نه گره React، و `CardArt` آن را با `dangerouslySetInnerHTML` تزریق می‌کند – یعنی React هیچ‌چیز را برایمان escape نمی‌کند و مرورگر هرچه اینجا نوشتیم را همان‌طور نشانه‌گذاری می‌خواند.

```
const ESCAPES = { '<': '&lt;', '>': '&gt;', '&': '&amp;', '"': '&quot;', "'": '&#39;' };
export const esc = (s) => String(s ?? '').replace(/[<>"']/g, (c) => ESCAPES[c]);
```

نام کالا را مدیر می‌نویسد و از پایگاه داده می‌آید. بدون این تابع، نامی مثل:

```
" onload="alert(1)
```

از `aria-label` بیرون می‌زد و به صفتِ اجراشدنی تبدیل می‌شد:

```
<-- esc بدون --!>
<svg aria-label="" onload="alert(1)">
<-- esc با --!>
<svg aria-label="&quot; onload=&quot;alert(1)">
```

پس هر متنی که از پایگاه داده می‌آید از اینجا رد می‌شود – نام کالا، نام دسته، و در حالت تصویری، نام میکس و نام تک‌تک دانه‌ها. سه جزئیات:

(۱) `String(s ?? '')` – نبود مقدار به رشته خالی تبدیل می‌شود، نه به `"undefined"` که روی کارت چاپ می‌شد.

(۲) یک بار `escape`، نه دو بار. چون `&` هم در همان یک `replace` است، نه در یک گذر جدا، رشته `A & B` می‌شود `A & B` و نه `A &amp; B`.

(۳) عددها هم خطر دارند. `zoom` داخل صفت `transform` می‌نشیند، پس اصلاً از راه رشته وارد نمی‌شود:

```
const numOr = (v, fallback) => {
  const n = Number(v);
  return Number.isFinite(n) && n > 0 ? n : fallback;
};
```

عدد نامعتبر به پیش فرض برمی گردد؛ چیزی برای escape کردن باقی نمی ماند.

هر چهار تولیدکننده (قهوه، ابزار، پودر، بسته میکس) را با نام خصمانه می سنجد و – نکته کمیاب – خودِ `card-art.test.js` سنجه را هم می سنجد: یک تست عمداً رشته `escape` نشده می سازد و انتظار دارد سنجه شکست را ببیند، تا اگر روزی `regex` بررسی خراب شد بقیه تست ها بی صدا سبز نمانند.

مولد شبه تصادفی با بذر

```
/**
 * مولد عدد شبه تصادفی با بذر ثابت (تا طرح هر قهوه تغییر نکند)
 */
function seeded(str) {
  let h = 2166136261;
  for (let i = 0; i < str.length; i++) { h ^= str.charCodeAt(i); h = Math.imul(h, 16777619); }
  return () => {
    h = Math.imul(h ^ (h >>> 15), 2246822507);
    h ^= h >>> 13;
    return ((h >>> 0) % 100000) / 100000;
  };
}
```

مرحله اول هاش FNV-1a است: `2166136261` و `16777619` ثابت های استاندارد FNV هستند. رشته به یک عدد ۳۲ بیتی تبدیل می شود.

مرحله دوم یک مولد `xorshift/multiply` است که هر بار عددی بین ۰ و ۱ برمی گرداند.

`Math.imul` ضرب ۳۲ بیتی صحیح انجام می دهد (بدون تبدیل به شناور) و `h >>> 0` عدد را به بدون علامت تبدیل می کند.

چرا اهمیت دارد؟ چون طرح هر کالا باید همیشه یکسان باشد. با `Math.random()` هر بار که صفحه رفرش می شد، چیدمان دانه ها عوض می شد و کارتها بی ثبات به نظر می رسیدند.

```
export function productArt(p) {
  const rnd = seeded(p.slug);
  const [bg1, bg2, accent] = GROUP_SCENE[p.group] || GROUP_SCENE.medium;
  const [c1, c2, crease] = ROAST_TONE[p.meter] || ROAST_TONE[3];
  const uid = 'k' + p.slug.replace(/^[a-z0-9]/gi, '');

  // چیدمان پایه؛ هر دانه کمی جابه‌جا و چرخانده می‌شود
  const spots = [
    [140, 78, 1.75], [64, 44, 1.0], [212, 92, 1.05],
    [50, 112, .82], [196, 34, .78], [104, 122, .7], [246, 126, .62]
  ];

  const beans = spots.map(([x, y, s]) => {
    const rot = Math.round(rnd() * 160 - 80);
    const sc = (s * (0.9 + rnd() * 0.22)).toFixed(2);
    const dx = Math.round(rnd() * 18 - 9);
    const dy = Math.round(rnd() * 14 - 7);
    const op = (0.72 + rnd() * 0.28).toFixed(2);
    return `<g transform="translate(${x + dx} ${y + dy}) rotate(${rot}) scale(${sc})" opacity="${op}">
      <ellipse rx="25" ry="17.5" fill="url(#b${uid})"/>
      <path d="M-19 0c6-7.5 6 7.5 19-7.5" fill="none" stroke="${crease}" stroke-width="2.6" stroke-
linecap="round"/>
      <ellipse cx="-7" cy="-7" rx="7" ry="3.4" fill="#FFFFFF" opacity=".16"/>
    </g>`;
  }).join('');
  ...
}
```

«تصادفی کنترل‌شده»: موقعیت پایه هفت دانه دستی طراحی شده (تا ترکیب بصری خوب باشد)، اما چرخش $\pm 80^\circ$ درجه، مقیاس $0.9-1.2$ برابر، جابه‌جایی $\pm 9^\circ$ پیکسل و شفافیت $0.72-0.98$ از مولد بذردار می‌آیند. هر دانه سه عنصر دارد: بیضی با گرادیان، شیار وسط، و یک بازتاب سفید کم‌رنگ.

رنگ‌ها از دو منبع:

```
const ROAST_TONE = {
  1: ['#D3A468', '#A9743A', '#EBC894'], // روشن‌ترین
  2: ['#BE8B4C', '#8E5C2A', '#DEB47F'],
  3: ['#9C6B3C', '#66401F', '#C79763'],
  4: ['#77492A', '#442715', '#A5734A'],
  5: ['#523020', '#26150B', '#84543A'] // تیره‌ترین
};
```

meter (درجهٔ رست ۱ تا ۵) رنگ دانه را تعیین می‌کند – یعنی قهوهٔ رست تیره واقعاً تیره‌تر کشیده می‌شود. این یک اطلاعات بصری واقعی است، نه تزئین.

GROUP_SCENE[group] پس‌زمینه را می‌دهد و کامنتش هوشمندانه است: «روشن نگه داشته می‌شوند تا دانه‌های تیره روی‌شان خوانا بمانند.»

```
const uid = 'k' + p.slug.replace(/^[a-z0-9]/gi, '');
...
<linearGradient id="b${uid}" ...>
<ellipse fill="url(#b${uid})"/>
```

شناسه‌های SVG در کل سند سراسری‌اند. اگر کارت همه `id="beanGradient"` داشتند، همه از اولی استفاده می‌کردند و همه کارت‌ها یک رنگ می‌شدند. با پیشوند slug، هر کارت گرادیان خودش را دارد.

پیشوند `'k'` (و `'v'` برای ابزار، `'p'` برای پودر) تضمین می‌کند شناسه با رقم شروع نشود — که در HTML4 نامعتبر بود و هنوز در بعضی موتورها مشکل‌ساز است.

کارت ابزار و پودر

```
export function gearArt(it) {
  const c = MATERIAL[it.mat] || MATERIAL.steel;
  const z = it.zoom || 1;
  const draw = (SHAPES[it.shape] || SHAPES.mug)(c, accent);
  ...
  <g transform="translate(140 76) scale(${z})">${draw}</g>
}
```

SHAPES یک شیء از ۳۶ تابع است که هرکدام رنگ‌های جنس را می‌گیرند و مسیر SVG برمی‌گردانند:

```
dripper: c => `
  <path d="M-40 -26 L40 -26 L11 24 L-11 24 Z" fill="${c.main}"/>
  <path d="M-40 -26 L0 -26 L0 24 L-11 24 Z" fill="${c.dk}" opacity=".18"/>
  ...`;
```

هر طرح در مختصات محلی حدود ۹۰×۹۰ با مرکز (۰،۰) کشیده شده، و `translate(140 76)` آن را وسط قاب ۱۵۰×۲۸۰ می‌گذارد. هم اجازه می‌دهد مدیر برای اشیای کوچک (مثل دماسنج، `zoom: 1.15`) یا بزرگ (موکاپات سه‌کاپ، `zoom: 0.85`) اندازه را تنظیم کند.

تفکیک رنگ از شکل: ۳۶ شکل × ۱۱ جنس = ۳۹۶ ترکیب ممکن، با نوشتن ۳۶ + ۱۱ تعریف.

```
const MATERIAL = {
  steel: { main: '#C3CAD0', dk: '#8A939B', lt: '#EFF3F6' },
  copper: { main: '#C8874E', dk: '#8E5729', lt: '#E8B683' },
  ...
};
```

هر جنس سه رنگ دارد: اصلی، سایه، روشنایی — که در همه طرح‌ها به‌طور یکنواخت استفاده می‌شوند.

برای پودر، `heap(tone, width)` تپه پودر را می‌کشد و `POWDER_SHAPES` ظرف یا ادویه کنارش را.

۷.۱۰ قالب‌بندی اعداد فارسی

```
export const toFa = (n) => Number(n || 0).toLocaleString('fa-IR');
```

دو کار همزمان: رقم‌های فارسی (۰-۹) و جداکننده هزارگان فارسی (، - U+066C، نه کاما).

۱,۸۵۰,۰۰۰ → 1850000

```
export function formatWeight(g) {
  if (g < 1000) return toFa(g) + ' گرم';
  const kg = g / 1000;
  return toFa(Number.isInteger(kg) ? kg : kg.toFixed(2)) + ' کیلوگرم';
}
```

ورودی	خروجی
250	۲۵۰ گرم
1000	۱ کیلوگرم
1500	۱٫۵۰ کیلوگرم
2750	۲٫۷۵ کیلوگرم

Number.isInteger(kg) جلوی «۱/۰۰ کیلوگرم» را می‌گیرد.

تبدیل معکوس

```
export function toLatinDigits(str) {
  return String(str ?? '')
    .replace(/[۰-۹]/g, (d) => String('۰۱۲۳۴۵۶۷۸۹'.indexOf(d)))
    .replace(/[۰-۹]/g, (d) => String('۰۱۲۳۴۵۶۷۸۹'.indexOf(d)))
    .replace(/,/g, '');
}
```

سه تبدیل: رقم فارسی → لاتین، رقم عربی → لاتین، حذف کاما.

چرا دو بلوک جدا؟ رقم فارسی (U+06F0–U+06F9) و رقم عربی (U+0660–U+0669) کدهای یونیکد متفاوتی دارند، هرچند شکل بعضی‌شان شبیه است. کاربر ایرانی معمولاً رقم فارسی می‌زند اما کیبوردهای عربی رقم عربی می‌دهند.

استفاده‌ها:

- CartDrawer: قبل از اعتبارسنجی شماره تلفن
- ClubSection: همان
- AdminItemForm: روی قیمت، rank، و درصدهای میکس

با کامنت: «تا مدیر بتواند قیمت را با کیبورد فارسی هم بنویسد.»


```
export function faDate(iso) {
  try {
    return new Intl.DateTimeFormat('fa-IR', {
      year: 'numeric', month: 'long', day: 'numeric',
      hour: '2-digit', minute: '2-digit'
    }).format(new Date(iso));
  } catch { return ''; }
}
```

Intl با locale fa-IR خودکار تقویم شمسی می‌دهد. یعنی در کلاینت اصلاً نیازی به jalali.js نیست. سرور آن را لازم دارد چون باید بازه‌ها را روی مرز روزهای شمسی ببندد — کاری که Intl انجام نمی‌دهد.

۷.۱۱ الگوریتم‌های کوچک‌تر

شناسهٔ ردیف سبد — web/src/lib/cartLine.js

```
export function mixSignature(mix) {
  if (!Array.isArray(mix) || mix.length === 0) return '';
  return [...mix]
    .filter((m) => Number(m.percent) > 0)
    .map((m) => `${m.slug}:${Math.round(Number(m.percent) || 0)}%`)
    .sort()
    .join(',');
}

export const lineKey = (slug, grind, mix) => `${slug}|${grind || ''}|${mixSignature(mix)}%`;
```

سه تصمیم:

- `[...mix]` کپی می‌سازد تا `.sort()` آرایهٔ اصلی را عوض نکند. این فقط ادب نیست: حالت تصویری با ترتیب انتخاب مشتری کیسه‌ها را می‌ریزد، پس مرتب کردن آرایهٔ اصلی نمایش را به هم می‌زد.
- `.filter(percent > 0)` داده‌های برداشته‌شده را حذف می‌کند، پس «۶۰/۴۰/۰» و «۶۰/۴۰» یک امضا دارند.
- `.sort()` ترتیب را نرمال می‌کند — داخل امضا، نه روی داده.

نتیجه: espresso-70-30|espresso|cerrado:60,monsooned:40

این تابع همراه `buildLine` از `ShopContext` بیرون کشیده شد وقتی ساز میکس دو حالت پیدا کرد. حالا هر دو راه — اهرم‌های ساده و حالت تصویری — از همین یک تابع رد می‌شوند، پس «یک میکس، از هر راهی که ساخته شود، دقیقاً همان ردیف سبد» یک ادعای تست‌پذیر است نه یک امید.

پیشنهاد روز

```
const dayIndex = () => Math.floor(Date.now() / 86400000);
const daily = featured.length ? featured[dayIndex() % featured.length] : null;
```

کامنت: «پیشنهاد روز هر روز عوض می‌شود، ولی در طول یک روز ثابت می‌ماند — تا اگر مشتری صفحه را دوباره باز کرد همان چیزی را ببیند که صبح دیده بود.»

86400000 میلی‌ثانیه یک روز است. تقسیم صحیح، شماره روز از epoch را می‌دهد. باقیمانده آن بر تعداد کالاهای منتخب، چرخش روزانه می‌سازد.

توجه: این محاسبه بر پایه UTC است، نه وقت تهران. پس پیشنهاد ساعت ۳:۳۰ بامداد عوض می‌شود، نه نیمه‌شب.

و یک نکته تازه که با رندر سمت سرور اضافه شد: این تنها محاسبه پروژه است که به لحظه اجرا وابسته است. سرور و مرورگر آن را در دو لحظه متفاوت حساب می‌کنند، و اگر آن دو لحظه دو طرف نیمه‌شب UTC بیفتند، دو کالای متفاوت انتخاب می‌شود – یعنی یک ناسازگاری hydration، یک بار در هر شبانه‌روز و فقط برای کسی که دقیقاً همان چند ثانیه صفحه را باز کرده باشد.

راه بستنش این است که انتخاب روی سرور انجام شود و نتیجه به شکل prop پایین برود. الان بسته نشده، چون اثرش یک کارت جابه‌جاشده است نه از دست رفتن داده – ولی جای دانستن دارد.

پله تخفیف بعدی

```
const nextTier = useMemo(() => {
  if (totals.grams <= 0) return null;
  return [...TIERS].reverse().find((t) => t.min > totals.grams) || null;
}, [totals.grams]);
```

و نمایشش در سبد:

```
{nextTier ? (
  <p className="next-tier">
    {formatWeight(nextTier.min - totals.grams)} تخفیف تا {nextTier.label} دیگر اضافه کنید تا تخفیف
  </p>
) : null}
```

با ۶۰۰ گرم: «۴۰۰ گرم دیگر اضافه کنید تا تخفیف ۵٪ فعال شود.» با ۶۰۰۰ گرم: `find` چیزی پیدا نمی‌کند → `null` → پیامی نشان داده نمی‌شود.

تولید slug از نام فارسی

```
function suggestSlug(name) {
  const map = {
    'kh': 'خ', 'h': 'ح', 'ch': 'چ', 'j': 'ج', 's': 'ث', 't': 'ت', 'p': 'پ', 'b': 'ب', 'a': 'آ', 'a': 'ا'
    , 't': 'ط', 'z': 'ض', 's': 'ص', 'sh': 'ش', 's': 'س', 'zh': 'ژ', 'z': 'ز', 'r': 'ر', 'z': 'ذ', 'd': 'د'
    , 'n': 'ن', 'm': 'م', 'l': 'ل', 'g': 'گ', 'k': 'ک', 'gh': 'ق', 'f': 'ف', 'gh': 'غ', 'a': 'ع', 'z': 'ظ'
    , '-': '-', 'y': 'ی', 'h': 'ه', 'v': 'و'
  };
  return [...String(name).toLowerCase()]
    .map((ch) => (/[a-z0-9]/.test(ch) ? ch : map[ch] ?? ''))
    .join('')
    .replace(/-+/g, '-')
    .replace(/^-|-$/g, '')
    .slice(0, 40);
}
```

پنج مرحله: تبدیل به آرایه نویسه (با `[...]`) که با یونیکد درست کار می‌کند، نگاشت هر نویسه، حذف خط تیره‌های تکراری، حذف خط تیره ابتدا و انتها، برش به ۴۰ نویسه.

`map[ch] ?? ''` یعنی نویسه‌های ناشناخته (اعراب، علائم) حذف می‌شوند.

نکته ظریف: '' (نیم فاصله، U+200C) هم در نگاشت هست. «رست‌خانه» بدون این، به rstkhaneh تبدیل می‌شد؛ با آن، rst-khaneh.

این فقط یک پیشنهاد است و مدیر می‌تواند تغییرش دهد؛ تا وقتی که slugTouched نشده باشد، با تایپ نام به‌روز می‌شود.

محافظه‌های regex در جست‌وجو

```
const rx = new RegExp(q.replace(/[\.\*\?^\$\{\}\(\)\[\]\|\]/g, '\\$&'), 'i');
```

این خط در سه فایل تکرار شده (reports.js, club.js, items.js). همه کاراکترهای خاص regex را escape می‌کند.

بدون این، جست‌وجوی (a+)+\$ می‌توانست یک ReDoS (حمله منع سرویس با regex) بسازد که CPU سرور را قفل کند. و جست‌وجوی . همه نتایج را برمی‌گرداند به‌جای اینکه دنبال نقطه بگردد.

\$& در رشته جایگزین یعنی «همان چیزی که تطبیق یافت»، پس . به \. تبدیل می‌شود.

۷.۱۲ نگهبان‌های ShopContext – الگوریتمی که هیچ عددی حساب نمی‌کند

این بخش درباره ریاضی نیست، ولی جایش همین‌جاست: یک الگوریتم ترتیبی است که اگر خراب شود، سبد مشتری بی‌صدا پاک می‌شود – بدون هیچ خطایی، بدون هیچ لاگی. با مورد ۲۶ سه نگهبان لازم شد و هر سه یک چیز می‌گویند: «هنوز نه».

مسئله

نسخه ویت سبد را در همان اولین رندر می‌خواند:

```
const [cart, setCart] = useState(loadCart); // ← دیگر این نیست
```

با رندر سمت سرور، همین یک خط سه مسئله می‌سازد و هر سه به یک نتیجه می‌رسند:

#	مسئله	نتیجه
۱	روی سرور localStorage وجود ندارد	خطا، یا سبد همیشه خالی در HTML
۲	سرور سبد خالی می‌فرستد و مرورگر در همان رندر اول سبد پُر می‌سازد	ری‌اکت ناسازگاری می‌بیند و می‌تواند درخت را دور بریزد
۳	افکت ذخیره پیش از افکت خواندن اجرا شود	آرایه خالی اولیه روی سبد ذخیره‌شده نوشته می‌شود

نگهبان یکم – hydrated

سبد همیشه با آرایه خالی شروع می‌شود، پس HTML سرور و اولین رندر مرورگر مو به مو یکی‌اند. خواندن در یک افکت یک‌باره است و نوشتن پشت پرچم:

```
const [cart, setCart] = useState([]);
const [hydrated, setHydrated] = useState(false);

/* خواندن سبد ذخیره‌شده، یک بار */
useEffect(() => {
  const saved = loadCart(browserStorage());
  if (saved.length) setCart(saved);
  setHydrated(true);
}, []);

/* نوشتن - نگه‌بان: پیش از خواندن، هیچ‌وقت ننویس */
useEffect(() => {
  if (!hydrated) return;
  saveCart(browserStorage(), cart);
}, [cart, hydrated]);
```

کامنت کد صریح است: «ترتیب مهم است، نه اینکه چه زمانی اجرا می‌شود.»

بهایش یک فریم است: تا اجرای افکت اول، شمارندهٔ سبد در هدر «گرم» است. عمداً با `suppressHydrationWarning` پنهان نشده — آن پرچم هشدار را خاموش می‌کند، نه مسئله را.

از کانتکست هم بیرون داده می‌شود، تا هرچه به سبد بستگی دارد بتواند تا آن لحظه حالت خنثی نشان بدهد. `hydrated`

نگهبان دوم — `catalogueComplete`

صفحهٔ اصلی همهٔ کالاها را دارد؛ صفحهٔ یک کالا عمداً فقط خودش و دانه‌هایش را (`fullCatalogue={false}`). خواندن صد و شش کالا برای نشان دادن یکی بی‌معنی است.

ولی افکتِ پاک‌سازی سبد ردیفی را که «کالایش در فهرست نیست» حذف می‌کند. روی فهرست ناقص، این یعنی دیدنِ صفحهٔ یک قهوه کل سبد را پاک می‌کرد.

```
useEffect(() => {
  if (!hydrated || !catalogueComplete || loading || items.length === 0) return;
  setCart((prev) => {
    const kept = prev.filter(
      (l) => bySlug.has(l.slug) && (l.mix || []).every((m) => bySlug.has(m.slug))
    );
    return kept.length === prev.length ? prev : kept;
  });
}, [hydrated, catalogueComplete, loading, items.length, bySlug]);
```

دو شرط اول هر دو لازم‌اند و هر کدام چیز جدایی می‌گویند:

- `catalogueComplete` — روی فهرست ناقص، «کالا در فهرست نیست» دلیلی برای حذف نیست.
- `hydrated` — پیش از خوانده شدن سبد، «ردیف نامعتبر» معنایی ندارد؛ سبد هنوز خالی است.

و `kept.length === prev.length ? prev : kept` هم عمدی است: اگر چیزی حذف نشده، همان آرایهٔ قبلی برمی‌گردد، نه کپی برابر آن. وگرنه هر بار که فهرست کالاها عوض شود ارجاع سبد هم عوض می‌شد و افکتِ نوشتن بی‌جهت اجرا می‌شد.

نگهبان سوم — «کامل کردن فهرست»

فهرست ناقص فقط یک حالت گذراست. سبد مال کل سایت است: مشتری می‌تواند دو قهوه در سبد داشته باشد و بعد صفحهٔ یک ابزار را باز کند. آن‌وقت `resolved` — که ردیف‌ها را با سند کالا جفت می‌کند — ردیف‌های بی‌جفت را کنار می‌گذارد، و `CartDrawer` سفارش را از همین `resolved` می‌سازد. یعنی ثبت سفارش از صفحهٔ یک کالا بی‌صدا بقیهٔ ردیف‌ها را می‌انداخت.

```
const fillTried = useRef(false);

useEffect(() => {
  if (!hydrated || catalogueComplete || fillTried.current || cart.length === 0) return;

  const missing = cart.some(
    (l) => !bySlug.has(l.slug) || (l.mix || []).some((m) => !bySlug.has(m.slug))
  );
  if (!missing) return;

  fillTried.current = true;
  reload();
}, [hydrated, catalogueComplete, cart, bySlug, reload]);
```

سه چیز که این افکت را بی‌خطر می‌کند:

۱) حلقه نمی‌سازد. بعد از `reload()` مقدار `catalogueComplete` روشن می‌شود و شرط دوم دیگر برقرار نیست. `fillTried` قفل دومی است، برای حالتی که `reload` خطا بدهد.

۲) برای بیشتر بازدیدکننده‌ها هیچ کاری نمی‌کند. شرط `cart.length === 0` همان اول برمی‌گرداند، پس صفحه کالا برای کسی که سبد ندارد سبک می‌ماند – یک درخواست، نه دو تا.

۳) جای «ردیف واقعاً حذف‌شده» را نمی‌گیرد. آن را نگهبان دوم برمی‌دارد، بعد از اینکه فهرست کامل شد.

و یک نگهبان چهارم که به سبد ربطی ندارد

`loadOnMount` – تنها جایی که `ShopProvider` هنوز خودش داده می‌خواند:

```
useEffect(() => {
  if (loadOnMount) reload();
}, [loadOnMount, reload]);
```

فقط پنل مدیریت این پرچم را می‌فرستد، چون صفحه‌هایش سروری نیستند که کسی فهرست کالاها را برایشان آماده کند – و فرم کالا برای پیش‌نمایش زنده میکس‌ها به قیمت دانه‌ها نیاز دارد. برای صفحه‌های فروشگاه هیچ‌وقت اجرا نمی‌شود.

چرا این‌ها الگوریتم‌اند، نه جزئیات پیاده‌سازی

چون هیچ‌کدام از این شرط‌ها را نمی‌شود با نگاه کردن به صفحه پیدا کرد. شکستشان دیده نمی‌شود: سبد پاک می‌شود و کاربر فکر می‌کند خودش کاری کرده. به همین دلیل دو تست اختصاصی دارند:

- `tests/cart-storage.test.js` – سبد ذخیره‌شده بازاعتبارسنجی می‌شود، شناسه ردیف از نو ساخته می‌شود، و حافظه خطا (حالت خصوصی مرورگر) فروشگاه را زمین نمی‌زند.
- `tests/cart-hydration.test.jsx` – یک چرخه کامل «رندر سرور ← hydrate» اجرا می‌شود و سبد باید دست‌نخورده بیرون بیاید. تنها تستی که یک شکست نامرئی را می‌گیرد.

۸. سیستم طراحی و استایل

۸.۱ فلسفه بصری

کامنت بالای `web/src/style.css` منبع الهام پالت را می‌گوید:

```
===== /*
رُست‌خانه دانه - استایل‌نامه
پالت از خود قهوه گرفته شده:
دانه خام سبز، پوسته گیلای قرمز، دانه رست‌شده تیره،
شیر و کارامل فنجان.
*/ =====
```

یعنی هر رنگ در سیستم، معادلی در دنیای واقعی قهوه دارد. این باعث می‌شود پالت منسجم باشد بدون اینکه از یک ابزار تولید پالت آمده باشد.

۸.۲ توکن‌های طراحی

همه توکن‌ها به صورت متغیر CSS روی `:root` تعریف شده‌اند (`web/src/style.css`)، خطوط ۸ تا ۳۷):

پالت رنگ

```
:root{
  --paper:      #F6EFE3;    /* کاغذ کاهی گرم */
  --paper-2:    #EFE4D2;
  --paper-3:    #FBF6EA;
  --line:       #DCCDB4;

  --espresso:   #1E1710;    /* دانه رست تیره */
  --espresso-2: #2E241A;

  --soft:       #6B5B45;    /* متن فرعی */
  --ink-2:      #4A3E2E;    /* متن معرفی روی زمینه روشن */

  --sage:       #A9B78C;    /* دانه خام */
  --sage-deep:  #5C6B47;
  --cherry:     #B23A2B;    /* گیلای قهوه */
  --cherry-dark: #8E2A1E;
  --brass:      #C8963C;    /* برنج ترازو */
  --latte:      #C9A87C;    /* شیر و قهوه */
  --crema:      #E8CBA0;
}
```

جدول کاربرد هر رنگ

توکن	مقدار	کجا استفاده می‌شود
--paper	#F6EFE3	پس‌زمینه <code>body</code> ، دکمه وزن، منوی موبایل
--paper-2	#EFE4D2	بخش‌های <code>.soft</code> ، برچسب‌های <code>.notes</code> ، <code>.mix-row</code> ، <code>.club</code>
--paper-3	#FBF6EA	بدنه کارت‌ها، <code>.input</code> ، <code>.search</code> ، فرم باشگاه
--line	#DCCDB4	همه حاشیه‌ها و جداکننده‌ها
--espresso	#1E1710	متن اصلی، <code>.band</code> ، هدر پنل، چپ فعال
--espresso-2	#2E241A	نوار اطمینان، hover دکمه سبد
--soft	#6B5B45	متن فرعی، <code>.section-desc</code> ، <code>.card-process</code>
--ink-2	#4A3E2E	<code>.lead</code> روی زمینه روشن
--sage	#A9B78C	گرادین سنج «سطح مهارت» ابزار
--sage-deep	#5C6B47	<code>.eyebrow</code> ، <code>.card-origin</code> ، focus حاشیه، تیک موفقیت
--cherry	#B23A2B	دکمه اصلی، <code>.badge</code> ، <code>.mix-val</code> ، <code>outline</code> فوکوس
--cherry-dark	#8E2A1E	hover دکمه اصلی، متن خطا
--brass	#C8963C	عدد سبد، <code>.readout-price</code> ، دسته اسلایدر، مرز پنل
--latte	#C9A87C	گرادین سنج رست، زیرعنوان پنل
--crema	#E8CBA0	<code>.eyebrow</code> روی زمینه تیره، آیکن نوار اطمینان

دو تصمیم درباره کنتراست

در کد دو کامنت وجود دارد که نشان می‌دهد رنگ‌ها بعد از بازبینی خوانایی تیره‌تر شده‌اند:

```

/* متن فرعی - کمی تیره‌تر از پالت اولیه، چون رنگ قبلی روی
زمینه کاهی به‌سختی خوانده می‌شد. */
--soft: #6B5B45;

/* تیره‌تر شد تا «تیترو کوچک» خوانا باشد */
--sage-deep: #5C6B47;
```

این نشان می‌دهد دسترس‌پذیری در طراحی لحاظ شده — نه به‌صورت خودکار، اما به‌صورت آگاهانه.

توکن‌های غیررنگی

```

--radius: 16px;
--radius-s: 10px;
--shadow: 0 18px 40px -28px rgba(30,23,16,.5);
--shadow-lift: 0 26px 50px -30px rgba(30,23,16,.62);
--wrap: 1200px;
--ease: cubic-bezier(.22,.61,.36,1);
```

دو شعاع، نه بیشتر. `--radius` (۱۶px) برای کارت‌ها و جعبه‌ها، `--radius-s` (۱۰px) برای فیلدها و برچسب‌ها. عناصر گرد کامل `.btn`، `.chip`، `.badge`) از `border-radius: 100px` استفاده می‌کنند.

سایه با **offset منفی**. `-28px` در پارامتر چهارم یعنی سایه کوچک‌تر از خود عنصر است، پس فقط زیرش دیده می‌شود نه دورش. این «سایه شناور» ظریف‌تر از سایهٔ یکنواخت است.

یک تابع **easing واحد**. `cubic-bezier(.22,.61,.36,1)` تقریباً معادل `ease-out` است اما نرم‌تر. استفاده از یک تابع در همهٔ انیمیشن‌ها، حس یکدستی می‌سازد.

۸.۳ تایپوگرافی

دو فونت، و یک لایهٔ واسط

```
body {
  font-family: var(--font-vazirmatn), system-ui, 'Segoe UI', Tahoma, sans-serif;
  font-size: 16px;
  line-height: 1.85;
  -webkit-font-smoothing: antialiased;
}

h1, h2, h3, h4 {
  font-family: var(--font-lalezar), var(--font-vazirmatn), sans-serif;
  font-weight: 400;
  line-height: 1.28;
  margin: 0;
  letter-spacing: 0.2px;
}
```

فونت	نقش	وزن‌ها
Vazirmatn	متن بدنه، برچسب، دکمه	فونت متغیر — همهٔ وزن‌های ۳۰۰ تا ۹۰۰
Lalezar	تیترها، قیمت، عدد بزرگ	فقط ۴۰۰

Lalezar یک فونت نمایشی فارسی است که فقط یک وزن دارد — به همین دلیل `font-weight: 400` صریحاً تنظیم شده تا مرورگر فونت مصنوعی (faux bold) نسازد.

line-height: 1.85 برای متن فارسی عمده است. خط فارسی به‌خاطر زیرنویس‌ها و کشیدگی‌ها به فضای عمودی بیشتری از لاتین نیاز دارد.

چرا نام فونت‌ها در استایل‌نامه نوشته نشده (مورد ۲۵)

تا پیش از مهاجرت، `index.html` یک `<link>` به Google Fonts داشت و استایل‌نامه فونت‌ها را با اسم صدا می‌زد:

```
<!-- دیگر وجود ندارد -->
<link href="https://fonts.googleapis.com/css2?family=Lalezar&family=Vazirmatn:wght@300;400;500;700;900&display=swap" rel="stylesheet" />
```

دو مشکل داشت. یکی، یک دامنهٔ سوم در **مسیر بحرانی رندر** و مهم‌تر — که برای مخاطب این فروشگاه تعیین‌کننده است — دامنه‌ای که برای بسیاری از کاربران ایرانی کند یا بسته است؛ نتیجه‌اش صفحه‌ای بود که تا تایم‌اوت شدن درخواست با فونت جایگزین دیده می‌شد.

حالا `next/font/google` فایل فونت را موقع **build** می‌گیرد و کنار بقیه دارایی‌ها می‌گذارد. در زمان اجرا هیچ درخواستی به بیرون نمی‌رود — و دقیقاً به همین دلیل شد در `web/src/proxy.js` بند `font-src` را روی `'self'` بست.

`web/src/app/fonts.js`:

```
export const vazirmatn = Vazirmatn({
  subsets: ['arabic', 'latin'],
  variable: '--font-vazirmatn',
  display: 'swap'
});

export const lalezar = Lalezar({
  subsets: ['arabic', 'latin'],
  weight: '400',
  variable: '--font-lalezar',
  display: 'swap'
});
```

همان معنای قبلی را دارد: متن با فونت جایگزین دیده می‌شود تا فونت اصلی برسد — بهتر از صفحه خالی. وزیرمتن روی گوگل‌فونتس فونت **متغیر** است، پس `weight` نمی‌گیرد: همان یک فایل همه وزن‌های ۳۰۰ تا ۹۰۰ را که استایل‌نامه می‌خواهد پوشش می‌دهد. لاله‌زار **متغیر** نیست و وزنش اجباری است.

لایه واسط: متغیر CSS به جای نام

نام خانواده‌ای که `next/font` تولید می‌کند **هش‌دار و غیرقابل‌پیش‌بینی** است (چیزی شبیه `__Vazirmatn_a1b2c3`). پس هیچ‌جای `style.css` و `admin.css` رشته `'Vazirmatn'` یا `'Lalezar'` نوشته نمی‌شود؛ همه‌چیز از راه متغیر می‌آید:

```
app/fonts.js      → variable: '--font-vazirmatn'
app/layout.jsx    → <html className={` ${vazirmatn.variable} ${lalezar.variable}` >
style.css / admin.css → font-family: var(--font-vazirmatn), ...
```

کامنت بالای `style.css` این زنجیره را با هشدارش می‌گوید:

اگر روزی آن دو کلاس از `<html>` برداشته شوند، همه متن سایت به فونت جانشین می‌افتد — بی هیچ خطایی.

همین است که فهرست جانشین‌ها را مهم می‌کند: `system-ui, 'Segoe UI', Tahoma, sans-serif` انتخاب شده چون هر سه روی ویندوز فارسی را با اتصال حروف درست رندر می‌کنند. یعنی حتی در بدترین حالت، صفحه ناخواناست نه شکسته.

نکته

بهایش این است که `npm run build` به شبکه نیاز دارد: فایل فونت‌ها همان موقع گرفته می‌شوند. در محیط بی‌اینترنت، **build** شکست می‌خورد — نه اجرا. این معامله آگاهانه‌ای است: یک بار موقع ساخت، به جای هر بار بازدیدکننده.

پنل مدیریت یک خانواده سوم هم دارد که از این زنجیره بیرون است: `ui-monospace, monospace` برای شناسه‌ها و اعداد لاتین (`slug`، شماره سفارش، JSON پیش‌نمایش). آنجا عمداً فونت سیستمی است — هیچ فایلی برایش دانلود نمی‌شود.

مقیاس اندازه

مقیاس سیال است و با `clamp(min, preferred, max)` کار می‌کند:

عناصر	اندازه
h1 (تصویر اصلی)	clamp(2.5rem, 5.6vw, 4.4rem)
h2 (عنوان بخش)	clamp(1.9rem, 3.6vw, 2.7rem)
.readout-price	clamp(2rem, 4.4vw, 2.9rem)
.cat-head h3	1.55rem
.card h3	1.4rem
.price b	1.35rem
.blend-title h3	1.3rem
.scale-title	1.15rem
بدنه	1rem (۱۶px)
.lead	1.03rem
.section-desc	.95rem
.card-process	.8rem
.eyebrow	.78rem
.field-label	.78rem
.notes li	.74rem
.card-origin	.72rem
.scale-note	.72rem

فاصله حروف برای متن‌های کوچک بزرگ‌نویس‌مانند:

```
.eyebrow{ font-size:.78rem; letter-spacing:.14em; }
.card-origin{ font-size:.72rem; letter-spacing:.1em; }
.readout-weight{ font-size:.9rem; letter-spacing:.06em; }
```

هرچه متن کوچک‌تر، فاصله حروف بیشتر – قاعده کلاسیک تایپوگرافی که در فارسی هم کار می‌کند چون این متن‌ها کوتاه‌اند.

الگوی logo-text

```
.logo-text{
  display:flex; flex-direction:column;
  font-family:var(--font-lalezar),sans-serif; font-size:1.22rem; line-height:1.1;
}
.logo-text em{
  font-family:var(--font-vazirmatn),sans-serif; font-style:normal;
  font-size:.66rem; letter-spacing:.12em; color:var(--soft);
}
```

یک الگوی تکرارشونده در پروژه: عنصر اصلی با فونت نمایشی، `<em>` داخلش با فونت بدنه و اندازه خیلی کوچک‌تر. همین الگو در `.price`، `.about-facts`، `.hero-facts` و `.readout-price` تکرار می‌شود.

۸.۴ فاصله‌گذاری و چیدمان

هیچ مقیاس عددی رسمی نیست

برخلاف سیستم‌هایی مثل Tailwind که مقیاس ۴px دارند، این پروژه از مقادیر موردی استفاده می‌کند. اما الگوها منظم‌اند:

نوع	مقدار
ارتفاع بخش	<code>padding-block: clamp(58px, 7.5vw, 104px)</code>
فاصله بین کارتها	<code>gap: 22px</code>
فاصله بین دسته‌ها	<code>gap: clamp(34px, 5vw, 56px)</code>
بدنه کارت	<code>padding: 16px 20px 20px</code>
فاصله نوار ابزار	<code>gap: 16px; margin-block-end: 32px</code>
فاصله فیلدها	<code>margin-block-end: 14px</code>
گروه اهرم میکس	<code>gap: 14px</code>

اعداد ۶، ۱۰، ۱۲، ۱۴، ۱۶، ۲۰، ۲۲ بیشترین تکرار را دارند.

شبکه‌ها

```
.grid-inner{ display:grid; grid-template-columns:repeat(auto-fill,minmax(285px,1fr)); gap:22px; }
.blend-grid{ display:grid; grid-template-columns:repeat(auto-fill,minmax(330px,1fr)); gap:22px; }
.why-grid{ ... }
```

یعنی شبکه واکنش‌گرا بدون **media query**. مرورگر خودش تصمیم می‌گیرد چند ستون جا می‌شود. کارت میکس عریض‌تر است (۳۳۰px) چون اهرم‌ها فضای بیشتری می‌خواهند.

چیدمان‌های دوستونه

```
.hero-inner{ display:grid; grid-template-columns:1.15fr .85fr; align-items:center; }
.club-inner{ display:grid; grid-template-columns:1.05fr .95fr; align-items:center; }
```

نسبت‌های غیر ۵۰/۵۰ عمدی‌اند: در تصویر اصلی، متن فضای بیشتری از «ترازوی قیمت» می‌گیرد.

کانتینر

```
.wrap{ width:min(var(--wrap), 92%); margin-inline:auto; }
```

یعنی: تا ۱۳۰۴px عرض صفحه، ۹۲٪ استفاده می‌شود؛ بعد از آن ۱۲۰۰px ثابت. `margin-inline` (نه `margin-left/right`) در RTL هم درست کار می‌کند.

۸.۵ راست‌به‌چپ – مهم‌ترین جنبه فنی استایل

ویژگی‌های منطقی به‌جای فیزیکی

کل استایل‌نامه از CSS Logical Properties استفاده می‌کند:

نوشته شده	به‌جای
<code>margin-inline-end</code> / <code>margin-inline-start</code>	<code>margin-right</code> / <code>margin-left</code>
<code>padding-block-end</code>	<code>padding-bottom</code>
<code>inset-block-end</code> / <code>inset-block-start</code>	<code>bottom</code> / <code>top</code>
<code>inset-inline-end</code> / <code>inset-inline-start</code>	<code>right</code> / <code>left</code>
<code>border-block-end</code>	<code>border-bottom</code>
<code>inline-size</code>	<code>width</code> (در انیمیشن)
<code>margin-inline: auto</code>	<code>margin: 0 auto</code>

نمونه‌ها از کد واقعی:

```
.nav{ margin-inline-start:auto; }

.skip-link{
  position:absolute; inset-block-start:-100px; inset-inline-start:16px;
}

.cat-head h3{ padding-inline-end:16px; border-inline-end:2px solid var(--line); }

.card-chip{ inset-block-start:12px; inset-inline-start:12px; }
.card-media .badge{ inset-block-start:12px; inset-inline-end:12px; }
```

چرا این مهم است؟ با `dir="rtl"` روی `<html>`، مرورگر خودکار `inline-start` را به «راست» و `inline-end` را به «چپ» ترجمه می‌کند. یعنی یک استایل‌نامه برای هر دو جهت کار می‌کند و نیازی به فایل جداگانه RTL یا ابزار آینه‌کاری (RTLCSS) نیست. مثال عملی: `.card-chip` (برچسب دسته روی تصویر) در RTL بالا-راست ظاهر می‌شود و `.badge` بالا-چپ – بدون یک خط کد اضافه.

جزیره‌های LTR

بعضی محتواها باید همیشه چپ‌به‌راست بمانند. این‌ها صریحاً علامت خورده‌اند:

```

{*/ شماره سفارش */}
<b dir="ltr">{order.code}</b>

{*/ شماره تلفن */}
<span dir="ltr">{order.customer.phone}</span>

{*/ شناسه کالا در پل / *}
<small dir="ltr">{item.slug}</small>

{*/ فیلدهای ورودی لاتین */}
<input className="input" dir="ltr" value={form.price} />
<input className="input" dir="ltr" autoComplete="username" />

```

و در جدول باشگاه، یک کلاس اختصاصی:

```
<td dir="ltr" className="cell-ltr">{m.phone}</td>
```

چرا لازم است؟ الگوریتم دوجته (bidi) یونیکد اعداد را در متن RTL درست نمایش می‌دهد، اما رشته‌ای مثل **M3K2-8412** را ممکن است به هم بریزد چون خط تیره در جهت خنثی است.

ورودی‌های عددی

```
<input className="input" dir="ltr" inputMode="numeric" value={c.percent} />
```

روی موبایل کیبورد عددی باز می‌کند، و `dir="ltr"` باعث می‌شود مکان‌نما درست حرکت کند. `inputMode="numeric"`

۸.۶ قرارداد نام‌گذاری کلاس‌ها

سیستم نه BEM است، نه utility — یک قرارداد میانی و ساده:

الگوی بلوک-عنصر با خط تیره

.card	بلوک
.card-media	عنصر
.card-body	
.card-top	
.card-origin	
.card-process	
.card-foot	
.card-chip	
.card-grind	
.card-more	

بدون `--` یا `--` مثل BEM؛ فقط خط تیره ساده.

.is-active چپ فیلتر، دکمه وزن، ردیف گزارش
 .is-in کارتی که با اسکرول وارد شده
 .is-open کشو، منو، مودال
 .is-off کالای پنهان، مجموع درصد نادرست
 .is-on نمایان toast، سوییچ روشن
 .is-locked ردیف میکس قفل
 .is-current دکمه وضعیت فعلی سفارش
 .is-empty ردیف بازه بدون سفارش

این یکی از قواعد BEM است که حفظ شده و بسیار خواناست: هر کلاسی که با **is-** شروع شود، یک **حالت موقت** است نه ساختار.

گونه‌ها با کلاس دوم

```
const CARD_CLASS = { coffee: 'card', gear: 'card card-gear', powder: 'card card-powder' };
```

```
.card-gear .card-art{ height:158px; }  
.card-gear:hover{ border-color:var(--brass); }  
.card-powder:hover{ border-color:var(--cherry); }
```

کارت ابزار و پودر همان کارت قهوه هستند با چند تفاوت. کامنت CSS این را می‌گوید:

```
/* — کارت ابزار دم‌آوری — */  
/* همان کارت قهوه است؛ فقط سنجه رست جای خود را به «سطح مهارت»  
و دکمه‌های وزن جای خود را به «سازگار با» می‌دهد. */
```

همین الگو برای دکمه‌ها: **.btn** + **.btn-primary** / **.btn-ghost** / **.btn-block**، و چپ‌ها: **.chip** + **.chip-sm**.

تفکیک با فضای نام

پنل مدیریت پیشوند **admin-** دارد:

```
.admin-bar    .admin-nav    .admin-link    .admin-page    .admin-head  
.admin-sub    .admin-table .admin-row    .admin-cell    .admin-thumb  
.admin-btn    .admin-ghost .admin-actions .admin-notice
```

پس هیچ‌وقت با کلاس‌های فروشگاه تداخل نمی‌کند، حتی اگر هر دو CSS در یک باندل باشند.

۸.۷ اجزای رابط کاربری

```
.btn{
  display:inline-flex; align-items:center; justify-content:center; gap:8px;
  padding:.72em 1.5em; border-radius:100px; border:1.5px solid transparent;
  font-weight:700; font-size:.95rem; cursor:pointer;
  transition:transform .18s var(--ease), background-color .18s, color .18s,
    border-color .18s, box-shadow .18s;
}
.btn:active{ transform:translateY(1px); }
.btn-primary{ background:var(--cherry); color:#FFF6F2; box-shadow:0 10px 22px -14px rgba(178,58,43,.9); }
.btn-primary:hover{ background:var(--cherry-dark); }
.btn-ghost{ border-color:currentColor; color:var(--espresso); background:transparent; }
.btn-ghost:hover{ background:var(--espresso); color:var(--paper); border-color:var(--espresso); }
```

سه جزئیات:

- **padding با em** یعنی دکمه با اندازه فونتش مقیاس می‌گیرد.
- **border: 1.5px solid transparent** روی حالت پایه — تا وقتی **.btn-ghost** حاشیه می‌گیرد، اندازه دکمه تغییر نکند.
- **transform: translateY(1px)** روی **:active** — بازخورد لمسی «فشردن شدن».

چیپ فیلتر

```
.chip{
  background:transparent; border:1.5px solid var(--line); color:var(--soft);
  border-radius:100px; padding:.42em 1.05em; cursor:pointer; font-size:.88rem;
  display:inline-flex; align-items:center; gap:7px;
}
.chip b{ font-size:.72rem; font-weight:700; background:var(--paper-2); color:var(--soft);
  border-radius:100px; padding:0 .5em; }
.chip.is-active{ background:var(--espresso); border-color:var(--espresso); color:var(--paper); }
.chip.is-active b{ background:var(--brass); color:var(--espresso); }
```

نکته خوب: عدد داخل چیپ (****) در حالت فعال به برنجی تغییر می‌کند — پس هم چیپ و هم شمارنده‌اش با هم حالت عوض می‌کنند.

سنجه پنج‌تایی

```
.roast-bar{ display:flex; gap:4px; margin-block-start:6px; }
.roast-bar i{ height:7px; flex:1; border-radius:100px; background:var(--paper-2); }
.roast-bar i.on{ background:linear-gradient(90deg,var(--latte),var(--espresso)); }

.level-bar i.on{ background:linear-gradient(90deg,var(--sage),var(--sage-deep)); }
```

یک کامپوننت، سه معنی:

نوع کالا	برچسب	رنگ
قهوه	درجه رست	قهوه‌ای (لاته → اسپرسو)
ابزار	سطح مهارت لازم	سبز (.level-bar)
پودر	شدت طعم	قهوه‌ای (.taste-bar)

```
<div className={`roast-bar ${kind.meterClass}`.trim()}
  role="img" aria-label={` ${kind.meterLabel} ${toFa(item.meter)} از ۵`} >
  {[1,2,3,4,5].map((i) => <i key={i} className={i <= item.meter ? 'on' : ''} />)}
</div>
```

این نوار بصری را برای screen reader معنادار می‌کند. `aria-label` + `role="img"`

اسلایدر سفارشی

```
.range{
  -webkit-appearance:none; appearance:none;
  width:100%; height:6px; border-radius:100px;
  background:linear-gradient(90deg,var(--crema),var(--espresso));
  cursor:pointer;
}
.range::-webkit-slider-thumb{
  -webkit-appearance:none; width:26px; height:26px; border-radius:50%;
  background:var(--brass); border:3px solid var(--paper-3); cursor:grab;
  box-shadow:0 4px 10px rgba(0,0,0,.3);
}
.range::-moz-range-thumb{
  width:22px; height:22px; border-radius:50%;
  background:var(--brass); border:3px solid var(--paper-3); cursor:grab;
}
```

هر دو موتور پوشش داده شده. گرادیان پس‌زمینه از کِرِما به اسپرسو، استعارهٔ درجهٔ رست را تکرار می‌کند.

«ترازوی قیمت» – امضای بصری صفحه

```
.scale{
  background:linear-gradient(170deg,#FCF7EC,#E7DAC2);
  border-radius:22px; padding:24px;
  box-shadow:0 34px 66px -30px rgba(0,0,0,.8);
  border:1px solid rgba(255,255,255,.7);
}
.scale-dot{
  width:9px; height:9px; border-radius:50%; background:var(--sage-deep);
  box-shadow:0 0 0 4px rgba(110,127,87,.2); animation:pulse 2.6s infinite;
}
@keyframes pulse{ 50%{ box-shadow:0 0 0 8px rgba(110,127,87,0); } }

.readout{
  background:var(--espresso); color:var(--paper);
  border-radius:var(--radius); padding:18px 20px; text-align:center;
  box-shadow:inset 0 2px 14px rgba(0,0,0,.45);
}
.readout-price{ font-family:var(--font-lalezar),sans-serif; font-size:clamp(2rem,4.4vw,2.9rem);
color:var(--brass); }
```

این تنها جایی است که `border-radius: 22px` استفاده می‌شود (نه توکن) – عمدی، چون این عنصر باید متمایز باشد.

روی صفحهٔ نمایش عدد، حس صفحهٔ فرورفتهٔ ترازوی دیجیتال می‌سازد. `box-shadow: inset`

با حلقهٔ سایه، چراغ «روشن بودن» ترازو را شبیه‌سازی می‌کند. `@keyframes pulse`

۸.۸ جزئیات بصری خاص

بافت کاغذ روی کل صفحه

```
body::after{
  content:""; position:fixed; inset:0; z-index:3; pointer-events:none; opacity:.05;
  background-image:url("data:image/svg+xml;utf8,<svg xmlns='http://www.w3.org/2000/svg' width='160'
  height='160'><filter id='n'><feTurbulence type='fractalNoise' baseFrequency='.85' numOctaves='3'/>
  </filter><rect width='160' height='160' filter='url(%23n)'/></svg>");
}
```

یک SVG با فیلتر `feTurbulence` (نویز فراکتال) به صورت **data-URI** روی کل صفحه با شفافیت ۵٪ نشسته است.

- `position: fixed; inset: 0` – با اسکرول حرکت نمی‌کند، پس حس «کاغذی که صفحه رویش چاپ شده» می‌دهد.
- `pointer-events: none` – کلیک‌ها از آن رد می‌شوند.
- بدون درخواست شبکه.

گلاس مورفیزم

```
.site-header{
  position:sticky; inset-block-start:0; z-index:40;
  background:rgba(246,239,227,.88);
  backdrop-filter:blur(12px);
  border-block-end:1px solid var(--line);
}

.card-chip{
  background:rgba(30,23,16,.72); color:var(--paper);
  backdrop-filter:blur(4px);
}
```

هدر نیمه‌شفاف با تارای پس‌زمینه، و برچسب دسته روی تصویر کارت هم همین تکنیک را با شدت کمتر دارد.

منحنی رست در تصویر اصلی

```
<svg className="roast-curve" viewBox="0 0 800 260" preserveAspectRatio="none" aria-hidden="true">
  <path d="M0,240 C140,238 210,150 300,110 S520,62 800,34" fill="none" stroke="currentColor"
  strokeWidth="2" />
  <path d="M0,255 C140,252 210,180 300,145 S520,110 800,86" fill="none" stroke="currentColor"
  strokeWidth="1" strokeDasharray="4 7" opacity=".5" />
</svg>
```

```
.roast-curve{
  position:absolute; inset-block-end:0; inset-inline:0; width:100%; height:55%;
  color:var(--brass); opacity:.3; pointer-events:none;
}

.roast-curve #curveLine{ stroke-dasharray:1400; stroke-dashoffset:1400; animation:draw 2.4s var(--ease)
.3s forwards; }
@keyframes draw{ to{ stroke-dashoffset:0; } }
```

این یک نمودار منحنی رست واقعی است (دما بر حسب زمان) که به عنوان عنصر تزئینی پس زمینه به کار رفته — جزئیاتی که فقط کسی که با قهوه آشناست تشخیص می دهد.

یعنی منحنی با عرض صفحه کشیده می شود. `preserveAspectRatio="none"`

نکته

انیمیشن `draw` روی `#curveLine` تعریف شده، اما در نسخه React هیچ عنصری این `id` را ندارد. یعنی این قاعده CSS بی اثر است — بازمانده نسخه ایستای اولیه.

انیمیشن ورود کارت ها

```
.card{
  opacity:0; transform:translateY(16px);
  transition:opacity .5s var(--ease), transform .5s var(--ease),
    border-color .2s, box-shadow .3s;
}
.card.is-in{ opacity:1; transform:none; }
.card:hover{ border-color:var(--sage-deep); box-shadow:var(--shadow-lift); transform:translateY(-3px); }
```

کارت ها با شفافیت صفر و ۱۶px پایین تر شروع می کنند. هوک `useReveal` کلاس `.is-in` را اضافه می کند.

رنگ حاشیه `hover` با نوع کالا فرق می کند: قهوه → `--sage-deep` · ابزار → `--brass` · پودر → `--cherry`

طرح رنگ دانه در راهنمای رست

```
const GUIDE_CARDS = [
  { filter: 'light', level: '1', fill: '#D3A468', stroke: '#8E5C2A', title: 'روشن', ... },
  { filter: 'medium', level: '3', fill: '#9C6B3C', stroke: '#C79763', title: 'متوسط', ... },
  { filter: 'dark', level: '5', fill: '#3B2213', stroke: '#84543A', title: 'تیره', ... },
  { filter: 'decaf', level: 'd', fill: '#7E8A63', stroke: '#E4EBD2', title: 'بدون کافئین', ... }
];
```

این رنگ ها با `ROAST_TONE` در `art.js` هماهنگ اند: `#D3A468` دقیقاً همان رنگ دانه `meter: 1` است. یعنی کارت راهنما و کارت محصول یک زبان بصری دارند.

۸.۹ واکنش گرایی

نقاط شکست فروشگاه

```

@media (max-width:1000px){
  .gallery{ grid-auto-rows:170px; }
}

@media (max-width:960px){
  .hero-inner{ grid-template-columns:1fr; }
  .hero-photo{ object-position:60% center; }
  .bulk-inner{ grid-template-columns:1fr; }
  .story-inner{ grid-template-columns:1fr; }
  .footer-inner{ grid-template-columns:1fr 1fr; }
  .toolbar{ align-items:flex-start; }
  .club-inner{ grid-template-columns:1fr; }
  .about-top{ grid-template-columns:1fr; }
}

@media (max-width:760px){
  .btn-burger{ display:block; }
  .nav{
    position:absolute; inset-block-start:100%; inset-inline:0;
    flex-direction:column; gap:0; background:var(--paper);
    max-height:0; overflow:hidden; transition:max-height .3s var(--ease);
  }
  .nav.is-open{ max-height:520px; overflow-y:auto; }
  .site-header{ position:relative; }
  .btn-cart span{ display:none; }
  .blend-grid{ grid-template-columns:1fr; }
  .sheet-head{ flex-direction:column; }
  .taste-chip{ flex:1 1 44%; }
  ...
}

```

سه نقطه شکست، هرکدام با هدف مشخص:

نقطه	چه اتفاقی می افتد
1000px	ارتفاع گالری کم می شود
960px	همهٔ چیدمان های دوستونه تک ستونه می شوند
760px	منوی برگر، شبکه های تک ستونه، پنهان کردن متن های ثانویه

دو جزئیات هوشمندانه در 760px :

(۱) `.site-header{ position:relative; }` — هدر از چسبان بودن درمی آید. دلیل: منوی موبایل با `position: absolute` نسبت به هدر باز می شود و اگر هدر چسبان بماند، منو روی محتوا گیر می کند.

(۲) `.btn-cart span{ display:none; }` — کلمهٔ «سبد» حذف می شود اما آیکون و عدد می مانند. صرفه جویی در فضا بدون از دست دادن اطلاعات.

منوی موبایل با `max-height`

```

.nav{ max-height:0; overflow:hidden; transition:max-height .3s var(--ease); }
.nav.is-open{ max-height:520px; overflow-y:auto; }

```

انیمیشن روی `max-height` (نه `height: auto` که قابل انیمیشن نیست). مقدار ۵۲۰px از ارتفاع واقعی ده لینک بیشتر است تا همیشه جا شود.

بسیاری از چیدمان‌ها اصلاً **media query** لازم ندارند:

```
.grid-inner{ grid-template-columns:repeat(auto-fill,minmax(285px,1fr)); }
```

از دسکتاپ چهارستونه تا موبایل یک‌ستونه، همه با یک خط.

کاهش حرکت

```
@media (prefers-reduced-motion:reduce){
  *{
    animation-duration:.01ms !important;
    animation-iteration-count:1 !important;
    transition-duration:.01ms !important;
  }
  html{ scroll-behavior:auto; }
  .card{ opacity:1; transform:none; }
}
```

سه لایه: همهٔ انیمیشن‌ها تقریباً آنی، پیمایش نرم خاموش، و کارت‌ها فوراً نمایان.

خط آخر حیاتی است: بدون آن، کارت‌ها با `opacity: 0` می‌مانند چون `transition` حذف شده اما حالت اولیه نه.

و همین بررسی در جاوااسکریپت هم تکرار شده (`useReveal.js`):

```
if (window.matchMedia('(prefers-reduced-motion: reduce)').matches) {
  cards.forEach((c) => c.classList.add('is-in'));
  return;
}
```

۸.۱۰ استایل چاپ رسید

این تنها جای پروژه است که یک **رسانهٔ دوم** طراحی می‌شود. همه‌چیزش در `admin.css` است، بعد از استایل رسید و پیش از پوستهٔ پنل، و میزباننش `web/src/components/PrintSheet.jsx`.

هدفش در کامنت بالای بلوک نوشته شده:

هدف: یک برگ A4 برای سفارش‌های معمولی، و برای سفارش بلند، شکستنِ تمیز — نه سرصفحهٔ تنها، نه ردیفی که وسطش دو نیم شود.

```
.print-sheet {
  display: none;
}

@media print {
  body.has-print-sheet > *:not(.print-sheet) {
    display: none !important;
  }

  .print-sheet {
    display: block !important;
    font-size: 10pt;
    line-height: 1.5;
    orphans: 3;
    widows: 3;
  }
}
```

بیرون از چاپ، میزبان پنهان است. داخل چاپ، هر فرزند دیگر `body` برداشته می‌شود و فقط همان می‌ماند.

سه چیز این سادگی را می‌سازند و هیچ‌کدام CSS نیستند:

(۱) رسید فرزند مستقیم `body` است. `PrintSheet` با `createPortal` یک بار دیگر رندرش می‌کند، بیرون از درخت صفحه. بدون این، قاعده باید نیاکان رسید را پنهان می‌کرد — و هر نیایی که پنهان شود خود رسید را هم می‌برد.

(۲) کلاس `has-print-sheet` روی `body`. بدون این شرط، قاعده روی هر صفحه‌ای برقرار بود و `Ctrl+P` روی خود فروشگاه یک برگ سفید می‌داد. کلاس را `PrintSheet` موقع mount می‌گذارد و موقع unmount برمی‌دارد.

(۳) `widows: 3 / orphans` — سه سطر تنها در بالا یا پایین صفحه نماند.

چرا `display: none` و نه `visibility` — و ۳۳ صفحه‌ای که پشتش بود

نسخه پیشین دقیقاً برعکس بود:

```
/* نسخه قدیمی — دیگر در پروژه نیست */
@media print{
  body *{ visibility:hidden !important; }
  .drawer, .drawer *{ visibility:visible !important; }
  .drawer{ position:absolute !important; transform:none !important; ... }
}
```

منطقش موجه به نظر می‌رسید: کشوی سبد چند لایه داخل صفحه است، پس به جای پنهان کردن فرزندان `body`، همه چیز نامرئی می‌شد و بعد فقط کشو برمی‌گشت — این‌طور جایگاه لایه‌های والد به هم نمی‌ریخت و `position: absolute` کشو مرجع درستی داشت.

ولی `visibility` عنصر را از چیدمان حذف نمی‌کند. فقط رنگش را برمی‌دارد. کل فروشگاه — هدر، سه فهرست کالا، ده‌ها کارت و SVG‌هایشان، متن‌های «درباره ما»، فوتر — همچنان ارتفاع واقعی خودشان را داشتند و مرورگر همان ارتفاع را صفحه‌بندی می‌کرد.

نتیجه اندازه‌گیری شد: ۳۳ صفحه، که ۳۱ تای آخرش کاغذ سفید خالی بود.

همان چیزی است که واقعاً لازم بود — و به محض اینکه رسید فرزند مستقیم `body` شد، هیچ مانعی برای استفاده‌اش نماند. ارتفاع سند حالا دقیقاً ارتفاع خود رسید است.

فرق‌های سبک چاپ با سبک نمایشگر

نمایشگر	کاغذ
رنگ	پالت کامل (--espresso ، --cherry ، --) ، بی‌استثنا <code>#fff</code> روی <code>#000</code>
زمینه و سایه	بافت کاغذ، <code>box-shadow</code> ، گلاس‌مورفیزم <code>filter:</code> ، <code>box-shadow: none</code> ، <code>transparent</code> <code>none</code>
واحد اندازه	<code>rem</code> و <code>vw</code> — تنها واحدی که چاپگر می‌فهمد <code>pt</code>
جداکننده	<code>border-radius</code> و رنگ ملایم خط‌های <code>dotted</code> / <code>solid</code> سیاه
قاب	<code>.receipt-sheet</code> کادر و padding دارد کادر و padding هر دو صفر
جهت	RTL از <code>dir</code> روی <code><html></code> همان — دست نمی‌خورد

آن سطر آخر مهم است: سبک چاپ هیچ کاری با جهت نمی‌کند. `dir="rtl"` روی `<html>` است و همه فاصله‌ها منطقی‌اند (`border-block-end` ، `margin-inline-start`) ، پس راست‌به‌چپ خودبه‌خود به کاغذ می‌رسد.

```
.print-sheet,
.print-sheet * {
  background: transparent !important;
  color: #000 !important;
  box-shadow: none !important;
  text-shadow: none !important;
  filter: none !important;
}
```

* اینجا موجه است: هیچ رنگی نباید از قلم بیفتد. رنگ‌های برزد روی کاغذ یا جوهر هدر می‌دهند یا خاکستری ناخوانا می‌شوند.

کاغذ، و چه چیزی روی آن نمی‌آید

```
@page {
  size: A4;
  margin: 12mm;
}
```

۲۷۳×۱۸۶ میلی‌متر فضای مفید — برای رسیدن چندرزیفه با ته‌مانده جا می‌ماند.

```
html, body {
  background: #fff !important;
  color: #000 !important;
  margin: 0 !important;
  padding: 0 !important;
  block-size: auto !important;
  overflow: visible !important;
}
```

لازم است چون کشوی سبد موقع باز بودن اسکرول `body` را قفل می‌کند — و چاپ معمولاً درست وقتی اتفاق می‌افتد که کشو باز است. `overflow: visible`

دو چیز عمداً حذف می‌شوند، چون فقط روی نمایشگر معنی دارند:

```
.print-sheet .order-done-mark,
.print-sheet .receipt-foot {
  display: none !important;
}
```

تیک سبز «ثبت شد» یک تزئین است، و `.receipt-foot` نوشته «این رسید را می‌توانید چاپ کنید یا از آن عکس بگیرید» – جمله‌ای که روی خود کاغذ بی‌معنی است. لینک‌ها هم زیرخطشان را از دست می‌دهند (`.print-sheet a { text-decoration: none }`). نشانی‌شان به درد نمی‌خورد، چون رسید شماره سفارش را دارد.

شکستن صفحه – چهار چیز که نباید دو نیم شوند

مهم‌ترین بخش این سبک، و چیزی که سبک قبلی اصلاً نداشت.

```
/* سرصفحه هیچ‌وقت آخرین چیز یک صفحه نباشد */
.print-sheet .receipt-h {
  break-after: avoid;
  page-break-after: avoid;
}

/* یک ردیف سفارش – با آسیاب و ترکیب میکسش – یک واحد است */
.print-sheet .receipt-lines > li,
.print-sheet .receipt-mix,
.print-sheet .receipt-block {
  break-inside: avoid;
  page-break-inside: avoid;
}

/* جمع‌ها یک تکه‌اند: «مبلغ قابل پرداخت» نباید تنها سر صفحه بعد بیفتد */
.print-sheet .totals { break-inside: avoid; page-break-inside: avoid; }

/* راه تماس فروشگاه – همیشه با هم */
.print-sheet .receipt-contact { break-inside: avoid; page-break-inside: avoid; }
```

هر قاعده دو بار نوشته شده: `break-inside` استاندارد امروز است و `page-break-inside` نام قدیمی‌اش. موتورهای چاپ عقب‌تر از موتورهای نمایش‌اند و این یکی از محدود جابجایی است که تکرار ارزش دارد.

مقیاس تایپوگرافی روی کاغذ

عنصر	اندازه	چرا
بدنه برگه	10pt	چگالی متعارف سند اداری
عنوان بالا (<code>.order-done h3</code>)	13pt	تنها تیتیر برگه
شماره سفارش	12pt	چیزی که مشتری پای تلفن می‌خواند
«مبلغ قابل پرداخت»	12pt	هموزن شماره سفارش، عمداً
سرسفحه‌ها (<code>.receipt-h</code>)	11pt	یک پله بالاتر از متن
فراداده، میکس، تماس	9pt	خوانا، ولی در حاشیه

دو چیز هم‌اندازه‌اند و این تصادفی نیست: شماره سفارش و مبلغ قابل پرداخت – دو عددی که هر کسی روی یک رسید دنبالشان می‌گردد.

می‌شوند و در یک خط کنار هم می‌نشینند، جایی که روی `display: inline` هم روی کاغذ `.receipt-date` و `.order-code` نمایشگر دو بلوک جدا بودند. یک سطر کمتر، و سرصفحهٔ برگه جمع‌وجورتر.

۸.۱۱ دسترس‌پذیری در سطح CSS

حلقهٔ فوکوس

```
:focus-visible{
  outline:3px solid var(--cherry);
  outline-offset:3px;
  border-radius:4px;
}
```

`:focus-visible` (نه `:focus`) یعنی حلقه فقط برای کاربر صفحه‌کلید ظاهر می‌شود، نه هنگام کلیک با موس. ضخامت ۳px و `outline-offset` آن را کاملاً قابل تشخیص می‌کند.

پیوند پرش

```
.skip-link{
  position:absolute; inset-block-start:-100px; inset-inline-start:16px; z-index:100;
  background:var(--espresso); color:var(--paper); padding:10px 16px;
  border-radius:0 0 10px 10px;
}
.skip-link:focus{ inset-block-start:0; }
```

بالای صفحه پنهان است و با اولین Tab ظاهر می‌شود: «رفتن به فهرست قهوه‌ها».

احترام به `hidden`

```
/* صفت hidden باید همیشه برنده باشد؛ وگرنه display دلخواه کلاس‌ها آن را بی‌اثر می‌کند */
[hidden]{ display:none !important; }
```

مشکل کلاسیک: `<div hidden class="drawer">` که `.drawer{ display: flex }` دارد، همچنان دیده می‌شود چون کلاس بر صفت `hidden` غلبه می‌کند. این قاعده آن را می‌بندد — و در `CartDrawer` واقعاً استفاده می‌شود:

```
<aside className={`drawer ${open ? 'is-open' : ''}`} hidden={!open}>
```

۸.۱۲ استایل پنل مدیریت

همان توکن‌ها را به ارث می‌برد اما چیدمان کاری‌تری دارد: `admin.css`

```
.admin-bar{
  position:sticky; inset-block-start:0; z-index:50;
  background:var(--espresso); color:var(--paper);
  border-block-end:3px solid var(--brass);
}
```


نوار تیره با مرز برنجی ۳px – تمایز بصری فوری بین پنل و فروشگاه.

جدول کالاها یک شبکه است، نه `<table>`:

```
<div className="admin-table" role="table">
  <div className="admin-row admin-row-head" role="row">
```

با `role` های ARIA برای حفظ معنا. دلیل انتخاب grid به جای table، انعطاف چیدمان در نقاط شکست است.

نقاط شکست پنل: `1080px` , `900px` , `760px` – متفاوت از فروشگاه، چون محتوای متراکم‌تری دارد.

۹. تصمیمات معماری: «چرا این روش؟»

هر تصمیم در این فصل به یک قالب ثابت نوشته شده است: تصمیم، گزینه‌های دیگر، معامله‌ها، و چرا این یکی – با شواهدی از خودِ کد.

تصمیم ۱ – یک مدل `Item` برای هر سه نوع کالا

تصمیم: قهوه، ابزار و پودر همه در یک مجموعه `items` با فیلد `kind` ذخیره می‌شوند.

گزینه‌های دیگر:

- سه مدل و سه مجموعه جدا (`Powder`, `Gear`, `Coffee`)
- یک مدل پایه با discriminator های Mongoose
- جدول اصلی + جدول‌های ویژگی (الگوی EAV)

معامله‌ها:

به سود	به زیان
یک API، یک فرم مدیریت، یک کارت	فیلدهای بی‌معنی روی بعضی نوع‌ها (<code>mat</code> روی قهوه)
فیلتر و مرتب‌سازی بین نوع‌ها ساده	قلاب <code>pre('validate')</code> پیچیده
افزودن نوع چهارم = یک ورودی در <code>taxonomy.js</code>	اعتبارسنجی نمی‌تواند فقط از schema بیاید

چرا این یکی: کامنت خط ۳۴۲ `server/src/models/Item.js` دلیل را صریح می‌گوید:

یکی بودن مدل باعث می‌شود پنل مدیریت و مسیرهای API هم یکی باشند و جای کمتری برای اشتباه بماند.

و شواهد در کد این را تأیید می‌کند. یک `ItemCard` سه نوع را رندر می‌کند:

```
const CARD_CLASS = { coffee: 'card', gear: 'card card-gear', powder: 'card card-powder' };
const kind = KINDS[item.kind] || KINDS.coffee;
```

یک `CatalogSection` سه بار با `kind` متفاوت صدا زده می‌شود. یک `AdminItemForm` هر سه را ویرایش می‌کند. جدول `KINDS` در `groups.js` همه تفاوت‌ها را در یک جا نگه می‌دارد:

```
coffee: { weighed: true, weights: [100,250,500,1000], meterLabel: 'درجه رست', ... },
gear: { weighed: false, weights: [], meterLabel: 'سطح مهارت لازم', ... },
powder: { weighed: true, weights: [50,100,250,500], meterLabel: 'شدت طعم', ... }
```

اگر سه مدل جدا بود، این ۱۰۶ کالا نیازمند سه صفحه فهرست، سه فرم و سه مجموعه مسیر API می‌شدند – با سه برابر کد و سه برابر باگ.

هزینه‌ای که پرداخت شده: قلاب `pre('validate')` با هفت گروه قاعده، که در فصل ۷ دیدیم. این پیچیدگی در یک جا متمرکز است، که بهتر از پخش شدنش در سه مدل است.

تصمیم ۲ – slug به عنوان کلید پیوند، نه ObjectId

تصمیم: میکس‌ها با `components[].slug` به دانه‌ها اشاره می‌کنند و سفارش‌ها با `lines[].slug` به کالاها. هیچ `ref` و `populate`‌ای در کل پروژه نیست.

گزینه‌های دیگر:

- `components: [{ bean: { type: ObjectId, ref: 'Item' }, percent: Number }]`
- ذخیره کل سند دانه به صورت تودرتو

معامله‌ها:

به سود	به زیان
کلید خوانا در URL، فایل خروجی و SVG	بدون یکپارچگی ارجاعی
بدون <code>populate</code> – کوئری ساده‌تر	باید دستی اعتبارسنجی شود
کلید ثابت بین محیط‌ها (dev/prod)	تغییر <code>slug</code> پیوندها را می‌شکند

چرا این یکی: `slug` در این پروژه چهار نقش همزمان دارد که `ObjectId` نمی‌توانست ایفا کند:

۱. بذر تولید تصویر – `web/src/lib/art.js`:

```
const rnd = seeded(p.slug);
const uid = 'k' + p.slug.replace(/^[a-z0-9]/gi, '');
```

با `ObjectId`، طرح کالا بین محیط توسعه و تولید فرق می‌کرد.

۲. کلید خوانا در محتوا – در `default-content.js`:

```
picks: ['tarrazu', 'huila', 'pacamara']
```

مدیر می‌تواند این را در پنل تایپ کند. با `ObjectId` غیرممکن بود.

۳. کلید فایل خروجی – ستون «شناسه کالا» در CSV.

۴. پایداری در سفارش – `slug` معنی دارد حتی اگر کالا حذف شده باشد.

کامنت مدل هم نقش دوگانه‌اش را می‌گوید:

```
/* شناسه انگلیسی و یکتا – هم در آدرس‌ها استفاده می‌شود
و هم بذر تولید تصویر کارت است، پس نباید تکراری باشد. */
```

هزینه‌ای که پرداخت شده: سه محافظ دستی – `checkBlend` در `routes/items.js`، بررسی میکس در `routes/orders.js`، و پاک‌سازی سبد در `ShopContext.jsx`.

تصمیم ۳ – قیمت‌ها در سرور دوباره حساب می‌شوند

تصمیم: بدنه درخواست ثبت سفارش هیچ قیمتی ندارد. سرور کالاها را از پایگاه داده می‌خواند و همه‌چیز را از نو حساب می‌کند.

گزینه‌های دیگر:

- اعتماد به جمع کل ارسالی از مرورگر
- ارسال قیمت و بررسی تطابقش با پایگاه داده
- محاسبه کامل در کلاینت و ارسال فقط `total`

معامله‌ها:

به سود	به زیان
دستکاری قیمت غیرممکن	یک کوئری اضافه در هر سفارش
منبع حقیقت واحد	باید همان فرمول در دو جا باشد
قیمت همیشه به‌روز	اگر مدیر وسط کار قیمت را عوض کند، مبلغ نهایی فرق می‌کند

چرا این یکی: این مهم‌ترین کنترل امنیتی کل پروژه است. کامنت `routes/orders.js`:

/* کالاها را از پایگاه داده می‌خوانیم – قیمت ارسالی از مرورگر اصلاً استفاده نمی‌شود. */

بدون این، مهاجم می‌توانست با یک درخواست دستی، ماشین اسپرسوی ۹۶ میلیون تومانی را با `total: 1000` سفارش دهد.

و کامنت مدل `Order` نکته مکمل را می‌گوید:

/* قیمت‌ها هنگام ثبت از روی پایگاه داده دوباره حساب می‌شوند ... ولی همان لحظه در سفارش ذخیره می‌شوند تا تغییر قیمت بعدی، فاکتور قدیمی را عوض نکند. */

یعنی دو نگرانی جدا: امنیت (بازمحاسبه) و درستی حسابداری (snapshot).

تصمیم ۴ – یک بسته مشترک، نه دو کپی آینه‌ای

تصمیم: منطقی که مرورگر و سرور باید یکسان ببینند در یک workspace جداگانه به نام `@ghahve/shared` است: `pricing.js`، `seo.js` و `taxonomy.js`. هیچ کپی‌ای وجود ندارد.

این تصمیم یک بار عوض شد. متن اصلی این بخش «دو نسخه عمده» را توجیه می‌کرد؛ آن استدلال و شکستش پایین آمده، چون بخشی از تاریخ همین انتخاب است.

گزینه‌های دیگر:

- دو کپی که دستی همگام بمانند (آنچه بود)
- فقط سرور محاسبه کند و مرورگر با هر تغییر درخواست بزند
- فقط مرورگر محاسبه کند (ناامن – سرور به عدد ارسالی اعتماد می‌کرد)
- انتشار روی رجیستری npm

معامله‌ها:

به سود	به زیان
واگرایی از اساس ممکن نیست	یک workspace بیشتر، و <code>transpilePackages</code> در Next
قیمت آتی، بدون تأخیر شبکه	باید <code>exports</code> را دستی نگه داشت
تست یک بار، برای هر دو طرف	فایل مشترک نمی‌تواند به <code>window</code> یا مונگوس دست بزند

چرا این یکی:

(۱) **دلیل اول عوض نشده – تجربه کاربری.** وقتی مشتری اهرم میکس را می‌کشد، قیمت باید در همان فریم عوض شود. اگر هر حرکت اهرم یک درخواست شبکه بود: پنجاه تا سیصد میلی‌ثانیه تأخیر در هر تغییر، ده‌ها درخواست هنگام کشیدن یک اهرم، و نیاز به debounce که تجربه را کندتر می‌کرد. همین برای اسلایدر «ترازوی قیمت» که ۳۹ حالت دارد.

پس محاسبه باید در مرورگر هم باشد. پرسش این نبود؛ پرسش این بود که آن نسخه از کجا بیاید.

(۲) **دلیل دوم، چیزی است که کپی‌ها را زمین زد.** خطرشان این بود که کسی پله‌های تخفیف را در یکی عوض کند و عددی که مشتری می‌بیند با عددی که ثبت می‌شود فرق کند – **بی هیچ خطایی.** نه تستی قرمز می‌شد، نه لاگی می‌آمد؛ فقط فاکتور با سبد نمی‌خواند.

کامنت آن روزها این را می‌دانست و کاری از دستش برنمی‌آمد:

```
/* نسخه یکسانی از این فایل در server/src/lib/pricing.js هست
تا عددی که کاربر می‌بیند با عددی که سرور ثبت می‌کند یکی باشد. */
```

«تا ... یکی باشد» یک آرزو بود، نه یک تضمین. راه‌حل پیشنهادی آن روز یک تست برابری (`pricing-parity.test.js`) بود که دو فایل را با هم بسنجد – یعنی یک تست برای نگه داشتن چیزی که از اول نباید دو تا می‌شد.

(۳) و سرِبارِ workspace آن‌قدری نبود که تصور می‌شد. فقط سه چیز لازم داشت: بند `workspaces` در `package.json` ریشه، یک `package.json` هفت‌خطی در `shared/`، و `transpilePackages: ['@ghahve/shared']` در `next.config.mjs`. npm خودش بسته را در `node_modules` پیوند می‌کند و هر دو طرف با همان نام `import` می‌کنند:

```
import { computeTotals } from '@ghahve/shared/pricing.js'; // هر دو طرف
import { KINDS } from '@ghahve/shared/taxonomy.js'; // هر دو طرف
```

بدون مسیر نسبی، بدون symlink دستی، بدون مرحله `build`.

آنچه این تصمیم را نمی‌گیرد: جای بازمحاسبه سمت سرور را. کامنت خودِ فایل هنوز همان هشدار قدیمی را دارد و هنوز درست است:

سرور همچنان قیمت را از نو حساب می‌کند و به عدد ارسالی از مرورگر اعتماد نمی‌کند. یکی بودن فرمول جای آن بازمحاسبه را نمی‌گیرد؛ فقط تضمین می‌کند نتیجه با چیزی که کاربر دیده یکی دربیاید.

آنچه اضافه شد و در نقشه اول نبود: با رندر سمت سرور (تصمیم ۲۱)، این بسته حالا در سه محیط اجرا می‌شود، نه دو تا – و آن سومی همان چیزی است که ارزشش را بیشتر کرد: قیمتی که در HTML سرور نوشته می‌شود و قیمتی که مرورگر بعد از hydrate حساب می‌کند باید یکی باشند، وگرنه ری‌اکت ناسازگاری می‌بیند. با یک فایل، هستند. جدول کاملش در ۷.۰.

بازمانده باز: `taxonomy.js` فقط کلید دارد و برچسب فارسی‌اش در `web/src/lib/groups.js` است. این دوتایی دوگانگی نیست، تقسیم کار است – ولی همان خطر را در مقیاس کوچک‌تر دارد، و برای همین `tests/taxonomy.test.js` هر دو سمت را می‌سنجد: نه کلید بی‌برچسب، نه برچسب بی‌کلید.

تصمیم ۵ – تقویم شمسی دست‌نویس

تصمیم: ۲۸۳ خط کد در `server/src/lib/jalali.js` به‌جای نصب `moment-jalaali` یا `date-fns-jalali`.

گزینه‌های دیگر:

- `moment-jalaali` (حدود ۳۰۰ KB با moment)
- `jalaali-js` (سبک، فقط تبدیل)
- `date-fns-jalali`
- بازه‌بندی روی تقویم میلادی و فقط نمایش شمسی

معامله‌ها:

به سود	به زیان
صفر وابستگی	۲۸۳ خط کد برای نگهداری
کنترل کامل روی DST تهران	نیاز به تخصص تقویم
فقط همان چیزی که لازم است	بدون تست

چرا این یکی: دو دلیل مشخص در کد.

دلیل اول – بازه‌بندی، نه فقط نمایش. کامنت بالای فایل:

گزارش‌ها باید با تقویمی دسته‌بندی شوند که مدیر می‌بیند: «مرداد» یعنی مرداد، نه August.

کتابخانه‌های تبدیل تقویم فقط تاریخ را ترجمه می‌کنند. اما این پروژه به چیز دیگری نیاز دارد: مرز دقیق نیمه‌شب اول هر ماه شمسی به وقت تهران، به‌صورت یک لحظه UTC – تا بشود در MongoDB کوئری زد:

```
Order.find({ createdAt: { $gte: bucket.start, $lt: bucket.end } })
```

دلیل دوم – ساعت تابستانی تاریخی ایران. این بخش را هیچ کتابخانه تبدیل تقویمی حل نمی‌کند:

/ ایران از سال ۱۴۰۱ ساعت تابستانی ندارد و اختلافش با UTC همیشه ۳:۳۰ است – ولی سفارش‌های قدیمی‌تر ممکن است از دوره ساعت تابستانی (۴:۳۰) باشند. پس به‌جای عدد ثابت، اختلاف واقعی همان لحظه را از خود سیستم می‌پرسیم تا هیچ سفارشی یک روز جابه‌جا نیفتد. /

راحل، استفاده از Intl.DateTimeFormat با timeZone: 'Asia/Tehran' است که پایگاه داده IANA داخل موتور جاوااسکریپت را به کار می‌گیرد.

و «هفته از شنبه» هم نکته‌ای است که کتابخانه‌های عمومی معمولاً اشتباه می‌گیرند:

```
getUTCDay: ... ۶ شنبه. فاصله تا شنبه قبل: /*
const back = (p.weekday + 1) % 7;
```

تصمیم ۶ – تصویر تولیدی به جای عکس محصول

تصمیم: ۱,۰۲۵ خط کد در art.js که برای هر کالا یک SVG می‌سازد.

گزینه‌های دیگر:

- عکاسی از ۱۰۶ محصول
- تصویر placeholder یکسان
- تصویر از سرویس بیرونی (Unsplash API)

معامله‌ها:

به سود	به زیان
بدون هیچ فایل تصویری	۱,۰۲۵ خط کد برای نگهداری
صفر درخواست شبکه	افزودن شکل تازه یعنی کد نوشتن
هر کالای تازه بلافاصله تصویر دارد	ظاهر «تصویرسازی» نه عکس واقعی
کاملاً واکنش‌گرا و بدون افت کیفیت	حجم HTML بیشتر (SVG درون‌خطی)

چرا این یکی: برای یک پروژه نمایشی با ۱۰۶ کالا، عکاسی غیرممکن بود. اما تصمیم فراتر از «راحل موقت» رفته و به یک زبان بصری منسجم تبدیل شده:

(۱) تصویر اطلاعات واقعی حمل می‌کند. رنگ دانه از meter می‌آید:

```
const ROAST_TONE = {
  1: ['#D3A468', '#A9743A', '#EBC894'], // روشن
  5: ['#523020', '#26150B', '#84543A'] // تیره
};
const [c1, c2, crease] = ROAST_TONE[p.meter] || ROAST_TONE[3];
```

یعنی قهوه رست تیره واقعاً تیره‌تر کشیده می‌شود.

(۲) طرح‌ها با جنس تفکیک شده‌اند. ۳۶ شکل × ۱۱ جنس = ۳۹۶ ترکیب، با نوشتن ۴۷ تعریف.

(۳) قطعی بودن. seeded(slug) تضمین می‌کند طرح هر کالا همیشه یکسان باشد – بدون آن، هر refresh چیدمان را عوض می‌کرد.

(۴) راه فرار وجود دارد. مدیر می‌تواند عکس واقعی آپلود کند و آن جای طرح می‌نشیند:

```
if (item?.image) return <img className={className} src={item.image} alt={item.name} loading="lazy" />;
return <span className="art-holder" dangerouslySetInnerHTML={{ __html: svg }} />;
```

۵) اعتبارسنجی از سرور محافظت می‌کند. `taxonomy.js` فهرست شکل‌های موجود را دارد و مدل اجازه نمی‌دهد کالایی با شکل ناموجود ذخیره شود:

مدل Item با همین‌ها اعتبارسنجی می‌شود تا هیچ‌وقت کالایی ذخیره نشود که طرح تصویرش وجود ندارد.

تصمیم ۷ – محتوای سایت در پایگاه داده با schema بیرونی

تصمیم: مدل `Content` با `{key, data: Mixed}` + فایل توصیف `contentSchema.js` که فرم پنل از رویش ساخته می‌شود.

گزینه‌های دیگر:

- متن‌ها در کد (hardcode)
- فایل‌های ترجمه (i18n JSON)
- CMS خارجی (Strapi, Sanity)
- مدل Mongoose با schema دقیق برای هر بخش

معامله‌ها:

به سود	به زیان
مدیر بدون برنامه‌نویس متن را عوض می‌کند	بدون اعتبارسنجی نوع در پایگاه داده
افزودن بخش بدون مهاجرت	خطای شکل داده تا زمان اجرا معلوم نمی‌شود
فرم خودکار ساخته می‌شود	مدیر می‌تواند شکل داده را خراب کند

چرا این یکی: کامنت مدل `Content` دلیل اصلی را می‌گوید:

هر بخش یک کلید دارد و شکل داده خودش را – تا وقتی بخواهیم بخش تازه‌ای اضافه کنیم مجبور به مهاجرت پایگاه داده نشویم.

و `contentSchema.js` دلیل مکمل را:

پنل مدیریت از روی همین توصیف، فرم را می‌سازد – پس برای اضافه کردن یک فیلد تازه فقط همین‌جا و فایل `server/src/data/default-content.js` عوض می‌شوند.

شش نوع فیلد تعریف شده: `list`, `lines`, `select`, `check`, `textarea`, `text`. کامپوننت `Field` و `ListField` در `AdminContent.jsx` هرکدام را رندر می‌کنند. `list` حتی دکمه‌های بالا/پایین دارد چون «ترتیب همان‌طور که اینجاست در سایت دیده می‌شود».

محافظت در برابر خرابی: سایت هیچ‌وقت با بخش خالی بالا نمی‌آید:

```
out[key] = stored[key] !== undefined ? stored[key] : DEFAULT_CONTENT[key];
```

و هر بخش کلاینت هم دفاعی رندر می‌شود:


```
if (!c || !Array.isArray(c.reasons) || c.reasons.length === 0) return null;
```

هزینه: اگر مدیر شکل داده را خراب کند (مثلاً `reasons` را رشته کند)، بخش بی‌صدا ناپدید می‌شود بدون پیام خطا. یک اعتبارسنجی شکل در روتر (مثلاً با `Zod`) این را حل می‌کند.

تصمیم ۸ – Context API به جای کتابخانه مدیریت حالت

تصمیم: دو Provider، بدون Redux/Zustand/Jotai.

معامله‌ها:

به سود	به زیان
صفر وابستگی	بدون DevTools برای زمان سفر
کد کمتر و مستقیم‌تر	rerender غیرضروری اگر مراقب نباشیم
منحنی یادگیری صفر	بدون middleware برای لاگ

چرا این یکی: حجم حالت سراسری کوچک است. `ShopContext` فقط پنج چیز نگه می‌دارد: کالاها، محتوا، سبد، آسیاب پیش‌فرض و toast. بقیه محلی‌اند:

Home.jsx	→ (هم عوضشان می‌کنند BrewSection و GuideSection چون) سه فیلتر
CatalogSection	→ ترتیب، جست‌وجو، وزن هر کارت
BlendCard	→ ترکیب میکس هر کارت
CartDrawer	→ مرحله تسویه، فرم، رسید
AdminReports	→ دوره، بازه، وضعیت، صفحه

این تفکیک درست است: حالتی که فقط یک زیردرخت لازمش دارد، نباید سراسری باشد.

بهینه‌سازی انجام‌شده:

```
const bySlug = useMemo(() => new Map(items.map((i) => [i.slug, i])), [items]);
const lookup = useCallback((slug) => bySlug.get(slug), [bySlug]);
const totals = useMemo(() => computeTotals(resolved, lookup), [resolved, lookup]);
```

صادقانه: شیء `value` که به Provider داده می‌شود با `useMemo` پوشیده نشده. در هر رندر `ShopProvider` یک شیء تازه ساخته می‌شود و همه مصرف‌کنندگان rerender می‌شوند. با اندازه فعلی مشکلی نیست، اما با ۱۰۰۰ کالا محسوس می‌شد. این در فصل ۱۱ به‌عنوان مسیر بهبود آمده.

تصمیم ۹ – کلید مرکب برای ردیف سبد

تصمیم: شناسه هر ردیف سبد ``${slug}|${grind}|${mixSignature(mix)}`` است، نه فقط `slug`.

گزینه‌های دیگر:

- کلید فقط `slug` (یک کالا = یک ردیف)
- شناسه تصادفی برای هر ردیف (UUID)
- آرایه ردیف‌ها بدون کلید، با `index`

معامله‌ها:

به سود	به زیان
یک قهوه با دو آسیاب = دو ردیف	منطق ادغام لازم است
یک میکس با دو نسبت = دو ردیف	کلید طولانی
افزودن دوباره همان چیز = جمع شدن	باید نرمال‌سازی شود

چرا این یکی: کامنت `ShopContext.jsx` سناریوی واقعی را می‌گوید:

یک قهوه می‌تواند چند بار در سبد باشد — یک بار دانه کامل و یک بار آسیاب اسپرسو — و یک میکس با دو نسبت متفاوت هم دو ردیف جداست.

این یک نیاز واقعی است. مشتری ممکن است ۵۰۰ گرم یرگاجف برای وی ۶۰ و ۲۵۰ گرم همان یرگاجف دانه کامل برای هدیه بخواهد.

نرمال‌سازی امضا حیاتی است:

```
return [...mix]
  .filter((m) => Number(m.percent) > 0)
  .map((m) => `${m.slug}:${Math.round(Number(m.percent) || 0)}%`)
  .sort()
  .join(',');
```

`.sort()` تضمین می‌کند ترتیب دانه‌ها اهمیت نداشته باشد، و `.filter(percent > 0)` دانه‌های برداشته‌شده را حذف می‌کند.

منطق ادغام برای وقتی است که تغییر آسیاب دو ردیف را یکی می‌کند:

```
const twin = prev.findIndex((l, idx) => idx !== i && l.key === newKey);
if (twin !== -1) {
  return prev
    .map((l, idx) => idx === twin ? { ...l, grams: l.grams + line.grams, qty: l.qty + line.qty } : l)
    .filter((_, idx) => idx !== i);
}
```

بدون این، سبد دو ردیف با کلید یکسان داشت و `key` در React تکراری می‌شد.

تصمیم ۱۰ — تخفیف بر پایه وزن، نه مبلغ

تصمیم: پله‌های تخفیف با مجموع گرم سبد فعال می‌شوند و فقط روی کالاهای وزنی اعمال می‌شوند.

گزینه‌های دیگر:

- تخفیف بر پایه مبلغ کل سبد
- تخفیف بر پایه وزن هر کالا جداگانه
- کد تخفیف

به سود	به زیان
با مدل «به گرم بفروش» جور است	خرید گران ولی سبک تخفیف نمی‌گیرد
تشویق به خرید از چند خاستگاه	ابزار در تخفیف سهیم نیست
قابل توضیح در یک جمله	سبد فقط-ابزار همیشه ارسال می‌دهد

چرا این یکی: توجیه تجاری در متن خودِ سایت آمده (BulkSection, StaticSections.jsx):

تخفیف روی مجموع وزن سبد اعمال می‌شود، نه روی تک‌تک قهوه‌ها. یعنی می‌توانید ۵۰۰ گرم از سه خاستگاه بردارید و باز هم پله ۱.۵ کیلو را بگیرید.

این یک استراتژی هوشمندانه فروش است: مشتری را تشویق می‌کند به جای یک کیلو از یک قهوه، ۵۰۰ گرم از سه خاستگاه بردارد – که هم فروش بیشتر است و هم مشتری قهوه‌های بیشتری را می‌چشد.

و توجیه اقتصادی هم درست است: تخفیف حجمی برای صرفه‌جویی در رست و بسته‌بندی است، نه پاداش خرید گران. بلو مانتین ۸/۵ میلیون تومانی در ۱۰۰ گرم، همان زحمت را دارد که ۱۰۰ گرم قهوه ارزان.

تفکیک در کد صریح است:

```
const discount = Math.round((weighedBase * tier.rate) / 1000) * 1000;
```

weighedBase، نه base. و سایت هم شفاف است:

تخفیف پلکانی فقط روی کالاهای وزنی (قهوه و پودر) اعمال می‌شود.

تصمیم ۱۱ – پرفروش‌ها از سفارش‌های واقعی + دو سوپاپ دستی

تصمیم: بخش «پرفروش‌ها» با aggregate روی سفارش‌های واقعی ساخته می‌شود، اما pinnedTop و excludeTop به مدیر اجازه استثنا می‌دهند.

گزینه‌های دیگر:

- فهرست کاملاً دستی
- کاملاً خودکار بدون دخالت
- الگوریتم وزن‌دار با زمان (trending)

چرا این یکی: دو مشکل واقعی حل می‌شود.

مشکل یک – فروشگاه تازه. بدون سفارش، بخش خالی می‌ماند. راه‌حل: pinnedTop به مدیر اجازه می‌دهد چند کالا را از روز اول نشان دهد. و اگر هیچ‌کدام نبود، پیام مناسب:

'هنوز سفارشی ثبت نشده – به‌زودی پرفروش‌ها اینجا می‌آیند' emptyTop:

مشکل دو – کالای پرفروش ولی نامناسب برای ویتترین. مثلاً فیلتر کاغذی (`filters-v60`)، تگ «مصرفی» احتمالاً پرفروش‌ترین کالای فروشگاه است چون مصرفی است – اما «پرفروش‌ترین کالای ما: فیلتر کاغذی» تبلیغ خوبی نیست. `excludeTop` این را حل می‌کند.

متن پنل هم این را روشن می‌کند:

پرفروش‌ها از روی سفارش‌های واقعی ساخته می‌شوند؛ این دو کلید فقط استثناها را تعیین می‌کنند.

انتخاب معیار هم توجه دارد. کامنت `stats.js`:

واحدها متفاوت‌اند، پس «تعداد دفعات سفارش» را ملاک می‌گیریم که بین وزنی و عددی قابل مقایسه است.

تصمیم ۱۲ – پیام‌های کنسول انگلیسی، پیام‌های مرورگر فارسی

تصمیم: `db.js`، `seed.js` و پیام‌های راه‌اندازی `index.js` انگلیسی؛ همهٔ پیام‌های API و رابط کاربری فارسی.

چرا این یکی: کامنت `db.js` دقیقاً می‌گوید:

پیام‌های این فایل عمدهً انگلیسی‌اند: اینجا خروجی خط فرمان است، و پنجرهٔ `cmd` ویندوز حروف فارسی را «?» چاپ می‌کند. متن‌هایی که مشتری در سایت می‌بیند همچنان فارسی‌اند.

این یک **تصمیم مبتنی بر محیط اجرا** است، نه سلیقه. کنسول کلاسیک ویندوز (`cmd.exe`) با کدپیچ پیش‌فرض ۴۳۷ یا ۱۲۵۶، حروف فارسی را «?» نشان می‌دهد. یک پیام خطای ناخوانا بدتر از پیام خطای انگلیسی است.

حتی نمادها هم ASCII انتخاب شده‌اند:

```
console.log('OK Database connected:', mongoose.connection.name);
console.error('\nX Could not connect to MongoDB.');
```

```
console.warn('.. Database disconnected, retrying');
```

OK, X, .. به جای ✓, ✕, ... – چون یونیکد هم در کنسول قدیمی ویندوز مشکل دارد.

README هم این را برای کاربر توضیح می‌دهد:

پیام‌های خط فرمان (سرور و `seed`) **انگلیسی** هستند، چون پنجرهٔ `cmd` ویندوز حروف فارسی را «?» چاپ می‌کند.

تصمیم ۱۳ – به‌جای فهرست سیاه توکن `tokenVersion`

تصمیم: یک عدد در سند مدیر که با تغییر رمز افزایش می‌یابد.

گزینه‌های دیگر:

- مجموعهٔ `revoked_tokens` در MongoDB
- Redis با TTL
- عمر کوتاه توکن + refresh token
- هیچ ابطالی (فقط منتظر انقضا)

معامله‌ها:

به سود	به زیان
بدون جدول یا سرویس اضافه	ابطال «همه یا هیچ» – نمی‌شود یک دستگاه را خارج کرد
ابطال فوری	یک <code>findById</code> در هر درخواست محافظت‌شده
منطق ساده و قابل فهم	JWT دیگر کاملاً stateless نیست

چرا این یکی: سیستم فقط یک حساب مدیر دارد. سناریوی «کاربر می‌خواهد فقط از تلفن قدیمی‌اش خارج شود» وجود ندارد. تنها سناریوی واقعی «رمز لو رفته، همهٔ نشست‌ها را ببند» است – که دقیقاً همان کاری است که این می‌کند.

پیاده‌سازی سه خط است:

```
// امضا
jwt.sign({ sub: String(admin._id), v: admin.tokenVersion }, ...)

// بررسی
if (admin.tokenVersion !== payload.v) return res.status(401).json({ error: 'رمز عوض شده است، دوباره وارد' });

// ابطال
admin.tokenVersion += 1;
```

و پنل به کاربر توضیح می‌دهد چه اتفاقی می‌افتد:

بعد از تغییر رمز، دستگاه‌های دیگری که با رمز قبلی وارد شده‌اند از پنل بیرون می‌آیند. همین مرورگر باز می‌ماند.

آن «همین مرورگر باز می‌ماند» به‌خاطر این است که سرور بلافاصله توکن تازه برمی‌گرداند.

تصمیم ۱۴ – مسیر نسبی و پراکسی، به‌جای آدرس مطلق API

تصمیم: همهٔ فراخوانی‌های API از مرورگر با مسیر نسبی نوشته شده‌اند و پراکسی فریم‌ورک آن‌ها را به Express می‌فرستد. اول این کار را `vite.config.js` می‌کرد؛ حالا `next.config.mjs`.

گزینه‌های دیگر:

- `NEXT_PUBLIC_API_URL` در متغیر محیطی و آدرس مطلق در `api.js`
- CORS باز در توسعه
- آدرس سخت‌کدشده با شرط محیط

چرا این یکی: نتیجهٔ عملی‌اش این است که در `web/src/lib/api.js` هیچ آدرس مطلق نیست:

```
items: () => request('/api/items'),
```

و همین کد در سه حالت کار می‌کند: توسعه با پراکسی، تولید با پراکسی جلوی هر دو فرآیند، و هر استقرار دیگری که API و صفحه‌ها روی یک دامنه باشند. `/uploads` هم همین‌طور، پس تصویرهای آپلودی در توسعه دیده می‌شوند.

از دید مرورگر همه‌چیز same-origin است، پس نه preflight لازم است نه پیکربندی CORS.

چه چیزی با مورد ۲۶ عوض شد: بند `server.proxy` رفت و `rewrites` آمد — با دو مقصد اضافه:

```
async rewrites() {
  return [
    { source: '/api/:path*', destination: `${API_URL}/api/:path*` },
    { source: '/uploads/:path*', destination: `${API_URL}/uploads/:path*` },
    { source: '/sitemap.xml', destination: `${API_URL}/sitemap.xml` },
    { source: '/robots.txt', destination: `${API_URL}/robots.txt` }
  ];
}
```

دو مسیر آخر عمداً اینجا هستند: نقشه سایت از فهرست زنده کالاهای ساخته می‌شود و دلیلی برای دو نسخه شدنش نیست، پس همان Express می‌سازدش. تفاوت مهمشان با `/api` این است که در ریشه می‌نشینند — ربات‌ها جای دیگری دنبالشان نمی‌گردند — و در تولید، پراکسی واقعی هم باید همین چهار را به Express بفرستد، نه به Next.

و یک مصرف‌کننده دوم که تازه است. تا دیروز فقط مرورگر با API حرف می‌زد. حالا خود Next هم روی سرور می‌خواند، و آنجا `fetch` مبدأ ندارد: `/api/items` برایش معنایی ندارد. برای همین `lib/data.js` جدا وجود دارد و آدرس Express را از `API_URL` می‌گیرد.

فایل	کجا اجرا می‌شود	آدرس
<code>lib/api.js</code>	مرورگر	نسبی — <code>/api/...</code>
<code>lib/data.js</code>	Next روی سرور	مطلق — <code>\${API_URL}/api/...</code>

`API_URL` عمداً پیشوند `NEXT_PUBLIC_` ندارد: آدرس داخلی است و هیچ‌وقت نباید به مرورگر برسد. چون `data.js` فقط در کامپوننت سروری import می‌شود، Next اصلاً آن را در باندل مرورگر نمی‌گذارد.

تصمیم ۱۵ — seed غیرمخرب به صورت پیش‌فرض

تصمیم: `npm run seed` فقط چیزهای نبوده را اضافه می‌کند؛ برای بازسازی کامل باید `--reset` داد.

چرا این یکی: سناریوی واقعی این است که توسعه‌دهنده بعد از چند ساعت کار در پتل، دوباره `npm run seed` می‌زند تا کالای تازه‌ای که به `seed-items.js` اضافه شده بیاید. اگر seed مخرب بود، همه کارش از بین می‌رفت.

پیاده‌سازی در سه سطح:

```
// سطح ۱ - کالای موجود دست‌نخورده می‌ماند
const exists = await Item.exists({ slug: data.slug });
if (exists) { skipped++; continue; }

// سطح ۲ - فقط فیلدهای خالی پر می‌شوند
for (const field of ['story', 'taste', 'recommend']) {
  if (!item[field] && story[field]) { item[field] = story[field]; changed = true; }
}

// سطح ۳ - میکسی که مدیر تعریف کرده دست نمی‌خورد
if (item.isBlend && item.components.length && !RESET) { skipped++; continue; }
```

و مهم‌ترین بخش - کامنتی که توضیح می‌دهد چرا `featured` عمداً تنظیم نمی‌شود:

```
/* «پیشنهاد ما» و «پرفروش‌ها» را seed تعیین نمی‌کند.
این‌ها انتخاب مدیرند و فقط در فرم خود کالا (بخش «ویترین»)
روشن می‌شوند - وگرنه هر بار seed، سلیقه مدیر را بازنویسی
می‌کرد و کالاها خودبه‌خود سر از پیشنهادها درمی‌آوردند. */
```

این نشان می‌دهد به مالکیت داده فکر شده: بعضی فیلدها مال کد هستند (متن معرفی اولیه) و بعضی مال مدیر (انتخاب ویترین).

تصمیم ۱۶ - پیش‌نمایش زنده با همان کامپوننت واقعی

تصمیم: فرم کالا یک شیء موقتی می‌سازد و به همان `ItemCard` سایت می‌دهد.

گزینه‌های دیگر:

- بدون پیش‌نمایش
- کامپوننت پیش‌نمایش جداگانه
- ذخیره و باز کردن سایت در تب دیگر

چرا این یکی: کامنت `AdminItemForm.jsx`:

کالای موقتی برای پیش‌نمایش کارت... تا پیش‌نمایش دقیقاً مثل سایت رفتار کند.

با کامپوننت جداگانه، دو پیاده‌سازی باید همگام می‌ماندند و هر تغییری در کارت سایت باید در پیش‌نمایش هم تکرار می‌شد. با استفاده از همان کامپوننت، واگرایی از نظر ساختاری غیرممکن است.

شیء `preview` دقیقاً شکل یک کالای واقعی را تقلید می‌کند، حتی فیلدهای مشتق:

```
grindable: form.kind === 'coffee',
isBlend: form.kind === 'coffee' && form.isBlend,
```

که در سرور با `pre('validate')` تنظیم می‌شوند.

تصمیم ۱۷ – سبد در localStorage با پاک‌سازی هوشمند

تصمیم: سبد در localStorage می‌ماند، اما بعد از رسیدن کالاها اعتبارسنجی می‌شود.

گزینه‌های دیگر:

- سبد در حافظه (با refresh از بین برود)
- سبد در سرور (نیازمند session یا حساب کاربری)
- sessionStorage

چرا این یکی: فروشگاه حساب کاربری ندارد، پس سبد سمت سرور یعنی ساختن یک سیستم نشست فقط برای سبد. localStorage رایگان‌ترین راه است.

اما با بی‌اعتمادی کامل:

```
/* — سبد ذخیره‌شده را می‌خوانیم ولی به آن اعتماد نمی‌کنیم —
هرچه در localStorage است دست کاربر بوده و ممکن است
دستکاری یا از نسخه قدیمی‌تر سایت مانده باشد. پس هر
ردیف از نو ساخته می‌شود، نه اینکه همان‌طور پذیرفته شود. */
export function parseCart(raw) {
  let data;
  try { data = JSON.parse(raw || '[]'); } catch { return []; }
  if (!Array.isArray(data)) return [];

  return data
    .filter((l) => l && typeof l.slug === 'string')
    .map((l) => ({ key: lineKey(l.slug, grind, mix), /* ... نوع‌دهی مجدد هر فیلد ... */ }));
}

export function loadCart(storage) {
  if (!storage) return [];
  try { return parseCart(storage.getItem(CART_KEY)); } catch { return []; }
}
```

شناسه ردیف از نو ساخته می‌شود، نه از حافظه خوانده – وگرنه ردیفی با شناسه جعلی می‌توانست با ردیف دیگری قاطی شود.

و لایه دوم بعد از رسیدن کالاها:

```
const kept = prev.filter(
  (l) => bySlug.has(l.slug) && (l.mix || []).every((m) => bySlug.has(m.slug))
);
```

با کامنت:

اگر مدیر کالایی را پاک یا خاموش کند، ردیفش از سبد کاربر هم برداشته می‌شود تا موقع ثبت سفارش خطا نگیرد. دانه‌های میکس هم باید هنوز موجود باشند، وگرنه قیمت میکس قابل محاسبه نیست.

و قیمت ذخیره نمی‌شود. فقط slug, grams, qty, grind, mix. پس اگر مدیر قیمت را عوض کند، سبد کاربر خودکار به‌روز می‌شود.

دو چیز با مورد ۲۶ به این تصمیم اضافه شد و هر دو ادامه همان بی‌اعتمادی‌اند، این بار نه به داده کاربر بلکه به ترتیب اجرا:

۱) منطق از کامپوننت بیرون آمد. حالا در `web/src/lib/cartStorage.js` است و حافظه به شکل پارامتر تزریق می‌شود، نه `localStorage` سراسری — همان کاری که `reserveStock` در سرور با مدل مونگوس می‌کند. روی سرور `browserStorage()` مقدار `null` می‌دهد و همه چیز بی‌خطا سبب خالی برمی‌گرداند، و در تست بدون مرورگر اجرا می‌شود.

۲) خواندن از رندر به افکت رفت. `useState(loadCart)` — که در نسخه ویت بی‌عیب بود — با رندر سمت سرور دقیقاً همان چیزی را می‌شکست که این تصمیم می‌خواست حفظ کند: سبب مشتری. سه نگهبانی که جایش را گرفتند در ۷.۱۲ خط به خط آمده‌اند، و `tests/cart-hydration.test.jsx` نگهبانشان است.

تصمیم ۱۸ — دو فرآیند و یک مبدأ

تصمیم: در تولید دو فرآیند Node اجرا می‌شوند — Next روی ۳۰۰۰ و Express روی ۴۰۰۰ — و از دید بازدیدکننده همه چیز یک مبدأ است.

این تصمیم هم عوض شد. متن اصلی یک فرآیند را توجیه می‌کرد: همان Express هم API می‌داد و هم خروجی build فرانت را. با مورد ۲۶ آن ممکن نماند و استدلالش پایین آمده.

گزینه‌های دیگر:

- یک فرآیند (آنچه بود) — با Next ممکن نیست، چون Next خودش سرور دارد
- فرانت روی API + CDN جدا
- Docker با دو کانتینر

معامله‌ها:

به سود	به زیان
رندر سمت سرور، با همه چیز که مورد ۲۶ آورد	دو فرآیند برای نگه داشتن، نه یکی
هر لایه کار خودش را می‌کند	یک پراکسی جلوی هر دو لازم است
از دید مرورگر هنوز same-origin	یک پرش شبکه اضافه برای <code>/api</code>

چرا این یکی: ناگزیر بود، ولی هزینه‌اش کمتر از چیزی است که به نظر می‌رسد.

۱) چرا ناگزیر: Next خودش یک سرور است. صفحه‌ها را در زمان درخواست رندر می‌کند (تصمیم ۲۲)، پس چیزی به نام «خروجی ایستا که Express سروش کند» وجود ندارد. سه چیزی که در `server/src/index.js` بود همان روز حذف شد: `express.static` روی `client/dist`، بازگشت SPA با regex منفی‌نگر، و `routes/itemPages.js` که `<head>` صفحه کالا را داخل HTML تزریق می‌کرد.

۲) چرا هزینه‌اش کم است: مرورگر فقط با ۳۰۰۰ حرف می‌زند. `/api`، `/uploads`، `/sitemap.xml` و `/robots.txt` را Next به Express می‌فرستد. یعنی همان مزیت same-origin تصمیم ۱۴ سر جایش است: نه preflight، نه پیکربندی CORS برای بازدیدکننده. پرش شبکه اضافه هم روی یک ماشین ناچیز است، و برای صفحه‌ها کمتر شده، نه بیشتر: پیش‌تر مرورگر HTML می‌گرفت و بعد خودش `/api/items` را می‌خواست؛ حالا Next آن را روی سرور می‌خواند و مرورگر یک رفت‌وبرگشت کمتر دارد.

۳) مقیاس هدف عوض نشده. این یک رستری کوچک با چند ده سفارش در روز است. `concurrently` هر دو را با یک `npm start` بالا می‌آورد.

آنچه در تولید باید مراقبش بود:

- اگر دو فرآیند روی یک ماشین نیستند. `API_URL`
- پراکسی جلویی (nginx یا مانندش) باید چهار مسیر را به Express بفرستد و بقیه را به Next – `/sitemap.xml` و `/robots.txt` را فراموش نکنید، چون در ریشه‌اند نه زیر `/api`.
- تا محدودیت نرخ آدرس واقعی مشتری را ببیند. `TRUST_PROXY`
- وگرنه Express در تولید اصلاً بالا نمی‌آید. `CLIENT_ORIGIN`

اگر روزی مقیاس لازم شد، این تقسیم کار را آسان‌تر کرده نه سخت‌تر: دو فرآیند را می‌شود جدا مقیاس داد، و چون مرورگر آدرس مطلق ندارد، جابه‌جا کردنشان فقط یک تغییر در پیکربندی پراکسی است.

تصمیم ۱۹ – بدون درگاه پرداخت

تصمیم: سفارش ثبت می‌شود و فروشگاه تماس می‌گیرد.

چرا این یکی: این تصمیم در کد صریح است. رسید می‌گوید:

همکاران ما برای هماهنگی ارسال با شما تماس می‌گیرند. شماره سفارش‌تان را دم دست داشته باشید.

و هیچ فیلد `paid`، `paymentRef` یا `transactionId` در مدل `Order` نیست.

برای یک کسب‌وکار کوچک ایرانی این کاملاً رایج است: اتصال به درگاه نیازمند نماد اعتماد الکترونیکی، قرارداد با PSP و مراحل اداری است. مدل «ثبت سفارش + تماس تلفنی» راه‌اندازی را ممکن می‌کند.

اما این یک محدودیت جدی است و در فصل ۱۱ به‌عنوان اولین مسیر توسعه آمده – همراه با اینکه چه چیزهایی در مدل داده باید اضافه شود.

تصمیم ۲۰ – بدون حساب کاربری برای مشتری

تصمیم: فقط مدیر لاگین دارد. سفارش با نام و شماره ثبت می‌شود و عضویت باشگاه بدون رمز است.

چرا این یکی: کامنت `ClubMember.js` صریح است:

عضویت ساده است: نام و شماره موبایل، و ایمیل اگر خودش بخواهد. رمز عبوری در کار نیست. شماره کلید یکتاست، پس یک نفر دو بار عضو نمی‌شود.

و متن سایت هم این را به‌عنوان مزیت می‌فروشد:

اسم و شماره‌تان را بگذارید تا از رست‌های تازه ... باخبر شوید. رمز عبوری در کار نیست.

منطق UX روشن است: هر مرحله اضافه در قیف خرید، درصدی از مشتری را از دست می‌دهد. برای فروشگاه‌هایی که مشتری‌هایش ممکن است سالی چند بار خرید کنند، اجبار به ساختن حساب، مانع است.

هزینه‌ها واقعی‌اند و در فصل ۱۱ آمده‌اند: مشتری نمی‌تواند سفارش‌های قبلی‌اش را یکجا ببیند، و شماره‌ای که وارد می‌شود تأیید نشده است (بدون OTP، مورد ۳۶).

یک هزینه کم شد. «نمی‌تواند سفارش را پیگیری کند» در متن اصلی همین‌جا بود و با مورد ۳۵ رفع شد: صفحه `/track` وضعیت سفارش را با جفت «شماره سفارش + شماره موبایل» نشان می‌دهد – بدون حساب، بدون رمز. این جای حساب کاربری را نمی‌گیرد (تاریخچه سفارش‌ها همچنان نیست) ولی همان چیزی را که مشتری واقعاً می‌خواست می‌دهد. جریانش در ۶.۶ و تحلیل امنیتی‌اش در ۷.۶.

تصمیم ۲۱ – رفتن به Next.js، بعد از اینکه یک بار «نه» گفته بودیم

تصمیم: فروشگاه و پنل از ویت به Next.js با App Router منتقل شدند و صفحه‌ها روی سرور رندر می‌شوند. `client/` حذف شد و Express فقط API ماند.

این تصمیم، تصمیم پیشین را برگرداند. متن اول فصل ۲ که رد کردن Next را توجیه می‌کرد عمداً سر جایش است: استدلال رد شدن، بخشی از تاریخ همین انتخاب است.

استدلال اولیه (که رد شد): Next یک مدل مسیریابی مبتنی بر فایل، مرز `server/client component` و زنجیره `build` سنگین‌تر می‌آورد. برای پروژه‌ای که هدفش نمایش معماری یک فروشگاه است، ویت ساده‌تر و شفاف‌تر به نظر می‌رسید.

چه چیزی آن استدلال را برگرداند – سه چیز، و هیچ‌کدام سلیقه نبود:

۱) راه میانی مسئله را حل نکرد. برای اینکه کارت پیش‌نمایش تلگرام درست شود، Express تگ‌های `<head>` را داخل `index.html` تزریق می‌کرد. این کارت را درست کرد ولی متن صفحه را نه – و مسئله اصلی همان بود: گوگل باید محتوای کالا را ببیند، نه فقط عنوانش را.

۲) نگه‌داشتن آن راه میانی خودش هزینه داشت. `<head>` هر صفحه کالا دو بار ساخته می‌شد – یک بار رشته در سرور (`htmlHead.js`)، یک بار عنصر DOM در مرورگر (`head.js`) – و یک پشته سه‌حالتی و یک تست همگامی (`seo-parity.test.js`) لازم داشت تا واگرا نشوند. صد خط کد که تنها کارش جلوگیری از واگرایی بود.

۳) «prerender جواب نمی‌دهد» درست بود، ولی راه سومی هم بود. آن روز فقط دو گزینه دیده شد: `build` ایستا (که با ۱۰۶ کالای متغیر بی‌فایده است) یا هیچ. راه سوم رندر سمت سرور در هر درخواست بود: نه `build` تازه برای هر کالا، نه فهرست موقع ساخت.

معامله‌ها:

به سود	به زیان
متن صفحه در پاسخ اول – برای گوگل و برای کاربر	دو فرآیند به‌جای یک (تصمیم ۱۸)
یک رندرکننده <code><head></code> ، نه دو تا	جاوااسکریپت بیشتر: <code>gzip</code> ۲۲۵KB در برابر <code>~۱۲۹KB</code>
مسیریابی بدون کتابخانه	کشی ایستا نداریم (تصمیم ۲۲)
مودال با آدرس واقعی (تصمیم ۲۳)	مرز <code>server/client</code> یک مفهوم تازه برای نگه داشتن
تقسیم خودکار باندل بر اساس مسیر	<code>npm run build</code> به شبکه نیاز دارد (فونت‌ها)

چه چیزی حذف شد: `client/` کامل، `react-router-dom`، `web/src/lib/head.js` با صفت‌های `data-head-*`، `server/src/lib/htmlHead.js`، `server/src/routes/itemPages.js`، `server/src/lib/clientRoutes.js`، `tests/seo-parity.test.js`. جمعاً چند صد خط که همه‌شان برای جبران رندر سمت سرور نوشته شده بودند.

چه چیزی دست‌نخورده ماند: `shared/`، کل `server/src` به‌جز سه فایل بالا، همه کامپوننت‌های فروشگاه، `style.css`، `admin.css`، و منطق سبد. مهاجرت لایه رندر بود، نه بازنویسی پروژه.

تصمیم ۲۲ – سیاست امنیتی با nonce، و بهایش: force-dynamic

تصمیم: سیاست امنیتی صفحه‌ها برای هر درخواست یک nonce تازه می‌سازد، و در نتیجه هیچ صفحه‌ای از پیش ساخته یا کش نمی‌شود:

```
export const dynamic = 'force-dynamic'; // app/layout.jsx
```

مسئله: سیاست قبلی می‌گفت `script-src 'self'`. برای یک باندل ویت کافی بود، چون هیچ اسکریپت درون خطی نداشت. Next اما داده رندر سمت سرور را به شکل چند `<script>` درون خطی داخل صفحه می‌گذارد – و آن سیاست دقیقاً همان‌ها را می‌بست. نتیجه: صفحه رندر می‌شد و هیچ‌وقت زنده نمی‌شد.

گزینه‌های دیگر:

گزینه	چرا نه
'unsafe-inline' اضافه شود	عملاً کل ارزش سیاست را دور می‌ریخت – و مورد ۷ عمداً آن را نداشت
هش هر اسکریپت به جای nonce	هش‌ها با هر build عوض می‌شوند و باید در پیکربندی بمانند
رندر ایستا با nonce ثابت	nonce تکراری بین دو بازدیدکننده یعنی سیاست بی‌اثر

چرا این یکی: بین «کش ایستا» و «سیاست سخت‌گیرانه» فقط یکی را می‌شد داشت. سیاست را نگه داشتیم.

nonce برای هر درخواست تازه ساخته می‌شود؛ صفحه‌ای که موقع build ساخته شده باشد آن را ندارد، پس اسکریپت‌های بسته می‌شوند. force-dynamic همین را تضمین می‌کند.

هزینه واقعی‌اش کمتر از چیزی است که به نظر می‌رسد: تقریباً هیچ صفحه این فروشگاه واقعاً ایستا نیست. فهرست کالاها، موجودی و متن‌های سایت همه از پایگاه داده می‌آیند و مدیر هر روز عوضشان می‌کند. نسخه کش‌شده یعنی فروختن چیزی که نداریم. نسخه ویت هم هر بار همین داده‌ها را از سرور می‌گرفت؛ فرق اینجاست که حالا یک رفت‌وبرگشت انجام می‌شود نه دو تا.

آنچه واقعاً از دست رفت: revalidate، صفحه کش‌شده روی CDN، و generateStaticParams برای صفحه‌های کالا. اگر روزی ترافیک آن‌قدر شد که این‌ها لازم شوند، راهش این است که برای مسیرهای کم‌تغییر nonce را کنار بگذاریم – نه اینکه سیاست را برای کل سایت شل کنیم.

دو استثنا که عمداً باز ماندند و در proxy.js کامنت دارند:

- `style-src 'unsafe-inline'` – صفت `style` با nonce کار نمی‌کند، و نوار نمودار گزارش و متغیرهای CSS حالت تصویری میکس هر دو درون خطی‌اند. اگر روزی برداشته شود، نوار نمودارها بی‌طول رندر می‌شوند.
- `img-src data:` – تصویر تولیدی SVG کارت‌ها و بافت `style.css`.

تصمیم ۲۳ – مودال کالا پشت یک آدرس واقعی

تصمیم: پنجره «درباره این قهوه» یک مسیر واقعی است (`/coffee/:slug`)، و اینکه به شکل مودال دیده شود یا صفحه کامل، به نحوه رسیدن بستگی دارد – با مسیرهای رهگیری‌شده App Router.

مسئله: تا پیش از مورد ۲۷، مودال هیچ آدرسی نداشت. یعنی هیچ کالایی لینک قابل اشتراک نداشت، هیچ صفحه‌ای برای ایندکس شدن نبود، و دکمه back مرورگر کاری نمی‌کرد.

گزینه‌های دیگر:

گزینه	چرا نه
فقط صفحه کامل، بدون مودال	هر کلیک یک پیمایش کامل – تجربه مرور فهرست را می‌شکست
فقط مودال، با <code>history.pushState</code> دستی	همان کاری است که نسخه ویت با <code>backgroundLocation</code> می‌کرد؛ حالت دستی و شکننده
مودال با query string (<code>?item=...</code>)	<code>og:url</code> و <code>canonical</code> را کثیف می‌کرد و از دید گوگل همان صفحه اصلی است

چرا این یکی: App Router دقیقاً همین را به شکل بومی دارد. یک شکاف موازی (`app/@modal`) و سه پوشه رهگیری شده (`coffee/[slug]`) و هم‌تاها) کافی است:

- کلیک از داخل سایت → مودال روی صفحه فعلی، صفحه زیر دست نخورده.
- بازدید سرد، رفرش، یا لینک کپی‌شده → صفحه کامل با `<head>` خودش.

همان دو رفتاری که پیش‌تر با `state.backgroundLocation` دستی ساخته می‌شد، بی هیچ حالت دستی.

سه پیامد که ارزش گفتن دارند:

۱) `<head>` مودال اصلاً ساخته نمی‌شود. پیش‌تر مجبور بودیم عنوان صفحه را با باز شدن مودال عوض کنیم و با بستنش برگردانیم – همان چیزی که آن پشته سه‌حالت را لازم می‌کرد. حالا آدرس واقعاً عوض شده و مسیر کامل همان تگ‌ها را دارد، پس چیزی برای عوض کردن نمانده.

۲) بستن مودال یک قدم به عقب در تاریخچه است (`router.back()`)، پس دکمه back مرورگر و دکمه بستن دقیقاً یک کار می‌کنند.

۳) داده مودال از سرور می‌آید، نه از کانتکست. مودال در شکاف layout ریشه رندر می‌شود، یعنی بیرون از `ShopProvider`. می‌شد یک Provider دوم گذاشت، ولی آن‌وقت دو سید جدا می‌داشتیم و افزودن از یکی روی دیگری اثر نمی‌کرد. پس فقط نام دانه‌ها و برچسب دو آسیاب، به شکل داده ساده، پاس داده می‌شوند.

هزینه‌اش: یک فایل `default.jsx` که فقط `null` برمی‌گرداند و بدون آن هر بازدید سرد ۴۰۴ می‌شود – نوعی دانش فریم‌ورک که باید در سند بماند، چون از خود کد پیدا نیست.

تصمیم ۲۴ – `og:type: product` را خود صفحه رندر می‌کند

تصمیم: همه تگ‌های `<head>` از راه Metadata API می‌روند، به جز یکی: `og:type` صفحه کالا، که خود کامپوننت رندرش می‌کند.

```
{ogType ? <meta property="og:type" content={ogType} /> : null}
```

مسئله: Next برای `openGraph.type` فهرست بسته خودش را دارد (`music.*` , `profile` , `book` , `article` , `website`) و برای هر چیز دیگری موقع رندر خطا می‌دهد. خطای متادیتا یعنی کل `<head>` از دست می‌رود، نه فقط یک تگ. اولین بار همین شد: صفحه کالا بدنه درست داشت و هیچ عنوانی نداشت.

و `product` چیزی نیست که بشود از آن گذشت: همان است که فیس‌بوک و تلگرام برای کارت کالا می‌خوانند، و پیش از مهاجرت هم در `<head>` صفحه کالا بود.

گزینه	چرا نه
<code>metadata.other</code>	تگ را با <code>name</code> می‌نویسد نه <code>property</code> ، و خزندهٔ Open Graph سراغ <code>property</code> می‌رود – تگی که هست ولی خوانده نمی‌شود، از نبودش بدتر
عوض کردنش به <code>'website'</code>	کالا دیگر کالا نیست؛ عقب‌گرد واقعی نسبت به وضع قبل
کنار گذاشتن Metadata API و رندر دستی همهٔ تگ‌ها	برگشت به همان دوگانگی‌ای که مورد ۲۶ حذفش کرد

چرا این یکی: ری‌اکت ۱۹ عنصرهای `<meta>` را از هر جای درخت به `<head>` می‌برد، پس یک خط JSX کافی است و بقیهٔ تگ‌ها مسیر همیشگی‌شان را می‌روند.

مهم‌تر اینکه از `shared` می‌آید، نه دستی نوشته می‌شود:

```
export const ogTypeFallback = (state = {}) =>
  state.ogType && !NEXT_OG_TYPES.has(state.ogType) ? state.ogType : '';
```

صفحه همان `itemHeadState` را دوباره می‌سازد (تابع خالص است) و فقط می‌پرسد «چیزی هست که Next نپذیرد؟». اگر روزی Next فهرستش را باز کند، فقط `NEXT_OG_TYPES` بزرگ می‌شود و آن یک خط از صفحه برداشته می‌شود – بدون هیچ تغییر دیگری.

هزینه‌اش: یک استثنا در قاعده‌ای که وگرنه بی‌استثنا بود، و یک تست که باید بداند این یک تگ جای دیگری است (`tests/next-metadata.test.js`).

تصمیم ۲۵ – صفحهٔ کالا فهرست کامل را فقط وقتی می‌خواند که سبد لازمش دارد

تصمیم: صفحهٔ اختصاصی یک کالا فقط خود آن کالا (و اگر میکس است، دانه‌هایش) را به `ShopProvider` می‌دهد، نه صد و شش کالا. اگر سبد بازدیدکننده به کالایی اشاره کند که در این فهرست نیست، همان لحظه فهرست کامل می‌شود.

مسئله: خواندن کل فهرست برای نشان دادن یک کالا اسراف است – و چون هیچ صفحه‌ای کش نمی‌شود (تصمیم ۲۲)، این اسراف در هر بازدید تکرار می‌شد. برای همین `GET /api/items/:kind/:slug` اصلاً ساخته شد.

ولی سبد مال کل سایت است، نه مال این صفحه. مشتری می‌تواند دو قهوه در سبد داشته باشد و بعد صفحهٔ یک ابزار را باز کند. آن وقت دو چیز می‌شکست و هر دو بی‌صدا:

- شمارندهٔ سبد در هدر کمتر از واقع نشان می‌داد؛
- و بدتر: `CartDrawer` سفارش را از `resolved` می‌سازد، پس ثبت سفارش از صفحهٔ یک کالا بقیهٔ ردیف‌ها را می‌انداخت.

گزینه‌های دیگر:

گزینه	چرا نه
همیشه کل فهرست را بخوان	همان اسراف‌ای که این مسیر برای حذفش ساخته شد
سبد را از <code>resolved</code> جدا کن و بدون سند کالا بفرست	سرور به قیمت ارسالی اعتماد نمی‌کند، ولی نام و نوع کالا هم لازم است؛ و شمارندهٔ هدر باز هم غلط می‌ماند
صفحهٔ کالا اصلاً سبد نداشته باشد	صفحه‌ای که دکمهٔ افزودن دارد ولی سبد ندارد، بدترین حالت است

چرا این یکی: چون حالتِ ناقص را موقتی می‌کند، نه دائمی. پرچم `fullCatalogue={false}` می‌گوید «این فهرست کامل نیست»، و دو نگهبان از آن استفاده می‌کنند:

- افکتِ پاک‌سازی روی فهرست ناقص اصلاً اجرا نمی‌شود.
- افکتِ «کامل کردن فهرست» اگر سبد به کالایی بی‌رد اشاره کند، یک بار `reload()` می‌زند.

برای بازدیدکنندهٔ بی‌سبد – یعنی بیشترشان، و همان‌هایی که از گوگل می‌آیند – هیچ‌کدام از این‌ها اجرا نمی‌شود و صفحه با یک درخواست تمام می‌شود.

هزینه‌اش: یک حالت اضافه در `ShopContext` و سه شرط که اگر یکی‌شان پاک شود، سبد مشتری بی‌صدا خالی می‌شود. به همین دلیل هر سه در ۷.۱۲ خط‌به‌خط مستند شده‌اند و کامنت‌های خودِ فایل هم دلیلشان را نگه داشته‌اند.

تصمیم ۲۶ – توست از کانتکست بیرون آمد، به یک انبارۀ ماژولی

تصمیم: پیام کوتاه («به سبد اضافه شد»، «لینک کپی شد») دیگر حالتِ `ShopContext` نیست. در `web/src/lib/toastStore.js` زندگی می‌کند – چند متغیر ماژولی و یک `Set` از مشترک‌ها – و `Toast.jsx` با `useSyncExternalStore` به آن وصل می‌شود.

مسئله: توست هیچ‌وقت واقعاً حالتِ فروشگاه نبود؛ فقط آنجا زندگی می‌کرد چون اولین صداکننده‌اش سبد بود. مرزِ غلطی بود که تا وقتی همه‌چیز روی صفحهٔ اصلی می‌گذشت هزینه‌ای نداشت.

بعد مودال کالا رویش گیر کرد. مودال در شکافِ `@modal` رندر می‌شود، یعنی بیرون از `ShopProvider` صفحه (تصمیم ۲۳). وقتی دکمهٔ هم‌رسانی به مودال اضافه شد، به چیزی نیاز داشت که بگوید «کپی شد» – و آنجا هیچ Provider ای بالای سرش نبود.

و این تنها نشانه نبود. `Toast` تا آن موقع چند خط داخل `HomeShell` بود، یعنی فقط صفحهٔ اصلی توست داشت: صفحهٔ اختصاصی کالا و مودالش هر دو بی‌صدا بودند. مشتری چیزی به سبد اضافه می‌کرد و هیچ تأییدی نمی‌گرفت.

گزینه‌های دیگر:

گزینه	چرا نه
یک <code>ShopProvider</code> دوم دور مودال	همان دام تصمیم ۲۳: دو سبد جدا. افزودن از یکی روی دیگری اثر نمی‌کرد
بردن <code>ShopProvider</code> به <code>app/layout.jsx</code>	آن‌وقت هر صفحه‌ای – از جمله <code>/track</code> و کل پنل – کالاها را حمل می‌کرد، و <code>force-dynamic</code> یعنی این هزینه در هر بازدید تکرار می‌شد
یک کانتکست دومِ کوچک، فقط برای توست	باز هم یک Provider که باید جایی بنشیند که بالای هر دو شکاف باشد – یعنی همان layout ریشه، با یک <code>'use client'</code> روی پوستهٔ کل سایت
پاس دادن <code>onToast</code> به شکل prop	تابع از مرز سروری به مشتری رد نمی‌شود؛ و مسیرِ کارت تا مودال چند لایه است

چرا این یکی: چون توست یک چیز است در کل صفحه، نه یک چیز به‌ازای هر زیردرخت. کانتکست ابزارِ درستی برای حالتی است که دامنه دارد؛ اینجا دامنه‌ای در کار نیست.

با انبارۀ ماژولی، هر کامپوننتی – در هر شکافی از درخت – می‌تواند `toast('...')` صدا بزند، و همان یک نوارِ پایین صفحه نشانش می‌دهد. `<Toast />` یک بار در `app/layout.jsx` می‌نشیند، بالای `children` و `modal` هر دو.


```
let message = '';
const listeners = new Set();

export function toast(msg) { ... emit(); }
export function subscribeToast(listener) { ... }
export const getToast = () => message;
export const getServerToast = () => '';
```

سه چیزی که این تصمیم را ارزان نگه می‌دارد:

(۱) `useSyncExternalStore` دقیقاً برای همین ساخته شده. هوک رسمی وصل کردن ری‌اکت به حالت بیرون از ری‌اکت. هیچ چیز دستی‌ای در کار نیست.

(۲) `getServerToast` جدا است و همیشه خالی. متغیرهای ماژول روی سرور بین درخواست‌ها مشترک‌اند. در عمل `toast()` فقط از دل رویدادهای مرورگر صدا زده می‌شود، ولی این snapshot تضمین می‌کند حتی در بدترین حالت هم پیام یک بازدیدکننده داخل HTML بازدیدکننده بعدی نریند.

(۳) صداکننده‌ها دست نخوردند. یک پوشش دوطرفی در `ShopContext` ماند:

```
const toast = useCallback((msg) => showToast(msg), []);
```

ده جایی که `useShop().toast(...)` می‌نوشتند همان‌طور ماندند. مهاجرت یک فایل بود، نه ده تا.

هزینه‌اش: یک تکه حالت سراسری که ری‌اکت نمی‌بیندش، و دو خرابی ممکن که هیچ‌کدام در رابط کاربری فوراً دیده نمی‌شوند – پیامی که پاک نشود و تا ابد بماند، و پیام دومی که شمارش‌اولی را به ارث ببرد و زودتر برود. هر دو در `tests/toast-store.test.js` بسته شده‌اند.

تصمیم ۲۷ – چاپ با `display: none`، به بهای یک رندر دوم

تصمیم: رسید برای چاپ یک بار دیگر رندر می‌شود، به شکل فرزند مستقیم `body` و از راه `createPortal`. سبک چاپ بعد هر فرزند دیگر `body` را با `display: none` برمی‌دارد.

مسئله: نسخه پیشین رسید را با `visibility: hidden` روی `body *` چاپ می‌کرد و فقط کشوی سبد را برمی‌گرداند:

```
/* نسخه قدیمی */
body *{ visibility:hidden !important; }
.drawer, .drawer *{ visibility:visible !important; }
```

منطقش موجه به نظر می‌رسید – کشو چند لایه داخل درخت است و `display: none` روی نیاکانش خودش را هم می‌برد. ولی `visibility` عنصر را از چیدمان حذف نمی‌کند؛ فقط رنگش را برمی‌دارد. کل فروشگاه – هدر، سه فهرست کالا، ده‌ها کارت و SVG‌هایشان، فوتر – همچنان ارتفاع واقعی خودشان را داشتند و مرورگر همان ارتفاع را صفحه‌بندی می‌کرد.

نتیجه اندازه‌گیری شد: ۳۳ صفحه، که ۳۱ تای آخرش کاغذ سفید خالی بود.

و این خرابی از آن دسته‌ای است که در توسعه دیده نمی‌شود: پیش‌نمایش چاپ صفحه اول را درست نشان می‌دهد، و کسی که فقط نگاهش می‌کند هیچ ایرادی نمی‌بیند.

گزینه‌های دیگر:

گزینه	چرا نه
ماندن با <code>visibility</code> و بستن ارتفاع با <code>block-size: 0</code> روی نیاکان	زنجره نیاکان ثابت نیست؛ هر تغییری در چیدمان کشف دوباره می‌شکستش
<code>display: none</code> مستقیم روی نیاکان رسید	هر نیایی که پنهان شود خود رسید را هم می‌برد
یک مسیر جدا برای چاپ (<code>/track/print</code>)	یک بارگذاری کامل برای گرفتن کاغذی از چیزی که همین حالا روی صفحه است
باز کردن پنجره تازه و نوشتن HTML رسید داخلش	همان قیمت‌ها باید دوباره ساخته می‌شدند؛ یعنی همان دام «دو نسخه از یک رسید»

چرا این یکی: چون مسئله را از CSS به جایگاه در درخت منتقل می‌کند. وقتی رسید فرزند مستقیم `body` باشد، قاعده چاپ یک خط است و هیچ فرضی درباره ساختار صفحه ندارد:

```
body.has-print-sheet > *:not(.print-sheet) { display: none !important; }
```

ارتفاع سند دقیقاً ارتفاع خود رسید است، و اگر روزی چیدمان کشف یا صفحه پیگیری عوض شود این قاعده دست نمی‌خورد. و شرط `has-print-sheet` بخشی از خود تصمیم است. بدون آن، قاعده روی هر صفحه‌ای برقرار بود و `Ctrl+P` روی فروشگاه یک برگ سفید می‌داد. کلاس را `PrintSheet` موقع mount می‌گذارد و در پاک‌سازی برمی‌دارد، پس دقیقاً وقتی زنده است که رسیدی برای چاپ وجود دارد.

هزینه‌اش، و اینکه چرا کم است: همان رسید دو بار در DOM هست. سه چیز این را ارزان می‌کند:

- میزبان روی صحنه `display: none` است، پس مرورگر چیدمانش را حساب نمی‌کند.
 - `aria-hidden="true"` دارد، پس صفحه‌خوان دو بار نمی‌خواندش.
 - هر دو نسخه همان کامپوننت `Receipt` اند. اگر دو پیاده‌سازی می‌شد، روزی یکی «تخفیف وزنی» را نشان می‌داد و دیگری نه – دقیقاً همان دامی که بیرون کشیدن `Receipt` از `CartDrawer` برای بستنش بود.
- و یک احتیاط: portal به `document` نیاز دارد و روی سرور چنین چیزی نیست، پس تا mount نشدن `null` برمی‌گردد. HTML سرور و اولین رندر مرورگر یکی می‌مانند و `hydrate` شکایتی ندارد.

تصمیم ۲۸ – هم‌رسانی را نوع نشانگر تعیین می‌کند، نه بودن `navigator.share`

تصمیم: دکمه هم‌رسانی وقتی سراغ برگه سیستم می‌رود که دستگاه واقعاً لمسی باشد – یعنی `(pointer: coarse)`. همه‌جای دیگر، حتی اگر `navigator.share` وجود داشته باشد، مستقیم به کپی در کلیپ‌بورد می‌رود.

مسئله: شرط اول همان چیزی بود که هر راهنمایی می‌گوید:

```
if (typeof nav.share === 'function') { ... }
```

با این فرض که «تقریباً هیچ مرورگر دسکتاپی `share` ندارد». آن فرض دیگر درست نیست. کروم روی ویندوز `navigator.share` دارد – ولی چیزی که باز می‌کند برگه اشتراک خود ویندوز است، که معمولاً خالی است و کاربر بی‌کار می‌بنددش.

و بستن آن برگه یک `AbortError` می‌دهد، که کد به‌درستی «کاربر نظرش عوض شد» تفسیرش می‌کرد و ساکت می‌ماند. یعنی در آن مسیر هیچ‌وقت کاری به کلیپ‌بورد نمی‌رسید: کاربر دسکتاپ دکمه را می‌زد، پنبلی می‌آمد و می‌رفت، و لینک هیچ‌جا کپی نشده بود. دکمه ظاهراً کار می‌کرد و عملاً هیچ نمی‌کرد.

گزینه	چرا نه
همان <code>typeof nav.share === 'function'</code>	همان اشکال بالا – و روی پرکاربردترین مرورگر دسکتاپ
اول <code>share</code> ، و اگر لغو شد کپی کن	لغو واقعی هم کپی می‌شد؛ کاربری که پشیمان شده بی‌دلیل یک توست می‌گرفت
<code>navigator.userAgent</code> و تشخیص موبایل	همان بازی همیشگی رشته عامل کاربر: شکننده، جعل‌شدنی، و همیشه یک دستگاه تازه که در فهرست نیست
<code>navigator.maxTouchPoints > 0</code>	هر نمایشگر لمسی وصل به دسکتاپ آن را بالای صفر می‌کند
هر دو را نشان بده: دو دکمه	برای یک کار کوچک، دو دکمه در کارتی که جا هم ندارد

چرا این یکی: چون پرسش درست «مرورگر چه چیزی دارد؟» نیست، «کاربر با چه چیزی کار می‌کند؟» است.

```
const mq = win?.matchMedia?.('(pointer: coarse)');
if (typeof mq?.matches === 'boolean') return mq.matches;

return Number(nav?.maxTouchPoints) > 0;
```

`(pointer: coarse)` یعنی نشانگر اصلی دستگاه انگشت است. آنجا برگه سیستم واقعاً خوب است: تلگرام، واتساپ، پیامک، هرچه نصب است. لپ‌تاپ لمسی که ماوس هم دارد `fine` گزارش می‌دهد و درست هم همین است – آنجا کپی به کار کاربر می‌آید، نه فهرستی از اپ‌هایی که ندارد.

`maxTouchPoints` فقط تکیه‌گاه آخر است، برای مرورگر بی `matchMedia`.

این یک نمونه از یک الگوی کلی‌تر است. «مرورگر این API را دارد؟» و «این API اینجا چیز خوبی می‌دهد؟» دو پرسش متفاوت‌اند، و تشخیص ویژگی فقط به اولی جواب می‌دهد. هر جا که پاسخ دوم به زمینه بستگی داشته باشد – نه به موجود بودن – تشخیص ویژگی ابزار اشتباهی است.

هزینه‌اش: یک انحراف از قاعده جافتاده «ویژگی را تشخیص بده، نه دستگاه را»، که در نگاه اول شبیه بدعادت به نظر می‌رسد. برای همین دلیلش هم بالای `isTouchDevice` نوشته شده و هم اینجا – و چهار تست `tests/share.test.js` در بخش «دسکتاپی» که `navigator.share` دارد» تنها چیزی‌اند که جلوی برگشتن رفتار قبلی را می‌گیرند.

تصمیم ۲۹ – تصویر در زمان سرو بهینه می‌شود، نه در زمان آپلود

تصمیم: عکس آپلودی مدیر دست‌نخورده روی دیسک می‌ماند. کوچک‌سازی و تبدیل قالب در لحظه درخواست انجام می‌شود، با بهینه‌ساز خود Next (`/_next/image`)، و نتیجه روی دیسک کش می‌شود.

مسئله: عکس تا چهار مگابایت خام سرو می‌شد، در قابی که بزرگ‌ترینش ۳۸۵ پیکسل عرض دارد و کوچک‌ترینش ۷۰. مشتری موبایل عکسی می‌گرفت که هیچ‌وقت بیش از یک بیستمش را نمی‌دید.

راه‌حلی که سند ضعیف‌ها (مورد ۲۴) پیشنهاد داده بود `sharp` در خود مسیر آپلود بود: یک بار کوچک‌کش کن، همان را نگه دار.

گزینه‌های دیگر:

گزینه	چرا نه
sharp در upload.js ، نسخه اصلی دور ریخته شود	تصمیم برگشتناپذیر. قالب فردا اندازه دیگری دارد و اصل عکس دیگر نیست
sharp در آپلود، با نگه داشتن اصل و ساختن چند نسخه ثابت	همان دیسکی که می‌خواستیم صرفه‌جویی کنیم، ضربدر تعداد نسخه‌ها؛ و فهرست اندازه‌ها دستی نگه داشته می‌شود
یک سرویس بیرونی (Cloudinary و مانندش)	وابستگی بیرونی و هزینه ماهانه، برای فروشگاه‌هایی که روی یک ماشین اجرا می‌شود
هیچ – همان خام	همان چیزی که مورد ۲۴ بود

چرا این یکی: چون کلاینت دیگر ویت نیست. **Next خودش بهینه‌ساز تصویر دارد و پشتش هم همان sharp** است. با `next/image`:

- هر اندازه‌ای که واقعاً خواسته شود ساخته می‌شود، نه فهرستی که ما حدس زده‌ایم؛
- قالب از هدر **Accept** همان مرورگر می‌آید (AVIF، وگرنه WebP، وگرنه اصل)؛
- نتیجه روی دیسک کش می‌شود، پس هزینه یک‌بار است؛
- **و نسخه اصلی می‌ماند.** اگر روزی قالب کارت عوض شد، همه‌چیز از نو ساخته می‌شود بی‌آنکه کسی مجبور باشد عکس‌ها را دوباره آپلود کند.

اندازه‌گیری – صفحه فهرست با ۳۸ کالای روشن که همه‌شان عکس آپلودی داشتند (عکس آزمایشی ۲۰۱۶ در ۱۵۱۲ پیکسل، میانگین ۴۵۲ کیلوبایت):

صرفه	با next/image	خام	
۹۴.۸٪	۴۷,۴۴۰	۹۱۴,۵۷۲	بالای صفحه (۳ عکس، بدون اسکرول)
۹۶.۶٪	۶۰۳,۷۶۰	۱۷,۵۸۰,۷۶۳	کل صفحه (۵۴ قالب، ۳۸ عکس یکتا)
۹۶.۷٪	۱۶,۰۴۸ (AVIF)	۴۷۹,۹۱۹	یک عکس در قالب کارت

و نکته‌ای که ارزش گفتن دارد: بیشتر این صرفه از **اندازه** می‌آید نه از قالب. همان عکس در ۴۴۸ پیکسل و به‌شکل JPEG هم ۲۵,۳۶۴ بایت می‌شود – یعنی مرورگری که AVIF نمی‌فهمد هم ۹۴.۷٪ کمتر می‌گیرد.

دو چیزی که این تصمیم را از یک در باز جدا می‌کند:

(۱) **LocalPatterns**. بدون آن هر مسیر محلی‌ای بهینه‌شدنی است و `/_next/image` عملاً یک واکنش عمومی می‌شود. دو الگو تعریف شده‌اند – `/uploads/**` و `/img/**` – و بس.

(۲) **lib/img.js**. SVG هیچ‌وقت نباید به بهینه‌ساز برسد: `dangerouslyAllowSVG` خاموش است (مورد ۵: SVG یک سند اجرایشده‌ای است) و بهینه‌ساز به SVG پاسخ ۴۰۰ می‌دهد – یعنی **قالب خالی**، نه عکس بهینه‌نشده. همان قاعده آدرس بیرونی را هم بیرون می‌گذارد.

هزینه‌اش: فایل خام تا چهار مگابایت روی دیسک می‌ماند، و اولین درخواست هر اندازه‌کننده از بقیه است. برای این پروژه معامله‌ای است که می‌ارزد؛ اگر روزی فضای دیسک تنگ شد، فشرده‌کردن در خود `upload.js` گام بعدی است – این بار کنار بهینه‌ساز زمان سرو، نه به‌جایش.

جدول جمع‌بندی

#	تصمیم	ارزش اصلی	هزینه اصلی
۱	یک مدل <code>Item</code>	سادگی UI و API	قلاب اعتبارسنجی پیچیده
۲	<code>slug</code> به جای <code>ObjectId</code>	خوانایی و نقش چندانگانه	یکپارچگی دستی
۳	بازمحاسبهٔ قیمت در سرور	امنیت	یک کوئری اضافه
۴	بستهٔ مشترک <code>@ghahve/shared</code>	واگرایی ممکن نیست	یک workspace بیشتر
۵	تقویم شمسی دست‌نویس	کنترل کامل	۲۸۳ خط نگهداری
۶	تصویر تولیدی	صفر فایل تصویر	۱,۰۲۵ خط کد
۷	محتوا در پایگاه داده	ویرایش بدون برنامه‌نویس	بدون اعتبارسنجی نوع
۸	Context API	صفر وابستگی	rerender اضافه
۹	کلید مرکب سبد	انعطاف واقعی	منطق ادغام
۱۰	تخفیف وزنی	چور با مدل کسب‌وکار	سبد فقط-ابزار ارسال می‌دهد
۱۱	پرفروش + سوپاپ دستی	داده + کنترل	دو فیلد اضافه
۱۲	کنسول انگلیسی	خوانایی در ویندوز	ناهمگونی زبان
۱۳	<code>tokenVersion</code>	ابطال ساده	یک کوئری در هر درخواست
۱۴	مسیر نسبی + پراکسی	کد یکسان همه‌جا	فقط same-origin
۱۵	seed غیرمخرب	حفظ کار مدیر	منطق پیچیده‌تر
۱۶	پیش‌نمایش با کامپوننت واقعی	صفر واگرایی	شیء تقلیدی لازم
۱۷	سبد در <code>localStorage</code>	بدون حساب کاربری	پاک‌سازی و سه‌نگهبان hydration
۱۸	دو فرآیند، یک مبدأ	رندر سمت سرور	دو چیز برای نگه داشتن
۱۹	بدون درگاه پرداخت	راه‌اندازی سریع	محدودیت جدی
۲۰	بدون حساب مشتری	قیف کوتاه‌تر	پیگیری فقط با «کد + موبایل»
۲۱	مهاجرت به Next.js	متن صفحه در پاسخ اول	جاوااسکریپت بیشتر
۲۲	nonce و <code>force-dynamic</code>	سیاست امنیتی واقعی	بدون کش ایستا
۲۳	مودال پشت آدرس واقعی	لینک قابل‌اشتراک + دکمهٔ back	یک <code>default.jsx</code> که باید بدانی
۲۴	دستی <code>og:type</code>	کارت کالا در تلگرام	یک استثنا در قاعده
۲۵	فهرست ناقص در صفحهٔ کالا	یک درخواست به‌جای صد و شش	سه شرط که سبد به آن‌ها بند است
۲۶	توست در انبارۀ ماژولی	مودال هم می‌تواند پیام بدهد	حالت سراسری بیرون از ری‌اکت
۲۷	چاپ با <code>display: none</code>	یک برگ به‌جای ۳۳	رسید دو بار در DOM
۲۸	هم‌رسانی بر پایهٔ نوع نشانگر	دکمه روی دکستاپ واقعاً کار می‌کند	انحراف از «ویژگی را تشخیص بده»
۲۹	بهینه‌سازی در زمان سرو	نسخهٔ اصلی می‌ماند، ۹۶٪ کمتر روی سیم	فایل خام روی دیسک

۱۰. پرسش و پاسخ آماده

چهل و پنج پرسش واقعی که ممکن است در مصاحبه یا بازبینی کد از شما بپرسند، با پاسخ‌های دقیق و برگرفته از خود کد. پنج پرسش بخش ز دربارهٔ موجودی انبار، پیگیری سفارش، آپلود و حالت تصویری میکس‌اند، چهار پرسش بخش ح دربارهٔ مهاجرت به Next.js، و شش پرسش بخش ط دربارهٔ هم‌رسانی، چاپ و بهینه‌سازی تصویر – همان چیزهایی که یک بازبین بیرونی اول از همه می‌پرسد.

بخش الف – معماری کلی

۱. این پروژه چه می‌کند و پشت‌فناوری‌اش چیست؟

یک فروشگاه آنلاین قهوه تخصصی با پنل مدیریت کامل: MongoDB با Express 4، Mongoose 8 به‌عنوان API، و Next.js 16 با App Router و React 19 برای فروشگاه و پنل. منطق مشترک هر دو طرف در یک workspace به‌نام `@ghahve/shared` است.

پروژه اول روی ویت و React Router بود و با مورد ۲۶ به Next منتقل شد؛ پوشه `client/` حذف شد و Express فقط API ماند. دلیل کاملش در فصل ۹، تصمیم ۲۱.

سه نکتهٔ متمایزش این است که قهوه به گرم فروخته می‌شود (قیمت ذخیره‌شده قیمت هر کیلوست)، مشتری می‌تواند نسبت دانه‌های میکس را خودش بچیند و قیمت آنی حساب شود، و هیچ عکس محصولی وجود ندارد – تصویر همهٔ ۱۰۶ کالا از روی `slug` به‌صورت SVG تولید می‌شود.

وابستگی‌های تولیدی کم‌اند: دوتا در سرور (`cors`, `multer`, `bcryptjs`, `jsonwebtoken`, `mongoose`, `express`) و سه‌تا در وب (`react-dom`, `react`, `next`) – به‌علاوه `@ghahve/shared` که بستهٔ محلی خودمان است.

بدون کتابخانهٔ مدیریت حالت، بدون UI kit، بدون CSS framework، بدون کتابخانهٔ تقویم شمسی، بدون کتابخانهٔ نمودار و بدون کتابخانهٔ سئو. مسیریابی هم کتابخانه نیست: جدول مسیرها ساختار پوشه‌های `web/src/app` است.

۲. چرا MongoDB و نه یک پایگاه دادهٔ رابطه‌ای؟

سه دلیل که هر سه در ساختار داده پیداست.

اول، ناهمگونی کالاها. سه نوع کالا داریم که فیلدهای مشترک دارند اما هرکدام فیلدهای اختصاصی هم دارند: `mat` (جنس) فقط برای ابزار، `tone` (رنگ) فقط برای پودر، `components` و `surcharge` فقط برای میکس‌های قهوه. در SQL یا سه جدول با `join` لازم بود، یا یک جدول پهن با ستون‌های `NULL`. در MongoDB سند فقط فیلدهای لازم را دارد.

دوم، مدل `Content`. متن‌های سایت با شکل `{key, data: Mixed}` ذخیره می‌شوند. کامنت مدل می‌گوید: «تا وقتی بخواهیم بخش تازه‌ای اضافه کنیم مجبور به مهاجرت پایگاه داده نشویم.» هر بخش سایت شکل دادهٔ خودش را دارد.

سوم، زیرسندهای تودرتو. `Order.lines[].mix[]` سه سطح تودرتوست و همیشه با هم خوانده می‌شوند. در SQL سه جدول با `join` لازم بود.

هزینه‌اش نبود یکپارچگی ارجاعی است که با اعتبارسنجی دستی جبران شده – `checkBlend` در `routes/items.js` و بررسی‌های `routes/orders.js`.

۳. چطور یک مدل واحد برای سه نوع کالای متفاوت کار می‌کند؟

با فیلد `kind` و یک فلاب `pre('validate')` که هفت گروه قاعده اعمال می‌کند. مثلاً:

```
if (this.kind === 'coffee') {
  this.shape = ''; this.mat = ''; this.tone = ''; this.pairs = [];
  this.grindable = true;
} else {
  this.grindable = false;
  this.isBlend = false;
  this.components = [];
  this.surcharge = 0;
}
```

سمت کلاینت، جدول `KINDS` در `web/src/lib/groups.js` همه تفاوت‌ها را داده‌محور می‌کند:

```
coffee: { weighed: true, weights: [100,250,500,1000], defaultWeight: 250, step: 100,
  minWeight: 100, meterLabel: 'درجه رست', priceLabel: 'هر کیلو', pairsLabel: null },
gear: { weighed: false, weights: [], step: 1, minWeight: 1,
  meterLabel: 'سطح مهارت لازم', priceLabel: 'قیمت هر عدد', pairsLabel: 'سازگار با' },
powder: { weighed: true, weights: [50,100,250,500], defaultWeight: 100, step: 50,
  minWeight: 50, meterLabel: 'شدت طعم', priceLabel: 'هر کیلو', pairsLabel: 'پیشنهاد سرو' }
```

نتیجه: یک `ItemCard`، یک `CatalogSection` و یک `AdminItemForm` برای هر سه نوع. کامنت مدل دلایل را می‌گوید: «یکی بودن مدل باعث می‌شود پنل مدیریت و مسیرهای API هم یکی باشند و جای کمتری برای اشتباه بماند.»

۴. مدیریت حالت را چطور انجام داده‌اید و چرا Redux نیست؟

با Context API و دو `Provider`: `ShopContext` (کالاها، محتوا، سبد، آسیاب پیش‌فرض، toast) و `AuthContext` (نشست مدیر).

Redux لازم نبود چون حالت سراسری کوچک است. حالت‌های دیگر عمداً محلی مانده‌اند: فیلترها در `HomeShell.jsx` (چون `BrewSection` و `GuideSection` هم عوضشان می‌کنند)، ترتیب و جست‌وجو در `CatalogSection`، ترکیب میکس در `BlendCard`، مرحله تسویه در `CartDrawer`.

مشکل `rerender` با `useMemo` و `useCallback` مدیریت شده:

```
const bySlug = useMemo(() => new Map(items.map((i) => [i.slug, i])), [items]);
const lookup = useCallback((slug) => bySlug.get(slug), [bySlug]);
const totals = useMemo(() => computeTotals(resolved, lookup), [resolved, lookup]);
```

صادقانه بگویم که یک نقطه بهبود هست: شیء `value` که به `Provider` داده می‌شود خودش با `useMemo` پوشیده نشده، پس در هر رندر `ShopProvider` یک شیء تازه ساخته می‌شود. با اندازه فعلی مشکلی ایجاد نمی‌کند اما با داده بزرگ‌تر محسوس می‌شد.

بخش ب – قیمت‌گذاری و منطق تجاری

۵. قیمت چطور محاسبه می‌شود؟

هسته محاسبه یک تابع است:

```
export const priceFor = (pricePerKg, grams) =>
  Math.round((pricePerKg * grams) / 1000 / 1000) * 1000;
```

قیمت هر کیلو ضربدر وزن، تقسیم بر ۱۰۰۰ (گرم به کیلو)، و بعد گرد به نزدیک‌ترین ۱۰۰۰ تومان. مثال: ۱,۸۵۰,۰۰۰ تومان هر کیلو در ۲۵۰ گرم می‌شود ۴۶۲,۵۰۰ که به ۴۶۳,۰۰۰ گرد می‌شود.

تکنیک استاندارد گرد کردن به مضرب است. دلیل گرد کردن این است که ضرب‌های اعشاری برای فاکتور دستی و پرداخت نقدی ناخوشایند است. `Math.round(x/1000)*1000`

جمع سبد در `computeTotals` انجام می‌شود که چهار انباشتگر جدا نگه می‌دارد: `grams`, `pieces`, `weighedBase`, `gearBase` – چون قواعد تخفیف و ارسال فقط به وزن کار دارند.

۶. پله‌های تخفیف چطور کار می‌کنند؟ چرا ترتیبشان نزولی است؟

```
export const TIERS = [
  { min: 5000, rate: 0.15, label: '۱۵%' },
  { min: 3000, rate: 0.10, label: '۱۰%' },
  { min: 1000, rate: 0.05, label: '۵%' },
  { min: 0, rate: 0, label: '' }
];

const tier = TIERS.find((t) => grams >= t.min);
```

ترتیب نزولی است چون `Array.prototype.find` اولین تطبیق را برمی‌گرداند. با ۳۵۰۰ گرم: `3500 >= 5000` ؟ نه. `3500 >= 3000` ؟ بله $\rightarrow 10\%$. اگر صعودی بود، همیشه اولین ردیف انتخاب می‌شد.

و ردیف `{min: 0, rate: 0}` نقش `else` را بازی می‌کند: `.find()` هیچ‌وقت `undefined` بر نمی‌گرداند، پس نیازی به بررسی `null` نیست.

نکته مهم: تخفیف روی `weighedBase` اعمال می‌شود نه `base`:

```
const discount = Math.round((weighedBase * tier.rate) / 1000) * 1000;
```

یعنی ابزار تخفیف نمی‌گیرد – و متن سایت هم این را صریح می‌گوید.

۷. چرا تخفیف بر پایه وزن است و نه مبلغ؟

دو دلیل، یکی تجاری و یکی اقتصادی.

تجاری: تخفیف روی مجموع وزن سبد است، نه هر کالا جداگانه. متن بخش «خرید کیلویی» این را به‌عنوان مزیت می‌فروشد:

تخفیف روی مجموع وزن سبد اعمال می‌شود، نه روی تک‌تک قهوه‌ها. یعنی می‌توانید ۵۰۰ گرم از سه خاستگاه بردارید و باز هم پله ۱.۵ کیلو را بگیرید.

این مشتری را تشویق می‌کند قهوه‌های بیشتری امتحان کند.

اقتصادی: تخفیف حجمی برای صرفه‌جویی در رست و بسته‌بندی است، نه پاداش خرید گران. ۱۰۰ گرم بلو مانتین ۸/۵ میلیون تومانی همان زحمت را دارد که ۱۰۰ گرم قهوه ارزان.

پیامد این تصمیم: خرید ۸/۵ میلیون تومان در ۱۰۰ گرم هیچ تخفیفی نمی‌گیرد، ولی ۱ کیلو میکس ۸۲۰ هزار تومانی ۵٪ می‌گیرد.

۸. قیمت میکس چطور حساب می‌شود؟

میانگین وزنی قیمت دانه‌ها به‌علاوه دستمزد میکس:

```
for (const p of parts) {
  const bean = lookup(p.slug);
  if (!bean) return item.price;
  total += bean.price * (Number(p.percent) || 0);
  percentSum += Number(p.percent) || 0;
}
if (percentSum <= 0) return item.price;
const weighted = total / percentSum;
return Math.round((weighted + (item.surcharge || 0)) / 1000) * 1000;
```

مثال با `espresso-70-30`: سرادو ۷۰٪ (۱,۲۰۰,۰۰۰) + مونسون ۳۰٪ (۱,۴۸۰,۰۰۰) = ۱۲۸,۴۰۰,۰۰۰ ÷ ۱۰۰ = ۱,۲۸۴,۰۰۰، به‌علاوه ۴۰,۰۰۰ دستمزد = ۱,۳۲۴,۰۰۰ تومان هر کیلو.

نکته ظریف: تقسیم بر `percentSum` است، نه بر ۱۰۰. کامنت توضیح می‌دهد: «تقسیم بر خودش یعنی اگر کمی کمتر یا بیشتر بود هم نتیجه معنی‌دار بماند.» اگر به‌خاطر گرد شدن مجموع ۹۹ شود، با تقسیم بر ۱۰۰ یک درصد از قیمت گم می‌شد.

و سه fallback امن دارد که همه به `item.price` برمی‌گردند: ترکیب خالی، دانه پیدانشده، و مجموع صفر. یعنی هیچ‌وقت `NaN` روی صفحه نمی‌آید.

۹. الگوریتم اهرم‌های میکس چطور مجموع را ۱۰۰ نگه می‌دارد؟

مفهوم کلیدی «جای خالی» (`room`) است. وقتی کاربر یک اهرم را می‌کشد:

```
const roomOf = (p) => (delta > 0 ? p.percent : 100 - p.percent);
const totalRoom = others.reduce((s, p) => s + Math.max(0, roomOf(p)), 0);
const need = Math.min(Math.abs(delta), totalRoom);
```

اگر این دانه زیاد می‌شود، بقیه باید کم شوند و حداکثر کاهش هرکدام مقدار فعلی‌اش است. اگر کم می‌شود، بقیه زیاد می‌شوند و حداکثر افزایش `100 - percent` است.

یعنی اگر جا نبود، اهرم زودتر متوقف می‌شود. `need = min(|delta|, totalRoom)`

سپس تخصیص نسبی:


```
take: Math.floor((need * room) / totalRoom)
```

و چون `floor` باقیمانده می‌گذارد، یک حلقه round-robin آن را پخش می‌کند:

```
let left = need - shares.reduce((s, x) => s + x.take, 0);
for (let i = 0; left > 0 && i < shares.length * 200; i++) {
  const s = shares[i % shares.length];
  if (s.take < s.room) { s.take++; left--; }
}
```

مثال: `A=50, B=30, C=20` و کاربر A را به ۸۰ می‌برد. `need=30, totalRoom=50, delta=+30`. سهم B سهم `floor(30*30/50)=18` و سهم C سهم `floor(30*20/50)=12` می‌گیرند. نتیجه: `A=80, B=12, C=8` – مجموع ۱۰۰.

آن `i < shares.length * 200` محافظ حلقه بی‌نهایت است.

۱۰. چرا `addBean` و `removeBean` منطق جداگانه ندارند؟

چون هر دو از `applyPercent` استفاده می‌کنند:

```
export function addBean(parts, slug, share = 20) {
  if (parts.some(p => p.slug === slug)) return parts;
  const withNew = [...parts, { slug, percent: 0 }];
  return applyPercent(withNew, slug, share);
}

export function removeBean(parts, slug) {
  if (parts.length <= 2) return parts;
  const zeroed = applyPercent(parts, slug, 0);
  return zeroed.filter(p => p.slug !== slug);
}
```

افزودن یعنی دانه را با صفر درصد اضافه کن (مجموع همچنان ۱۰۰ است) و بعد با `applyPercent` به ۲۰ ببر. حذف یعنی اول صفر کن (تا سهمش پخش شود) بعد حذف کن.

نتیجه: یک الگوریتم به‌جای سه، و همیشه مجموع درست می‌ماند.

نکته ظریف: `removeBean` وقتی بی‌اثر است همان آرایه را برمی‌گرداند، و `BlendsSection` با مقایسه مرجع این را تشخیص می‌دهد:

```
const next = removeBean(prev, slug);
if (next === prev) toast('میکس دست‌کم به دو دانه نیاز دارد');
```

بخش ج – امنیت

۱۱. چطور جلوی دستکاری قیمت را می‌گیرید؟

با اینکه قیمت اصلاً از مرورگر خوانده نمی‌شود. بار درخواست فقط این است:

```
{ slug: 'yirgacheffe', grams: 500, grind: 'v60' }
```

سرور کالاها را از پایگاه داده می‌خواند و همه‌چیز را از نو حساب می‌کند:

```
const items = await Item.find({ slug: { $in: [...wanted] }, active: true });
const bySlug = new Map(items.map((i) => [i.slug, i]));
const lookup = (slug) => bySlug.get(slug);
...
const totals = computeTotals(lines, lookup);
```

کامنت کد صریح است: «کالاها را از پایگاه داده می‌خوانیم – قیمت ارسالی از مرورگر اصلاً استفاده نمی‌شود.»

سپس نتیجه در سفارش ذخیره می‌شود (snapshot) تا تغییر قیمت آینده فاکتور قدیمی را عوض نکند. و پاسخ هم از روی سند ذخیره‌شده ساخته می‌شود: «تا مشتری دقیقاً همان چیزی را ببیند که ثبت شده.»

۱۲. جلوی mass assignment را چطور می‌گیرید؟

با whitelist در `routes/items.js`:

```
const WRITABLE = ['kind', 'slug', 'name', 'origin', 'spec', 'group', 'meter', 'price', 'stock',
  'notes', 'pairs', 'tag', 'shape', 'mat', 'tone', 'zoom', 'image', 'rank', 'active',
  'story', 'taste', 'recommend', 'tastes',
  'isBlend', 'house', 'customizable', 'components', 'pool', 'surcharge',
  'featured', 'pinnedTop', 'excludeTop'];

function pickBody(body) {
  const out = {};
  for (const key of WRITABLE) {
    if (body[key] === undefined) continue;
    out[key] = body[key];
  }
  ...
}
```

هر فیلدی که در این فهرست نباشد بی‌صدا حذف می‌شود – از جمله `_id`، `createdAt`، `weighed` و مهم‌تر از همه `grindable` که فیلد مشتق است و باید همیشه از `kind` محاسبه شود.

همین الگو برای محتوا هم هست:

```
const KEYS = Object.keys(DEFAULT_CONTENT);
if (!KEYS.includes(key)) return res.status(400).json({ error: 'این بخش محتوا وجود ندارد' });
```

۱۳. احراز هویت را چطور پیاده کرده‌اید و چطور توکن را باطل می‌کنید؟

JWT با یک شمارنده نسخه. توکن دو چیز حمل می‌کند:

```
jwt.sign({ sub: String(admin._id), v: admin.tokenVersion }, process.env.JWT_SECRET,
  { expiresIn: `${hours}h` });
```

و `requireAdmin` سه لایه بررسی می‌کند: وجود توکن، اعتبار امضا/انقضا، و همخوانی نسخه:

```
if (admin.tokenVersion !== payload.v) {
  return res.status(401).json({ error: 'رمز عوض شده است، دوباره وارد شوید' });
}
```

با تغییر رمز، `tokenVersion += 1` می‌شود و همه توکن‌های قبلی باطل می‌شوند – بدون جدول blacklist و بدون Redis. و بلافاصله توکن تازه برگردانده می‌شود تا خودِ مدیر بیرون نیفتد.

معامله‌ای که پذیرفته شده: `requireAdmin` در هر درخواست یک `findById` می‌زند، پس این JWT دیگر کاملاً stateless نیست. اما در ازایش ابطال فوری داریم.

۱۴. چرا پیام خطای ورود برای نام کاربری غلط و رمز غلط یکسان است؟

برای جلوگیری از `user enumeration`. کامنت خودِ کد این را می‌گوید:

```
/* پیام یکسان برای نام کاربری اشتباه و رمز اشتباه،
تا نشود فهمید کدام نام کاربری وجود دارد. */
const ok = admin && (await admin.checkPassword(password));
if (!ok) return res.status(401).json({ error: 'نام کاربری یا رمز عبور درست نیست' });
```

اگر پیام‌ها فرق می‌کردند، مهاجم می‌توانست با آزمون‌وخطا فهرست نام‌های کاربری معتبر را کشف کند و بعد فقط روی آن‌ها brute force کند.

توجه کنید که `(... await) admin &&` باعث می‌شود اگر کاربر پیدا نشود، `bcrypt.compare` اصلاً اجرا نشود. این از نظر امنیتی یک نقطهٔ ضعف کوچک است (تفاوت زمانی پاسخ می‌تواند وجود کاربر را لو دهد) که در فصل ۱۱ آمده.

۱۵. حملهٔ CSV Injection را می‌شناسید؟ در این پروژه چطور مدیریت شده؟

بله. اکسل هر سلولی را که با `=`، `+`، `-` یا `@` شروع شود به‌عنوان فرمول تفسیر می‌کند. اگر مشتری در فیلد نشانی بنویسد `=cmd|'/c calc'!A1` و مدیر فایل CSV را باز کند، ممکن است دستور اجرا شود.

راه‌حل در `routes/reports.js`:

```
/* اکسل هر سلولی را که با = + - @ شروع شود «فرمول» حساب
می‌کند. نشانی و توضیح مشتری را خود مشتری نوشته، پس
ابتدایش یک آپاستروف می‌گذاریم تا متن بماند. */
function csvCell(v) {
  let s = String(v ?? '');
  if (/^[+=\-\@\t\r]/.test(s)) s = "'" + s;
  return `"${s.replace(/"/g, '""')}"`;
}
```

آپاستروف ابتدای سلول به اکسل می‌گوید «این متن است». و `.replace(/"/g, '""')` هم escape استاندارد RFC 4180 برای نقل‌قول است.

۱۶. جست‌وجو چطور در برابر regex injection محافظت شده؟

با escape کردن همه کاراکترهای خاص regex:

```
const rx = new RegExp(q.replace(/[\.\*\?^\$\{\}\(\)\[\]\|\]\]/g, '\\$&'), 'i');
```

این خط در سه فایل تکرار شده: `reports.js`, `club.js`, `items.js`.

بدون آن دو مشکل داشتیم: ۱. جست‌وجوی `.` همه نتایج را برمی‌گرداند (چون `.` یعنی «هر نویسه»). ۲. جست‌وجوی الگویی مثل `(a+)+$` می‌توانست یک **ReDoS** بسازد که با backtracking نمایی CPU سرور را قفل کند.

`$&` در رشته جایگزین یعنی «همان چیزی که تطبیق یافت»، پس `.` به `\.` تبدیل می‌شود.

۱۷. آپلود تصویر چه محافظت‌هایی دارد؟

چهار لایه، در دو فایل.

لایه ۱ – فهرست سفید نوع فایل (`server/src/lib/upload.js`):

```
export const ALLOWED = {
  'image/jpeg': '.jpg', 'image/png': '.png', 'image/webp': '.webp',
  'image/gif': '.gif', 'image/avif': '.avif'
};

limits: { fileSize: 4 * 1024 * 1024 },
fileFilter: (_req, file, cb) => {
  if (!ALLOWED[file.mimetype]) {
    return cb(new Error('فقط تصویر JPG، PNG، WebP، AVIF یا GIF قابل آپلود است'));
  }
  cb(null, true);
}
```

SVG عمداً در این فهرست نیست. یک سند XML است، نه تصویر خام، و می‌تواند `<script>` داشته باشد؛ چون از `/uploads` روی همان دامنه سایت سرو می‌شود، آن اسکریپت در مبدأ خود فروشگاه اجرا می‌شد – یعنی به توکن مدیر در `localStorage` هم دسترسی داشت.

لایه ۲ – نام تصادفی:

```
const ext = ALLOWED[file.mimetype] || path.extname(file.originalname).toLowerCase() || '.bin';
cb(null, crypto.randomBytes(12).toString('hex') + ext);
```

هم مشکل نام فارسی و برخورد نام‌ها را حل می‌کند، هم **path traversal** را می‌بندد. اگر نام اصلی حفظ می‌شد، مهاجم می‌توانست بفرستد `../../server/src/index.js` بفرستد. توجه کنید پسوند از **mimetype** می‌آید، نه از نامی که کاربر فرستاده.

لایه ۳ – **هدرهای پوشه** `/uploads`. بستن فهرست مجاز فقط جلوی آپلود تازه را می‌گیرد؛ هرچه پیش از این آپلود شده هنوز روی دیسک است:

```
res.setHeader('X-Content-Type-Options', 'nosniff');
res.setHeader('Content-Security-Policy', "default-src 'none'; sandbox");
res.setHeader('Cross-Origin-Resource-Policy', 'same-origin');
res.setHeader('X-Frame-Options', 'DENY');

if (DOCUMENT_LIKE.has(ext)) res.setHeader('Content-Disposition', 'attachment');
```

سند را در مبدأ یکتا و بدون اسکرپت می‌گذارد، پس حتی SVG قدیمی آلوده هم دیگر به مبدأ فروشگاه دسترسی ندارد. `sandbox` نمی‌گذارد مرورگر یک PNG با محتوای HTML را HTML بخواند. و پسوندهای سندی (`.svg .svgz .xml .html ...`) `nosniff` به‌جای رندر، دانلود می‌شوند – که روی `<img src>` اثری ندارد، پس کارتها سالم می‌مانند.

لایهٔ ۴ – پاک‌سازی فایل یتیم هنگام حذف یا جایگزینی:

```
function removeImageFile(imagePath) {
  if (!imagePath || !imagePath.startsWith('/uploads/')) return;
  const file = path.join(UPLOAD_DIR, path.basename(imagePath));
  fs.promises.unlink(file).catch(() => {});
}
```

`path.basename()` هر `../` را حذف می‌کند – لایهٔ دفاعی دوم در برابر همان `path traversal`.

بخش د – پایگاه داده و مدل‌سازی

۱۸. چرا سفارش‌ها اطلاعات کالا را کپی می‌کنند به‌جای ارجاع؟

چون فاکتور باید در زمان منجمد بماند. کامنت مدل `Order`:

قیمت‌ها هنگام ثبت از روی پایگاه داده دوباره حساب می‌شوند ... ولی همان لحظه در سفارش ذخیره می‌شوند تا تغییر قیمت بعدی، فاکتور قدیمی را عوض نکند.

هر ردیف سفارش این‌ها را کپی می‌کند: `name`, `unitPrice`, `lineTotal`, `grindLabel`, و حتی نام هر دانه میکس:

```
mix: { slug: String, name: String, percent: Number }
```

با کامنت: «با نام دانه‌ها، تا فاکتور بعداً هم خوانا بماند.»

اگر فقط `slug` بود و مدیر نام «یرگاچف» را عوض می‌کرد یا کالا را حذف می‌کرد، فاکتور شش ماه پیش `yirgacheffe` خام نشان می‌داد.

همین‌طور `totals` هم ذخیره می‌شود که از `lines` قابل محاسبه است. دلیلش کارایی گزارش است: `reports/summary` فقط سه فیلد کوچک از هر سفارش می‌خواند:

```
Order.find({ createdAt: { $gte: windowStart } }, { createdAt: 1, status: 1, totals: 1 })
```

۱۹. چرا slug به جای ObjectId برای پیوند میکس‌ها؟

چون slug در این پروژه چهار نقش دارد که ObjectId نمی‌توانست ایفا کند:

۱. بذر تولید تصویر – seeded(p.slug) در art.js. با ObjectId طرح کالا بین محیط توسعه و تولید فرق می‌کرد. ۲. کلید خوانا در محتوا – مدیر در پنل picks: ['tarrazu', 'huila'] تایپ می‌کند. ۳. ستون فایل خروجی – «شناسه کالا» در CSV. ۴. پایداری در سفارش – معنی دارد حتی اگر کالا حذف شده باشد.

کامنت مدل هم می‌گوید: «هم در آدرس‌ها استفاده می‌شود و هم بذر تولید تصویر کارت است، پس نباید تکراری باشد.»

هزینه‌اش سه محافظ دستی است: checkBlend هنگام ذخیره، بررسی میکس هنگام سفارش، و پاک‌سازی سبد هنگام بارگذاری.

۲۰. چرا مسیر ویرایش از findByIdAndUpdate استفاده نمی‌کند؟

چون findByIdAndUpdate قلاب‌های سند (document middleware) را دور می‌زند. کد عمداً این‌طور نوشته شده:

```
const item = await Item.findById(req.params.id);
Object.assign(item, data);
await item.save();
```

همهٔ منطق اعتبارسنجی مدل Item – هفت گروه قاعده شامل «فقط قهوه آسیاب می‌شود»، «مجموع درصدهای میکس باید ۱۰۰ باشد»، «دسته کالا باید متعلق به نوعش باشد» – در pre('validate') است. با findByIdAndUpdate هیچ‌کدام اجرا نمی‌شدند و سند نامعتبر در پایگاه داده می‌نشست.

save() تضمین می‌کند که از هر مسیری (POST، PUT، seed.js) قواعد یکسان اعمال شوند.

۲۱. شماره سفارش چطور ساخته می‌شود؟ آیا یکتاست؟

```
orderSchema.pre('validate', function (next) {
  if (!this.code) {
    const stamp = Date.now().toString(36).slice(-4).toUpperCase();
    const rand = Math.floor(Math.random() * 9000 + 1000);
    this.code = `${stamp}-${rand}`;
  }
  next();
});
```

زمان به مبنای ۳۶ تبدیل می‌شود، چهار نویسهٔ آخرش گرفته می‌شود، و یک عدد تصادفی چهاررقمی به آن می‌چسبد. نتیجه چیزی مثل P9PT-8412 است – کوتاه، قابل خواندن روی تلفن، و بهتر از ObjectId بیست‌وچهار نویسه‌ای.

اما قطعاً یکتا نیست. فیلد code قید unique دارد، پس برخورد باعث خطای 11000 می‌شود که به «این شماره سفارش قبلاً استفاده شده است» تبدیل می‌شود – و هیچ منطق تلاش مجددی وجود ندارد. احتمالش کم است اما صفر نیست، و راه‌حلش یک حلقهٔ retry با چند تلاش است.

۲۲. رمز عبور چطور ذخیره می‌شود؟

هرگز خام ذخیره نمی‌شود. سه لایهٔ محافظت:

```
passwordHash: { type: String, required: true, select: false },

adminSchema.methods.setPassword = async function (plain) {
  this.passwordHash = await bcrypt.hash(plain, 12);
};

toJSON: { transform(_d, ret) { delete ret.passwordHash; delete ret.__v; return ret; } }
```

۱. **bcrypt** با ضریب کار ۱۲ – استاندارد امروزی. ۲. **select: false** – در کوئری معمولی خوانده نمی‌شود؛ برای خواندنش باید نوشت. **select('+passwordHash')**. ۳. **حذف در toJSON** – حتی اگر خوانده شد، در سریال‌سازی حذف می‌شود.

این «امنیت در عمق» است: اگر برنامه‌نویس اشتباهاً کل سند مدیر را برگرداند، هش بیرون نمی‌رود.

bcryptjs انتخاب شده به جای **bcrypt** چون ماژول native نیست و روی ویندوز بدون Visual Studio Build Tools نصب می‌شود. کندتر است اما در سالی چند ده عملیات هش، بی‌اهمیت.

بخش ۵ – تقویم شمسی و گزارش‌ها

۲۳. چرا تقویم شمسی را دستی نوشتید؟

دو دلیل که هیچ کتابخانه‌ای حلشان نمی‌کرد.

اول، بازه‌بندی نه فقط نمایش. کتابخانه‌های تبدیل تقویم فقط تاریخ را ترجمه می‌کنند. اما گزارش‌ها به مرز دقیق نیمه‌شب اول هر ماه شمسی به وقت تهران، به صورت یک لحظهٔ UTC نیاز دارند تا بشود در MongoDB کوئری زد:

```
Order.find({ createdAt: { $gte: bucket.start, $lt: bucket.end } })
```

کامنت فایل می‌گوید: «گزارش‌ها باید با تقویمی دسته‌بندی شوند که مدیر می‌بیند: «مرداد» یعنی مرداد، نه August.»

دوم، ساعت تابستانی تاریخی ایران. ایران تا ۱۴۰۱ ساعت تابستانی داشت. به جای offset ثابت، اختلاف واقعی همان لحظه پرسیده می‌شود:

```
function offsetMinAt(date) {
  const p = zoned(date); // Intl.DateTimeFormat timeZone: 'Asia/Tehran'
  const asUTC = Date.UTC(p.year, p.month - 1, p.day, p.hour, p.minute, p.second);
  return Math.round((asUTC - date.getTime()) / 60000);
}
```

این کار پایگاه دادهٔ IANA داخل موتور جاوااسکریپت را به کار می‌گیرد.

و نکتهٔ سوم: هفته از شنبه شروع می‌شود – نه یکشنبه (آمریکا) و نه دوشنبه (ISO):

```
const back = (p.weekday + 1) % 7; // getUTCDay: ۰ یکشنبه ... ۶ شنبه
```

۲۴. `tehranMidnight` چرا دو بار تصحیح می‌کند؟

مسئله مرغ و تخم‌مرغ: برای دانستن اختلاف زمانی در یک لحظه باید آن لحظه را بدانیم؛ اما برای دانستن آن لحظه باید اختلاف را بدانیم.

```
function tehranMidnight(gy, gm, gd) {
  const naive = Date.UTC(gy, gm - 1, gd);
  let t = naive - FALLBACK_OFFSET_MIN * 60000; // حدس اولیه ۳:۳۰
  for (let i = 0; i < 2; i += 1) t = naive - offsetMinAt(new Date(t)) * 60000;
  return new Date(t);
}
```

راه‌حل تکرار است: با حدس اولیه شروع کن، اختلاف واقعی را بپرس، تصحیح کن، دوباره بپرس. کامنت می‌گوید: «دو بار تصحیح می‌کنیم تا اگر آن روز مرز ساعت تابستانی بوده هم به نیمه‌شب درست برسیم.»

۲۵. فهرست بازه‌های گزارش چطور ساخته می‌شود؟

```
export function bucketSeries(period, count, now = new Date()) {
  const list = [];
  let b = bucketOf(now, period);
  for (let i = 0; i < count; i += 1) {
    list.unshift(b);
    b = previousBucket(b, period);
  }
  return list;
}

export function previousBucket(bucket, period) {
  return bucketOf(new Date(bucket.start.getTime() - DAY_MS), period);
}
```

ترفند اصلی در `previousBucket` است: به‌جای محاسبه حسابی «ماه قبل» (که با طول‌های متغیر ماه شمسی — ۳۱، ۳۰ یا ۲۹ روز — پیچیده است)، یک روز از شروع بازه کم می‌کند و می‌پرسد «این لحظه در کدام بازه است؟» چون یک روز قبل از اول مرداد، حتماً در تیر است.

`unshift` باعث می‌شود آرایه از قدیم به جدید مرتب باشد، و بازه‌های خالی هم می‌مانند تا نمودار حفره نداشته باشد.

۲۶. پرفروش‌ها چطور محاسبه می‌شوند؟

دو مرحله. اول `aggregate` روی سفارش‌ها:

```
{ $match: { status: { $ne: 'canceled' } } },
{ $unwind: '$lines' },
{ $group: { _id: '$lines.slug', orders: { $sum: 1 }, grams: {...}, qty: {...}, revenue: {...} } },
{ $sort: { orders: -1, revenue: -1 } },
{ $limit: 60 }
```

معیار «تعداد دفعات سفارش» (`$sum: 1`) است، نه وزن یا مبلغ. کامنت دلایلش را می‌گوید: «واحدها متفاوت‌اند، پس «تعداد دفعات سفارش» را ملاک می‌گیریم که بین وزنی و عددی قابل مقایسه است.» نمی‌شود ۵۰۰ گرم قهوه را با ۲ عدد ماگ مقایسه کرد.

مرحله دوم در جاوااسکریپت، join با کالاهای دو سوپاپ دستی:

```
.filter((it) => it.pinnedTop || it.sales)
.sort((a, b) => {
  if (a.pinnedTop !== b.pinnedTop) return a.pinnedTop ? -1 : 1;
  const ao = a.sales?.orders || 0, bo = b.sales?.orders || 0;
  if (bo !== ao) return bo - ao;
  return (a.rank || 999) - (b.rank || 999);
})
```

برای فروشگاه تازه که هنوز سفارشی ندارد، و `excludeTop` برای کالاهایی مثل فیلتر کاغذی که پرفروش‌اند اما ویتترین خوبی نیستند.

بخش و – رابط کاربری و RTL

۲۷. راست‌به‌چپ را چطور پیاده کرده‌اید؟

با CSS Logical Properties در کل استایل‌نامه، نه با فایل RTL جداگانه یا ابزار آینه‌کاری:

```
.nav{ margin-inline-start:auto; }
.card-chip{ inset-block-start:12px; inset-inline-start:12px; }
.card-media .badge{ inset-block-start:12px; inset-inline-end:12px; }
.cat-head h3{ padding-inline-end:16px; border-inline-end:2px solid var(--line); }
.wrap{ width:min(var(--wrap), 92%); margin-inline:auto; }
```

با `dir="rtl"` روی `<html>`، مرورگر خودکار `inline-start` را به «راست» ترجمه می‌کند. یعنی یک استایل‌نامه برای هر دو جهت.

برای محتوایی که باید LTR بماند، صریحاً علامت خورده:

<code><b dir="ltr">{order.code}</code>	<code>{*/ شماره سفارش */}</code>
<code>{order.customer.phone}</code>	<code>{*/ تلفن */}</code>
<code><small dir="ltr">{item.slug}</small></code>	<code>{*/ شناسه کالا */}</code>
<code><input className="input" dir="ltr" /></code>	<code>{*/ قیمت، نام کاربری */}</code>

دلیلش این است که الگوریتم دوجهته یونیکد رشته‌ای مثل `M3K2-8412` را ممکن است به هم بریزد، چون خط تیره جهت خنثی دارد. و اعداد فارسی با `toLocaleString('fa-IR')` تولید می‌شوند که هم رقم‌ها را فارسی می‌کند و هم جداکننده هزارگان فارسی (،) می‌گذارد.

۲۸. چرا شناسه ردیف سبد مرکب است و نه فقط slug؟

چون یک کالا می‌تواند چند بار با تنظیمات مختلف در سبد باشد. کامنت `web/src/lib/cartLine.js`:

یک قهوه می‌تواند چند بار در سبد باشد – یک بار دانه کامل و یک بار آسیاب اسپرسو – و یک میکس با دو نسبت متفاوت هم دو ردیف جداست.

```
export const lineKey = (slug, grind, mix) => `${slug}|${grind || ''}|${mixSignature(mix)}`;

export function mixSignature(mix) {
  return [...mix]
    .filter((m) => Number(m.percent) > 0)
    .map((m) => `${m.slug}:${Math.round(Number(m.percent) || 0)}%`)
    .sort()
    .join(',');
}
```

`.sort()` حیاتی است: `[cerrado 60, monsooned 40]` و `[monsooned 40, cerrado 60]` باید یک امضا بدهند.

و چون تغییر آسیاب کلید را عوض می‌کند، ممکن است دو ردیف یکی شوند — `setLineGrind` این را با ادغام مدیریت می‌کند:

```
return prev
  .map((l, idx) => idx === twin ? { ...l, grams: l.grams + line.grams, qty: l.qty + line.qty } : l)
  .filter((_, idx) => idx !== i);
```

۲۹. تصویر محصولات را از کجا آورده‌اید؟

هیچ‌جا — تولید می‌شوند. `web/src/lib/art.js` با ۱,۰۲۵ خط، برای هر کالا یک SVG می‌سازد.

هسته کار یک مولد شبه‌تصادفی با بذر ثابت است:

```
function seeded(str) {
  let h = 2166136261;
  for (let i = 0; i < str.length; i++) { h ^= str.charCodeAt(i); h = Math.imul(h, 16777619); }
  return () => { h = Math.imul(h ^ (h >> 15), 2246822507); h ^= h >> 13;
    return ((h >> 0) % 100000) / 100000; };
}
```

هش FNV-1a روی `slug` و بعد یک `xorshift`. چون بذر ثابت است، طرح هر کالا همیشه یکسان می‌ماند — با `Math.random()` هر `refresh` چیدمان عوض می‌شد.

تصویر اطلاعات واقعی حمل می‌کند: رنگ دانه از `meter` (درجه رست) می‌آید، پس قهوه رست تیره واقعاً تیره‌تر کشیده می‌شود:

```
const ROAST_TONE = { 1: ['#D3A468',...], 5: ['#523020',...] };
const [c1, c2, crease] = ROAST_TONE[p.meter] || ROAST_TONE[3];
```

برای ابزار، ۳۶ طرح در `SHAPES` و ۱۱ جنس در `MATERIAL` تعریف شده — یعنی ۳۹۶ ترکیب ممکن با نوشتن ۴۷ تعریف.

نکته فنی مهم: هر SVG یک `uid` از `slug` می‌گیرد چون شناسه‌های SVG سراسری هستند و بدون آن همه ۱۰۶ کارت یک گرادیان می‌گرفتند.

و مدیر می‌تواند عکس واقعی آپلود کند که جای طرح می‌نشیند.

۳۰. بزرگ‌ترین ضعف‌های این پروژه چیست و چطور رفعشان می‌کنید؟

اول آنچه رفع شده، چون سؤال بعدی همیشه همین است:

ضعف	چه شد
صفر تست	بیست‌ودو فایل تست با Vitest (۳۸۰ سنجه)، بدون نیاز به مونگو + CI روی هر push
بدون rate limiting	سه محدودکننده روی ورود، ثبت سفارش و پیگیری
دوگانگی <code>pricing.js</code>	workspace به نام <code>@ghahve/shared</code> ؛ کپی‌ها حذف شدند
آپلود SVG	از فهرست مجاز بیرون رفت + هدرهای سخت‌گیر روی <code>/uploads</code>
CORS باز در تولید	نبود <code>CLIENT_ORIGIN</code> حالا خطای مرگبار است
بدون هدر امنیتی	<code>helmet</code> برای API، و CSP با <code>nonce</code> برای صفحه‌ها
بدون موجودی انبار	<code>Item.stock</code> با رزرو اتمی هنگام ثبت سفارش
بی‌خبری مشتری از سفارش	صفحه <code>/track</code> با جفت «کد + موبایل»
تزییق در SVG	<code>esc()</code> روی هر متنی که از پایگاه داده می‌آید
رندر کاملاً سمت کلاینت	مهاجرت به Next.js App Router – متن صفحه در پاسخ اول
کالا بدون URL اختصاصی	<code>/coffee/:slug</code> و هم‌تاهایش؛ مودال هم آدرس گرفت
بدون sitemap، Open Graph و robots	<code>generateMetadata</code> روی سرور + <code>sitemap.xml</code> زنده Express
بدون داده‌های ساختاریافته	<code>Product</code> روی صفحه کالا، <code>LocalBusiness</code> روی صفحه اصلی
فونت از Google Fonts	<code>next/font</code> – فایل‌ها موقع build گرفته و روی خودمان میزبانی می‌شوند
تصویر خام تا ۴ مگابایت	<code>next/image</code> روی عکس آپلودی – تصویرهای صفحه فهرست ۹۷٪ کوچک‌تر

پنج سطر آخر همان چیزی است که در نسخه پیشین این پاسخ **بزرگ‌ترین ضعف** نامیده می‌شد. آن بند این‌طور بود و امروز دیگر درست نیست:

بدون SSR – بزرگ‌ترین ضعف از دید SEO. پروژه CSR خالص است و خزنده بدون جاوااسکریپت صفحه خالی می‌بیند ... راه‌حل کامل مهاجرت به Next.js است.

راه‌حل کامل انجام شد. شرحش در فصل ۹، تصمیم ۲۱، و پرسش‌های بخش ح همین فصل.

و آنچه هنوز باقی است – پنج موردی که اگر فردا وقت داشتم سراغشان می‌رفتم:

۱) توکن در `localStorage`. در برابر XSS آسیب‌پذیر است. امن‌تر `httpOnly cookie` با `SameSite=Strict` است – که آن‌وقت CSRF را برمی‌گرداند و به توکن CSRF نیاز پیدا می‌کنیم. `dangerouslySetInnerHTML` در `CardArt` دیگر خطر مستقیم نیست (هر متنی از `esc` رد می‌شود) ولی همچنان جایی است که باید مراقبش بود.

۲) بدون درگاه پرداخت. سفارش ثبت می‌شود و تماس تلفنی می‌گیرند. برای یک فروشگاه واقعی، این بزرگ‌ترین شکاف کارکردی است – و حالا که آن ضعف SEO رفع شده، بزرگ‌ترین ضعف باقی‌مانده همین است.

۳) کد اسپلیت، نیمه‌کاره. جاوااسکریپت خودبه‌خود بر اساس مسیر تقسیم شد (بازدیدکننده فروشگاه دیگر کد پنل را نمی‌گیرد)، ولی CSS همچنان یکجاست: هر دو فایل در `app/layout.jsx` وارد می‌شوند.

(۴) شمارنده محدودیت نرخ درون حافظه‌ای است. با چند پروسه، سقف واقعی چند برابر می‌شود. برای استقرار جدی باید انبار مشترک (مثلاً Redis) گذاشت – و حالا که دو فرآیند داریم این مهم‌تر هم شده.

(۵) بدون کش ایستا. بهای آگاهانه nonce است (تصمیم ۲۲): همه صفحه‌ها پویا رندر می‌شوند، نه `revalidate` داریم و نه صفحه کش‌شده روی CDN.

فهرست کامل‌تر با وضعیت هر مورد در فصل ۱۱ آمده – از جمله نبود تله فوکوس در مودال‌ها، اینکه موجودی میکس از دانه‌های اجزایش کم نمی‌شود، و ۱۷ هشدار `set-state-in-effect` که عمداً روی `warn` مانده‌اند (مورد ۴۲).

بخش ز – تغییرهای تازه

۳۱. موجودی انبار چطور کم می‌شود و چرا فروش بیش از موجودی ممکن نیست؟

با یک عملیات اتمی مونگو به جای الگوی «اول بخوان، بعد بنویس».

مسئله را با یک سناریو نشان می‌دهم. فرض کنید ۵۰۰ گرم یرگاچف مانده و دو مشتری هم‌زمان همان ۵۰۰ گرم را سفارش می‌دهند:

زمان	مشتری الف	مشتری ب
t1	را می‌خواند $\text{stock} = 500$	
t2	را می‌خواند $\text{stock} = 500$	
t3	(بررسی موجودی) $500 \geq 500 \checkmark$	
t4	$500 \geq 500 \checkmark$	
t5	می‌نویسد $\text{stock} = 0$	
t6	می‌نویسد $\text{stock} = 0$	

یک کیلو فروختیم، ۵۰۰ گرم داشتیم

فاصله بین بررسی و نوشتن همان چیزی است که باید حذف شود. کد این کار را با گذاشتن شرط داخل خود به‌روزرسانی انجام می‌دهد:

(`server/src/lib/stock.js`)

```
const updated = await ItemModel.findOneAndUpdate(
  { slug: item.slug, stock: { $gte: need } }, // شرط
  { $inc: { stock: -need } }, // کاهش
  { new: true, projection: { stock: 1 } }
);
```

مونگو تضمین می‌کند که به‌روزرسانی یک سند اتمی است – شرط و `$inc` زیر یک قفل سند اجرا می‌شوند. پس همان سناریو این‌طور تمام می‌شود:

t1	$500 \leq 500 \rightarrow$ سند قفل $\checkmark$ الف: $\text{stock} = 0$
t2	$500 \leq 0 \rightarrow$ سند قفل $\times$ ب: <code>null</code>

دقیقاً یکی برنده می‌شود و دیگری `null` می‌گیرد که یعنی «نرسید».

چرا transaction نه؟ چون لازم نیست، و چون روی مونگوی تک‌گرهی – که محیط توسعه این پروژه است – اصلاً در دسترس نیست. برای یک شمارنده ساده، `findOneAndUpdate` شرطی همان ضمانت را با هزینه خیلی کمتر می‌دهد.

دو نکته ظریف:

الف) `null` یعنی نامحدود و اصلاً لمس نمی‌شود.

```
if (!isTracked(item)) continue;
```

به جای خراب شدن «ناموجود» می‌شد — یک شکست امن ولی اشتباه. پس نگهبان صریح گذاشته‌ایم.

ب) همه یا هیچ. هر ردیف سبد جدا رزرو می‌شود، پس ردیف سوم می‌تواند بعد از موفقیت دو ردیف اول شکست بخورد. `taken` هر رزرو موفق را نگه می‌دارد و در شکست همه با هم برمی‌گردند:

```
if (!updated) {
  await releaseStock(taken, ItemModel);
  ...
  return { error: shortMessage(item, left), taken: [] };
}
```

و اگر خود `Order.create` هم خطا بدهد، روتر همین `releaseStock` را صدا می‌زند. نتیجه یک ضمانت ساده است: سفارش یا کامل ثبت می‌شود یا اصلاً — و انبار هیچ‌وقت بی‌دلیل کم نمی‌ماند.

چرا ۴۰۹ و نه ۴۰۰؟ چون ورودی مشتری غلط نبود؛ وضعیت سرور عوض شده بود. با ۴۰۹، رابط کاربری سبد را دست‌نخورده نگه می‌دارد و فقط پیام موجودی را نشان می‌دهد.

چطور تست شده بدون مونگو؟ مدل تزریق می‌شود (`reserveStock(lines, ItemModel)`) و تست یک بدل چند خطی می‌سازد که همان ضمانت را تقلید می‌کند — شرط `$gte` را واقعاً بررسی می‌کند، و گزینه تست «یک گرم بیشتر از موجودی» بی‌معنی بود.

۳۲. چرا مسیر پیگیری برای شماره غلط و کد ناموجود یک پاسخ می‌دهد؟

چون اگر فرق می‌کرد، صفحه پیگیری به ابزار شمارش سفارش‌ها تبدیل می‌شد.

عمومی است — مشتری حساب کاربری ندارد که با آن وارد شود. فرض کنید پاسخ‌ها این‌طور بودند:

- کد وجود ندارد → «چنین سفارشی نداریم»

- کد هست ولی شماره جور نیست → «شماره موبایل درست نیست»

آن‌وقت یک اسکریپت می‌توانست کدها را یکی‌یکی امتحان کند و بدون دانستن هیچ شماره‌ای بفهمد کدام کدها وجود دارند. از آنجا: تعداد سفارش‌های فروشگاه، نرخ رشدشان، و اینکه رسید افتاده‌ای که پیدا کرده هنوز معتبر است یا نه.

پس یک پیام برای هر دو حالت، از یک ثابت صادرشده:

```
export const NOT_FOUND_MESSAGE = 'سفارشی با این شماره و شماره موبایل پیدا نشد';
```

ثابت بودنش عمدی است — تا اگر کسی خواست پیام را عوض کند، مجبور شود در یک جا عوضش کند و دو پیام متفاوت درنیاید.

ولی پیام یکسان کافی نیست. اگر برای کد ناموجود سریع‌تر پاسخ بدهیم، خود زمان پاسخ همان چیزی را لو می‌دهد که پنهانش کردیم. پس کد عمداً زودتر برنمی‌گردد:

```
const doc = await OrderModel.findOne({ code: wantedCode }).lean();

/* حتی وقتی سفارشی پیدا نشده، یک مقایسه انجام می‌دهیم -
تا زمان پاسخ در هر دو حالت یکی باشد. */
const stored = normalizePhone(doc?.customer?.phone);
const matches = safeEqual(stored || ' no-order', wantedPhone);

if (!doc || !matches) {
  return { status: 404, error: NOT_FOUND_MESSAGE };
}
```

و خود مقایسه هم زمان ثابت است:

```
export function safeEqual(a, b) {
  const ha = crypto.createHash('sha256').update(String(a ?? ''), 'utf8').digest();
  const hb = crypto.createHash('sha256').update(String(b ?? ''), 'utf8').digest();
  return crypto.timingSafeEqual(ha, hb);
}
```

چرا اول هش؟ چون `timingSafeEqual` طول‌های نابرابر را قبول نمی‌کند و پرتاب خطا می‌کند – که خودش یک نشت زمانی است: شماره ۵ رقمی فوراً خطا می‌گرفت، شماره ۱۱ رقمی غلط کندتر رد می‌شد. خروجی sha256 همیشه ۳۲ بایت است، پس مقایسه همیشه یک شکل و یک هزینه دارد.

استثنا: ورودی ناقص (کد یا شماره خالی) `400` می‌گیرد با پیام راهنما، نه `404`. اینجا چیزی برای پنهان کردن نیست – کاربر هنوز چیزی نپرسیده.

و لایه آخر: `trackLimiter` با ۲۰ تلاش در ۱۵ دقیقه. سخاوتمندتر از ثبت سفارش، چون مشتری واقعی ممکن است چند بار غلط تایپ کند؛ ولی آن قدر کم که جست‌وجوی فراگیر روی فضای کدها معنا نداشته باشد.

۳۳. چرا نشانی مشتری در صفحه پیگیری نشان داده نمی‌شود؟

چون دیدن وضعیت سفارش به نشانی نیاز دارد، و هر فیلد اضافه‌ای که برگردانیم چیزی است که در یک حدس موفق به دست مهاجم می‌افتد.

اثبات مالکیت اینجا یک جفت کوتاه است: شماره سفارش + شماره موبایل. این از یک رمز عبور خیلی ضعیف‌تر است – کد هشت نویسه است و شماره را ممکن است کسی از جای دیگری داشته باشد. پس فرض را بر این می‌گذاریم که روزی یک حدس موفق می‌شود، و می‌پرسیم: آن وقت چه چیزی لو رفته؟

با طراحی فعلی: نام، وضعیت، فهرست کالاها و مبلغ. بدون آن: نشانی خانه یک نفر، به علاوه شماره تماس و یادداشتش.

روش پیاده‌سازی هم اهمیت دارد. `publicOrderView` سند را بازسازی می‌کند، نه اینکه فیلدهای ناخواسته را حذف کند:

```
return {
  code: order.code,
  status: order.status,
  createdAt: order.createdAt,
  customer: { name: order.customer?.name || '' },
  lines: (order.lines || []).map((l) => ({ kind: l.kind, name: l.name, ... })),
  totals: { base: ..., discount: ..., shipping: ..., total: ... }
};
```

فرق whitelist و blacklist در آینده است. اگر با `delete` کار می‌کردیم، هر فیلدی که فردا به مدل `Order` اضافه شود (یادداشت داخلی، حاشیه سود، شماره پیک) به طور پیش فرض بیرون می‌رفت و کسی متوجه نمی‌شد. با `Whitelist`، به طور پیش فرض داخل نمی‌آید.

و این ادعا تست دارد:

```
it('فیلد تازه‌ای که فردا به مدل اضافه شود، خودبه‌خود بیرون می‌ماند', () => {
  const withInternal = { ...sample(), adminNote: 'مشتری بدحساب', profitMargin: 42 };
  const flat = JSON.stringify(publicOrderView(withInternal));
  expect(flat).not.toContain('adminNote');
  expect(flat).not.toContain('بدحساب');
});
```

چرا نام می‌ماند؟ چون به مشتری اطمینان می‌دهد سفارش خودش را می‌بیند، و چیزی بیشتر از آنچه خودش نوشته به او نشان نمی‌دهد.

نکته آخر: این تصمیم سمت سرور گرفته شده، نه در `Track.jsx`. اگر فقط رابط کاربری نشانی را نشان نمی‌داد، هر کسی که مستقیماً API را صدا می‌زد آن را می‌دید.

۳۴. اگر SVG آپلودی خطرناک است، چرا خودتان SVG تولید می‌کنید؟

چون این دو اصلاً یک چیز نیستند: یکی فایلی است که از بیرون می‌آید و سرو می‌شود، دیگری رشته‌ای است که خودمان می‌سازیم.

SVG آپلودی. یک سند XML کامل است که مرورگر آن را به‌عنوان سند می‌خواند، نه تصویر. می‌تواند `<script>` داشته باشد، `<foreignObject>` با HTML داشته باشد، و منابع بیرونی صدا بزند. و چون از `/uploads` روی همان دامنه فروشگاه سرو می‌شود، آن اسکریپت در مبدأ خود سایت اجرا می‌شود — یعنی به `localStorage` و در نتیجه به توکن مدیر دسترسی داشت.

مهاجم لازم نیست مدیر باشد؛ کافی است مدیر را قانع کند فایلی را آپلود کند، یا خود حساب مدیر یک بار به هر دلیلی در دست کسی بیفتد. پس از فهرست مجاز بیرون رفت.

SVG تولیدی. خروجی `art.js` هیچ‌وقت روی دیسک نمی‌نشیند و هیچ‌وقت به‌عنوان سند مستقل باز نمی‌شود. یک رشته است که مستقیماً به DOM تزریق می‌شود:

```
<span className="art-holder" dangerouslySetInnerHTML={{ __html: svg }} />
```

ساختارش را ما نوشته‌ایم؛ تنها چیزی که از بیرون می‌آید متنهای داخلش است (نام کالا، نام دسته، نام دانه‌های میکس) که مدیر واردشان کرده. و دقیقاً همین‌ها از یک صافی رد می‌شوند:

```
const ESCAPES = { '<': '&lt;', '>': '&gt;', '&': '&amp;', '"': '&quot;', "'": '&#39;' };
export const esc = (s) => String(s ?? '').replace(/[<>'"']/g, (c) => ESCAPES[c]);
```

چرا لازم است؟ چون خروجی این فایل رشته است نه گره React، پس React هیچ‌چیز را برایمان escape نمی‌کند. بدون `esc`، نامی مثل `onload="alert(1)"` از `aria-label` بیرون می‌زد:

```
<!-- esc بدون --!>
<svg aria-label="" onload="alert(1)">
```

هر چهار تولیدکننده را با نام خصمانه می‌سنجد – و یک تست هم عمداً رشته escape می‌سازد تا مطمئن شویم خود سنج کار می‌کند.

خلاصه‌اش: خطر از «فرمت SVG» نمی‌آید، از مسیر رسیدن می‌آید. فایل ناشناخته‌ای که با مبدأ سایت سرو شود خطرناک است؛ رشته‌ای که خودمان با ورودی escape شده می‌سازیم نه.

و چون فایل‌های قدیمی هنوز روی دیسک‌اند، لایهٔ دومی هم هست: `default-src 'none'; sandbox` و `nosniff` و دانلود اجباری روی پسوندهای سندی – یعنی حتی SVG آلودهٔ باقی‌مانده از قبل هم دیگر به مبدأ فروشگاه دسترسی ندارد.

۳۵. ساز میکس دو حالت دارد؛ چطور منطقشان دوتا نشده؟

با یک قاعدهٔ ساده: حالت تصویری هیچ حالتی از خودش دربارهٔ ترکیب ندارد. فقط یک نمای دیگر از همان داده است.

همه‌چیز ترکیب در `BlendCard` بالای هر دو می‌ماند:

```
const [mix, setMix] = useState(() => startingMix(item));
const [grams, setGrams] = useState(KINDS.coffee.defaultWeight);
const [pick, setPick] = useState(grind);

const [visual, setVisual] = useState(false); // تنها چیزی که فرق می‌کند
```

`MixVisual` همان `mix` را به‌عنوان prop می‌گیرد و برای تغییرش همان تابعی را صدا می‌زند که اهرم‌های ساده صدا می‌زنند:

```
<MixVisual mix={mix} onPercent={change} onDrop={drop} onJoin={join} onAdd={add} ... />
```

و `change` چیزی جز `applyPercent` نیست – همان ریاضی‌ای که مجموع را ۱۰۰ نگه می‌دارد. یعنی رفت‌وبرگشت بین دو حالت هیچ انتخابی را از دست نمی‌دهد، چون اصلاً چیزی جابه‌جا نمی‌شود.

سه چیز عمداً مشترک‌اند:

چه چیزی	کجا	چرا
ریاضی درصدها	<code>lib/blend.js</code> → <code>applyPercent</code>	یک قاعده برای «کشیدن اهرم»
شکل ردیف سبد	<code>lib/cartLine.js</code> → <code>buildLine</code>	تا ردیف هر دو راه مو به مو یکی باشد
فهرست دانه‌های افزودنی	<code>BeanPicker.jsx</code>	یک قاعده برای «چه چیزی می‌شود اضافه کرد»

مهم‌ترینشان دومی است. `mixSignature` و `buildLine` قبلاً داخل `ShopContext` زندگی می‌کردند؛ وقتی راه دوم ساختن میکس اضافه شد، به یک فایل خالص منتقل شدند تا هر دو راه از یک تابع رد شوند و ادعای «یک میکس، از هر راهی که ساخته شود، دقیقاً همان ردیف سبد» تست‌پذیر باشد نه یک امید.

و تنها یک راه به سبد وجود دارد:

```
const add = useCallback(
  () => addWeighed(item.slug, grams, { grind: pick, mix }),
  [addWeighed, item.slug, grams, pick, mix]
);
```

دکمهٔ پایان مرحلهٔ ۴ در حالت تصویری دقیقاً همین را صدا می‌زند، و دکمهٔ پاورقی هم در هر دو حالت سر جایش است – راه انداختن دستگاه یک خوشی اختیاری است، نه دروازهٔ خرید.

پس `blendVisual.js` چه کاره است؟ فقط تصمیم‌های نمایشی: هر کیسه چقدر بریزد (`pourPlan`)، کیف بعد از هر کیسه تا کجا پر شود (`hopperLevel`)، و کیسه‌ها کجای صحنه بایستند (`sackLayout`). هیچ درصدی آنجا حساب نمی‌شود. همه هم تابع خالص‌اند، پس بدون مرورگر تست می‌شوند.

یک نکته همگام‌سازی: اگر ترکیب عوض شود، آنچه دستگاه ساخته دیگر معتبر نیست. `MixVisual` این را با مقایسه امضا حین رندر می‌گیرد، نه در `useEffect`:

```
const [ranWith, setRanWith] = useState(signature);
if (ranWith !== signature) {
  setRanWith(signature);
  setStage('pick');
  setPoured(0);
}
```

با `useEffect`، کاربر یک فریم بسته قبلی را می‌دید و بعد پرش می‌خورد.

بخش ح – مهاجرت به Next.js

۳۶. کامپوننت سروری در این پروژه دقیقاً چه چیزی می‌خرد؟

سه چیز، و فقط یکی‌شان «سرعت» است.

۱) متن صفحه در پاسخ اول. این دلیل اصلی بود و بقیه جانبی‌اند. پیش‌تر خزنده‌ای که جاوااسکریپت اجرا نمی‌کند – تلگرام، واتساپ، بخشی از خزنده‌های گوگل – یک `<div id="root">` خالی می‌دید. راه میانی‌مان تزریق `<head>` از Express بود که کارت پیش‌نمایش را درست کرد ولی بدنه را نه. حالا معرفی کامل هر کالا، فهرست کارت‌ها و JSON-LD همه داخل همان HTML اولی‌اند که سرور می‌فرستد.

۲) یک رفت‌وبرگشت به جای سه تا. ترتیب قبلی این بود: HTML خالی → دانلود و اجرای باندل → `useEffect` در `ShopContext` → `/api/items` و `/api/content` → اولین قهوه. حالا `app/page.jsx` هر دو را روی سرور و با `Promise.all` می‌گیرد و نتیجه به شکل prop پایین می‌رود.

۳) کدی که برای مرورگر فرستاده نمی‌شود. این ظریف‌ترین سودش است. مثلاً `ItemBody.jsx` – متن معرفی کالا – هیچ هوکی ندارد، پس سروری است و جاوااسکریپتش اصلاً در باندل نمی‌آید. همین‌طور `art.js` که SVG کارت‌ها را می‌سازد.

و چه چیزی نمی‌خرد. این نکته را بازبین معمولاً می‌پرسد: `HomeShell.jsx` و همه کامپوننت‌های زیرش `'use client'` دارند، چون فیلترهای دسته حالت‌اند. یعنی جاوااسکریپت‌شان همچنان فرستاده می‌شود.

ولی کامپوننت مشتری هم روی سرور رندر می‌شود – متنش در HTML هست. تفاوت «سروری» و «مشتری» این نیست که کدامیک در HTML می‌آید (هر دو می‌آیند)، بلکه این است که کدامیک جاوااسکریپت هم می‌فرستد. پس نسبت به نسخه ویت چیزی از دست نرفته و متن صفحه اضافه شده.

مرزها در عمل با سه چیز کشیده شدند:

مرز	نمونه	چرا
حالت لازم است	HomeShell, ItemBuyCard, CartDrawer	useState فقط در کامپوننت مشتری کار می‌کند
تابع باید پاس داده شود	SiteHeader	تابع از مرز سروری به مشتری رد نمی‌شود؛ فقط داده قابل‌سریال‌سازی
نه این و نه آن	StaticSections, ItemBody	سروری می‌ماند و هیچ کدی نمی‌فرستد

۳۷. سبد چطور از hydration جان سالم به در می‌برد؟

با یک پرچم به‌نام `hydrated` و سه نگهبان دور آن. این حساس‌ترین جای کل مهاجرت بود، چون شکستش دیده نمی‌شود: سبد پاک می‌شود و کاربر فکر می‌کند خودش کاری کرده.

مسئله. نسخهٔ ویت سبد را در همان اولین رندر می‌خواند:

```
// با SSR سه‌چور می‌شکند
const [cart, setCart] = useState(loadCart);
```

- روی سرور `localStorage` وجود ندارد؛
 - اگر سرور سبد خالی بفرستد و مرورگر در همان رندر اول سبد پُر بسازد، ری‌اکت ناسازگاری می‌بیند و می‌تواند درخت را دور بریزد؛
 - و اگر افکت ذخیره پیش از افکت خواندن اجرا شود، همان آرایهٔ خالی اولیه روی سبد ذخیره‌شده نوشته می‌شود.
- جواب.** سبد همیشه با آرایهٔ خالی شروع می‌شود، پس HTML سرور و اولین رندر مرورگر مو به مو یکی‌اند. خواندن در یک افکت یک‌باره است و نوشتن پشت پرچم:

```
useEffect(() => {
  const saved = loadCart(browserStorage());
  if (saved.length) setCart(saved);
  setHydrated(true);
}, []);

useEffect(() => {
  if (!hydrated) return; // ← نگهبان
  saveCart(browserStorage(), cart);
}, [cart, hydrated]);
```

بهایش یک فریم است. تا اجرای افکت اول، شمارندهٔ سبد در هدر «گرم» است. عمداً با `suppressHydrationWarning` پنهان نشده — آن پرچم هشدار را خاموش می‌کند، نه مسئله را. `hydrated` از کانتکست بیرون داده می‌شود تا هرچه به سبد بستگی دارد بتواند تا آن لحظه حالت خنثی نشان بدهد.

دو نگهبان دیگر به فهرست کالاها مربوط‌اند، نه به زمان‌بندی: افکت پاک‌سازی سبد فقط وقتی اجرا می‌شود که هم `hydrated` باشد و هم `catalogueComplete` — وگرنه صفحهٔ یک کالا، که عمداً فهرست ناقص دارد، کل سبد را پاک می‌کرد. جزئیاتش در ۷.۱۲ و در تصمیم ۲۵ فصل ۹.

چطور مطمئنیم؟ `tests/cart-hydration.test.jsx` یک چرخهٔ کامل «رندر سرور ← hydrate» را در jsdom اجرا می‌کند و می‌سنجد که سبد ذخیره‌شده دست‌نخورده بیرون بیاید. یکی از دو تست کامپوننتی کل پروژه است، و دلایش همین است: هیچ تابع خالصی نمی‌تواند این را نگه دارد.

۳۸. چرا SVG کارت‌ها روی سرور رندر می‌شود، با اینکه HTML را بزرگ می‌کند؟

چون گزینه دیگر بدتر است، و بزرگ شدن HTML آن قدر که به نظر می‌رسد گران نیست.

`art.js` یک تابع خالص است که رشته SVG برمی‌گرداند، و `CardArt` هیچ هوکی ندارد. پس هر ۳۲ کارت صفحه اصلی طرحشان داخل همان HTML اول می‌آید.

اگر در مرورگر می‌ساختیم چه می‌شد؟ یکی از این دو:

- کارت‌ها در HTML بی‌تصویر می‌آمدند و بعد از hydrate پر می‌شدند – یعنی یک پرش چیدمانی روی هر کارت، و صفحه‌ای که برای خزنده بدون جاوااسکریپت هیچ تصویری ندارد. دقیقاً همان چیزی که مورد ۲۶ می‌خواست حلش کند، این بار فقط برای تصویرها.
- یا `CardArt` را کامپوننت مشتری می‌کردیم و SVG را در `useEffect` تزریق می‌کردیم – که همان مسئله است، به علاوه جاوااسکریپت ۱،۰۲۵ خطی `art.js` که حالا باید دانلود هم بشود.

سه چیز هزینه را قابل تحمل می‌کند:

۱) **gzip روی SVG تکراری بسیار مؤثر است.** طرح‌ها از یک مجموعه محدود (۳۶ طرح ابزار، ۱۴ طرح پودر، یک مولد دانه) ساخته می‌شوند و رشته‌های تکراری زیادی دارند. `compression` روی API روشن است و Next هم پاسخ‌های خودش را فشرده می‌کند.

۲) **بدیلش یک فایل تصویر واقعی بود، نه هیچ.** اگر عکس محصول داشتیم، به ازای هر کارت یک درخواست شبکه و ده‌ها کیلوبایت می‌رفت. SVG درون خطی از هر دو ارزان‌تر است.

۳) **هزینه‌اش یک بار است.** با `force-dynamic` هیچ صفحه‌ای کش نمی‌شود، ولی SVG هم چیزی نیست که با گذشت زمان بزرگ‌تر شود.

و یک سود امنیتی جانبی: چون این SVG روی سرور تولید می‌شود و از `esc()` رد می‌شود، نام خصمانه کالا نمی‌تواند صفتی به تگ اضافه کند – همان چیزی که `tests/card-art.test.js` می‌سنجد. اگر در مرورگر ساخته می‌شد، همان تابع لازم بود ولی حالا در دسترس کد سمت کاربر هم بود.

کجا واقعاً هزینه دارد؟ صفحه اصلی که سه فهرست کامل دارد. اگر روزی تعداد کالاها چند برابر شود، راه حل صفحه‌بندی فهرست است (مورد ۲۱ فصل ۱۱)، نه بردن تولید تصویر به مرورگر.

۳۹. اگر `force-dynamic` را بردارید، چه چیزی می‌شکند؟

سایت. نه کند می‌شود، نه بد ایندکس می‌شود – بالا نمی‌آید.

زنجیره‌اش این است:

- `web/src/proxy.js` برای هر درخواست یک `nonce` تازه می‌سازد و در سیاست امنیتی می‌نویسد: `script-src 'self'`
- Next همان `nonce` را از هدر درخواست برمی‌دارد و روی `<script>` های خودش می‌گذارد.
- صفحه‌ای که موقع **build** ساخته شده باشد، `nonce` آن درخواست را ندارد – `nonce`ی که در HTML کش‌شده نشسته، مال چند روز پیش است.
- مرورگر همه آن اسکریپت‌ها را می‌بندد.

نتیجه: صفحه رندر می‌شود، متنش دیده می‌شود، و هیچ وقت زنده نمی‌شود. دکمه سبد کار نمی‌کند، فیلترها کار نمی‌کنند، ساز میکس تکان نمی‌خورد. و چون HTML درست است، در نگاه اول به نظر می‌رسد همه چیز سالم است – بدترین نوع خرابی.

اگر **nonce** را هم با آن بردارید چه؟ آن وقت باید `'unsafe-inline'` بگذارید، چون Next داده رندر سمت سرور را به شکل `<script>` درون خطی می نویسد. و `'unsafe-inline'` عملاً کل ارزش سیاست را دور می ریزد – یعنی مورد ۷ سند ضعف ها را نیمه کاره می کند. این معامله ای بود که آگاهانه رد شد.

پس چه چیزی از دست رفته؟ `revalidate`، صفحه کش شده روی CDN، و `generateStaticParams` برای صفحه های کالا.

و چقدر واقعاً از دست رفته؟ کمتر از آنچه به نظر می رسد. تقریباً هیچ صفحه این فروشگاه ایستا نیست: فهرست کالاها، موجودی انبار و متن های سایت همه از پایگاه داده می آیند و مدیر هر روز عوضشان می کند. صفحه کش شده یعنی نشان دادن کالایی که تمام شده – که با مورد ۳۴ دقیقاً همان چیزی است که نمی خواهیم.

راه بازش، اگر روزی لازم شد: برای مسیرهای کم تغییر **nonce** را کنار بگذارید و همان ها را ایستا کنید، نه اینکه سیاست را برای کل سایت شل کنید. `force-dynamic` در `app/layout.jsx` است و می شود در یک مسیر خاص `override` اش کرد؛ دلیل کاملش هم در کامنت خود همان فایل نوشته شده تا کسی بی خبر برش ندارد.

بخش ط – هم رسانی، چاپ و تصویر

۴۰. دکمه هم رسانی چرا روی دسکتاپ سراغ `navigator.share` نمی رود، با اینکه هست؟

چون بودنش با مفید بودنش یکی نیست.

شرط اول همان چیزی بود که هر راهنمایی می گوید:

```
if (typeof nav.share === 'function') { ... }
```

با این فرض که «تقریباً هیچ مرورگر دسکتاپی `share` ندارد». کروم روی ویندوز آن را دارد – ولی چیزی که باز می کند برگه اشتراک خود ویندوز است که معمولاً خالی است و کاربر بی کار می بنددش.

و اینجا زنجیره بد تمام می شد: بستن آن برگه یک `AbortError` می دهد، و کد به درستی «کاربر نظرش عوض شد» تفسیرش می کرد و ساکت می ماند. یعنی در آن مسیر هیچ وقت کاری به کلیک پوردد نمی رسید. کاربر دسکتاپ دکمه را می زد، پنبلی می آمد و می رفت، و لینک هیچ جا کپی نشده بود – دکمه ای که ظاهراً کار می کرد و عملاً هیچ نمی کرد.

پس ملاک عوض شد از «مرورگر `share` دارد؟» به «دستگاه واقعاً لمسی است؟»:

```
export function isTouchDevice(opts = {}) {
  const { nav = globalThis.navigator, win = globalThis } = opts;

  const mq = win?.matchMedia?.('(pointer: coarse)');
  if (typeof mq?.matches === 'boolean') return mq.matches;

  return Number(nav?.maxTouchPoints) > 0;
}
```

`(pointer: coarse)` یعنی نشانگر اصلی دستگاه انگشت است. لپ تاپ لمسی که ماوس هم دارد `fine` گزارش می دهد و درست هم همین است – آنجا کپی به کار کاربر می آید، نه فهرستی از اپ هایی که ندارد. `maxTouchPoints` فقط تکیه گاه آخر است، برای مرورگر بی `matchMedia`؛ به تنهایی ملاک بدی است چون هر نمایشگر لمسی وصل به دسکتاپ آن را بالای صفر می کند.

و اگر بگویند این خلاف «ویژگی را تشخیص بده، نه دستگاه را» است؟ درست می گویند، و جواب این است که آن قاعده به پرسش دیگری جواب می دهد. «این API وجود دارد؟» و «این API اینجا چیز خوبی می دهد؟» دو چیزند، و تشخیص ویژگی فقط به اولی جواب می دهد. ضمناً `(pointer: coarse)` خودش یک پرسمان استاندارد است، نه تجزیه `userAgent`.

چهار تست در `tests/share.test.js` – بخش «دسکتاپی که `navigator.share` دارد» – تنها چیزی‌اند که جلوی برگشتن رفتار قبلی را می‌گیرند.

۴۱. لینکی که دکمه هم‌رسانی می‌فرستد از کجا می‌آید؟

از `canonical` خود صفحه، و نه از رشته‌بافی دستی:

```
export function itemShareUrl(item, env) {
  return itemHeadState(item, { baseUrl: siteBaseUrl(env) }).canonical;
}
```

چرا این مهم است. لینکی که مشتری برای دوستش می‌فرستد باید دقیقاً همان آدرسی باشد که در `canonical` و `og:url` صفحه نوشته شده. اگر نبود، پیش‌نمایشی که تلگرام و واتساپ می‌سازند به یک آدرس نگاه می‌کند و لینک به آدرسی دیگر می‌رود – و کاربر عکس و عنوان یک کالا را می‌بیند و به کالای دیگری (یا به ۴۰۴) می‌رسد.

`itemHeadState` همان تابعی است که `generateMetadata` صفحه کالا از آن می‌خواند و `buildSitemap` هم بر پایه‌اش کار می‌کند. سه مصرف‌کننده، یک منبع – همان قاعده ۴ پروژه.

اگر کسی روزی `'/coffee/' + item.slug` بنویسد، تا روز عوض‌شدن مسیرها یا آدرس پایه هیچ‌چیز خراب به‌نظر نمی‌رسد، و بعد لینک‌هایی که قبلاً فرستاده شده‌اند به ۴۰۴ می‌رسند. `tests/share.test.js` دقیقاً این را می‌بندد:

```
const canonical = itemHeadState(item, { baseUrl: 'https://daneh.example' }).canonical;

expect(itemShareUrl(item, ENV)).toBe(canonical);
expect(itemShareUrl(item, ENV)).toBe(itemUrl('https://daneh.example', item));
```

و یک حالت مرزی که تست جدا دارد: بدون `NEXT_PUBLIC_SITE_URL`، `siteBaseUrl` رشته خالی می‌دهد و نتیجه یک مسیر نسبی می‌شود، نه آدرسی حدسی. مسیر نسبی بهتر از آدرس غلط است.

۴۲. چرا توست از `ShopContext` بیرون کشیده شد؟ کانتکست که مشکلی نداشت.

کانتکست مشکلی نداشت؛ مرزش غلط بود.

توست هیچ‌وقت واقعاً حالت فروشگاه نبود. فقط آنجا زندگی می‌کرد چون اولین صداکننده‌اش سبک بود. تا وقتی همه‌چیز روی صفحه اصلی می‌گذشت، هزینه‌ای نداشت.

بعد دو چیز هم‌زمان روشنش کرد:

(۱) **مودال کالا بیرون از `Provider` است.** در شکاف `@modal` رندر می‌شود (تصمیم ۲۳). وقتی دکمه هم‌رسانی به مودال اضافه شد، به چیزی نیاز داشت که بگوید «کپی شد» – و آنجا هیچ `Provider` ای بالای سرش نبود.

(۲) **`Toast` فقط در `HomeShell` بود.** یعنی صفحه اختصاصی کالا و مودالش هر دو بی‌صدا بودند: مشتری چیزی به سبد اضافه می‌کرد و هیچ تأییدی نمی‌گرفت.

راه آسان‌تر چه بود و چرا نه؟ یک `ShopProvider` دوم دور مودال. ولی آن‌وقت دو سبک جدا می‌داشتیم و افزودن از یکی روی دیگری اثر نمی‌کرد – همان دامی که تصمیم ۲۳ برای پرهیز از آن، داده مودال را به‌شکل `prop` می‌فرستد.

پس به‌جای بزرگ‌کردن کانتکست، این یک تکه از آن بیرون آمد:

```
let message = '';
const listeners = new Set();

export function toast(msg) { ... emit(); }
export function subscribeToast(listener) { listeners.add(listener); ... }
export const getToast = () => message;
export const getServerToast = () => '';
```

و `Toast.jsx` با `useSyncExternalStore` وصل می‌شود:

```
const message = useSyncExternalStore(subscribeToast, getToast, getServerToast);
```

چرا انبارهٔ مازولی و نه یک کانتکستِ دومِ کوچک؟ چون توست یک چیز است در کل صفحه، نه یک چیز به‌ازای هر زیردرخت. کانتکست ابزارِ دامنه‌بندی است و اینجا دامنه‌ای در کار نیست. با انبارهٔ مازولی، `<Toast />` یک بار در `app/layout.jsx` می‌نشیند — بالای `children` و `modal` هر دو — و هر کامپوننتی در هر شکافی می‌تواند پیام بدهد.

و ده صداکننده دست نخوردند. یک پوششِ دوخطی در `ShopContext` ماند:

```
const toast = useCallback((msg) => showToast(msg), []);
```

۴۳. `getServerToast` چرا جداست و چرا همیشه رشتهٔ خالی می‌دهد؟

چون متغیرهای یک مازول روی سرور بین درخواست‌ها مشترک‌اند.

آرگومان سوم `useSyncExternalStore` همان snapshot سرور است. اگر آن را نمی‌دادیم دو چیز می‌شد: Next موقع رندر سروری خطا می‌داد، و اگر هم `getToast` را جای هر دو می‌گذاشتیم، هر پیامی که به هر دلیلی روی میز مانده بود داخل HTML بازدیدکنندهٔ بعدی می‌نشست — و `hydrate` هم به‌هم می‌ریخت، چون مرورگر همان لحظه پیامی ندارد.

در عمل `toast()` فقط از دل رویدادهای مرورگر صدا زده می‌شود، پس آن متغیرها روی سرور هیچ‌وقت پر نمی‌شوند. ولی این از آن دسته چیزهایی است که «در عمل اتفاق نمی‌افتد» دلیل خوبی برای باز گذاشتنش نیست: هزینهٔ بستنش یک خط است و هزینهٔ نبستنش نشتبِ داده بین بازدیدکننده‌هاست.

یک تست جدا همین را نگه می‌دارد:

```
it('snapshot', () => {
  // 'سرور همیشه خالی است، حتی وقتی پیامی روی میز باشد'
  toast('چیزی که فقط مال مرورگر است');

  expect(getToast()).toBe('چیزی که فقط مال مرورگر است');
  expect(getServerToast()).toBe('');
});
```

۴۴. چرا رسید برای چاپ دو بار رندر می‌شود؟ این اسراف نیست؟

نه، و بدیش گران‌تر بود.

مسئله‌ای که حل می‌کند. نسخهٔ پیشین رسید را با `visibility: hidden` روی `body *` چاپ می‌کرد و فقط کشوی سبد را برمی‌گرداند. منطقش موجه به‌نظر می‌رسید: کشو چند لایه داخل درخت است و `display: none` روی نیاکانش خودش را هم می‌برد.

ولی `visibility` عنصر را از چیدمان حذف نمی‌کند؛ فقط رنگش را برمی‌دارد. کل فروشگاه – هدر، سه فهرست کالا، ده‌ها کارت و SVG‌هایشان، فوتر – همچنان ارتفاع واقعی خودشان را داشتند و مرورگر همان ارتفاع را صفحه‌بندی می‌کرد. نتیجه اندازه‌گیری شد: ۳۳ صفحه، که ۳۱ تای آخرش کاغذ سفید خالی بود.

این خرابی در توسعه دیده نمی‌شود: پیش‌نمایش چاپ صفحه اول را درست نشان می‌دهد.

راه‌حل مسئله را از CSS به جایگاه در درخت منتقل کرد. رسید یک بار دیگر و به‌شکل فرزند `body` مستقیم رندر می‌شود، از راه `createPortal`. آن‌وقت قاعده چاپ یک خط است و هیچ فرضی درباره ساختار صفحه ندارد:

```
body.has-print-sheet > *:not(.print-sheet) { display: none !important; }
```

و اینکه چرا ارزان است:

- میزبان روی صحنه `display: none` است، پس مرورگر چیدمانش را حساب نمی‌کند؛
 - `aria-hidden="true"` دارد، پس صفحه‌خوان دو بار نمی‌خواندش؛
 - و هر دو نسخه همان کامپوننت `Receipt` اند. اگر دو پیاده‌سازی می‌شد، روزی یکی «تخفیف وزنی» را نشان می‌داد و دیگری نه.
- آن شرط `has-print-sheet` هم تزئینی نیست. بدون آن، قاعده روی هر صفحه‌ای برقرار بود و `Ctrl+P` روی خود فروشگاه یک برگ سفید می‌داد. کلاس را `PrintSheet` موقع mount روی `body` می‌گذارد و در پاک‌سازی افکت برمی‌دارد.
- و یک احتیاط: portal به `document` نیاز دارد و روی سرور چنین چیزی نیست، پس تا mount نشدن `null` برمی‌گردد – HTML سرور و اولین رندر مرورگر یکی می‌مانند.

۴۵. چرا تصویر را در زمان آپلود کوچک نمی‌کنید؟ فایل ۴ مگابایتی روی دیسک می‌ماند.

می‌ماند، و این معامله‌ای است که آگاهانه انتخاب شد.

سند ضعیف‌ها (مورد ۲۴) `sharp` در خود مسیر آپلود را پیشنهاد داده بود. آن‌طور انجام نشد چون کلاینت دیگر ویت نیست: `Next` خودش بهینه‌ساز تصویر دارد و پشتش هم همان `sharp` است. با `next/image` هیچ نسخه‌ای در لحظه آپلود ساخته نمی‌شود؛ هر اندازه‌ای که واقعاً خواسته شود ساخته و روی دیسک کش می‌شود، و قالبش از هدر `Accept` همان مرورگر می‌آید.

چهار چیزی که این راه می‌دهد و کوچک‌سازی زمان آپلود نمی‌دهد:

۱. نسخه اصلی می‌ماند. اگر فردا قالب کارت عوض شد، همه چیز از نو ساخته می‌شود بی‌آنکه کسی مجبور باشد عکس‌ها را دوباره آپلود کند. کوچک‌سازی زمان آپلود یک تصمیم برگشت‌ناپذیر است. ۲. فهرست اندازه‌ها دستی نگه داشته نمی‌شود. مرورگر از `sizes` می‌گوید چه می‌خواهد. ۳. `AVIF` و `WebP` رایگان‌اند و برای مرورگری که نمی‌فهمد، اصل می‌رود. ۴. هزینه یک‌بار است – نتیجه روی دیسک کش می‌شود، با `minimumCacheTTL` هفت‌روزه که هم‌تراز پوشه `uploads` است.

و اندازه‌گیری روی صفحه فهرست، با ۳۸ کالای روشن که همه‌شان عکس آپلودی داشتند (عکس آزمایشی ۲۰۱۶ در ۱۵۱۲ پیکسل، میانگین ۴۵۲ کیلوبایت):

صرفه	با <code>next/image</code>	خام	
۹۴.۸٪	۴۷,۴۴۰	۹۱۴,۵۷۲	بالای صفحه (۳ عکس، بدون اسکرول)
۹۶.۶٪	۶۰۳,۷۶۰	۱۷,۵۸۰,۷۶۳	کل صفحه (۵۴ قاب، ۳۸ عکس یکتا)
۹۶.۷٪	(AVIF) ۱۶,۰۴۸	۴۷۹,۹۱۹	یک عکس در قاب کارت

بیشتر این صرفه از اندازه می‌آید نه از قالب: همان عکس در ۴۴۸ پیکسل و به شکل JPEG هم ۲۵,۳۶۴ بایت می‌شود – یعنی مرورگری که AVIF نمی‌فهمد هم ۹۴.۷٪ کمتر می‌گیرد.

دو چیزی که اگر نبودند این یک در باز می‌شد:

(۱) `LocalPatterns`. بدون آن هر مسیر محلی‌ای بهینه‌شدنی است و `/_next/image` عملاً یک واکنش عمومی می‌شود. دو الگو بیشتر تعریف نشده: `/img/**` و `/uploads/**`.

(۲) `lib/img.js`. SVG هیچ‌وقت نباید به بهینه‌ساز برسد. `dangerouslyAllowSVG` خاموش است (مورد ۵) و بهینه‌ساز به SVG پاسخ ۴۰۰ می‌دهد – یعنی قاب خالی، نه عکس بهینه‌نشده. همان قاعده آدرس بیرونی دستی‌نوشته مدیر را هم بیرون می‌گذارد.

و هدرها؟ با این تغییر مرورگر دیگر مستقیم `/uploads/...` را نمی‌خواند، پس هدرهای `setUploadHeaders` روی پاسخی که واقعاً می‌رسد نیستند. آن‌ها سر جای خودشان ماندند (هر کس آدرس را مستقیم باز کند همان‌ها را می‌گیرد) و معادلشان برای مسیر تازه در فایل `next.config.mjs` ساخته شد: `nosniff`، `Cross-Origin-Resource-Policy` و `X-Frame-Options`. دوتای مهم‌تر – CSP با `sandbox` و `Content-Disposition: attachment` – را خود Next می‌گذارد.

آنچه هنوز باز است: فایل خام تا چهار مگابایت روی دیسک می‌ماند. اگر روزی فضا تنگ شد، فشرده‌کردن در خود `upload.js` گام بعدی است – این بار کنار بهینه‌ساز زمان سرو، نه به‌جایش.

۱۱. نقاط ضعف فعلی و مسیر بهبود

این فصل صادقانه است. هر مورد از خودِ کد استخراج شده، با ارجاع دقیق و راه‌حل پیشنهادی.

وضعیت فعلی

این فهرست یک‌بار نوشته نشد و بعد رها؛ بخشی از آن اجرا شده و متن هر مورد هم به‌روز است. موردی که رفع شده، بالای خودش یک بلوک ✓ حل شد دارد که می‌گوید دقیقاً چه شد، چه چیزی از راه‌حل پیشنهادی عوض شد، و چه باقی‌مانده‌ای هست.

✓ رفع‌شده (بیشتر مورد)

مورد	عنوان	چه شد
۱	بدون محدودیت نرخ روی ورود	<code>loginLimiter</code> – ۵ در ۱۵ دقیقه، از <code>.env</code>
۲	مسیر عمومی ثبت سفارش بدون محافظ	<code>orderLimiter</code> – ۵ در ساعت (+ <code>trackLimiter</code>)
۴	<code>dangerouslySetInnerHTML</code> برای SVG	<code>esc()</code> روی هر متنی که وارد SVG می‌شود
۵	آپلود SVG مجاز است	از فهرست مجاز بیرون رفت + هدرهای سخت‌گیر <code>/uploads</code>
۶	CORS باز در نبود <code>CLIENT_ORIGIN</code>	در تولید خطای مرگبار، در <code>lib/cors.js</code>
۷	بدون Helmet	CSP متناسب همین پروژه + HSTS فقط در تولید
۱۱	صفر تست	بیشتر وودو فایل Vitest (۳۸۰ سنجه)، بدون نیاز به مونگو
۱۲	دوگانگی <code>pricing.js</code>	<code>@ghahve/shared</code> workspace؛ کپی‌ها حذف شدند
۱۳	دوگانگی <code>taxonomy.js</code> و <code>groups.js</code>	کلید در <code>shared</code> ، برجسب در <code>web</code> ، همگامی با تست
۱۴	بدون ESLint و Prettier	<code>flat config</code> + Prettier، هر دو گام در CI
۲۲	بدون ایندکس روی <code>Order.createdAt</code>	دو ایندکس، یکی مرکب با <code>status</code>
۲۴	بدون فشرده‌سازی و بدون بهینه‌سازی تصویر	<code>compression</code> روی API (فهرست کالاها ۷۸٪ کوچک‌تر) و <code>next/image</code> روی عکس آپلودی (تصویرهای صفحه فهرست ۹۷٪ کوچک‌تر)
۲۵	فونت از Google Fonts	<code>next/font</code> – فایل‌ها موقع <code>build</code> گرفته و روی خودمان میزبانی می‌شوند
۲۶	رندر کاملاً سمت کلاینت	مهاجرت به Next.js App Router؛ تصمیم «انجام نشود» برگشت
۲۷	هیچ محصولی URL اختصاصی ندارد	<code>/coffee/:slug</code> و هم‌تاهایش؛ مودال هم آدرس گرفت
۲۸	بدون داده‌های ساختاریافته	<code>Product</code> روی صفحه کالا، <code>LocalBusiness</code> روی صفحه اصلی
۲۹	بدون sitemap، Open Graph و robots	<code>generateMetadata</code> سمت سرور + <code>sitemap.xml</code> زنده Express

مورد	عنوان	چه شد
۳۴	بدون موجودی انبار	<code>Item.stock</code> با رزرو اتمی و پس‌دادن در شکست
۳۵	بدون حساب کاربری و تاریخچه سفارش	صفحه <code>/track</code> با جفت «کد + موبایل»
۳۹	بدون CI/CD و بدون Docker	نیمه CI انجام شد؛ استقرار و Docker نه

○ باقی‌مانده (بیست‌ویک مورد)

دسته	موردها
امنیت	۳ (توکن در <code>localStorage</code>) ۸۰ (حملة زمانی ورود) ۹۰ (بدون sanitize) ۱۰۰ (بدون لاگ حسابرسی)
کیفیت کد	۱۵ (بدون TypeScript) ۱۶۰ (فیلدهای مرده) ۱۷۰ (CSS مرده) ۱۸۰ (از نسخه ایستا فقط <code>img/</code> ریشه مانده) ۱۹۰ (شماره سفارش بدون <code>retry</code>) ۴۲۰ (<code>setState</code> در افکت – تصمیمش گرفته شد، الگو ماند)
کارایی	۲۰ (بدون کد اسپلیت) ۲۱۰ (بدون صفحه‌بندی کالا) ۲۳۰ (<code>value</code> بدون <code>useMemo</code>)
SEO	– همه چهار مورد این دسته (۲۶ تا ۲۹) رفع شده‌اند
دسترس‌پذیری	۳۰ (focus trap) ۳۱۰ (عناصر غیرمعنایی) ۳۲۰ (بدون بررسی خودکار)
کسب‌وکار	۳۳ (درگاه پرداخت) ۳۶۰ (OTP و اطلاع‌رسانی) ۳۷۰ (ارسال منطقه‌ای) ۳۸۰ (ماشین حالت سفارش)
زیرساخت	۴۰ (لاگ ساخت‌یافته) ۴۱۰ (پشتیبان خودکار) و نیمه CD/Docker از مورد ۳۹

باقی‌مانده‌های ریز داخل موردهای رفع‌شده

شش چیز که با رفع مورد اصلی تمام نشدند و عمداً باز مانده‌اند:

- **مورد ۱ –** شمارنده محدودیت نرخ درون‌حافظه‌ای است؛ با چند پروسه سقف واقعی چند برابر می‌شود. برای استقرار جدی `rate-limit-redis` لازم است.
- **مورد ۳۴ –** موجودی میکس از دانه‌های اجزایش کم نمی‌شود، و لغو سفارش موجودی را برنمی‌گرداند.
- **مورد ۳۹ –** CD و Docker هنوز نیستند؛ CI فقط می‌گوید کد سالم است.
- **مورد ۲۶ –** با آمدن nonce در سیاست امنیتی، همه صفحه‌ها پویا رندر می‌شوند؛ کش ایستا و `revalidate` روی میز نیست. تصمیم آگاهانه است، نه جا افتادن (تصمیم ۲۲ فصل ۹).
- **مورد ۴۲ –** قاعده `set-state-in-effect` روی `warn` ماند و ۱۷ هشدار باقی است. بخشی‌شان مثبت کاذب‌اند (نگهبان‌های hydration)، بخشی‌شان بدهی واقعی پتل.
- **مورد ۲۴ –** عکس آپلودی در زمان سرو بهینه می‌شود، پس فایل خام تا ۴ مگابایت روی دیسک می‌ماند. برخلاف چهار مورد بالا این یک بدهی نیست: انتخابی است که در تصمیم ۲۹ فصل ۹ توضیح داده شده – نسخه اصلی می‌ماند تا قاب فردا هم بتواند از آن نسخه بسازد.

پنج کار خارج از این فهرست

- (۱) **حالت تصویری ساز میکس.** ویژگی تازه است، نه رفع ضعف: نمای دومی از همان ترکیب، با همان `applyPercent` و همان ردیف سبد. جزئیاتش در فصل ۵ و ۷ آمده.
- (۲) **در `art.js`.** در همان گذری که آپلود SVG بسته شد (مورد ۵)، تزریق از راه نام کالا هم بسته شد – که مورد ۴ بود.
- (۳) **دکمه هم‌رسانی کالا.** روی کارت، صفحه کالا و مودال رهگیری‌شده. نشانی‌اش از `canonical` همان صفحه می‌آید و ملاک «برگه سیستم یا کپی؟» نوع نشانگر است، نه بودن `navigator.share` – تصمیم ۲۸، و جریان‌ش در ۶.۱۱.

(۴) **توست** از `ShopContext` بیرون آمد. لازمهٔ مورد ۳ بود: مودال بیرون از Provider رندر می‌شود. حالا `lib/toastStore.js` با `useSyncExternalStore`، و `<Toast />` یک بار در `app/layout.js` – پس صفحهٔ کالا و مودالش هم دیگر بی‌صدا نیستند. تصمیم ۲۶.

(۵) **رسید مشترک و بازنویسی سبک چاپ**. `Receipt.js` از `CartDrawer` بیرون کشیده شد تا صفحهٔ پیگیری هم همان برگه را چاپ کند، و سبک چاپ از `visibility` به `display: none` رفت – یک برگ کاغذ به‌جای ۳۳ تا. تصمیم ۲۷، و سبکش در ۸۰۱۰. سه تای آخر در یک گذر انجام شدند و به هم بند بودند: دکمهٔ هم‌رسانی توست لازم داشت، توست باید از کانتکست بیرون می‌آمد، و رسید چاپی راهی به `/track` می‌خواست.

۱۱.۱ امنیت

۱) بدون محدودیت نرخ روی ورود

✓ **حل شد**. `server/src/middleware/rateLimit.js` ساخته شد و `loginLimiter` روی `POST /api/auth/login` نشست. پیش‌فرض همان چیزی است که این بند پیشنهاد داده – ۵ تلاش در ۱۵ دقیقه – ولی از `.env` خوانده می‌شود (`RATE_LIMIT_LOGIN_WINDOW_MIN`، `RATE_LIMIT_LOGIN_MAX`) تا بشود بدون دست زدن به کد سفتش کرد یا شل.

هر دو نکتهٔ حیاتی زیر هم اجرا شد:

- `trust proxy` **مشروط** در `server/src/index.js`، دقیقاً با همان شرط `NODE_ENV === 'production' || TRUST_PROXY === '1'`.
 - **انبارۀ درون‌حافظه‌ای** هنوز همان است و **باقی‌ماندهٔ این مورد است**: با چند پروسه، سقف واقعی چند برابر می‌شود. برای استقرار چندپروسه‌ای `rate-limit-redis` لازم است.
- در محیط تست محدودیت خاموش است (`skip: isOff`)، وگرنه مجموعهٔ تست بعد از پنج فراخوانی به ۴۲۹ می‌خورد. تست: `tests/rate-limit.test.js`.

وضعیت پیشین: `POST /api/auth/login` هیچ محدودیتی نداشت. مهاجم می‌توانست هزاران رمز در ثانیه امتحان کند.

راه‌حل:

```
import rateLimit from 'express-rate-limit';

const loginLimiter = rateLimit({
  windowMs: 15 * 60 * 1000,
  max: 5,
  message: { error: 'تلاش‌های ناموفق زیاد بود. ۱۵ دقیقه دیگر امتحان کنید' },
  standardHeaders: true
});

router.post('/login', loginLimiter, async (req, res, next) => { ... });
```

اولویت: بالا.

دو نکته حیاتی که خود محدودیت نرخ را بی‌اثر می‌کنند

۱) `trust proxy` – وگرنه محدودیت به ضد خودش تبدیل می‌شود. `express-rate-limit` روی `req.ip` کلید می‌زند. اگر سرور پشت `nginx` یا کلادفلر باشد، `req.ip` آدرس پراکسی است نه مشتری – یعنی همه بازدیدکنندگان یک سطل مشترک دارند و با ۵ ورود ناموفق یک نفر، کل سایت برای همه قفل می‌شود.

راه‌حل، ولی مشروط:

```
if (process.env.NODE_ENV === 'production' || process.env.TRUST_PROXY === '1') {
  app.set('trust proxy', 1);
}
```

نمی‌شود بی‌قید روشنش گذاشت: جایی که پراکسی نیست، هر کلاینتی می‌تواند `X-Forwarded-For` جعلی بفرستد و با هر درخواست یک آدرس تازه بسازد – و محدودیت نرخ عملاً از کار بیفتد. پس یا تولید، یا پرچم صریح.

۲) انباره درون‌حافظه‌ای. پیش‌فرض `express-rate-limit` شمارنده‌ها را در حافظه همان پروسه نگه می‌دارد. یعنی:

- با هر ری‌استارت سرور، همه شمارنده‌ها صفر می‌شوند.
 - بین چند پروسه به اشتراک گذاشته نمی‌شوند؛ با `pm2 -i` یا چند کانتینر، هر پروسه سقف خودش را دارد و سقف واقعی چند برابر می‌شود.
- تا وقتی یک پروسه اجرا می‌شود مشکلی نیست. برای بیش از یکی، `rate-limit-redis` (یا مشابهش) لازم است.

۲) مسیر عمومی ثبت سفارش بدون محافظ

✓ حل شد. `orderLimiter` روی `POST /api/orders` نشست – ۵ سفارش در ساعت از هر IP، با همان الگوی مورد ۱ و قابل تنظیم از `.env`.

بعداً یک محدودکننده سوم هم به همین خانواده اضافه شد: `trackLimiter` روی `GET /api/orders/track` (مورد ۳۵) با ۲۰ تلاش در ۱۵ دقیقه. خطر آن مسیر نوشتن نیست، حدس زدن است – پس سقفش سخاوتمندتر ولی هنوز بسته است. بخش دوم راه‌حل (تأیید شماره با OTP) اجرا نشد و همچنان مورد ۳۶ است.

وضعیت پیشین: `POST /api/orders` عمداً `requireAdmin` ندارد (مشتری باید بتواند سفارش دهد). سقف ۱۰۰ ردیف در هر سفارش هست، اما هیچ محدودیتی روی تعداد سفارش نبود:

```
if (raw.length > 100) {
  return res.status(400).json({ error: 'تعداد ردیف‌های سبد بیش از حد مجاز است' });
}
```

یک اسکریپت ساده می‌تواند هزاران سفارش جعلی بسازد و پنل مدیریت را غیرقابل استفاده کند.

راه‌حل: `rate limit` بر پایه IP (مثلاً ۵ سفارش در ساعت) به‌علاوه تأیید شماره با کد پیامکی (OTP) – که مشکل بعدی را هم حل می‌کند.

هر دو نکته مورد ۱ (`trust proxy`) و انباره درون‌حافظه‌ای (عیناً اینجا هم صدق می‌کند: بدون `trust proxy` سقف «۵ سفارش در ساعت» روی کل مشتری‌ها اعمال می‌شود، نه روی هر مشتری).

اولویت: بالا.

۳) توکن در localStorage

وضعیت: web/src/lib/api.js :

```
const TOKEN_KEY = 'ghahve.admin.token';  
export const getToken = () => localStorage.getItem(TOKEN_KEY) || '';
```

هر اسکریپت مخربی که در صفحه اجرا شود، توکن را می‌خواند.

راه‌حل: httpOnly cookie با SameSite=Strict و Secure. توکن دیگر برای جاوااسکریپت قابل خواندن نیست. نیازمند افزودن cookie-parser و مدیریت CSRF (که با SameSite=Strict عملاً پوشش داده می‌شود).

اولویت: متوسط (چون سطح حمله XSS در این پروژه کوچک است).

۴) dangerouslySetInnerHTML برای SVG تولیدی

✓ حل شد. تابع esc در web/src/lib/art.js اضافه شد – همان‌جا که رشته SVG ساخته می‌شود، نه در CardArt.jsx که فقط رندرش می‌کند:

```
const ESCAPES = { '<': '&lt;', '>': '&gt;', '&': '&amp;', '"': '&quot;', "'": '&#39;' };  
export const esc = (s) => String(s ?? '').replace(/[<>&"]/g, (c) => ESCAPES[c]);
```

هر سه aria-label (در productArt، gearArt و powderArt) حالا هم نام کالا و هم برچسب دسته را از esc رد می‌کنند. (String(s ?? '')) هم جلوی چاپ شدن undefined داخل تصویر را می‌گیرد.

دو چیز دیگر هم در همان گذر بسته شد:

- zoom تنها عدد کالا بود که مستقیم داخل صفت transform می‌نشست. حالا از numOr رد می‌شود: هر چیزی جز عدد متناهی مثبت، پیش‌فرض ۱ می‌گیرد. (در مدل Number با کران است، ولی این تابع فقط شیء ورودی را می‌بیند نه اسکیمای).
- slug به esc نیاز ندارد و عمداً همان‌طور ماند: replace(/^[a-z0-9]/gi, '') هرچه حرف و رقم نباشد حذف می‌شود، که هم شناسه id معتبر می‌سازد و هم سختگیرانه‌تر از escape است. یک کامنت کنارش این را توضیح می‌دهد تا کسی «اصلاحش» نکند.

جای دیگری در کد همین الگو را نداشت: dangerouslySetInnerHTML فقط یک بار در کل web/src می‌آید (همین CardArt.jsx) و تنها سازنده نشانه‌گذاری با رشته art.js است. سرور اصلاً HTML تولید نمی‌کند. نسخه ایستای میراثی هم که با innerHTML کار می‌کرد دیگر در مخزن نیست (مورد ۱۸)؛ آنجا هم داده خودش را از هیچ ورودی بیرونی نمی‌گرفت – نه fetch، نه localStorage، نه پارامتر URL.

تست: tests/card-art.test.js. تگ باز <svg> را با همان قاعده مرورگر (نام صفت، مساوی، مقدار داخل گیومه) توکن می‌کند و می‌سنجد که نام‌های onload="alert(1)" و "<script>alert(1)</script>" نتوانند صفت تازه‌ای بسازند یا تگ را ببندند. یک تست هم خود سنجه را می‌آزماید: همان تگ را بدون escape می‌سازد و انتظار دارد onload را ببیند – وگرنه تست بی‌دندان می‌شد و برداشتن esc را نمی‌گرفت.

وضعیت: web/src/components/CardArt.jsx :

```
return <span className="art-holder" dangerouslySetInnerHTML={{ __html: svg }} />;
```

SVG را خودمان تولید می‌کنیم، اما `slug` و `name` مدیر داخلی می‌روند:

```
aria-label="دانه‌های ${p.name} با ${GROUPS[p.group]?.label || ""}"
```

اگر مدیری بدخواه نامی مثل `" onload="alert(1)"` بگذارد، تزریق ممکن است.

راه‌حل: یک تابع `escape` کوچک روی هر متنی که وارد SVG می‌شود:

```
const esc = (s) => String(s).replace(/<>"/g, (c) =>
  ({ '<': '&lt;', '>': '&gt;', '&': '&amp;', '"': '&quot;', "'": '&#39;' }[c]));
```

اولویت: متوسط (نیازمند دسترسی مدیر است).

۵) آپلود SVG مجاز است

✓ حل شد. راه‌حل‌های ۱ و ۲ هر دو اجرا شدند – یکی جلوی فایل تازه را می‌گیرد و یکی فایل‌های قبلی را بی‌خطر می‌کند:

- `image/svg+xml`: `server/src/lib/upload.js` از `ALLOWED` برداشته شد و پیام خطا هم دیگر SVG را پیشنهاد نمی‌دهد. فرم پنل (`AdminItemForm.jsx`) به جای `accept="image/*"` فهرست صریح می‌دهد.
- `setUploadHeaders` در همان فایل، و بسته‌شدنش به `express.static` در `server/src/index.js`: هر پاسخ `/uploads` با `X-Content-Type-Options: nosniff`، `Content-Security-Policy: default-src 'none';`، `X-Frame-Options: DENY` و `Cross-Origin-Resource-Policy: same-origin`، `sandbox`، `Content-Disposition: attachment` (با `...html`، `...xml`، `...svgz`، `...svg`) سندی (حتی SVG ای که پیش از این آپلود شده باشد هم در مبدأ فروشگاه اجرا نمی‌شود).
راه‌حل ۳ (`DOMPurify`) لازم نشد: وقتی SVG اصلاً پذیرفته نمی‌شود، چیزی برای پاک‌سازی نمانده است.
هنگام اجرا، پایگاه داده بررسی شد: هیچ کالایی به تصویر SVG آپلودی ارجاع نداشت (فیلد `image` هر ۱۰۶ کالا خالی بود و `server/uploads/` فقط `.gitkeep` داشت). تنها `.svg` موجود در پایگاه داده، `/img/roastery.svg` در محتوای «درباره ما» است که فایل ایستای `web/public/img/` است و ربطی به آپلود ندارد.
تست: `tests/upload.test.js`.

وضعیت: `server/src/lib/upload.js`:

```
const ALLOWED = { ..., 'image/svg+xml': '.svg', ... };
```

SVG می‌تواند `<script>` داشته باشد و چون از `/uploads` روی همان دامنه سرو می‌شود، اسکریپت‌ش در همان مبدأ اجرا می‌شود.

راه‌حل‌ها (به ترتیب سادگی): ۱. حذف `image/svg+xml` از فهرست مجاز. ۲. سرو کردن `/uploads` با هدر `Content-Security-Policy: sandbox` و `Content-Disposition: attachment` برای ۳. SVG. پاک‌سازی SVG با `DOMPurify` هنگام آپلود.

اولویت: متوسط.

۶ CORS باز در نبود CLIENT_ORIGIN

✓ حل شد. تصمیم به `server/src/lib/cors.js` منتقل شد — چون تابعی که کنارش `process.exit` باشد تست‌پذیر نیست:

```
export function resolveCorsOrigin(env = process.env) {
  const origins = parseOrigins(env.CLIENT_ORIGIN);
  if (!origins.length && env.NODE_ENV === 'production') {
    throw new Error('CLIENT_ORIGIN must be set in production');
  }
  return origins.length ? origins : true;
}
```

`index.js` این `throw` را می‌گیرد و به پیام خط فرمان و `exit(1)` تبدیل می‌کند — با یک نمونه آماده `CLIENT_ORIGIN=...` در همان پیام، تا کسی که اولین بار مستقر می‌کند دنبال مستندات نگردد.

در توسعه هنوز باز است تا کلون تازه بدون `.env` هم اجرا شود. `env` پارامتر است، پس تست هر چهار ترکیب را بدون دست‌کاری محیط واقعی می‌سنجد. تست: `tests/cors.test.js`.

وضعیت پیشین: `server/src/index.js`:

```
app.use(cors({ origin: process.env.CLIENT_ORIGIN?.split(',') || true }));
```

اگر متغیر تنظیم نشده بود، `true` یعنی هر مبدأیی مجاز است.

راه‌حل: در تولید صریحاً خطا بده:

```
const origins = process.env.CLIENT_ORIGIN?.split(',');
if (!origins && process.env.NODE_ENV === 'production') {
  console.error('X CLIENT_ORIGIN must be set in production');
  process.exit(1);
}
app.use(cors({ origin: origins || true }));
```

اولویت: بالا در تولید.

✓ **حل شد.** `helmet` نصب شد و **اولین** میان‌افزار است، تا هدرها روی هر پاسخی بنشینند – حتی پاسخ‌های خطا. پیکربندی CSP دقیقاً همان است که این بند پیشنهاد داده، و هر بندش کامنت دارد که چرا لازم است: `'unsafe-inline'` برای `style` (نوار نمودار گزارش)، دامنه‌های Google Fonts (مورد ۲۵)، و `data:` برای تصویر تولیدی و بافت نویز. یک تصمیم فراتر از پیشنهاد هم گرفته شد:

```
strictTransportSecurity: process.env.NODE_ENV === 'production'
```

HSTS فقط وقتی معنا دارد که سایت روی `https` باشد. روشن بودنش روی `http://localhost` دامنه `localhost` را برای **پروژه‌های دیگر** هم به `https` قفل می‌کند – دردسری که پاک کردنش از مرورگر سخت است. `scriptSrc` دست‌نخورده ماند (`'self'` از پیش‌فرض `helmet`): باندل ویت اسکریپت درون‌خطی ندارد.

وضعیت پیشین: هیچ هدر امنیتی تنظیم نمی‌شد – نه CSP، نه HSTS، نه `X-Content-Type-Options`، نه `X-Frame-Options`.

راه‌حلی که آن روز نوشته شد:

```
import helmet from 'helmet';
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ['self'],
      styleSrc: ['self', 'unsafe-inline', 'https://fonts.googleapis.com'],
      fontSrc: ['self', 'https://fonts.gstatic.com'],
      imgSrc: ['self', 'data:']
    }
  }
}));
```

و آنچه امروز واقعاً هست، کوچک‌تر و تقسیم‌شده است. دو تغییر بعدی این بند را عوض کردند:

- **مورد ۲۵** دو بند گوگل‌فونتس را بی‌مصرف کرد: فونت‌ها با `next/font` روی همان دامنه‌اند.
- **مورد ۲۶** این سیاست را از دوش صفحه‌ها برداشت: Express دیگر HTML نمی‌دهد، پس سیاستش فقط برای JSON و XML و تصویر آپلودی است.

```
helmet({
  contentSecurityPolicy: {
    directives: { defaultSrc: ['self'], imgSrc: ['self', 'data:'] }
  },
  strictTransportSecurity: process.env.NODE_ENV === 'production'
})
```

سیاست **صفحه‌ها** به `web/src/proxy.js` رفت و آنجا سخت‌گیرانه‌تر هم شد: به جای `'unsafe-inline'` برای اسکریپت، یک `nonce` تازه در هر درخواست (تصمیم ۲۲ فصل ۹). `'unsafe-inline'` فقط برای `style` باقی ماند، چون صفت `style` با `nonce` کار نمی‌کند – نوار نمودار گزارش و متغیرهای CSS حالت تصویری میکس هر دو درون‌خطی‌اند.

پس امروز سه سیاست جدا داریم و این عمده است: `proxy.js` برای صفحه‌ها، `helmet` برای پاسخ‌های API، و `lib/upload.js` برای `./uploads`.

۸) حمله زمانی روی ورود

وضعیت: `server/src/routes/auth.js`:

```
const ok = admin && (await admin.checkPassword(password));
```

اگر کاربر پیدا نشود، `bcrypt.compare` اجرا نمی‌شود – پس پاسخ سریع‌تر برمی‌گردد. مهاجم با اندازه‌گیری زمان می‌تواند بفهمد کدام نام کاربری وجود دارد، هرچند پیام یکسان است.

راه‌حل: همیشه یک `compare` انجام بده:

```
const DUMMY_HASH = '$2a$12$' + 'x'.repeat(53);
const ok = admin
  ? await admin.checkPassword(password)
  : (await bcrypt.compare(password, DUMMY_HASH), false);
```

اولویت: پایین (حمله دقیق نیست و rate limit عملاً بی‌اثرش می‌کند).

۹) بدون sanitize روی ورودی‌های متنی

وضعیت: فیلدهای `customer.note`، `customer.address`، `recommend`، `taste`، `story` هیچ پاک‌سازی نمی‌شوند. چون React به‌طور پیش‌فرض escape می‌کند، خطر مستقیمی نیست – اما در خروجی CSV یا اگر روزی جایی با `innerHTML` رندر شوند، مشکل‌ساز است.

راه‌حل: حداقل محدودیت طول روی این فیلدها در `schema`، و در صورت نیاز به HTML، استفاده از `sanitize-html` با `whitelist` محدود.

اولویت: پایین.

۱۰) بدون لاگ حسابرسی

وضعیت: وقتی کالایی حذف یا قیمتی عوض می‌شود، هیچ ردی نمی‌ماند. با یک مدیر مشکلی نیست، اما با چند مدیر جدی است.

راه‌حل: یک مدل `AuditLog` با `{ admin, action, target, before, after, at }` و یک میان‌افزار که روی مسیرهای مخرب می‌نشیند.

اولویت: پایین (تا وقتی یک مدیر هست).

۱۱.۲ تست و کیفیت کد

(۱۱) صفر تست

✓ حل شد. Vitest اضافه شد و `tests/` در ریشه نشست — نه در client و نه در server، چون از هر سه workspace می‌خواند. یازده فایل:

فایل	چه چیزی را نگه می‌دارد
<code>pricing.test.js</code>	مرزهای دقیق پله‌ها، ارسال، قیمت میکس
<code>blend.test.js</code>	مجموع ۱۰۰ بعد از هزار حرکت تصادفی
<code>blend-visual.test.js</code>	بودجه زمانی، سطح قیف، هندسه کیسه‌ها
<code>card-art.test.js</code>	escape شدن نام کالا داخل SVG (مورد ۴)
<code>stock.test.js</code>	رزرو و پس‌دادن موجودی (مورد ۳۴)
<code>track.test.js</code>	پیگیری سفارش و نمای عمومی (مورد ۳۵)
<code>upload.test.js</code>	فهرست مجاز و هدرهای <code>/uploads</code> (مورد ۵)
<code>cors.test.js</code>	خطا در تولید بدون <code>CLIENT_ORIGIN</code> (مورد ۶)
<code>rate-limit.test.js</code>	خاموش بودن محدودیت در محیط تست (مورد ۱، ۲)
<code>jalali.test.js</code>	نوروزهای شناخته‌شده، شنبه بودن شروع هفته
<code>taxonomy.test.js</code>	هر کلید shared یک برچسب فارسی دارد

بعداً پنج فایل دیگر در گذر سئو اضافه شدند:

فایل	چه چیزی را نگه می‌دارد
<code>item-routes.test.js</code>	شکل آدرس کالا و بازگشت SPA (مورد ۲۷)
<code>json-ld.test.js</code>	شکل داده‌های ساختاریافته و escape شدنشان (مورد ۲۸)
<code>sitemap.test.js</code>	XML نقشه سایت، robots و تگ‌های head (مورد ۲۹)
<code>seo-parity.test.js</code>	یکی بودن متن سرور و کلاینت، و تزریق <code><head></code> (مورد ۲۹)
<code>item-card.test.jsx</code>	کارت هر نوع کالا لینک صفحه خودش را دارد (مورد ۲۷)

(دو ردیف این جدول با مورد ۲۶ عوض شدند: تزریق سمت سرور حذف شد، پس `seo-parity.test.js` موضوعش را از دست داد و جایش را `next-metadata.test.js` گرفت؛ نیمه «بازگشت SPA» در `item-routes.test.js` هم به `next-routes.test.js` منتقل شد. فهرست کامل امروز در فصل ۳ است.)

و با مورد ۲۶ چهار فایل دیگر:

فایل	چه چیزی را نگه می‌دارد
<code>next-routes.test.js</code>	هر نوع کالا پوشه مسیر خودش را دارد – در هر دو جهت
<code>next-metadata.test.js</code>	هر تگی که <code>headTags</code> می‌سازد به شیء <code>metadata</code> نکست می‌رسد
<code>cart-storage.test.js</code>	بازاعتبارسنجی سبد ذخیره‌شده، با حافظه تزریق‌شده
<code>cart-hydration.test.jsx</code>	سبد از چرخه «رندر سرور ← hydrate» دست‌نخورده بیرون می‌آید

و سه فایل تازه‌تر، در گذر هم‌رسانی و چاپ:

فایل	چه چیزی را نگه می‌دارد
<code>share.test.js</code>	نشانی فرستاده‌شده همان <code>canonical</code> صفحه است، و دسکتاپ به کپی می‌رود نه به برگه سیستم
<code>toast-store.test.js</code>	عمر پیام، شمارش تازه برای پیام دوم، و خالی بودن <code>snapshot</code> سرور
<code>item-detail.test.jsx</code>	مودال کالا بدون <code>ShopProvider</code> رندر می‌شود

جمع امروز: بیست‌ودو فایل و ۳۸۰ سنجه.

سه استثنای قاعده «فقط تابع خالص» – `item-card.test.jsx`، `cart-hydration.test.jsx` و `item-detail.test.jsx` – و عمده هم فقط همین سه می‌مانند. هر کدام چیزی را نگه می‌دارد که هیچ تابع خالصی نمی‌تواند: یکی اینکه کارت کالا لینک صفحه خودش را رندر کند، دومی اینکه سبد ذخیره‌شده بی‌صدا پاک نشود، و سومی اینکه مودال کالا – بیرون از هر `Provider` ای – همچنان باز شود. با `// @vitest-environment jsdom` بالای خودشان محیط را عوض می‌کنند تا نوزده فایل دیگر بی‌جهت داخل `jsdom` اجرا نشوند، و `ShopContext` را با `vi.mock` بدل می‌گیرند تا به شبکه دست نزنند.

دو فایل دیگر همان گذر – `share.test.js` و `toast-store.test.js` – عمده استثنا **نشدند**: `share.js` وابستگی‌هایش (`window`، `document`، `navigator`) را پارامتر می‌گیرد و `toastStore.js` یک انبار ساده است، پس هر دو در محیط `node` اجرا می‌شوند.

دلیل وجودش یک شکست واقعی است: در گذر اول، لینک صفحه کالا پشت شرط `hasStory` بود و هر کالای بدون معرفی – یعنی همه ابزارها و پودرها – هیچ لینکی به صفحه خودش نداشت. مسیرها کار می‌کردند، نقشه سایت درست بود، هر ۲۸۳ تست سبز بودند، و کالاها از داخل سایت دست‌نیافتنی. هیچ تابع خالصی این را نمی‌گرفت.

هیچ تستی به پایگاه داده دست نمی‌زند. جایی که کد به مونگو نیاز دارد، مدل تزریق می‌شود (`reserveStock(lines, ItemModel)`) و تست یک بدل چند خطی می‌سازد. پس CI هم به سرویس `MongoDB` نیاز ندارد.

در `vitest.config.mjs` ثابت شده، چون گزارش‌ها روی وقت تهران بسته می‌شوند و تست باید مستقل از ساعت ماشین اجراکننده باشد.

که پایین پیشنهاد شده بود **لازم نشد**: با یکی شدن فایل در `shared/` (مورد ۱۲) دیگر دو نسخه‌ای نمانده که واگرا شوند.

وضعیت پیشین: هیچ فایل تستی در مخزن نبود. این جدی‌ترین ضعف مهندسی پروژه بود، چون منطق‌های حساس تست‌پذیرند اما تست نشده بودند.

راه‌حل: با `Vitest` شروع کنید. اولویت‌ها:

```
// tests/pricing.test.js
describe('computeTotals', () => {
  it('مرز ۹۹۹ گرم: بدون تخفیف و با هزینه ارسال', () => {
    const t = computeTotals([{ kind: 'coffee', grams: 999, item: { price: 1000000 } }], () => null);
    expect(t.discount).toBe(0);
    expect(t.shipping).toBe(65000);
  });

  it('مرز ۱۰۰۰ گرم: ۵٪ تخفیف و ارسال رایگان', () => {
    const t = computeTotals([{ kind: 'coffee', grams: 1000, item: { price: 1000000 } }], () => null);
    expect(t.discountLabel).toBe('۵%');
    expect(t.shipping).toBe(0);
  });

  it('تخفیف روی ابزار اعمال نمی‌شود', () => {
    const t = computeTotals([
      { kind: 'coffee', grams: 5000, item: { price: 1000000 } },
      { kind: 'gear', qty: 1, item: { price: 10000000 } }
    ], () => null);
    expect(t.discount).toBe(750000); // ۱۵٪ فقط روی ۵ میلیون
  });
});

// tests/blend.test.js
describe('applyPercent', () => {
  it('مجموع همیشه ۱۰۰ می‌ماند', () => {
    let mix = [{ slug: 'a', percent: 50 }, { slug: 'b', percent: 30 }, { slug: 'c', percent: 20 }];
    for (const v of [0, 15, 33, 67, 100, 42]) {
      mix = applyPercent(mix, 'a', v);
      expect(sumOf(mix)).toBe(100);
    }
  });
});

// tests/jalali.test.js
describe('bucketOf', () => {
  it('هفته از شنبه شروع می‌شود', () => { ... });
  it('اول فروردین در سطل درست می‌افتد', () => { ... });
});

tests/pricing-parity.test.js // ← مهم‌ترین
describe('همگامی کلاینت و سرور', () => {
  it('هر دو نسخه نتیجه یکسان می‌دهند', () => {
    // برای مجموعه‌ای از ورودی‌های تصادفی، خروجی هر دو فایل را مقایسه کن
  });
});
```

آن تست آخر مستقیماً خطر واگرایی `pricing.js` را می‌بندد.

اولویت: بالا.

✓ حل شد. «راه‌حل کامل» اجرا شد، نه «راه‌حل فوری». هر دو کپی حذف شدند و یک بسته خصوصی به نام `@ghahve/shared` جای‌شان را گرفت:

```
ریشه package.json //
{ "workspaces": ["shared", "client", "server"] }
```

```
// هر دو طرف، با همین یک نام
import { computeTotals } from '@ghahve/shared/pricing.js';
```

نتیجه: یک `npm install` در ریشه، یک `package-lock.json` و **صفر** امکان واگرایی. بدون مرحله `build` و بدون `symlink` دستی — `exports` در `shared/package.json` فقط همان دو فایل را بیرون می‌دهد.

`shared/taxonomy.js` هم در همان گذر منتقل شد (مورد ۱۳).

کامنت بالای `shared/pricing.js` عمداً یادآوری می‌کند که این جای بازمحاسبه سمت سرور را **نمی‌گیرد**: سرور همچنان به عددِ ارسالی از مرورگر اعتماد نمی‌کند؛ یکی بودنِ فرمول فقط تضمین می‌کند نتیجه با آنچه کاربر دیده یکی دربیاید.

وضعیت پیشین: دو فایل یکسان که باید دستی همگام می‌ماندند:

```
web/src/lib/pricing.js    (خط ۱۰۱)
server/src/lib/pricing.js (خط ۱۰۲)
```

اگر کسی پله‌های تخفیف را در یکی عوض می‌کرد، عددی که مشتری می‌بیند با عددی که ثبت می‌شود فرق می‌کرد — **بدون هیچ خطایی**.

راه‌حل کامل: `npm workspace` با پوشه `shared/`:

```
ریشه package.json //
{ "workspaces": ["shared", "client", "server"] }
```

```
// web/src/context/ShopContext.jsx
import { computeTotals } from '@ghahve/shared/pricing.js';
```

راه‌حل فوری: تست همگامی (مورد ۱۱).

اولویت: بالا.

✓ حل شد – ولی نه با ساختار پیشنهادی پایین. taxonomy.js به shared/ منتقل شد، و کلید و برچسب عمداً جدا ماندند:

- shared/taxonomy.js فقط کلید دارد. سرور با متن فارسی کاری ندارد و بردن برچسب‌ها به shared فقط بار اضافه بود.
- web/src/lib/groups.js برچسب فارسی هر کلید را کنارش می‌گذارد.

ساختار { steel: { fa: 'استیل' } } که این بند پیشنهاد داده بود، متن رابط کاربری را داخل بسته‌ای می‌برد که سرور هم import می‌کند – یعنی مسئولیت‌ها را قاطی می‌کرد. به‌جایش، همگامی با تست تضمین شد:

```
// tests/taxonomy.test.js
it('طرح‌های ابزار', () => sameKeys(GEAR_SHAPES, GEAR_SHAPE_KEYS));
```

sameKeys نه یکی کم را می‌بخشد و نه یکی زیاد، پس کلید بی‌برچسب و برچسب بی‌کلید هر دو CI را قرمز می‌کنند. یک دسته تست هم با toBe (نه toEqual) می‌سنجد که ترتیب‌های ال همان آرایه shared باشند، نه کپی برابر آن – چون کپی امروز برابر است و فردا واگرا می‌شود.

قاعده کار حالا روشن است: کلید تازه اول در shared، بعد برچسبش در groups.js.

وضعیت پیشین: کلیدهای POWDER_TONES, POWDER_SHAPES, GEAR_MATERIALS, GEAR_SHAPES, KINDS و TASTE_KEYS و DEFAULT_GRINDS در هر دو تکرار شده بودند، با یک کامنت هشدار در groups.js و هیچ تضمین خودکاری.

۱۴) بدون ESLint و Prettier

✓ حل شد. eslint.config.mjs (flat config) با سه ناحیه – سرور (globals نود)، کلاینت (globals مرورگر + JSX با eslint-plugin-react و react-hooks)، و تست‌ها و فایل‌های پیگیری – به‌علاوه eslint-config-prettier در انتهای صف تا قاعده‌های سلیقه‌ای قالب‌بندی با Prettier دعوا نکنند. prettierignore و prettierignore. هم اضافه شدند و هر دو گام lint و format:check در CI اجرا می‌شوند.

پنج دسته در prettierignore کنار گذاشته شده‌اند و هر پنج کامنت دارند: نسخه ایستای ریشه (مورد ۱۸)، دو فایل seed (هر ردیف عمداً یک خط بلند است)، خروجی ساخته‌شده docs/documentation.html، چهار فایل ستون‌تراز scripts/docs/ و همه *.md – چون تنها کار Prettier با متن فارسی هم‌تراز کردن ستون‌های جدول است و با پهنای حروف فارسی نتیجه به‌هم‌ریخته‌تر می‌شود. جاهایی که ستون‌بندی دستی معنا دارد، prettier-ignore گذاشته شده.

یک git-blame-ignore-revs هم آماده شد تا قالب‌بندی یک‌باره سراسری git blame را خراب نکند (SHA کامیت باید هنگام اعمال داخلش نوشته شود).

دو چیز عمداً باز ماند:

- react-hooks/set-state-in-effect روی warn است، نه error. قاعده تازه افزونه الگوی «با mount داده را بگیر و در state بگذار» را در ۱۴ جا خطا می‌داند – همان الگویی که کل بارگذاری داده پروژه رویش سوار است. درست کردنش یعنی بازنویسی لایه داده؛ پس هشدار می‌ماند تا دیده شود ولی جلوی CI را نگیرد. این همان مورد ۴۲ است.
- react/prop-types خاموش است، چون پروژه نه TypeScript دارد و نه prop-types: شکل داده در schema مونگوس و کامنت‌هاست (مورد ۱۵).

وضعیت پیشین: هیچ فایل پیکربندی در مخزن نبود، فقط چند `// eslint-disable-next-line react-hooks/exhaustive-` پراکنده که نشان می‌داد زمانی ESLint اجرا می‌شده.

باقی‌مانده: بررسی همان `exhaustive-deps` های خاموش‌شده – بعضی‌شان ممکن است باگ واقعی باشند. مورد ۴۲ یکی از همین‌هاست.

۱۵) بدون TypeScript

وضعیت: کل پروژه JavaScript ساده است. شکل داده فقط در schema Mongoose و در ذهن برنامه‌نویس وجود دارد.

راه‌حل: حتی بدون مهاجرت کامل، می‌شود با JSDoc نوع‌دار شروع کرد:

```
/**
 * @typedef {Object} CartLine
 * @property {string} key
 * @property {string} slug
 * @property {'coffee'|'gear'|'powder'} kind
 * @property {number} grams
 * @property {number} qty
 * @property {string} grind
 * @property {{slug: string, percent: number}[]} mix
 */
```

با `// @ts-check` بالای فایل، VS Code همان الان بررسی می‌کند.

اولویت: پایین (بهبود تدریجی).

۱۶) فیلدهای مرده در مدل

وضعیت: سه فیلد در مدل هستند اما استفاده نمی‌شوند:

فیلد	جا	چرا مرده
<code>components[].min</code> / <code>.max</code>	<code>Item.js</code>	فرم همیشه <code>0</code> / <code>100</code> می‌فرستد و اهرم‌ها همیشه <code>۱۰۰۰۰</code> حرکت می‌کنند
<code>components[].locked</code>	<code>Item.js</code>	فرم همیشه <code>false</code> می‌فرستد
<code>pool[]</code>	<code>Item.js</code>	<code>BlendsSection</code> از همه قهوه‌ها استفاده می‌کند، نه از <code>pool</code>

کامنت `AdminItemForm.jsx` نشان می‌دهد این عمدی است:

ترکیب ما فقط پیشنهاد است – بازه مجاز و قفل نداریم، چون مشتری در انتخابش کاملاً آزاد است.

یعنی سیاست تجاری عوض شده اما فیلدها مانده‌اند.

راه‌حل: یا حذفشان (با یک اسکریپت مهاجرت که از اسناد موجود پاکشان کند)، یا زنده کردنشان در ال. حالت فعلی گمراه‌کننده است.

اولویت: پایین.

۱۷) کد مرده در CSS

وضعیت: `web/src/style.css`:

```
#grid,#gearGrid,#powderGrid{ display:grid; gap:clamp(34px,5vw,56px); }
.roast-curve #curveLine{ stroke-dasharray:1400; ... animation:draw ... }
```

هیچ‌کدام از این `id` ها در نسخه React وجود ندارند — بازمانده نسخه ایستای اولیه. همچنین `.mix-row.is-locked` و `.mix-lock` که به فیلد مرده `locked` مربوط‌اند.

راه‌حل: حذف، یا اجرای یک ابزار پوشش CSS.

اولویت: پایین.

۱۸) نسخه ایستای اولیه در مخزن

🔗 بیشترش رفت، یک تکه ماند.

سه فایل اصلی نسخه ایستا — `index.html`، `script.js` و `style.css` — دیگر در مخزن نیستند. آنچه مانده پوشه `img/` در ریشه است: همان ۱۱ فایل SVG، که نسخه زنده‌شان در `web/public/img/` است و همان یکی سرو می‌شود. پس پوشه ریشه امروز یک کپی بی‌مصرف است، نه بخشی از سایت.

دو جا هنوز طوری تنظیم شده‌اند که انگار هر چهار تا سر جایشان‌اند: `.prettierignore` (`/script.js`، `/index.html`، `/style.css`، `/img`) و بلوک `ignores` در `eslint.config.mjs`. این سطرها بی‌ضررند — الگویی که به فایلی نمی‌خورد کاری نمی‌کند — ولی همان گیجی «این‌ها چیستند؟» را نگه می‌دارند.

وضعیت پیشین: `index.html` (۲۷ KB)، `script.js` (۱۲۱ KB)، `style.css` (۲۹ KB) و پوشه `img/` در ریشه — که اجرا نمی‌شدند.

راه‌حل: انتقال به شاخه `legacy` یا پوشه `docs/legacy/` با فایل README ای که توضیح دهد چیستند. حالت فعلی برای هر تازه‌واردی گیج‌کننده است.

آنچه مانده: برداشتن `img/` ریشه (یا آگاهانه نگه‌داشتنش با یک `README`)، و پاک کردن سه سطر مرده از `.prettierignore` و `eslint.config.mjs`.

اولویت: پایین.

۱۹) شماره سفارش بدون تلاش مجدد

وضعیت: `server/src/models/Order.js`:

```
const stamp = Date.now().toString(36).slice(-4).toUpperCase();
const rand = Math.floor(Math.random() * 9000 + 1000);
this.code = `${stamp}-${rand}`;
```

اگر برخورد رخ دهد، خطای `11000` به کاربر می‌رسد و سفارش از دست می‌رود.


```
async function createOrderWithCode(data, tries = 5) {
  for (let i = 0; i < tries; i++) {
    try {
      return await Order.create(data);
    } catch (err) {
      if (err.code === 11000 && err.keyPattern?.code && i < tries - 1) continue;
      throw err;
    }
  }
}
```

اولویت: متوسط.

۴۲) الگوی بارگذاری داده با `useState` داخل `useEffect`

شماره ۴۲ است چون بعد از نوشتن این سند پیدا شد؛ شماره‌های ۱ تا ۴۱ عمداً دست‌نخورده ماندند تا ارجاع‌های بیرونی نشکنند.

❶ تصمیم‌گیری گرفته شد و مستند است، ولی الگو هنوز هست.

با مورد ۲۶ بخش بزرگی از این هشدارها موضوعشان را از دست دادند: `ShopContext` دیگر موقع mount داده نمی‌گیرد، چون کالاهای و متن‌ها روی سرور خوانده و به شکل پارامتر تحویل می‌شوند.

ولی جای خالی‌شان را دو دسته تازه پر کرد که هر دو درست‌اند و نمی‌شود حذفشان کرد:

- نگهدارنده‌های hydration در `ShopContext` (خواندن سبک از `localStorage`، پاک‌سازی، و کامل کردن فهرست) — شرحشان در ۷.۱۲؛

- `loadOnMount` در پنل، که صفحه‌هایش سروری نیستند.

پس تصمیم این شد که قاعده روی `warn` بماند، با کامنتی در `eslint.config.mjs` که همین دو استثنا را نام می‌برد. یعنی مورد تازه دیده می‌شود، ولی CI قرمز نمی‌شود.

وضعیت: با راه‌اندازی ESLint (مورد ۱۴)، قاعده `react-hooks/set-state-in-effect` در نسخه ۷ افزونه، تقریباً هر جایی را که پروژه داده می‌گرفت خطا گرفت:

```
const load = useCallback(async () => { ... setItems(data); }, [...]);
useEffect(() => { load(); }, [load]);
```

امروز ۱۷ هشدار در ۱۱ فایل باقی است:

فایل	تعداد	چرا هست
ShopContext.jsx	۵	خواندن سید، نوشتن سید، آسیاب پیش فرض، کامل کردن فهرست، پاک سازی – همه در ۷.۱۲ توضیح داده شده‌اند
AdminReports.jsx	۲	خلاصهٔ بازه‌ای و فهرست صفحه‌بندی‌شده
AdminContent.jsx	۲	فرم ساخته‌شده از schema
AdminClub , AdminOrders , AdminItems , AdminItemForm	هر کدام ۱	صفحه‌های پنل، که سروری نیستند
ItemCard.jsx , Hero.jsx , ContentSections.jsx	هر کدام ۱	حالت محلی همگام‌شونده با prop
PrintSheet.jsx	۱	setReady(true) بعد از mount – نگهداری hydration، چون portal روی سرور document ندارد

استدلال قاعده این است که `setState` همگام داخل افکت، رندر آبخاری می‌سازد. در این اندازه محسوس نیست، ولی الگو با جهت‌گیری React جدید (و React Compiler) جور نیست.

تصمیم فعلی: قاعده روی `warn` است، نه `error`. کامنت `eslint.config.mjs` دلیلش را می‌گوید:

این قاعده الگوی «با mount داده را بگیر و در state بگذار» را خطا می‌داند. با آمدن رندر سمت سرور، داده دیگر این‌طور نمی‌آید – ولی چند جا هنوز لازم است و درست است: خواندن سید از `localStorage`، و فهرست کالاها در پنل مدیریت که صفحه‌اش سروری نیست.

راه‌حل واقعی: بازنویسی لایهٔ بارگذاری داده در پنل – یا با یک هوک `useAsync` مشترک، یا با کتابخانه‌ای مثل TanStack Query که `loading/error/data` را خودش می‌دهد. این کار به مورد ۲۳ (`useMemo` نداشتن `value` در Provider) هم گره خورده است.

نگهبان‌های `ShopContext` اما با هیچ‌کدام از این‌ها **حل نمی‌شوند**: آن‌ها واقعاً همگام‌سازی با یک سیستم بیرونی‌اند (`localStorage`)، یعنی دقیقاً همان کاری که افکت برایش ساخته شده. هشدارشان مثبت کاذب است.

اولویت: پایین (بدهی معماری در پنل، نه باگ).

۱۱.۳ کارایی

۲۰ بدون کد اسپلیت

🕒 نیمه‌اش خودبه‌خود حل شد، نیمهٔ دیگرش نه.

با مهاجرت به Next (مورد ۲۶)، **کد جاوااسکریپت** خودبه‌خود بر اساس مسیر تقسیم می‌شود: بازدیدکنندهٔ فروشگاه دیگر کد هشت صفحهٔ پنل را دانلود نمی‌کند، چون `app/admin/**` باندل خودش را دارد. آن `lazy` و `Suspense` دستی راه‌حل پایین لازم نشد.

ولی CSS همچنان یکجاست: هر دو فایل در `app/layout.jsx` وارد می‌شوند، پس مشتری استایل پنل را هم می‌گیرد. برای همین این مورد بسته نشده.

وضعیت: `web/src/app/layout.jsx`

```
import '../style.css';
import '../admin.css';
```

یعنی بازدیدکننده فروشگاه، استایل پنل را هم دانلود می‌کند. (پیش‌تر، در نسخه ویت، کد صفحه‌های پنل هم به همین شکل ایستا import می‌شد؛ آن نیمه با تقسیم خودکار Next حل شد.)

راه‌حل:

```
import { lazy, Suspense } from 'react';
const AdminLayout = lazy(() => import('./pages/admin/AdminLayout.jsx'));
const AdminReports = lazy(() => import('./pages/admin/AdminReports.jsx'));
...

<Route path="/admin" element={
  <Suspense fallback={<div className="admin-boot">در حال بارگذاری</div>}>
    <AdminLayout />
  </Suspense>
} />
```

و `admin.css` را داخل `AdminLayout.jsx` import کنید تا Vite خودش جدایش کند.

تأثیر تخمینی: کاهش قابل توجه باندل اولیه فروشگاه.

اولویت: بالا.

۲۱) `GET /api/items` بدون صفحه‌بندی

وضعیت:

```
const items = await Item.find(filter).sort({ rank: 1, name: 1 }).lean({ virtuals: true });
```

همه ۱۰۶ کالا در یک پاسخ. با ۱۰۰۰ کالا این چند صد کیلوبایت می‌شود.

راه‌حل: صفحه‌بندی یا بارگذاری تنبل بر اساس `kind`. اما توجه کنید که معماری فعلی به فهرست کامل نیاز دارد: `bySlug` برای قیمت میکس، و `houseBlends` برای بخش میکس‌ها. پس این تغییر ساده نیست.

اولویت: پایین (تا وقتی کاتالوگ کوچک است).

۲۲) بدون ایندکس روی `Order.createdAt`

✓ حل شد. هر دو ایندکس پیشنهادی دقیقاً همان‌طور به `server/src/models/Order.js` اضافه شدند:

```
orderSchema.index({ createdAt: -1 });
orderSchema.index({ status: 1, createdAt: -1 });
```

اولی برای فهرست و بازه گزارش، دومی برای وقتی که فیلتر وضعیت هم فعال است — ترتیب فیلدها در ایندکس مرکب مهم است: `status` که با تساوی فیلتر می‌شود جلو، `createdAt` که بازه و مرتب‌سازی است بعد.

وضعیت پیشین: مدل `Order` فقط روی `code` و `status` ایندکس داشت. اما گزارش‌ها مدام روی `createdAt` فیلتر و مرتب می‌کنند:

```
Order.find({ createdAt: { $gte: windowStart } }, ...)
Order.find(filter).sort({ createdAt: -1 }).skip(...).limit(...)
```

با هزاران سفارش، این‌ها به `collection scan` تبدیل می‌شوند.

راه‌حل:

```
orderSchema.index({ createdAt: -1 });
orderSchema.index({ status: 1, createdAt: -1 });
```

اولویت: متوسط (بالا با رشد داده).

۲۳) شیء `value` در `Provider` بدون `useMemo`

وضعیت: `web/src/context/ShopContext.jsx`:

```
const value = {
  items, byKind, bySlug, lookup, houseBlends, content,
  loading, loadError, reload,
  cart, resolved, totals, cartSummary, nextTier,
  addWeighed, addPiece, ..., toast, toastMsg, priceFor
};

return <ShopContext.Provider value={value}>{children}</ShopContext.Provider>;
```

در هر رندر `ShopProvider` یک شیء تازه ساخته می‌شود، پس همهٔ مصرف‌کنندگان `rerender` می‌شوند – حتی اگر هیچ داده‌ای عوض نشده باشد.

راه‌حل:

```
const value = useMemo(() => ({ ... }), [
  items, byKind, bySlug, lookup, houseBlends, content,
  loading, loadError, reload, cart, resolved, totals,
  cartSummary, nextTier, addWeighed, ..., toastMsg
]);
```

یا بهتر: تقسیم به دو `context` – یکی برای داده (کم تغییر) و یکی برای کنش‌ها (ثابت).

اولویت: پایین (با اندازهٔ فعلی محسوس نیست).

۲۴) بدون فشرده‌سازی و بدون بهینه‌سازی تصویر

✓ حل شد – هر دو نیمه.

`compression` به Express اضافه شد، پیش از helmet و مسیره‌ها، تا خروجی‌های استریمی CSV و JSON گزارش‌ها هم از آن رد شوند. اثرش روی متن فارسی بزرگ است، چون هر حرف در UTF-8 سه بایت می‌گیرد:

مسیر	خام	با gzip	صرفه
/api/items	۵۲,۹۹۶	۱۱,۷۰۰	۷۸٪
/api/content	۱۱,۵۵۵	۴,۱۰۲	۶۵٪
/api/stats/top-sellers	۳,۸۰۰	۱,۷۲۹	۵۵٪

دو پیش‌فرض خود کتابخانه دست‌نخورده ماند: پاسخ‌های کوچک‌تر از یک کیلوبایت فشرده نمی‌شوند، و `Cache-Control: no-transform` کنار گذاشته می‌شود.

— نیمه دوم: بهینه‌سازی تصویر —

راه‌حل پیشنهادی پایین `sharp` در خود مسیر آپلود بود. آن‌طور انجام نشد، و دلیلش این است که کلاینت دیگر ویت نیست: **Next خودش بهینه‌ساز تصویر دارد** و پشتش هم همان `sharp` است. با `next/image` یک نسخه در لحظه آپلود ساخته نمی‌شود؛ هر اندازه‌ای که واقعاً خواسته شود ساخته و روی دیسک کش می‌شود، و قالبش را از هدر `Accept` همان مرورگر می‌گیرد.

سه چیز عوض شد:

- `web/src/components/CardArt.jsx` – تنها جای رندر عکس کالا. عکس آپلودی از `next/image` رد می‌شود و طرح تولیدی SVG دست‌نخورده ماند. هر قاب سایت جایگاه خودش را نام می‌برد (`card`، `sheet`، `blend`، `thumb`) تا `sizes` درست به مرورگر برسد؛ یک کلاس در پنج قاب با پنج اندازه متفاوت استفاده می‌شود و بدون این کار مرورگر نمی‌داند کدام نسخه را بردارد.
- `web/src/lib/img.js` – قاعده «کدام تصویر بهینه می‌شود». برداری (SVG) و آدرس بیرونی نه؛ بقیه بله.
- `web/next.config.mjs` – `formats` با AVIF و WebP، `localPatterns` روی `/uploads` و `/img` (بی‌آن، `/_next/image` یک واکنشی عمومی می‌شود)، و ۴۴۸ در `imageSizes` چون کارت فهرست روی دسکتاپ ۳۸۵ پیکسل است و با فهرست پیش‌فرض نزدیک‌ترین نسخه ۶۴۰ می‌شد.

اندازه‌گیری روی صفحه فهرست، با ۳۸ کالای روشن که همه‌شان عکس آپلودی داشتند (عکس آزمایشی ۲۰۱۶ در ۱۵۱۲ پیکسل، میانگین ۴۵۲ کیلوبایت – اندازه معمول عکس موبایل):

خام	با next/image	صرفه
۹۱۴,۵۷۲	۴۷,۴۴۰	۹۴.۸٪
۱۷,۵۸۰,۷۶۳	۶۰۳,۷۶۰	۹۶.۶٪
۴۷۹,۹۱۹	۱۶,۰۴۸ (AVIF)	۹۶.۷٪

بیشتر این صرفه از **اندازه** می‌آید نه از قالب: همان عکس در ۴۴۸ پیکسل و به شکل JPEG هم ۲۵,۳۶۴ بایت می‌شود. یعنی مرورگری که AVIF نمی‌فهمد هم ۹۴.۷٪ کمتر می‌گیرد.

— هدرها —

با این تغییر مرورگر دیگر مستقیم `/uploads/...` را نمی‌خواند، پس هدرهای `setUploadHeaders` روی پاسخی که واقعاً می‌رسد نیستند. آن هدرها سر جای خودشان ماندند (هر کس آدرس را مستقیم باز کند همان‌ها را می‌گیرد) و برای مسیر تازه معادلشان ساخته شد: Next خودش `Content-Security-Policy: script-src 'none'; frame-src 'none'; sandbox` و `Content-Disposition: attachment` می‌گذارد، و `nosniff` و `Cross-Origin-Resource-Policy` و `X-Frame-Options` در `headers()` اضافه شدند. میان‌افزار `proxy.js` عمداً `_next/image` را رد می‌کند، پس این کار از آنجا برنمی‌آمد.

`dangerouslyAllowSVG` خاموش ماند (مورد ۵)، و `lib/img.js` مطمئن می‌شود هیچ SVG ای اصلاً به بهینه‌ساز نرسد – وگرنه پاسخ ۴۰۰ می‌شد و قاب خالی می‌ماند. CSP صفحه‌ها دست نخورد: `/_next/image` هم‌مبدأ است و `img-src 'self'` پوشش می‌دهدش.

وضعیت:

- بدون میان‌افزار `compression` — پاسخ‌های JSON فشرده نمی‌شوند: (حل شد)
- تصویر آپلودی تا ۴ MB خام سرو می‌شود، بدون تولید نسخهٔ کوچک: (حل شد)

راه‌حلی که اجرا شد:

```
// server/src/index.js
app.use(compression());
```

```
// web/src/components/CardArt.jsx
<Image src={item.image} alt={item.name} width={box.w} height={box.h} sizes={box.sizes} />
```

آنچه هنوز باز است — و چرا باز است. فایل خام تا ۴ مگابایت روی دیسک می‌ماند؛ بهینه‌سازی در زمان سرو انجام می‌شود، نه در زمان آپلود.

این یک باقی‌ماندهٔ جاافتاده نیست، یک انتخاب است و در تصمیم ۲۹ فصل ۹ شرح داده شده: نگه‌داشتن نسخهٔ اصلی یعنی اگر فردا قابِ کارت عوض شد، همه‌چیز از نو ساخته می‌شود بی‌آنکه کسی مجبور باشد عکس‌ها را دوباره آپلود کند. کوچک‌سازی در زمان آپلود تصمیمی برگشت‌ناپذیر بود.

بهایش فضای دیسک است و بس. اگر روزی تنگ شد، فشرده‌کردن در خودِ `upload.js` گام بعدی است — کنارِ بهینه‌سازِ زمانِ سرو، نه به‌جایش.

پس این مورد به‌طور کامل بسته است: هر دو نیمه‌اش — فشرده‌سازی پاسخ‌ها و بهینه‌سازی تصویر — اجرا شده‌اند، هر دو اندازه‌گیری شده‌اند، و آنچه مانده یک قید طراحی است نه یک کار انجام‌نشده.

✓ حل شد – ولی نه با `font-face@` دستی.

با مهاجرت به Next (مورد ۲۶)، فونت‌ها از `next/font/google` می‌آیند: فایل فونت **موقع build** گرفته و کنار بقیه دارایی‌ها گذاشته می‌شود، پس در زمان اجرا هیچ درخواستی به دامنه بیرونی نمی‌رود. آن `<link>` به `fonts.googleapis.com` حذف شد و در سیاست امنیتی صفحه‌ها `font-src` روی `'self'` بسته شد.

چرا راه‌حل پیشنهادی پایین اجرا نشد: `next/font` همان کار را می‌کند و دو چیز اضافه هم می‌دهد – فونت‌جانشین با متریک تنظیم‌شده (تا موقع بارگذاری صفحه نپرد) و `preload` خودکار. وزیرمن هم روی گوگل‌فونتس **متغیر** است، پس همان یک فایل همه وزن‌های ۳۰۰ تا ۹۰۰ را می‌دهد و «حذف وزن‌های بی‌استفاده» موضوعیت ندارد.

اندازه‌گیری پس از تغییر: عرض و ارتفاع متن مو به مو همان است که با فونت گوگل بود (h2 در ۳۱×۵۹۹ پیکسل، بندر متن در ۹۱×۴۸۲).

یک بهای کوچک: `npm run build` از این به بعد به شبکه نیاز دارد.

وضعیت (متن اصلی): `client/index.html`:

```
<link href="https://fonts.googleapis.com/css2?family=Lalezar&family=Vazirmatn:wght@300;400;500;700;900&display=swap" rel="stylesheet" />
```

پنج وزن Vazirmatn بارگذاری می‌شود – حتی وزن‌هایی که کم استفاده می‌شوند. به‌علاوه یک وابستگی به شبکه بیرونی که در ایران ممکن است کند یا مسدود باشد.

راه‌حل: میزبانی محلی فونت‌ها با زیرمجموعه فارسی و `font-display: swap`:

```
@font-face {
  font-family: 'Vazirmatn';
  src: url('/fonts/Vazirmatn-Regular.woff2') format('woff2');
  font-weight: 400;
  font-display: swap;
  unicode-range: U+0600-06FF, U+200C-200E, U+0020-007F;
}
```

و حذف وزن‌های بی‌استفاده.

اولویت: بالا (تأثیر مستقیم روی کاربر ایرانی).

✓ حل شد – و این موردی است که یک بار رد شد و بعد پذیرفته.

این تنها مورد این سند است که دو تصمیم پشت سرش دارد، و هر دو در متن می‌مانند. پاک کردن استدلال اول یعنی جا انداختن چیزی که بعداً معلوم شد کدام بخشش درست بود و کدام بخشش نه.

— تصمیم اول: انجام نشود —

استدلالش این بود:

- هر دو راه حل پیشنهادی پایین معماری را عوض می‌کنند، نه یک فایل را. مهاجرت به Next.js یعنی بازنویسی مسیریابی، لایه داده ShopContext، و کل پنل مدیریت؛ prerender در زمان build هم یعنی فهرست کالاها باید موقع ساخت معلوم باشد، در حالی که مدیر هر روز کالا اضافه و خاموش می‌کند و هر تغییر یک build تازه می‌خواست.
- سه مورد دیگر همین فصل (۲۷، ۲۸، ۲۹) بیشتر سود سئو را بدون این تغییر می‌دهند: هر کالا آدرس خودش را دارد، داده‌های ساختاریافته سر جایشان‌اند، و نقشه سایت زنده است. گوگل جاوااسکریپت را اجرا می‌کند، پس این‌ها را می‌بیند – با تأخیر، ولی می‌بیند.
- هزینه‌اش با اندازه فعلی کسب‌وکار جور نبود.

و راه میانی‌اش این بود: Express برای مسیرهای `/coffee/:slug` همان `index.html` را می‌خواند و پیش از فرستادن، تگ‌های `<head>` مخصوص همان کالا را داخلش می‌گذاشت – چون خزنده پیش‌نمایش تلگرام و واتساپ جاوااسکریپت اجرا نمی‌کند. بدنه صفحه دست‌نخورده و سمت کلاینت می‌ماند.

— چه چیزی آن تصمیم را برگرداند —

نه فشار بیرونی، بلکه سه چیزی که خود همان راه میانی نشان داد:

۱) آنچه از دست رفته بود، همان چیزی بود که مورد ۲۶ درباره‌اش بود. متن کالا – داستان خاستگاه، نت‌های طعمی، پیشنهاد دم‌آوری – در هیچ پاسخی نبود. تزریق `<head>` کارت تلگرام را درست کرد، ولی صفحه‌ای که چهار پاراگراف متن دارد و آن متن در HTML نیست، همان مسئله اول است.

۲) هزینه نگاه‌داشتن راه میانی از هزینه خود کار کمتر نبود. آن راه یک ناسازگاری ساختاری ساخت: `<head>` هر صفحه کالا دو بار ساخته می‌شد – یک بار در `server/src/lib/htmlHead.js` به شکل رشته، یک بار در `web/src/lib/head.js` به شکل عنصر DOM – و هر دو باید حرف‌به‌حرف یکی می‌ماندند. برای همین `head.js` یک پشته سه‌حالتی لازم داشت (پیش‌فرض ثابت، تزریق‌شده سرور، نوشته مرورگر) و یک تست اختصاصی (`seo-parity.test.js`) فقط برای اینکه آن دو واگرا نشوند. صد خط از ظریف‌ترین کد پروژه، برای جبران چیزی که رندر سمت سرور از اساس ندارد.

۳) «معماری عوض می‌شود» درست بود، ولی «prerender جواب نمی‌دهد» هم درست بود – و راه سومی وجود داشت که در استدلال اول دیده نشده بود: رندر سمت سرور در هر درخواست. نه build تازه برای هر تغییر کالا، نه فهرست موقع ساخت. همان چیزی که مدیر روزانه عوض‌کن لازم داشت.

— چه چیزی شد —

کلاینت به Next.js ۱۶ با App Router منتقل شد و `client/` (وبت + react-router) حذف شد. Express دست‌نخورده ماند و فقط API شد.

- جدول مسیره‌ها = ساختار پوشه‌های `web/src/app` . `react-router-dom` رفت.
- صفحه‌ کالا کامپوننت سروری است: `<head>` و بدنه هر دو در پاسخ اول.
- `generateMetadata` جای مدیر `head` را گرفت. آن پشته سه‌حالت، `data-head-*` ، `htmlHead.js` ، `itemPages.js` ، بازگشت SPA و `clientRoutes.js` همه حذف شدند.
- مودال کالا با مسیره‌های رهگیری‌شده (`app/@modal`) بازسازی شد؛ `backgroundLocation` لازم نبود.
- `sitemap.xml` و `robots.txt` همان‌جا که بودند ماندند — Express با مونگو حرف می‌زند و دلیلی برای جابه‌جایی نبود.
- فونت‌ها با `next/font` روی دامنه خودمان آمدند (مورد ۲۵)، و `compression` به API اضافه شد (مورد ۲۴).

— چه چیزی بدتر شد، و آگاهانه پذیرفته شد —

- همه صفحه‌ها پویا رندر می‌شوند. سیاست امنیتی صفحه‌ها به جای `'unsafe-inline'` از `nonce` استفاده می‌کند، و `nonce` برای هر درخواست تازه ساخته می‌شود — پس صفحه از-پیش-ساخته آن را ندارد. یعنی نه کش ایستا و نه `revalidate = 3600` ای که خود همین سند پیشنهاد داده بود. بین «کش ایستا» و «سیاست سخت‌گیرانه»، دومی انتخاب شد.
- جاوااسکریپت بیشتر روی سیم: ۲۲۵ کیلوبایت gzip در برابر ۱۲۹ کیلوبایت باندل ویت. ولی دیگر مانع اولین رنگ نیست، چون HTML خودش کامل است.
- شمارنده سبک یک فریم دیر می‌رسد. HTML سرور سبک بازدیدکننده را نمی‌داند. عمداً با `suppressHydrationWarning` پنهان نشده.
- `og:type: product` از `Metadata API` نمی‌رود — Next این نوع را نمی‌پذیرد و موقع رندر خطا می‌دهد، که یعنی کل `<head>` از دست می‌رود. آن یک تگ را خود صفحه رندر می‌کند و ری‌اکت ۱۹ به `<head>` می‌بردش.
- دو فرایند به‌جای یک. در تولید یک پراکسی لازم است که `/api` ، `/uploads` ، `/sitemap.xml` و `/robots.txt` را به Express بفرستد و بقیه را به Next.

— چه چیزی بهتر شد —

- متن کالا در پاسخ اول است — هدف اصلی مورد ۲۶.
- `LocalBusiness` و `Product` هم سروری شدند.
- آدرس ناشناخته حالا ۴۰۴ واقعی می‌دهد، نه هدایت بی‌صدا با ۲۰۰.
- آن صد خط مدیر `head` و تست همگامی‌اش دیگر وجود ندارند.

— چه چیزی هنوز نیست —

استقرار (مورد ۳۹). بهینه‌سازی تصویر — که وقتی این متن نوشته شد هنوز باز بود — با `next/image` انجام شد و شرحش پای مورد ۲۴ است.

وضعیت (متن اصلی): `client/index.html` فقط `<div id="root"></div>` دارد. تمام محتوا با جاوااسکریپت تولید می‌شود. گوگل امروز جاوااسکریپت را اجرا می‌کند، اما با تأخیر و اولویت کمتر. بقیه موتورها و پیش‌نمایش شبکه‌های اجتماعی، صفحه خالی می‌بینند.

راه‌حل کامل: مهاجرت به Next.js با App Router و `export const revalidate = 3600` برای صفحه محصولات.

راه‌حل میانی: `vite-plugin-ssr` یا `prerender` ساده در زمان `build`.

اولویت: بالا اگر SEO مهم است.

✓ حل شد. سه مسیر تازه – `/coffee/:slug`، `/gear/:slug` و `/powder/:slug` – که هر کدام صفحه کامل `web/src/app/_item/itemRoute.js` را نشان می‌دهند: کارت خرید در یک ستون، متن معرفی در ستون دیگر.

مسیرها به جای یک `/:segment/:slug` عمداً برای هر نوع جداگانه ساخته می‌شوند (از روی `KIND_SEGMENTS`)، تا هیچ‌وقت با `/admin` و `/track` رقابت نکنند.

مودال هم آدرس گرفت، بدون اینکه مودال بودنش را از دست بدهد. الگوش `backgroundLocation` است: لینک کارت موقعیت فعلی را در `location.state` همراه خودش می‌فرستد، و `App.jsx` جدول اصلی را با همان موقعیت رندر می‌کند تا فروشگاه زیر مودال سر جایش بماند. نتیجه:

کار کاربر	چه می‌بیند
کلیک روی «درباره این قهوه»	همان مودال قبلی، ولی آدرس عوض می‌شود
دکمه back مرورگر	مودال بسته می‌شود و به همان‌جای فهرست برمی‌گردد
رفرش یا کپی لینک	صفحه کامل کالا، از صفر

`state` با رفرش دور ریخته می‌شود، و همین است که تضمین می‌کند آنچه گوگل و تلگرام از آن آدرس می‌گیرند صفحه کامل باشد، نه چیز دیگری.

از کجا می‌شود به این صفحه رسید – نکته‌ای که اول از قلم افتاد. در گذر اول فقط دکمه «درباره این قهوه» به لینک تبدیل شده بود، و آن دکمه پشت شرط `hasStory(item) || item.isBlend` بود. نتیجه دو ایراد واقعی داشت:

- کلیک روی خود کالا (نام، تصویر) هیچ کاری نمی‌کرد؛ تنها راه، یک لینک کوچک ته کارت بود.
- کالایی که مدیر هنوز برایش معرفی ننوشته، هیچ لینکی به صفحه خودش نداشت. روی پایگاه داده فعلی یعنی هر شش کالای ابزار و پودر: صفحه‌شان وجود داشت و در `sitemap.xml` هم بود، ولی از داخل سایت به آن‌ها لینکی نمی‌رفت – که برای سئو تقریباً یعنی وجود ندارند.

حالا نام کالا لینک است و همیشه هست؛ شرط `hasStory` فقط همان لینک فرعی را کنترل می‌کند. خود کارت عمداً کلیک‌پذیر نشد: پر از کنترل است (وزن، آسیاب، افزودن) و کلیک سرتاسری آن‌ها را می‌بلعد.

ظاهر لینک نام هم عمداً مثل متن معمولی است و فقط با هور و فوکوس خودش را نشان می‌دهد – کارت شلوغ است و لینک پررنگ، چشم را از دکمه «افزودن» می‌دزد.

دو کار جانبی که لازم شد:

- متن‌های مودال به `components/ItemBody.jsx` منتقل شدند تا صفحه و مودال یک متن نشان بدهند، نه دو نسخه واگرا.
- لنگرهای سربرج (`#products`، `#gear`، ...) بیرون از صفحه اصلی به `Link` تبدیل می‌شوند؛ لنگر تنها روی صفحه کالا هیچ کاری نمی‌کرد.

سمت سرور: بازگشت به `index.html` از قبل بود، ولی `regex`ش داخل `index.js` نوشته شده بود و تست نمی‌شد. حالا در `server/src/lib/clientRoutes.js` است، با `/sitemap.xml` و `/robots.txt` هم در فهرست استثناها. تست: `tests/item-routes.test.js` – از جمله این که `/apiary` باید SPA بگیرد ولی `/api/items` نه.

باقی‌مانده: آدرس فهرست‌ها هنوز لنگر است (`/#gear`)، نه مسیر (`/gear`). با معماری تک‌صفحه فعلی این عمدی است.

وضعیت: کل فروشگاه یک صفحه با لنگر است. `#products` و `#gear` فقط موقعیت اسکرول‌اند. مودال «درباره این قهوه» هم آدرس ندارد.

نتیجه: نمی‌شود لینک یک قهوه خاص را فرستاد، و گوگل نمی‌تواند هر قهوه را جدا ایندکس کند.

راه‌حل: مسیر `/coffee/:slug` که هم `ItemDetail` را نشان دهد و هم عنوان و توضیح اختصاصی داشته باشد:

```
<Route path="/coffee/:slug" element={<ItemPage />} />
```

اولویت: بالا اگر SEO مهم است.

۲۸) بدون داده‌های ساختاریافته

✓ حل شد. هر صفحه کالا یک گره `Product` دارد و صفحه اصلی یک گره `LocalBusiness` (که خودش زیرشاخه `Organization` است، پس یک گره هر دو کار را می‌کند).

همه چیز از داده واقعی ساخته می‌شود، هیچ چیز ثابت نوشته نشده:

فیلد	از کجا
<code>sku</code> , <code>name</code>	<code>item.slug</code> , <code>item.name</code>
<code>description</code>	ترکیب <code>origin</code> , <code>spec</code> , <code>notes</code> و <code>taste</code> – بریده روی ۱۵۸ نویسه
<code>category</code>	برچسب فارسی همان گروه، از <code>groups.js</code>
<code>offers.price</code>	<code>item.price</code>
<code>availability</code>	<code>isSoldOut(item)</code> – همان تعریفی که کارت هم با آن «ناموجود» می‌زند
نشانی و ساعت کار	بخش «درباره ما»ی پنل مدیریت

سه تصمیم که ارزش گفتن دارند:

- **واحد پول.** قیمت‌ها در پایگاه داده تومان‌اند، ولی JSON-LD کد ISO 4217 می‌خواهد و کد رسمی ایران ریال است (`IRR`؛ «IRT» کد استاندارد نیست). پس عدد در `TOMAN_TO_RIAL` ضرب می‌شود تا آنچه به گوگل می‌گوییم واقعاً همان مبلغ باشد. اگر روزی قیمت‌ها به ریال ذخیره شدند، فقط همان ضریب ۱ می‌شود.
 - **قیمت هر کیلو.** قهوه و پودر وزنی‌اند و `price` قیمت یک کیلوست. بدون توضیح، گوگل ۱۸,۵۰۰,۰۰۰ ریال را قیمت یک بسته می‌فهمید. پس کالای وزنی یک `UnitPriceSpecification` با `referenceQuantity: 1 KGM` هم می‌گیرد.
 - **ساعت کار.** در پنل یک جمله فارسی است («شنبه تا چهارشنبه ۱۰ تا ۲۰ · پنجشنبه ۱۰ تا ۱۶») و `schema.org` ساعت ماشین‌خوان می‌خواهد. به جای اینکه مدیر مجبور شود یک چیز را دو بار بنویسد، `parseOpeningHours` همان جمله را می‌خواند و بازه روزها را باز می‌کند. اگر شکلش را نشناسد چیزی تولید نمی‌کند – ساعت غلط در داده‌های ساختاریافته از نبودنش بدتر است.
- escape:** رشته JSON با `jsonLdText` از `<` , `>` , `&` و `U+2028/U+2029` پاک می‌شود و با `textContent` می‌نشیند، نه `innerHTML`. پس نام کالایی مثل `</script><script>...` بی‌اثر است – همان درس مورد ۴، این بار برای JSON.
- تست: `tests/json-ld.test.js` (مورد ۴۳) – شکل خروجی، نگاشت موجودی به `InStock` / `OutOfStock` روی مرزهای واقعی (صفر، کمتر از کمینه خرید، دقیقاً کمینه)، تبدیل واحد پول، پارس ساعت کار، و اینکه نام مخرب از مسیر واقعی هم بی‌خطر بیرون بیاید.

وضعیت: هیچ JSON-LD در پروژه نیست.

راه‌حل: برای هر محصول:

```
{
  "@context": "https://schema.org",
  "@type": "Product",
  "name": "یرگاجف",
  "description": "اتیوپی • گدئو - فرآوری شسته • ارتفاع ۲۰۵۰ متر",
  "brand": { "@type": "Brand", "name": "رُست‌خانه دانه" },
  "offers": {
    "@type": "Offer",
    "price": "1850000",
    "priceCurrency": "IRR",
    "availability": "https://schema.org/InStock"
  }
}
```

و برای خود فروشگاه، `LocalBusiness` با نشانی و ساعت کار – که داده‌اش همین حالا در `content.about` هست. **اولویت: متوسط.**

۲۹) بدون Open Graph، sitemap و robots

✓ **حل شد** – با یک نکته مهم که پایین می‌آید.

مدیر head. چون سایت SPA است، تگ‌ها باید در زمان اجرا و برای هر مسیر جدا نوشته شوند. `web/src/lib/head.js` همین را می‌کند و کتابخانه هم نیامد: کل کار همین صد خط بود و `react-helmet` برای SSR و ادغام درختی ساخته شده که هیچ‌کدام را نداریم. دو تصمیم ریز که بدون آن‌ها کار نمی‌کرد:

- **پشته، نه «آخرین نفر برنده».** یک لحظه دو مصرف‌کننده با هم زنده‌اند: مودال کالا روی صفحه فروشگاه. مودال باید عنوان را بگیرد و با بسته شدنش عنوان صفحه زیرش برگرورد – ولی افکت آن صفحه دوباره اجرا نمی‌شود. پس ورودی‌ها در پشته می‌نشینند و بالاترین اعمال می‌شود.

- **پیش‌فرض‌های `index.html` جایگزین می‌شوند، نه اینکه کنارشان تگ تازه بنشینند.** بدون این، هر صفحه دو تا `og:title` داشت و خزنده‌ها اولی را برمی‌دارند – یعنی همیشه عنوان صفحه اصلی. تگ‌های ثابت با `data-head-default` نشان‌دار شده‌اند و هرکدام که صفحه فعلی نسخه خودش را دارد برداشته می‌شود و بعد برمی‌گردد.

نقشه سایت را سرور می‌سازد، نه فایل ایستا. `server/src/routes/seo.js` روی `/sitemap.xml` و `/robots.txt` می‌نشیند – در ریشه، چون ربات‌ها فقط آنجا دنبالشان می‌گردند، و **پیش از** بازگشت SPA وگرنه `index.html` می‌گرفتند. فهرست از کالاهای **روشن** خوانده می‌شود، با همان قاعده `GET /api/items`: چیزی که در فروشگاه دیده نمی‌شود به گوگل هم معرفی نمی‌شود. فایل ایستا یعنی هر بار مدیر کالایی اضافه یا خاموش کند، یک نفر باید یادش باشد فایل را دستی به‌روز کند – و نخواهد بود.

آدرس پایه از متغیر محیطی می‌آید، نه از کد: `SITE_URL` در `server/.env` و `NEXT_PUBLIC_SITE_URL` در `web/.env.local` (هر دو در فایل‌های `.env.example` توضیح داده شده‌اند). اگر تنظیم نشده باشند، سرور از `Host` درخواست استفاده می‌کند و سمت وب آدرس نسبی می‌سازد – در توسعه راحت است، در تولید نباید به آن تکیه کرد. این دو باید یکی باشند، وگرنه canonical صفحه و لینک داخل `sitemap.xml` فرق می‌کنند.

تصویر پیش‌نمایش. تصویر کارت‌ها به شکل SVG **درون‌خطی** ساخته می‌شود (از مورد ۲۶ به بعد روی سرور، ولی همچنان درون‌خطی)، پس نه فایلی دارد که بشود آدرسش را داد و نه پیش‌نمایش‌سازها SVG را رندر می‌کنند. یک تصویر ۶۳۰×۱۲۰۰ ساخته شد (`web/public/img/og-default.png`) و سازنده‌اش هم در مخزن است: `npm run og:image`. کالایی که مدیر برایش عکس آپلود کرده باشد، عکس خودش را می‌گیرد.

تزریق سمت سرور – چیزی که بدون آن کل این بند نصفه بود. خزنده پیش‌نمایش تلگرام و واتساپ **جاوااسکریپت اجرا نمی‌کند** و فقط HTML خام را می‌خواند. یعنی تگ‌هایی که در زمان اجرا نوشته می‌شوند را اصلاً نمی‌بیند. در گذر اول همین‌طور رها شد و لینک هر قهوه در تلگرام همان کارت پیش‌فرض فروشگاه را نشان می‌داد.

حالا `server/src/routes/itemPages.js` برای مسیرهای کالا، پیش از فرستادن `index.html`، تگ‌های همان کالا را داخلش می‌نویسد (`server/src/lib/htmlHead.js`). **این SSR نیست:** بدنه هنوز `<div id="root">` خالی است و React همان‌طور که بود در مرورگر رندرش می‌کند. فقط `<head>` عوض می‌شود.

سه چیز که این کار را درست نگه می‌دارد:

- **یک نسخه بودن متن.** `itemTitle`، `itemDescription` و `itemHeadState` از client به `shared/seo.js` رفتند، چون حالا هر دو طرف همان رشته را لازم دارند. اگر دو نسخه می‌ماند، لینک تلگرام با تب مرورگر فرق می‌کرد – بی‌سروصدا، مثل همان دوگانگی `pricing.js` (مورد ۱۲). `tests/seo-parity.test.js` می‌سنجد که کلاینت **همان تابع** را صادر کند، نه کپی‌اش.

- **پیش‌فرض‌ها برداشته می‌شوند، نه اینکه کنارشان تگ تازه بنشیند.** وگرنه هر صفحه دو `og:title` داشت و خزنده اولی را برمی‌دارد.

- **مرورگر که بالا آمد، صاحب head می‌شود.** تگ‌های تزریقی `data-head-injected` می‌گیرند و مدیر head با یک قاعده سه‌حالت کنارشان می‌گذارد: تا داده نیامده دست‌نخورده می‌ماند، روی همان صفحه تگ‌به‌تگ با نسخه تازه جایگزین می‌شوند، و با رفتن به صفحه دیگر همه می‌روند. بدون بند آخر، `og:title` یک قهوه روی صفحه `/track` هم می‌ماند.

JSON-LD عمداً تزریق نمی‌شود. برچسب فارسی دسته کالا در `groups.js` است و آوردنش به سرور یعنی دو نسخه شدن آن جدول؛ مصرف‌کننده JSON-LD هم گوگل است که جاوااسکریپت را اجرا می‌کند.

۴۰۴ واقعی برای آدرسی که کالایی پشتش نیست. پیش از این، هم شناسه ناموجود و هم کالای خاموش‌شده وضعیت ۲۰۰ می‌گرفتند و صفحه‌ای می‌دیدند که می‌گفت «پیدا نشد» – همان چیزی که گوگل «soft 404» صدایش می‌کند. دو عارضه داشت: کالای خاموش‌شده تا مدت‌ها دوباره خزیده می‌شد، و خزنده بی‌جاوااسکریپت هیچ‌وقت `noindex` را نمی‌دید چون آن را فقط مرورگر می‌نوشت.

حالا هر دو حالت ۴۰۴ می‌گیرند و تگ `noindex, follow` همان‌جا در HTML نشسته است. **برای بازدیدکننده انسانی هیچ چیز عوض نشده:** بدنه دست‌نخورده است، مرورگر با ۴۰۴ هم رندرش می‌کند، و همان پنجره فارسی «این کالا پیدا نشد» با دکمه بازگشت به فروشگاه دیده می‌شود.

صفحه ۴۰۴ عمداً `canonical` و `og:image` و `og:url` نمی‌گیرد (`notFoundHeadState`): آدرسی که وجود ندارد نسخه متعارف ندارد، و پیش‌نمایش لینکی که به بن‌بست می‌رسد نباید تصویر تبلیغاتی داشته باشد.

۴۱۰ نیست، و این یک تصمیم است. خاموش کردن در این فروشگاه یعنی «فعلاً فروخته نمی‌شود»، نه «برای همیشه رفت» – و ۴۱۰ حرف دوم را می‌زند. اگر روزی «برای همیشه» لازم شد، یک فیلد صریح می‌خواهد، نه استنباط از `active`.

باقی‌مانده مورد ۲۶ هنوز سر جایش است: بدنه صفحه همچنان سمت کلاینت رندر می‌شود، پس موتورهای بی‌جاوااسکریپت متن صفحه را نمی‌بینند – فقط `<head>` درست را می‌بینند.

تست: `tests/sitemap.test.js` (مورد ۳۴) – شکل XML و escape شدنش، کنار گذاشتن کالای بی‌شناسه، `lastmod` که برای تاریخ نامعتبر اصلاً ساخته نمی‌شود، خط‌های `robots.txt`، و اینکه هر پیش‌فرض `index.html` واقعاً نسخه زمان‌اجرا داشته باشد. و `tests/seo-parity.test.js` (مورد ۲۴) – یکی بودن متن دو طرف، برداشته شدن پیش‌فرض‌ها، دست‌نخورده ماندن بدنه، بی‌اثر شدن نام مخرب کالا داخل صفت، و شکل `<head>` صفحه ۴۰۴.

(با مورد ۲۶: بخش «پیش‌فرض‌های `index.html`» از `sitemap.test.js` و کلی `seo-parity.test.js` حذف شدند – هر دو موضوعشان را از دست دادند، چون نه `index.html` ای مانده و نه دو رندرکننده `<head>`. جانشینان `tests/next-metadata.test.js` است که همان خطر را در جای تازه‌اش می‌گیرد: افتادن بی‌صدای یک تگ.)

وضعیت: هیچ‌کدام وجود ندارند. لینک سایت در تلگرام یا واتساپ بدون تصویر و توضیح نمایش داده می‌شود.

راه‌حل:

```
<meta property="og:type" content="website" />
<meta property="og:title" content="رست‌خانه دانه - قهوه تخصصی به گرم و کیلو" />
<meta property="og:description" content="بیش از ۳۰ خاستگاه و میکس، رست‌شده همین هفته" />
<meta property="og:image" content="https://daneh.coffee/og.jpg" />
<meta name="twitter:card" content="summary_large_image" />
<link rel="canonical" href="https://daneh.coffee/" />
```

به‌علاوه `public/robots.txt` و `public/sitemap.xml`.

اولویت: متوسط (کم‌هزینه، اثر فوری).

۱۱.۵ دسترس‌پذیری

۳۰ بدون focus trap کامل در مودال‌ها

وضعیت: `ItemDetail` و `CartDrawer` فوکوس اولیه را می‌دهند و فوکوس قبلی را برمی‌گردانند:

```
lastFocus.current = document.activeElement;
document.body.style.overflow = 'hidden';
closeBtn.current?.focus();
...
lastFocus.current?.focus?();
```

اما Tab می‌تواند به پشت‌صحنه برود. کاربر صفحه‌کلید ممکن است در محتوای پنهان‌شده گم شود.

راه‌حل: حلقه فوکوس دستی:

```
const onKeyDown = (e) => {
  if (e.key !== 'Tab') return;
  const focusables = panel.current.querySelectorAll(
    'a[href], button:not([disabled]), input, select, textarea, [tabindex]:not([tabindex="-1"])'
  );
  const first = focusables[0], last = focusables[focusables.length - 1];
  if (e.shiftKey && document.activeElement === first) { e.preventDefault(); last.focus(); }
  else if (!e.shiftKey && document.activeElement === last) { e.preventDefault(); first.focus(); }
};
```

یا استفاده از عنصر بومی `<dialog>` که این را رایگان می‌دهد.

اولویت: متوسط.

۳۱ عناصر کلیک‌پذیر غیرمعنایی

وضعیت: در `StaticSections.jsx`:

```
<figure className={`shot ...`} onClick={() => setShot(s)}>
```

`<figure>` نه `role="button"` دارد، نه `tabIndex`، نه `onKeyDown`. با صفحه‌کلید قابل استفاده نیست.

همین‌طور `.sheet-back` در `ItemDetail` که با کلیک بسته می‌شود.

راه‌حل: تبدیل به `<button>` یا افزودن سه ویژگی. توجه کنید که در `AdminReports` این درست انجام شده:

```
<tr tabIndex={0} role="button" aria-pressed={active}
  onKeyDown={(e) => { if (e.key === 'Enter' || e.key === ' ') { ... } }}>
```

پس فقط نیاز به یکسان‌سازی است.

اولویت: متوسط.

۳۲) بدون بررسی خودکار دسترس‌پذیری

وضعیت: پروژه پایه خوبی دارد (ARIA، skip link، فوکوس، `prefers-reduced-motion`)، اما هیچ ابزار خودکاری اجرا نشده.

راه‌حل: `@axe-core/react` در توسعه و Lighthouse در CI. مواردی مثل کنتراست `--soft` روی `--paper-2` را باید سنجید.

اولویت: پایین.

۱۱.۶ قابلیت‌های تجاری غایب

۳۳) بدون درگاه پرداخت

وضعیت: سفارش ثبت می‌شود و فروشگاه تماس می‌گیرد. رسید می‌گوید: «همکاران ما برای هماهنگی ارسال با شما تماس می‌گیرند.»

هیچ فیلد `paid`، `paymentRef` یا `transactionId` در مدل نیست.

راه‌حل: اتصال به یکی از درگاه‌های ایرانی (زرین‌پال، آی‌دی‌پی، نکست‌پی). تغییرات لازم:

```
// models/Order.js
payment: {
  status: { type: String, enum: ['unpaid', 'pending', 'paid', 'failed', 'refunded'], default: 'unpaid' },
  gateway: String,
  refId: String,
  authority: String,
  paidAt: Date
}
```

و دو مسیر تازه: `POST /api/orders/:id/pay` و `GET /api/payment/callback`.

اولویت: بالا برای کسب‌وکار واقعی.

✓ **حل شد.** `Item.stock` اضافه شد، دقیقاً با همان معنای پیشنهادی — `null` یعنی نامحدود، و پیش‌فرض هم همین است تا ۱۰۶ کالای موجود بدون مهاجرت مثل قبل کار کنند. صفر یعنی «تمام شد»، که با `null` یکی نیست. واحدش واحد فروش است: گرم برای وزنی، عدد برای ابزار.

کاهش اتمی به `server/src/lib/stock.js` منتقل شد تا **بدون مونگو تست شود** (مدل پارامتر است، نه `import`). سه چیز فراتر از پیشنهاد این بند اجرا شد:

- **پس دادن رزروها.** هر ردیف جدا رزرو می‌شود، پس ردیف سوم می‌تواند بعد از موفقیت دو ردیف اول شکست بخورد. `releaseStock` همه را با هم برمی‌گرداند — و اگر خود `Order.create` هم خطا بدهد. سفارش یا کامل ثبت می‌شود یا اصلاً.
- **۴۰۹ به جای ۴۰۰.** ورودی مشتری غلط نبود؛ انبار عوض شده بود. سبد دست‌نخورده می‌ماند و فقط پیام موجودی نشان داده می‌شود.
- **عدد تازه در پیام خطا.** موجودی برای متن پیام دوباره از پایگاه داده خوانده می‌شود، وگرنه مشتری می‌خواند «فقط ۳۰۰ گرم مانده» و سفارش ۳۰۰ گرمی هم رد می‌شد.
- در رابط کاربری: `isSoldOut` در `groups.js` («موجودی از کمینه قابل خرید کمتر است»)، نشان «ناموجود» روی کارت و در ترازوی قیمت، گزینه خاموش در فهرست دانه‌های میکس، و ستون موجودی در فهرست کالاهای پنل. تست: `tests/stock.test.js`.
- دو دام باقی‌مانده:** موجودی میکس از دانه‌های اجزایش کم نمی‌شود (پس میکس‌ها را نامحدود بگذارید)، و لغو سفارش موجودی را برنمی‌گرداند — تغییر وضعیت هیچ کاری با انبار ندارد.

وضعیت پیشین: هیچ فیلد `stock` وجود نداشت. اگر ۱۰ کیلو یرگاف مانده بود، مشتری می‌توانست ۵۰ کیلو سفارش دهد.

راه‌حل:

```
stock: { type: Number, default: null },    نامحدود = null //
```

با کاهش اتمی هنگام ثبت سفارش:

```
const updated = await Item.findOneAndUpdate(
  { slug, stock: { $gte: grams } },
  { $inc: { stock: -grams } },
  { new: true }
);
if (!updated) return res.status(400).json({ error: `«${item.name}» موجودی `});
```

اولویت: بالا برای کسب‌وکار واقعی.

۳۵) بدون حساب کاربری و تاریخچه سفارش

✓ **حل شد** – با همان «راه‌حل کم‌هزینه» ای که خود این بند پیشنهاد داده بود. حساب کاربری همچنان وجود ندارد و عمداً هم ساخته نشد؛ چیزی که ساخته شد صفحه پیگیری است:

- `GET /api/orders/track?code=&phone=` در `server/src/routes/orders.js` عمداً `requireAdmin` ندارد – مشتری حسابی ندارد که با آن وارد شود – و به جای توکن، `trackLimiter` (میان‌افزار `server/src/middleware/rateLimit.js`، پیش‌فرض ۲۰ تلاش در ۱۵ دقیقه) جلوی حدس زدن کد را می‌گیرد. پیش از مسیر `/:id` ثبت شده، وگرنه `/track` پشت `requireAdmin` گیر می‌کرد.
 - منطقش در `server/src/lib/track.js` است تا بدون مونگو تست شود. سه ضمانتش: پاسخ «کد وجود ندارد» و «شماره جور نیست» حرف‌به‌حرف یکی است (۴۰۴ با یک پیام)، مقایسه در زمان ثابت انجام می‌شود و حتی برای کد ناموجود هم یک پرس‌وجو می‌رود تا زمان پاسخ چیزی لو ندهد، و خروجی یک whitelist صریح است (`publicOrderView`) – نه `_id`، نه `__v`، نه `updatedAt`، نه نشانی و شماره تماس و یادداشت مشتری.
 - سمت کاربر: مسیر `/track` با `web/src/components/Track.jsx`، لینکش در فوتر و در رسید خرید (که کد را با خودش می‌برد: `/track?code=...`).
 - تست: `tests/track.test.js` و یک مورد تازه در `tests/rate-limit.test.js`.
- آنچه هنوز نیست: حساب کاربری، تاریخچه چند سفارش با هم، و اطلاع‌رسانی تغییر وضعیت (مورد ۳۶).

وضعیت: مشتری بعد از بستن رسید، هیچ راهی برای دیدن سفارشش ندارد.

راه‌حل کم‌هزینه: صفحه پیگیری با شماره سفارش + شماره موبایل:

`/track → GET /api/orders/track?code=&phone=` و `code` ورود

بدون نیاز به حساب کاربری، اما با تأیید مالکیت.

اولویت: متوسط.

۳۶) بدون تأیید شماره و بدون اطلاع‌رسانی

وضعیت: شماره موبایل فقط با `^0\d{10}$` regex بررسی می‌شود. هیچ پیامکی هم فرستاده نمی‌شود – نه هنگام ثبت سفارش، نه هنگام تغییر وضعیت.

راه‌حل: OTP هنگام ثبت سفارش (که هم شماره را تأیید می‌کند و هم سفارش جعلی را دشوار)، و پیامک خودکار در تغییر وضعیت به `processing` و `done`.

اولویت: بالا برای کسب‌وکار واقعی.

۳۷) هزینه ارسال ثابت و بدون منطقه

وضعیت:

```
export const SHIPPING = 65000;
export const FREE_SHIPPING_FROM = 1000;
```

یک عدد ثابت برای کل کشور. سفارش داخل تهران و سفارش به زاهدان یکسان حساب می‌شوند.

و یک لبه تیز: سبدی که فقط ابزار دارد (`grams === 0`) همیشه ۶۵,۰۰۰ تومان ارسال می‌دهد – حتی اگر یک ماشین اسپرسو ۹۶ میلیون تومانی باشد.

راه‌حل: جدول هزینه ارسال بر پایه استان + وزن، و بازبینی قاعده ارسال رایگان برای سبدهای فقط-ابزار.

اولویت: متوسط.

۳۸ وضعیت سفارش بدون ماشین حالت

وضعیت: `PATCH /api/orders/:id/status` هر گذاری را می‌پذیرد. مدیر می‌تواند از `done` به `new` برگردد.

راه‌حل: تعریف گذارهای مجاز:

```
const TRANSITIONS = {
  new: ['processing', 'canceled'],
  processing: ['done', 'canceled'],
  done: [],
  canceled: []
};
if (!TRANSITIONS[order.status].includes(req.body.status)) {
  return res.status(400).json({ error: 'این تغییر وضعیت مجاز نیست' });
}
```

اولویت: پایین (برای این اندازه کسب‌وکار، انعطاف فعلی شاید بهتر باشد).

۱۱.۷ زیرساخت

۳۹ بدون CI/CD و بدون Docker

✓ **حل شد – نیمه CI.** `.github/workflows/ci.yml` روی هر `push` و هر `pull request` پنج گام را همان‌طور اجرا می‌کند که یک نفر روی ماشین خودش می‌زند:

```
npm ci → npm run lint → npm run format:check → npm test → npm run build
```

دو تفاوت با نمونه پایین: گام `format:check` اضافه شد، و `npm ci` جای `setup:ci` را گرفت چون با `npm workspaces` یک نصب برای هر سه بسته کافی است و کش `npm` روی همان تک `package-lock.json` ریشه بسته می‌شود.

پایگاه داده لازم نیست – تست‌ها روی توابع خالص‌اند و به مونگو دست نمی‌زنند (مورد ۱۱). همین است که CI را ساده و سریع نگه می‌دارد.

باقی‌مانده: استقرار خودکار (CD) و `docker-compose.yml`. اینجا فقط گفته می‌شود کد سالم است؛ هیچ چیزی مستقر نمی‌شود.

وضعیت پیشین: نه GitHub Actions، نه Dockerfile، نه اسکریپت استقرار.

```
# .github/workflows/ci.yml
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20 }
      - run: npm run setup:ci
      - run: npm run lint
      - run: npm test
      - run: npm run build
```

و `docker-compose.yml` با سرویس MongoDB برای راه‌اندازی یک‌دستوری.

اولویت: متوسط.

۴۰ بدون لاگ ساخت‌یافته و بدون پایش

وضعیت: فقط `console.error(err)` در مبدل خطای سراسری.

راه‌حل: `pino` با `pino-http` برای لاگ ساخت‌یافته، و Sentry برای ردیابی خطا در تولید.

اولویت: متوسط.

۴۱ بدون راهبرد پشتیبان خودکار

وضعیت: خروجی JSON دستی در پنل هست (که خوب است)، اما هیچ پشتیبان زمان‌بندی‌شده‌ای وجود ندارد و فقط `orders` را می‌گیرد — نه `items`، نه `contents`، نه `clubmembers`.

راه‌حل: `mongodump` روزانه با cron، و افزودن خروجی برای سه مجموعه دیگر.

اولویت: بالا در تولید.

۱۱.۸ نقشه راه پیشنهادی

ستون وضعیت در هر پنج جدول یک معنا دارد:

نشانه	یعنی
✓	انجام شد؛ توضیحش بالای همان مورد آمده
●	بخشی انجام شد؛ باقی‌مانده‌اش در همان مورد نوشته شده
○	هنوز انجام نشده

گام یک – پایه‌های مهندسی (۱ تا ۲ هفته)

کار	مورد	وضعیت
Vitest + تست <code>pricing</code> , <code>blend</code> , <code>jalali</code>	۱۱	✓
تست همگامی دو نسخه <code>pricing.js</code>	۱۲	✓ بی‌موضوع – با <code>shared/</code> دو نسخه‌ای نماند
ESLint + Prettier	۱۴	✓
GitHub Actions یا CI	۳۹	✓
rate limit روی login و orders	۲، ۱	✓
ایندکس <code>Order.createdAt</code>	۲۲	✓

گام یک کامل است.

گام دو – کارایی و امنیت (۱ هفته)

کار	مورد	وضعیت
کد اسپلیت پتل مدیریت	۲۰	○
میزبانی محلی فونت	۲۵	○
<code>helmet</code> + بستن CORS در تولید	۷، ۶	✓
<code>compression</code>	۲۴	○
retry برای شماره سفارش	۱۹	○
بستن آپلود SVG	۵	✓

گام سه – SEO (۲ تا ۳ هفته)

کار	مورد	وضعیت
مسیر <code>/coffee/:slug</code>	۲۷	✓
JSON-LD برای محصول و فروشگاه	۲۸	✓
Open Graph + sitemap + robots	۲۹	✓
prerender یا مهاجرت به Next.js	۲۶	✗ عمداً انجام نشد – دلیلش پای مورد ۲۶

گام سه انجام شد، به‌جز موردی که عمداً کنار گذاشته شد. نتیجه‌اش این است: هر کالا آدرس، عنوان، توضیح و داده ساختاریافته خودش را دارد و نقشه سایت زنده است – ولی چون رندر همچنان سمت کلاینت است، پیش‌نمایش لینک در تلگرام و واتساپ هنوز کارت پیش‌فرض فروشگاه است. تنها بازمانده این فصل همین است و راه میانی‌اش پای مورد ۲۶ نوشته شده.

گام چهار – کسب‌وکار (۳ تا ۴ هفته)

کار	مورد	وضعیت
درگاه پرداخت	۳۳	○
موجودی انبار با کاهش اتمی	۳۴	✓
OTP + پیامک وضعیت	۳۶	○
صفحه پیگیری سفارش	۳۵	✓
هزینه ارسال منطقه‌ای	۳۷	○

گام پنج – تمیزکاری (پیوسته)

کار	مورد	وضعیت
پوشه <code>shared/</code> با workspace	۱۲، ۱۳	✓
حذف فیلدهای مرده	۱۶	○
حذف CSS مرده	۱۷	○
انتقال نسخه ایستا به شاخه legacy	۱۸	● سه فایل اصلی‌اش رفتند؛ پوشه <code>img/</code> ریشه هنوز مانده
JSDoc نوع‌دار یا TypeScript	۱۵	○
focus trap و عناصر معنایی	۳۰، ۳۱	○
بازنویسی بارگذاری داده (<code>setState</code> در افکت)	۴۲	● قاعده‌اش روشن است و هشدار می‌دهد؛ خودِ بازنویسی نشده

کاری که در نقشه راه نبود و انجام شد

دو چیز بیرون از این پنج گام اضافه شدند:

کار	چرا
<code>esc</code> روی متن‌های داخل SVG (مورد ۴)	همان گذری که آپلود SVG بسته شد، تزریق از راه نام کالا هم بسته شد
حالت تصویری ساز میکس	ویژگی تازه، نه رفع ضعف – نمای دومی از همان ترکیب، با همان منطق

۱۱.۹ آنچه نباید عوض شود

فهرست ضعف‌ها نباید این تصور را بسازد که پروژه ضعیف است. این تصمیم‌ها درست‌اند و باید بمانند:

(۱) بازمحاسبه قیمت در سرور. درست‌ترین کار امنیتی پروژه.

(۲) snapshot کردن سفارش‌ها. فاکتور باید در زمان منجمد بماند.

(۳) تقویم شمسی دست‌نویس. هیچ کتابخانه‌ای بازه‌بندی با DST تاریخی تهران را حل نمی‌کرد.

(۴) یک مدل `Item` با `kind`. پیچیدگی در یک قلاب متمرکز است، نه پخش در سه مدل.

(۵) تصویر تولیدی. یک راه‌حل زیبا برای مسئله واقعی، با اطلاعات بصری معنادار (رنگ دانه از درجه رست).

۶) whitelist نوشتن. WRITABLE در routes/items.js mass assignment را کاملاً می‌بندد.

۷) محافظ CSV injection. جزئیاتی که خیلی پروژه‌ها نادیده می‌گیرند.

۸) استریم کردن خروجی با cursor. درست‌ترین راه برای فایل بزرگ.

۹) پیام‌های خطای فارسی در همهٔ لایه‌ها. از مدل تا رابط کاربری.

۱۰) CSS با ویژگی‌های منطقی. یک استایل‌نامه برای هر دو جهت.

۱۱) کامنت‌های فارسی که «چرا» را توضیح می‌دهند. این کد بعد از یک سال هم قابل فهم است – که ارزش کمیابی است.

۱۲.۱ اصطلاحات عمومی توسعه وب

فارسی	English	توضیح در بستر این پروژه
پشته فناوری	Tech stack	Node.js (فروشگاه و پنل) روی MongoDB + Express (API) + Next.js
رابط کاربری	User Interface (UI)	پوشه <code>web/</code>
واسط برنامه‌نویسی	API	۳۲ مسیر زیر <code>/api</code> در <code>server/src/routes/</code> ، به‌علاوه دو مسیر ریشه
نقطه پایانی	Endpoint	مثلاً <code>POST /api/orders</code>
مسیریابی	Routing	ساختار پوشه‌های <code>web/src/app</code> – بدون کتابخانه
میان‌افزار	Middleware	<code>requireAdmin</code> ، <code>express.json</code> ، <code>cors</code> ، <code>helmet</code> ، محدودکننده‌های نرخ
مؤلفه / کامپوننت	Component	<code>CartItem.jsx</code> ، <code>CartDrawer.jsx</code>
قلاب	Hook	<code>useShop</code> ، <code>useAuth</code> ، <code>useReveal</code> ، <code>useStorefrontRefresh</code>
حالت	State	<code>useState</code> ، <code>Context</code>
ویژگی / پراپ	Prop	<code><CatalogSection kind="coffee" /></code>
رندر	Render	تولید HTML یا DOM از کامپوننت
رندر دوباره	Re-render	اجرای مجدد کامپوننت با تغییر حالت
بارگذاری تنبل	Lazy loading	<code>loading="lazy"</code> روی تصویرها
باندل	Bundle	خروجی <code>next build</code> در <code>web/.next/</code>
ابزار ساخت	Build tool	خود Next (پیش‌تر Vite)
جایگزینی داغ مازول	HMR	به‌روزرسانی آتی در توسعه – <code>'unsafe-eval'</code> فقط برای همین باز است
پراکسی	Proxy	بند <code>rewrites</code> در <code>next.config.mjs</code> (پیش‌تر <code>server.proxy</code> در <code>vite.config.js</code>)
برنامه تک‌صفحه‌ای	SPA	آنچه دیگر نیست؛ هر کالا صفحه خودش را دارد
بازگشت SPA	SPA fallback	الگویی که با مورد ۲۶ حذف شد – آدرس ناشناخته حالا ۴۰۴ واقعی است
رندر سمت کاربر	CSR	وضعیت پیش از مورد ۲۶؛ امروز فقط تعامل‌ها
رندر سمت سرور	SSR	وضعیت فعلی – هر صفحه در هر درخواست روی سرور ساخته می‌شود
تولید ایستا	SSG	آنچه عمداً ندارد – بهای <code>nonce</code> (تصمیم ۲۲)
بازاعتبارسنجی افزایشی	ISR	<code>revalidate</code> – همان، روی میز نیست
تفکیک کد	Code splitting	برای JS خودکار (بر اساس مسیر)؛ برای CSS هنوز نه – نیمه باز مورد ۲۰
فضای کاری	Workspace	<code>shared</code> ، <code>server</code> ، <code>web</code> زیر یک <code>package-lock.json</code>

فارسی	English	توضیح در بستر این پروژه
بسته مشترک	Shared package	<code>seo.js</code> , <code>taxonomy.js</code> , <code>pricing.js</code> – <code>@ghahve/shared</code>
تنزل مطبوع	Graceful degradation	<code>getContent()</code> که در خطا <code>{}</code> می‌دهد

۱۲.۲ پایگاه داده و مدل‌سازی

فارسی	English	توضیح در بستر این پروژه
مجموعه	Collection	<code>clubmembers</code> , <code>contents</code> , <code>admins</code> , <code>orders</code> , <code>items</code>
سند	Document	یک کالا، یک سفارش
زیرسند	Subdocument	<code>mix[]</code> , <code>lines[]</code> , <code>components[]</code>
طرح‌واره	Schema	<code>orderSchema</code> , <code>itemSchema</code>
مدل	Model	<code>Admin</code> , <code>Order</code> , <code>Item</code>
نگاشنگر شیء-سند	ODM	Mongoose
فیلد مجازی	Virtual field	<code>weighed</code> – محاسبه می‌شود، ذخیره نمی‌شود
قلاب پیش از اعتبارسنجی	hook <code>pre('validate')</code>	هشت گروه قاعده در <code>Item.js</code>
اعتبارسنجی	Validation	<code>enum</code> , <code>match</code> , <code>max</code> , <code>min</code> , <code>required</code>
ایندکس	Index	<code>unique {code:1}</code> , <code>{kind:1, rank:1}</code>
ایندکس یکتا	Unique index	<code>code</code> , <code>phone</code> , <code>username</code> , <code>slug</code>
ایندکس متنی	Text index	روی <code>name/origin/spec</code> – ساخته شده ولی بی‌استفاده
تجمیع	Aggregation	<code>stats.js</code> در <code>\$sort</code> , <code>\$group</code> , <code>\$unwind</code> , <code>\$match</code>
باز کردن آرایه	<code>\$unwind</code>	تبدیل هر ردیف سفارش به یک سند مستقل
گروه‌بندی	<code>\$group</code>	جمع فروش هر <code>slug</code>
فراقتی	Projection	<code>{createdAt:1, status:1, totals:1}</code> در گزارش
کوئری سبک	<code>.lean()</code>	برگرداندن شیء ساده به جای سند Mongoose
مکان‌نما	Cursor	استریم کردن خروجی CSV و JSON
درج یا به‌روزرسانی	Upsert	<code>findOneAndUpdate(..., {upsert:true})</code> در محتوا
غیرعادی‌سازی	Denormalization	<code>Order.totals</code> که از <code>lines</code> قابل محاسبه است
عکس لحظه‌ای	Snapshot	<code>Order.lines[].name/unitPrice/grindLabel</code>
کلید منطقی	Logical key	<code>slug</code> – بدون <code>ObjectId ref</code>
کلید طبیعی	Natural key	<code>ClubMember.phone</code>
یکپارچگی ارجاعی	Referential integrity	آنچه MongoDB ندارد و <code>checkBlend</code> دستی جبران می‌کند

فارسی	English	توضیح در بستر این پروژه
مهاجرت	Migration	آنچه با <code>Mixed</code> در <code>Content</code> از آن فرار شده
وراثت تکجدولی	Single-table inheritance	یک مدل <code>Item</code> با فیلد <code>kind</code>
حذف نرم	Soft delete	فیلد <code>active</code> – کالا پنهان می‌شود نه حذف
عملیات اتمی سند	Atomic document update	<code>findOneAndUpdate</code> با شرط <code>\$gte</code> در <code>stock.js</code>
سه‌حالتی با <code>null</code>	Tri-state field	<code>Item.stock</code> – نشمرده / مانده / تمام شد

۱۲.۳ امنیت

فارسی	English	توضیح در بستر این پروژه
احراز هویت	Authentication	ورود مدیر با نام کاربری و رمز
اجازه‌سنجی	Authorization	<code>requireAdmin</code> روی مسیرهای مدیریتی
توکن وب JSON	JWT	<code>{sub, v}</code> امضا شده با <code>JWT_SECRET</code>
بار توکن	Payload	<code>{ sub: adminId, v: tokenVersion }</code>
توکن حامل	Bearer token	<code>Authorization: Bearer <token></code>
هش	Hash	<code>bcrypt.hash(plain, 12)</code>
ضریب کار	Cost factor / rounds	۱۲ در <code>Admin.setPassword</code>
نسخه توکن	Token version	<code>tokenVersion</code> – ابطال بدون blacklist
ابطال توکن	Token revocation	افزایش <code>tokenVersion</code> هنگام تغییر رمز
فهرست سفید	Whitelist	<code>WRITABLE</code> در <code>routes/items.js</code>
تخصیص انبوه	Mass assignment	حمله‌ای که <code>pickBody</code> می‌بندد
شمارش کاربر	User enumeration	پیام یکسان ورود جلویش را می‌گیرد
حمله زمانی	Timing attack	<code>safeEqual</code> در <code>lib/track.js</code> – هش سپس <code>timingSafeEqual</code>
مقایسه زمان ثابت	Constant-time comparison	همان – تا زمان پاسخ چیزی لو ندهد
تزریق فرمول	CSV / Formula injection	تابع <code>csvCell</code> جلویش را می‌گیرد
تزریق regex	Regex injection	escape کردن <code>q</code> در جست‌وجو
منع سرویس با regex	ReDoS	همان escape کردن
پیمایش مسیر	Path traversal	نام تصادفی فایل + <code>path.basename</code>
اسکرپت‌نویسی بین‌سایتی	XSS	<code>esc()</code> در <code>art.js</code> + بستن آپلود SVG
بی‌خطرسازی خروجی	Output escaping	<code>esc()</code> – پنج نویسه پیش از رفتن داخل SVG
جعل درخواست بین‌سایتی	CSRF	با Bearer token خطر کم است

فارسی	English	توضیح در بستر این پروژه
اشتراک منابع بین‌مبدأ	CORS	<code>resolveCorsOrigin()</code> – در تولید بدون <code>CLIENT_ORIGIN</code> خطای مرگبار
محدودیت نرخ	Rate limiting	<code>loginLimiter</code> , <code>orderLimiter</code> , <code>trackLimiter</code>
هدرهای امنیتی	Security headers	<code>helmet</code> – CSP, HSTS, nosniff, frameguard
سیاست امنیت محتوا	CSP	سه سیاست جدا: <code>proxy.js</code> برای صفحه‌ها، <code>helmet</code> برای API، <code>upload.js</code> برای <code>/uploads</code>
نانس	Nonce	رشته تصادفی هر درخواست در <code>proxy.js</code> – به جای <code>'unsafe-inline'</code>
ارث پویا	<code>'strict-dynamic'</code>	اسکرپتی که اسکرپت دیگری می‌سازد اعتبار nonce را به ارث می‌برد
سندبکس	Sandbox	<code>default-src 'none'; sandbox</code> روی <code>/uploads</code>
منع حدس نوع	<code>nosniff</code>	<code>X-Content-Type-Options</code> روی فایل‌های آپلودی
اعتماد به پراکسی	Trust proxy	شرط درست کار کردن کلید IP در محدودیت نرخ
امنیت در عمق	Defense in depth	<code>select:false</code> + <code>toJSON</code> روی <code>passwordHash</code>
رمز یک‌بارمصرف	OTP	آنچه ندارد – ضعف ۳۶
نمای عمومی	Public view / projection	<code>publicOrderView</code> – whitelist صریح فیلدها
نشت وجود	Existence disclosure	همان ۴۰۴ یکسان برای «کد نیست» و «شماره غلط»

۱۲.۴ اصطلاحات دامنه قهوه

فارسی	English	توضیح
رست / برشته‌کاری	Roast	فرآیند حرارت دادن دانه خام
رست‌خانه	Roastery	کارگاه رست – نام فروشگاه
درجه رست	Roast level	فیلد <code>meter</code> (۱ تا ۵)
رست روشن	Light roast	<code>group: 'light'</code> – اسیدیته زنده، میوه و گل
رست متوسط	Medium roast	<code>group: 'medium'</code> – تعادل شیرینی و اسیدیته
رست تیره	Dark roast	<code>group: 'dark'</code> – بدنۀ سنگین، کاکائو
بدون کافئین	Decaf	<code>group: 'decaf'</code> – کافئین‌زدایی با آب
خاستگاه	Origin	فیلد <code>origin</code> – کشور و منطقه
تک‌خاستگاه	Single origin	قهوه غیرمیکس
میکس / ترکیب	Blend	<code>isBlend: true</code>
میکس ویژه خانه	House blend	<code>house: true</code> – بخش جداگانه دارد

فارسی	English	توضیح
اجزای میکس	Blend components	components[]
دستمزد میکس	Blend surcharge	surcharge – به ازای هر کیلو
فرآوری شسته	Washed process	در فیلد spec
فرآوری نچرال	Natural process	همان‌جا
فرآوری هانی	Honey process	همان‌جا
گیلینگ باساه	Wet-hulled	روش سوماترایبی
مونسونی	Monsooned	روش هندی
عربیکا	Arabica	گونه اصلی
روبوستا	Robusta	گونه پرکافئین، در میکس‌های اسپرسو
کاپینگ	Cupping	چشیدن حرفه‌ای
اسیدیته	Acidity	ترشی مطبوع قهوه
بدنه	Body	حس غلظت در دهان
نوت طعمی	Tasting note	فیلد notes[]
پروفایل طعمی	Taste profile	فیلد tastes[] – هشت کلید
کرما	Crema	کف روی اسپرسو
آسیاب	Grind	فیلد grind – هفت گزینه
دانه کامل	Whole bean	grind: 'whole'
دم‌آوری	Brewing	روش تهیه قهوه
دم‌آور دستی	Pour-over	group: 'pourover'
قیف وی‌۶۰	V60 dripper	shape: 'dripper'
کمکس	Chemex	shape: 'chemex'
کالیتا ویو	Kalita Wave	shape: 'wave'
ایروپرس	AeroPress	shape: 'aeropress'
فرنچ پرس	French press	shape: 'frenchpress'
موکاپات	Moka pot	shape: 'moka'
چذوه	Cezve / Ibrik	shape: 'cezve' – قهوه ترک
سایفون	Siphon	shape: 'siphon'
دم سرد	Cold brew	میکس cold-brew
پرتافیلتر	Portafilter	دسته اسپرسوساز
گروپ‌هد	Group head	، shape: 'machine' E61
تمپر	Tamper	shape: 'tamper'

فارسی	English	توضیح
دیستریبیوتر	Distributor / Leveler	shape: 'leveler'
ابزار WDT	WDT tool	shape: 'wdt'
پیچر شیر	Milk pitcher	shape: 'pitcher'
ناک باکس	Knock box	shape: 'knockbox'
کتری گردن‌غازی	Gooseneck kettle	shape: 'kettle'
تی‌دی‌اس‌متر	Refractometer	shape: 'refracto'
دمی‌تاس	Demitasse	فنجان اسپرسو
لته‌آرت	Latte art	نقش روی فوم شیر
ماچا	Matcha	پودر چای سبز ژاپنی
هوجیچا	Hojicha	چای سبز برشته
ماسالا	Masala chai	چای ادویه‌ای هندی
کرک	Karak	چای غلیظ خلیجی

۱۲.۵ اصطلاحات کسب‌وکار پروژه

فارسی	English	توضیح در کد
سبد خرید	Shopping cart	ShopContext.cart
ردیف سبد	Cart line	یک شیء با mix, grind, grams, slug, key
شناسهٔ ردیف	Line key	slug grind mixSignature - در lib/cartLine.js
امضای میکس	Mix signature	تابع mixSignature
تسویه	Checkout	فرم step: 'form' در CartDrawer
رسید	Receipt	کامپوننت Receipt
سفارش	Order	مدل Order
شمارهٔ سفارش	Order code	P9PT-8412
وضعیت سفارش	Order status	canceled, done, processing, new
پیگیری سفارش	Order tracking	/track - جفت «کد + موبایل»، بدون حساب کاربری
موجودی انبار	Stock	Item.stock - null نامحدود، صفر یعنی تمام شد
رزرو موجودی	Stock reservation	reserveStock با findOneAndUpdate اتمی
پس دادن رزرو	Stock release	releaseStock - در شکست وسط کار
فروش بیش از موجودی	Overselling	همان چیزی که رزرو اتمی جلویش را می‌گیرد

فارسی	English	توضیح در کد
ناموجود	Sold out	<code>isSoldOut</code> – موجودی از کمینۀ قابل خرید کمتر است
جمع کالا	Subtotal	<code>totals.base</code>
تخفیف وزنی	Weight-tier discount	<code>totals.discount</code>
پله تخفیف	Discount tier	<code>TIERS</code>
هزینه ارسال	Shipping	ثابت ۶۵,۰۰۰ تومان
ارسال رایگان	Free shipping	از ۱۰۰۰ گرم
مبلغ پرداختی	Total	<code>totals.total</code>
کالای وزنی	Weighed item	<code>weighed: true</code> – قهوه و پودر
کالای عددی	Piece item	ابزار
پیشنهاد ما	Featured	<code>featured: true</code>
پیشنهاد روز	Daily pick	<code>featured[dayIndex() % length]</code>
پرفروش‌ها	Top sellers	<code>/api/stats/top-sellers</code>
سنجاق شده	Pinned	<code>pinnedTop: true</code>
کنارگذاشته	Excluded	<code>excludeTop: true</code>
وبترین	Showcase	بخش «وبترین» در فرم کالا
حالت تصویری میکس	Visual blend mode	<code>MixVisual.jsx</code> – نمای دوم همان ترکیب
باشگاه مشتریان	Customer club	مدل <code>ClubMember</code>
پنل مدیریت	Admin panel	مسیرهای <code>/admin/*</code>
محتوای سایت	Site content	مدل <code>Content</code> – هفت کلید
پر کردن اولیه	Seeding	<code>server/src/seed.js</code>
پشتیبان	Backup	<code>GET /api/reports/export.json</code>

۱۲.۶ اصطلاحات رابط کاربری و طراحی

فارسی	English	توضیح در کد
راست‌به‌چپ	RTL	<code>dir="rtl"</code> روی <code><html></code>
ویژگی منطقی	Logical property	<code>margin-block-end</code> , <code>inset-inline-start</code>
دوجهته	Bidirectional (bidi)	چرا <code>dir="ltr"</code> روی شماره لازم است
توکن طراحی	Design token	متغیرهای CSS در <code>:root</code>
متغیر CSS	CSS custom property	<code>--cherry</code> , <code>--espresso</code> , <code>--paper</code>
نقطه شکست	Breakpoint	<code>760px</code> , <code>960px</code> , <code>1000px</code>
واکنش‌گرا	Responsive	<code>minmax()</code> , <code>auto-fill</code> , <code>clamp()</code>
کشو	Drawer	<code>CartDrawer</code>
مودال / پنجره شناور	Modal	<code>ItemDetail</code> , پنجره گزارش
پوشش	Overlay	<code>.overlay</code> پشت کشو
توست / پیام کوتاه	Toast	<code>lib/toastStore.js</code> + <code>Toast.jsx</code> – بیرون از هر کانتکست
چیپ	Chip	<code>.chip</code> – دکمه فیلتر
نشان	Badge	<code>.badge</code> – برچسب گوشه تصویر
برچسب	Tag	فیلد <code>tag</code>
نوار ابزار	Toolbar	<code>.toolbar</code>
آکاردئون	Accordion	بخش‌های <code>AdminContent</code> , ردیف سفارش
پیوند پرش	Skip link	«رفتن به فهرست قهوه‌ها»
حلقه فوکوس	Focus ring	<code>:focus-visible</code>
تله فوکوس	Focus trap	آنچه ندارد – ضعف ۳۰
تأخیر ورودی	Debounce	۱۶۰ms جست‌وجو، ۲۵۰ms پفل
ناظر تقاطع	IntersectionObserver	<code>useReveal.js</code>
کاهش حرکت	Reduced motion	<code>prefers-reduced-motion</code>
گلاس‌مورفیزم	Glassmorphism	<code>backdrop-filter: blur(12px)</code>
اسکلت بارگذاری	Skeleton	آنچه ندارد (فقط متن «در حال خواندن»، که با SSR عملاً دیده نمی‌شود)
پرش یک‌فریمی شمارنده	Hydration flash	شمارنده سبد تا اجرای افکت خواندن «و گرم» است

۱۲.۷ اصطلاحات تقویم و بومی‌سازی

فارسی	English	توضیح در کد
تقویم شمسی / جلالی	Jalali / Solar Hijri calendar	<code>server/src/lib/jalali.js</code>
تقویم میلادی	Gregorian calendar	مبنای ذخیره در <code>createdAt</code>
شماره روز ژولین	Julian Day Number (JDN)	پل بین دو تقویم – <code>g2d</code> , <code>d2g</code>
سال کبیسه	Leap year	<code>jalCal(jy).leap</code>
نقاط شکست	Breaks	آرایه ۱۸ عددی در <code>jalCal</code>
منطقه زمانی	Timezone	<code>Asia/Tehran</code>
اختلاف با UTC	UTC offset	۲۱۰ دقیقه (۳:۳۰)
ساعت تابستانی	DST	تا ۱۴۰۱ در ایران
بازه / سطل	Bucket	<code>bucketOf(date, period)</code>
سری زمانی	Time series	<code>bucketSeries(period, count)</code>
بومی‌سازی	Localization (l10n)	<code>toLocaleString('fa-IR')</code>
بین‌المللی‌سازی	Internationalization (i18n)	<code>Intl.DateTimeFormat</code>
رقم فارسی	Persian digits	<code>(U+06F0–U+06F9)</code> ۰۱۲۳۴۵۶۷۸۹
رقم عربی	Arabic-Indic digits	<code>(U+0660–U+0669)</code> ۰۱۲۳۴۵۶۷۸۹
جداکننده هزارگان	Thousands separator	<code>(U+066C)</code> ,
نیم‌فاصله	ZWNJ	<code>U+200C</code> – در <code>suggestSlug</code> هست
علامت ترتیب بایت	BOM	در ابتدای CSV برای اکسل

۱۲.۸ الگوها و اصول

فارسی	English	جا در پروژه
منبع یگانه حقیقت	Single source of truth	<code>shared/</code> - <code>pricing.js</code> و <code>taxonomy.js</code>
تفکیک دغدغه‌ها	Separation of concerns	<code>lib/</code> بدون React و بدون Express
بالا بردن حالت	Lifting state up	فیلترها در <code>HomeShell.jsx</code>
تزریق وابستگی	Dependency injection	<code>lookup</code> به <code>computeTotals</code>
تابع خالص	Pure function	<code>computeTotals</code> , <code>applyPercent</code> , <code>priceFor</code>
تغییرناپذیری	Immutability	<code>setCart((prev) => [...prev, ...])</code>
بازگشت زودهنگام	Early return	<code>if (!item) return;</code>
اصل شکست امن	Fail-safe default	<code>blendPricePerKg</code> که به <code>item.price</code> برمی‌گردد
اعتبارسنجی دولایه	Client + server validation	<code>mixError</code> و بررسی <code>orders.js</code>
کهنه در حین اعتبارسنجی	Stale-while-revalidate	<code>openOrder</code> در <code>AdminReports</code>
محاسبه خوش‌بینانه	Optimistic calculation	قیمت میکس همان لحظه، بی رفت‌وبرگشت شبکه
عملیات اتمی	Atomic operation	شرط و کاهش یکجا در <code>findOneAndUpdate</code>
شرایط رقابتی	Race condition	دو سفارش هم‌زمان روی آخرین موجودی
همه یا هیچ	All-or-nothing	سفارش یا کامل ثبت می‌شود یا اصلاً
تزریق مدل برای تست	Model injection	<code>reserveStock(lines, ItemModel)</code>
تزریق حافظه	Storage injection	<code>loadCart(storage)</code> - تا بدون مرورگر تست شود
نگهبان ترتیب	Ordering guard	<code>if (!hydrated) return;</code> پیش از نوشتن سبد
الگوی مؤلفه مرکب	Compound component	<code>OrderDetail</code> مشترک بین دو صفحه
فرم داده‌محور	Schema-driven form	<code>contentSchema.js</code>
بذر ثابت	Deterministic seed	<code>seeded(slug)</code> در <code>art.js</code>
بودجه زمانی	Time budget	<code>POUR_BUDGET</code> - کل مرحله ثابت، سهم هر کیسه متغیر
تنظیم حالت حین رندر	Set state during render	بازنشانی <code>MixVisual</code> با تغییر ترکیب

۱۲.۹ تست و ابزار توسعه

فارسی	English	جا در پروژه
تست واحد	Unit test	بسیار و دو فایل در <code>tests/</code> ، ۳۸۰ سنج
تست کامپوننتی	Component test	دو فایل <code>.jsx</code> که محیط خود را به <code>jsdom</code> عوض می‌کنند
اجراکننده تست	Test runner	<code>Vitest</code> – <code>npm test</code>
بدل / جایگزین	Test double / fake	مدل ساختگی در <code>stock.test.js</code> و <code>track.test.js</code>
ادعا	Assertion	<code>expect(...).toBe(...)</code>
برابری ارجاعی	Reference equality	<code>toBe</code> در <code>taxonomy.test.js</code> – همان آرایه، نه کپی
تست آزمون‌شونده	Meta-test	تستی که خود سنج را می‌سنجد (<code>card-art.test.js</code>)
تست مرزی	Boundary test	۹۹۹ در برابر ۱۰۰۰ گرم
تست ویژگی‌محور	Property-based test	هزار حرکت تصادفی روی <code>applyPercent</code>
لینتر	Linter	ESLint با <code>flat config</code>
قالب‌بند	Formatter	<code>Prettier</code> – <code>format:check</code> گام CI
یکپارچه‌سازی پیوسته	CI	<code>.github/workflows/ci.yml</code>
منطقه زمانی ثابت در تست	Fixed TZ	<code>env: { TZ: 'Asia/Tehran' }</code> در <code>vitest.config.mjs</code>
گرد کردن به مضرب	Round to multiple	<code>Math.round(x/1000)*1000</code>
نگاه به جلوی منفی	Negative lookahead	<code>matcher</code> در <code>proxy.js</code> – مسیرهای Express و فایل ایستا را کنار می‌گذارد
پخش نوبتی باقیمانده	Round-robin remainder	حلقه توزیع در <code>applyPercent</code>

۱۲.۱۰ فرمان‌ها و فایل‌های کلیدی

فرمان	کار
<code>npm run setup</code>	نصب هر سه <code>workspace + seed</code>
<code>npm run dev</code>	اجرای همزمان (concurrently)
<code>npm run dev:server</code>	فقط سرور با <code>node --watch</code>
<code>npm run dev:web</code>	فقط Next روی ۳۰۰۰
<code>npm run seed</code>	افزودن چیزهای نبوده
<code>npm run seed:reset</code>	بازسازی کامل
<code>npm run build</code>	ساخت <code>web/.next</code>
<code>npm start</code>	اجرای تولید – دو فرآیند: Next و Express

فرمان	کار
<code>npm run docs:pdf</code>	ساخت <code>documentation.html</code> + بازرسی نمودارها + رندر PDF
<code>npm run og:image</code>	ساخت دوباره تصویر پیش‌نمایش لینک
<code>npm test</code>	Vitest، یک بار
<code>npm run test:watch</code>	Vitest در حالت پیوسته
<code>npm run lint</code>	ESLint روی هر سه workspace
<code>npm run format</code>	Prettier، با نوشتن
<code>npm run format:check</code>	همان، فقط بررسی – گام CI
<code>net start MongoDB</code>	روشن کردن سرویس (ویندوز، با دسترسی مدیر)

فایل	چیست
<code>server/.env</code>	تنظیمات محرمانه (در git نیست)
<code>server/.env.example</code>	نمونه قابل انتشار
<code>shared/pricing.js</code>	تنها نسخه محاسبه قیمت – هر دو طرف
<code>shared/taxonomy.js</code>	تنها نسخه کلیدهای مجاز – هر دو طرف
<code>shared/seo.js</code>	آدرس کالا، متن <code><head></code> ، sitemap، JSON-LD، robots
<code>web/next.config.mjs</code>	پراکسی به Express + <code>transpilePackages</code>
<code>web/src/proxy.js</code>	سیاست امنیتی صفحه‌ها – CSP با nonce
<code>web/src/app/layout.jsx</code>	پوسته همه‌جا، شکاف مودال، و <code>force-dynamic</code>
<code>web/src/lib/data.js</code>	خواندن داده روی سرور، با <code>cache</code> ریاکت
<code>web/src/lib/metadata.js</code>	از توصیف <code><head></code> به شیء metadata نکست
<code>web/src/lib/cartStorage.js</code>	خواندن و نوشتن سبد، با حافظه تزریق‌شده
<code>web/src/lib/groups.js</code>	برچسب فارسی همان کلیدها
<code>web/src/lib/art.js</code>	تولید SVG تصویر کارتها + <code>esc</code>
<code>web/src/lib/cartLine.js</code>	شکل ردیف سبد – مشترک بین دو حالت میکس
<code>web/src/lib/blendVisual.js</code>	زمان‌بندی و هندسه حالت تصویری
<code>server/src/lib/stock.js</code>	رزرو و پس‌دادن اتمی موجودی
<code>server/src/lib/track.js</code>	پیگیری سفارش و نمای عمومی
<code>server/src/lib/cors.js</code>	تصمیم CORS، جدا و تست‌پذیر
<code>server/src/middleware/rateLimit.js</code>	سه محدودکننده نرخ
<code>web/src/components/admin/contentSchema.js</code>	توصیف فرم محتوا

فایل	چیسٲ
eslint.config.mjs · vitest.config.mjs	پیکربندی تست و لینت
.github/workflows/ci.yml	نصب · لینت · قالب‌بندی · تست · build

۱۲.۱۱ اصطلاحات معماری (Next (App Router)

همهٔ این‌ها با مورد ۲۶ وارد پروژه شدند. ستون آخر می‌گوید در کدام فایل می‌شود نمونه‌اش را دید.

فارسی	English	در این پروژه
مسیریاب اپ	App Router	web/src/app/ — ساختار پوشه‌ها همان جدول مسیرهاست
کامپوننت سروری	Server Component	پیش‌فرض همه‌جا: app/page.jsx, _item/itemRoute.jsx, ItemBody.jsx
کامپوننت مشتری	Client Component	با 'use client' بالای فایل: AdminShell, ShopContext, HomeShell
مرز سرور—مشتری	Server/client boundary	تابع از آن رد نمی‌شود؛ فقط دادهٔ قابل‌سریال‌سازی — دلیل وجود SiteHeader.jsx
آبدهی / آب‌بندی	Hydration	زنده شدن HTML سروری در مرورگر — سه نگهبانش در ۷.۱۲
ناسازگاری آبدهی	Hydration mismatch	چیزی که سبب خالی اولیه عمداً از آن جلوگیری می‌کند
پارامتر مسیر	Dynamic segment	[slug] در app/coffee/[slug]/
گروه مسیر	Route group	(panel) — در آدرس دیده نمی‌شود، فقط برای layout مشترک
پوشهٔ خصوصی	Private folder	_item/ — با _ شروع می‌شود، پس Next مسیر حسابش نمی‌کند
مسیر موازی	Parallel route	@modal — شکاف دوم app/layout.jsx
مسیر رهگیری‌شده	Intercepting route	(.)coffee/[slug] — پیمایش نرم مودال، بازدید سرد صفحهٔ کامل
پیش‌فرض شکاف	Slot default	@modal/default.jsx که null می‌دهد — بی‌آن، بازدید سرد ۴۰۴ می‌شود
پوسته / چیدمان	Layout	app/layout.jsx, app/admin/layout.jsx, (panel)/layout.jsx
صفحهٔ نیافته	not-found.jsx	۴۰۴ عمومی، و ۴۰۴ اختصاصی هر نوع کالا
مرز خطا	error.jsx	وقتی سرور نتوانست صفحه را بسازد (۵۰۰، نه ۴۰۴)
رندر پویای اجباری	force-dynamic	بهای nonce — هیچ صفحه‌ای از پیش ساخته نمی‌شود
متادیتا API	Metadata API	export const metadata و generateMetadata
متادیتای پویا	generateMetadata	itemMetadata(segment, props) در _item/itemRoute.jsx
پایهٔ متادیتا	metadataBase	در lib/site.js — تا آدرس نسبی og:image مطلق شود
بازنویسی مسیر	Rewrite	rewrites در next.config.mjs
میان‌افزار	Middleware	web/src/proxy.js — نامش در ۱۶ Next به proxy عوض شد

فارسی	English	در این پروژه
کش درخواست ری‌اکت	<code>cache()</code>	در <code>lib/data.js</code> – یک کالا در یک درخواست، یک بار خوانده می‌شود
بدون ذخیره کش	<code>cache: 'no-store'</code>	چون فهرست کالاها و موجودی زنده‌اند
کش مسیریاب	Router cache	همان چیزی که <code>router.refresh()</code> باطلش می‌کند
تازه‌سازی مسیریاب	<code>router.refresh()</code>	<code>useStorefrontRefresh()</code> – جانشین <code>reload()</code> در پنل
فونت خودمیزبان	Self-hosted font	<code>next/font</code> – فایل‌ها موقع <code>build</code> گرفته می‌شوند
متغیر فونت	Font CSS variable	<code>--font-lalezar</code> و <code>--font-vazirmatn</code> روی <code><html></code>
بسته ترنسپایل‌شونده	<code>transpilePackages</code>	برای <code>@ghahve/shared</code> که ترنسپایل نشده است
ریشه ردیابی فایل	<code>outputFileTracingRoot</code>	چون <code>node_modules</code> در ریشه <code>workspace</code> هویت می‌شود

چهار چیز که نامشان شبیه است ولی یکی نیستند

چیت	مثال
<code>proxy.js</code>	میان‌افزار Next که هدر امنیتی می‌گذارد <code>web/src/proxy.js</code>
<code>rewrites</code>	فرستادن درخواست به Express <code>next.config.mjs</code>
<code>lib/api.js</code>	<code>fetch</code> از مرورگر، با مسیر نسبی <code>request('/api/items')</code>
<code>lib/data.js</code>	<code>fetch</code> از سرور، با آدرس مطلق <code>fetch(`\${API_URL}/api/items`)</code>